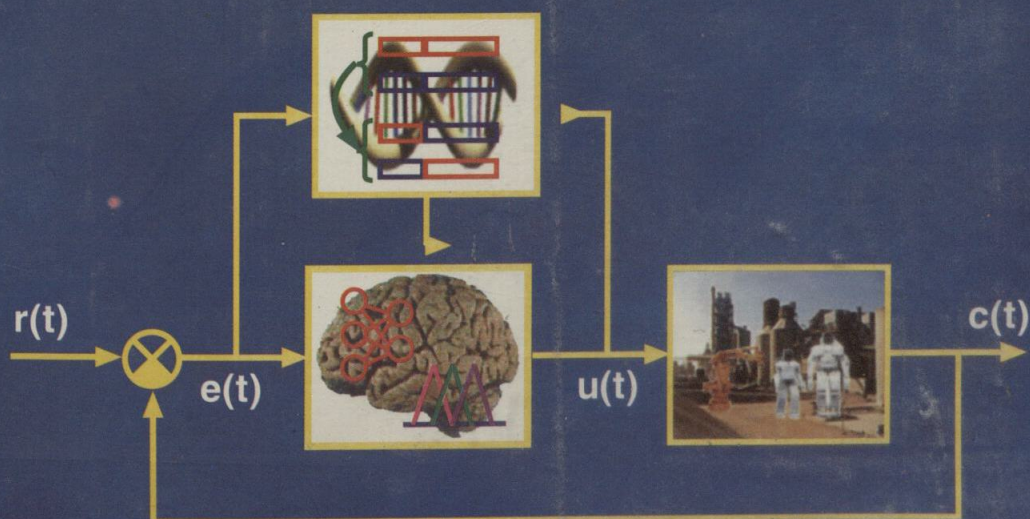


HUỲNH THÁI HOÀNG

EBOOKBKMT.COM

HỖ TRỢ TÀI LIỆU HỌC TẬP

HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH



THU VIEN DAI HOC NHA TRANG



3000013422



NHÀ XUẤT BẢN
ĐẠI HỌC QUỐC GIA

*Chào mừng bạn đã đến với
thư viện của chúng tôi*

Xin vui lòng:

- Không xé sách
- Không gạch, viết, vẽ lên sách

ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

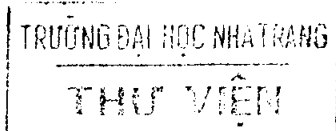
Huỳnh Thái Hoàng

EBOOKBKMT.COM

HỖ TRỢ TÀI LIỆU HỌC TẬP

HỆ THỐNG

ĐIỀU KHIỂN THÔNG MINH



M13422

NHÀ XUẤT BẢN ĐẠI HỌC QUỐC GIA
TP HỒ CHÍ MINH - 2006

HỖ TRỢ TÀI LIỆU HỌC TẬP

MỤC LỤC

LỜI NÓI ĐẦU	7
Chương 1 TỔNG QUAN VỀ ĐIỀU KHIỂN THÔNG MINH	9
1.1 Giới thiệu	9
1.2 Khái niệm về điều khiển thông minh	10
1.3 Đặc điểm của hệ thống thông minh	14
1.4 Cấu trúc của hệ thống điều khiển thông minh	16
1.5 Các lĩnh vực liên quan đến điều khiển thông minh	18
1.6 Hướng nghiên cứu điều khiển thông minh	22
Chương 2 HỆ MỜ	25
2.1 Giới thiệu	25
2.2 Tập hợp mờ	26
2.3 Quan hệ mờ	41
2.4 Biến ngôn ngữ và giá trị ngôn ngữ	44
2.5 Logic mờ	45
2.6 Quy tắc mờ	49
2.7 Suy luận mờ	54
2.8 Hệ mờ	60
2.9 Tóm tắt	85
Chương 3 MẠNG THẦN KINH	86
3.1 Giới thiệu	86
3.2 Tế bào thần kinh và mạng thần kinh nhân tạo	88
3.3 Các phương pháp huấn luyện mạng thần kinh nhân tạo	92
3.4 Mạng truyền thẳng một lớp	95
3.5 Mạng truyền thẳng nhiều lớp	107
3.6 Mạng ham cơ sở xuyên tâm	118
3.7 Mạng thần kinh suy diễn mờ thích nghi	122
3.8 Tóm tắt	125

Chương 4 GIẢI THUẬT DI TRUYỀN	126
4.1 Giới thiệu	126
4.2 Giải thuật di truyền cơ bản	129
4.3 Giải thuật di truyền mã số thực	147
4.4 Giải thuật di truyền song song	153
4.5 Một số ví dụ	156
4.6 Tóm tắt	167
Chương 5 ĐIỀU KHIỂN TRỰC TIẾP	168
5.1 Giới thiệu	168
5.2 Điều khiển mờ trực tiếp	173
5.3 Điều khiển PID mờ	190
5.4 Điều khiển trực tiếp dùng mạng thần kinh	207
5.5 Tự chỉnh bộ điều khiển trực tiếp dùng giải thuật di truyền	212
5.6 Tóm tắt	233
Chương 6 NHẬN DẠNG HỆ PHI TUYẾN	234
6.1 Giới thiệu	234
6.2 Nhận dạng hệ phi tuyến	237
6.3 Nhận dạng hệ phi tuyến dùng mô hình mờ	247
6.4 Nhận dạng hệ phi tuyến dùng mô hình mạng thần kinh	270
6.5 Ước lượng thông số mô hình phi tuyến dùng GA	277
6.6 Tóm tắt	280
Chương 7 ĐIỀU KHIỂN DỰA VÀO MÔ HÌNH	281
7.1 Giới thiệu	281
7.2 Điều khiển dùng mô hình ngược	282
7.3 Điều khiển mô hình nội	288
7.4 Điều khiển theo mô hình chuẩn	292
7.5 Điều khiển dự báo dựa vào mô hình	296
7.6 Tóm tắt	306

Chương 8 ĐIỀU KHIỂN THÍCH NGHI	307
8.1 Giới thiệu	307
8.2 Hồi tiếp tuyến tính hóa hệ phi tuyến	312
8.3 Điều khiển thích nghi gián tiếp	316
8.4 Điều khiển thích nghi trực tiếp	327
8.5 Tóm tắt	336
Chương 9 ĐIỀU KHIỂN HỌC	337
9.1 Giới thiệu	337
9.2 Hệ thống điều khiển học	339
9.3 Một số ví dụ	343
9.4 Tóm tắt	356
Chương 10 HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH	357
10.1 Cấu trúc hệ thống điều khiển thông minh	357
10.2 Phát sinh hành vi	367
10.3 Mô hình thế giới	370
10.4 Xử lý cảm biến	374
10.5 Đánh giá giá trị	376
10.6 Ứng dụng hệ thống điều khiển thông minh	378
10.7 Tóm tắt	386
Phụ lục A: CHƯƠNG TRÌNH MATLAB	387
TÀI LIỆU THAM KHẢO	412

LỜI NÓI ĐẦU

Sự phát triển không ngừng của khoa học công nghệ làm xuất hiện các đối tượng điều khiển có độ phức tạp ngày càng tăng. Yêu cầu thực tiễn đặt ra là phải điều khiển các hệ thống động ngày càng phức tạp, trong điều kiện các yếu tố bất định ngày càng gia tăng với yêu cầu chất lượng điều khiển ngày càng cao. Các yêu cầu trên không thể đáp ứng được trọn vẹn, đồng thời nếu dùng lý thuyết điều khiển thông thường sẵn có. Đây chính là động lực cho ra đời một lĩnh vực nghiên cứu mới đầy tiềm năng và nhiều thách thức, đó là **điều khiển thông minh**. Tổng quát, điều khiển thông minh là phương pháp điều khiển phỏng theo các đặc điểm cơ bản của trí thông minh của con người, bao gồm khả năng học, khả năng xử lý thông tin không chắc chắn và khả năng tìm kiếm lời giải tối ưu. Ngày nay điều khiển thông minh có xu hướng bao gồm tất cả các vấn đề không thuộc phạm vi điều khiển thông thường. Các kỹ thuật điều khiển thông minh được sử dụng phổ biến hiện nay bao gồm mạng thần kinh (nhân tạo), logic mờ và giải thuật di truyền. Khác biệt cơ bản giữa điều khiển thông minh và điều khiển thông thường là khi thiết kế hệ thống điều khiển thông minh về nguyên tắc không cần biết mô hình toán học của đối tượng điều khiển. Chính vì lý do đó mà các kỹ thuật điều khiển thông minh có thể áp dụng để giải được nhiều bài toán điều khiển phức tạp, có yếu tố bất định. Những năm gần đây, điều khiển thông minh phát triển rất mạnh về lý thuyết và được ứng dụng rất rộng rãi trong các lĩnh vực công nghiệp, dân dụng cũng như an ninh quốc phòng.

Quyển sách **HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH** gồm 10 chương, trình bày một cách có hệ thống cơ sở lý thuyết và ứng dụng các kỹ thuật thông minh (logic mờ, mạng thần kinh, giải thuật di truyền) trong việc thiết kế các hệ thống điều khiển. Đa số các ví dụ minh họa trong quyển sách đều được trình bày chi tiết từ cách thiết kế hệ thống điều khiển đến cách thực thi chương trình mô phỏng để độc giả có thể tự lập trình kiểm chứng lý thuyết dùng phần mềm Matlab. Quyển sách được biên soạn nhằm mục đích làm tài liệu tham khảo cho học viên Cao học các ngành Tự động hóa, Cơ điện tử, Công nghệ chế tạo máy... khi học môn Hệ thống điều khiển thông minh.

Tuy vậy, tác giả hy vọng rằng sinh viên đại học và cán bộ khoa học kỹ thuật liên quan đến lĩnh vực điều khiển cũng có thể tìm thấy trong quyển sách này những thông tin bổ ích cần thiết cho việc học tập hoặc công tác chuyên môn.

Tác giả chân thành gửi lời cảm ơn đến các đồng nghiệp ở Bộ môn Điều khiển tự động đã tạo điều kiện và giúp đỡ nhiệt tình để quyển sách được xuất bản; và những người thân trong gia đình đã dành nhiều tình cảm và động viên để quyển sách được hoàn thành.

Đây là lần đầu tiên quyển sách ra mắt độc giả nên mặc dù tác giả đã rất cố gắng biên soạn dựa trên cơ sở tham khảo nhiều tài liệu mới công bố trong các sách chuyên khảo và tạp chí chuyên ngành trong những năm gần đây, nhưng quyển sách chắc chắn chưa bao quát hết tất cả các vấn đề liên quan đến hệ thống điều khiển thông minh và còn nhiều thiếu sót. Tác giả rất mong nhận được các ý kiến đóng góp chân thành của độc giả và đồng nghiệp để lần xuất bản sau quyển sách được hoàn thiện hơn.

Mọi ý kiến đóng góp xin gửi về: TS Huỳnh Thái Hoàng, Bộ môn Điều khiển tự động, Khoa Điện - Điện tử, Trường Đại học Bách Khoa - Đại học Quốc gia TP HCM: 268 Lý Thường Kiệt - Quận 10.

Điện thoại: (08) 8654357

Email: hthoang@hcmut.edu.vn

Tác giả

TS Huỳnh Thái Hoàng

Chương 1

TỔNG QUAN VỀ ĐIỀU KHIỂN THÔNG MINH

1.1 GIỚI THIỆU

Hệ thống điều khiển thông thường hiện nay được thiết kế dựa vào mô hình toán học của hệ thống vật lý. Bằng kỹ thuật mô hình hóa và/hoặc nhận dạng hệ thống, một mô hình toán học mô tả đặc tính động học của đối tượng điều khiển được đưa ra. Mô hình toán học của hệ thống phải “đủ đơn giản” để có thể phân tích bằng lý thuyết điều khiển thông thường hiện có, và “đủ chính xác” để có thể mô tả được các đặc điểm động học quan trọng của đối tượng, đặc biệt là trong miền lân cận điểm làm việc. Sau khi đã chọn được mô hình toán học mô tả đối tượng, các kỹ thuật thiết kế dựa vào mô hình được áp dụng để đưa ra mô hình toán học của bộ điều khiển thích hợp. Lý thuyết toán học cơ sở và các phương pháp điều khiển đã phát triển không ngừng trong thời gian qua để đáp ứng nhu cầu điều khiển đối tượng ngày càng phức tạp với yêu cầu chất lượng ngày càng cao. Trong quá trình phát triển, lý thuyết điều khiển đã có những bước tiến quan trọng từ lý thuyết kinh điển (phân tích và thiết kế hệ thống điều khiển trong miền tần số sử dụng phép biến đổi Laplace và hàm truyền trong những năm 1930–1940) đến lý thuyết hiện đại (phân tích và thiết kế hệ thống điều khiển trong miền thời gian sử dụng mô hình không gian trạng thái trong những năm 1950–1960).

Điều khiển tối ưu trong những năm 1950–1960, theo sau đó là các phương pháp điều khiển ngẫu nhiên, thích nghi, phi tuyến từ sau năm 1960 đến phương pháp điều khiển bền vững từ những năm 1980 đến nay, đã cung cấp cơ sở toán học để có thể điều khiển chính xác hơn các hệ thống động phức tạp. Bộ điều khiển sau khi thiết kế sẽ được thực thi bằng phần cứng và/hoặc phần mềm và nó được dùng để điều khiển hệ thống vật lý. Quá trình thiết kế có thể phải lặp lại nhiều lần cho đến khi chất lượng của hệ thống điều khiển đáp ứng yêu cầu.

Một yếu tố gây ra khó khăn trong việc thiết kế các bộ điều khiển cho các hệ thống động là sự bất định. Nếu hệ thống làm việc trong phạm vi hẹp, hệ thống có thể được xem là tuyến tính và các yếu tố bất định không đáng kể thì các bộ điều khiển kinh điển, chẳng hạn như PID, có thể thiết kế và thực thi dễ dàng và đáp ứng được yêu cầu chất lượng. Khi miền làm việc của hệ thống càng rộng thì các đặc tính phi tuyến xuất hiện và các yếu tố bất định càng gia tăng, trong trường hợp này phải sử dụng các phương pháp hiện đại mới có thể thiết kế được bộ điều khiển đạt yêu cầu. Để đối phó với đặc tính phi tuyến của đối tượng phải sử dụng bộ điều khiển phi tuyến và để đối phó với các yếu tố bất định có thể dùng bộ điều khiển thích nghi hoặc bền vững. Bộ điều khiển thích nghi có thông số và/hoặc cấu trúc thay đổi thích ứng với sự thay đổi của các yếu tố bất định nhằm đảm bảo chất lượng của hệ thống không đổi (hoặc thay đổi không đáng kể), trong khi bộ điều khiển bền vững được thiết kế sao cho chất lượng của hệ thống không đổi (hoặc thay đổi không đáng kể) khi các yếu tố bất định nằm trong giới hạn cho phép. Thực tế đã cho thấy rằng, các bộ điều khiển thích nghi, điều khiển bền vững và gần đây hơn là điều khiển thích nghi bền vững có thể điều khiển rất tốt các tàu du hành vũ trụ, các đối tượng trong các dây chuyền sản xuất công nghiệp vốn có nhiều yếu tố bất định ngẫu nhiên.

Sự phát triển không ngừng của khoa học công nghệ làm xuất hiện các đối tượng điều khiển có độ phức tạp ngày càng tăng. Yêu cầu thực tiễn đặt ra là phải điều khiển các hệ thống động ngày càng phức tạp, trong điều kiện các yếu tố bất định ngày càng gia tăng với yêu cầu chất lượng điều khiển ngày càng cao. Các yêu cầu trên không thể đáp ứng được trọn vẹn, đồng thời nếu dùng lý thuyết điều khiển thông thường sẵn có. Đây chính là động lực cho ra đời một lĩnh vực nghiên cứu mới đầy tiềm năng và nhiều thách thức: *lý thuyết điều khiển thông minh*.

1.2 KHÁI NIỆM VỀ ĐIỀU KHIỂN THÔNG MINH

Điều khiển thông minh là phương pháp điều khiển phỏng theo các đặc điểm cơ bản của trí thông minh của con người. Các đặc điểm cơ bản này bao gồm khả năng học, khả năng xử lý thông tin không chắc chắn và khả năng tìm kiếm lời giải tối ưu. Ngày nay, lĩnh vực điều khiển thông minh (ĐKTM) có xu hướng bao gồm tất cả các vấn đề không thuộc phạm vi điều khiển thông thường. Tuy nhiên, ranh

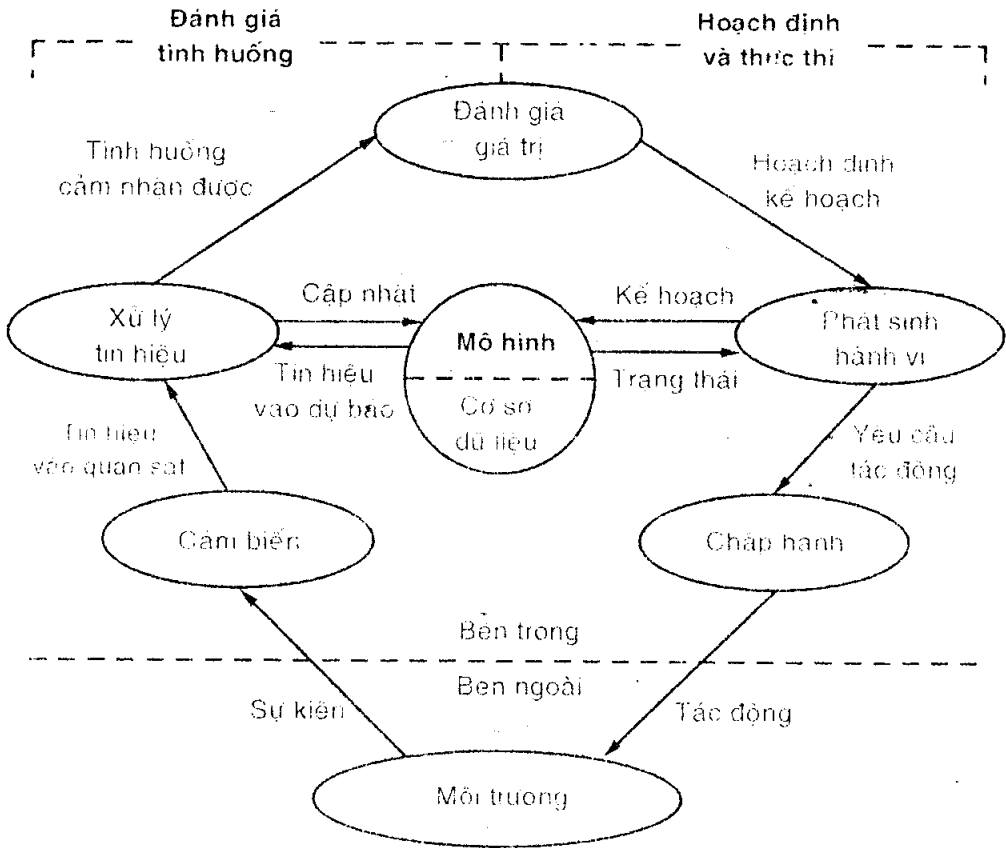
giới giữa ĐKTM và điều khiển thông thường là tương đối và thay đổi theo thời gian. Những hệ thống hiện nay được gọi là hệ thống ĐKTM trong tương lai có thể sẽ chỉ được gọi đơn giản là hệ thống điều khiển. Khó khăn lớn nhất trong việc định nghĩa chính xác thuật ngữ “*Điều khiển thông minh*” là do không có sự thống nhất trong việc định nghĩa trí thông minh của con người và hành vi thông minh. Các tranh luận về trí thông minh đã diễn ra hàng thế kỷ và vẫn tiếp tục hiện nay giữa các chuyên gia giáo dục, chuyên gia tâm lý, chuyên gia máy tính... Một định nghĩa về trí thông minh được nhiều người đồng ý nhất là “*Thông minh là khả năng thu thập và sử dụng tri thức*”. Theo định nghĩa này, hệ thống ĐKTM là hệ thống điều khiển có khả năng thu thập và sử dụng tri thức. Do thông minh là khái niệm mang tính tương đối, có nhiều cấp độ thông minh khác nhau nên hệ thống ĐKTM cũng có nhiều cấp độ khác nhau. Sau đây là một số định nghĩa về hệ thống thông minh và cách phân cấp mức độ thông minh.

Theo Albus ([3]), “*Hệ thống thông minh là hệ thống có khả năng hoạt động thích hợp trong môi trường bất định, trong đó một hoạt động thích hợp là hoạt động làm tăng xác suất thành công, và thành công là đạt được mục tiêu con hướng tới đạt được mục tiêu chung của hệ thống*”. Albus phân ra ba cấp độ thông minh:

- Ở mức độ tối thiểu, thông minh đòi hỏi khả năng cảm nhận môi trường, ra quyết định và điều khiển hành động.
- Ở mức độ trung bình, thông minh có thể bao gồm khả năng nhận biết đối tượng và sự kiện, biểu diễn tri thức trong một mô hình thế giới, suy luận và hoạch định tương lai.
- Ở mức độ cao cấp, thông minh đòi hỏi khả năng nhận thức và hiểu, chọn lựa một cách khôn ngoan, và hành động thành công trong nhiều hoàn cảnh khác nhau để tồn tại và phát triển trong môi trường phức tạp và đối nghịch.

Cấp độ thông minh của hệ thống tùy thuộc vào các yếu tố cấu thành sự thông minh (H.1.1), theo Albus bao gồm:

- Khả năng tính toán của bộ não (máy tính) của hệ thống.
- Sự phức tạp, tinh vi của các thuật toán mà hệ thống sử dụng để xử lý thông tin từ cảm biến, mô hình hóa đối tượng, phát sinh hành vi, đánh giá hoạt động, truyền tin toàn cục...
- Thông tin mà hệ thống lưu trữ trong bộ nhớ.



Hình 1.1 Các yếu tố cấu thành sự thông minh và liên hệ giữa chúng

Theo Meystel ([5]), “ĐKTM là quá trình tính toán một cách hiệu quả hướng tới đạt được mục tiêu của hệ thống phức tạp, thông tin không đầy đủ và trong điều kiện không có chỉ dẫn cụ thể làm thế nào để đạt được mục tiêu đó”. Meystel phân ra bốn cấp độ thông minh:

- *Mức thấp (mức 1):* bù sai số (error compensation)
- *Mức trung bình (mức 2):* hoạch định và bù sai số theo cách người thiết kế lập trình sẵn.
- *Mức cao (mức 3):* hoạch định và bù sai số theo cách mới không đưa ra trước bởi người thiết kế.
- *Mức cao rất cao (mức 4):* phát biểu lại nhiệm vụ khi tình huống thay đổi.

Krishnakumar ([68]) đề xuất cách phân loại ĐKTM dựa trên khả năng tự cải thiện của cấu trúc điều khiển.

- *Mức 0: Điều khiển bền vững.* Tự cải thiện sai số bám là mục tiêu quan trọng của nhiều kỹ thuật điều khiển. Để đạt được điều này, có thể thiết kế các bộ điều khiển hồi tiếp bền vững với độ lợi bằng hằng số để cải thiện sai số khi thời gian tiến đến vô cùng.

- *Mức 1: Điều khiển thích nghi.* Tự cải thiện thông số bộ điều khiển hướng tới mục tiêu đạt được sai số bám (hoặc một mục tiêu nào đó liên quan đến sai số) tốt hơn. Bộ ĐKTM mức 1 thông thường là bộ điều khiển hồi tiếp bền vững với các thông số thích nghi giúp chất lượng điều khiển bù sai số của hệ thống vẫn đảm bảo khi điều kiện làm việc thay đổi.

- *Mức 2: Điều khiển tối ưu.* Tự cải thiện sai số bám hướng đến mục tiêu cực tiểu hoặc cực đại một phẩm hàm theo thời gian (sai số tiến về 0 và chỉ số chất lượng đạt cực trị). Bộ ĐKTM mức 2 bên cạnh khả năng như mức 1 (bền vững + thích nghi) còn phải có khả năng tối ưu hóa một hàm mục tiêu, mà hàm mục tiêu này được hệ thống tự điều chỉnh tùy theo tình huống làm việc.

- *Mức 3: Điều khiển hoạch định.* Bên cạnh khả năng của mức 2, bộ ĐKTM mức 3 còn có khả năng tự cải thiện chức năng hoạch định. Chức năng hoạch định bao gồm hoạch định các tình huống ngẫu nhiên, các tình huống khẩn cấp, các tình huống lỗi...

Chú ý rằng, từ “*điều khiển*” trong thuật ngữ “*ĐKTM*” mang ý nghĩa tổng quát hơn từ “*điều khiển*” trong thuật ngữ “*điều khiển thông thường*”. Trước tiên, đối tượng điều khiển tổng quát hơn, có thể mô tả bằng mô hình hệ thống sự kiện rời rạc hoặc mô hình phương trình vi phân/sai phân hoặc cả hai. Điều này dẫn đến sự phát triển lý thuyết điều khiển hệ thống lai, trong đó nghiên cứu vấn đề điều khiển quá trình động học có trạng thái liên tục bằng các bộ điều khiển trạng thái rời rạc. Ví dụ hệ thống ĐKTM trong một nhà máy cán thép, ngoài chức năng thông thường là ổn định tốc độ của máy cán thép, còn có thể bao gồm thêm chức năng hoạch định điểm đặt của bộ điều chỉnh tốc độ, chức năng chuẩn đoán lỗi và cảnh báo, chức năng chọn lựa các quyết định liên quan đến hiệu quả kinh tế, lịch trình bảo trì, sự sẵn sàng của máy móc thiết bị liên quan... Như vậy, đối tượng điều khiển của hệ thống ĐKTM trong nhà máy cán thép là cả quá trình sản xuất. Bên cạnh việc khảo sát đối tượng tổng quát hơn, mục tiêu của hệ thống ĐKTM cũng tổng quát hơn. Ví dụ, công

việc tổng quát của bộ điều khiển cánh tay máy trong không gian là “*thay thế bộ phận A trong vệ tinh*”; công việc này có thể chia ra làm nhiều việc nhỏ, trong số đó có thể bao gồm nhiệm vụ “*bám theo một quỹ đạo xác định*”, bài toán nhỏ này có thể giải bằng phương pháp điều khiển thông thường. Để đạt được mục tiêu điều khiển tổng quát cho hệ thống phức tạp trong một khoảng thời gian, bộ điều khiển phải ứng phó với các yếu tố bất định lớn mà bộ điều khiển bền vững hoặc điều khiển thích nghi thông thường không giải quyết được. Tóm lại bài toán điều khiển trong ĐKTM là phiên bản nâng cao của bài toán điều khiển trong điều khiển thông thường, nó tổng quát hơn và nhiều thách thức hơn.

1.3 ĐẶC ĐIỂM CỦA HỆ THỐNG THÔNG MINH

Để đạt được mục tiêu điều khiển tổng quát trong điều kiện có nhiều yếu tố bất định hệ thống thông minh phải có một số đặc điểm sau đây ([7]):

- Khả năng học và thích nghi
- Khả năng suy luận, xử lý thông tin phức tạp, không chắc chắn
- Khả năng xử lý các tình huống lỗi và sửa sai
- Tính tối ưu
- Khả năng tái cấu hình và mở rộng
- Tính tự chủ
- Khả năng hoạch định và ra quyết định.

Các đặc điểm nêu trên có thể được xem là tiêu chuẩn để đánh giá mức độ thông minh của hệ thống, tùy theo cấp độ thông minh mà hệ thống có thể có một vài hoặc tất cả đặc điểm nêu trên. Dưới đây chúng ta xét các đặc điểm được xem là cơ bản nhất trong hệ thống ĐKTM.

1- Khả năng học và thích nghi

Khả năng thích nghi với điều kiện làm việc thay đổi là rất cần thiết đối với hệ thống thông minh. Mặc dù tính thích nghi không nhất thiết đòi hỏi khả năng học, tuy nhiên đối với các hệ thống có thể thích ứng nhanh với nhiều thay đổi không tiên liệu trước khả năng học là cần thiết. Do vậy, khả năng học là một đặc điểm quan trọng của hệ thống thông minh (cấp cao).

Mối liên hệ giữa điều khiển thích nghi và điều khiển học là gì? Trước tiên cần nhấn mạnh rằng để có tính thích nghi hệ thống không cần khả năng nhớ, ngược lại để có thể học hệ thống nhất thiết phải có bộ nhớ. Việc học được hoàn thành, theo một ý nghĩa nào đó, khi một thuật toán điều khiển thích nghi được sử dụng để điều chỉnh thông số của bộ điều khiển sao cho tính ổn định và chất lượng của hệ thống được đảm bảo. Trong trường hợp này hệ thống học và tri thức thu thập được ghi vào bộ nhớ là giá trị mới của các thông số. Trong quá trình vận hành, nếu điều kiện làm việc thay đổi giống như tình huống đã được học thì hệ thống sẽ truy xuất ngay đến giá trị thông số bộ điều khiển đã lưu trữ, nhờ đó mà hệ thống có thể thích ứng nhanh với môi trường biến động. Trong khi đó thuật toán điều khiển thích nghi khi gặp lại tình huống cũ vẫn cần phải tính toán lại bộ điều khiển vì không có thông tin gì được lưu trữ trong bộ nhớ.

Bài toán học trong hệ thống điều khiển tự động đã được nghiên cứu từ lâu, đặc biệt là từ những năm cuối thập niên 1960, và nó là đề tài của rất nhiều công trình nghiên cứu. Có nhiều sơ đồ điều khiển trong đó quá trình học được áp dụng:

- Học về đối tượng, tức là học cách kết hợp các thay đổi và sau đó là cách đưa ra mô hình mới của đối tượng.
- Học về môi trường, điều này có thể được thực hiện bằng cách sử dụng các phương pháp đi từ các quan sát thụ động đến thử nghiệm tích cực.
- Học về bộ điều khiển, ví dụ học cách chỉnh một thông số điều khiển nào đó để làm tăng chất lượng.
- Học cách đưa ra mục tiêu và điều kiện ràng buộc mới...

Tóm lại, hệ thống có khả năng học bao hàm khả năng thích nghi, đồng thời có bộ nhớ để lưu trữ tri thức trong quá khứ bằng một hình thức nào đó sao cho nó có thể hữu ích khi tình huống tương tự xuất hiện. Nhờ khả năng học mà hệ thống thông minh không lặp lại sai lầm trong quá khứ.

2- Khả năng suy luận

Hệ thống thông minh phải có khả năng suy luận để tự ứng xử và ra quyết định trong những trường hợp môi trường làm việc biến động mạnh dẫn đến xuất hiện các tình huống mới mà khi thiết kế hệ thống không thể tiên liệu được hết. Hệ thống không thể được gọi là thông minh nếu nó chỉ có thể làm được những điều đã được chỉ dẫn. Khả năng suy luận cũng rất cần thiết để hệ thống có thể xử lý thông tin không chắc chắn, chuẩn đoán các tình huống lỗi và sửa sai.

3- Tính tối ưu

Tính tối ưu trong việc đạt được mục tiêu là một đặc điểm quan trọng của hệ thống thông minh. Hệ thống không thể được gọi là thông minh (cấp cao) nếu có hệ thống khác có thể hoạt động tốt hơn. Để đạt được tối ưu trong điều kiện môi trường làm việc có nhiều biến động, hệ thống phải có khả năng tái cấu trúc và mở rộng.

4- Tính tự chủ

Tự chủ trong việc đặt ra và đạt các mục tiêu là một đặc điểm quan trọng của hệ thống ĐKTM. Khi hệ thống có khả năng ứng xử thích hợp trong một môi trường bất định trong một thời gian dài mà không có sự can thiệp bên ngoài, nó được xem là có tính tự chủ cao. Liên quan đến tính tự chủ là khả năng hoạch định và ra quyết định.

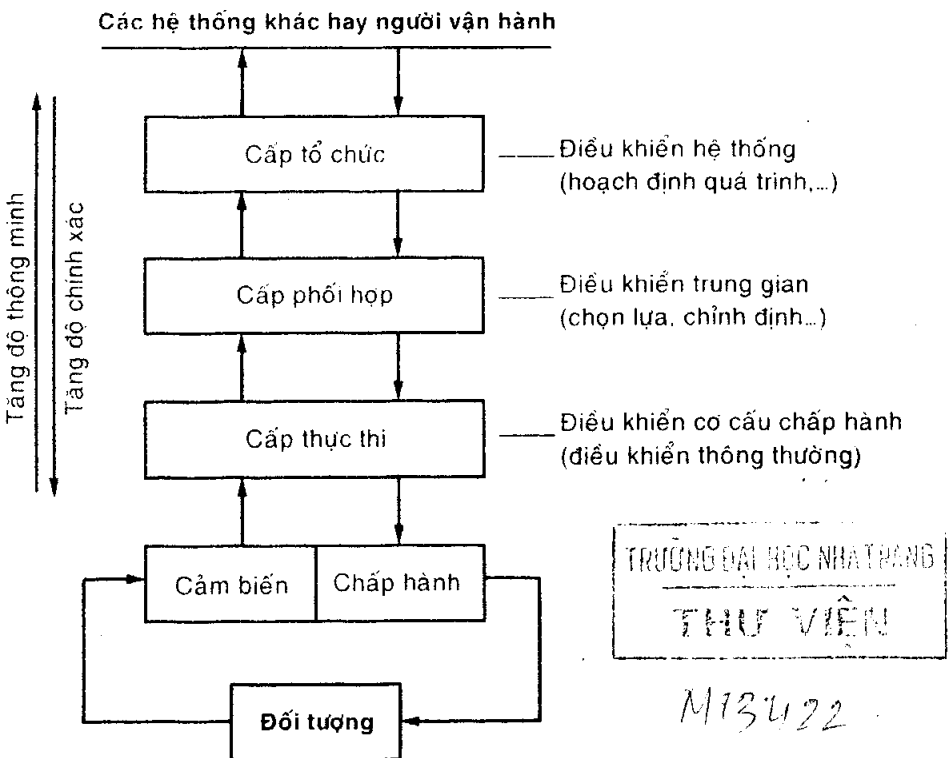
1.4 CẤU TRÚC CỦA HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH

Hệ thống thông minh phải có cấu trúc phân cấp thích hợp để phân tích và đánh giá các chiến lược điều khiển một cách có hiệu quả để có thể đương đầu với sự phức tạp của đối tượng điều khiển, mục tiêu điều khiển và sự biến động của môi trường. Hình 1.2 là cấu trúc phân cấp hệ thống ĐKTM được sử dụng phổ biến nhất hiện nay. *Cấu trúc này có ba cấp từ cao đến thấp: cấp tổ chức (hay quản lý), cấp phối hợp và cấp thực thi.* Cần nhấn mạnh rằng phân cấp ở đây là phân cấp chức năng, không nhất thiết ngụ ý phân cấp phần cứng mà còn có thể là phân cấp phần mềm.

Cần nhấn mạnh rằng, hệ thống có thể có nhiều hơn hoặc ít hơn ba cấp, tuy nhiên về khái niệm chia làm ba cấp. Cấp thực thi, gồm các thuật toán điều khiển thông thường, có giao tiếp với đối tượng và

môi trường qua phần cứng cảm biến và cơ cấu chấp hành. Cấp tổ chức, gồm các phương pháp ra quyết định thông minh, có giao tiếp với người vận hành. Cấp phối hợp liên kết cấp thực thi và cấp tổ chức, nó được dùng để kết hợp các phương pháp ra quyết định thông thường và thông minh. Cấp thực thi gồm có cả phần cứng và phần mềm, trong khi đó cấp phối hợp và cấp tổ chức chủ yếu là phần mềm.

Trong hình 1.2, các chỉ thị được đưa ra bởi cấp cao hơn tới cấp thấp hơn và dòng dữ liệu đáp ứng đi từ cấp thấp lên trên. Các thông số của các hệ thống con có thể thay đổi bởi hệ thống ở trên đó một cấp. Có sự ủy thác và phân bố tác vụ từ cấp cao đến cấp thấp và có sự phân bố theo lớp quyền ra quyết định. Tại mỗi cấp, một số xử lý được thực hiện trước khi thông tin được gửi đến cấp cao hơn. Nếu có yêu cầu, dữ liệu có thể được truyền từ các hệ thống con cấp thấp nhất đến cấp cao nhất để hiển thị. Tất cả các hệ thống con cung cấp thông tin trạng thái đến cấp cao hơn. Sự can thiệp của con người được cho phép thậm chí ở cấp giám sát thực thi điều khiển, tuy nhiên chỉ thị được truyền xuống từ các cấp cao trong sơ đồ phân cấp.



Hình 1.2 Cấu trúc chức năng bộ điều khiển tự động thông minh

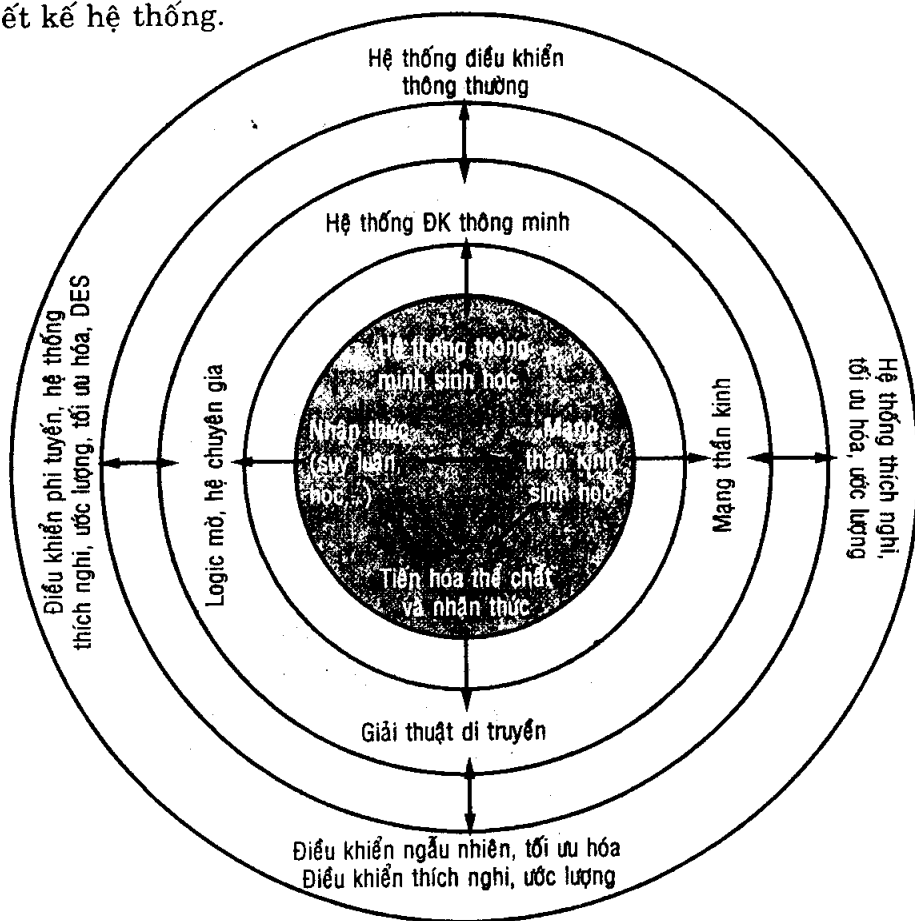
Có một sự ủy thác các nhiệm vụ từ cấp cao đến cấp thấp; hệ quả là số các tác vụ phân biệt tăng lên khi chúng ta đi từ trên xuống theo cấu trúc phân cấp. Các cấp cao hơn không đề cập chi tiết đến đáp ứng của hệ thống mà đề cập đến khía cạnh rộng hơn của đáp ứng. Do đó, cấp cao hơn phải ra các quyết định điều khiển bằng cách xem xét ít thông tin hơn. Cũng chú ý rằng, cấp cao quan tâm đến phạm vi thời gian dài hơn so với cấp thấp. Do các cấp cao trong sơ đồ phân cấp phải ra quyết định trong điều kiện thiếu thông tin hơn cấp thấp nên có sự gia tăng mức độ thông minh khi đi từ cấp thấp đến cấp cao. Điều này được phản ánh qua việc ở cấp cao ít dùng các phương pháp thuật toán số thông thường hơn, thay vào đó là dùng nhiều hơn các phương pháp ra quyết định dựa vào ký hiệu. Đây chính là *“nguyên tắc tăng độ thông minh bằng cách giảm độ chính xác”* của Saradis (1998). Việc giảm độ chính xác được phản ánh qua việc giảm mật độ các giai đoạn kế tiếp nhau, giảm băng thông hoặc tốc độ hệ thống, và giảm tốc độ quyết định (tác động điều khiển). Tất cả những đặc điểm này dẫn đến giảm độ cụ thể của mô hình được sử dụng, hoặc tương đương, tăng độ trừu tượng của mô hình. Độ cụ thể của mô hình còn phụ thuộc vào khả năng của bộ điều khiển tự động.

1.5 CÁC LĨNH VỰC LIÊN QUAN ĐẾN ĐIỀU KHIỂN THÔNG MINH

Điều khiển thông minh là lĩnh vực khoa học liên ngành vì nó kết hợp lý thuyết từ các lĩnh vực khác nhau như toán học, điều khiển học, máy tính, trí tuệ nhân tạo... Khoa học máy tính và đặc biệt là trí tuệ nhân tạo cung cấp phương pháp biểu diễn tri thức, phương pháp suy diễn dựa trên cơ sở tri thức. Khái niệm và thuật toán thích nghi trong lĩnh vực điều khiển giúp các bộ ĐKTM thích nghi và học. Sự phát triển công nghệ chế tạo cảm biến, cơ cấu chấp hành, công nghệ tính toán và mạng thông tin tạo điều kiện cần thiết để thực hiện phân cứng hệ thống ĐKTM.

Điều khiển thông minh là kỹ thuật điều khiển thay thế bộ não con người trong việc ra quyết định, hoạch định chiến lược điều khiển, và học các chức năng mới. Đối với nhiều người, thuật ngữ *“ĐKTM”* có nghĩa là *điều khiển dùng phương pháp mạng thần kinh, logic mờ, giải thuật di truyền (H.1.3)*. Tuy nhiên, ĐKTM không hạn chế trong các phương pháp trên. Thật ra, theo các định nghĩa về ĐKTM đã trình bày ở mục 1.2 không phải tất cả các bộ điều khiển thần

kinh/mờ/di truyền đều được coi là thông minh. Cần phải hiểu rằng mạng thần kinh, logic mờ, giải thuật di truyền chỉ là các công cụ phù hợp để thực thi các thuật toán ĐKTM. Hệ thống có thực sự thông minh hay không, mức độ thông minh đến đâu hoàn toàn phụ thuộc vào mức độ sáng tạo của người vận dụng các kỹ thuật nêu trên để thiết kế hệ thống.



Hình 1.3 Các kỹ thuật điều khiển thông minh

Mạng thần kinh (Neural Networks) ([39], [76]) có thể xem là mô hình toán học đơn giản của bộ não và nó hoạt động như là một mạng tính toán phân bố song song. Tuy nhiên, khác với máy tính thông thường cần phải lập trình để thực hiện một công việc cụ thể nào đó, phần lớn mạng thần kinh phải được huấn luyện, nó có thể học các liên kết mới, quan hệ chức năng mới và mẫu mới. Mạng thần kinh là công cụ cơ bản để trang bị cho hệ thống thông minh khả năng học

Có lẽ lợi điểm quan trọng nhất của mạng thần kinh là tính thích nghi. Mạng thần kinh có thể tự động điều chỉnh trọng số của nó để tối ưu hóa các hoạt động như nhận dạng mẫu, ra quyết định, điều khiển hệ thống, dự báo. Tính thích nghi cho phép mạng thần kinh vẫn thực hiện tốt chức năng của nó khi môi trường hoặc đối tượng điều khiển thay đổi theo thời gian. Nhiều sơ đồ điều khiển dùng mạng thần kinh với thuật toán huấn luyện lan truyền ngược sử dụng khả năng ánh xạ phi tuyến của mạng thần kinh để giải bài toán nhận dạng và điều khiển hệ phi tuyến phức tạp. Những năm gần đây, mạng thần kinh đã được áp dụng thành công vào nhiều lĩnh vực kỹ thuật như hàng không, xử lý tín hiệu số, điện tử, robot, thị giác máy tính, xử lý tiếng nói, hệ thống sản xuất, giao thông, điều khiển và y học. Một vài ứng dụng của mạng thần kinh trong lĩnh vực liên quan đến điều khiển là: điều khiển nhiệt độ, cánh tay robot, điều khiển động cơ tủy động, quá trình hóa học, điều khiển cột chưng cất, điều khiển máy bay, nhận dạng hệ thống, điều khiển tàu ngầm, máy cán thép, hệ thống phun xăng...

Logic mờ (Fuzzy Logic) ([28], [76], [82], [119]) cung cấp một phương thức suy diễn có thể bắt chước khả năng suy luận của con người để áp dụng vào các hệ thống cơ sở tri thức. Dựa trên lý thuyết tập mờ (*Fuzzy Set*), một công cụ biểu diễn và xử lý dữ liệu không chính xác, logic mờ cung cấp cơ sở toán học để mô tả sự không chắc chắn liên quan đến quá trình nhận thức của con người. Hệ mờ (*Fuzzy System*) là hệ thống xử lý thông tin dựa vào logic mờ và suy luận mờ (*Fuzzy Reasoning*). Hai đặc điểm chính làm cho hệ mờ có nhiều ứng dụng trong các hệ thống thông minh là:

- Hệ mờ phù hợp cho việc suy luận không chắc chắn hay gần đúng, đặc biệt là cho hệ thống phức tạp rất khó rút ra được mô hình toán học.
- Logic mờ cho phép ra quyết định với giá trị ước lượng trong điều kiện thông tin không đầy đủ hoặc không chắc chắn.

Thiết kế dựa vào logic mờ có thể giải quyết các điểm yếu của các phương pháp thông thường. Điều khiển mờ dựa trên logic mờ cung cấp một kiểu thiết kế mới mà một bộ điều khiển có thể thiết kế cho các quá trình phức tạp, thiếu xác định mà không cần tri thức về dữ liệu định lượng liên quan đến quan hệ vào ra, những thông tin

định lượng này cần thiết để thiết kế hệ thống điều khiển theo phương pháp thông thường. Công trình đầu tiên ứng dụng bộ điều khiển mờ là mô hình động cơ hơi nước. Sau công trình tiên phong này, nhiều sản phẩm công nghiệp cũng như dân dụng dùng logic mờ đã phát triển ở Nhật, Mỹ, châu Âu: điều khiển xe lửa tự động, điều khiển robot, động cơ tùy động, điều khiển ổ đĩa, máy giặt, điều khiển xe hơi và máy bay trực thăng, thiết bị điện tử và đồ dùng gia đình, cảm biến, điều khiển nhiệt độ, thị giác máy tính, điều khiển hệ thống hạ cánh của máy bay, điều khiển tàu hàng, nhà máy xi măng...

Giải thuật di truyền (Genetic Algorithm – GA) ([36], [62], [112]) là giải thuật tìm kiếm lời giải tối ưu trên nguyên tắc phỏng theo quá trình tiến hóa và quy luật di truyền của sinh vật trong tự nhiên. GA sử dụng ba phép toán cơ bản là chọn lọc, lai ghép và đột biến để tiến hóa một tập hợp hữu hạn các nhiễm sắc thể biểu diễn bằng chuỗi ký hiệu (nhị phân, thập phân...). Bản chất toán học của GA là giải thuật tìm kiếm theo xác suất. GA có thể hoạt động tốt trong quá trình không chắc chắn và phi tuyến. Ngoài ra, GA không phụ thuộc vào bài toán cần giải, rất ít thông tin (nếu có) về hệ thống được sử dụng để thiết kế GA. Vì vậy GA là một thuật tối ưu hóa tổng quát có thể áp dụng vào một lớp rộng các bài toán có độ phức tạp cao. Cơ sở toán học của GA được đưa ra bởi Holland và sau đó lý thuyết liên quan đến GA được phát triển mạnh bởi các nhà nghiên cứu khác. Chương 4 của quyển sách này sẽ trình bày chi tiết về các vấn đề liên quan đến GA. Hiện nay GA đã được ứng dụng trong nhiều bài toán khác nhau như đo lường tốc độ lưu chất, hoạch định đường đi cho robot, thiết kế hệ thống điện, thiết kế mạch in, tối ưu hóa thông số các bộ điều khiển.

Như đã giới thiệu ở trên, mạng thần kinh, logic mờ, giải thuật di truyền tương ứng là ba công cụ cho phép thiết kế hệ thống có các đặc điểm thông minh cơ bản là khả năng học, khả năng suy luận và khả năng tối ưu hóa. Trong những năm gần đây, mạng thần kinh, logic mờ và giải thuật di truyền là các phương pháp bổ sung cho nhau trong việc thiết kế và thực thi các hệ thống thông minh nhằm phát huy triệt để ưu điểm và giảm thiểu khuyết điểm của từng phương pháp. Hệ thống kết hợp mạng thần kinh, logic mờ, giải thuật di truyền gọi là hệ thống lai. Sau đây là một số kết hợp được sử dụng phổ biến hiện nay:

- *Hệ mờ thần kinh (Neural Fuzzy System)*: cung cấp cho hệ mờ một hệ thống tự chỉnh dùng công cụ mạng thần kinh, ANFIS (*Adaptive Neuro Fuzzy Inference Systems*) là một ví dụ điển hình về hệ mờ thần kinh.
- *Mạng thần kinh mờ (Fuzzy Neural Network)*: là mạng thần kinh trong đó một số phép toán trong mạng được mờ hóa, ví dụ logic mờ có thể sử dụng để xác định bước học cấu trúc mạng.
- *Hệ mờ di truyền (Genetic Fuzzy System)*: là hệ mờ trong đó các tập mờ cũng như các quy tắc suy luận mờ được chỉnh định tối ưu nhờ GA.

Các hệ thống lai đã chứng minh được tính hiệu quả trong việc thiết kế hệ thống ĐKTM và đã được ứng dụng vào nhiều quá trình công nghiệp như cánh tay máy, điều khiển độ pH (trong quá trình hóa học), quỹ đạo robot, hệ thống giám sát chủ động trong ô tô, phân loại sản phẩm...

1.6 HƯỚNG NGHIÊN CỨU ĐIỀU KHIỂN THÔNG MINH

1.6.1 Điều khiển thông minh là một lĩnh vực nghiên cứu riêng biệt

Có ý kiến cho rằng ĐKTM đơn giản là tập hợp các khái niệm và phương pháp đã đề cập đến ở lĩnh vực khác, do đó nghiên cứu về ĐKTM chỉ đơn giản là đưa ra các phương án kết hợp và ứng dụng các kỹ thuật đã biết. Ý kiến này đương nhiên là không đúng. Chúng ta đều biết rằng, lý thuyết điều khiển không đơn giản chỉ là lý thuyết toán ứng dụng, bởi vì điều khiển có các yêu cầu khác, và do đó cần giải quyết những vấn đề khác. Ví dụ trong toán ứng dụng, các lời giải khác nhau của phương trình vi phân với điều kiện đầu và hàm kích thích khác nhau được quan tâm, trong điều khiển người ta thường quan tâm đến việc tìm hàm kích thích (tín hiệu điều khiển) để tạo ra lời giải (đáp ứng của hệ thống) thỏa mãn một số điều kiện nào đó. Đây là một bài toán khác, liên quan đến bài toán đầu tiên, nhưng lời giải của nó cần được phát triển bằng các cách hoàn toàn khác. Tương tự như vậy, tuy liên quan đến các lĩnh vực khác nhưng lý thuyết ĐKTM vẫn cần phát triển các khái niệm, cách tiếp cận và phương pháp mới do lĩnh vực điều khiển có những bài toán cần giải quyết hoàn toàn khác so với các lĩnh vực khác. Ví dụ, trong khi các phương pháp tính toán mềm (mạng thần kinh, logic mờ, giải thuật di truyền...) trong

khoa học máy tính thường liên quan đến các bài toán tĩnh và không cần thời gian thực, hệ thống điều khiển thường là hệ thống động và tất cả các luật điều khiển, dù thông minh hay không, đều phải có thể điều khiển hệ thống trong thời gian thực. Do đó, trong hầu hết các trường hợp chúng ta không thể áp dụng trực tiếp các phương pháp của khoa học máy tính vào những bài toán điều khiển. Sự cải tiến và mở rộng lý thuyết đã biết ở các lĩnh vực khác để áp dụng vào các hệ thống ĐKTM là rất cần thiết. Thêm vào đó, có những bài toán mới trong hệ thống thông minh chưa từng được nghiên cứu trước đây. hăng hạn như trường hợp hệ thống lai là sự kết hợp của hệ thống đang thái liên tục và trạng thái rời rạc.

6.2 Hướng nghiên cứu điều khiển thông minh

Có thể phân chia nghiên cứu trong lĩnh vực ĐKTM làm hai hướng chính:

- *Nghiên cứu lý thuyết* điều khiển thông thường, hướng tới các chức năng điều khiển ở cấp thực thi và cấp phối hợp
- *Nghiên cứu mô hình hóa*, phân tích và thiết kế hệ thống ra quyết định ở cấp cao hơn tìm thấy trong cấp tổ chức và quản lý.

Về lý thuyết, hướng nghiên cứu ĐKTM hiện nay là:

- Phân tích toán học tính hội tụ, ổn định, bền vững của hệ thống ĐKTM
- Phân tích so sánh toán học giữa các hệ thống ĐKTM với hệ thống điều khiển thông thường
- Các phương pháp phân tích, thiết kế hệ thống lai
- Kỹ thuật tái cấu trúc hệ thống điều khiển khi các sự cố lỗi xuất hiện
- Phương pháp đưa ra các mô hình trừu tượng từ các mô hình phương trình vi phân/sai phân để sử dụng ở cấp cao hơn của hệ thống ĐKTM
- Vấn đề giao tiếp ký hiệu/số ở cấp trung gian và cấp tổ chức
- Vấn đề tiết kiệm bộ nhớ, thời gian tính toán
- Các chiến lược học phức tạp dựa vào trực giác
- Nghiên cứu thiết kế hệ thống có khả năng suy luận, đưa ra những phương án giải quyết vấn đề mới (không đưa ra bởi người thiết kế) từ thông tin phản hồi không đầy đủ, không chắc chắn.

Về ứng dụng, hướng nghiên cứu ĐKTM hiện nay là:

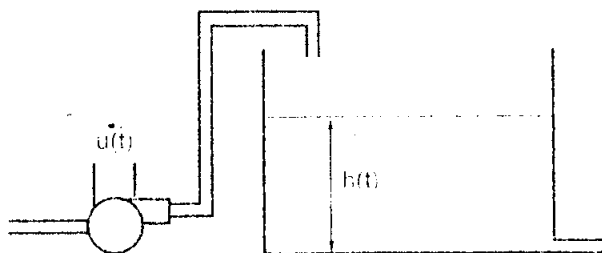
- Sử dụng mạng thần kinh để thực hiện chức năng học ở tất cả các cấp của hệ thống ĐKTM. Cụ thể ở cấp thực thi sử dụng tính chất xấp xỉ tổng quát của mạng thần kinh để thiết kế các bộ điều khiển thích nghi, ở cấp cao hơn sử dụng khả năng phân loại mẫu và khả năng lưu trữ thông tin để sử dụng trong các bộ hoạch định.
- Sử dụng logic mờ để thực hiện chức năng suy luận. Ở cấp thực thi thiết kế các bộ điều khiển mờ có khả năng suy luận theo cách của người thiết kế. Ở cấp phối hợp, có thể dùng logic mờ để giải quyết vấn đề giao tiếp ký hiệu/số.
- Sử dụng GA để giải bài toán tối ưu hóa ở tất cả các cấp của hệ thống thông minh. Ở cấp thực thi, tìm thông số bộ điều khiển để tối ưu hóa một chỉ tiêu chất lượng (liên quan đến sai số, thời gian đáp ứng, năng lượng...). Ở cấp tổ chức, sử dụng GA để hoạch định tối ưu các tác vụ để hoàn thành mục tiêu chung của hệ thống.
- Sử dụng kết hợp các kỹ thuật ĐKTM: hệ mờ thần kinh, hệ mờ di truyền, mạng thần kinh mờ... để giải các bài toán điều khiển ở cấp thực thi.

Chương 2

HỆ MỜ

2.1 GIỚI THIỆU

Xét hệ thống bồn chứa chất lỏng ở hình 2.1, ta cần thiết kế bộ điều khiển để giữ cho mực chất lỏng đầy bồn.



Hình 2.1 Hệ bồn chứa chất lỏng

Phương pháp điều khiển ON-OFF theo logic kinh điển điều khiển máy bơm theo chiến lược sau:

- Nếu chất lỏng chưa đầy bồn thì máy bơm chạy
- Nếu chất lỏng đầy bồn thì máy bơm dừng.

Phương pháp điều khiển ON-OFF đơn giản, có thể lập trình dễ dàng dùng PLC (*Programmable Logic Controller*). Tuy nhiên, do máy bơm chỉ có hai trạng thái “chạy” hoặc “dừng” nên chất lượng điều khiển không cao, đáp ứng của hệ thống có vọt lố, dao động.

Để nâng cao chất lượng điều khiển có thể thiết kế bộ điều khiển PID để điều khiển hệ thống, tuy nhiên cần phải biết mô hình toán học của hệ bồn chứa.

Trong khi đó, người vận hành mặc dù không biết mô hình toán của hệ bồn chứa vẫn có thể điều khiển hệ thống đạt chất lượng tốt theo chiến lược sau:

- Nếu mực chất lỏng “thấp” thì điện áp cấp cho máy bơm “lớn”.
- Nếu mực chất lỏng “cao” thì điện áp cấp cho máy bơm “nhỏ”.

Trong chiến lược điều khiển trên, ý nghĩa của các từ “thấp”, “cao” không rõ ràng, với một giá trị mức chất lỏng cụ thể, chúng ta không thể khẳng định được chắc chắn mức chất lỏng là “cao” hay “thấp”; tương tự ta cũng không khẳng định được chắc chắn điện áp cấp cho động cơ bằng bao nhiêu volt thì gọi là “lớn”, bao nhiêu volt thì gọi là “nhỏ”. Nói chung, ngôn ngữ tự nhiên mà con người sử dụng để suy nghĩ, trao đổi thông tin, xử lý thông tin có tính chất không rõ ràng. Mặc dù thông tin trao đổi không rõ ràng nhưng con người vẫn hiểu nhau, vẫn suy nghĩ và điều khiển rất tốt các đối tượng, do đó con người mong muốn thiết kế được bộ điều khiển bắt chước được khả năng xử lý thông tin không rõ ràng. Bộ điều khiển bắt chước theo chiến lược điều khiển dựa vào thông tin không rõ ràng của con người gọi là bộ điều khiển mờ (FLC – *Fuzzy Logic Controller*).

Cơ sở toán học của bộ điều khiển mờ là lý thuyết tập mờ và logic mờ đã được Zadeh đưa ra từ năm 1965 ([119]). Đến năm 1975 Mamdani áp dụng thành công lý thuyết mờ để thiết kế và thực thi bộ điều khiển mờ đầu tiên điều khiển động cơ hơi nước. Sau những công trình mở đường này, rất nhiều sản phẩm dân dụng cũng như sản phẩm kỹ thuật cao dùng logic mờ đã được phát triển ở Nhật, Mỹ và EU.

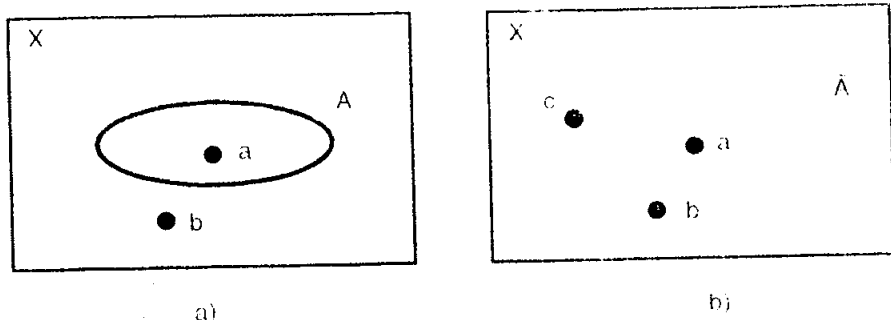
Chương này trình bày lý thuyết tập mờ, logic mờ, suy luận mờ và hệ mờ ([28], [76], [82], [119]). Các ứng dụng của hệ mờ vào lĩnh vực điều khiển sẽ được trình bày ở các chương sau.

2.2 TẬP HỢP MỜ

2.2.1 Khái niệm về tập hợp mờ

Đối với tập hợp kinh điển, biên của tập hợp là rõ ràng. Cho một phần tử bất kỳ chúng ta hoàn toàn có thể xác định được phần tử đó có thuộc tập hợp hay không. Xét tập hợp A ở hình 2.2a, trực quan ta thấy $a \in A$ và $b \notin A$.

Trái với tập hợp kinh điển, biên của tập hợp mờ không rõ ràng, do đó có một số phần tử ta không thể khẳng định là thuộc tập hợp mờ hay không. Ví dụ ở hình 2.2b ta không khẳng định được phần tử c thuộc tập hợp mờ $\tilde{A}$ hay không thuộc tập mờ $\tilde{A}$ (để phân biệt giữa tập mờ và tập kinh điển, trong quyển sách này chúng ta dùng các chữ cái có “dấu ngã trên” để đặt tên cho các tập mờ).



Hình 2.2 a) Biên của tập hợp rõ; b) Biên của tập mờ

Ví dụ 2.1 Chúng ta hãy xét các tập hợp được mô tả “mờ” sau đây:

- Tập $\tilde{B}$ gồm những số thực nhỏ hơn nhiều so với 6:

$$\tilde{B} = \{x \in R \mid x \ll 6\}$$

- Tập $\tilde{C}$ gồm những số thực gần bằng 6:

$$\tilde{C} = \{x \in R \mid x \approx 6\}$$

Những tập hợp được mô tả bởi các định nghĩa “mờ” như vậy có biên không rõ ràng, chúng ta không thể khẳng định $x = 2,5$ có thuộc $\tilde{B}$ hoặc $x = 3,5$ có thuộc $\tilde{C}$ hay không.

Nếu đã không khẳng định được một phần tử có thuộc tập mờ hay không thì cũng không khẳng định được là phần tử đó không thuộc tập mờ. Vậy một phần tử bất kỳ thuộc tập mờ bao nhiêu phần trăm? Giả sử câu trả lời đó có thì độ phụ thuộc của một phần tử vào tập mờ phải là một giá trị nằm trong đoạn $[0,1]$ (tức là từ 0% đến 100%). Hàm số cho biết độ phụ thuộc của các phần tử vào tập mờ gọi là *hàm liên thuộc* (*membership function*). Từ phát biểu mô tả tập hợp mờ ta không thể suy ra hàm liên thuộc. Do đó, hàm liên thuộc phải được nêu lên như là một điều kiện trong định nghĩa tập mờ.

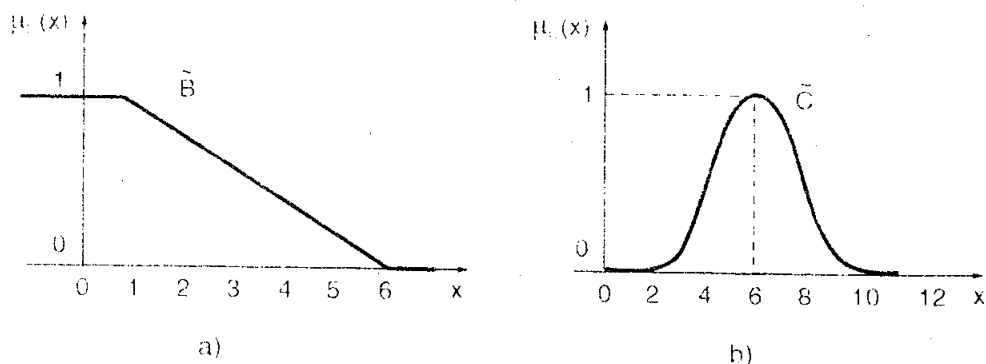
Định nghĩa 2.1

Tập mờ $\tilde{A}$ xác định trên tập cơ sở X là một tập hợp mà mỗi phần tử của nó là một cặp giá trị $(x, \mu_{\tilde{A}}(x))$, trong đó $x \in X$ và $\mu_{\tilde{A}}(x)$ là ánh xạ:

$$\mu_{\tilde{A}} : X \rightarrow [0,1] \quad (2.1)$$

Ảnh xạ $\mu_{\tilde{A}}(x)$ được gọi là hàm liên thuộc của tập mờ $\tilde{A}$. Hàm liên thuộc đặc trưng cho độ phụ thuộc của một phần bất kỳ thuộc tập cơ sở X vào tập mờ $\tilde{A}$. Nói cách khác, tập mờ xác định bởi hàm liên thuộc của nó.

Hàm liên thuộc có thể có dạng tuyến tính từng đoạn như hình 2.3a hay dạng trơn như hình 2.3b. Các đặc điểm của hàm liên thuộc sẽ được trình bày chi tiết trong mục kế tiếp.



Hình 2.3 a) Hàm liên thuộc của tập mờ $\tilde{B}$

b) Hàm liên thuộc của tập mờ $\tilde{C}$

Ký hiệu tập mờ

Tập mờ $\tilde{A}$ định nghĩa trên tập cơ sở X rời rạc hữu hạn được ký hiệu như sau:

$$\tilde{A} = \sum_i \frac{\mu_{\tilde{A}}(x_i)}{x_i} \quad (2.2)$$

Tập mờ $\tilde{A}$ định nghĩa trên tập cơ sở X liên tục vô hạn được ký hiệu như sau:

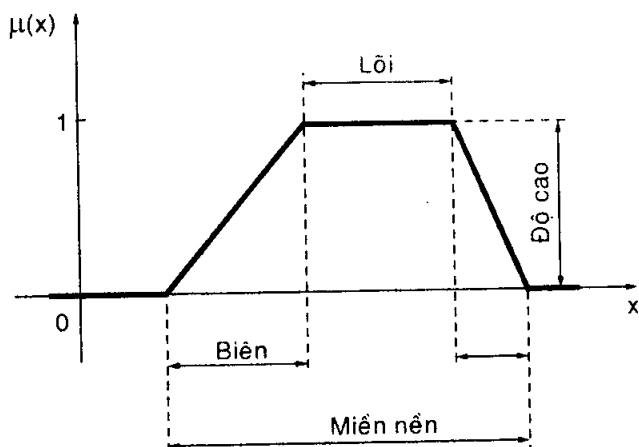
$$\tilde{A} = \int \frac{\mu_{\tilde{A}}(x)}{x} \quad (2.3)$$

Chú ý, trong cách ký hiệu như trên, dấu gạch ngang không phải là “dấu chia” mà chỉ là dấu phân cách, dấu Σ và $\int$ tương ứng không phải là “dấu cộng” hay “dấu tích phân” mà phải hiểu là “gồm các phần tử”.

2.2.2 Hàm liên thuộc

1- Các đặc điểm của hàm liên thuộc

Vì tập mờ được xác định bởi hàm liên thuộc nên cần định nghĩa một số thuật ngữ để mô tả các đặc điểm của hàm này. Để đơn giản, các hàm liên thuộc được trình bày ở các hình dưới đây đều liên tục nhưng các thuật ngữ được sử dụng tương đương cho tập mờ liên tục và tập mờ rời rạc.



Hình 2.4 Miền nền, lõi, biên và độ cao của tập mờ

Miền nền

Miền nền của hàm liên thuộc của tập mờ $\tilde{A}$ là vùng gồm các phần tử có độ phụ thuộc khác 0. Nghĩa là miền nền gồm các phần tử x của tập cơ sở X sao cho $\mu_{\tilde{A}}(x) > 0$.

Lõi

Lõi của hàm liên thuộc của tập mờ $\tilde{A}$ là vùng gồm các phần tử có độ phụ thuộc bằng 1, nghĩa là lõi gồm các phần tử x của tập cơ sở X sao cho $\mu_{\tilde{A}}(x) = 1$.

Biên

Biên của hàm liên thuộc của tập mờ $\tilde{A}$ là vùng gồm các phần tử có độ phụ thuộc khác 0 và nhỏ hơn 1, nghĩa là biên của tập mờ gồm các phần tử x của tập cơ sở X sao cho $0 < \mu_{\tilde{A}}(x) < 1$.

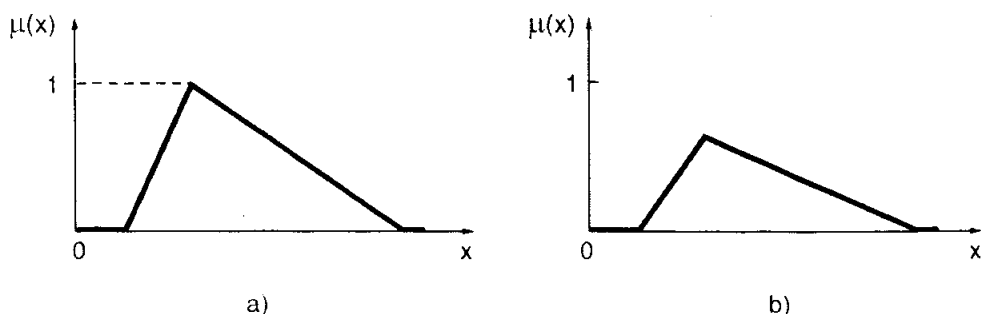
Độ cao

Độ cao của tập mờ $\tilde{A}$ là cận trên nhỏ nhất của hàm liên thuộc:

$$\text{hgt}(\tilde{A}) = \sup_{x \in X} \mu_{\tilde{A}}(x) \quad (2.4)$$

Tập mờ chính tắc

Tập mờ chính tắc là tập mờ có độ cao bằng 1. Hình 2.5 minh họa tập mờ chính tắc và không chính tắc.

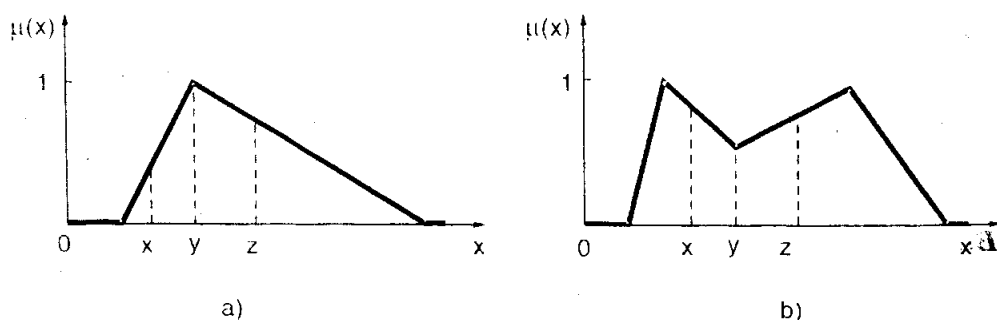


Hình 2.5 a) Tập mờ chính tắc; b) Tập mờ không chính tắc

Tập mờ lồi

Tập mờ lồi là tập mờ mà hàm liên thuộc của nó đơn điệu tăng, hoặc đơn điệu giảm, hoặc đơn điệu tăng sau đó đơn điệu giảm. Nói cách khác, cho ba phần tử bất kỳ x, y và z thuộc tập mờ $\tilde{A}$, nếu $x < y < z$ kéo theo $\mu_{\tilde{A}}(y) \geq \min[\mu_{\tilde{A}}(x), \mu_{\tilde{A}}(z)]$ thì $\tilde{A}$ được gọi là tập mờ lồi.

Hình 2.6 biểu diễn dạng tập mờ lồi và không lồi.



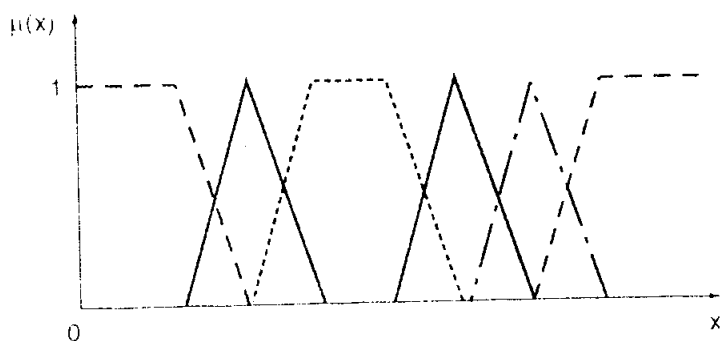
Hình 2.6 a) Tập mờ lồi; b) Tập mờ không lồi

Sự phân hoạch mờ

Các tập mờ $\tilde{A}_1, \tilde{A}_2, \dots, \tilde{A}_n$ định nghĩa trên tập cơ sở X được gọi là phân hoạch mờ nếu $\tilde{A}_i \neq \emptyset, \tilde{A}_i \neq X$ và:

$$\forall x \in X, \sum_{i=1}^n \mu_{\tilde{A}_i}(x) = 1 \quad (2.5)$$

Hình 2.7 trình bày một ví dụ về sự phân hoạch mờ. Nếu các tập mờ chính tắc và lồi tạo nên sự phân hoạch mờ thì không có nhiều hơn hai tập mờ chồng nhau.

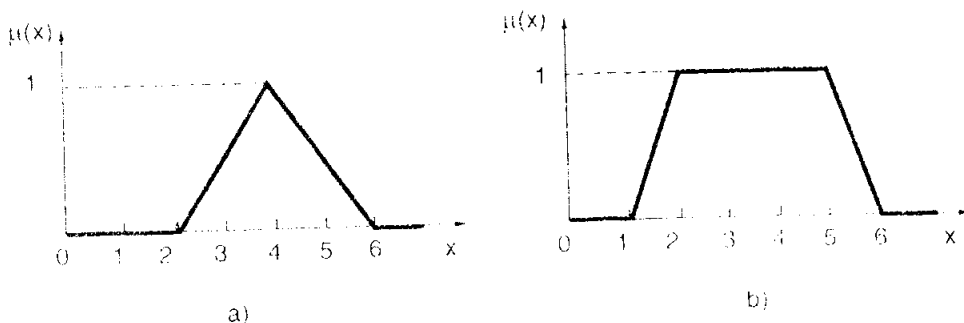


Hình 2.7 Các tập mờ được phân hoạch mờ

Số mờ

Nếu $\tilde{A}$ là một tập mờ lồi chính tắc xác định trên trục thực và chỉ có một phần tử có độ phụ thuộc bằng 1 thì $\tilde{A}$ được gọi là số mờ (H.2.8a).

Khoảng mờ



Hình 2.8 a) Số mờ “gần bằng 4”

b) Khoảng mờ “gần bằng 2 đến gần bằng 5”

Nếu $\tilde{A}$ là một tập mờ lõi chính tắc xác định trên trục thực và có nhiều hơn một phần tử có độ phụ thuộc bằng 1 thì $\tilde{A}$ được gọi là khoảng mờ (H.2.8b).

2- Các dạng hàm liên thuộc thường gặp

Đa số các dạng thường gặp của hàm liên thuộc là chính tắc và lõi. Tuy nhiên, nhiều phép toán trên tập mờ cũng là phép toán trên hàm liên thuộc, tạo ra kết quả là các tập mờ không chính tắc (chẳng hạn như phép giao, mục 1 của phần 2.2.3) và không lõi (chẳng hạn như phép hợp, mục 2 của phần 2.2.3).

Hàm liên thuộc có thể đối xứng hoặc không đối xứng. Các hàm này thường được định nghĩa trên các tập cơ sở một chiều, nhưng tất nhiên các hàm này cũng có thể được định nghĩa trên các tập cơ sở nhiều chiều. Hàm liên thuộc định nghĩa trên tập cơ sở một chiều là các đường, hàm liên thuộc định nghĩa trên tập cơ sở hai chiều là các mặt, hàm liên thuộc định nghĩa trên tập cơ sở từ ba chiều trở lên là các mặt bậc cao (siêu diện – *hypersurface*).

Hàm liên thuộc định nghĩa trên tập cơ sở một chiều tổng quát có dạng tuyến tính từng đoạn (H.2.9a,d), hoặc là các đường cong “trơn” (H.2.9g,o), hàm liên thuộc có dạng hình chữ nhật (H.2.9e) tương ứng với tập rõ và hàm liên thuộc dạng vạch (H.2.9f) là giá trị rõ. Hàm liên thuộc dạng “trơn” có biểu thức $\mu(x)$ thường chứa hàm mũ nên để tính được độ phụ thuộc của các phần tử cần thực hiện nhiều phép tính, thời gian tính toán lâu và rất khó thực hiện trên vi xử lý cấp thấp. Vì vậy, trong kỹ thuật điều khiển mờ thông thường các hàm liên thuộc dạng “trơn” thường được thay bằng hàm liên thuộc tuyến tính hóa từng đoạn.

Trong các biểu thức hàm liên thuộc dưới đây, để gợi nhớ thông số của hàm liên thuộc sử dụng các ký hiệu sau:

$$L = \text{Left}, R = \text{Right}, C = \text{Center}, S = \text{Slope}, W = \text{Width}$$

Dạng vai trái (H.2.9a):

$$\mu_{\text{left}}(x, L, R) = \begin{cases} \frac{R-x}{R-L} & \text{nếu } L \leq x \leq R \\ 1 & \text{nếu } x < L \\ 0 & \text{nếu } x > R \end{cases} \quad (2.6)$$

Dạng vai phải (H.2.9b):

$$\mu_{right}(x, L, R) = \begin{cases} \frac{x-L}{R-L} & \text{nếu } L \leq x \leq R \\ 0 & \text{nếu } x < L \\ 1 & \text{nếu } x > R \end{cases} \quad (2.7)$$

Dạng tam giác (H.2.9c):

$$\mu_{tri}(x, L, C, R) = \begin{cases} \frac{x-L}{C-L} & \text{nếu } L \leq x \leq C \\ \frac{R-x}{R-C} & \text{nếu } C < x \leq R \\ 0 & \text{nếu } x < L \text{ hoặc } x > R \end{cases} \quad (2.8)$$

Dạng hình thang (H.2.9d):

$$\mu_{trap}(x, L, C_1, C_2, R) = \begin{cases} \frac{x-L}{C_1-L} & \text{nếu } L \leq x \leq C_1 \\ \frac{R-x}{R-C_2} & \text{nếu } C_2 \leq x \leq R \\ 1 & \text{nếu } C_1 < x < C_2 \\ 0 & \text{nếu } x < L \text{ hoặc } x > R \end{cases} \quad (2.9)$$

Dạng chữ nhật (tập rõ) (H.2.9e):

$$\mu_{rec}(x, C_1, C_2) = \begin{cases} 1 & \text{nếu } C_1 \leq x \leq C_2 \\ 0 & \text{nếu } x < C_1 \text{ hoặc } x > C_2 \end{cases} \quad (2.10)$$

Dạng vạch (Singleton) (H.2.9f):

$$\mu_{singleton}(x, C) = \begin{cases} 1 & \text{nếu } x = C \\ 0 & \text{nếu } x \neq C \end{cases} \quad (2.11)$$

Dạng Z (H.2.9g):

$$\mu_Z(x, L, R) = \begin{cases} 1 - 2\left(\frac{x-L}{R-L}\right)^2 & \text{nếu } L \leq x \leq (L+R)/2 \\ 2\left(\frac{R-x}{R-L}\right)^2 & \text{nếu } (L+R)/2 < x \leq R \\ 1 & \text{nếu } x < L \\ 0 & \text{hoặc } x > R \end{cases} \quad (2.12)$$

Dạng S (H.2.9h):

$$\mu_S(x, L, R) = \begin{cases} 2\left(\frac{x-L}{R-L}\right)^2 & \text{nếu } L \leq x \leq (L+R)/2 \\ 1 - 2\left(\frac{R-x}{R-L}\right)^2 & \text{nếu } (L+R)/2 < x \leq R \\ 0 & \text{nếu } x < L \\ 1 & \text{hoặc } x > R \end{cases} \quad (2.13)$$

Dạng pi (H.2.9i):

$$\mu_{pi}(x, L, C, R) = \begin{cases} 2\left(\frac{x-L}{C-L}\right)^2 & \text{nếu } L \leq x \leq (L+C)/2 \\ 1 - 2\left(\frac{C-x}{C-L}\right)^2 & \text{nếu } (L+C)/2 < x \leq C \\ 1 - 2\left(\frac{x-C}{R-C}\right)^2 & \text{nếu } C < x \leq (C+R)/2 \\ 2\left(\frac{R-x}{R-C}\right)^2 & \text{nếu } (C+R)/2 < x \leq R \\ 0 & \text{nếu } x < L \text{ hoặc } x > R \end{cases} \quad (2.14)$$

Dạng sigmoid ($S < 0$: hình 2.9j và $S > 0$: hình 2.9k):

$$\mu_{sig}(x, S, C) = \frac{1}{1 - e^{-S(x-C)}} \quad (2.15)$$

Dạng dsig: Hiệu hai hàm sigmoid (H.2.9l)

$$\mu_{sig}(x, S_1, C_1, S_2, C_2) = \frac{1}{1 - e^{-S_1(x-C_1)}} - \frac{1}{1 - e^{-S_2(x-C_2)}} \quad (2.16)$$

Dạng Gauss (H.2.9m):

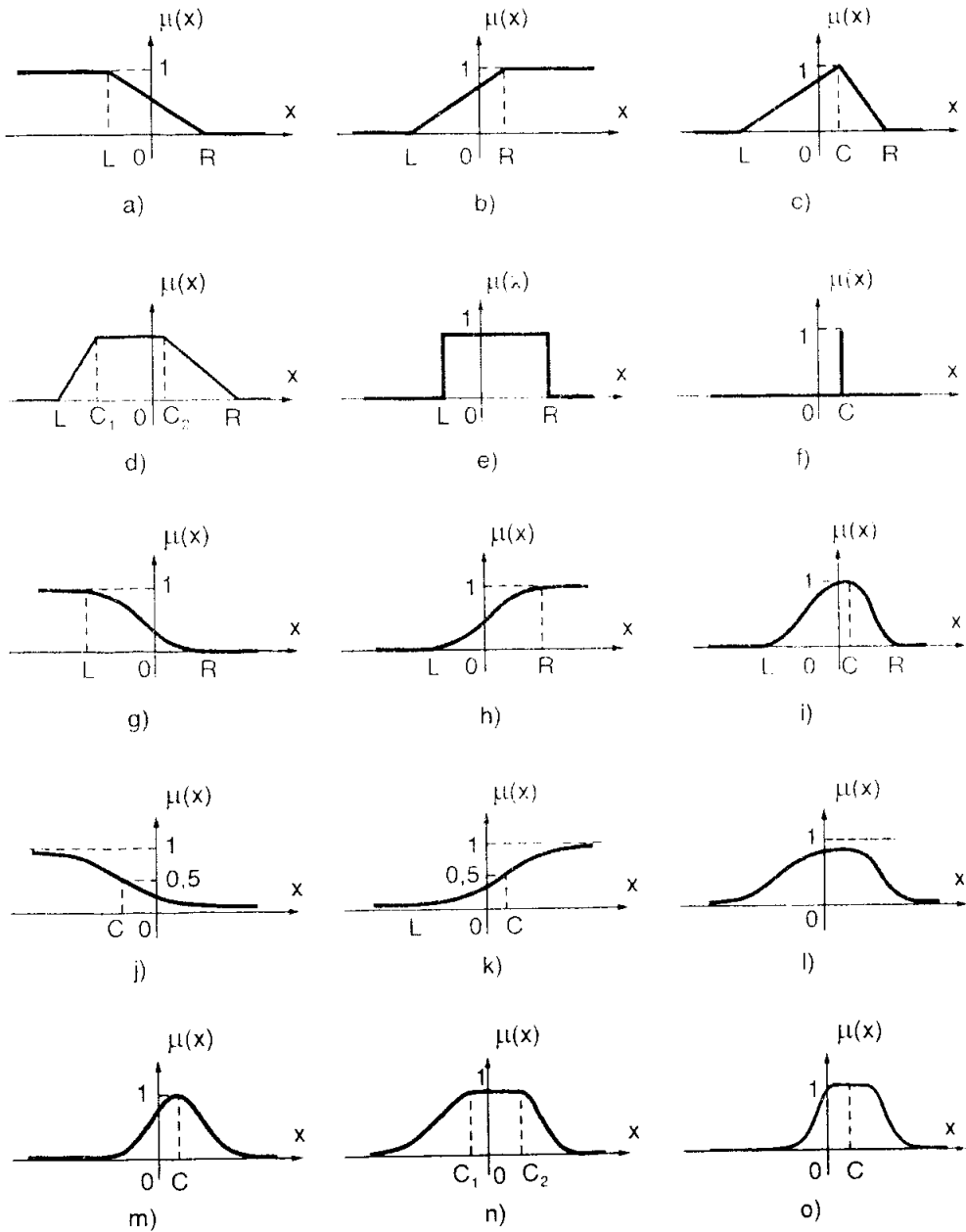
$$\mu_{gauss}(x, W, C) = \exp\left[-(x-C)^2 / 2W^2\right] \quad (2.17)$$

Dạng Gauss hai phía (H.2.9n):

$$\mu_{gauss2}(x, W, C) = \begin{cases} \exp\left[-(x-C_1)^2 / 2W_1^2\right] & \text{nếu } x \leq C_1 \\ 1 & \text{nếu } C_1 < x < C_2 \\ \exp\left[-(x-C_2)^2 / 2W_2^2\right] & \text{nếu } x \geq C_2 \end{cases} \quad (2.18)$$

Dạng chuông (H.2.9o):

$$\mu_{bell}(x, W, S, C) = 1 / \left(1 + |(x-C)/W|^{2S}\right) \quad (2.19)$$



Hình 2.9 Các dạng hàm liên thuộc cơ bản

- | | | |
|---------------------|----------------------------|------------------|
| a) Dạng vai trái; | b) Dạng vai phải; | c) Dạng tam giác |
| d) Dạng hình thang; | e) Dạng chữ nhật (tập rõ); | f) Singleton |
| g) Dạng Z; | h) Dạng S; | i) Dạng pi |
| j, k) Dạng sigmoid | l) Dạng dsigmoid; | |
| m) Dạng gauss | n) Dạng gauss hai phía; | o) Dạng chuông |

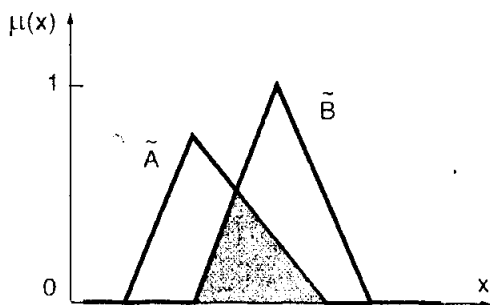
2.2.3 Các phép toán trên tập mờ

1- Phép giao

Định nghĩa 2.2 (theo Zadeh)

Giao của hai tập mờ $\tilde{A}$ và $\tilde{B}$ có cùng cơ sở X là một tập mờ xác định trên cơ sở X có hàm liên thuộc xác định bởi biểu thức:

$$\tilde{A} \cap \tilde{B} : \mu_{\tilde{A} \cap \tilde{B}}(x) = \min\{\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)\} \quad (2.20)$$



Hình 2.10 Giao của hai tập mờ

Tuy nhiên có nhiều cách khác định nghĩa giao của hai tập mờ. Tổng quát giao của hai tập mờ được biểu diễn bởi chuẩn T (T -norm).

$$\mu_{\tilde{A} \cap \tilde{B}}(x) = T(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) \quad (2.21)$$

Chuẩn T là ánh xạ $[0,1] \times [0,1] \rightarrow [0,1]$ thỏa mãn tính chất:

- $T(a,1) = a$
- $T(a,b) \leq T(c,d)$ khi $a \leq c, b \leq d$
- $T(a,b) = T(b,a)$ (tính giao hoán)
- $T(T(a,b),c) = T(a,T(b,c))$ (tính kết hợp)

Các công thức sau đây thường được sử dụng để lấy giao hai tập mờ. Trong điều khiển mờ, chuẩn T thường sử dụng là MIN (công thức Zadeh) hay PROD (công thức xác suất).

Công thức Lukasiewicz:

$$T(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) = \max\{0, \mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x) - 1\} \quad (2.22)$$

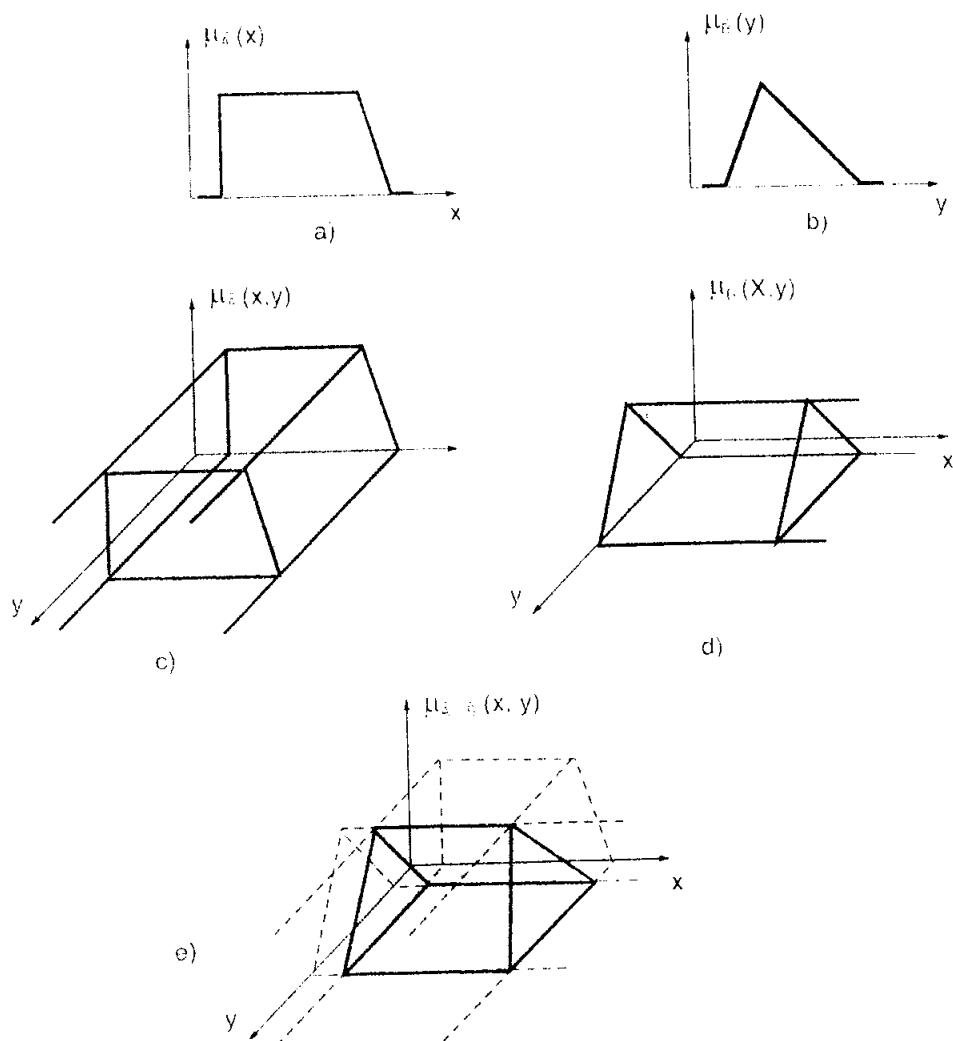
Công thức Einstein:

$$T(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) = \frac{\mu_{\tilde{A}}(x)\mu_{\tilde{B}}(x)}{2 - (\mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)) - \mu_{\tilde{A}}(x)\mu_{\tilde{B}}(x)} \quad (2.23)$$

Công thức xác suất (toán tử PROD):

$$T(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(y)) = \mu_{\tilde{A}}(x) \mu_{\tilde{B}}(y) \quad (2.24)$$

Trên đây chúng ta vừa xét các công thức tìm giao hai tập mờ cùng cơ sở. Các công thức trên cũng có thể sử dụng để tìm giao hai tập mờ không cùng cơ sở bằng cách đưa cả hai tập mờ về chung một cơ sở là tích hai cơ sở đã cho.



Hình 2.11 a) Tập mờ $\tilde{A}$ trên tập cơ sở X ; b) Tập mờ $\tilde{B}$ trên tập cơ sở Y
 c) Tập mờ $\tilde{A}'$ trên tập cơ sở $X \times Y$; d) Tập mờ $\tilde{B}'$ trên tập cơ sở $X \times Y$
 e) Giao của hai tập mờ $\tilde{A}' \cap \tilde{B}'$ trên tập cơ sở $X \times Y$

Chẳng hạn có hai tập mờ $\tilde{A}$ định nghĩa trên cơ sở X và $\tilde{B}$ định nghĩa trên cơ sở Y . Do hai cơ sở X và Y độc lập với nhau nên hàm liên thuộc $\mu_{\tilde{A}}(x)$, $x \in X$ của tập mờ $\tilde{A}$ sẽ không phụ thuộc vào Y và ngược lại $\mu_{\tilde{B}}(y)$, $y \in Y$ của tập mờ $\tilde{B}$ không phụ thuộc vào X . Trên cơ sở mới là tập tích $X \times Y$, hàm liên thuộc của tập mờ $\tilde{A}$ là một mặt dọc theo trục Y (H.2.11c) và hàm liên thuộc của tập mờ $\tilde{B}$ là một mặt dọc theo trục X (H.2.11d). Để phân biệt, ký hiệu $\tilde{A}'$ (hoặc $\tilde{B}'$) dùng để chỉ tập mờ $\tilde{A}$ (hoặc $\tilde{B}$) trên cơ sở mới là $X \times Y$. Với những ký hiệu đó thì:

$$\mu_{\tilde{A}'}(x, y) = \mu_{\tilde{A}}(x), \forall y \in Y \quad (2.25)$$

$$\mu_{\tilde{B}'}(x, y) = \mu_{\tilde{B}}(x), \forall x \in X \quad (2.26)$$

Sau khi đưa hai tập mờ về cùng một cơ sở, ta có thể áp dụng các công thức đã trình bày ở phần trên để tìm giao. Chẳng hạn, với công thức Zadeh ta có:

$$\mu_{\tilde{A}' \cap \tilde{B}'}(x, y) = \min\{\mu_{\tilde{A}'}(x, y), \mu_{\tilde{B}'}(x, y)\} \quad (2.27)$$

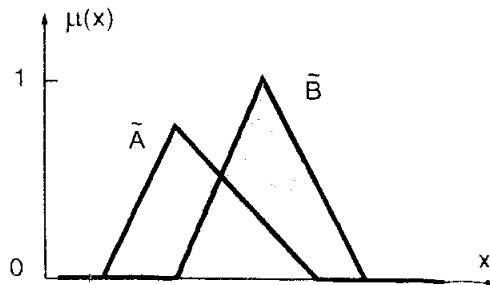
Các công thức khác cũng áp dụng tương tự.

2- Phép hợp

Định nghĩa 2.3 (theo Zadeh)

Hợp của hai tập mờ $\tilde{A}$ và $\tilde{B}$ có cùng cơ sở X là một tập mờ xác định trên cơ sở X có hàm liên thuộc:

$$\tilde{A} \cup \tilde{B} : \mu_{\tilde{A} \cup \tilde{B}}(x) = \max\{\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)\} \quad (2.28)$$



Hình 2.12 Hợp của hai tập mờ

Tổng quát hợp của hai tập mờ được biểu diễn bởi chuẩn S (S -norm).

$$\mu_{\tilde{A} \cup \tilde{B}}(x) = S(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) \quad (2.29)$$

Chuẩn S là ánh xạ $[0,1] \times [0,1] \rightarrow [0,1]$ thỏa mãn tính chất:

- $S(a,0) = a$
- $S(a,b) \leq S(c,d)$ khi $a \leq c, b \leq d$
- $S(a,b) = S(b, a)$ (tính giao hoán)
- $S(S(a,b),c) = S(a,S(b,c))$ (tính kết hợp)

Các công thức sau đây thường được sử dụng để lấy hợp hai tập mờ. Trong điều khiển mờ, chuẩn S thường được sử dụng là MAX (công thức Zadeh).

Công thức Lukasiewicz (tổng bị chặn BSUM – Bounded Sum):

$$S(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) = \min\{1, \mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)\} \quad (2.30)$$

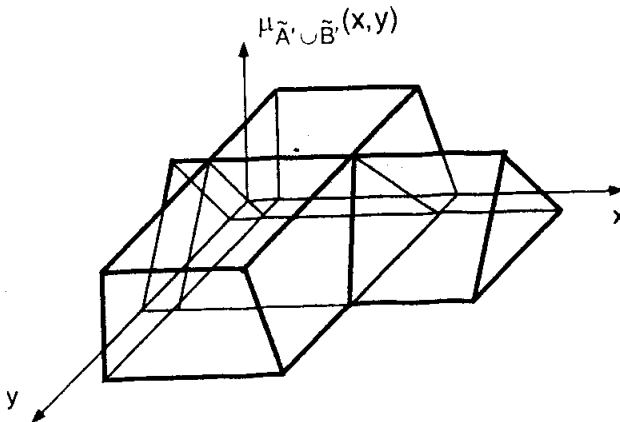
Công thức Einstein:

$$S(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) = \frac{\mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)}{1 + \mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x)} \quad (2.31)$$

Công thức xác suất:

$$S(\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(x)) = \mu_{\tilde{A}}(x) + \mu_{\tilde{B}}(x) - \mu_{\tilde{A}}(x)\mu_{\tilde{B}}(x) \quad (2.32)$$

Để tìm hợp của hai tập mờ không cùng cơ sở, trước tiên ta đưa hai tập mờ về cùng một cơ sở tương tự như đối với phép giao. Sau đó áp dụng các công thức tính hợp như vừa trình bày.



Hình 2.13 .Phép hợp hai tập mờ không cùng cơ sở

Ví dụ hợp của hai tập mờ $\tilde{A}$ định nghĩa trên cơ sở X và $\tilde{B}$ định nghĩa trên cơ sở Y theo công thức Zadeh là:

$$\mu_{\tilde{A} \cup \tilde{B}}(x, y) = \max\{\mu_{\tilde{A}}(x, y), \mu_{\tilde{B}}(x, y)\} \quad (2.33)$$

trong đó
$$\mu_{\tilde{A}}(x, y) = \mu_{\tilde{A}}(x), \forall y \in Y \quad (2.34)$$

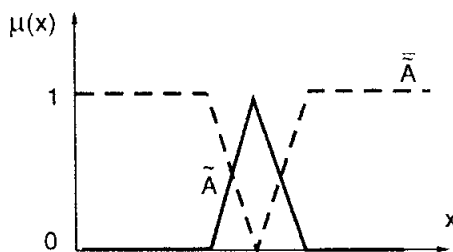
$$\mu_{\tilde{B}}(x, y) = \mu_{\tilde{B}}(y), \forall x \in X \quad (2.35)$$

3- Phép bù

Định nghĩa 2.4

Bù của tập mờ $\tilde{A}$ định nghĩa trên tập cơ sở X là một tập mờ xác định trên cơ sở X có hàm liên thuộc xác định bởi biểu thức:

$$\bar{\tilde{A}} : \mu_{\bar{\tilde{A}}}(x) = 1 - \mu_{\tilde{A}}(x) \quad (2.36)$$



Hình 2.14 Phép bù của tập mờ

2.2.4 Tính chất của tập hợp mờ

Tập mờ có các tính tương tự như tập rõ. Cụ thể như sau:

Tính giao hoán:

$$\begin{aligned} \tilde{A} \cup \tilde{B} &= \tilde{B} \cup \tilde{A} \\ \tilde{A} \cap \tilde{B} &= \tilde{B} \cap \tilde{A} \end{aligned} \quad (2.37)$$

Tính kết hợp:

$$\begin{aligned} (\tilde{A} \cup \tilde{B}) \cup \tilde{C} &= \tilde{A} \cup (\tilde{B} \cup \tilde{C}) \\ (\tilde{A} \cap \tilde{B}) \cap \tilde{C} &= \tilde{A} \cap (\tilde{B} \cap \tilde{C}) \end{aligned} \quad (2.38)$$

Tính phân phối:

$$\begin{aligned} \tilde{A} \cup (\tilde{B} \cap \tilde{C}) &= (\tilde{A} \cup \tilde{B}) \cap (\tilde{A} \cup \tilde{C}) \\ \tilde{A} \cap (\tilde{B} \cup \tilde{C}) &= (\tilde{A} \cap \tilde{B}) \cup (\tilde{A} \cap \tilde{C}) \end{aligned} \quad (2.39)$$

Tính bắc cầu:

$$\text{Nếu } \tilde{A} \subseteq \tilde{B} \subseteq \tilde{C} \text{ thì } \tilde{A} \subseteq \tilde{C} \quad (2.40)$$

Tính lặp:

$$\overline{\overline{\tilde{A}}} = \tilde{A} \quad (2.41)$$

2.3 QUAN HỆ MỜ

2.3.1 Định nghĩa

Để biểu diễn mối tương quan không rõ ràng, không chắc chắn giữa các phần tử trong không gian tích, người ta dùng **quan hệ mờ**.

Định nghĩa 2.5

Quan hệ mờ n ngôi trên $X_1 \times X_2 \times \dots \times X_n$, ký hiệu $\tilde{R}$, là một tập mờ định nghĩa trên tập cơ sở n chiều $X_1 \times X_2 \times \dots \times X_n$. Trường hợp đặc biệt $n=2$ ta gọi $\tilde{R}$ là quan hệ mờ hai ngôi.

Quan hệ mờ được xác định bởi hàm liên thuộc. Hàm liên thuộc của quan hệ mờ cho biết mức độ tương quan giữa các phần tử:

$$\mu_{\tilde{R}}(x_1, x_2, \dots, x_n) : X_1 \times X_2 \times \dots \times X_n \rightarrow [0, 1] \quad (2.42)$$

Vì quan hệ mờ là cũng tập mờ (trên cơ sở nhiều chiều) nên quan hệ mờ được ký hiệu như tập mờ. Quan hệ mờ trên tập cơ sở rời rạc được ký hiệu là:

$$\tilde{R} = \sum_{x_1 \in X_1, \dots, x_n \in X_n} \frac{\mu_{\tilde{R}}(x_1, x_2, \dots, x_n)}{(x_1, x_2, \dots, x_n)} \quad (2.43)$$

hay quan hệ mờ $\tilde{R}$ trên tập cơ sở liên tục vô hạn được ký hiệu như sau:

$$\tilde{R} = \int_{X_1 \times \dots \times X_n} \frac{\mu_{\tilde{R}}(x_1, x_2, \dots, x_n)}{(x_1, x_2, \dots, x_n)} \quad (2.44)$$

Quan hệ mờ trên tập cơ sở rời rạc hai chiều còn có thể biểu diễn dưới dạng ma trận.

Ví dụ 2.2 Cho $X_1 = \{1, 2, 3\}$; $X_2 = \{2, 3, 4\}$

Hàm liên thuộc của quan hệ mờ “gần bằng” định nghĩa trên tập tích $X_1 \times X_2$ là:

$$\mu_{R_s}(x_1, x_2) = \min\left\{\frac{x_1}{x_2}, \frac{x_2}{x_1}\right\}, x_1 \in X_1, x_2 \in X_2$$

Theo ký hiệu của Zadeh:

$$R_{\approx} = \frac{0,5}{(1,2)} + \frac{0,33}{(1,3)} + \frac{0,25}{(1,4)} + \frac{1}{(2,2)} + \frac{0,67}{(2,3)} + \frac{0,5}{(2,4)} + \frac{0,67}{(3,2)} + \frac{1}{(3,3)} + \frac{0,75}{(3,4)}$$

Viết dưới dạng ma trận quan hệ mờ:

$$R_{\approx} = \begin{matrix} & \begin{matrix} 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0,5 & 0,33 & 0,25 \\ 1 & 0,67 & 0,5 \\ 0,67 & 1 & 0,75 \end{bmatrix} \end{matrix}$$

Vì bản chất quan hệ mờ là tập mờ nên quan hệ mờ có các phép toán hợp, giao, bù như tập hợp mờ. Các tính chất của tập mờ như giao hoán, kết hợp, phân phối cũng áp dụng được cho quan hệ mờ.

2.3.2 Sự hợp thành của quan hệ mờ

Có hai loại hợp thành là sự hợp thành giữa quan hệ mờ với quan hệ mờ và sự hợp thành giữa tập mờ với quan hệ mờ. Tuy nhiên bản chất của hai sự hợp thành trên là như nhau vì quan hệ mờ là tập mờ định nghĩa trên tập cơ sở nhiều chiều. Quyển sách này *chỉ đề cập đến sự hợp thành giữa tập mờ với quan hệ mờ* vì sự hợp thành giữa tập mờ với quan hệ mờ là cơ sở toán học của việc suy luận mờ, và sẽ được ứng dụng trong điều khiển mờ.

Định nghĩa 2.6

Giả sử $\tilde{R}$ là quan hệ mờ trên $X \times Y$, $\tilde{A}$ là một tập mờ trên X , sự hợp thành của $\tilde{R}$ và $\tilde{A}$ là tập mờ $\tilde{B}$, ký hiệu là:

$$\tilde{B} = \tilde{A} \circ \tilde{R} \quad (2.45)$$

và $\tilde{B}$ được xác định bởi:

$$\mu_{\tilde{B}}(y) = \mu_{\tilde{A} \circ \tilde{R}}(y) = S\{T(\mu_{\tilde{A}}(x), \mu_{\tilde{R}}(x, y))\} \quad (2.46)$$

Toán tử S được dùng là MAX hoặc SUM, toán tử T được dùng là MIN hoặc PROD, vì vậy có bốn công thức hợp thành thường dùng là:

Công thức hợp thành MAX-MIN:

$$\mu_{\tilde{B}}(y) = \mu_{\tilde{A} \circ \tilde{R}}(y) = \max_x \min(\mu_{\tilde{A}}(x), \mu_{\tilde{R}}(x, y)) \quad (2.47)$$

Công thức hợp thành MAX-PROD:

$$\mu_{\tilde{B}}(y) = \mu_{\tilde{A} \circ \tilde{R}}(y) = \max_x (\mu_{\tilde{A}}(x) \cdot \mu_{\tilde{R}}(x, y)) \quad (2.48)$$

Công thức hợp thành SUM-MIN:

$$\mu_{\tilde{B}}(y) = \mu_{\tilde{A} \circ \tilde{R}}(y) = \sum_x \min(\mu_{\tilde{A}}(x), \mu_{\tilde{R}}(x, y)) \quad (2.49)$$

Công thức hợp thành SUM-PROD:

$$\mu_{\tilde{B}}(y) = \mu_{\tilde{A} \circ \tilde{R}}(y) = \sum_x (\mu_{\tilde{A}}(x) \cdot \mu_{\tilde{R}}(x, y)) \quad (2.50)$$

Trong điều khiển, thường áp dụng công thức hợp thành MAX-MIN hoặc MAX-PROD.

Ý nghĩa của sự hợp thành là khi biết quan hệ $\tilde{R}$ trên tập cơ sở $X \times Y$, nếu cho tập mờ $\tilde{A}$ thì chúng ta có thể xác định được tập mờ $\tilde{B}$ có quan hệ $\tilde{R}$ với $\tilde{A}$.

Ví dụ 2.3 Cho $X_1 = \{1, 2, 3\}$; $X_2 = \{2, 3, 4\}$

Quan hệ mờ “gần bằng” trong ví dụ 2.2 là:

$$R_{\approx} = \begin{matrix} & \begin{matrix} 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} & \begin{bmatrix} 0,5 & 0,33 & 0,25 \\ 1 & 0,67 & 0,5 \\ 0,67 & 1 & 0,75 \end{bmatrix} \end{matrix}$$

Giả sử tập mờ $\tilde{A}$ “gần bằng 3” (trên tập cơ sở X_1) định nghĩa như sau:

$$\tilde{A} = \frac{0}{1} + \frac{0,5}{2} + \frac{1}{3}$$

Tính $\tilde{B} = \tilde{A} \circ \tilde{R}$.

Theo công thức hợp thành MAX-MIN (2.47), tập mờ $\tilde{B}$ được tính như sau:

$$\begin{aligned} \mu_{\tilde{B}}(2) &= \max \left\{ \min(\mu_{\tilde{A}}(1), \mu_{\tilde{R}}(1,2)), \min(\mu_{\tilde{A}}(2), \mu_{\tilde{R}}(2,2)), \right. \\ &\quad \left. \min(\mu_{\tilde{A}}(3), \mu_{\tilde{R}}(3,2)) \right\} \\ &= \max \{ \min(0; 0,5), \min(0,5; 1), \min(1; 0,67) \} = 0,67 \end{aligned}$$

$$\begin{aligned}
\mu_{\tilde{B}}(3) &= \max \left\{ \begin{array}{l} \min(\mu_{\tilde{A}}(1), \mu_{\tilde{R}}(1, 3)), \min(\mu_{\tilde{A}}(2), \mu_{\tilde{R}}(2, 3)), \\ \min(\mu_{\tilde{A}}(3), \mu_{\tilde{R}}(3, 3)) \end{array} \right\} \\
&= \max \{ \min(0; 0,33), \min(0,5; 0,67), \min(1, 1) \} = 1 \\
\mu_{\tilde{B}}(4) &= \max \left\{ \begin{array}{l} \min(\mu_{\tilde{A}}(1), \mu_{\tilde{R}}(1, 4)), \min(\mu_{\tilde{A}}(2), \mu_{\tilde{R}}(2, 4)), \\ \min(\mu_{\tilde{A}}(3), \mu_{\tilde{R}}(3, 4)) \end{array} \right\} \\
&= \max \{ \min(0; 0,25), \min(0,5; 0,5), \min(1; 0,75) \} = 0,75
\end{aligned}$$

Vậy:
$$\tilde{B} = \frac{0,67}{2} + \frac{1}{3} + \frac{0,75}{4}$$

Do đó khi biết quan hệ mờ “gần bằng” $\tilde{R}_z$, nếu biết tập mờ “gần bằng 3” $\tilde{A}$ thì ta suy ra được tập mờ $\tilde{B}$ có quan hệ “gần bằng” tập mờ $\tilde{A}$ (rõ ràng như ví dụ trên ta thấy $\tilde{B}$ cũng là tập mờ “gần bằng 3” (định nghĩa trên tập cơ sở X_2)).

Sự hợp thành có vai trò rất quan trọng trong việc suy luận mờ. Các mục kế tiếp sẽ trình bày chi tiết về vấn đề này.

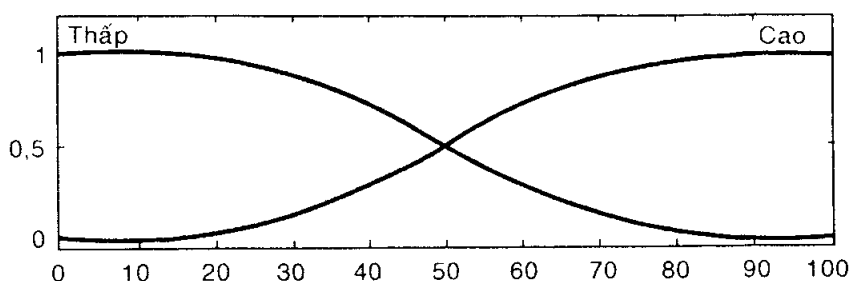
2.4 BIẾN NGÔN NGỮ VÀ GIÁ TRỊ NGÔN NGỮ

Con người suy nghĩ và giao tiếp với nhau bằng ngôn ngữ tự nhiên. Vì vậy muốn thiết kế một bộ điều khiển bắt chước sự suy nghĩ, xử lý thông tin và ra quyết định của con người, trước tiên chúng ta cần phải biểu diễn được ngôn ngữ tự nhiên bằng mô hình toán học. Đặc điểm của ngôn ngữ tự nhiên là chứa thông tin mơ hồ, không chắc chắn, mà tập hợp mờ cũng chứa thông tin mơ hồ không chắc chắn nên rõ ràng chúng ta thấy ngay rằng có thể dùng tập mờ để biểu diễn ngôn ngữ tự nhiên. Trong mục này chúng ta sẽ xét khái niệm biến ngôn ngữ. Trước tiên chúng ta xét khái niệm biến mờ.

Biến mờ

Biến mờ là biến được đặc trưng bởi bộ 3 phần tử $(\alpha, X, \mu_\alpha(x))$, trong đó α là tên biến, X là tập hợp cơ sở, $\mu_\alpha(x)$ là một tập mờ định nghĩa trên cơ sở X biểu diễn sự hạn chế mờ (*fuzzy restriction*) ngụ ý bởi α .

Ví dụ trong bài toán điều khiển mức chất lỏng, chúng ta có thể định nghĩa các biến mờ sau: (*cao*, X , $\mu_{cao}(x)$) và (*thấp*, X , $\mu_{thấp}(x)$). Hình 2.15 minh họa hàm liên thuộc của $\mu_{cao}(x)$ và $\mu_{thấp}(x)$ của hai biến mờ trên.



Hình 2.15 Hàm liên thuộc của hai tập mờ mô tả biến mờ “cao”, “thấp”

Biến ngôn ngữ

Biến ngôn ngữ là biến mà giá trị của nó là các từ (*word*). Ví dụ “mức chất lỏng” là biến ngôn ngữ thì nó có thể có giá trị là các từ “cao”, “thấp”.

Trong lý thuyết tập mờ biến ngôn ngữ được định nghĩa như sau: Biến ngôn ngữ là biến bậc cao hơn biến mờ, nó lấy biến mờ làm giá trị. Theo định nghĩa này, biến ngôn ngữ “mức chất lỏng” có thể nhận một trong các giá trị (*cao*, X , $\mu_{cao}(x)$) và (*thấp*, X , $\mu_{thấp}(x)$). Dễ thấy định nghĩa này rõ ràng hơn, “mức chất lỏng” có thể nhận giá trị “cao” (hoặc “thấp”), trong đó từ “cao” (hoặc “thấp”) được mô tả bởi tập mờ $\mu_{cao}(x)$ (hoặc $\mu_{thấp}(x)$) xác định trên cơ sở X .

Vì biến mờ là giá trị của biến ngôn ngữ nên nó còn được gọi là *giá trị ngôn ngữ* (*linguistic term*).

2.5 LOGIC MỜ

Tương tự như lý thuyết tập hợp kinh điển là nền tảng cho logic kinh điển, lý thuyết tập mờ là nền tảng cho logic mờ. Điều này có nghĩa là các phép toán logic mờ dựa trên các phép toán trên tập mờ. Tương ứng với các phép toán giao, hợp, bù tập mờ là các phép toán logic mờ **và** (AND), **hoặc** (OR), **phủ định** (NOT). Tuy nhiên, như đã trình bày trong mục 2.2.3, mỗi phép toán trên tập mờ có thể tính bằng nhiều công thức khác nhau nên mỗi phép toán logic mờ cũng có thể tính bằng nhiều công thức khác nhau. Điều này khác với logic kinh điển, mỗi phép toán logic chỉ có một cách tính duy nhất.

2.5.1 Logic kinh điển

1- Mệnh đề kinh điển

Định nghĩa 2.7

Mệnh đề kinh điển, ký hiệu P , là phát biểu chỉ có một trong hai khả năng hoặc đúng hoặc sai. Nói cách khác, mệnh đề kinh điển là phát biểu chính xác không chứa thông tin mơ hồ, không rõ ràng.

Ví dụ, các phát biểu sau đây là mệnh đề kinh điển:

- P : “Điện áp điều khiển là 5,5V”

- Q : “Nhiệt độ lò là 75°C”.

Mệnh đề P đúng nếu điện áp điều khiển là 5,5V, ngược lại nếu điện áp điều khiển khác 5,5V thì mệnh đề P sai. Tương tự, mệnh đề Q đúng nếu nhiệt độ lò là 75°C, mệnh đề Q sai nếu nhiệt độ lò không phải là 75°C.

Gọi $T(P)$ là giá trị thật (*true value*) của mệnh đề P thì $T(P)$ chỉ có thể bằng 0 hay 1 tùy theo mệnh đề P sai hay đúng.

2- Các phép toán trên mệnh đề kinh điển

Các mệnh đề kinh điển có thể kết hợp với nhau bằng các phép toán logic để tạo nên các mệnh đề mới. Bảng sau đây tóm tắt các phép toán logic trên mệnh đề kinh điển.

Bảng 2.2 Tóm tắt các phép toán logic trên mệnh đề kinh điển

$T(P)$	$T(Q)$	Phép phủ định		Phép giao	Phép hợp	Phép kéo theo
		$T(\bar{P})$	$T(\bar{Q})$	$T(P \wedge Q)$	$T(P \vee Q)$	$T(P \rightarrow Q)$
0	0	1	1	0	0	1
0	1	1	0	0	1	1
1	0	0	1	0	1	0
1	1	0	0	1	1	1

Với P và Q là hai mệnh đề trong ví dụ ở phần trên thì:

- $\bar{P}$: “Điện áp điều khiển khác 5,5V”

- $\bar{Q}$: “Nhiệt độ lò khác 75°C”

- $P \wedge Q$: “Điện áp điều khiển là 5,5V và nhiệt độ lò là 75°C”

- $P \vee Q$: “Điện áp điều khiển là 5,5V hoặc nhiệt độ lò là 75°C”

- $P \rightarrow Q$: “Nếu điện áp điều khiển là 5,5V thì nhiệt độ lò là 75°C”.

2.5.2 Logic mờ

1- Mệnh đề mờ (Fuzzy Proposition)

Định nghĩa 2.8

Mệnh đề mờ, ký hiệu $\tilde{P}$, là phát biểu có chứa thông tin không rõ ràng.

Các phát biểu diễn tả ý tưởng chủ quan như mô tả chiều cao hoặc trọng lượng của một người thường là các mệnh đề mờ. Trong kỹ thuật, các phát biểu có dạng sau đây là các mệnh đề mờ:

- “Nhiệt độ” là “cao”
- “Mức chất lỏng” là “thấp”
- “Vận tốc” là “trung bình”...

Như vậy, mệnh đề mờ là phát biểu có dạng: “biến ngôn ngữ” là “giá trị ngôn ngữ”. Về mặt toán học, mệnh đề mờ là biểu thức:

$$\tilde{P} : x \in \tilde{A} \quad (2.51)$$

Tập mờ $\tilde{A}$ đặc trưng cho giá trị ngôn ngữ trong mệnh đề mờ. Khác với mệnh đề kinh điển chỉ có hai khả năng sai hoặc đúng (0 hoặc 1), giá trị thật của mệnh đề mờ là một giá trị bất kỳ nằm trong đoạn $[0,1]$. Gọi $T(\tilde{P})$ là giá trị thật của mệnh đề mờ $\tilde{P}$, $T(\tilde{P})$ chính là ánh xạ:

$$\begin{aligned} T(\tilde{P}) : X &\rightarrow [0,1] \\ x &\mapsto \mu_{\tilde{A}}(x) \end{aligned} \quad (2.52)$$

trong đó X là tập cơ sở của tập mờ $\tilde{A}$.

Nói cách khác:

$$T(\tilde{P}) = \mu_{\tilde{A}}(x) \quad \text{với} \quad 0 \leq \mu_{\tilde{A}}(x) \leq 1 \quad (2.53)$$

Biểu thức trên cho thấy “độ đúng” của mệnh đề $\tilde{P} : x \in \tilde{A}$ bằng độ phụ thuộc của x vào tập mờ $\tilde{A}$.

2- Các phép toán trên mệnh đề mờ

Các mệnh đề mờ có thể kết hợp với nhau qua các phép toán luận lý. Gọi $\tilde{P}$ là mệnh đề mờ tương ứng với tập mờ $\tilde{A}$, $\tilde{Q}$ là mệnh

đề mờ tương ứng với tập mờ $\tilde{B}$. Tương tự như với mệnh đề kinh điển, chúng ta có các phép toán sau đối với mệnh đề mờ.

Phép phủ định (Negation)

Mệnh đề phủ định:

$$\tilde{\tilde{P}} : x \notin \tilde{A} \quad (2.54)$$

Giá trị thật của mệnh đề phủ định:

$$T(\tilde{\tilde{P}}) = 1 - T(\tilde{P}) = 1 - \mu_{\tilde{A}}(x) \quad (2.55)$$

Phép hợp (Disjunction)

Mệnh đề hợp:

$$\tilde{P} \vee \tilde{Q} : x \in \tilde{A} \text{ hoặc } x \in \tilde{B} \quad (2.56)$$

$$\Rightarrow \tilde{P} \vee \tilde{Q} : x \in \tilde{A} \cup \tilde{B} \quad (2.57)$$

Từ (2.52) và (2.57) suy ra giá trị thật của mệnh đề hợp là:

$$T(\tilde{P} \vee \tilde{Q}) = \mu_{\tilde{A} \cup \tilde{B}}(x) \quad (2.58)$$

Phép giao (Conjunction)

Mệnh đề giao:

$$\tilde{P} \wedge \tilde{Q} : x \in \tilde{A} \text{ và } x \in \tilde{B} \quad (2.59)$$

$$\Rightarrow \tilde{P} \wedge \tilde{Q} : x \in \tilde{A} \cap \tilde{B} \quad (2.60)$$

Từ (2.52) và (2.60) suy ra giá trị thật của mệnh đề giao là:

$$T(\tilde{P} \wedge \tilde{Q}) = \mu_{\tilde{A} \cap \tilde{B}}(x) \quad (2.61)$$

Công thức tính hàm liên thuộc của hợp và giao hai tập mờ xem mục 2.2.3.

Phép kéo theo (Implication)

Mệnh đề kéo theo:

$$\tilde{P} \rightarrow \tilde{Q} : \text{nếu } x \in \tilde{A} \text{ thì } y \in \tilde{B} \quad (2.62)$$

trong đó mệnh đề $\tilde{P} : x \in \tilde{A}$ được gọi là *mệnh đề điều kiện* và mệnh đề $\tilde{Q} : y \in \tilde{B}$ được gọi là *mệnh đề kết luận*.

Giá trị thật của mệnh đề kéo theo được xác định bởi *toán tử I* (Implication).

$$T(\tilde{P} \rightarrow \tilde{Q}) = I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) \quad (2.63)$$

trong đó, I là ánh xạ:

$$I : [0, 1] \times [0, 1] \rightarrow [0, 1] \quad (2.64)$$

Các toán tử I thường dùng để xác định giá trị thật của mệnh đề kéo theo là:

Công thức Kleene (dựa trên logic kinh điển):

$$I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) = \max\{1 - \mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))\} \quad (2.65)$$

Công thức Zadeh

$$I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) = \max\{1 - \mu_{\tilde{A}}(x), \min[\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(y)]\} \quad (2.66)$$

Công thức Lukasiewics

$$I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) = \min\{1, 1 - \mu_{\tilde{A}}(x), \mu_{\tilde{B}}(y)\} \quad (2.67)$$

Toán tử I thực hiện phép kéo theo có thể là toán tử T , trong trường hợp này ta gọi phép kéo theo là *phép kéo theo T*. Có hai toán tử T thường dùng để thực hiện phép kéo theo là:

Công thức Mamdani

$$I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) = \min\{\mu_{\tilde{A}}(x), \mu_{\tilde{B}}(y)\} \quad (2.68)$$

Công thức Larsen

$$I(\mu_{\tilde{A}}(x), (\mu_{\tilde{B}}(y))) = \mu_{\tilde{A}}(x) \cdot \mu_{\tilde{B}}(y) \quad (2.69)$$

Trong điều khiển mờ, toán tử I thường được sử dụng là MIN (công thức Mamdani) hay PROD (công thức Larsen).

2.6 QUY TẮC MỜ

Quy tắc mờ là phát biểu **nếu-thì**, trong đó mệnh đề điều kiện và mệnh đề kết luận là các mệnh đề mờ. Trong mệnh đề điều kiện có thể có các phép giao, phép hợp hoặc phép phủ định. Để đơn giản xét quy tắc sau đây:

$$\text{nếu } x_1 \text{ là } \tilde{A}_1 \text{ và } x_2 \text{ là } \tilde{A}_2 \text{ thì } y \text{ là } \tilde{B} \quad (2.70)$$

trong đó các tập mờ $\tilde{A}_1$, $\tilde{A}_2$ và $\tilde{B}$ tương ứng được xác định bởi các hàm liên thuộc $\mu_{\tilde{A}_1}(x_1)$, $\mu_{\tilde{A}_2}(x_2)$, và $\mu_{\tilde{B}}(y)$.

$$\text{Đặt: } \tilde{P}_1 : x_1 \text{ là } \tilde{A}_1 \quad (2.71)$$

$$\tilde{P}_2 : x_2 \text{ là } \tilde{A}_2 \quad (2.72)$$

$$\tilde{Q} : y \text{ là } \tilde{B}_1 \quad (2.73)$$

Quy tắc (2.70) có thể viết lại như sau:

$$(\tilde{P}_1 \wedge \tilde{P}_2) \rightarrow \tilde{Q} \quad (2.74)$$

Áp dụng công thức (2.61) cho phép giao và (2.63) cho phép kéo theo, ta tính được giá trị thật của quy tắc (2.70) là:

$$T((\tilde{P}_1 \wedge \tilde{P}_2) \rightarrow \tilde{Q}) = I(T(\mu_{\tilde{A}_1}(x_1), \mu_{\tilde{A}_2}(x_2)), \mu_{\tilde{B}}(y)) \quad (2.75)$$

trong đó T là toán tử thực hiện phép giao và I toán tử thực hiện phép kéo theo. Phép giao dùng toán tử T biểu diễn liên từ **và** (AND), phép kéo theo dùng toán tử I biểu diễn sự suy luận, liên từ **nếu-thì**.

Ta thấy biểu thức (2.75) chính là hàm liên thuộc của một tập mờ định nghĩa trên tập cơ sở ba chiều. Mà chúng ta đã biết tập mờ trong cơ sở nhiều chiều là quan hệ mờ (xem mục 2.2.3). Vì vậy *quy tắc mờ có thể được biểu diễn bởi quan hệ mờ*. Hàm liên thuộc của quan hệ mờ $\tilde{R}$ biểu diễn quy tắc (2.70) được cho bởi:

$$\mu_{\tilde{R}}(x_1, x_2, y) = I(T(\mu_{\tilde{A}_1}(x_1), \mu_{\tilde{A}_2}(x_2)), \mu_{\tilde{B}}(y)) \quad (2.76)$$

Do đó, quan hệ mờ $\tilde{R}$ biểu diễn quy tắc trên có thể viết dưới dạng sau:

$$\tilde{R} = I(T(\tilde{A}_1, \tilde{A}_2), \tilde{B}) \quad (2.77)$$

2.6.1 Kết hợp các quy tắc mờ (Fuzzy Rules Aggregation)

Chúng ta đã biết cách biểu diễn mỗi quy tắc mờ bằng một quan hệ mờ. Mục này sẽ trình bày sự kết hợp các quy tắc mờ thành quan hệ mờ.

Xét k quy tắc mờ đối với n biến ngõ vào:

r_1 : nếu x_1 là $\tilde{A}_{11}$ và ... và x_n là $\tilde{A}_{n1}$ thì y là $\tilde{B}_1$

r_2 : nếu x_1 là $\tilde{A}_{12}$ và ... và x_n là $\tilde{A}_{n2}$ thì y là $\tilde{B}_2$

...

r_k : nếu x_1 là $\tilde{A}_{1k}$ và ... và x_n là $\tilde{A}_{nk}$ thì y là $\tilde{B}_k$

Việc chuyển hệ quy tắc mờ như trên thành một quan hệ mờ được thực hiện bằng cách xác định quan hệ mờ $\tilde{R}_i$ cho từng quy tắc mờ r_i (xem mục trên), sau đó kết hợp các quan hệ mờ $\tilde{R}_i$ này thành một quan hệ mờ $\tilde{R}$ duy nhất theo công thức sau:

$$\tilde{R} = \bigcup_{i=1}^k \tilde{R}_i \quad (2.78)$$

Công thức (2.78) cho thấy *quan hệ mờ biểu diễn hệ quy tắc bằng hợp của tất cả các quan hệ mờ biểu diễn từng quy tắc*. Vì quan hệ mờ là tập mờ định nghĩa trên tập cơ sở nhiều chiều nên phép hợp (2.78) chính là phép hợp các tập mờ và được thực hiện bởi chuẩn S (xem mục 2 phần 2.2.3). Trong điều khiển mờ, chuẩn S thường dùng để kết hợp hệ quy tắc mờ là toán tử MAX.

2.6.2 Tính chất của hệ quy tắc mờ

1- Tính liên tục (Continuity)

Định nghĩa 2.9

Hệ quy tắc mờ được gọi là liên tục nếu các quy tắc mờ có mệnh đề điều kiện kề nhau thì mệnh đề kết luận phải kề nhau.

Để làm rõ điều này trước hết chúng ta xét khái niệm tập mờ kề nhau đối với các tập mờ có thứ tự:

$$\tilde{A}_1 < \tilde{A}_2 < \dots < \tilde{A}_{i-1} < \tilde{A}_i < \tilde{A}_{i+1} < \dots$$

Trong đó $\tilde{A}_{i-1}$ và $\tilde{A}_i$ là hai tập mờ kề nhau, $\tilde{A}_i$ và $\tilde{A}_{i+1}$ cũng là hai tập mờ kề nhau. *Hai tập mờ kề nhau chồng lên nhau*. Ví dụ như sự phân hoạch mờ, chỉ có các tập mờ kề nhau là chồng nhau.

Chúng ta đã biết mệnh đề điều kiện của quy tắc mờ có thể là giao hay hợp của nhiều mệnh đề mờ. Do đó, *hai mệnh đề điều kiện được gọi là kề nhau nếu chúng chỉ khác nhau một mệnh đề thành*

phần, và mệnh đề thành phần khác nhau này phải tương ứng với hai tập mờ kề nhau.

Xét ví dụ sau, giả sử hệ quy tắc mờ gồm các quy tắc:

$$r_i: \text{nếu } x_1 \text{ là } \tilde{A}_{1i} \text{ và } x_2 \text{ là } \tilde{A}_{2i} \text{ thì } y \text{ là } \tilde{B}_i \quad (i = \overline{1..k})$$

Mệnh đề điều kiện của hai quy tắc mờ r_i và r_j ($i \neq j$) được gọi là kề nhau nếu:

- 1- $\tilde{A}_{1i} = \tilde{A}_{1j}$ và $\tilde{A}_{2i}, \tilde{A}_{2j}$ là hai tập mờ kề nhau
 hoặc 2- $\tilde{A}_{2i} = \tilde{A}_{2j}$ và $\tilde{A}_{1i}, \tilde{A}_{1j}$ là hai tập mờ kề nhau

Hệ quy tắc mờ nêu trên được gọi là liên tục khi $\tilde{B}_i$ và $\tilde{B}_j$ kề nhau trong trường hợp các mệnh đề điều kiện kề nhau (trường hợp 1 hoặc 2).

Chú ý rằng, khi $\tilde{A}_{1i}$ và $\tilde{A}_{1j}$ kề nhau, đồng thời $\tilde{A}_{2i}$ và $\tilde{A}_{2j}$ kề nhau thì hai mệnh đề điều kiện không gọi là kề nhau. Tính liên tục của hệ quy tắc mờ đóng vai trò quan trọng khi phép kéo theo trong các quy tắc mờ được biểu diễn bởi toán tử I dựa trên phép kéo theo kinh điển.

2- Tính nhất quán (Consistency)

Tính nhất quán của hệ quy tắc mờ thể hiện sự thống nhất của tri thức được biểu diễn bởi các quy tắc mờ. Ví dụ, hệ quy tắc mờ điều khiển lò nhiệt sau đây không nhất quán:

nếu nhiệt độ thấp thì công suất đốt nóng tăng

nếu nhiệt độ thấp thì công suất đốt nóng giảm

Chúng ta có thể cho rằng đây là hệ quy tắc được thiết kế không tốt. Điều này có thể đúng, tuy nhiên sự không nhất quán rất có thể xảy ra đối với hệ quy tắc phức tạp. Việc sử dụng liên từ **hoặc** (OR) có thể dẫn đến sự không nhất quán.

Ví dụ xét hệ quy tắc sau đây (xem bảng 2.3):

nếu x_1 là $\tilde{A}$ hoặc x_2 là $\tilde{E}$ thì y là $\tilde{H}$

nếu x_1 là $\tilde{C}$ hoặc x_2 là $\tilde{F}$ thì y là $\tilde{I}$

nếu x_1 là $\tilde{B}$ và x_2 là $\tilde{D}$ thì y là $\tilde{G}$

Hệ quy tắc trên dẫn đến kết luận y không nhất quán như trình bày ở bảng 2.3. Trong bảng này chúng ta thấy kết luận không thống nhất trong trường hợp x_1 là $\tilde{A}$ và x_2 là $\tilde{F}$ và trong trường hợp x_1 là $\tilde{C}$ và x_2 là $\tilde{E}$.

Bảng 2.3 Ví dụ về sự không nhất quán của hệ quy tắc mờ

$x_1 \backslash x_2$	$\tilde{A}$	$\tilde{B}$	$\tilde{C}$
$\tilde{D}$	$\tilde{H}$	$\tilde{G}$	$\tilde{I}$
$\tilde{E}$	$\tilde{H}$	$\tilde{H}$	$\tilde{H}, \tilde{I}$
$\tilde{F}$	$\tilde{H}, \tilde{I}$	$\tilde{I}$	$\tilde{I}$

Trong ví dụ đơn giản trên chúng ta dễ dàng thấy rằng trong một số trường hợp, hai quy tắc đầu tiên dẫn đến các kết luận không rõ ràng. Tuy nhiên, trong thực tế với các hệ quy tắc phức tạp thì hiện tượng này rất khó nhận biết. Sự không nhất quán cũng thường xảy ra khi toán tử phủ định (NOT) được sử dụng ở mệnh đề điều kiện của các quy tắc.

3- Tính hoàn chỉnh (Completeness)

Tính hoàn chỉnh của hệ quy tắc mờ thể hiện sự hoàn chỉnh của tri thức biểu diễn bởi các quy tắc mờ. Một hệ quy tắc mờ không hoàn chỉnh có các điểm trống (*blank spots*), nghĩa là có một số trường hợp tín hiệu ở ngõ vào sẽ dẫn đến không xác định được tác động ở ngõ ra. Điều này không có nghĩa là đối với một hệ quy tắc không hoàn chỉnh kết quả suy diễn không tồn tại. Tính hoàn chỉnh của hệ quy tắc mờ phụ thuộc rất nhiều vào hình dạng và vị trí của các tập mờ sử dụng ở mệnh đề điều kiện của các quy tắc. Độ hoàn chỉnh (*Completeness Measure*) của hệ quy tắc mờ gồm k quy tắc và n tín hiệu vào được xác định bởi:

$$CM(x) = \sum_{j=1}^k \left\{ \prod_{i=1}^n \mu_{\tilde{A}_{ij}}(x_i) \right\} \quad (2.79)$$

Trong đó $\mu_{\tilde{A}_{ij}}(x_i)$ là hàm liên thuộc của tập mờ tương ứng với giá trị ngôn ngữ của biến ngôn ngữ thứ i trong quy tắc thứ j ; x là vector tín hiệu vào.

Độ hoàn chỉnh $CM(x)$ của hệ quy tắc mờ có giá trị lớn hơn 0 khi vector tín hiệu vào x nằm trong miền xác định của mệnh đề điều kiện của ít nhất một quy tắc mờ trong hệ. Dùng hàm $CM(x)$, chúng ta có thể đánh giá độ hoàn chỉnh của hệ quy tắc mờ đối với vector tín hiệu vào bất kỳ như sau:

$CM(x) = 0$: hệ quy tắc mờ không hoàn chỉnh (*incompleteness*)

$0 < CM(x) < 1$: hệ quy tắc mờ thiếu hoàn chỉnh (*subcompleteness*)

$CM(x) = 1$: hệ quy tắc mờ hoàn chỉnh (*strict completeness*)

$CM(x) > 1$: hệ quy tắc mờ quá hoàn chỉnh (*overcompleteness*)

Sự thiếu hoàn chỉnh, hoàn chỉnh và quá hoàn chỉnh của hệ quy tắc mờ là các mức độ hoàn chỉnh trái ngược với sự không hoàn chỉnh. Dễ thấy rằng hệ quy tắc mờ có thể có tính chất khác nhau trong các miền khác nhau trong không gian tín hiệu vào. Ví dụ, trong một số miền hệ quy tắc có thể quá hoàn chỉnh, trong khi trong các miền khác hệ quy tắc có thể thiếu hoàn chỉnh.

Từ biểu thức (2.5) ta thấy hệ quy tắc mờ là hoàn chỉnh (*strict completeness*) khi các tập mờ trên mỗi tập cơ sở dùng trong các mệnh đề điều kiện được phân hoạch mờ và hệ quy tắc mờ phải gồm tất cả các quy tắc có thể xác định được. Sự không hoàn chỉnh của hệ quy tắc mờ có thể dẫn đến các tác động điều khiển không mong muốn.

2.7 SUY LUẬN MỜ

2.7.1 Sự suy diễn của một quy tắc mờ

1- Suy diễn bằng sự hợp thành

Giả sử chúng ta có quy tắc mờ:

nếu x là $\tilde{A}$ thì y là $\tilde{B}$

Nếu biết ngõ vào x là $\tilde{A}'$ thì có thể suy ra giá trị ngõ ra y là $\tilde{B}'$ được không? Nếu được thì $\tilde{B}'$ được tính bằng cách nào?

nếu x là $\tilde{A}$ thì y là $\tilde{B}$

x là $\tilde{A}'$

y là $\tilde{B}'$?

Câu trả lời là được. Quá trình suy ra giá trị $\tilde{B}'$ được gọi là sự suy diễn.

Phương pháp suy diễn bằng sự hợp thành được Zadeh đưa ra năm 1973 và phương pháp này giả sử rằng mỗi quy tắc mờ:

nếu x là $\tilde{A}$ thì y là $\tilde{B}$

được biểu diễn bởi một quan hệ mờ $\tilde{R}$ (xem mục 2.3.1). Sau đó, nếu biết $\tilde{A}'$ thì có thể suy ra $\tilde{B}'$ thông qua sự hợp thành của $\tilde{A}'$ và $\tilde{R}$ (xem mục 2.3.2):

$$\tilde{B}' = \tilde{A}' \circ \tilde{R} \quad (2.80)$$

Công thức suy diễn (2.80) trong trường hợp tổng quát tương đối phức tạp. Khi phép kéo theo thực hiện bởi toán tử MIN và sự hợp thành được tính bởi công thức MAX-MIN hoặc phép kéo theo được thực hiện bởi toán tử PROD và sự hợp thành được tính bởi công thức MAX-PROD thì công thức (2.80) có thể khai triển để dễ dàng áp dụng trong việc suy luận mờ như trình bày ở hai mục kế tiếp sau đây.

2- Phương pháp suy diễn MAX-MIN (MAX-MIN Inference Method)

Xét quy tắc thứ k của một hệ quy tắc mờ:

r_k : nếu $(x_1 \text{ là } \tilde{A}_{1k}) \text{ và } (x_2 \text{ là } \tilde{A}_{2k}) \text{ thì } (y \text{ là } \tilde{B}_k)$

Giả sử trong quy tắc trên toán tử T thực hiện phép giao là MIN, toán tử I thực hiện phép kéo theo cũng là MIN, và sự hợp thành áp dụng công thức MAX-MIN. Áp dụng công thức (2.77), ta được quan hệ mờ biểu diễn quy tắc trên là:

$$\tilde{R}_k = I(T(\tilde{A}_{1k}, \tilde{A}_{2k}), \tilde{B}_k)$$

$$\begin{aligned} \Rightarrow \mu_{\tilde{R}_k}(x_1, x_2, y) &= \min[\min(\mu_{\tilde{A}_{1k}}(x_1), \mu_{\tilde{A}_{2k}}(x_2)), \mu_{\tilde{B}_k}(y)] \\ &= \min[\mu_{\tilde{A}_{1k}}(x_1), \mu_{\tilde{A}_{2k}}(x_2), \mu_{\tilde{B}_k}(y)] \end{aligned} \quad (2.81)$$

Nếu biết tín hiệu vào là $\tilde{A}'_1$ và $\tilde{A}'_2$ thì áp dụng công thức hợp thành MAX-MIN (2.47) ta có thể tính được $\tilde{B}'_k$ bằng sự hợp thành của quan hệ mờ $\tilde{R}_k$ và giao hai tập mờ $\tilde{A}'_1 \cap \tilde{A}'_2$ như sau:

$$\begin{aligned}
\mu_{\tilde{B}_k} &= \max_{x_1, x_2} \left\{ \min[\mu_{\tilde{A}_1 \cap \tilde{A}_2}(x_1, x_2), \mu_{\tilde{R}_k}(x_1, x_2, y)] \right\} \\
&= \max_{x_1, x_2} \left\{ \min[\min(\mu_{\tilde{A}_1}(x_1), \mu_{\tilde{A}_2}(x_2)), \min(\mu_{\tilde{A}_{1k}}(x_1), \mu_{\tilde{A}_{2k}}(x_2), \mu_{\tilde{B}_k}(y))] \right\} \\
&= \min \left\{ \max_{x_1, x_2} \left\{ \min[\min(\mu_{\tilde{A}_1}(x_1), \mu_{\tilde{A}_2}(x_2)), \min(\mu_{\tilde{A}_{1k}}(x_1), \mu_{\tilde{A}_{2k}}(x_2))] \right\}, \mu_{\tilde{B}_k}(y) \right\} \\
&= \min \left\{ \min \left\{ \max_{x_1} [\min(\mu_{\tilde{A}_1}(x_1), \mu_{\tilde{A}_{1k}}(x_1))], \max_{x_2} [\min(\mu_{\tilde{A}_2}(x_2), \mu_{\tilde{A}_{2k}}(x_2))] \right\}, \mu_{\tilde{B}_k}(y) \right\} \\
&= \min \left\{ \text{hgt}(\tilde{A}'_1 \cap \tilde{A}_{1k}), \text{hgt}(\tilde{A}'_2 \cap \tilde{A}_{2k}), \mu_{\tilde{B}_k}(y) \right\} \quad (2.82)
\end{aligned}$$

Vì vậy, việc suy diễn mờ bằng sự hợp thành MAX-MIN được rút gọn lại là “cắt” tập mờ $\tilde{B}_k$ ở mệnh đề kết luận của quy tắc bởi hệ số β_k :

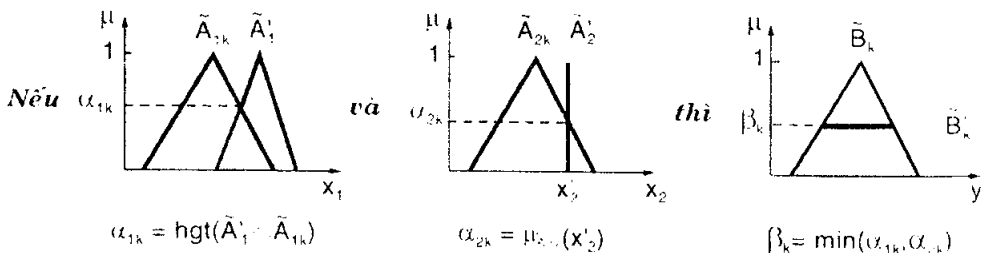
$$\mu_{\tilde{B}_k}(y) = \min \left\{ \beta_k, \mu_{\tilde{B}_k}(y) \right\} \quad (2.83)$$

$$\text{Với } \beta_k = \min_{i=1}^n \left\{ \text{hgt}(\tilde{A}'_i \cap \tilde{A}_{ik}) \right\} \quad (n: \text{số tín hiệu vào}) \quad (2.84)$$

$$\text{Hay } \beta_k = \min_{i=1}^n \{ \alpha_{ik} \} \quad (2.85)$$

$$\text{trong đó } \alpha_{ik} = \text{hgt}(\tilde{A}'_i \cap \tilde{A}_{ik}) \quad (2.86)$$

Giá trị β_k gọi là “độ phù hợp” (*degree of matching*) giữa dữ liệu vào và mệnh đề điều kiện của quy tắc. Hình 2.16 minh họa sự suy diễn của một quy tắc mờ theo phương pháp MAX-MIN.



Hình 2.16 Sự suy diễn của một quy tắc mờ theo phương pháp MAX-MIN

Phương pháp suy diễn MAX-MIN bằng các công thức (2.83)-(2.86) đã loại trừ được việc tính toán trên không gian tích, vì sự hợp thành của quan hệ mờ đã bị khử đi. Trong các công thức trên

rõ ràng không có sự phân biệt giữa dữ liệu vào là rõ hay mờ. Phương pháp suy diễn MAX-MIN thường được dùng trong điều khiển mờ.

3- Phương pháp suy diễn MAX-PROD (MAX-PROD Inference Method)

Sự suy diễn của một quy tắc mờ cũng có thể đơn giản khi toán tử thực hiện phép giao và phép kéo theo là toán tử PROD, và sử dụng công thức hợp thành MAX-PROD (2.48). Tương tự như mục trên, ta dễ dàng chứng minh được kết quả suy diễn (tập mờ $\tilde{B}_k$) bằng tập mờ ở mệnh đề kết luận của quy tắc (tập mờ $\tilde{B}_k$) nhân với “độ phù hợp” β_k :

$$\mu_{\tilde{B}_k}(y) = \beta_k \cdot \mu_{\tilde{B}_k}(y) \quad (2.87)$$

trong đó:
$$\beta_k = \prod_{i=1}^n \left\{ \max_{x_i} (\mu_{\tilde{A}_i}(x_i) \cdot \mu_{\tilde{A}_{ik}}(x_i)) \right\} \quad (2.88)$$

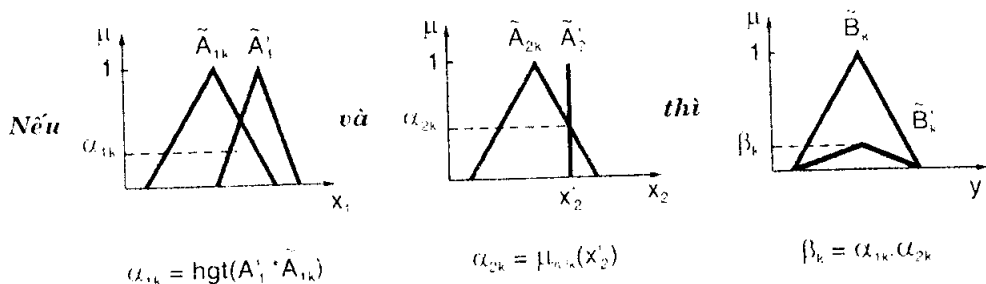
$$\Rightarrow \beta_k = \prod_{i=1}^n \left\{ \text{hgt}(\tilde{A}_i * \tilde{A}_{ik}) \right\} \quad (2.89)$$

Trong công thức (2.89) ký hiệu $*$ biểu diễn phép giao của hai tập mờ được thực hiện bởi toán tử PROD. Biểu thức (2.89) có thể viết lại như sau:

$$\beta_k = \prod_{i=1}^n \alpha_{ik} \quad (2.90)$$

trong đó:
$$\alpha_{ik} = \left\{ \text{hgt}(\tilde{A}_i * \tilde{A}_{ik}) \right\} \quad (2.91)$$

Phương pháp suy diễn MAX-PROD được minh họa ở hình 2.17. Phương pháp này cũng được sử dụng phổ biến trong điều khiển mờ. Ngoài hai phương pháp suy diễn vừa trình bày ở trên, còn một số phương pháp khác như SUM-PROD... Các phương pháp suy diễn khác có thể tham khảo thêm trong các tài liệu.



Hình 2.17 Sự suy diễn của một quy tắc mờ theo phương pháp MAX-PROD

2.7.2 Sự suy diễn của hệ quy tắc mờ

Xét một hệ gồm r quy tắc mờ, trong đó mỗi quy tắc có dạng:

$$r_k: \text{nếu } (x_1 \text{ là } \tilde{A}_{1k}) \text{ và } \dots \text{ và } (x_n \text{ là } \tilde{A}_{nk}) \text{ thì } (y \text{ là } \tilde{B}_k)$$

(k : quy tắc thứ k , $k = \overline{1..r}$; n là số ngõ vào)

Nếu biết các ngõ vào là $\tilde{A}'_1, \dots, \tilde{A}'_n$ thì từ hệ quy tắc trên chúng ta có thể suy ra giá trị ngõ ra $\tilde{B}'$ bằng một trong hai cách: suy diễn cục bộ và suy diễn toàn cục.

1- Suy diễn cục bộ

Phương pháp suy diễn cục bộ là thực hiện sự suy diễn với mỗi quy tắc riêng lẻ, sau đó kết hợp các kết quả suy diễn riêng lẻ lại để được kết quả suy diễn của hệ quy tắc.

Theo mục 2.7.1, kết quả suy diễn của quy tắc thứ k là:

$$\tilde{B}'_k = \left\{ \bigcap_{i=1}^n \tilde{A}'_i \right\} \circ \tilde{R}_k \quad (2.92)$$

trong đó $\tilde{R}_k$ là quan hệ mờ biểu diễn quy tắc thứ k .

Kết hợp các kết quả suy diễn của từng quy tắc, ta được kết quả suy diễn của hệ quy tắc như sau:

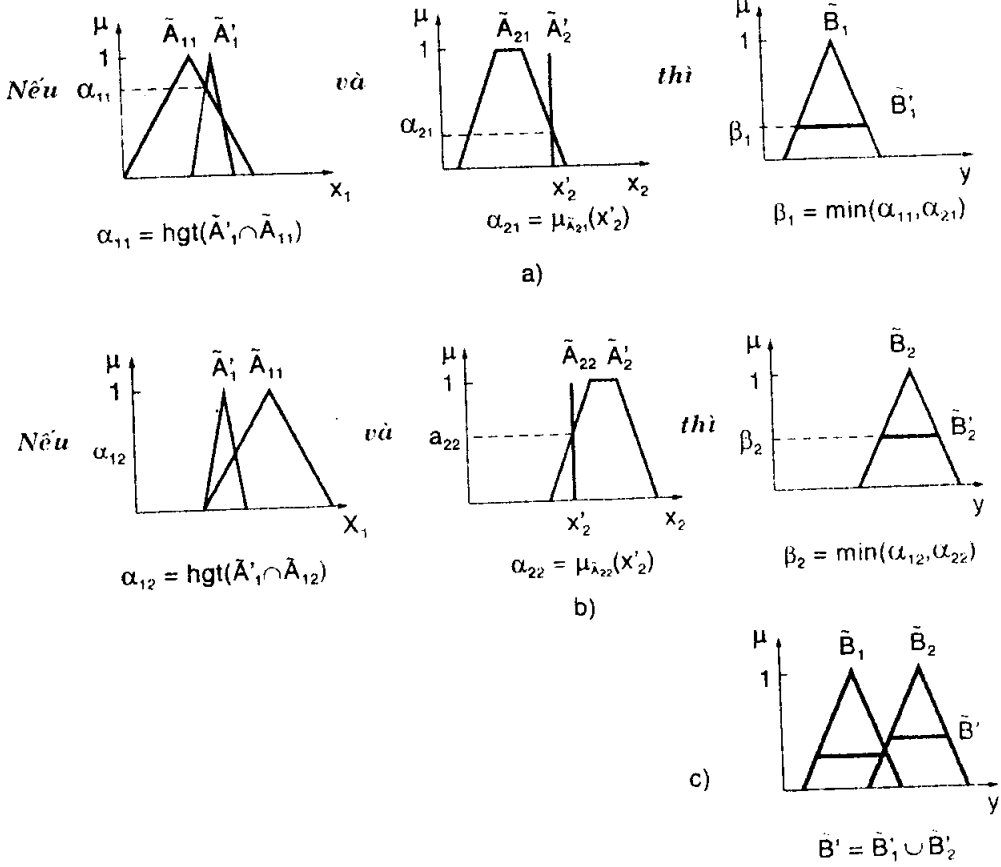
$$\tilde{B}' = \bigcup_{k=1}^r \tilde{B}'_k \quad (2.93)$$

Để minh họa, chúng ta hãy xét hệ gồm hai quy tắc sau:

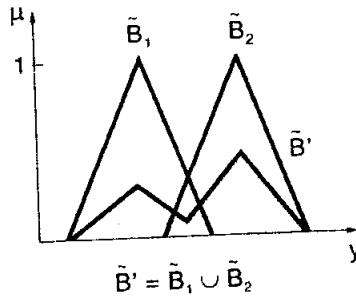
$$r_1: \text{nếu } (x_1 \text{ là } \tilde{A}_{11}) \text{ và } (x_2 \text{ là } \tilde{A}_{21}) \text{ thì } (y \text{ là } \tilde{B}_1)$$

$$r_2: \text{nếu } (x_1 \text{ là } \tilde{A}_{12}) \text{ và } (x_2 \text{ là } \tilde{A}_{22}) \text{ thì } (y \text{ là } \tilde{B}_2)$$

Áp dụng phương pháp suy diễn cục bộ, chúng ta tính được kết quả suy diễn của mỗi quy tắc như ở hình 2.18a và hình 2.18b. Sau đó kết hợp các kết quả trên ta được kết quả suy diễn của hệ quy tắc như hình 2.18c. Hình 2.18 minh họa kết quả khi mỗi quy tắc áp dụng phương pháp suy diễn MAX-MIN. Nếu mỗi quy tắc áp dụng phương pháp suy diễn MAX-PROD, ta có kết quả như hình 2.19.



Hình 2.18 Suy diễn cục bộ của hệ quy tắc mờ theo phương pháp MAX-MIN



Hình 2.19 Suy diễn cục bộ của hệ quy tắc mờ theo phương pháp MAX-PROD

2- Suy diễn toàn cục

Phương pháp suy diễn toàn cục thực hiện sự kết hợp các quan hệ mờ biểu diễn các quy tắc lại với nhau để được quan hệ mờ biểu diễn hệ quy tắc, sau đó dùng quan hệ mờ này và tín hiệu vào để suy diễn và được kết quả.

Khi phép kéo theo được thực hiện bởi toán tử T , quan hệ mờ $\tilde{R}$ biểu diễn hệ quy tắc mờ là:

$$\tilde{R} = \bigcup_{k=1}^r \tilde{R}_k \quad (2.94)$$

trong đó $\tilde{R}_k$ là quan hệ mờ biểu diễn quy tắc r_k . Kết quả suy diễn toàn cục được cho bởi:

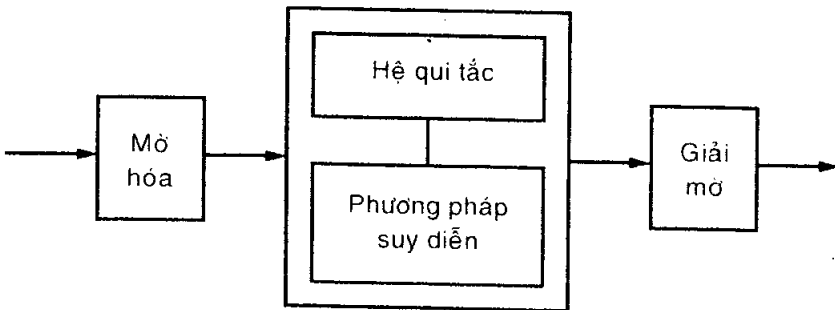
$$\begin{aligned} \tilde{B}' &= \left\{ \bigcap_{i=1}^n \tilde{A}'_i \right\} \circ \tilde{R} = \left\{ \bigcap_{i=1}^n \tilde{A}'_i \right\} \circ \left\{ \bigcup_{k=1}^r \tilde{R}_k \right\} = \bigcup_{k=1}^r \left\{ \left\{ \bigcap_{i=1}^n \tilde{A}'_i \right\} \circ \tilde{R}_k \right\} \\ &\Rightarrow \tilde{B}' = \bigcup_{k=1}^r \tilde{B}'_k \end{aligned} \quad (2.95)$$

Biểu thức (2.93) và (2.95) trên cho thấy kết quả suy diễn cục bộ và kết quả suy diễn toàn cục như nhau. Tuy nhiên, thực tế các bộ điều khiển mờ đều chọn phương pháp suy diễn cục bộ vì phương pháp này việc tính toán đơn giản hơn, ít tốn bộ nhớ và thực thi nhanh hơn.

2.8 HỆ MỜ

2.8.1 Sơ đồ khối

Hệ mờ cơ bản gồm ba thành phần: *khối mờ hóa, hệ quy tắc và khối giải mờ* (H.2.20). Chú ý rằng hệ mờ cơ bản là hệ tĩnh, điều này có nghĩa là giá trị ngõ ra của hệ mờ ở một thời điểm chỉ phụ thuộc vào giá trị ngõ vào ở thời điểm đó.

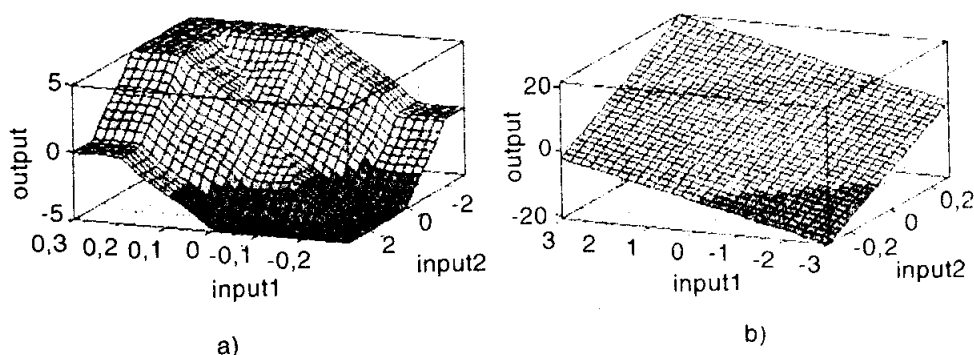


Hình 2.20 Sơ đồ khối hệ mờ cơ bản

Mặt đặc tính (Surface)

Hệ mờ có thể có nhiều ngõ vào và nhiều ngõ ra. Đồ thị biểu diễn mối quan hệ giữa một ngõ ra theo hai ngõ vào bất kỳ gọi là mặt đặc tính. Trường hợp đặc biệt khi hệ mờ chỉ có một ngõ vào thì mặt đặc tính trở thành đường đặc tính. Tổng quát mặt đặc tính của hệ mờ là mặt phi tuyến, tuy nhiên trong một số trường hợp mặt đặc tính có thể là mặt tuyến tính (mặt phẳng). Hình 2.21 minh họa mặt đặc tính phi tuyến và tuyến tính.

Mặt đặc tính phụ thuộc chủ yếu vào hệ quy tắc mờ vì hệ quy tắc mờ quyết định quan hệ giữa ngõ vào và ngõ ra dưới dạng giá trị ngôn ngữ. Tuy nhiên, hình dạng, vị trí các tập mờ, phương pháp suy diễn, phương pháp giải mờ cũng ảnh hưởng đến mặt đặc tính. Các ảnh hưởng này sẽ được phân tích trong mục 2.8.5.



Hình 2.21 Mặt đặc tính biểu diễn mối quan hệ vào ra của hệ mờ

a) Mặt đặc tính phi tuyến; b) Mặt đặc tính tuyến tính

2.8.2 Mờ hóa

Khối đầu tiên bên trong hệ mờ cơ bản là khối mờ hóa, khối này có chức năng biến đổi giá trị rõ sang giá trị ngôn ngữ, hay nói cách khác là sang tập mờ, vì hệ quy tắc mờ chỉ có thể suy diễn trên các tập mờ.

Giả sử tín hiệu vào hệ mờ là giá trị rõ x' , khối mờ hóa sẽ biến đổi giá trị rõ x' thành tập mờ $\tilde{A}'$ như sau:

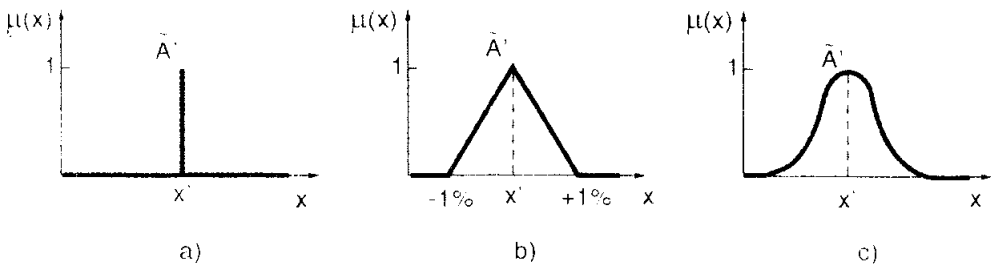
$$\tilde{A}' = \text{fuzz}(x')$$

trong đó fuzz là hàm mờ hóa.

Thường các tín hiệu vào bộ điều khiển chính là tín hiệu phản hồi từ ngõ ra của đối tượng hoặc sai lệch giữa tín hiệu đặt và tín hiệu phản hồi từ ngõ ra của đối tượng. Nếu *mạch đo chính xác, không có nhiễu thì giá trị rõ x' đơn giản được biến đổi thành tập mờ $\tilde{A}'$ có dạng singleton (H.2.22a).*

$$\mu_{\tilde{A}'}(x) = \begin{cases} 1, & \text{nếu } x = x' \\ 0, & \text{nếu } x \neq x' \end{cases}$$

Nếu mạch đo không chính xác hoặc có nhiễu thì giá trị phản hồi x' là giá trị không chắc chắn, do đó khối mờ hóa sẽ biến đổi giá trị không chắc chắn x' thành tập mờ có dạng như hình 2.22b hay hình 2.22c.



Hình 2.22 Tập mờ ở ngõ ra của khâu mờ hóa

- a) Tập mờ $\tilde{A}'$ khi tín hiệu vào x' không có sai số, không có nhiễu
- b) Tập mờ $\tilde{A}'$ khi tín hiệu vào x' có sai số $\pm 1\%$
- c) Tập mờ $\tilde{A}'$ khi tín hiệu vào x' có nhiễu phân bố Gauss

Trong điều khiển mờ các tín hiệu vào bộ điều khiển thường được mờ hóa thành các tập mờ có dạng singleton. Các tín hiệu vào sau khi đã mờ hóa sẽ được xử lý bởi hệ quy tắc mờ.

2.8.3 Giải mờ

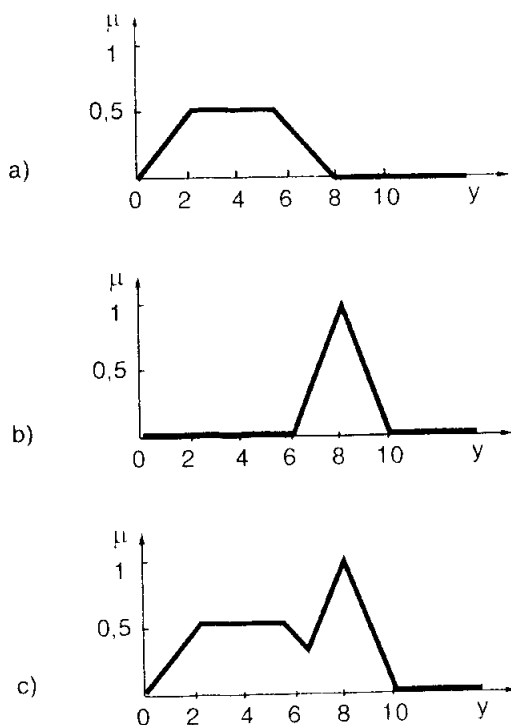
1- Các phương pháp giải mờ

Định nghĩa 2.10

Giải mờ là biến đổi tập mờ (giá trị ngôn ngữ) sang giá trị rõ (giá trị vật lý).

Ngõ ra của một hệ mờ có thể là hợp của hai hay nhiều tập mờ xác định trên tập cơ sở của biến ở ngõ ra. Ví dụ, giả sử ngõ ra của một hệ mờ gồm hai phần: phần thứ nhất là tập mờ $\tilde{C}_1$ có hàm liên thuộc

dạng hình thang (H.2.23a), và phần thứ hai tập mờ $\tilde{C}_2$ có là hàm liên thuộc dạng tam giác (H.2.23a). Hợp của tập mờ này, $\tilde{C} = \tilde{C}_1 \cup \tilde{C}_2$, có hàm liên thuộc trình bày ở hình 2.23c.



Hình 2.23 Ngõ ra của một bộ xử lý mờ a) Phần thứ nhất của ngõ ra b) Phần thứ hai của ngõ ra; c) Hợp của hai phần

Tất nhiên, ngõ ra tổng quát có thể gồm nhiều phần (nhiều hơn hai) và hàm liên thuộc của mỗi phần có thể có dạng khác tam giác hoặc hình thang. Hơn nữa, các hàm liên thuộc này không phải lúc nào cũng là hàm liên thuộc chính tắc. Tổng quát ta có:

$$\tilde{C} = \bigcup_{i=1}^k \tilde{C}_i \quad (2.96)$$

Gọi Y là tập cơ sở của biến ở ngõ ra, $\tilde{C}$ là tập mờ kết quả của hệ mờ, y^* là giá trị rõ sau khi giải mờ. Có ít nhất bảy phương pháp giải mờ đã được đưa ra, tuy nhiên các phương pháp này có thể phân làm hai nhóm dựa vào hai nguyên tắc giải mờ khác nhau, nhóm thứ nhất xác định giá trị rõ dựa vào độ cao, nhóm thứ hai dựa vào trọng tâm.

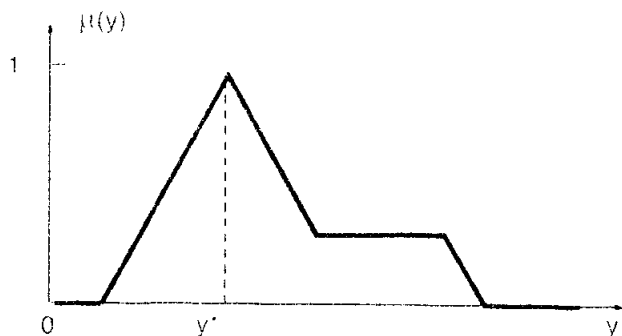
a- Nguyên tắc giải mờ dựa vào độ cao

Phương pháp độ cao (Height Method)

Phương pháp này chỉ áp dụng cho các hàm liên thuộc ở ngò ra có *đỉnh cực đại*, giá trị rõ y^* được cho bởi biểu thức đại số:

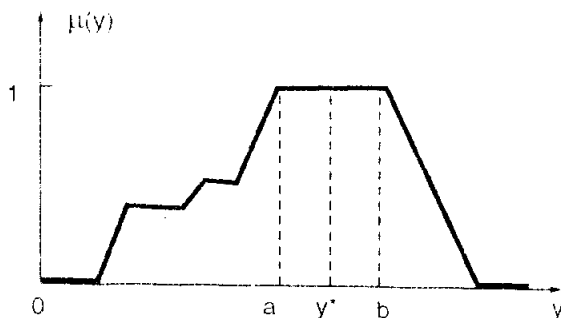
$$y^* \in Y : \mu_{\tilde{C}}(y^*) > \mu_{\tilde{C}}(y), \forall y \in Y \quad (2.97)$$

Ý nghĩa của biểu thức trên là y^* là giá trị mà tại đó hàm liên thuộc đạt cực đại. Phương pháp này trình bày dưới dạng đồ thị (H.2.24).



Hình 2.24 Phương pháp độ phụ thuộc cực đại

Phương pháp trung bình của độ phụ thuộc cực đại (Mean-Of-Maximum)



Hình 2.25 Phương pháp trung bình của độ phụ thuộc cực đại

Phương pháp này là dạng mở rộng của phương pháp độ phụ thuộc cực đại để có thể áp dụng cho các hàm liên thuộc có độ phụ thuộc cực đại là một vùng chứ không chỉ là một điểm. Phương pháp này được cho bởi biểu thức:

$$y^* = \frac{a + b}{2} \quad (2.98)$$

trong đó a và b được xác định ở hình 2.25.

Phương pháp cận trái cực đại (hay cận phải cực đại)

Phương pháp này xác định giá trị rõ như sau:

Đầu tiên xác định độ cao của tập mờ $\tilde{C}$ ở ngõ ra theo biểu thức:

$$\text{hgt}(\tilde{C}) = \sup_{y \in Y} \mu_{\tilde{C}}(y) \quad (2.99)$$

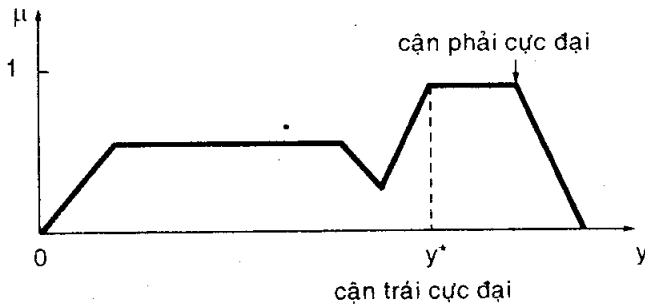
sau đó gán giá trị rõ y^* bằng cận trái cực đại (Left Of Maximum-LOM):

$$y^* = \inf_{y \in Y} \{y \in Y \mid \mu_{\tilde{C}}(y) = \text{hgt}(\tilde{C})\} \quad (2.100)$$

Một dạng khác của phương pháp này gọi là *phương pháp cận phải cực đại (Right Of Maximum-ROM)* và y^* được cho bởi biểu thức:

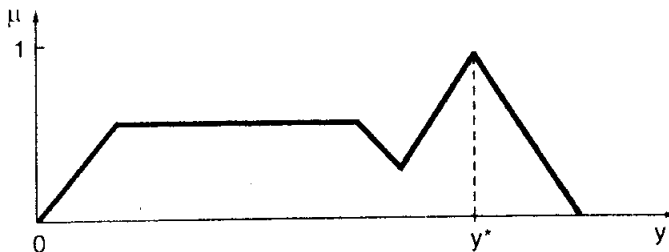
$$y^* = \sup_{y \in Y} \{y \in Y \mid \mu_{\tilde{C}}(y) = \text{hgt}(\tilde{C})\} \quad (2.101)$$

Trong các biểu thức trên, $\sup$ (*supremum*) là cận trên nhỏ nhất và $\inf$ (*infimum*) là cận dưới lớn nhất. Phương pháp này được trình bày dưới dạng đồ thị ở hình 2.26.



Hình 2.26 Phương pháp cận trái cực đại (hay cận phải cực đại)

Chú ý: Nếu tập mờ ở ngõ ra có dạng như hình 2.27 thì các phương pháp giải mờ độ cao, trung bình của độ thuộc cực đại, cận trái cực đại, cận phải cực đại đều cho giá trị rõ y^* như nhau.



Hình 2.27 Các phương pháp giải mờ dựa vào độ cao cho kết quả y^* như nhau

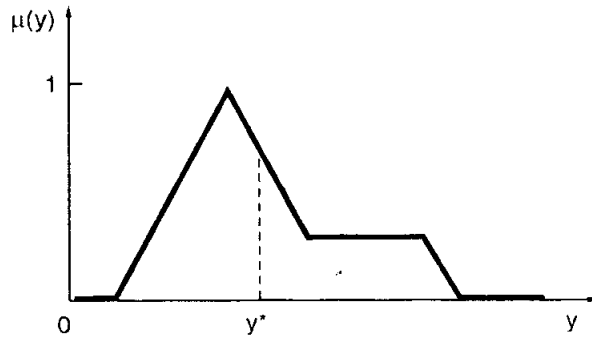
b- Nguyên tắc giải mờ dựa vào trọng tâm

Phương pháp trọng tâm (Centroid Method)

Phương pháp này còn được gọi là *Center Of Area–COA* hay *Center Of Gravity–COG*. Đây là phương pháp giải mờ thường dùng nhất trong điều khiển. Phương pháp này được cho bởi biểu thức đại số:

$$y^* = \frac{\int \mu_{\tilde{C}}(y) \cdot y dy}{\int \mu_{\tilde{C}}(y) dy} \quad (2.102)$$

Về mặt hình học, phương pháp trọng tâm gán giá trị rõ cho biến ở ngõ ra là giá trị mà tại đó hàm liên thuộc được chia làm hai phần có diện tích bằng nhau. Phương pháp này được trình bày ở hình 2.28.



Hình 2.28 Phương pháp trọng tâm

Phương pháp trung bình có trọng số (Weighted Average Method)

Phương pháp này chỉ sử dụng khi ngõ ra là hợp của các hàm liên thuộc đối xứng và được cho bởi biểu thức đại số:

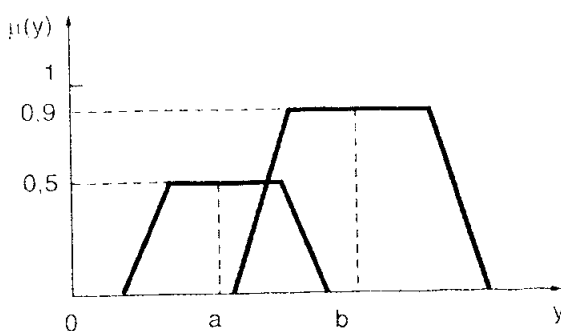
$$y^* = \frac{\sum_i \mu_{\tilde{C}}(\bar{y}_i) \cdot \bar{y}_i}{\sum_i \mu_{\tilde{C}}(\bar{y}_i)} \quad (2.103)$$

trong đó dấu Σ là ký hiệu phép tính tổng đại số, $\bar{y}_i$ là giá trị trung bình của các hàm liên thuộc thành phần. Phương pháp này gán y^* bằng giá trị trung bình có trọng số của các giá trị $\bar{y}_i$. Phương pháp này được trình bày ở hình 2.29, giá trị giải mờ là:

$$y^* = \frac{a(0.5) + b(0.9)}{0.5 + 0.9} \quad (2.104)$$

Vì phương pháp này chỉ áp dụng cho các hàm liên thuộc đối xứng nên giá trị a và b tương ứng là giá trị trung bình trên hàm liên thuộc của nó. Phương pháp trung bình có trọng số thực thi rất nhanh và còn có tên gọi khác là *phương pháp trung bình mờ* (fuzzy-mean defuzzification).

So sánh hai biểu thức (2.102) và (2.103), chúng ta thấy ngay phương pháp trung bình có trọng số chính là dạng rời rạc của phương pháp trọng tâm. Trong khi phương pháp trọng tâm chỉ áp dụng được cho hàm liên thuộc liên tục, phương pháp trung bình có trọng số có thể áp dụng cho hàm liên thuộc liên tục và rời rạc.



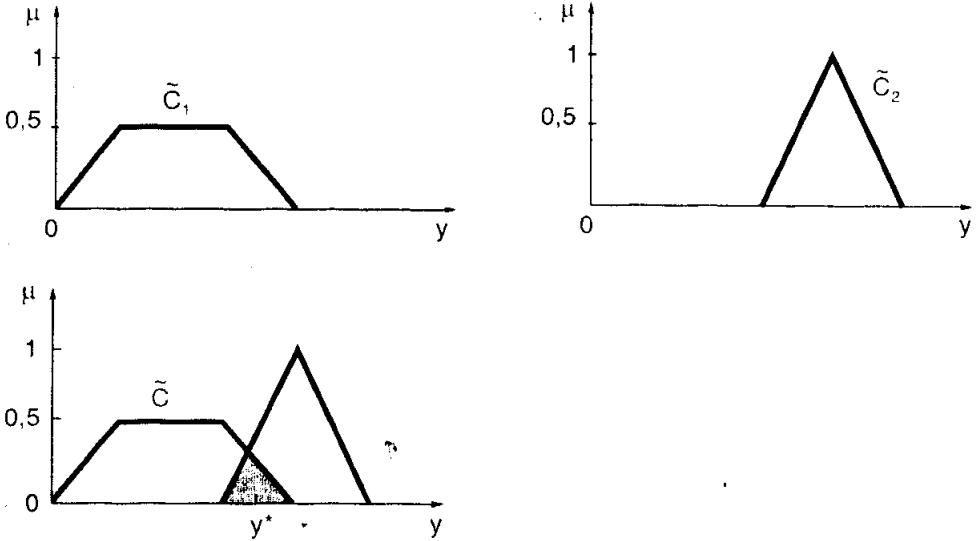
Hình 2.29 Phương pháp trung bình có trọng số

Phương pháp trọng tâm của tổng (Center of Sum)

Phương pháp trọng tâm của tổng thực thi nhanh hơn phương pháp trọng tâm, phương pháp này lấy tổng đại số của các ngõ ra riêng lẻ của các tập mờ thay vì lấy hợp của chúng. Khuyết điểm của phương pháp này là vùng giao tăng lên hai lần. Giá trị sau khi giải mờ được cho bởi phương trình:

$$y^* = \frac{\int_Y \sum_{k=1}^n \mu_{\tilde{C}_k}(y).dy}{\int_Y \sum_{k=1}^n \mu_{\tilde{C}_k}(y).dy} \quad (2.105)$$

Phương pháp trọng tâm của tổng tương tự như phương pháp trung bình có trọng số, ngoại trừ trong phương pháp trọng tâm của tổng thì trọng số là diện tích của các hàm liên thuộc tương ứng, trong khi phương pháp trung bình có trọng số thì trọng số là các giá trị độ phụ thuộc riêng lẻ. Hình 2.30 minh họa phương pháp trọng tâm của tổng.



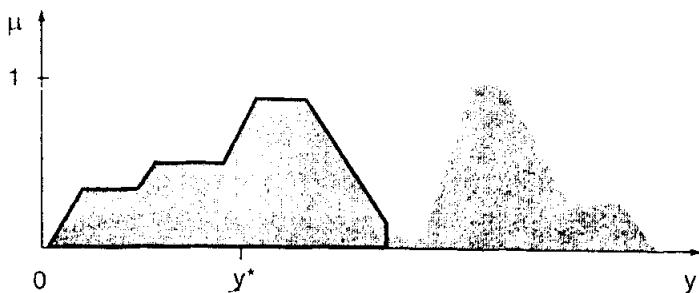
Hình 2.30 Phương pháp giải mờ trọng tâm của tổng

Phương pháp trọng tâm vùng lớn nhất (Center Of Largest Area)

Phương pháp này là dạng mở rộng của phương pháp trọng tâm. Nếu tập mờ ở ngõ ra có ít nhất hai vùng con lồi, thì trọng tâm của vùng mờ lồi có diện tích lớn nhất được dùng để suy ra giá trị rõ ở ngõ ra. Phương pháp này được trình bày bằng đồ thị ở hình 2.31 và được cho bởi biểu thức đại số:

$$y^* = \frac{\int \mu_{\tilde{C}_m}(y) \cdot y dy}{\int \mu_{\tilde{C}_m}(y) dy} \quad (2.106)$$

trong đó $\tilde{C}_m$ là vùng lồi con có diện tích lớn nhất. Trong trường hợp tập mờ ở ngõ ra chỉ có một vùng lồi thì y^* được xác định hoàn toàn giống như phương pháp trọng tâm.



Hình 2.31 Phương pháp trọng tâm vùng có diện tích lớn nhất

2- Ý nghĩa và ứng dụng của các phương pháp giải mờ

Mục đích của một phương pháp giải mờ là rút ra giá trị rõ thể hiện tốt nhất giá trị mờ của biến ngôn ngữ ở ngõ ra. Như đã trình bày ở trên có nhiều phương pháp giải mờ khác nhau. Để chọn phương pháp giải mờ thích hợp, chúng ta cần hiểu ý nghĩa ngôn ngữ ẩn chứa dưới quá trình giải mờ.

a- Phương pháp giải mờ thỏa hiệp và phương pháp giải mờ hợp lý

Về mặt ý nghĩa, các phương pháp giải mờ có thể phân thành hai loại: phương pháp giải mờ “*thỏa hiệp*” (*compromise*) và phương pháp giải mờ “*hợp lý*” (*plausible*).

Một phương pháp giải mờ có tính chất “*thỏa hiệp*” là phương pháp trọng tâm vùng có diện tích lớn nhất COM (*Center Of Maximum*). Phương pháp COM đầu tiên tìm giá trị tiêu biểu nhất cho mỗi giá trị ngôn ngữ và sau đó tính giá trị thỏa hiệp nhất của kết quả suy luận mờ. Để được giá trị thỏa hiệp nhất đối với kết quả suy luận mờ là một số thực, kết quả suy diễn phải được xem như là trọng số tại các vị trí của giá trị tiêu biểu nhất cho mỗi giá trị ngôn ngữ. Giá trị rõ thỏa hiệp nhất là vị trí mà giá trị rõ cân bằng các trọng lượng. Trong một số trường hợp, phương pháp giải mờ này cho kết quả không tốt. Nếu chúng ta dùng phương pháp COM để giải mờ, sự thỏa hiệp của hai lời giải tốt có thể dẫn đến kết quả xấu. Các phương pháp giải mờ khác dựa vào trọng tâm như COA/COG, đều là các phương pháp giải mờ “*thỏa hiệp*”.

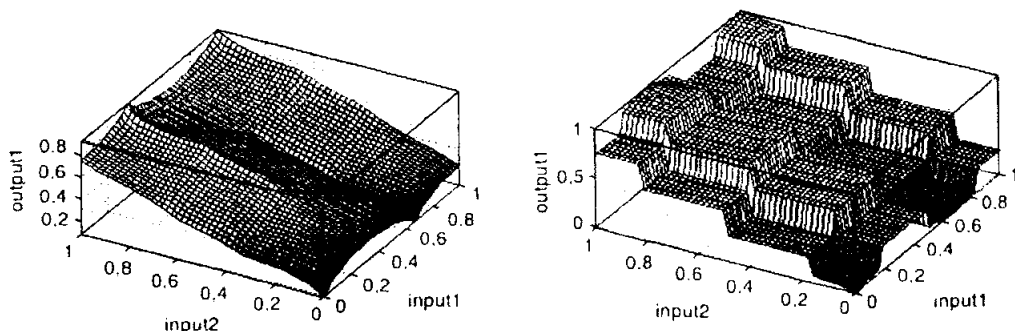
Một phương pháp giải mờ cho kết quả “*hợp lý*” là phương pháp giải mờ trung bình của độ phụ thuộc cực đại MOM (*Mean Of Maximum*). Khác với việc cân bằng các kết quả suy diễn khác nhau, MOM chọn giá trị tiêu biểu của giá trị ngôn ngữ hợp lý nhất. Ngoài MOM, các phương pháp giải mờ dựa trên độ cao như LOM, ROM đều là các phương pháp giải mờ “*hợp lý*”.

b- Sự liên tục của phương pháp giải mờ

Một tính chất quan trọng của phương pháp giải mờ là tính liên tục. Định nghĩa tính liên tục như sau:

Định nghĩa 2.11

Xét một hệ mờ với hệ quy tắc hoàn chỉnh (nghĩa là với mọi trạng thái có thể có của ngõ vào phải có ít nhất một quy tắc tác động – xem mục 2.3.3). Một phương pháp giải mờ được gọi là liên tục nếu sự thay đổi nhỏ của các biến ở ngõ vào không gây ra sự thay đổi gián đoạn ở các biến ngõ ra.



Hình 2.32 Sự liên tục của phương pháp giải mờ

- a) Giải mờ bằng phương pháp COA mặt đặc tính liên tục
- b) Giải mờ bằng phương pháp MOM mặt đặc tính gián đoạn

Các phương pháp giải mờ dựa vào trọng tâm COM, COA/COG là các phương pháp giải mờ liên tục trong khi các phương pháp giải mờ dựa vào độ cao MOM, LOM, ROM là các phương pháp giải mờ không liên tục. Điều này là do phương pháp giải mờ “thỏa hiệp” không bao giờ nhảy đến một giá trị khác đối với sự thay đổi của các biến ngõ vào. Trái lại, có những điểm mà phương pháp giải mờ “hợp lý” nhảy sang một giá trị khác, nói cách khác ngõ vào thay đổi liên tục nhưng ngõ ra gián đoạn.

c- Phạm vi ứng dụng của các phương pháp giải mờ

Tính liên tục là tính chất quan trọng trong hầu hết các ứng dụng điều khiển vòng kín. Nếu ngõ ra của bộ điều khiển mờ điều khiển trực tiếp đối tượng, thì sự nhảy gián đoạn của biến ở ngõ ra bộ điều khiển mờ có thể làm cho hệ thống mất ổn định hoặc dao động. Vì vậy trong hầu hết các ứng dụng điều khiển vòng kín, phương pháp giải mờ được dùng là phương pháp COM hoặc COA. Chỉ khi nào ngõ ra của bộ điều khiển mờ được nối với một khâu tích phân thì các phương pháp giải mờ không liên tục như MOM mới có thể sử dụng được.

Trong các ứng dụng nhận dạng (*recognition*), phương pháp giải mờ được sử dụng nhiều nhất là MOM. Nếu chúng ta muốn nhận dạng đối tượng bằng sự phân loại các tín hiệu đo, chúng ta mong muốn kết quả hợp lý nhất.

Trong các ứng dụng thực tế, sự khác nhau giữa các phương pháp giải mờ chỉ là phương pháp đó cho kết quả “*thỏa hiệp*” (COM, COA, COG) hay “*hợp lý*” (MOM, LOM, ROM), khi thay đổi các phương pháp giải mờ trong một nhóm quan hệ vào ra của hệ mờ thay đổi không đáng kể.

2.8.4 Hệ quy tắc mờ

Hệ quy tắc: Hệ quy tắc mờ có thể xem là mô hình toán học biểu diễn tri thức, kinh nghiệm của con người trong việc giải quyết bài toán dưới dạng các phát biểu ngôn ngữ. Hệ quy tắc mờ gồm các quy tắc có dạng **nếu-thì**, trong đó mệnh đề điều kiện và mệnh đề kết luận của mỗi quy tắc là các mệnh đề mờ liên quan đến một hay nhiều biến ngôn ngữ. Điều này có nghĩa là hoàn toàn có thể sử dụng hệ mờ để giải các bài toán điều khiển một ngõ vào một ngõ ra (SISO) hay nhiều ngõ vào nhiều ngõ ra (MIMO).

Phương pháp suy diễn: Suy diễn là sự kết hợp các giá trị ngôn ngữ của ngõ vào sau khi mờ hóa với hệ quy tắc để rút ra kết luận giá trị mờ của ngõ ra. Hai phương pháp suy diễn thường dùng trong điều khiển là MAX-MIN và MAX-PROD (xem mục 2.7).

Có hai loại quy tắc mờ sử dụng phổ biến trong các hệ mờ hiện nay là quy tắc mờ Mamdani và quy tắc mờ Takagi-Sugeno (hay còn gọi tắt là quy tắc Sugeno), hai loại quy tắc này sẽ được mô tả chi tiết trong các mục sau đây.

1- Quy tắc mờ Mamdani

Quy tắc mờ Mamdani là loại quy tắc mờ đã xét ở các mục trước, **kết luận của quy tắc mờ Mamdani là mệnh đề mờ**. Đây là loại quy tắc được dùng trong các ứng dụng đầu tiên của điều khiển mờ và có dạng tổng quát sau đây:

$$\begin{aligned} r_i: \quad & \text{nếu } (x_1 \text{ là } \tilde{A}_{1i}) \text{ và } \dots \text{ và } (x_n \text{ là } \tilde{A}_{ni}) \\ & \text{thì } (y_1 \text{ là } \tilde{B}_{1i}), \dots, (y_m \text{ là } \tilde{B}_{mi}) \end{aligned} \quad (2.107)$$

trong đó n là số tín hiệu vào, m là số tín hiệu ra, $i = \overline{1..k}$, với k là số quy tắc.

Ví dụ, một quy tắc mờ Mamdani như sau:

**nếu sai số “lớn” và tốc độ thay đổi sai số “nhỏ”
thì tín hiệu điều khiển “lớn”**

Quy tắc trên có thể so sánh với thuật toán điều khiển PD rời rạc: tín hiệu ra của bộ điều khiển phụ thuộc vào giá trị sai số và đạo hàm bậc nhất của sai số (sai số là sai lệch giữa tín hiệu đặt và tín hiệu ra của quá trình điều khiển). Trong mệnh đề điều kiện của quy tắc mờ Mamdani, chúng ta có thể sử dụng các biến ngôn ngữ khác, ví dụ như dùng sự biến thiên của tín hiệu ra thay vì sự biến thiên của sai số; hoặc dùng sự phân loại tín hiệu đặt để khắc phục sự phi tuyến của quá trình điều khiển.

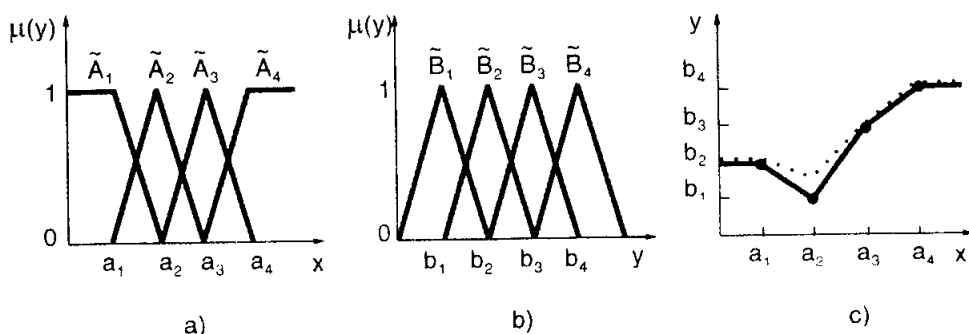
Thông thường, toán tử MIN được dùng cho phép giao và phép kéo theo, toán tử MAX được dùng để kết hợp các quy tắc và sự hợp thành áp dụng công thức MAX-MIN. Phương pháp này gọi là phương pháp suy diễn MAX-MIN. Mamdani sử dụng loại quy tắc mờ này và là người đầu tiên trình bày loại quy tắc này trong các bài báo về ứng dụng của logic mờ trong điều khiển nên loại quy tắc điều khiển này được gọi là quy tắc Mamdani.

Có thể nhận thấy rằng với các quy tắc Mamdani, việc dùng một số hữu hạn các hàm liên thuộc cho ngõ ra sẽ hạn chế mặt đặc tính nếu các tập mờ ở mệnh đề điều kiện là chính tắc và được phân hoạch mờ, và các tín hiệu vào là giá trị rõ. Để thấy rõ hơn sự hạn chế này, chúng ta hãy xét một hệ mờ có một ngõ vào và một ngõ ra gồm bốn quy tắc sau:

1. Nếu x là $\tilde{A}_1$ thì y là $\tilde{B}_2$
2. Nếu x là $\tilde{A}_2$ thì y là $\tilde{B}_1$
3. Nếu x là $\tilde{A}_3$ thì y là $\tilde{B}_3$
4. Nếu x là $\tilde{A}_4$ thì y là $\tilde{B}_4$

Các tập mờ ở ngõ vào và ngõ ra như hình 2.33a và hình 2.33b. Ta thấy rằng khi ngõ vào x' nằm ở điểm giữa của tập mờ $\tilde{A}_i$ thì giá trị hàm liên thuộc $\mu_{\tilde{A}_i}(x') = 1$ và chỉ có quy tắc thứ i tác động đến ngõ ra. Ví dụ, nếu $x' = a_3$ (điểm giữa của tập mờ $\tilde{A}_3$) thì chỉ có quy tắc 3 tác động đến ngõ ra, kết quả suy diễn của các quy tắc khác đều bằng 0. Do đó kết quả suy diễn của hệ quy tắc chính là kết quả suy

diễn của quy tắc 3 và là tập mờ $\tilde{B}_3$. Giải mờ bằng phương pháp trọng tâm, ta được $y' = b_3$. Điểm (a_3, b_3) gọi là điểm đặc tính (*characteristic point*). Dễ thấy rằng, ngoài điểm (a_3, b_3) , bộ điều khiển trên còn có các điểm đặc tính khác là (a_1, b_2) , (a_2, b_1) , và (a_4, b_4) . Đường đặc tính bị giới hạn bởi sự nội suy giữa các điểm đặc tính này. Tùy theo phương pháp suy diễn và phương pháp giải mờ mà ta sẽ có các đường nội suy khác nhau. Trong trường hợp sử dụng phương pháp suy diễn SUM-PROD và phương pháp giải mờ trung bình mờ thì đường nội suy là đường thẳng nối các điểm đặc tính (hình 2.33c, đường liền nét).



Hình 2.33 Sự hạn chế của đường đặc tính

- a) Các tập mờ ở ngõ vào; b) Các tập mờ ở ngõ ra
c) Đường đặc tính bị hạn chế bởi sự nội suy giữa các điểm đặc tính (đường liền nét) và có thể chọn như mong muốn bằng cách dùng nhiều kết luận với các trọng số (đường đứt nét)

Sự giới hạn của đường đặc tính (tổng quát là mặt đặc tính) có thể khắc phục bằng cách định nghĩa nhiều tập mờ hơn cho các ngõ ra. Điều này dẫn đến làm tăng số quy tắc có thể có, tăng độ phức tạp của hệ thống, và làm cho hệ quy tắc trở nên khó hiểu.

Một cách khác, để khắc phục sự hạn chế mặt đặc tính là trong mỗi quy tắc mờ dùng nhiều mệnh đề kết luận, mỗi mệnh đề kết luận được nhân với một trọng số. Dùng cách này ta có thể thiết kế được mặt điều khiển mong muốn bất kỳ. Ví dụ, thay quy tắc 2 của hệ mờ trong ví dụ vừa xét ở trên bằng quy tắc sau:

Nếu x là $\tilde{A}_2$ thì y là $\tilde{B}_1$ (50%), y là $\tilde{B}_2$ (50%)

ta được đường đặc tính mới (hình 2.33c, đường đứt nét). Bằng cách thay đổi các trọng số ta sẽ được đường đặc tính mong muốn.

2- Quy tắc mờ Sugeno

Một loại quy tắc mờ khác được cũng được sử dụng rất phổ biến là quy tắc Sugeno. Quy tắc mờ Sugeno được đưa ra vào năm 1983 và khác biệt so với quy tắc Mamdani ở mệnh đề kết luận: *kết luận của quy tắc mờ Sugeno là hàm của các tín hiệu vào của hệ mờ*. Tổng quát quy tắc Sugeno cho hệ mờ nhiều ngõ vào nhiều ngõ ra có dạng như sau:

$$\begin{aligned} r_i: & \text{ nếu } (x_1 \text{ là } \tilde{A}_{1i}) \text{ và } \dots \text{ và } (x_n \text{ là } \tilde{A}_{ni}) \\ & \text{thì } y_1 = f_{1i}(x_1, \dots, x_n), \dots, y_m = f_{mi}(x_1, \dots, x_n) \end{aligned} \quad (2.108)$$

Không mất tính tổng quát, xét hệ mờ có một ngõ ra, Sugeno dùng hàm tuyến tính ở kết luận của các quy tắc mờ như sau:

$$\begin{aligned} r_i: & \text{ nếu } (x_1 \text{ là } \tilde{A}_{1i}) \text{ và } \dots \text{ và } (x_n \text{ là } \tilde{A}_{ni}) \\ & \text{thì } y = b_{0i} + \sum_{j=1}^n b_{ji}x_j \end{aligned} \quad (2.109)$$

trong đó b_{0i} và b_{ji} là các hằng số.

Ví dụ, một quy tắc điều khiển mờ Sugeno có dạng:

$$\text{nếu } e \text{ "lớn" và } \Delta e \text{ "nhỏ" thì } u = 4e + 2\Delta e$$

trong đó u là tín hiệu điều khiển, e là sai số và Δe là biến thiên của sai số.

Sử dụng toán tử PROD để thực hiện phép giao, phép kéo theo, và sử dụng phương pháp giải mờ trung bình có trọng số (*weighted average*), chúng ta có thể tính được ngõ ra của hệ mờ Sugeno bằng công thức sau:

$$y' = \frac{\sum_{i=1}^k \beta_i \left\{ b_{0i} + \sum_{j=1}^n b_{ji}x_j \right\}}{\sum_{i=1}^k \beta_i} = \frac{\sum_{i=1}^k \beta_i b_{0i} + \sum_{j=1}^n \left\{ \sum_{i=1}^k \beta_i b_{ji}x_j \right\}}{\sum_{i=1}^k \beta_i} \quad (2.110)$$

trong đó β_i là độ phù hợp của mệnh đề điều kiện của quy tắc thứ i .

$$\text{Đặt: } b'_0 = \frac{\sum_{i=1}^k \beta_i b_{0i}}{\sum_{i=1}^k \beta_i}; \quad b'_j = \frac{\sum_{i=1}^k \beta_i b_{ji}}{\sum_{i=1}^k \beta_i}, \quad (j = 1, 2, \dots, n) \quad (2.111)$$

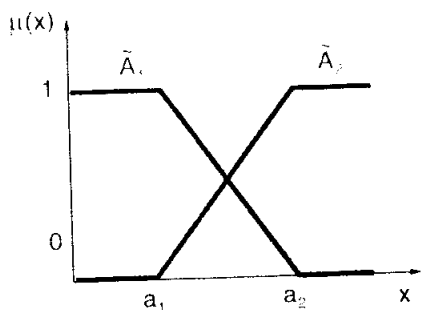
$$\text{Suy ra: } y' = b'_0 + \sum_{j=1}^n b'_j x_j \quad (2.112)$$

Công thức (2.110) cho thấy ngõ ra của hệ mờ dùng các quy tắc Sugeno là trung bình có trọng số của các hàm ở mệnh đề kết luận của mỗi quy tắc, nói cách khác ngõ ra của hệ quy tắc Sugeno là kết quả nội suy ngõ ra của các quy tắc. Tuy nhiên, nội suy giữa các hàm tuyến tính có khuyết điểm là kết quả không thể thấy bằng trực giác được, đặc biệt khi các quy tắc Sugeno được thiết kế bởi phương pháp đồ thị. Một ví dụ điển hình là các quy tắc sau:

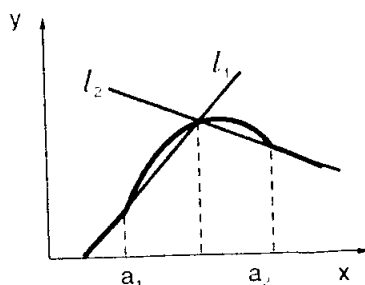
$$r_1: \text{nếu } (x \text{ là } \tilde{A}_1) \text{ thì } l_1: y = c_1x + d_1$$

$$r_2: \text{nếu } (x \text{ là } \tilde{A}_2) \text{ thì } l_2: y = c_2x + d_2$$

trong đó các thông số c_1, c_2, d_1 và d_2 được chọn sau cho hai đường thẳng l_1 và l_2 cắt nhau tại điểm giữa của hàm liên thuộc tương ứng với hai biến ngôn ngữ $\tilde{A}_1$ và $\tilde{A}_2$ của ngõ vào x . Giả sử các tập mờ $\tilde{A}_1, \tilde{A}_2$ tạo nên phân hoạch mờ, thì ngõ ra bộ điều khiển sẽ có giá trị nằm giữa hai hàm l_1 và l_2 (H.2.34).



a)



b)

Hình 2.34 Ngõ ra của bộ điều khiển dùng quy tắc Sugeno là sự nội suy giữa các hàm ở kết luận của từng quy tắc

a) Các tập mờ ở ngõ vào; b) Quan hệ ngõ vào và ngõ ra (đường đậm nét)

3- So sánh quy tắc mờ Mamdani và quy tắc mờ Sugeno

Như trình bày ở trên, quy tắc mờ Mamdani khác quy tắc mờ Sugeno. Trong các ứng dụng điều khiển, quy tắc mờ Mamdani thích hợp để điều khiển các đối tượng không xác định được mô hình toán, trong khi quy tắc mờ Sugeno thích hợp để điều khiển các đối tượng có mô hình không chính xác. Đặc biệt, quy tắc mờ Sugeno rất thích

hợp để giải các bài toán phi tuyến bằng phương pháp tuyến tính hóa từng đoạn.

Mặc dù quy tắc Mamdani và quy tắc Sugeno hoàn toàn khác nhau, tuy nhiên trong trường hợp đặc biệt, quy tắc mờ Mamdani với các tập mờ ở mệnh đề kết luận là các tập mờ dạng singleton hoàn toàn tương đương quy tắc mờ Sugeno có kết luận là hằng số (các quy tắc mờ trong trường hợp này còn được gọi là quy tắc Wang). Để rõ hơn sự tương đương này chúng ta hãy xét ví dụ sau:

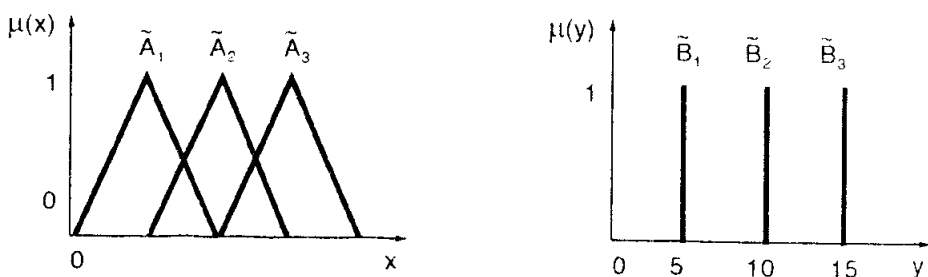
Xét hệ mờ một ngõ vào một ngõ ra, các tập mờ ở ngõ vào và ngõ ra định nghĩa như sau:

Giả sử các quy tắc mờ Mamdani điều khiển hệ thống là:

r_1 : nếu (x là $\tilde{A}_1$) thì (y là $\tilde{B}_1$)

r_2 : nếu (x là $\tilde{A}_2$) thì (y là $\tilde{B}_2$)

r_3 : nếu (x là $\tilde{A}_3$) thì (y là $\tilde{B}_3$)



Hình 2.35 Khi các tập mờ ở ngõ ra dạng singleton, quy tắc mờ Mamdani đồng nhất với quy tắc mờ Sugeno có kết luận là hằng số

Ba quy tắc trên tương đương với ba quy tắc Sugeno sau:

r_1 : nếu (x là $\tilde{A}_1$) thì $y = 5$

r_2 : nếu (x là $\tilde{A}_2$) thì $y = 10$

r_3 : nếu (x là $\tilde{A}_3$) thì $y = 15$

2.8.5 Phân tích quan hệ vào ra của hệ mờ

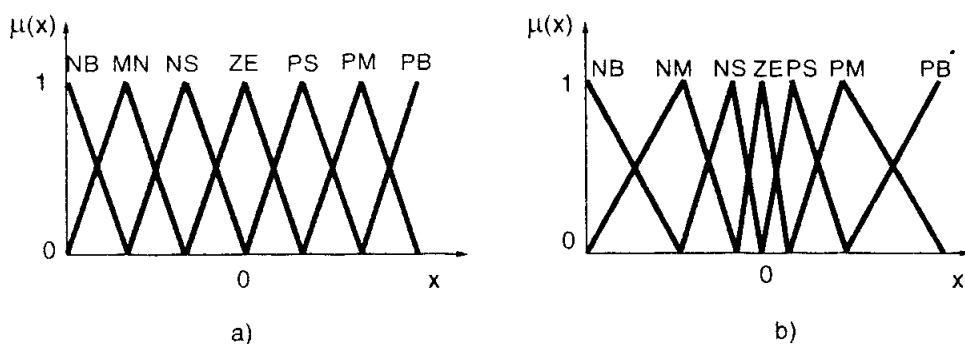
Mục này phân tích sự ảnh hưởng của các thành phần trong hệ mờ. Việc phân tích được thực hiện dựa trên phương pháp thực nghiệm đối với các hệ mờ cơ bản đã trình bày trong các mục trước.

Trong mục trước chúng ta đã biết hệ mờ có thể xem như là một ánh xạ phi tuyến từ ngõ vào đến ngõ ra, chúng ta sử dụng kết luận này để phân tích sự ảnh hưởng của các thành phần của hệ mờ.

1- Vai trò của các tập mờ

a- Số lượng các tập mờ

Khi thiết kế một hệ mờ, một trong những câu hỏi phát sinh đầu tiên sau khi chọn các tín hiệu vào là cần bao nhiêu tập mờ và các tập mờ được phân chia như thế nào trên tập cơ sở của các biến ngõ vào. Hình 2.36 trình bày các dạng tập mờ chuẩn trên tập cơ sở của các tín hiệu vào của hệ mờ.



Hình 2.36 Các kiểu phân chia tập mờ thường dùng

a) Phân chia tuyến tính; b) Phân chia phi tuyến

Việc dùng các tập mờ được phân chia tuyến tính, phi tuyến hay các kiểu tập mờ khác tùy thuộc vào từng bài toán cần giải quyết. Mật độ các tập mờ trên một miền nào đó của tập cơ sở càng cao thì trên miền đó có thể định nghĩa quan hệ giữa tín hiệu ra và các tín hiệu vào càng phức tạp. Thực ra, để chọn số lượng các tập mờ và cách phân chia các tập mờ trên tập cơ sở cần phải biết ngõ ra của hệ mờ quan hệ với ngõ vào của hệ mờ như thế nào. Ví dụ, khi thiết kế bộ điều khiển mờ để điều khiển một đối tượng phi tuyến cần phải biết tính phi tuyến của đối tượng này. Hiện nay không có phương pháp thiết kế chuẩn nào có thể sử dụng để chọn số lượng và vị trí các tập mờ, và rất ít người nhận ra rằng đây là một bài toán cần giải quyết.

Số lượng và vị trí của các tập mờ xác định mật đặc tính. Trong điều khiển để thiết kế mật đặc tính đúng cần biết đặc tính của đối tượng điều khiển. Nếu đặc tính của đối tượng tồn tại dưới dạng kinh

nghiệm của người vận hành thì người thiết kế phải thu thập kinh nghiệm này. Khi đã biết đặc tính phi tuyến của đối tượng điều khiển thì có thể sử dụng các đặc tính này để chọn số lượng và vị trí của các tập mờ. Ví dụ, nếu quá trình điều khiển có một vùng chết (*dead zone*) thì có thể định nghĩa một tập mờ chồng lên vùng chết này, sau đó dùng tập mờ này trong một quy tắc để bù vùng chết. Điều này tương tự việc bù đặc tính phi tuyến như trường hợp các bộ điều khiển kinh điển trong thực tế.

b- Sự chồng nhau của các tập mờ

Sự chồng nhau của các tập mờ cùng với phép suy diễn mờ và giải mờ dẫn đến ngõ ra của hệ mờ là sự nội suy giữa các ngõ ra của các quy tắc mờ. Nếu các hàm liên thuộc là chính tắc lỗi và các tập mờ được phân hoạch mờ trên tập cơ sở thì sự nội suy chỉ phụ thuộc vào các điểm đặc tính xung quanh gần nhất và mỗi điểm đặc tính được xác định duy nhất bởi một quy tắc mờ. Điều này bởi vì không có nhiều hơn hai hàm liên thuộc chồng nhau trên tập cơ sở đối với sự phân hoạch mờ.

Nếu có nhiều hơn hai tập mờ chồng nhau thì sự nội suy không chỉ phụ thuộc vào các điểm đặc tính xung quanh gần nhất. Trong trường hợp hàm liên thuộc có dạng phân bố Gauss thì tất cả các điểm đặc tính đều ảnh hưởng đến ngõ ra, điều này có nghĩa là tất cả các quy tắc mờ đều tác động. Có thể thấy rằng, khi có nhiều hơn hai tập mờ chồng nhau thì mặt điều khiển “trơn” hơn trường hợp chỉ có hai tập mờ chồng nhau. Để thấy điều này chúng ta xét ba quy tắc sau:

$$r_1: \text{nếu } x \text{ là } \tilde{A}_1 \text{ thì } y \text{ là } \tilde{B}_1$$

$$r_2: \text{nếu } x \text{ là } \tilde{A}_2 \text{ thì } y \text{ là } \tilde{B}_2$$

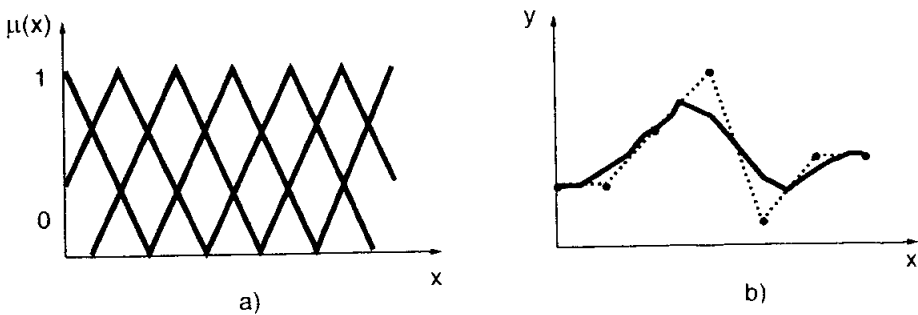
$$r_3: \text{nếu } x \text{ là } \tilde{A}_3 \text{ thì } y \text{ là } \tilde{B}_3$$

trong đó cả ba tập mờ $\tilde{A}_1$, $\tilde{A}_2$ và $\tilde{A}_3$ đều chồng nhau. Để đơn giản, chúng ta sử dụng phương pháp giải mờ trung bình mờ, chỉ dùng biểu diễn số b_1 , b_2 , và b_3 tương ứng của $\tilde{B}_1$, $\tilde{B}_2$ và $\tilde{B}_3$. Kết quả ngõ ra y^* phụ thuộc vào ngõ vào x' như sau:

$$y^* = \frac{\mu_{\tilde{A}_1}(x')b_1 + \mu_{\tilde{A}_2}(x')b_2 + \mu_{\tilde{A}_3}(x')b_3}{\mu_{\tilde{A}_1}(x') + \mu_{\tilde{A}_2}(x') + \mu_{\tilde{A}_3}(x')}$$

Ví dụ, khi $b_1, b_3 < b_2$ thì ngõ ra $y^* < b_2, \forall x'$ vì $\tilde{A}_1, \tilde{A}_2$ và $\tilde{A}_3$ chồng nhau. Điều này có nghĩa là các điểm đặc tính quan trọng của hàm điều khiển trở nên kém quan trọng hơn do việc chồng nhau của các tập mờ và một quy tắc bất kỳ không bao giờ là quy tắc tác động duy nhất.

Hình 2.37 trình bày ảnh hưởng làm “trơn” ngõ ra y khi có sự chồng nhau của nhiều hơn hai tập mờ trên tập cơ sở của ngõ vào x . Ảnh hưởng của việc chồng nhau của nhiều hơn hai tập mờ đến mặt đặc tính khó biết trước được. Khi có nhiều tập mờ chồng nhau thì sự thay đổi mệnh đề kết luận của một quy tắc mờ nào đó sẽ không dẫn đến sự thay đổi đáng kể hàm điều khiển (*controller function*) vì sự thay đổi này bị lọc bởi các quy tắc khác cũng tác động cùng lúc.



Hình 2.37 Ảnh hưởng làm trơn ngõ ra khi nhiều hơn hai tập mờ chồng nhau

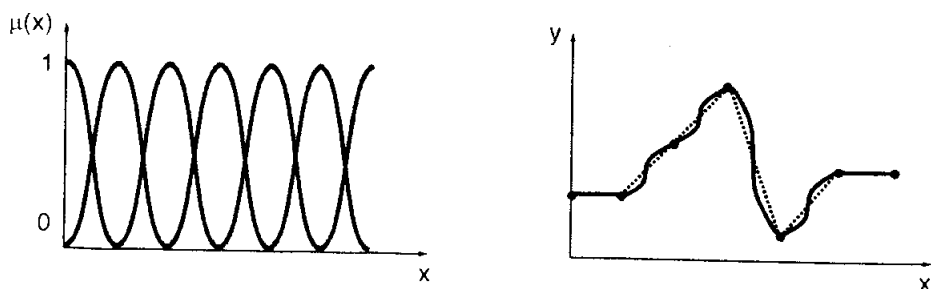
a) Ba tập mờ ở ngõ vào chồng nhau; b) Ngõ ra khi có ba tập mờ chồng nhau (đường liền nét) “trơn” hơn khi có hai tập mờ chồng nhau (đường đứt nét)

c- Dạng của các tập mờ

Mục trước đã trình bày ảnh hưởng của các hàm liên thuộc chồng nhau, khi có nhiều hơn hai tập mờ chồng nhau thì mặt đặc tính sẽ “trơn” hơn. Vì vậy trong phần này chúng ta chỉ tập trung nghiên cứu các tập mờ được phân hoạch mờ trên tập cơ sở. Như đã biết, hàm liên thuộc của các tập mờ có thể chia làm hai nhóm: hàm liên thuộc tuyến tính từng đoạn (tam giác, hình thang...) và hàm liên thuộc trơn (dạng sigmoid, dạng gauss, dạng chuông...).

Ta đã biết ngõ ra của hệ mờ là hàm của các ngõ vào và được xác định bởi sự nội suy giữa các kết luận của từng quy tắc riêng lẻ. Sự nội suy này là phi tuyến trong trường hợp hàm liên thuộc của các tập mờ

ở ngõ vào có dạng trơn. Như vậy, bằng việc sử dụng hàm liên thuộc dạng trơn, quan hệ vào ra của hệ mờ có đặc điểm phi tuyến. Hình 2.38a trình bày các hàm liên thuộc dạng hình chuông với lõi chỉ có duy nhất một điểm tạo thành một sự phân hoạch mờ. Hình 2.38b là hàm điều khiển tương ứng với hàm liên thuộc hình chuông (đường liền nét) và hàm liên thuộc tam giác (đường đứt nét). Mặc dù sự nội suy phi tuyến có thể dẫn đến mặt đặc tính mong muốn, tuy nhiên điều này theo R. Jager là không phù hợp với ý tưởng chủ đạo của hệ mờ: tính phi tuyến của hệ mờ nên được xác định bởi các quy tắc mờ, bao gồm số lượng và vị trí của các tập mờ, bởi vì những quy tắc này tiêu biểu cho tri thức về quá trình cần điều khiển.



Hình 2.38 a) Các hàm liên thuộc trơn; b) hàm điều khiển (đường liền nét). Đường đứt nét là hàm điều khiển khi tập cơ sở của ngõ vào được phân hoạch mờ với các tập mờ có dạng tam giác. Các điểm trên hình (b) là các điểm đặc tính của hàm điều khiển, mỗi điểm biểu diễn một quy tắc, và hàm điều khiển là sự nội suy giữa các điểm đặc tính này

d- Các tập mờ ở mệnh đề kết luận

Trong mục 2.8.4, chúng ta đã biết mệnh đề kết luận của các quy tắc mờ có thể là mệnh đề mờ (quy tắc Mamdani) hoặc là hàm của các tín hiệu vào (quy tắc Sugeno). Trong mục này chúng ta chỉ xét các tập mờ dùng trong mệnh đề kết luận của quy tắc Mamdani. Các tập mờ này được định nghĩa trên tập cơ sở của ngõ ra bộ điều khiển và thường có hàm liên thuộc cách đều nhau. Rất khó có thể đưa ra nhận xét tổng quát về hình dạng, số lượng và vị trí của các tập mờ dùng cho tín hiệu ra, bởi vì ảnh hưởng của nó phụ thuộc rất nhiều vào các quy tắc mờ.

Do đặc điểm nội suy của hệ mờ, việc chọn số các hàm liên thuộc không làm hạn chế độ phân giải của tín hiệu ra hệ mờ, giả sử rằng

chúng ta sử dụng phương pháp giải mờ trung bình. Số lượng và vị trí của các hàm liên thuộc trên tập cơ sở của tín hiệu ra, kết hợp với các quy tắc mờ và tập cơ sở của các tín hiệu vào cần được chọn tương ứng với mặt đặc tính mong muốn.

Ta đã biết rằng, hệ quy tắc mờ quyết định hình dạng của mặt đặc tính. Ở mục trên ta đã thấy rằng các tập mờ đã định trước cho ngõ ra hệ mờ làm hạn chế mặt đặc tính. Vì vậy, nên dùng mệnh đề kết luận của các quy tắc mờ là hằng số (tập mờ dạng singleton) thay vì các tập mờ có dạng khác trên tập cơ sở của ngõ ra. Điều này nghĩa là trong khi thiết kế, hệ quy tắc được đưa ra dựa trên sự phân loại các ngõ vào và kết luận được xác định tự do. Điều này cho phép điều chỉnh nhanh chóng và dễ dàng kết luận của các quy tắc mờ bằng phương pháp thử sai.

2- Vai trò của các toán tử

Mặt đặc tính thể hiện mối quan hệ giữa tín hiệu vào và tín hiệu ra của hệ mờ không chỉ phụ thuộc vào các tập mờ như đã trình bày ở các mục trước mà còn phụ thuộc vào các toán tử được dùng để thực hiện các phép toán logic. Mục này phân tích ảnh hưởng của các toán tử thực hiện các phép toán logic.

a- Phép phủ định trong mệnh đề điều kiện của các quy tắc

Để thuận lợi, đôi khi toán tử phủ định (NOT) được sử dụng trong mệnh đề điều kiện của các quy tắc mờ. Tuy nhiên, điều này có thể dẫn đến sự không nhất quán. Để minh họa điều này, chúng ta hãy xét ví dụ sau. Giả sử các tập mờ $\tilde{A}_1$, $\tilde{A}_2$, $\tilde{A}_3$ được định nghĩa trên tập cơ sở của tín hiệu vào x và tập mờ $\tilde{B}$ được định nghĩa trên tập cơ sở của tín hiệu ra y .

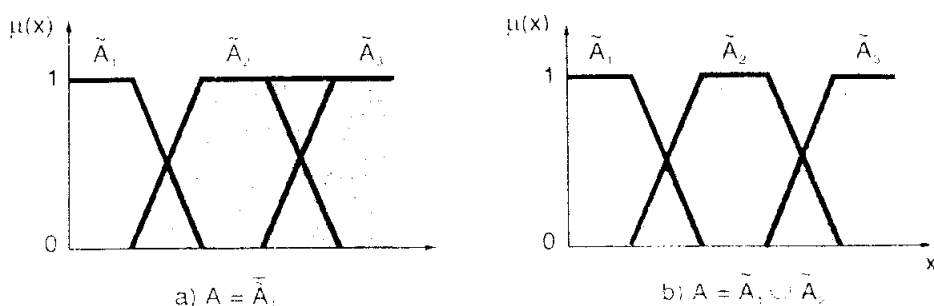
Xét các quy tắc sau:

r_1 : nếu (x không là $\tilde{A}_1$) thì (y là $\tilde{B}$)

r_2 : nếu (x là $\tilde{A}_2$ hoặc x là $\tilde{A}_3$) thì (y là $\tilde{B}$)

Nếu $\tilde{A}_1$, $\tilde{A}_2$, $\tilde{A}_3$ là tất cả các giá trị ngôn ngữ có thể có của tín hiệu vào x thì hai quy tắc trên là tương đương về mặt logic. Tuy nhiên, về mặt toán học, áp dụng lý thuyết logic mờ thì mệnh đề điều kiện của hai quy tắc trên có thể không giống nhau. Hình 2.39a là mệnh đề điều

kiện của quy tắc 1, hình 2.39b là mệnh đề điều kiện của quy tắc 2 với phép hợp (liên từ **hoặc**) được thực hiện bởi toán tử MAX. Rõ ràng hai mệnh đề điều kiện của hai quy tắc này khác nhau. Tuy nhiên, nếu các tập mờ ở ngõ vào được phân hoạch mờ trên tập cơ sở thì ta có thể tránh được sự không nhất quán bằng cách thực hiện phép hợp bằng toán tử BSUM (tổng bị chặn) và thực hiện phép phủ định bằng cách lấy bù chuẩn.



Hình 2.39 Tập mờ tương ứng với mệnh đề điều kiện dùng phép phủ định (a) và phép hợp (b)

b- Liên từ và (phép giao)

Liên từ **và** (AND) rất quan trọng trong hệ mờ khi có nhiều hơn hai tín hiệu vào. Nó được dùng để kết hợp các mệnh đề mờ trong mệnh đề điều kiện của các quy tắc mờ. Toán tử được sử dụng cho liên từ **và** có thể là một toán tử T bất kỳ (MIN, PROD...), tuy nhiên câu hỏi đặt ra là nên dùng toán tử T nào. Trong các tài liệu về lý thuyết mờ ta có thể thấy nhiều hệ mờ thực tế dùng toán tử MIN cho liên từ **và**, tuy nhiên đó có phải là sự lựa chọn đúng đắn không? Toán tử MIN được dùng chỉ vì nó được đưa ra đầu tiên bởi Zadeh, hay vì nó là sự lựa chọn tốt nhất trong một bài toán nào đó. Trong nhiều trường hợp, việc sử dụng toán tử MIN cho liên từ **và** trong các hệ mờ còn nhiều tranh cãi, bởi vì nó dẫn đến sự phi tuyến. Để minh họa điều này chúng ta xét ví dụ sau. Giả sử hệ mờ gồm các quy tắc:

r_1 : nếu x_1 là NE và x_2 là NE thì y là NE

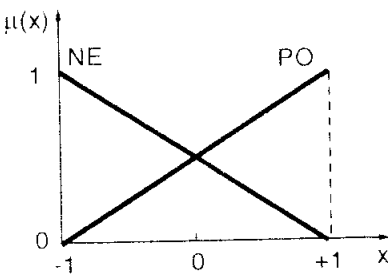
r_2 : nếu x_1 là NE và x_2 là PO thì y là ZE

r_3 : nếu x_1 là PO và x_2 là NE thì y là ZE

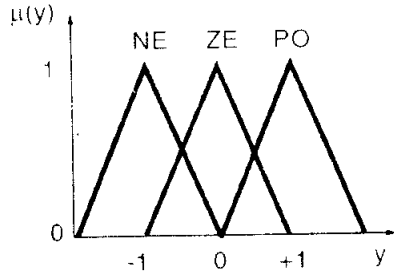
r_4 : nếu x_1 là PO và x_2 là PO thì y là PO

Hình 2.40a là các tập mờ mô tả các giá trị ngôn ngữ của x_1, x_2 , Hình 2.40b là các tập mờ mô tả các giá trị ngôn ngữ của y . Dùng toán tử MIN cho liên từ **và**, mặt đặc tính thể hiện quan hệ giữa tín hiệu vào và tín hiệu ra của bộ điều khiển mờ được trình bày ở hình 2.40c, để ý rằng trong trường hợp này mặt đặc tính là phi tuyến. Chú ý, loại toán tử thực hiện phép kéo theo ảnh hưởng không đáng kể vì chúng ta sử dụng phương pháp giải mờ trung bình mờ. Hình 2.46d trình bày mặt đặc tính khi toán tử PROD được dùng cho liên từ **và**. Như thấy trong hình vẽ đây là hàm tuyến tính.

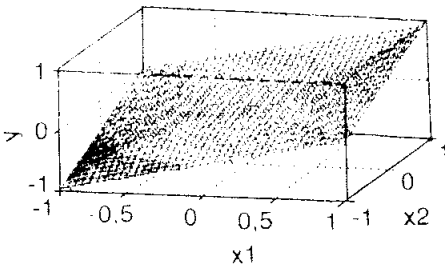
Từ những kết quả trên có thể kết luận rằng, việc dùng toán tử MIN cho liên từ **và** là một câu hỏi vì nó dẫn đến sự phi tuyến của mặt đặc tính. Thật ra bất cứ toán tử T nào khác ngoài toán tử PROD cũng đều cho mặt đặc tính phi tuyến.



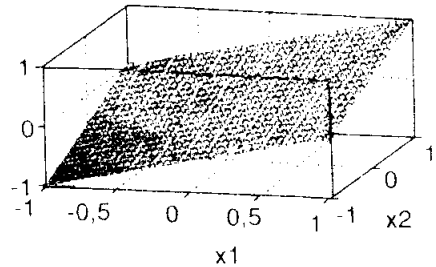
a)



b)



c)



d)

Hình 2.40 Ảnh hưởng của toán tử thực hiện phép giao đến mặt đặc tính

a) Các tập mờ ở ngõ vào; b) Các tập mờ ở ngõ ra

c) Toán tử MIN thực hiện phép giao dẫn đến mặt đặc tính phi tuyến

d) Sự phi tuyến có thể khắc phục bằng cách dùng toán tử PROD

c- Liên từ hoặc (phép hợp)

Như đã đề cập ở mục trước dùng toán tử **phủ định** trong các quy tắc mờ có một số thuận lợi. Việc dùng liên từ **hoặc** (OR) cũng dẫn đến một số thuận lợi khi thiết kế hệ quy tắc mờ. Số quy tắc có thể giảm xuống bằng cách dùng toán tử **phủ định** (NOT) và liên từ **hoặc** (OR). Tương tự như việc sử dụng toán tử **phủ định** (NOT), việc dùng liên từ **hoặc** (OR) cũng có thể dẫn đến sự không nhất quán (xem ví dụ ở mục 2 phần 2.6.2). Để giải quyết vấn đề không nhất quán của toán tử phủ định có thể dùng tổng bị chặn (BSUM) cho liên từ **hoặc** (OR).

3- Vai trò của hệ quy tắc

Hệ quy tắc mờ gồm các quy tắc biểu diễn tri thức của con người trong việc giải bài toán. Hệ quy tắc mờ là yếu tố ảnh hưởng chủ yếu đến quan hệ phi tuyến giữa ngõ vào và ngõ ra của hệ mờ. Nên chọn hệ quy tắc mờ thỏa mãn ba tính chất: liên tục, nhất quán và hoàn chỉnh (xem mục 2.6.2). Tuy nhiên, điều này không bắt buộc.

2.8.6 Hệ mờ là bộ xấp xỉ tổng quát

Hệ mờ (2.107) là bộ xấp xỉ tổng quát, có thể xấp xỉ hàm phi tuyến $f(x)$ trơn bất kỳ với sai số ε nhỏ tùy ý, điều này thể hiện qua định lý sau.

Định lý 2.1: Tính xấp xỉ tổng quát của hệ mờ

Cho $f(x)$ là hàm phi tuyến liên tục xác định trên tập chặt $X \subset \mathbb{R}^n$ và ε là số dương nhỏ tùy ý, tồn tại hệ mờ (2.107) sao cho:

$$\sup_{x \in X} |f(x) - y(x, \theta)| < \varepsilon \quad (2.113)$$

Định lý trên được Wang và Mendel [109] chứng minh lần đầu tiên với hàm liên thuộc của các tập mờ ở mệnh đề điều kiện là hàm Gauss, sau đó Kosko [66] chứng minh định lý trên cho các hệ mờ có các hàm liên thuộc ở mệnh đề điều kiện có các dạng khác như tam giác, dạng hình thang, π ... Chú ý rằng, định lý chỉ khẳng định tồn tại hệ mờ (2.107) xấp xỉ hàm phi tuyến $f(x)$ với sai số nhỏ tùy ý, định lý không cho biết phải chọn hệ mờ (2.107) như thế nào để đạt được sai số này. Ngoài ra, để đạt được độ chính xác nhỏ tùy ý có thể cần phải tăng số quy tắc mờ lên rất lớn. Gần đây hơn, một số công trình đưa ra điều kiện cần (tối thiểu) để mô hình mờ Mamdani có thể xấp xỉ hàm

phi tuyến $f(x)$ với sai số ε cho trước [117]. Thông thường, hệ mờ (2.107) được chọn thử sai với độ phức tạp tăng dần đến khi biểu thức (2.113) được thỏa mãn, đây chính là khuyết điểm lớn gây khó khăn trong việc sử dụng hệ mờ để xấp xỉ các hàm phi tuyến trong các ứng dụng thực tế. Giá trị của tính chất xấp xỉ tổng quát của hệ mờ đơn giản chỉ là nó chỉ ra rằng nếu chúng ta kiên nhẫn chỉnh định hệ mờ, chúng ta có thể sẽ đạt được hệ mờ với chất lượng mà ta mong muốn.

2.9 TÓM TẮT

Chương 2 đã trình bày một cách có hệ thống các vấn đề lý thuyết liên quan đến hệ mờ. Trên cơ sở lý thuyết tập mờ, lý thuyết suy luận mờ được đưa ra nhằm mục đích biểu diễn toán học sự suy luận không chắc chắn của con người. Hệ mờ là hệ thống suy luận mờ gồm ba thành phần cơ bản là mờ hóa, hệ quy tắc và giải mờ. Có hai loại quy tắc được sử dụng phổ biến trong các hệ mờ là quy tắc Mamdani và quy tắc Takagi–Sugeno. Trong điều khiển, hệ mờ có thể được sử dụng như là bộ điều khiển hoặc mô hình để nhận dạng đối tượng. Các ứng dụng của hệ mờ trong lĩnh vực điều khiển sẽ được trình bày từ chương 5 trở đi.

MẠNG THẦN KINH

3.1 GIỚI THIỆU

Bộ não con người (H.3.1) là hệ thống xử lý thông tin phức hợp, phi tuyến và song song có khả năng học, ghi nhớ, tổng quát hóa và xử lý lỗi. Bộ não con người gồm khoảng 10^{11} tế bào thần kinh liên kết với nhau thành mạng (H.3.2a). Tế bào thần kinh sinh học có cấu tạo như hình 3.2b, mỗi tế bào thần kinh sinh học gồm ba thành phần chính là thân tế bào (*soma*), một hệ thống hình cây các đầu dây thần kinh vào (*dendrite*) và một trục (*axon*) dẫn đến đầu dây thần kinh ra. Tại đầu của các dây thần kinh có các khớp thần kinh (*synapse*) để kết nối với các tế bào thần kinh khác. Mỗi tế bào thần kinh trong bộ não con người có khoảng 10^4 khớp thần kinh. Có hai loại khớp nối: *khớp nối kích thích* (*excitatory*) hoặc *khớp nối ức chế* (*inhibitory*).

Tín hiệu truyền trong các dây thần kinh vào và dây thần kinh ra của các tế bào thần kinh là tín hiệu điện phát sinh thông qua các quá trình phản ứng và giải phóng các chất hữu cơ. Các chất này được phát ra từ các khớp nối dẫn tới các dây thần kinh vào sẽ làm tăng hay giảm điện thế của nhân tế bào. Khi điện thế này đạt đến một ngưỡng nào đó, sẽ tạo ra một xung điện dẫn đến trục dây thần kinh ra. Xung này được truyền theo trục, tới các nhánh rẽ khi chạm tới các khớp nối với các tế bào thần kinh khác sẽ giải phóng các chất truyền điện. Quá trình lan truyền tín hiệu cứ tiếp tục như vậy cho đến khi đến đầu ra cuối cùng.

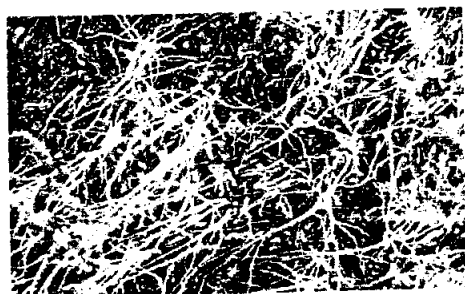
Mạng thần kinh nhân tạo là mô hình toán học đơn giản của bộ não con người, bản chất mạng thần kinh nhân tạo là mạng tính toán phân bố song song. Trái với các mô hình tính toán thông thường, hầu hết các mạng thần kinh phải được huấn luyện trước khi sử dụng. Các nghiên cứu về mạng thần kinh nhân tạo đã bắt đầu từ thập niên 1940.

Năm 1944, McCulloch và Pitts công bố công trình nghiên cứu về liên kết của các tế bào thần kinh. Năm 1949, Hebb công bố nghiên cứu về tính thích nghi của mạng thần kinh. Cuối những năm 1950, Rosenblatt đưa ra mạng Perceptron. Nghiên cứu về mạng thần kinh chỉ phát triển mạnh từ sau những năm 1980 sau giai đoạn thoái trào từ năm 1969, khi Minsky và Papert chỉ ra một số khuyết điểm của mạng perceptron. Năm 1985, mạng Hopfield ra đời và sau đó một năm là mạng lan truyền ngược. Đến nay đã có rất nhiều cấu hình mạng và các thuật toán huấn luyện tương ứng được công bố để giải quyết các bài toán khác nhau.

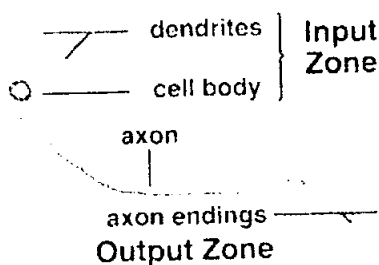
Chương này sẽ trình bày cơ bản về tế bào thần kinh, cấu trúc và thuật toán huấn luyện các loại mạng thần kinh sử dụng phổ biến trong lĩnh vực điều khiển thông minh [76]. Các dạng mạng thần kinh khác có thể tham khảo [39]. Các ứng dụng cụ thể của mạng thần kinh trong lĩnh vực điều khiển sẽ được trình bày ở các chương tiếp theo.



Hình 3.1 Bộ não con người



a)



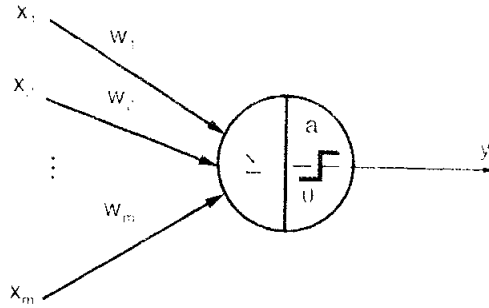
b)

Hình 3.2 a) Mạng thần kinh sinh học; b) Tế bào thần kinh sinh học

3.2 TẾ BÀO THẦN KINH VÀ MẠNG THẦN KINH NHÂN TẠO

3.2.1 Tế bào thần kinh

Mạng thần kinh nhân tạo (gọi ngắn gọn là *mạng thần kinh*) có thể xem như là mô hình toán học đơn giản của bộ não con người. Mạng thần kinh gồm các tế bào thần kinh (đơn vị xử lý) kết nối với nhau bởi các liên kết. Mỗi liên kết kèm theo một trọng số, đặc trưng cho đặc tính kích thích hay ức chế giữa các tế bào thần kinh.



Hình 3.3 Tế bào thần kinh nhân tạo

Mỗi tế bào thần kinh được nối với các tế bào thần kinh khác và nhận các tín hiệu x_j từ chúng với các trọng số w_j .

Đặt: $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_m]^T$: vector tín hiệu vào tế bào thần kinh (3.1)

$\mathbf{w} = [w_1 \ w_2 \ \dots \ w_m]^T$: vector trọng số của tế bào thần kinh (3.2)

Quá trình xử lý thông tin của tế bào thần kinh có thể chia thành hai phần: xử lý ở ngõ vào và xử lý ở ngõ ra. Hàm xử lý ở ngõ vào là hàm tổng có dạng như sau.

Hàm tuyến tính (linear function):

$$f = \text{net} = \left(\sum_{j=1}^m w_j x_j \right) - \theta = \mathbf{w}^T \mathbf{x} - \theta \quad (3.3)$$

Hàm toàn phương (quadratic function):

$$f = \text{net} = \left(\sum_{j=1}^m w_j x_j^2 \right) - \theta \quad (3.4)$$

Hàm cầu (spherical function):

$$f = \text{net} = \left(\rho^{-2} \sum_{j=1}^m (x_j - w_j)^2 \right) - \theta = \rho^{-2} (\mathbf{x} - \mathbf{w})^T (\mathbf{x} - \mathbf{w}) - \theta \quad (3.5)$$

Trong các biểu thức trên, θ là mức ngưỡng của tế bào thần kinh. Thường hàm tổng ở ngõ vào được sử dụng nhiều nhất là hàm tuyến tính (3.3). Ngõ ra của mỗi tế bào thần kinh được cho bởi biểu thức:

$$y = a(f) \quad (3.6)$$

Hàm $a(.)$ gọi là hàm tác động (*activation function*) hay hàm truyền (*transfer function*). Các dạng hàm tác động thường dùng là:

Hàm nấc (H.3.4a):

$$a(f) = \begin{cases} 1 & \text{nếu } f \geq 0 \\ 0 & \text{nếu } f < 0 \end{cases} \quad (3.7)$$

Hàm dấu (H.3.4b):

$$a(f) = \begin{cases} 1 & \text{nếu } f \geq 0 \\ -1 & \text{nếu } f < 0 \end{cases} \quad (3.8)$$

Hàm tuyến tính:

$$a(f) = f \quad (3.9)$$

Hàm dốc bão hòa (H.3.4c):

$$a(f) = \begin{cases} 1 & \text{nếu } f > 1 \\ f & \text{nếu } 0 \leq f \leq 1 \\ 0 & \text{nếu } f < 0 \end{cases} \quad (3.10)$$

Hàm tuyến tính bão hòa (H.3.4d):

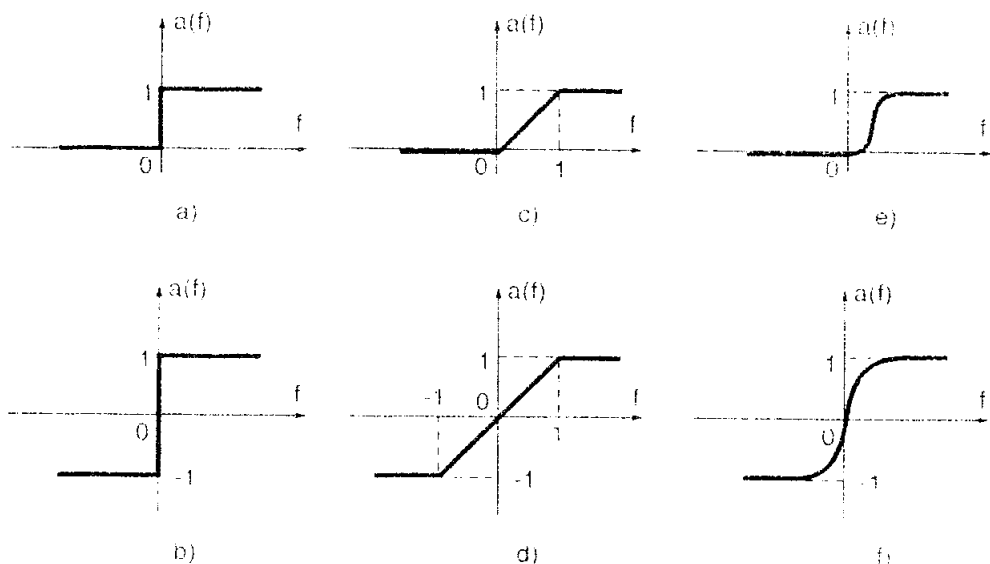
$$a(f) = \begin{cases} 1 & \text{nếu } f > 1 \\ f & \text{nếu } 0 \leq |f| \leq 1 \\ -1 & \text{nếu } f < -1 \end{cases} \quad (3.11)$$

Hàm dạng S đơn cực (H.3.4e):

$$a(f) = \frac{1}{1 + e^{-\lambda f}} \quad (3.12)$$

Hàm dạng S lưỡng cực (H.3.4f):

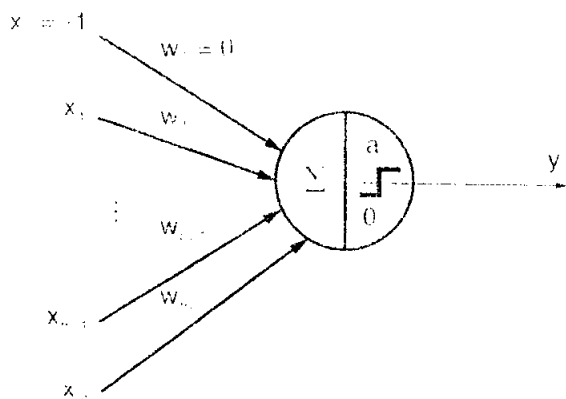
$$a(f) = \frac{2}{1 + e^{-\lambda f}} - 1 \quad (3.13)$$



Hình 3.4 Các dạng hàm tác động thường dùng

a) Hàm nấc; b) Hàm dấu; c) Hàm dốc; d) Hàm tuyến tính bão hòa
e) Hàm dạng S đơn cực; f) Hàm dạng S lưỡng cực

Chú ý rằng, tế bào thần kinh có m tín hiệu vào với mức ngưỡng θ có thể biến đổi tương đương thành tế bào thần kinh có $m+1$ tín hiệu vào với mức ngưỡng $\theta = 0$ như sau:



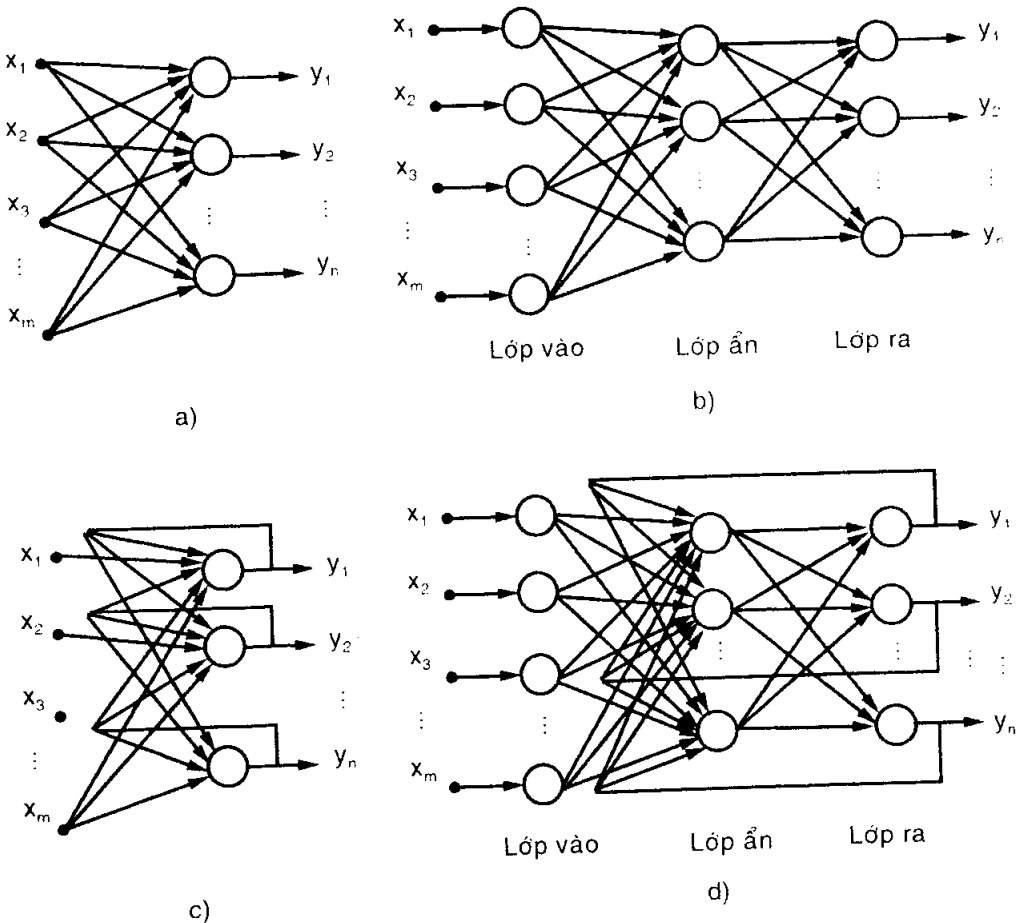
Hình 3.5 Biến đổi tương đương tế bào thần kinh về dạng có mức ngưỡng bằng 0

Tổng có trọng số của các tín hiệu vào của tế bào thần kinh ở hình 3.5 là:

$$f = net = \left(\sum_{j=0}^m w_j x_j \right) - \theta = \sum_{j=1}^m w_j x_j - \theta \quad (3.14)$$

Rõ ràng biểu thức (3.3) và (3.14) là tương đương nhau. Vì vậy, để cho đơn giản (và cũng không mất tính tổng quát), từ đây về sau chúng ta chỉ xét các tế bào thần kinh có mức ngưỡng $\theta = 0$.

3.2.2 Mạng thần kinh nhân tạo



Hình 3.6 Các loại mạng thần kinh nhân tạo thường gặp

- a) Mạng thần kinh truyền thẳng một lớp
- b) Mạng thần kinh truyền thẳng nhiều lớp
- c) Mạng thần kinh hồi quy một lớp
- d) Mạng thần kinh hồi quy nhiều lớp

Mạng thần kinh nhân tạo gồm nhiều tế bào thần kinh liên kết chặt chẽ với nhau. Tùy theo cấu trúc mạng, người ta chia ra các loại mạng sau (H.3.6).

- *Mạng một lớp* (H.3.6a, H.3.6c) là mạng chỉ gồm một lớp tế bào thần kinh xử lý.
- *Mạng nhiều lớp* (H.3.6b, H.3.6d) là mạng gồm nhiều lớp tế bào thần kinh xử lý
- *Mạng truyền thẳng* (H.3.6a, H.3.6c) là mạng mà trong đó tín hiệu chỉ truyền theo một chiều từ ngõ vào đến ngõ ra.
- *Mạng hồi quy* (H.3.6c, H.3.6d) là mạng mà trong đó có tín hiệu hồi tiếp từ ngõ ra trở về ngõ vào.

3.3 CÁC PHƯƠNG PHÁP HUẤN LUYỆN MẠNG THẦN KINH NHÂN TẠO

Mạng thần kinh nhân tạo phải được huấn luyện trước khi sử dụng. Có hai kiểu học:

- *Học thông số* (Parameter Learning): để cập nhật các trọng số liên kết giữa các tế bào thần kinh trong mạng.
- *Học cấu trúc* (Structure Learning): để thay đổi cấu trúc mạng, bao gồm số tế bào thần kinh và cách liên kết giữa chúng.

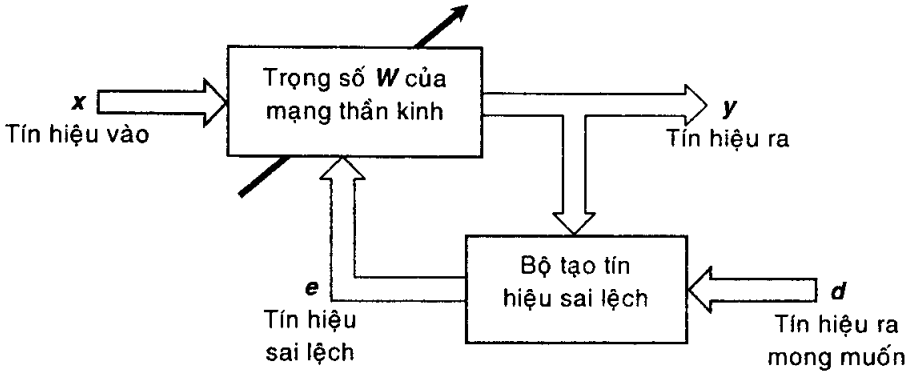
Hai kiểu học trên có thể sử dụng đồng thời hay riêng lẻ. Quyển sách này chỉ tập trung vào kiểu học thông số. Có ba phương pháp học thông số.

3.3.1 Học có giám sát (Supervised Learning)

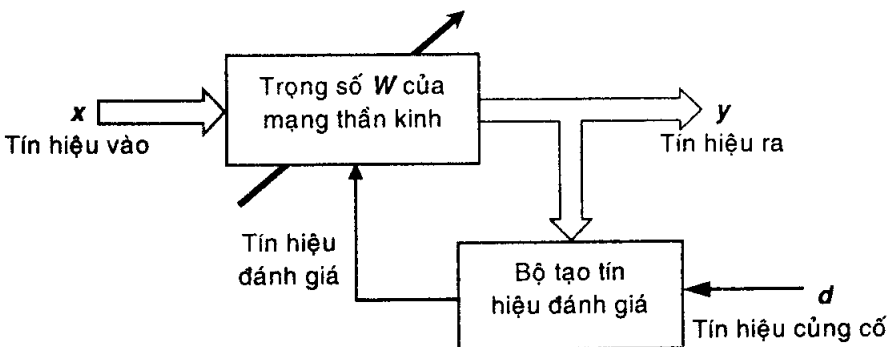
Trong phương pháp học có giám sát, dữ liệu huấn luyện mạng thần kinh là tập các mẫu (dữ liệu vào–dữ liệu ra) mong muốn. Giả sử có K mẫu dữ liệu huấn luyện mạng:

$$\{(x(1), d(1)); (x(2), d(2)); \dots; (x(K), d(K))\}$$

Ban đầu các trọng số của mạng được khởi động giá trị bất kỳ. Lần lượt từng dữ liệu vào $x(k)$ được đưa vào các ngõ vào của mạng, mạng thần kinh tính được dữ liệu ở ngõ ra là $y(k)$. Dữ liệu ra $y(k)$ được so sánh với dữ liệu ra mong muốn $d(k)$. Sai số $e(k) = d(k) - y(k)$ được sử dụng để cập nhật trọng số của mạng (H.3.7).

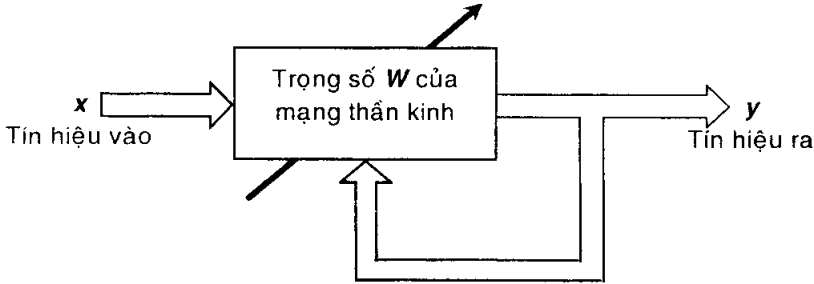
**Hình 3.7** Học có giám sát**3.3.2 Học củng cố (Reinforcement Learning)**

Trong phương pháp học có giám sát, chúng ta đã biết dữ liệu ở ngõ ra của mạng tương ứng với mỗi mẫu tín hiệu vào. Tuy nhiên trong thực tế, không phải lúc nào chúng ta cũng có được thông tin đầy đủ như vậy. Ví dụ, trong một số trường hợp chúng ta chỉ biết ngõ ra thực của mạng là “đúng” hay “sai” so với ngõ ra mong muốn. Phương pháp học dựa trên thông tin đánh giá như trên gọi là học củng cố. Thông tin đánh giá chỉ cho biết mức độ “đúng” hay “sai” của ma trận trọng số mà không chỉ dẫn được phải thay đổi ma trận trọng số như thế nào để đi đến kết quả đúng (H.3.8).

**Hình 3.8** Học củng cố**3.3.3 Học không có giám sát (Unsupervised Learning)**

Trong phương pháp học không có giám sát, không có thông tin phản hồi cho biết ngõ ra của mạng là đúng hay sai. Mạng thần kinh

phải tự phát hiện các đặc điểm, các mối tương quan, hay các nhóm của tập mẫu dữ liệu vào và mã hóa chúng thành dữ liệu ra của mạng. Trong khi phát hiện ra các đặc điểm này, mạng thần kinh thay đổi thông số của nó, quá trình này gọi là tự tổ chức (H.3.9).



Hình 3.9 Học không có giám sát

Có thể so sánh ba phương pháp học của mạng thần kinh nhân tạo với ba cách học của bộ não con người (bảng 3.1).

Bảng 3.1 So sánh các phương pháp học của mạng thần kinh nhân tạo

Bộ não con người	Mạng thần kinh nhân tạo
Học có sự hướng dẫn của giáo viên	Học có giám sát
Học có sự đánh giá của giáo viên	Học củng cố
Tự học	Học không có giám sát

Chúng ta vừa xét ba phương pháp huấn luyện mạng thần kinh nhân tạo. Dù huấn luyện mạng thần kinh nhân tạo theo phương pháp nào thì sơ đồ tổng quát để huấn luyện **một tế bào thần kinh** cũng có dạng như sau:

Trong hình 3.10, tín hiệu vào x_j ($j = 1, 2, \dots, m$) có thể là tín hiệu ra từ một tế bào thần kinh khác hoặc có thể là tín hiệu vào từ bên ngoài. Chú ý rằng mức ngưỡng θ của tế bào thần kinh cũng bao gồm trong quá trình học như là một trọng số của ngõ vào x_0 có giá trị cố định bằng -1 . Tín hiệu ra mong muốn d_i chỉ tồn tại trong phương pháp học có giám sát, hoặc trong phương pháp học củng cố, d_i là tín hiệu đánh giá. Tổng quát vector trọng số của tế bào thần kinh thay đổi theo quy luật sau:

$$w_i(k+1) = w_i(k) + \Delta w_i(k) \quad (3.15)$$

trong đó $\Delta w_i(k)$ là biến thiên của vector trọng số tại bước lặp thứ k .

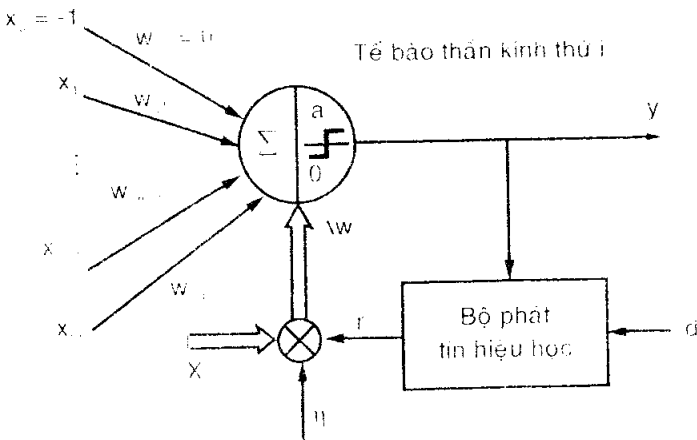
$$\Delta w_i(k) = \eta r x(k) \tag{3.16}$$

η là một số dương quyết định tốc độ học, gọi là hằng số học; r là tín hiệu học, tổng quát là hàm của w_i , x và tín hiệu mong muốn d_i (nếu tồn tại).

$$r = f_r(w_i(k), x(k), d_i) \tag{3.17}$$

Kết hợp (3.15) và (3.16) ta được:

$$w_i(k + 1) = w_i(k) + \eta r x(k) \tag{3.18}$$



Hình 3.10 Sơ đồ tổng quát huấn luyện một tế bào thần kinh nhân tạo

Trong các mục sau chúng ta sẽ xét thuật toán học cụ thể cho từng loại mạng thần kinh nhân tạo như thuật toán học perceptron, thuật toán học Widrow–Hoff, thuật toán học delta, thuật toán lan truyền ngược...

3.4 MẠNG TRUYỀN THĂNG MỘT LỚP

Mạng truyền thăng một lớp, hay còn gọi là mạng perceptron đơn giản (simple perceptron), có cấu trúc như hình 3.11a. Hàm tổng ở ngõ vào của Perceptron là hàm tuyến tính, hàm tác động ở ngõ ra có thể là hàm dấu, hàm tuyến tính hay hàm dạng S. Tùy theo từng loại hàm tác động mà ta có các thuật toán khác nhau để huấn luyện perceptron.

3.4.1 Thuật toán học Perceptron

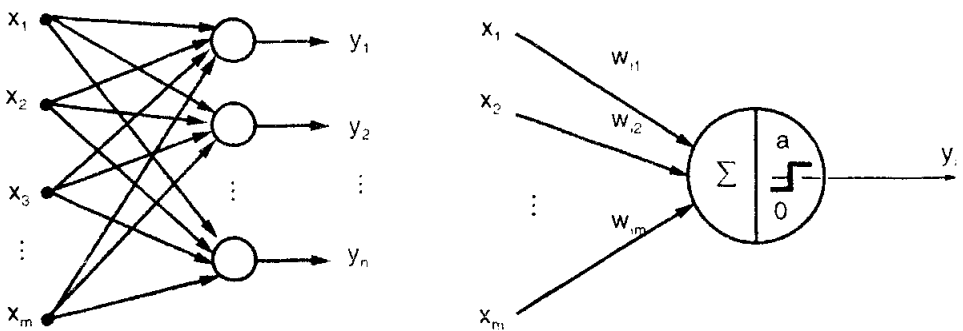
Mạng thần kinh nhân tạo đơn giản nhất là Perceptron có hàm tác động là hàm giới hạn cứng, mạng này thường được gọi tắt là Perceptron. Thuật toán huấn luyện mạng gọi là thuật toán học Perceptron. Perceptron được quan tâm nghiên cứu rất nhiều vì nó có khả năng tổng quát hóa từ những vector mẫu huấn luyện và khả năng học từ các trọng số và mức ngưỡng ngẫu nhiên bất kỳ. Perceptron đặc biệt thích hợp với những phân loại mẫu đơn giản. Mạng Perceptron có ưu điểm là mạng nhanh và tin cậy. Ngoài ra, việc hiểu hoạt động của perceptron cung cấp kiến thức cơ bản tốt để hiểu các mạng phức tạp hơn.

Xét mạng Perceptron có m ngõ vào và n ngõ ra (H.3.11). Tổng trọng số của các ngõ vào Perceptron thứ i :

$$net_i = \sum_{j=1}^m w_{ij} x_j = \mathbf{w}_i^T \mathbf{x} \quad (i = \overline{1, n}) \quad (3.19)$$

Ngõ ra của Perceptron thứ i (giả sử hàm tác động là hàm nấc):

$$y_i = a(net_i) = \text{step}(net_i) = \begin{cases} 1 & \text{nếu } net_i \geq 0 \\ 0 & \text{nếu } net_i < 0 \end{cases} \quad (i = \overline{1, n}) \quad (3.20)$$



Hình 3.11 Mạng Perceptron

Cho K mẫu dữ liệu vào-ra:

$$\{(\mathbf{x}(1), \mathbf{d}(1)); (\mathbf{x}(2), \mathbf{d}(2)); \dots; (\mathbf{x}(K), \mathbf{d}(K))\}$$

trong đó $\mathbf{x} = [x_1, x_2, \dots, x_m]^T$ và $\mathbf{d} = [d_1, d_2, \dots, d_n]^T$, (m là số ngõ vào, n là số ngõ ra). Thuật toán học Perceptron là một trường hợp đặc biệt của thuật toán học tổng quát (3.18), trong đó tín hiệu học r_i để cập nhật trọng số của Perceptron thứ i ở bước lặp thứ k được xác định bởi:

$$r_i(k) = d_i(k) - y_i(k) \quad (i = \overline{1, n}) \quad (3.21)$$

Vì $r_i(k)$ chính là sai số giữa ngõ ra mong muốn và ngõ ra thực của mạng nên ta còn sử dụng ký hiệu $\delta_i(k) = r_i(k)$, do đó thuật toán học Perceptron còn được gọi là *thuật toán học sửa sai (Error Correction Learning Rule)*. Thuật toán học Perceptron được tóm tắt như sau:

Thuật toán 3.1: Luật học Perceptron

Bước 1: Chọn tốc độ học $\eta > 0$.

Bước 2: Khởi động:

- Gán sai số $E = 0$.
- Gán biến chạy $k = 1$.
- Gán các vector trọng số $w_i(k)$ ($i = \overline{1, n}$) bằng giá trị ngẫu nhiên nhỏ bất kỳ.

Bước 3: Quá trình huấn luyện bắt đầu, tính:

$$y_i(k) = \text{step}(w_i^T(k)x(k)) = \text{step}\left(\sum_{j=1}^m w_{ij}(k)x_j(k)\right) \quad (i = \overline{1, n})$$

Bước 4: Cập nhật các vector trọng số:

$$w_i(k+1) = w_i(k) + \eta(\delta_i(k) - y_i(k))x(k) \quad (i = \overline{1, n})$$

Bước 5: Tính sai số tích lũy:

$$E = E + \frac{1}{2} \|d(k) - y(k)\|^2$$

Bước 6: Nếu $k < K$ thì gán $k = k+1$ và trở lại **bước 3**. Nếu $k = K$ thì tiếp tục **bước 7**.

Bước 7: Kết thúc một chu kỳ huấn luyện (epoch).

- Nếu $E = 0$ thì kết thúc quá trình học.
- Nếu $E \neq 0$ thì gán $E = 0$, $k = 1$ và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

Vấn đề đặt ra là thuật toán học Perceptron có hội tụ hay không. Định lý hội tụ perceptron cho phép đánh giá tính hội tụ của thuật toán học Perceptron. Trước khi trình bày định lý này chúng ta cần tìm hiểu khái niệm *khả phân tuyến tính*.

1- Khái niệm khả phân tuyến tính (linearly separable)

Để đơn giản, trước tiên chúng ta xét trường hợp mạng chỉ có một Perceptron. Gọi tập hợp mẫu vector vào huấn luyện Perceptron là:

$$S = \{x(1), x(2), \dots, x(K)\}$$

Gọi S_0 là tập hợp các vector vào có ngõ ra mong muốn là 0:

$$S_0 = \{x(k) \in S | d(k) = 0\}$$

Gọi S_1 là tập hợp các vector vào có ngõ ra mong muốn là 1:

$$S_1 = \{x(k) \in S | d(k) = 1\}$$

Nếu tồn tại một đường thẳng (trường hợp mạng có hai ngõ vào), một mặt phẳng (trường hợp mạng có ba ngõ vào) hay một mặt siêu phẳng (*hyper-plane*, trường hợp mạng có nhiều hơn ba ngõ vào) phân chia không gian vector vào thành hai miền riêng biệt: một miền chứa S_0 , một miền chứa S_1 thì bài toán được gọi là *khả phân tuyến tính*.

Mở rộng khái niệm khả phân tuyến tính cho trường hợp mạng có nhiều Perceptron. Tập vector vào dùng để huấn luyện mạng có nhiều Perceptron được gọi là khả phân tuyến tính nếu tập vector vào đó khả phân tuyến tính đối với từng Perceptron trong mạng.

Để hiểu rõ hơn khái niệm *khả phân tuyến tính*, ta xét hai ví dụ sau đây.

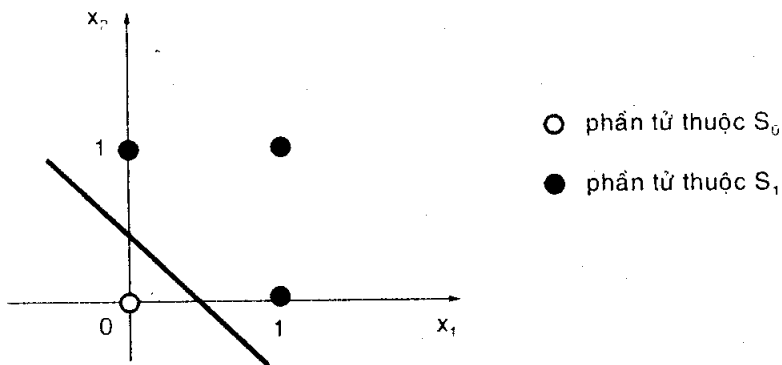
Ví dụ 3.1 Giả sử ta cần huấn luyện một Perceptron có hai ngõ vào, một ngõ ra để thực hiện phép toán logic OR. Dữ liệu huấn luyện mạng được cho ở bảng 3.2.

Bảng 3.2 Quan hệ vào ra của phép OR

x_1	x_2	d
0	0	0
0	1	1
1	0	1
1	1	1

Ta có: $S = \{(0,0); (0,1); (1,0); (1,1)\}$; $S_0 = \{(0,0)\}$; $S_1 = \{(0,1); (1,0); (1,1)\}$

Biểu diễn hai tập hợp trên trong một mặt phẳng. Dễ thấy rằng tồn tại một đường thẳng chia mặt phẳng thành hai miền: một miền chứa S_0 , một miền chứa S_1 nên bài toán OR là bài toán khả phân tuyến tính.



Hình 3.12 Tập mẫu vector vào của bài toán OR khả phân tuyến tính

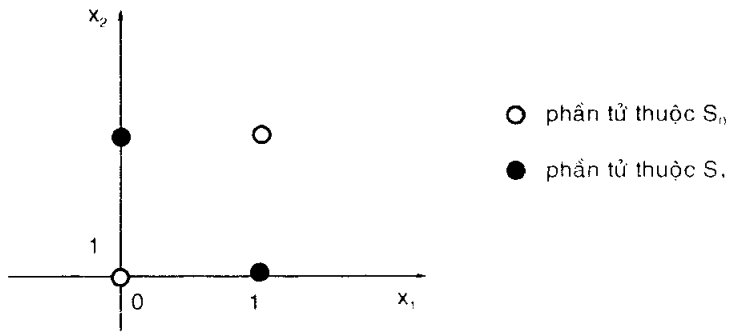
Ví dụ 3.2 Giả sử ta cần huấn luyện một Perceptron có hai ngõ vào, một ngõ ra để thực hiện phép toán logic XOR. Dữ liệu huấn luyện mạng cho ở bảng 3.3.

Bảng 3.3 Quan hệ vào ra của phép toán logic XOR

x_1	x_2	d
0	0	0
0	1	1
1	0	1
1	1	0

Ta có: $S = \{(0,0); (0,1); (1,0); (1,1)\}$; $S_0 = \{(0,0), (1,1)\}$; $S_1 = \{(0,1); (1,0)\}$

Biểu diễn hai tập hợp trên trong một mặt phẳng, ta thấy rằng không thể vẽ một đường thẳng chia mặt phẳng thành hai miền, một miền chứa S_0 , một miền chứa S_1 nên bài toán XOR không khả phân tuyến tính.



Hình 3.13 Tập mẫu vector vào của bài toán XOR không khả phân tuyến tính

Định lý 3.1: Định lý hội tụ perceptron

Nếu bài toán khả phân tuyến tính thì thuật toán 3.1 sẽ lặp lại bước 3 một số hữu hạn lần.

2- Ứng dụng mạng perceptron

Mạng perceptron thích hợp để giải các bài toán phân loại đơn giản. Sau đây là một ví dụ:

Ví dụ 3.3 Trong dây chuyền sản xuất gạch men, người ta dùng một mạng perceptron để phát hiện các viên gạch bị kém chất lượng. Người ta dùng búa gõ nhẹ vào các viên gạch và dùng micro thu tín hiệu âm thanh từ các viên gạch phát ra. Tín hiệu thu được qua mạch lọc nhiễu, phân tích phổ, người ta thu được số liệu ở bảng 3.4.

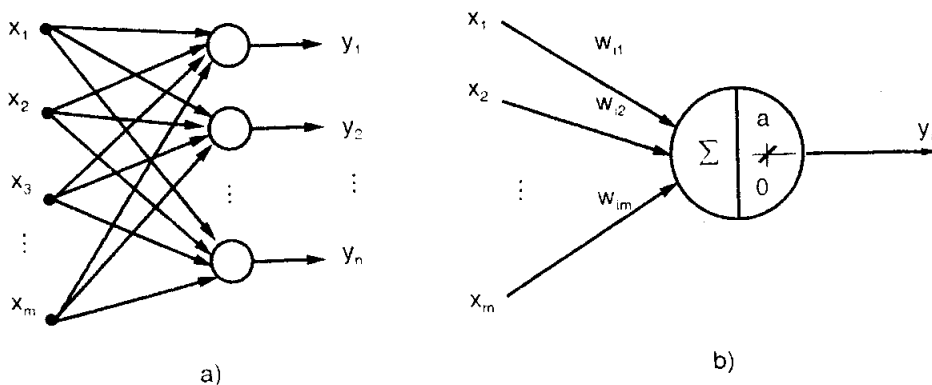
Bảng 3.4 Dữ liệu biên độ âm thanh theo chất lượng viên gạch

Biên độ tín hiệu 475Hz	Biên độ tín hiệu 555 Hz	Chất lượng viên gạch
0,958	0,003	Đạt
1,043	0,001	Đạt
1,907	0,003	Đạt
0,780	0,002	Đạt
0,579	0,001	Đạt
0,003	0,105	Không đạt
0,001	1,748	Không đạt
0,014	1,839	Không đạt
0,007	1,021	Không đạt
0,004	0,214	Không đạt

Dùng tập dữ liệu trên để huấn luyện mạng Perceptron hai ngõ vào, một ngõ ra. Sau khi huấn luyện xong, mạng có khả năng phát hiện các viên gạch kém chất lượng.

3.4.2 Adaline và mạng tuyến tính

Mạng tuyến tính (*Linear Network*) là mạng truyền thẳng một lớp gồm các phần tử thích nghi tuyến tính Adaline (*Adaptive Linear Element*). Adaline tương tự như Perceptron, chỉ khác ở chỗ hàm tác động của Adaline là hàm tuyến tính chứ không phải là hàm giới hạn cứng. Điều này cho phép ngõ ra của Adaline có thể nhận một giá trị bất kỳ. Trong mục này chúng ta xét thuật toán học Widrow–Hoff để huấn luyện Adaline. Thuật toán học Widrow–Hoff, bản chất là thuật toán trung bình bình phương tối thiểu (*LMS–Least Mean Squares*) nên hiệu quả hơn thuật toán học Perceptron và di chuyển giới hạn phân loại ra xa nhất có thể được so với các mẫu phân loại. Vì ngõ ra của Adaline có thể nhận một giá trị bất kỳ nên có thể sử dụng Adaline trong các bài toán điều khiển.



Hình 3.14 Mạng tuyến tính (a) và Adaline (b)

Xét mạng tuyến tính có m ngõ vào và n ngõ ra (H.3.14). Tổng trọng số của các ngõ vào Adaline thứ i là:

$$net_i = \sum_{j=1}^m w_{ij}x_j = \mathbf{w}_i^T \mathbf{x} \quad (3.22)$$

Ngõ ra của Adaline thứ i :

$$y_i = a(net_i) = net_i = \sum_{j=1}^m w_{ij}x_j \quad (3.23)$$

Cho K mẫu dữ liệu vào-ra:

$$\{(x(1), d(1)); (x(2), d(2)); \dots; (x(K), d(K))\}$$

trong đó $x = [x_1, x_2, \dots, x_m]^T$ và $d = [d_1, d_2, \dots, d_n]^T$, (m là số ngõ vào, n là số ngõ ra), ta cần tìm các vector trọng số w_i sao cho:

$$\sum_{j=1}^m w_{ij} x_j(k) = d_i(k), \quad (k = 1, \dots, K) \quad (3.24)$$

Dễ thấy rằng (3.24) là hệ gồm K phương trình tuyến tính. Lời giải w_i^* của hệ phương trình trên chỉ tồn tại nếu các mẫu dữ liệu huấn luyện $x(1), x(2), \dots, x(K)$ độc lập tuyến tính, điều này xảy ra chỉ khi $K \leq m$. Do đó chúng ta chỉ có thể lưu trữ tối đa m quan hệ bất kỳ trong một Adaline.

Nếu các mẫu dữ liệu huấn luyện mạng không thỏa mãn điều kiện độc lập tuyến tính, ta tìm các vector trọng số w_i sao cho tối thiểu tổng bình phương sai số giữa ngõ ra mong muốn và ngõ ra của mạng:

$$E(w) = \sum_{i=1}^n (d_i - y_i)^2 \rightarrow \min \quad (3.25)$$

Có thể tìm được lời giải của bài toán tìm cực trị (3.25) bằng thuật toán suy giảm độ dốc:

$$w_{ij}(k+1) = w_{ij}(k) - \eta \frac{\partial E(k)}{\partial w_{ij}} \quad (3.26)$$

Ta có:

$$\frac{\partial E(k)}{\partial w_{ij}} = \left[\frac{\partial E(k)}{\partial y_i} \right] \left[\frac{\partial y_i}{\partial net_i} \right] \left[\frac{\partial net_i}{\partial w_{ij}} \right] = -[d_i(k) - y_i(k)] \cdot 1 \cdot x_j(k) \quad (3.27)$$

Thay biểu thức (3.27) vào biểu thức (3.26) ta được:

$$w_{ij}(k+1) = w_{ij}(k) + \eta [d_i(k) - y_i(k)] x_j(k) \quad (3.28)$$

Thuật toán (3.28) được gọi là thuật toán học Adaline hay thuật toán học Widrow-Hoff. Đối với thuật toán học Widrow-Hoff trọng số ban đầu có thể khởi động giá trị bất kỳ. Thuật toán học Widrow-Hoff được tóm tắt như sau:

Thuật toán 3.2: Thuật toán học Widrow-Hoff

Bước 1: Chọn tốc độ học $\eta > 0$, chọn sai số cực đại E_{max}

Bước 2: Khởi động:

- Gán sai số $E = 0$
- Gán biến chạy $k = 1$
- Gán các trọng số $w_{ij}(k)$ ($i = \overline{1, n}; j = \overline{1, m}$) bằng giá trị bất kỳ.

Bước 3: Quá trình huấn luyện bắt đầu, tính:

$$y_i(k) = \sum_{j=1}^m w_{ij}(k)x_j(k) \quad (i = \overline{1, n})$$

Bước 4: Cập nhật các trọng số theo biểu thức:

$$w_{ij}(k+1) = w_{ij}(k) + \eta[d_i(k) - y_i(k)]x_j(k) \quad (i = \overline{1, n}; j = \overline{1, m})$$

Bước 5: Tính sai số tích lũy:

$$E = E + \frac{1}{2} \sum_{i=1}^n [d_i(k) - y_i(k)]^2$$

Bước 6: Nếu $k < K$ thì gán $k = k+1$ và trở lại **bước 3**.

Nếu $k = K$ thì tiếp tục **bước 7**.

Bước 7: Kết thúc một chu kỳ huấn luyện.

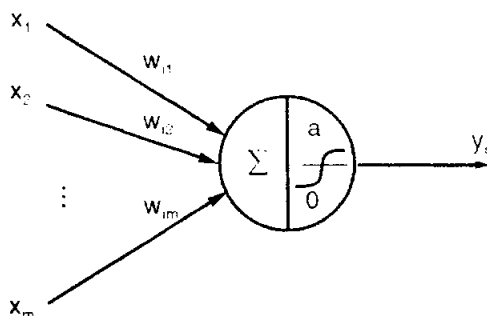
- Nếu $E < E_{max}$ thì kết thúc quá trình học.
- Nếu $E \geq E_{max}$ thì gán $E = 0$, $k = 1$ và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

Chú ý: Thoạt nhìn ta tưởng rằng hai thuật toán 3.1 và 3.2 là như nhau vì biểu thức cập nhật trọng số ở bước 4 như nhau. Tuy nhiên, về bản chất hai thuật toán trên khác nhau: thuật toán 3.1 không phải là thuật toán tối ưu, trong khi thuật toán 3.2 là thuật toán tối ưu, quá trình học theo thuật toán 3.2 làm cực tiểu hóa sai số, nghĩa là làm cho mặt siêu phẳng phân chia các nhóm nằm cách xa các nhóm nhất.

Adaline chỉ có thể học mối quan hệ tuyến tính giữa vector vào và vector ra. Vì vậy Adaline không thể tìm ra lời giải cho một số bài toán. Tuy nhiên, mặc dù lời giải hoàn chỉnh không tồn tại, Adaline cũng tối thiểu tổng của bình phương sai số nếu tốc độ học được chọn đủ nhỏ.

3.4.3 Perceptron với hàm tác động bán tuyến tính

Ta đã biết Perceptron đơn giản sử dụng hàm tác động là hàm giới hạn cứng (hàm nấc hay hàm dấu), mà hàm giới hạn cứng lại không khả vi tại mọi điểm nên ta không thể áp dụng thuật toán học tối ưu. Để khắc phục khuyết điểm trên, ta có thể thay hàm giới hạn cứng bằng hàm giới hạn mềm.



Hình 3.15 Tế bào thần kinh nhân tạo với hàm tác động là hàm giới hạn mềm

Xét mạng thần kinh truyền thẳng với hàm tác động bán tuyến tính:

$$y_i = a(net_i) = a\left(\sum_{j=1}^m w_{ij}x_j\right) \quad (3.29)$$

Sai số của mạng:

$$E(w) = \sum_{i=1}^n (d_i - y_i)^2 \quad (3.30)$$

Áp dụng thuật toán suy giảm độ dốc, trọng số của mạng được cập nhật bởi công thức:

$$w_{ij}(k+1) = w_{ij}(k) - \eta \frac{\partial E(k)}{\partial w_{ij}} \quad (3.31)$$

Để dàng thấy rằng:

$$\frac{\partial E(k)}{\partial w_{ij}} = \left[\frac{\partial E(k)}{\partial y_i} \right] \left[\frac{\partial y_i}{\partial net_i} \right] \left[\frac{\partial net_i}{\partial w_{ij}} \right] = [-(d_i(k) - y_i(k))] [a'(net_i(k))] [x_j(k)] \quad (3.32)$$

$$\text{Đặt: } \delta_i(k) = -\frac{\partial E(k)}{\partial net_i(k)} = -\left[\frac{\partial E(k)}{\partial y_i} \right] \left[\frac{\partial y_i}{\partial net_i} \right] = [d_i(k) - y_i(k)] a'(net_i(k)) \quad (3.33)$$

$\delta_i(k)$ gọi là tín hiệu sai số. Kết hợp (3.31), (3.32) và (3.33) ta được:

$$w_{ij}(k+1) = w_{ij}(k) + \eta \delta_i(k) x_j(k) \quad (3.34)$$

Biểu thức (3.34) gọi là thuật toán học Delta. Thuật toán học Delta được tóm tắt như sau.

Thuật toán 3.3: Thuật toán học Delta

Bước 1: Chọn tốc độ học $\eta > 0$, chọn sai số cực đại E_{max}

Bước 2: Khởi động:

- Gán sai số $E = 0$
- Gán biến chạy $k = 1$
- Gán các trọng số $w_{ij}(k)$ bằng giá trị **ngẫu nhiên** nhỏ bất kỳ.

Bước 3: Quá trình huấn luyện bắt đầu, tính:

$$net_i(k) = \sum_{j=1}^m w_{ij}(k) x_j(k) \quad (i = \overline{1, n})$$

$$y_i(k) = a(net_i(k)) \quad (i = \overline{1, n})$$

Bước 4: Cập nhật các trọng số theo biểu thức:

$$\delta_i(k) = [d_i(k) - y_i(k)] a'(net_i(k))$$

$$w_{ij}(k+1) = w_{ij}(k) + \eta \delta_i(k) x_j(k)$$

Bước 5: Tính sai số tích lũy:

$$E = E + \frac{1}{2} \sum_{i=1}^n [d_i(k) - y_i(k)]^2$$

Bước 6: Nếu $k < K$ thì gán $k = k+1$ và trở lại **bước 3**.

Nếu $k = K$ thì tiếp tục **bước 7**.

Bước 7: Kết thúc một chu kỳ huấn luyện.

- Nếu $E < E_{max}$ thì kết thúc quá trình học.
- Nếu $E \geq E_{max}$ thì gán $E = 0$, $k = 1$ và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

Chú ý rằng tương tự như mạng tuyến tính, điều kiện để tìm được lời giải $E(w) = 0$ cho mạng perceptron với hàm tác động dạng S là các mẫu huấn luyện phải thỏa mãn điều kiện độc lập tuyến tính.

Khác với mạng tuyến tính, thuật toán học Delta cho mạng perceptron với hàm tác động dạng S có thể tìm được các lời giải cục bộ. Do đó, để có khả năng tìm được lời giải toàn cục trọng số ban đầu của mạng phải được chọn là giá trị **ngẫu nhiên** nhỏ bất kỳ.

Ví dụ 3.4 Xét mạng thần kinh với hàm tác động là hàm dạng S đơn cực:

$$a(f) = \frac{1}{1 + e^{-\lambda f}} \quad (3.35)$$

Để ý rằng hàm dạng S đơn cực có tính chất:

$$\frac{\partial a(f)}{\partial f} = \frac{\lambda e^{-\lambda f}}{(1 + e^{-\lambda f})^2} = \frac{-\lambda + \lambda + \lambda e^{-\lambda f}}{(1 + e^{-\lambda f})^2} = \lambda a(f)[1 - a(f)] \quad (3.36)$$

Do đó tín hiệu sai số trong trường hợp hàm tác động dạng S đơn cực như sau:

$$\begin{aligned} \delta_i(k) &= [d_i(k) - y_i(k)]a'(net_i(k)) \\ &= [d_i(k) - y_i(k)]\lambda a(net_i(k))[1 - a(net_i(k))] \\ &= (d_i(k) - y_i(k))\lambda y_i(k)(1 - y_i(k)) \end{aligned} \quad (3.37)$$

Thuật toán huấn luyện mạng thần kinh một lớp với hàm tác động dạng S đơn cực là thuật toán Delta với tín hiệu sai số tính bởi biểu thức (3.37).

Ví dụ 3.5 Xét mạng thần kinh với hàm tác động là hàm dạng S lưỡng cực:

$$a(f) = \frac{2}{1 + e^{-f}} - 1 \quad (3.38)$$

$$\text{Do:} \quad \frac{\partial a(f)}{\partial f} = \frac{2e^{-f}}{(1 + e^{-f})^2} = \frac{1}{2}[1 - a^2(f)] \quad (3.39)$$

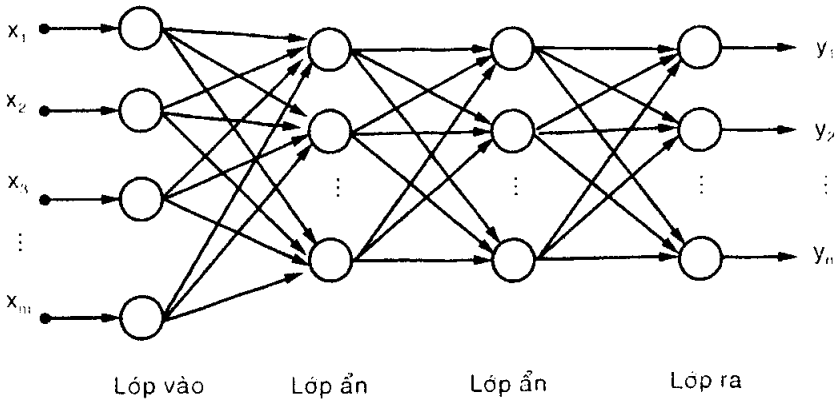
Nên tín hiệu sai số trong trường hợp hàm tác động dạng S lưỡng cực là:

$$\begin{aligned} \delta_i(k) &= [d_i(k) - y_i(k)]a'(net_i(k)) \\ &= [d_i(k) - y_i(k)]\frac{1}{2}[1 - a^2(net_i(k))] \\ &= (d_i(k) - y_i(k))\frac{1}{2}(1 - [y_i(k)]^2) \end{aligned} \quad (3.40)$$

3.5 MẠNG TRUYỀN THẮNG NHIỀU LỚP

3.5.1 Cấu trúc mạng

Mạng truyền thắng nhiều lớp là mạng truyền thắng có từ hai lớp tế bào thần kinh xử lý trở lên. Lớp tế bào thần kinh nối với ngõ vào gọi là lớp vào (thường lớp vào không thực hiện các phép toán xử lý), lớp tế bào thần kinh nối với ngõ ra gọi là lớp ra, lớp tế bào thần kinh không nối trực tiếp với ngõ vào và ngõ ra gọi là lớp ẩn. Kết nối giữa các tế bào thần kinh ở các lớp có thể đầy đủ hoặc không đầy đủ. Khác với mạng truyền thắng một lớp, mạng truyền thắng nhiều lớp có thể giải bài toán không khả phân tuyến tính. Thuật toán huấn luyện mạng truyền thắng nhiều lớp là thuật toán lan truyền ngược. Thuật toán lan truyền ngược thực hiện hai bước truyền thông tin. Thứ nhất, mẫu dữ liệu vào $x(k)$ được truyền từ ngõ vào đến ngõ ra, kết quả của việc truyền dữ liệu theo hướng thuận là tạo ra tín hiệu $y(k)$ ở ngõ ra của mạng. Sau đó sai số là sai lệch giữa $d(k)$ và $y(k)$ được truyền ngược từ lớp ra trở về lớp trước đó để cập nhật trọng số của mạng. Do thuật toán huấn luyện mạng truyền thắng là thuật toán lan truyền ngược nên mạng này còn được gọi là **mạng lan truyền ngược**.

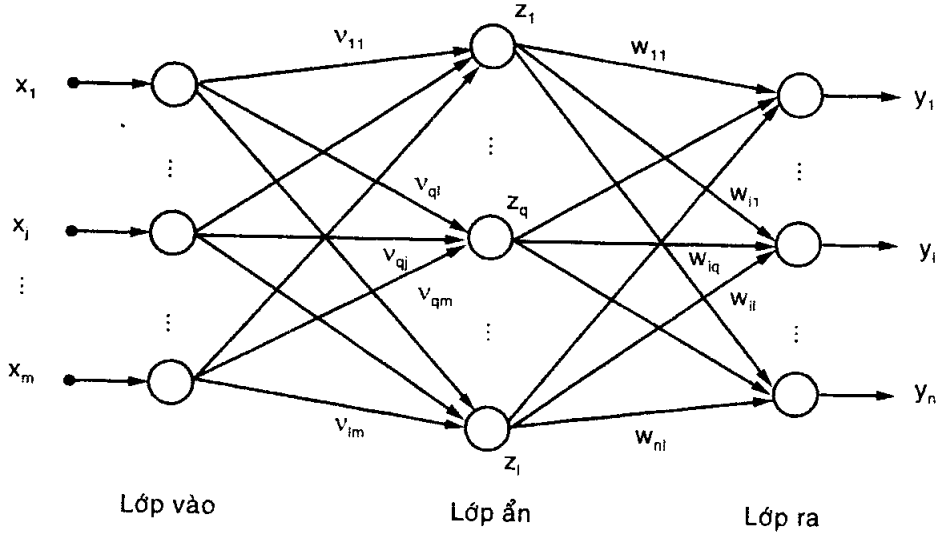


Hình 3.16 Mạng truyền thắng nhiều lớp

3.5.2 Thuật toán lan truyền ngược (Back Propagation – BP)

Xét mạng thần kinh truyền thắng ba lớp có m tế bào thần kinh ở lớp vào, l tế bào thần kinh ở lớp ẩn và n tế bào thần kinh ở lớp ra (H.3.17). Ký hiệu trọng số của tế bào thần kinh thứ q của lớp ẩn là v_q , trọng số của tế bào thần kinh thứ i ở lớp ra là w_i . Gọi hàm

tác động của các tế bào ở lớp ẩn là $a_h(.)$, của lớp ra là $a_o(.)$. Quan hệ giữa các tín hiệu truyền trong mạng như sau:



Hình 3.17 Mạng thần kinh truyền thẳng ba lớp

Tổng có trọng số của các tín hiệu vào tế bào thần kinh thứ q ở lớp ẩn là:

$$net_q = \sum_{j=1}^m v_{qj} x_j \quad (3.41)$$

Tín hiệu ra của tế bào thần kinh thứ q ở lớp ẩn là:

$$z_q = a_h(net_q) = a_h\left(\sum_{j=1}^m v_{qj} x_j\right) \quad (3.42)$$

Tổng có trọng số của các tín hiệu vào tế bào thần kinh thứ i ở lớp ra là:

$$net_i = \sum_{q=1}^l w_{iq} z_q = \sum_{q=1}^l w_{iq} a_h(net_q) = \sum_{q=1}^l w_{iq} a_h\left(\sum_{j=1}^m v_{qj} x_j\right) \quad (3.43)$$

Tín hiệu ra của tế bào thần kinh thứ i ở lớp ra là:

$$\begin{aligned} y_i &= a_o(net_i) = a_o\left(\sum_{q=1}^l w_{iq} z_q\right) \\ &= a_o\left(\sum_{q=1}^l w_{iq} a_h(net_q)\right) = a_o\left(\sum_{q=1}^l w_{iq} a_h\left(\sum_{j=1}^m v_{qj} x_j\right)\right) \end{aligned} \quad (3.44)$$

Giả sử ta có tập dữ liệu huấn luyện mạng gồm K mẫu $(x(k), d(k))$, $k = \overline{1..K}$. Tiêu chuẩn để huấn luyện mạng là làm tối thiểu sai số:

$$\begin{aligned} E(w, v) &= \frac{1}{2} \sum_{i=1}^n (d_i - y_i)^2 = \frac{1}{2} \sum_{i=1}^n [d_i - a_o(net_i)]^2 \\ &= \frac{1}{2} \sum_{i=1}^n \left[d_i - a_o \left(\sum_{q=1}^l w_{iq} z_q \right) \right]^2 = \frac{1}{2} \sum_{i=1}^n \left[d_i - a_o \left(\sum_{q=1}^l w_{iq} a_h(net_q) \right) \right]^2 \end{aligned} \quad (3.45)$$

Áp dụng thuật toán suy giảm độ dốc, trọng số của lớp ra được cập nhật:

$$w_{iq}(k+1) = w_{iq}(k) - \eta \frac{\partial E(k)}{\partial w_{iq}} \quad (3.46)$$

$$\frac{\partial E(k)}{\partial w_{iq}} = \left[\frac{\partial E(k)}{\partial y_i} \right] \left[\frac{\partial y_i}{\partial net_i} \right] \left[\frac{\partial net_i}{\partial w_{iq}} \right] = [-(d_i(k) - y_i(k))] [a'_o(net_i(k))] [z_q(k)] \quad (3.47)$$

$$\text{Đặt: } \delta_{oi}(k) = -\frac{\partial E(k)}{\partial net_i} = \left[\frac{\partial E(k)}{\partial y_i} \right] \left[\frac{\partial y_i}{\partial net_i} \right] = [(d_i(k) - y_i(k))] [a'_o(net_i(k))] \quad (3.48)$$

Thay (3.47) và (3.48) vào (3.46) ta được:

$$w_{iq}(k+1) = w_{iq}(k) + \eta \delta_{oi}(k) z_q(k) \quad (3.49)$$

Tương tự, trọng số của lớp ẩn được cập nhật bởi công thức:

$$v_{qj}(k+1) = v_{qj}(k) - \eta \frac{\partial E(k)}{\partial v_{qj}} \quad (3.50)$$

$$\begin{aligned} \frac{\partial E(k)}{\partial v_{iq}} &= \left[\frac{\partial E(k)}{\partial z_q} \right] \left[\frac{\partial z_q}{\partial net_q} \right] \left[\frac{\partial net_q}{\partial v_{qj}} \right] \\ &= \left[-\sum_{i=1}^n (d_i(k) - y_i(k)) a'_o(net_i(k)) w_{iq}(k) \right] [a'_h(net_q(k))] [x_j(k)] \end{aligned} \quad (3.51)$$

$$\begin{aligned} \text{Đặt: } \delta_{hq}(k) &= -\frac{\partial E(k)}{\partial net_q} = -\left[\frac{\partial E(k)}{\partial z_q} \right] \left[\frac{\partial z_q}{\partial net_q} \right] \\ &= \left[\sum_{i=1}^n (d_i(k) - y_i(k)) a'_o(net_i(k)) w_{iq}(k) \right] [a'_h(net_q(k))] \end{aligned} \quad (3.52)$$

Thay (3.48) vào (3.52) ta được:

$$\delta_{hq}(k) = \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] a'_h(net_q(k)) \quad (3.53)$$

Kết hợp (3.50), (3.51) và (3.53) ta được:

$$v_{qj}(k+1) = v_{qj}(k) + \eta \delta_{hq}(k) x_j(k) \quad (3.54)$$

Do trọng số của mạng được cập nhật dựa vào tín hiệu sai số $(\delta_{oi}(k), \delta_{hq}(k))$ và có thể mở rộng được cho mạng có nhiều lớp tế bào thần kinh tính toán hơn nên thuật toán lan truyền ngược còn được gọi là *thuật toán học Delta tổng quát*. Thuật toán lan truyền ngược để huấn luyện mạng truyền thẳng ba lớp được tóm tắt như sau:

Thuật toán 3.4: Thuật toán học lan truyền ngược sai số (BP)

Bước 1: Chọn tốc độ học $\eta > 0$, chọn sai số cực đại E_{max} .

Bước 2: Khởi động:

- Gán sai số $E = 0$.
- Gán biến chạy $k = 1$.
- Gán các trọng số $w_{iq}(k), v_{qj}(k)$ ($i = \overline{1, n}; j = \overline{1, m}; q = \overline{1, l}$) bằng giá trị ngẫu nhiên nhỏ bất kỳ.

Bước 3: (Truyền thuận dữ liệu) Tính ngõ ra của mạng với tín hiệu vào là $x^{(k)}$:

$$\text{Lớp ẩn: } net_q(k) = \sum_{j=1}^m v_{qj}(k) x_j(k) \quad (q = \overline{1, l})$$

$$z_q(k) = a_h(net_q(k)) \quad (q = \overline{1, l})$$

$$\text{Lớp ra: } net_i(k) = \sum_{q=1}^l w_{iq}(k) z_q(k) \quad (i = \overline{1, n})$$

$$y_i(k) = a_o(net_i(k)) \quad (i = \overline{1, n})$$

Bước 4: (Lan truyền ngược sai số) Cập nhật trọng số của mạng:

$$\text{Lớp ra: } \delta_{oi}(k) = [(d_i(k) - y_i(k))][a'_o(net_i(k))] \quad (i = \overline{1, n})$$

$$w_{iq}(k+1) = w_{iq}(k) + \eta \delta_{oi}(k) z_q(k)$$

$$(i = \overline{1, n}; \quad q = \overline{1, l})$$

$$\text{Lớp ẩn: } \delta_{hq}(k) = \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] a'_h(net_q(k)) \quad (q = \overline{1, l})$$

$$v_{qj}(k+1) = v_{qj}(k) + \eta \delta_{hq}(k) x_j(k)$$

$$(j = \overline{1, m}; \quad q = \overline{1, l})$$

Bước 5: Tính sai số tích lũy: $E = E + \frac{1}{2} \sum_{i=1}^n (d_i(k) - y_i(k))^2$

Bước 6: Nếu $k < K$ thì gán $k = k+1$ và trở lại **bước 3**.

Nếu $k = K$ thì tiếp tục **bước 7**.

Bước 7: Kết thúc một chu kỳ huấn luyện.

- Nếu $E < E_{max}$ thì kết thúc quá trình học.

- Nếu $E \geq E_{max}$ thì gán $E = 0$, $k = 1$ và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

Ví dụ 3.6 Xét mạng thần kinh ba lớp với hàm tác động ở lớp ẩn là hàm dạng S lưỡng cực và hàm tác động ở lớp ra là hàm tuyến tính.

$$a_h(f) = \frac{2}{1 + e^{-f}} - 1 \quad (3.55)$$

$$a_o(f) = f \quad (3.56)$$

$$\text{Do: } a'_h(f) = \frac{1}{2} [1 - a^2(f)] \quad (3.57)$$

$$a'_o(f) = 1 \quad (3.58)$$

Nên tín hiệu sai số ở lớp ra là:

$$\delta_{oi}(k) = [d_i(k) - y_i(k)] a'_o(\text{net}_i(k)) = d_i(k) - y_i(k) \quad (3.59)$$

và tín hiệu sai số ở lớp ẩn là:

$$\begin{aligned} \delta_{hq}(k) &= \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] a'_h(\text{net}_q(k)) \\ &= \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] \frac{1}{2} [1 - (a_h(\text{net}_q(k)))^2] \\ &= \frac{1}{2} \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] [1 - (z_q(k))^2] \end{aligned} \quad (3.60)$$

Thuật toán huấn luyện mạng là thuật toán lan truyền ngược với tín hiệu sai số xác định bởi biểu thức (3.59) và (3.60).

Tương tự, có thể rút ra được tín hiệu học cho mạng thần kinh ba lớp có hàm tác động ở lớp ẩn và lớp ra đều là hàm S lưỡng cực như sau:

$$\begin{aligned} \delta_{oi}(k) &= \frac{1}{2} [d_i(k) - y_i(k)] [1 - (y_i(k))^2] \\ \delta_{hq}(k) &= \frac{1}{2} \left[\sum_{i=1}^n \delta_{oi}(k) w_{iq}(k) \right] [1 - (z_q(k))^2] \end{aligned}$$

Định nghĩa 3.1

Hàm tác động $a(f) : R \rightarrow [0,1]$ (hoặc $[-1,1]$) được gọi là hàm tác động dạng nén nếu $a(f)$ không giảm, $\lim_{f \rightarrow +\infty} a(f) = 1$ và $\lim_{f \rightarrow -\infty} a(f) = 0$ (hoặc -1).

Rõ ràng hàm tác động dạng S đơn cực và lưỡng cực là hàm tác động dạng nén.

Định lý 3.2

Mạng truyền thẳng có ít nhất một lớp ẩn với hàm tác động dạng nén (squashing activation function) và hàm tổng dạng tuyến tính hoặc đa thức có thể xấp xỉ một hàm bất kỳ với độ chính xác mong muốn nếu số tế bào thần kinh ở lớp ẩn đủ lớn.

Định lý 3.2 khẳng định mạng truyền thẳng nhiều lớp là bộ xấp xỉ tổng quát. Do đó, nếu trong một ứng dụng nào đó mà thuật toán lan truyền ngược huấn luyện mạng không hội tụ thì nguyên nhân có thể do thông số huấn luyện mạng không thích hợp, số lượng tế bào thần kinh ở lớp ẩn không đủ lớn, hoặc có lỗi trong tập mẫu dữ liệu huấn luyện mạng.

Mặc dù định lý 3.2 khẳng định mạng truyền thẳng ba lớp (một lớp ẩn) có thể xấp xỉ một hàm bất kỳ nhưng thực tế đôi khi người ta sử dụng mạng truyền thẳng có nhiều lớp hơn. Lý do là trong nhiều ứng dụng, nếu dùng mạng ba lớp thì số tế bào thần kinh ở lớp ẩn quá lớn, trong khi đó nếu dùng mạng nhiều hơn ba lớp thì số lượng tế bào thần kinh ở các lớp ẩn sẽ ít hơn.

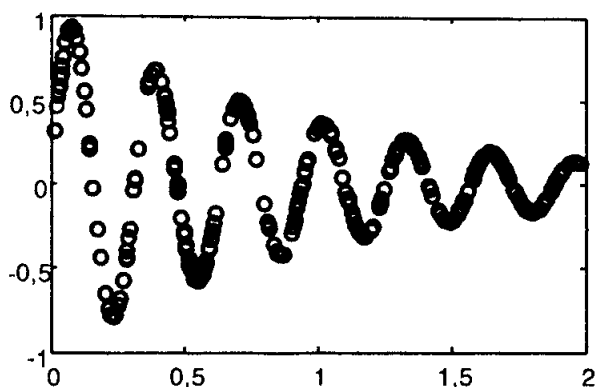
Ví dụ 3.7 Giả sử chúng ta cần xấp xỉ hàm phi tuyến:

$$y = e^{-x} \sin(10x); \quad 0 \leq x \leq 2 \quad (3.61)$$

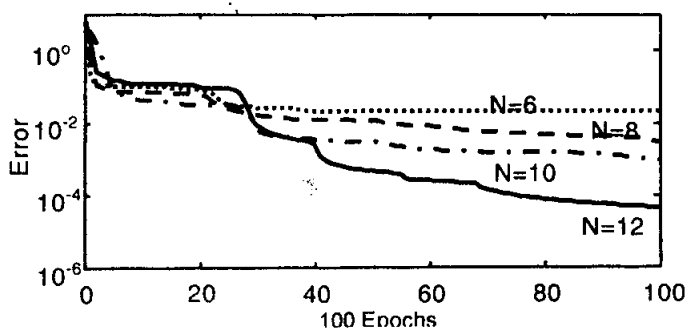
Cấu hình mạng thần kinh xấp xỉ hàm phi tuyến (3.61) như sau:

- Một ngõ vào là x , một ngõ ra là y .
- Số tế bào thần kinh ở lớp ẩn là N . Ví dụ này khảo sát khả năng xấp xỉ của mạng trong bốn trường hợp N bằng 6, 8, 10, 12.
- Hàm kích hoạt ở lớp ẩn là tansig, ở lớp ra là tuyến tính.

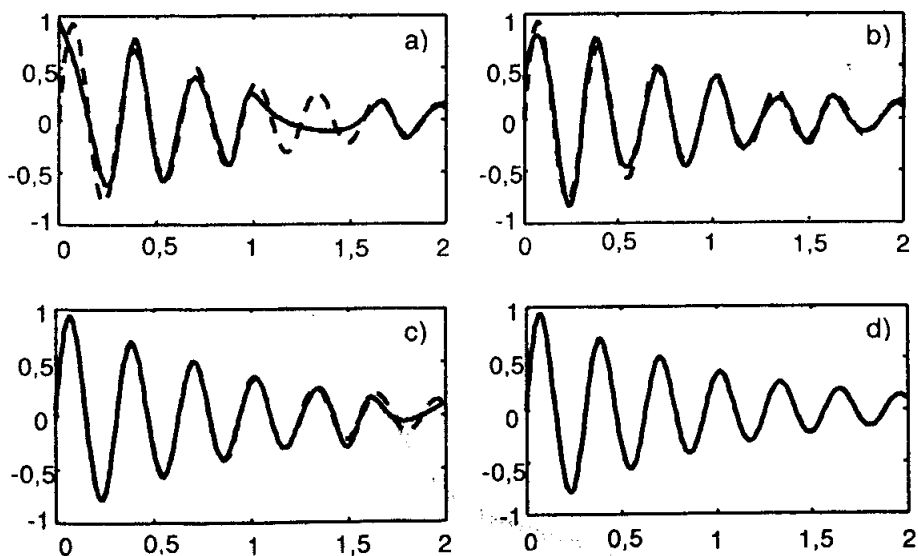
Hình 3.18 là tập dữ liệu ngẫu nhiên gồm 300 mẫu dùng để huấn luyện mạng. Theo biểu thức (3.61) và đồ thị hình 3.18 ta thấy hàm cần xấp xỉ có tính phi tuyến cao.



Hình 3.18 Tập dữ liệu huấn luyện mạng



Hình 3.19 Sai số huấn luyện mạng



Hình 3.20 Kết quả xấp xỉ hàm dùng mạng thần kinh

a) $N = 6$; b) $N = 8$; c) $N = 10$; d) $N = 12$

Hình 3.19 là sai số huấn luyện mạng trong các trường hợp số tế bào thần kinh ở lớp ẩn là 6, 8, 10, 12. Rõ ràng khi số tế bào thần kinh ở lớp ẩn càng tăng thì sai số càng giảm. Hình 3.20 là ngõ ra của mạng trong bốn trường hợp nêu trên, để thấy rằng khi số tế bào thần kinh ở lớp ẩn đủ lớn sai lệch giữa ngõ ra của mạng và giá trị hàm số (3.61) không đáng kể.

Ví dụ 3.8 Tính toán động học cánh tay máy

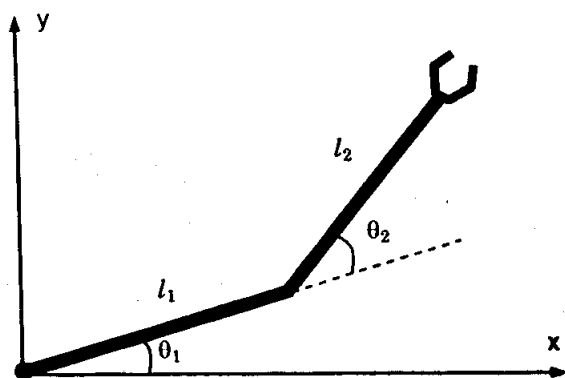
Trong ví dụ này một mạng thần kinh lan truyền ngược đơn giản được sử dụng để giải bài toán động học thuận của một cánh tay máy (H.3.21). Tín hiệu vào của mạng là tọa độ góc giữa các khớp nối (θ_1, θ_2) , tín hiệu ra của mạng là tọa độ đề-các (x, y) . Quan hệ phi tuyến giữa (θ_1, θ_2) và (x, y) như sau:

$$\begin{cases} x = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2) \\ y = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2) \end{cases} \quad (3.62)$$

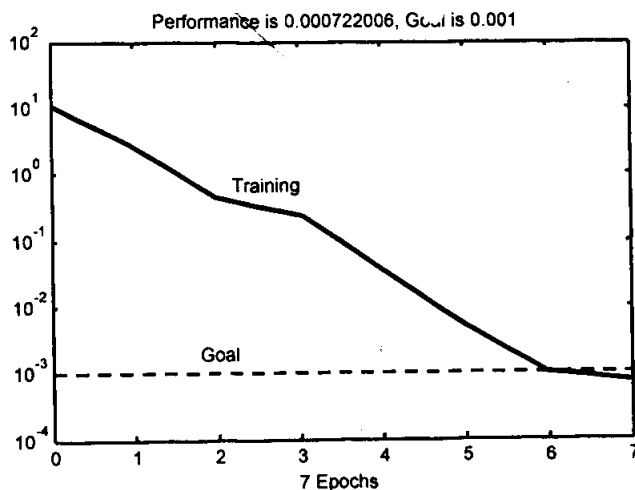
Cấu trúc mạng thần kinh dùng để xấp xỉ quan hệ (3.62) như sau:

- Hai ngõ vào là θ_1 và θ_2
- Hai ngõ ra là x và y
- Số tế bào thần kinh ở lớp ẩn là 10.
- Hàm kích hoạt ở lớp ẩn là tansig, ở lớp ra là tuyến tính.

Tập dữ liệu gồm 200 mẫu được sử dụng để huấn luyện mạng, cho thấy sai số của mạng giảm dần trong quá trình huấn luyện đến khi đạt được sai số mong muốn. Sau khi đã huấn luyện xong, chỉ cần đặt vào ngõ vào của mạng giá trị (θ_1, θ_2) bất kỳ mạng sẽ tính tọa độ (x, y) tương ứng.



Hình 3.21 Cánh tay máy



Hình 3.22 Sai số của mạng trong quá trình huấn luyện

3.5.3 Các yếu tố ảnh hưởng đến quá trình huấn luyện mạng

1- Trọng số khởi động

Trọng số ban đầu của mạng truyền thẳng nhiều lớp ảnh hưởng rất nhiều đến lời giải của thuật toán lan truyền ngược. Thường trọng số ban đầu được khởi động là giá trị ngẫu nhiên nhỏ. Nếu khởi động trọng số ban đầu bằng nhau thì không thể huấn luyện được mạng trong trường hợp lời giải đúng là các trọng số khác nhau. Trọng số ban đầu phải nhỏ vì nếu trọng số lớn sẽ làm cho hàm tác động dạng S bão hòa ngay từ đầu và thuật toán sẽ rơi vào cực trị cục bộ. Nên chọn các trọng số w_{ij} ban đầu ngẫu nhiên nằm trong đoạn $[-3/\sqrt{k_i}, 3/\sqrt{k_i}]$, trong đó k_i là số tín hiệu vào của tế bào thần kinh thứ i .

2- Hệ số học

Một yếu tố quan trọng khác ảnh hưởng mạnh đến tốc độ học và tính hội tụ của thuật toán lan truyền ngược là hệ số học η . Không có cách chọn η thích hợp cho các trường hợp khác nhau, thường η được chọn dựa vào kinh nghiệm cho mỗi bài toán. Giá trị η lớn làm tăng tốc độ học, nhưng nếu lớn quá thì có thể làm cho thuật toán không hội tụ, ngược lại giá trị η nhỏ bảo đảm thuật toán luôn hội tụ nhưng tốc độ học lại rất chậm. Thực nghiệm cho thấy giá trị η nằm trong khoảng 0,001 đến 10 là thích hợp trong nhiều trường hợp huấn luyện mạng.

Phương pháp hiệu quả nhất để đảm bảo thuật toán lan truyền ngược vừa hội tụ vừa huấn luyện mạng nhanh là dùng hệ số học thích nghi. Ý tưởng cơ bản của phương pháp học thích nghi như sau: ở mỗi bước lặp ta kiểm tra xem trọng số vừa được cập nhật có làm giảm tiêu chuẩn huấn luyện mạng không, nếu không có nghĩa là đã xảy ra vọt lố, trong trường hợp này nên giảm η ; ngược lại nếu trong vài bước lặp liên tiếp tiêu chuẩn huấn luyện mạng đều giảm thì η quá nhỏ, trong trường hợp này nên tăng η .

Cụ thể thuật toán học thích nghi được mô tả bởi các biểu thức:

$$\eta(k+1) = \eta(k) + \Delta\eta \quad (3.63)$$

$$\Delta\eta = \begin{cases} +a \\ -b\eta(k) \\ 0 \end{cases} \quad (3.64)$$

3- Tiêu chuẩn huấn luyện mạng

Tiêu chuẩn huấn luyện mạng thường dùng là chuẩn toàn phương L_2 (tổng bình phương của sai số, xem (3.44)), tổng quát ta có thể sử dụng chuẩn L_p .

$$E = \frac{1}{p} \sum_{i=1}^n (d_i - y_i)^p \quad (1 \leq p < +\infty) \quad (3.65)$$

Dễ thấy rằng khi thay đổi tiêu chuẩn huấn luyện mạng thì chỉ có tín hiệu học δ_{oi} của lớp ra là thay đổi, các biểu thức khác của thuật toán lan truyền ngược vẫn giữ nguyên.

4- Xung lượng

Phương pháp suy giảm độ dốc có thể rất chậm nếu hệ số học quá nhỏ và có thể dao động mạnh nếu hệ số học quá lớn. Hiện tượng này thường xảy ra nếu điểm cực tiểu có dạng thung lũng có độ dốc lớn ở sườn và độ dốc nhỏ ở đáy. Một phương pháp hiệu quả thường dùng cho phép hệ số học lớn mà không xảy ra dao động phân kỳ là cộng thêm một xung lượng vào phương pháp suy giảm độ dốc thông thường.

$$w_{ij}(k+1) = w_{ij}(k) + \Delta w_{ij}(k) \quad (3.66)$$

$$\Delta w_{ij}(k) = -\eta \frac{\partial E(k)}{\partial w_{ij}} + \alpha \Delta w_{ij}(k-1) \quad (3.67)$$

Trong công thức (3.66), α gọi là hệ số xung lượng ($0 \leq \alpha \leq 1$), giá trị α thường dùng là 0,9. Nhờ xung lượng cộng vào mà mỗi trọng số sẽ thay đổi theo hướng độ dốc trung bình, do đó thuật toán không bị dao động.

5- Luật cập nhật trọng số

Phương pháp suy giảm độ dốc là một trong những phương pháp tối ưu hóa đơn giản nhất nhưng không phải là phương pháp hiệu quả nhất. Phương pháp hiệu quả hơn để cập nhật trọng số của mạng thần kinh là phương pháp Newton:

$$w_{ij}(k+1) = w_{ij}(k) - \left[\frac{\partial^2 E(k)}{\partial w_{ij}^2} \right]^{-1} \left[\frac{\partial E(k)}{\partial w_{ij}} \right] \quad (3.68)$$

Phương pháp Newton hội tụ nhanh nhưng dễ rơi vào điểm cực trị cục bộ. Mặt khác tính toán đạo hàm bậc hai rất khó khăn nên thực tế phương pháp này ít được sử dụng.

6- Dữ liệu huấn luyện mạng

Dữ liệu huấn luyện mạng cần đủ và đúng. Không có quy tắc chung nào chỉ rõ cách chọn dữ liệu huấn luyện thích hợp trong mọi trường hợp. Thực nghiệm cho thấy dữ liệu huấn luyện mạng cần chọn ngẫu nhiên và bao hàm toàn bộ không gian của tín hiệu vào. Gọi K là số mẫu dữ liệu huấn luyện mạng, để huấn luyện mạng thần kinh phân loại mẫu tín hiệu vào cần chọn $K \gg M$, với M là số nhóm dữ liệu cần phân loại.

7- Số nút ẩn

Số lượng tế bào thần kinh ở lớp ẩn ảnh hưởng rất lớn đến tính hội tụ của thuật toán lan truyền ngược và chất lượng của mạng. Thường số lượng tế bào thần kinh ở lớp ẩn được chọn dựa vào kinh nghiệm. Nên bắt đầu với mạng có ít tế bào ở lớp ẩn, nếu thuật toán không hội tụ thì tăng dần số tế bào đến khi thuật toán hội tụ.

Mạng truyền thẳng với thuật toán huấn luyện lan truyền ngược đã được ứng dụng để giải nhiều bài toán. Tuy nhiên, bên cạnh những thành công thực tế, thuật toán lan truyền ngược có khuyết điểm là khó huấn luyện, chậm hội tụ. Mặc dù việc tối ưu hóa hệ số học và hệ số moment đã cải thiện được chất lượng của mạng lan truyền ngược nhưng nó vẫn còn kém hiệu quả và tốn nhiều thời gian, do đó không phù hợp để giải một số bài toán thời gian thực.

3.6 MẠNG HÀM CƠ SỞ XUYỀN TÂM

3.6.1 Cấu trúc mạng hàm cơ sở xuyến tâm

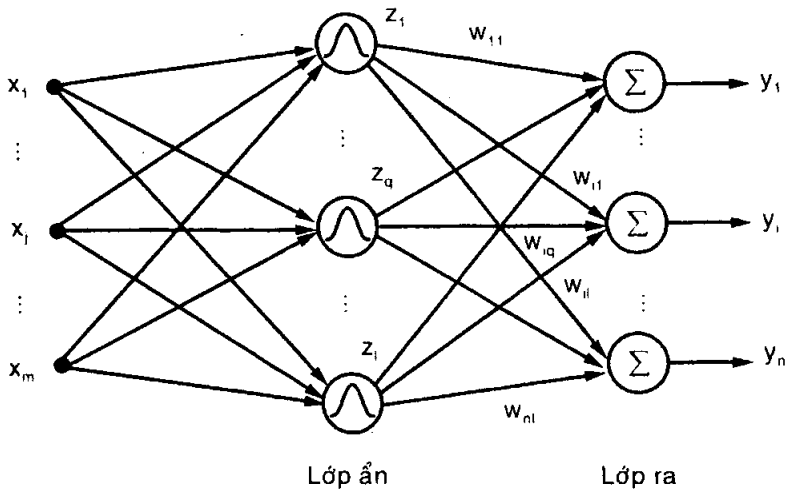
(Radial Basis Function Networks - RBF)

Mạng RBF là một trường hợp đặc biệt của mạng truyền thẳng hai lớp (H.3.23), trong đó:

- Hàm tổng ngõ vào của tế bào thần kinh ở lớp ẩn là hàm cầu (3.5).
- Hàm kích hoạt của tế bào thần kinh ở lớp ẩn là hàm mũ:

$$a_o(f) = e^{-f} \quad (3.69)$$

- Hàm tổng ngõ vào của tế bào thần kinh ở lớp ra là hàm tuyến tính (3.3).
- Hàm kích hoạt của tế bào thần kinh ở lớp ra là tuyến tính (3.9).
- Mức ngưỡng của các tế bào thần kinh bằng 0.



Hình 3.23 Mạng hàm cơ sở xuyến tâm

Do đó áp dụng công thức tính quan hệ vào ra của mạng truyền thẳng, ngõ ra của tế bào thần kinh thứ i của mạng RBF có m ngõ vào, n ngõ ra và l tế bào thần kinh ở lớp ẩn được tính theo như sau:

$$y_i = \sum_{q=1}^l w_{iq} z_q = \sum_{q=1}^l w_{iq} e^{-\frac{\|x - \mu_q\|^2}{2\sigma_q^2}} \quad (3.70)$$

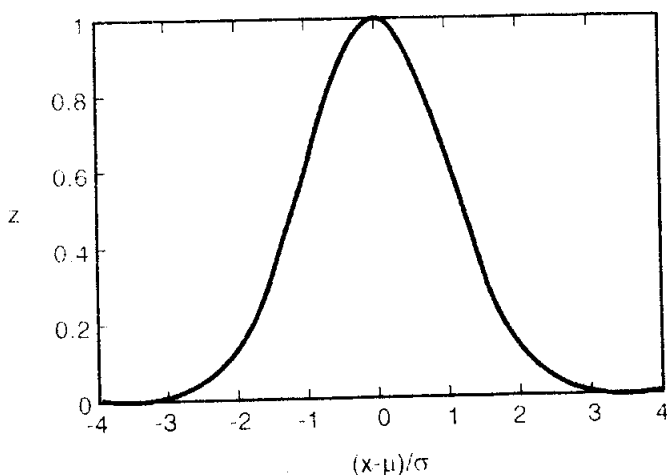
$$\|x - \mu_q\|^2 = (x - \mu_q)^T (x - \mu_q) \quad (3.71)$$

trong đó $\mu_q \in \mathfrak{R}^m$ là tâm của hàm cơ sở thứ q (trọng số của lớp ẩn)

$\sigma_q \in \mathfrak{R}$ là độ phân tán của hàm cơ sở thứ q

$w_i \in \mathfrak{R}^l$ là trọng số ở lớp ra (của ngõ ra thứ i).

Hình 3.24 là ngõ ra của tế bào thần kinh ở lớp ẩn của mạng RBF, ta thấy vector tín hiệu vào càng gần tâm của hàm cơ sở thì ngõ ra của tế bào thần kinh ở lớp ẩn càng lớn và ngược lại. Do ngõ ra của mạng là tổng có trọng số của các ngõ ra của các tế bào thần kinh ở lớp ẩn nên chỉ những vector tín hiệu vào nằm trong lân cận quanh tâm của hàm cơ sở mới ảnh hưởng đến ngõ ra của mạng. Nói cách khác, hàm cơ sở xuyên tâm có tính cục bộ.



Hình 3.24 Ngõ ra của tế bào thần kinh ở lớp ẩn của mạng RBF

3.6.2 Thuật toán huấn luyện mạng RBF

Do mạng RBF là một trường hợp đặc biệt của mạng truyền thẳng nên hoàn toàn có thể áp dụng thuật toán lan truyền ngược để huấn luyện mạng RBF. Tuy nhiên, do đặc điểm hàm cơ sở xuyên tâm có tính cục bộ nên thông thường mạng RBF được huấn luyện qua hai bước:

Bước 1: Xác định tâm và độ phân tán của các hàm cơ sở

- Xác định tâm μ_q ($q = \overline{1..l}$) của các hàm cơ sở: cách phổ biến nhất là phân các vector tín hiệu vào trong tập dữ liệu huấn luyện mạng thành q nhóm, sau đó tính tâm μ_q chính là tâm của mỗi nhóm.

- Xác định độ phân tán σ_q ($q = \overline{1, l}$) của các hàm cơ sở: giá trị độ phân tán σ_q có ảnh hưởng rất lớn đến chất lượng của mạng. Có nhiều cách khác nhau để xác định giá trị này, cách phổ biến nhất là dựa trên sự tương đồng giữa các nhóm dữ liệu vào. Đối với mỗi tế bào thần kinh ở lớp ẩn, khoảng cách từ tâm của nó đến tế bào thần kinh khác gần nhất chính là giá trị độ phân tán σ_q .

Bước 2: Huấn luyện trọng số ở lớp ra của mạng.

Do tín hiệu ra của mạng RBF tuyến tính theo trọng số của lớp ra nên trọng số lớp ra có thể tính được dễ dàng bằng cách sử dụng thuật toán bình phương tối thiểu tuyến tính.

$$w_i = \left[\sum_{k=1}^K z(k) z^T(k) \right]^{-1} \left[\sum_{k=1}^K z(k) d_i(k) \right] \quad (3.72)$$

trong đó:
$$z(k) = [z_1(k) \ z_2(k) \ \dots \ z_l(k)]^T \quad (3.73)$$

$$z_q(k) = e^{-\frac{\|x(k) - \mu_q\|^2}{2\sigma_q}} \quad (3.74)$$

(K là số mẫu dữ liệu huấn luyện mạng)

Ngoài ra, cũng có thể huấn luyện trọng số của các tế bào thần kinh ở lớp ra dùng thuật toán lặp như sau:

Thuật toán 3.5: Thuật toán huấn luyện trọng số lớp ra của mạng RBF

Bước 1: Chọn tốc độ học $\eta > 0$, chọn sai số cực đại E_{max} .

Bước 2: Khởi động:

- Gán sai số $E = 0$.
- Gán biến chạy $k = 1$.
- Gán các trọng số $w_{iq}(k)$, ($i = \overline{1, n}$; $q = \overline{1, l}$) bằng giá trị ngẫu nhiên bất kỳ.

Bước 3: Tính ngõ ra của mạng với tín hiệu vào là $x(k)$:

$$z_q(k) = e^{-\frac{\|x(k) - \mu_q\|^2}{2\sigma_q}}$$

$$y_i(k) = \sum_{q=1}^l w_{iq}(k) z_q(k) \quad (i = \overline{1, n})$$

Bước 4: Cập nhật trọng số của lớp ra của mạng:

$$w_{iq}(k+1) = w_{iq}(k) + \eta(d_i(k) - y_i(k))z_q(k) \quad (i = \overline{1, n}; \quad q = \overline{1, l})$$

Bước 5: Tính sai số tích lũy:

$$E = E + \frac{1}{2} \sum_{i=1}^n (d_i(k) - y_i(k))^2$$

Bước 6: Nếu $k < K$ thì gán $k = k+1$ và trở lại **bước 3**.

Nếu $k = K$ thì tiếp tục **bước 7**.

Bước 7: Kết thúc một chu kỳ huấn luyện.

- Nếu $E < E_{max}$ thì kết thúc quá trình học.
- Nếu $E \geq E_{max}$ thì gán $E = 0$, $k = 1$ và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

Nhận xét: Mặc dù mạng RBF và mạng BP đều có cấu trúc là mạng truyền thẳng nhưng chất lượng của mạng RBF lại tốt hơn mạng BP do hai loại mạng này có một số khác biệt. Khác biệt thứ nhất là giá trị khởi động của các trọng số. Đối với mạng BP trọng số thường được khởi động ngẫu nhiên, trong khi đó trọng số ban đầu của tế bào thần kinh ở lớp ẩn của mạng RBF (tâm của hàm cơ sở) có thể được chọn dựa trên sự phân nhóm dữ liệu vào. Khác biệt thứ hai là thuật toán huấn luyện mạng. Quá trình huấn luyện mạng BP chỉnh định tất cả các trọng số ở lớp ẩn và lớp ra, trong khi quá trình huấn luyện mạng RBF chủ yếu chỉnh định trọng số tuyến tính ở lớp ra.

Mạng RBF có khả năng xấp xỉ hàm rất tốt và có thể huấn luyện dễ dàng, nhanh chóng. Ngoài ra, mạng RBF không gặp vấn đề liên quan đến cực trị cục bộ khi huấn luyện như mạng BP. Một trong những ưu điểm của mạng RBF là các trọng số tuyến tính ở lớp ra có thể xem như độc lập với các tế bào thần kinh ở lớp ẩn. Sau khi trọng số ở lớp ẩn đã được huấn luyện dùng thuật toán tối ưu hóa phi tuyến, trọng số của lớp ra được huấn luyện dùng thuật toán tối ưu hóa tuyến tính. Khả năng xấp xỉ chính xác và tốc độ của mạng RBF còn có thể cải thiện hơn nữa bằng việc chọn tâm và độ phân tán phù hợp cho các hàm cơ sở ở lớp ẩn. Tuy nhiên, RBF thường đáp ứng chậm trong giai đoạn nhớ do lượng lớn tế bào thần kinh ở lớp ẩn.

3.7 MẠNG THẦN KINH SUY DIỄN MỜ THÍCH NGHI

3.7.1 Cấu trúc mạng thần kinh suy diễn mờ thích nghi

(Adaptive Neuro Fuzzy Inference System - ANFIS)

ANFIS là một mở rộng của mô hình mờ Takagi–Sugeno cho phép hệ mờ có thể học thông số dùng thuật toán học lan truyền ngược thích nghi. Tổng quát ANFIS phức tạp hơn nhiều so với hệ suy diễn mờ. Để đơn giản, ta xét hệ mờ có hai ngõ vào là x_1 và x_2 và một ngõ ra y . Giả sử hệ mờ Takagi–Sugeno gồm hai quy tắc:

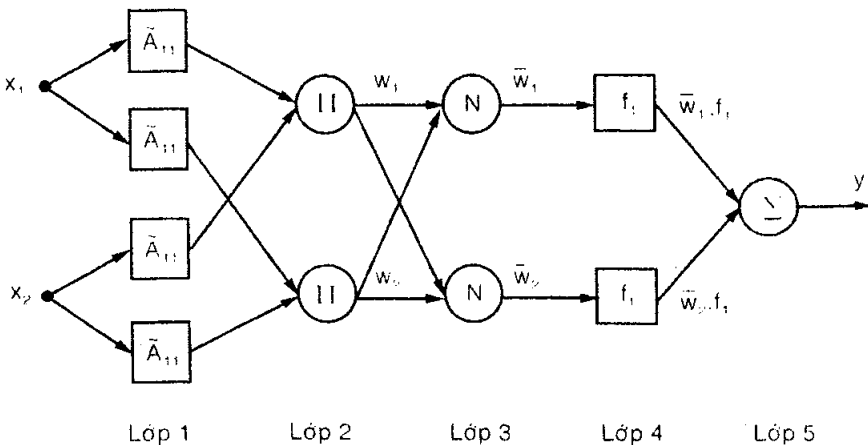
Nếu x_1 là $\tilde{A}_{11}$ và x_2 là $\tilde{A}_{21}$ thì $y = f_1 = b_{01} + b_{11}x_1 + b_{21}x_2$

Nếu x_1 là $\tilde{A}_{12}$ và x_2 là $\tilde{A}_{22}$ thì $y = f_2 = b_{02} + b_{12}x_1 + b_{22}x_2$ (3.75)

Để thực thi hệ mờ Takagi–Sugeno ở trên ta có thể sử dụng cấu trúc mạng ANFIS như hình 3.25. Mạng ANFIS gồm năm lớp, chức năng của các nút trong mỗi lớp như sau:

Lớp 1: Mỗi nút là một hàm liên thuộc của tập mờ biểu diễn giá trị ngôn ngữ liên quan đến các biến vào. Bất cứ dạng hàm liên thuộc nào liên tục và khả vi từng đoạn đều có thể sử dụng được như dạng tam giác, hình thang, gauss, sigmoid, chuông... (xem biểu thức các hàm liên thuộc ở chương 2). Các hàm liên thuộc ở lớp 1 có thông số thay đổi được, các thông số này gọi là thông số ở mệnh đề điều kiện.

$$O_i^1 = \mu_{\tilde{A}}(x) \quad (3.76)$$



Hình 3.25 Mạng ANFIS thực thi hệ mờ Takagi–Sugeno

Lớp 2: Mỗi nút ở lớp 2 được ký hiệu là một hình tròn có dấu Π (tích), ngõ ra của nút ở lớp 2 là tích của tất cả các ngõ vào, ví dụ ở hình 3.25.

$$O_i^2 = w_i = \mu_{\tilde{A}_{1i}}(x_1) \cdot \mu_{\tilde{A}_{2i}}(x_2) \quad (i = 1, 2) \quad (3.77)$$

Ngõ ra của mỗi nút chính là độ đúng của mệnh đề điều kiện của mỗi quy tắc.

Lớp 3: Mỗi nút ở lớp 3 được ký hiệu là một hình tròn có dấu N (Normalize—chuẩn hóa), ngõ ra của một nút ở lớp 3 độ đúng chuẩn hóa của mỗi quy tắc:

$$O_i^3 = \bar{w}_i = \frac{w_i}{w_1 + w_2} \quad (i = 1, 2) \quad (3.78)$$

Lớp 4: Chức năng của nút ở lớp 4 là tính ngõ ra của mỗi quy tắc:

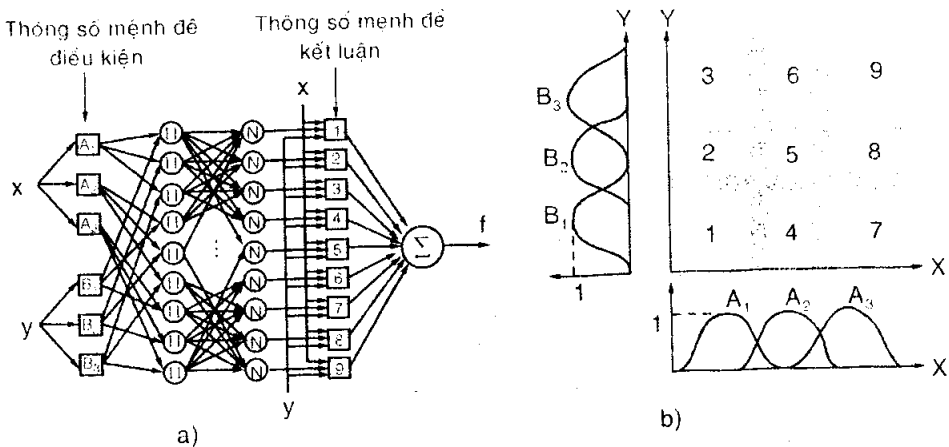
$$O_i^4 = \bar{w}_i f_i = \bar{w}_i (b_{0i} + b_{1i}x_1 + b_{2i}x_2) \quad (3.79)$$

trong đó $\bar{w}_i$ là ngõ ra của lớp 3. Các thông số của hàm f_i có thể chỉnh định và được gọi là thông số ở mệnh đề kết luận.

Lớp 5: Ngõ ra của nút ở lớp 5 là tổng của các ngõ vào nút (ngõ ra của lớp 4):

$$O_i^5 = y = \sum_i \bar{w}_i f_i \quad (3.80)$$

Rõ ràng công thức (3.80) đồng nhất với công thức tính ngõ ra của hệ mờ Takagi–Sugeno đã trình bày ở chương 2.



Hình 3.26 a) Mạng ANFIS chín quy tắc; b) Không gian con mờ tương ứng

Trên đây ta đã xây dựng được cấu trúc ANFIS để thực thi một hệ suy diễn mờ Takagi–Sugeno đơn giản. Các hệ mờ Takagi–Sugeno khác phức tạp hơn cũng đều có thể thực thi dùng cấu trúc ANFIS tương tự như đã trình bày. Hình 3.26 là mạng ANFIS thực thi hệ mờ hai ngõ

vào, chín quy tắc. Có ba hàm liên thuộc liên quan đến mỗi biến vào nên không gian tín hiệu vào chia làm chín không gian mờ con, mỗi không gian mờ con chịu sự chi phối của một quy tắc mờ. Mệnh đề điều kiện của các quy tắc xác định các không gian mờ con, trong khi mệnh đề kết luận xác định ngõ ra trong không gian mờ con tương ứng.

3.7.2 Thuật toán lai huấn luyện mạng ANFIS

Có thể áp dụng thuật toán suy giảm độ dốc để huấn luyện mạng ANFIS, tuy nhiên thuật toán suy giảm độ dốc có khuyết điểm là chậm hội tụ và lời giải có thể rơi vào cực trị cục bộ. Một thuật toán khác hiệu quả hơn để huấn luyện mạng ANFIS là thuật toán học lai như trình bày dưới đây.

Trước tiên để ý rằng ngõ ra của mạng ANFIS tuyến tính theo thông số ở mệnh đề kết luận và phi tuyến theo thông số ở mệnh đề điều kiện. Ví dụ, ngõ ra của mạng ANFIS ở hình 3.25 được tính bởi biểu thức:

$$y = \bar{w}_1 b_{01} + \bar{w}_1 b_{11} x_1 + \bar{w}_1 b_{21} x_2 + \bar{w}_2 b_{02} + \bar{w}_2 b_{12} x_2 + \bar{w}_2 b_{22} x_2 \quad (3.81)$$

Rõ ràng y tuyến tính theo các thông số $(b_{01}, b_{11}, b_{21}, b_{02}, b_{12}, b_{22})$ ở mệnh đề kết luận. Gọi θ_L , θ_N là lần lượt hai vector cột chứa tất cả các thông số tuyến tính và phi tuyến, tổng quát ngõ ra của mạng ANFIS có thể viết dưới dạng:

$$y = F(x, \theta_N, \theta_L) = H_\star(x, \theta_N) \theta_L \quad (3.82)$$

Thuật toán lai huấn luyện mạng ANFIS là sự kết hợp thuật toán suy giảm độ dốc để cập nhật các thông số phi tuyến θ_N và thuật toán bình phương tối thiểu tuyến tính để ước lượng thông số tuyến tính θ_L . Thuật toán lai huấn luyện mạng ANFIS được tóm tắt như sau:

Thuật toán 3.6: Thuật toán lai huấn luyện mạng ANFIS

Cho tập dữ liệu vào–ra dùng để huấn luyện gồm K mẫu

$$(x(1), d(1)), (x(2), d(2)), \dots, (x(K), d(K)) \quad (3.83)$$

Bước 1: Chọn tốc độ học $\eta > 0$, chọn sai số cực đại E_{max} .

Bước 2: Khởi động:

- Gán sai số $E = 0$

- Gán các giá trị đầu cho thông số phi tuyến $\theta_N(0)$

Bước 3: Ước lượng thông số tuyến tính dùng thuật toán bình phương tối thiểu:

$$\sum_{k=1}^K [d(k) - H(x(k), \theta_N(0))\theta_L]^2 \rightarrow \min$$

$$\Rightarrow \theta_L = \left(\sum_{k=1}^K H^T(x(k), \theta_N(0)) \cdot H(x(k), \theta_N(0)) \right)^{-1} \left(\sum_{k=1}^K H^T(x(k), \theta_N(0)) \cdot d(k) \right)$$

Bước 4: Cập nhật trọng số phi tuyến dùng thuật toán suy giảm độ dốc:

Với $k = 1 : K$

- Tính sai số:

$$E(k) = \frac{1}{2} [d(k) - H(x(k), \theta_N)\theta_L]^2$$

- Cập nhật thông số:

$$\theta_N(k) = \theta_N(k-1) + \eta \frac{\partial E(k)}{\partial \theta_N}$$

- Tính sai số tích lũy:

$$E = E + E(k)$$

Bước 5: Kết thúc một chu kỳ huấn luyện.

- Nếu $E < E_{max}$ thì kết thúc quá trình học.

- Nếu $E \geq E_{max}$ thì gán $E = 0$, và trở lại **bước 3** bắt đầu một chu kỳ huấn luyện mới.

3.8 TÓM TẮT

Chương 3 đã trình bày mô hình toán học đơn giản của tế bào thần kinh và các cấu hình mạng. Mạng truyền thẳng nhiều lớp MLP, mạng hàm cơ sở xuyên tâm RBF, mạng suy diễn mờ thích nghi ANFIS với các thuật toán huấn luyện tương ứng đều là có tính chất xấp xỉ tổng quát, có khả năng xấp xỉ hàm phi tuyến bất kỳ với sai số bé tùy ý. Nhờ tính chất này mà mạng thần kinh có thể sử dụng rất hiệu quả trong các thuật toán nhận dạng hệ thống, điều khiển dựa vào mô hình hoặc điều khiển thích nghi. Ứng dụng mạng thần kinh trong lĩnh vực điều khiển sẽ được trình bày trong các chương sau.

Chương 4

GIẢI THUẬT DI TRUYỀN

4.1 GIỚI THIỆU

4.1.1 Sự tiến hóa trong tự nhiên

Theo thuyết tiến hóa (*Darwin*) và quy luật di truyền (*Mendel*), sinh vật trong tự nhiên tồn tại và phát triển từ thế hệ (*generation*) này sang thế hệ khác thông qua quá trình **chọn lọc tự nhiên** (*natural selection*), **lai ghép** (*crossover*) và **đột biến** (*mutation*).

Chọn lọc tự nhiên

Các *quần thể* (*population*) trong tự nhiên gồm nhiều cá thể (*individual*), chỉ những cá thể thích nghi nhất với môi trường sống mới tồn tại trong cuộc đấu tranh sinh tồn, ngược lại những cá thể không thích nghi với môi trường sống sẽ bị đào thải.

Lai ghép

Đơn vị di truyền cấp tế bào là *nhễm sắc thể* (*chromosome*), mỗi loài có số lượng cặp nhiễm sắc thể riêng. Nhiễm sắc thể (NST) gồm nhiều *gien* (*gene*). Trong quá trình lai ghép, NST của bố mẹ bị phân chia và tạo nên NST của con. NST của con gồm một số gien của bố và một số gien của mẹ. Thông qua quá trình sinh sản (lai ghép) những tính trạng tốt được truyền từ thế hệ này sang thế hệ khác.

Đột biến

Gien của cá thể bị thay đổi một cách ngẫu nhiên do có “lỗi” trong quá trình di truyền. Xác suất xảy ra đột biến trong tự nhiên thường rất thấp.

Qua quá trình tiến hóa, thế hệ sau luôn có xu hướng thích nghi với môi trường sống tốt hơn thế hệ trước.

4.1.2 Giải thuật di truyền

Giải thuật di truyền (Gientic Algorithm – GA) được Holland đưa ra vào năm 1975 là giải thuật tìm kiếm lời giải tối ưu trên nguyên tắc phỏng theo quá trình tiến hóa và quy luật di truyền của sinh vật trong tự nhiên. Bản chất toán học của GA là giải thuật tìm kiếm theo xác suất.

GA cũng như các giải thuật tiến hóa nói chung dựa trên quan điểm cho rằng, quá trình tiến hóa của tự nhiên là quá trình hoàn hảo nhất, hợp lý nhất và tự nó đã mang tính tối ưu. Quan niệm này có thể xem như là một tiên đề đúng không chứng minh được nhưng phù hợp với thực tế khách quan. *Quá trình tiến hóa thể hiện tính tối ưu ở chỗ, thế hệ sau bao giờ cũng tốt hơn (phát triển hơn, hoàn thiện hơn) thế hệ trước.* Tiến hóa tự nhiên được duy trì nhờ hai quá trình cơ bản: ***sinh sản và chọn lọc tự nhiên***, các thế hệ mới luôn được sản sinh ra để bổ sung, thay thế thế hệ cũ. Cá thể nào phát triển hơn, thích ứng hơn với môi trường sẽ có nhiều khả năng tồn tại và phát triển, cá thể nào không thích ứng được với môi trường sẽ bị đào thải. Sự thay đổi của môi trường là động lực của quá trình tiến hóa. Ngược lại, tiến hóa cũng tác động trở lại góp phần làm thay đổi môi trường.

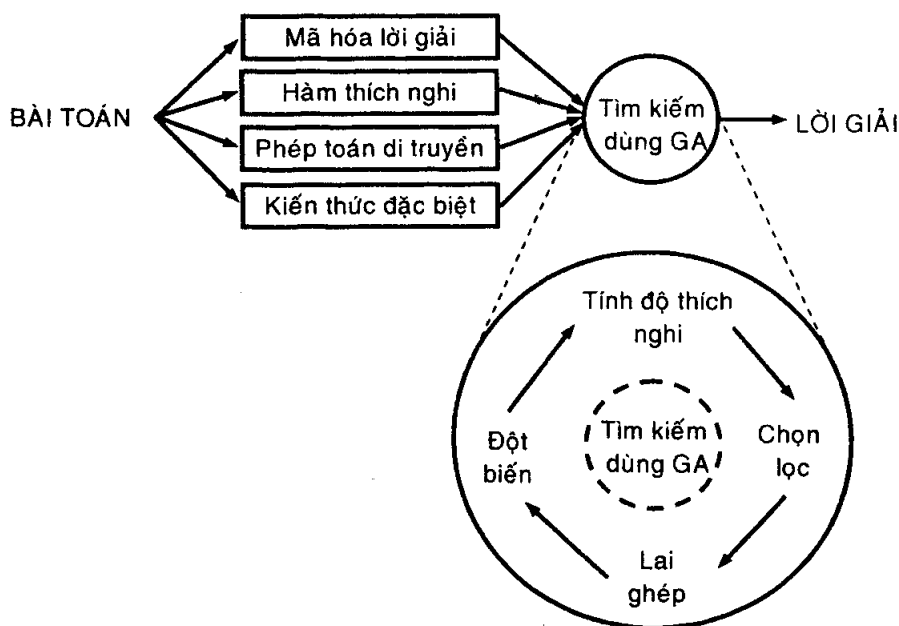
Các cá thể mới sinh ra trong quá trình tiến hóa nhờ sự lai ghép ở thế hệ cha-mẹ. Một cá thể mới có thể mang những tính trạng của cha-mẹ (*di truyền*), cũng có thể mang những tính trạng hoàn toàn mới (*đột biến*). Di truyền và đột biến là hai cơ chế có vai trò quan trọng như nhau trong tiến trình tiến hóa, dù rằng hiện tượng đột biến diễn ra với xác suất nhỏ hơn nhiều so với di truyền. GA nói riêng và các giải thuật tiến hóa nói chung tuy có điểm khác biệt nhưng đều mô phỏng ba quá trình cơ bản của tiến hóa tự nhiên: ***chọn lọc tự nhiên, lai ghép (sinh sản) và đột biến.***

GA là phương pháp tối ưu hóa ngẫu nhiên tổng quát dùng để giải các bài toán tìm kiếm. GA có một số khác biệt so với giải thuật tối ưu hóa và tìm kiếm thông thường. Thứ nhất, GA làm việc với quần thể gồm nhiều NST (tập hợp nhiều lời giải), tìm kiếm nhiều điểm cực trị cùng một lúc. Bằng các phép toán di truyền, GA trao đổi thông tin giữa những đỉnh cực trị, do đó hạn chế khả năng rơi vào cực trị cục bộ. Nói cách khác GA là phương pháp tìm kiếm lời giải tối ưu toàn cục. Thứ hai, GA làm việc với chuỗi ký hiệu (chuỗi NST) chứ không làm việc với các thông số. Thứ ba, GA chỉ cần đánh giá hàm mục tiêu để định hướng quá trình tìm kiếm mà không cần bất cứ

thông tin gì khác (như đạo hàm...). Do các đặc điểm trên, GA có khả năng giải quyết rất hiệu quả các bài toán tìm cực trị hàm phi tuyến, không khả vi, không liên tục...

4.1.3 Ứng dụng của giải thuật di truyền

GA là thuật toán tổng quát có khả năng giải quyết thành công các bài toán ở nhiều lĩnh vực, đặc biệt là những bài toán khó giải quyết bằng các phương pháp khác. Một bài toán được giải bằng GA được mô tả trong hình 4.1. GA không đảm bảo rằng sẽ tìm được lời giải tối ưu toàn cục cho bài toán, nhưng thông thường GA có thể tìm được lời giải “đủ tốt” trong thời gian “chấp nhận được”. Khi tồn tại một phương pháp cụ thể để giải bài toán thì phương pháp đó thường tốt hơn GA cả về tốc độ tìm lời giải và độ chính xác của kết quả. Vì vậy, phạm vi ứng dụng chính của GA là các bài toán khó không tồn tại phương pháp giải khác hoặc phương pháp giải khác chưa hiệu quả.

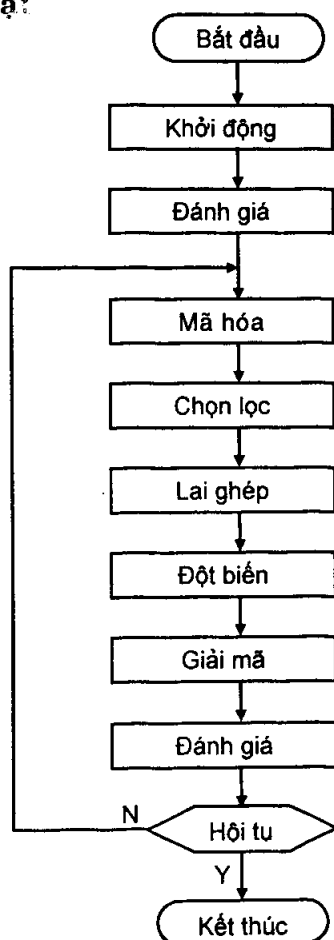


Hình 4.1 Giải bài toán dùng GA

Những năm gần đây, các ứng dụng của GA tăng mạnh trong nhiều lĩnh vực như: kỹ thuật (thiết kế động cơ, thiết kế máy bay...), tối ưu hóa, hoạch định hoạt động robot, hệ thống phân loại, hệ thống học, nhận dạng mẫu, huấn luyện mạng thần kinh, chỉnh định hệ mờ...

4.2 GIẢI THUẬT DI TRUYỀN CƠ BẢN

4.2.1 Lưu đồ giải thuật



Hình 4.2 Lưu đồ giải thuật di truyền

Lưu đồ GA dùng để giải bài toán tối ưu trong điều khiển được trình bày ở hình 4.2. Để áp dụng GA giải bài toán tối ưu trước hết phải **mã hóa** (*encode*) lời giải của bài toán thành chuỗi NST. Tùy theo phương pháp mã hóa mà chuỗi NST có thể là chuỗi số nhị phân, chuỗi số thập phân, chuỗi số tự nhiên hay chuỗi số thực. Mỗi NST đại diện cho một cá thể trong quần thể. Để đánh giá các cá thể phải định nghĩa một **hàm thích nghi** (*fitness function*), thường là hàm cần tìm cực trị hoặc một biến đổi tương đương của hàm cần tìm cực trị. Thế hệ đầu tiên gồm nhiều cá thể (lời giải) được khởi động ngẫu nhiên. Qua quá trình chọn lọc tự nhiên những cá thể thích nghi nhất với

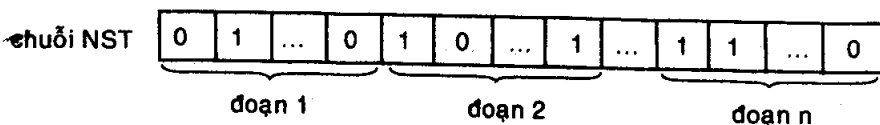
môi trường sống (có độ thích nghi cao) mới tồn tại và có cơ hội sinh sản để tạo ra thế hệ con có xu hướng thích nghi với môi trường sống tốt hơn thế hệ bố mẹ. Cá thể nào có độ thích nghi càng cao thì càng có nhiều cơ hội để tồn tại và bắt cặp với một cá thể khác để sinh ra thế hệ con. Nhờ quá trình lai ghép mà các đặc tính tốt của thế hệ trước được truyền lại cho thế hệ sau. Đôi khi do lỗi trong quá trình di truyền trong tự nhiên xảy ra hiện tượng đột biến với xác suất rất thấp. Hiện tượng đột biến có thể tạo ra những cá thể có độ thích nghi rất kém nhưng cũng có thể tạo ra những cá thể có độ thích nghi rất tốt, vượt trội hơn hẳn so với các cá thể còn lại trong quần thể. Chọn lọc tự nhiên sẽ loại bỏ những cá thể kém thích nghi, kết quả là qua quá trình tiến hóa được lặp lại từ thế hệ này sang thế hệ khác, thế hệ sau có xu hướng thích nghi với môi trường sống tốt hơn thế hệ trước, điều đó có nghĩa là lời giải của bài toán tiến dần đến lời giải tối ưu.

4.2.2 Mã hóa - Giải mã

Mã hóa là biểu diễn lời giải của bài toán cần giải quyết thành chuỗi NST. Có rất nhiều cách mã hóa khác nhau, tùy thuộc vào bài toán cụ thể. Trong lĩnh vực điều khiển ta thường tìm cực trị của hàm $J(\theta)$, trong đó $\theta = [\theta_1, \theta_2, \dots, \theta_d]^T$ là vector thông số. Mỗi lời giải của bài toán tìm cực trị được mã hóa thành một chuỗi NST, mỗi thông số θ_i sẽ được mã hóa thành một đoạn gen trên chuỗi NST. Trong tự nhiên, mỗi cá thể gồm nhiều NST. Trong các giải thuật di truyền, thường mỗi cá thể chỉ có một NST, vì vậy các thuật ngữ cá thể (*individual*) và nhiễm sắc thể (*chromosome*) thường đồng nghĩa nhau trong đa số trường hợp.

Có ba cách thường dùng để mã hóa bài toán tìm cực trị hàm là mã hóa nhị phân, mã hóa thập phân và mã hóa số thực. Phần này trình bày cách mã hóa nhị phân và thập phân, cách mã hóa số thực và các phép toán di truyền liên quan sẽ được trình bày ở mục 4.3.

1- Mã hóa nhị phân



Hình 4.3 Mã hóa nhị phân

Đây là phương pháp mã hóa cơ bản được Holland đưa ra để giải các bài toán tối ưu hóa hàm. Bộ gen dùng để mã hóa lời giải chỉ gồm có hai ký hiệu $\{0;1\}$. Biến thứ i được mã hóa thành một chuỗi nhị phân có độ dài là L_i (H.4.3), như vậy tổng số giá trị có thể có của biến thứ i là 2^{L_i} . Cận dưới $\theta_{i\min}$ của biến thứ i được biểu diễn bằng chuỗi (00...0), cận trên $\theta_{i\max}$ được biểu diễn bằng chuỗi (11...1). Một chuỗi NST s_i bất kỳ được giải mã thành lời giải θ_i như sau:

$$\theta_i = \theta_{i\min} + \frac{\theta_{i\max} - \theta_{i\min}}{2^{L_i} - 1} DV(s_i) \quad (4.1)$$

trong đó $DV(s_i)$ là giá trị của chuỗi nhị phân s_i .

Nếu $s_i = (s_{i,L_i-1}, \dots, s_{i2}, s_{i1}, s_{i0})$ thì $DV(s_i)$ được tính như sau:

$$DV(s_i) = \sum_{j=0}^{L_i-1} 2^j \cdot s_{ij} \quad (4.2)$$

Chiều dài của chuỗi nhị phân được xác định bởi độ chính xác mong muốn tương ứng với mỗi biến. Nếu độ chính xác mong muốn của biến θ_i là ε_i thì độ dài của đoạn gen nhị phân biểu diễn biến θ_i được tính như sau:

$$L_i = \log_2 \left(\frac{\theta_{i\max} - \theta_{i\min}}{\varepsilon_i} \right) \quad (4.3)$$

Chiều dài chuỗi NST biểu diễn lời giải của bài toán bằng tổng chiều dài của các chuỗi nhị phân mã hóa các biến:

$$L = \sum_{i=1}^n L_i$$

Phương pháp mã hóa nhị phân có ưu điểm là đơn giản, các phép toán di truyền cơ bản đều định nghĩa cho chuỗi NST mã hóa theo cách này. Khuyết điểm của cách mã hóa nhị phân là nếu muốn lời giải có độ chính xác cao phải dùng chuỗi NST rất dài để mã hóa, trong trường hợp này GA chạy rất chậm.

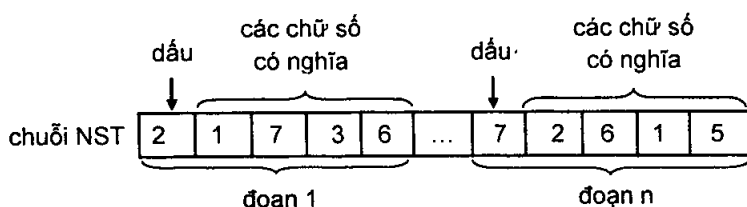
Ví dụ 4.1 Giả sử miền tìm kiếm giá trị tối ưu của biến θ_i là $\theta_{i\min} = -1,5$, $\theta_{i\max} = 4,6$, độ chính xác mong muốn của lời giải là $\varepsilon_i = 0,1$ thì độ dài của chuỗi nhị phân biểu diễn biến θ_i là:

$$L_i = \log_2 \left(\frac{4,6 - (-1,5)}{0,1} \right) = 5,93$$

Do số gien phải là số nguyên nên chọn $L_i = 6$. Chuỗi (000000) biểu diễn giá trị $-1,5$, chuỗi (111111) biểu diễn giá trị $4,6$. Theo công thức (4.1), chuỗi bất kỳ, chẳng hạn $s_i = (101110)$ biểu diễn giá trị:

$$\theta_i = -1,5 + \frac{4,5 - (-1,5)}{2^6 - 1} \cdot (2^5 + 2^3 + 2^2 + 2^1) = 2,8571$$

2- Mã hóa thập phân



Hình 4.4 Mã hóa thập phân

Bộ gien dùng để mã hóa lời giải gồm có mười ký hiệu $\{0; 1; 2; 3; 4; 5; 6; 7; 8; 9\}$. Theo phương pháp này lời giải của bài toán tìm cực trị được mã hóa thành chuỗi nhiễm sắc thể gồm các chữ số thập phân, mỗi chuỗi nhiễm sắc thể gồm nhiều đoạn gien, mỗi đoạn tương ứng với một biến. Mỗi đoạn gien trên chuỗi nhiễm sắc thể gồm một chữ số biểu diễn dấu của thông số và các chữ số có nghĩa. Vị trí dấu chấm thập phân của mỗi đoạn gien cũng được lưu trữ để sử dụng trong bước giải mã. Giá trị gien biểu diễn dấu của thông số giải mã như sau:

$$\begin{cases} \text{Giá trị gien bằng } 0 - 4 : \text{dấu } - \\ \text{Giá trị gien bằng } 5 - 9 : \text{dấu } + \end{cases} \quad (4.4)$$

Cách mã hóa thập phân có hai ưu điểm: thứ nhất độ dài chuỗi nhiễm sắc thể được rút ngắn lại đáng kể so với cách mã hóa nhị phân kinh điển, do đó làm cho thuật toán chạy nhanh hơn; thứ hai có thể áp dụng trực tiếp các phép toán di truyền quen thuộc đã đưa ra cho cách mã hóa nhị phân.

Ví dụ 4.2

- Nếu sử dụng cách mã hóa thập phân với 5 chữ số có nghĩa thì đoạn gen biểu diễn biến $\theta_i = -56,4172$ là $s_i = (056417)$, đồng thời lưu trữ vị trí dấu chấm thập phân là 2.
- Nếu sử dụng cách mã hóa thập phân với 4 chữ số có nghĩa, vị trí dấu chấm thập phân là -1 thì đoạn gen $s_i = (71428)$ sẽ được giải mã thành lời giải $\theta_i = 0,1428$.

4.2.3 Hàm thích nghi

Hàm thích nghi dùng để đánh giá các cá thể, cá thể nào có độ thích nghi tốt hơn sẽ tồn tại qua quá trình chọn lọc tự nhiên và có nhiều cơ hội để lai ghép. Thường hàm thích nghi chính là hàm cần tìm cực trị hoặc biến đổi tương đương của hàm cần tìm cực trị.

Do giải thuật di truyền cơ bản tìm kiếm lời giải làm cực đại hàm thích nghi (qua quá trình tiến hóa giá trị hàm thích nghi của các cá thể trong quần thể tăng dần) nên để áp dụng GA giải bài toán tìm cực trị, ta có thể định nghĩa hàm thích nghi như sau:

- Nếu tìm cực đại của hàm $J(\theta)$:

$$fitness = J(\theta) + C \quad (4.5)$$

- Nếu tìm cực tiểu của hàm $J(\theta)$:

$$fitness = \frac{1}{J(\theta) + C} \quad (4.6)$$

Trong hai công thức trên, hệ số C được cộng thêm vào hàm $J(\theta)$ để đảm bảo giá trị hàm thích nghi luôn dương.

4.2.4 Chọn lọc tự nhiên

Nguyên tắc cơ bản của các phương pháp chọn lọc là NST có độ thích nghi càng cao thì có xác suất chọn lựa càng lớn. Chọn lọc không chỉ xác định cá thể nào đó được tồn tại mà còn xác định số lượng con mà cá thể đó có thể có. Vì vậy phép toán chọn lọc có vai trò rất quan trọng đối với chất lượng của GA.

Phép toán chọn lọc gọi là có cường độ mạnh nếu cá thể thích nghi tốt nhất có xác suất chọn lọc cao hơn nhiều so với cá thể có độ

thích nghi kém nhất. Ngược lại, phép toán chọn lọc gọi là có cường độ yếu nếu xác suất chọn lọc của cá thể thích nghi tốt nhất chỉ hơi lớn hơn so với cá thể có độ thích nghi kém nhất. Nếu phương pháp chọn lọc có cường độ mạnh, các gen của các cá thể có độ thích nghi cao sẽ chiếm ưu thế ở thế hệ kế tiếp mặc dù rằng đây có thể chỉ là lời giải tối ưu cục bộ, hiện tượng này gọi là hội tụ sớm. Ngược lại, nếu phép chọn lọc có cường độ yếu các cá thể kém thích nghi có nhiều cơ hội được tái tạo do đó làm chậm quá trình tiến hóa. Cường độ chọn lọc được định nghĩa bởi biểu thức toán học sau:

$$I = \frac{M^* - M}{\sigma} \quad (4.7)$$

trong đó M và M^* là độ thích nghi trung bình của quần thể trước và sau chọn lọc, σ^2 là phương sai của độ thích nghi trước chọn lọc.

Qua quá trình chọn lọc tự nhiên, một số cá thể có độ thích nghi kém sẽ bị loại bỏ, điều này sẽ làm mất tính đa dạng gen trong quần thể. Độ mất tính đa dạng được đo bằng tỷ lệ cá thể trong quần thể không được chọn lọc. Quần thể càng mất tính đa dạng càng dễ bị hội tụ sớm.

Có nhiều phương pháp thực hiện phép toán chọn lọc đã được đưa ra trong các tài liệu về GA. Để thuận lợi cho việc trình bày các phương pháp chọn lọc, ta định nghĩa các ký hiệu sau:

P : quần thể (*Population*), gồm nhiều cá thể (*Individual*) $P = (I_1, I_2, \dots, I_N)$

I_k : cá thể (nhằm sắc thể) thứ k ($k = \overline{1..N}$, N : số cá thể trong quần thể)

f_k : độ thích nghi của nhằm sắc thể thứ k ($k = \overline{1..N}$).

1- Chọn lọc tỷ lệ (*Proportional Selection, Roulette Wheel Selection*)

Chọn lọc tỷ lệ là phương pháp chọn lọc cơ bản được đưa ra trong những nghiên cứu đầu tiên về GA, xác suất lựa chọn mỗi cá thể theo phương pháp này tỷ lệ với độ thích nghi. Chú ý rằng phương pháp chọn lọc tỷ lệ chỉ áp dụng được để giải các bài toán tìm cực đại và giá trị của hàm tìm cực đại luôn dương. Nếu bài toán cần giải là bài toán tìm cực tiểu hoặc hàm cần tìm cực trị có thể nhận giá trị âm ta phải biến đổi tương đương về bài toán tìm cực đại và hàm cần tìm cực đại chỉ có thể nhận giá trị dương.

Thuật toán 4.1: Chọn lọc tỷ lệ

```

 $P' = \text{chon\_loc\_ty\_le}(P)$ 
{
     $s_0 \leftarrow 0$ ;
    for  $k \leftarrow 1:N$  {
         $s_k \leftarrow s_{k-1} + f_k$ 
    }
    for  $k \leftarrow 1:N$  {
         $r \leftarrow \text{random}[0, s_N]$ ;
         $I'_k = I_j$ , với  $j$  thỏa  $s_{j-1} \leq r < s_j$ 
    }
     $P' \leftarrow (I'_1, I'_2, \dots, I'_N)$ 
}

```

Cường độ chọn lọc tỷ lệ xác định bởi công thức:

$$I \approx \frac{\sigma}{M} \quad (4.8)$$

trong đó M và σ^2 là trung bình và phương sai của độ thích nghi của quần thể trước chọn lọc.

Phương pháp chọn lọc tỷ lệ có khuyết điểm là cường độ chọn lọc rất kém khi $\sigma \approx 0$, tức là khi độ thích nghi của các cá thể trong quần thể xấp xỉ nhau. Do đó GA sử dụng phương pháp chọn lọc tỷ lệ thường chậm hội tụ khi lời giải của bài toán tiến gần đến cực trị.

2- Chọn lọc đấu vòng (Tournament Selection)**Thuật toán 4.2: Chọn lọc đấu vòng**

```

 $P' = \text{chon\_loc\_dau\_vong}(P, t)$ 
{
    for  $k \leftarrow 1:N$  {
        chọn ngẫu nhiên  $t$  nhiễm sắc thể;
         $I'_k = I^*$ , với  $I^*$  là nhiễm sắc thể có độ thích nghi
        tốt nhất trong  $t$  nhiễm sắc thể vừa chọn.
    }
     $P' \leftarrow (I'_1, I'_2, \dots, I'_N)$ 
}

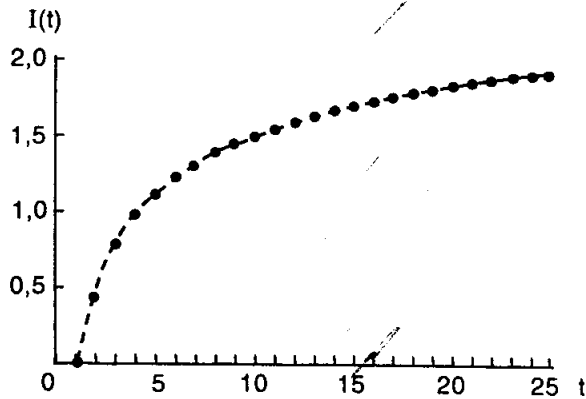
```

Chọn ra t cá thể ngẫu nhiên (t gọi là quy mô mẫu ngẫu nhiên), cá thể nào có độ thích nghi tốt nhất trong t cá thể trên sẽ được chọn để lai ghép. Lặp lại N lần bước trên để chọn đủ N cá thể.

Cường độ chọn lọc phụ thuộc vào quy mô mẫu ngẫu nhiên theo biểu thức:

$$I(t) \approx \sqrt{2(\ln(t) - \ln(\sqrt{4.14 \ln(t)}))} \quad (4.9)$$

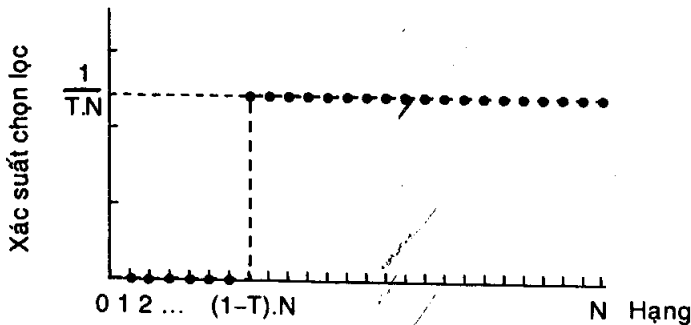
Hình 4.5 biểu diễn quan hệ giữa cường độ chọn lọc và quy mô mẫu ngẫu nhiên, dễ thấy rằng quy mô mẫu ngẫu nhiên càng lớn cường độ chọn lọc càng cao.



Hình 4.5 Cường độ chọn lọc mẫu ngẫu nhiên

3- Chọn lọc cắt (Truncation Selection)

Theo phương pháp chọn lọc cắt với mức ngưỡng T ($T \in [0, 1]$), chỉ có $T.N$ cá thể tốt nhất mới có cơ hội được lựa chọn và xác suất chọn lựa của các cá thể này như nhau, xem hình 4.6.



Hình 4.6 Xác suất chọn lọc cắt

Thuật toán 4.3: Chọn lọc cắt

```

P' = chon_loc_cat (P, T)
{
     $\bar{P} \leftarrow$  sắp xếp P theo thứ tự tăng dần độ thích nghi
    for k  $\leftarrow$  1:N {
        r  $\leftarrow$  random((1-T)N, N)
         $I'_k = I_r$ 
    }
    P'  $\leftarrow$  ( $I'_1, I'_2, \dots, I'_N$ )
}

```

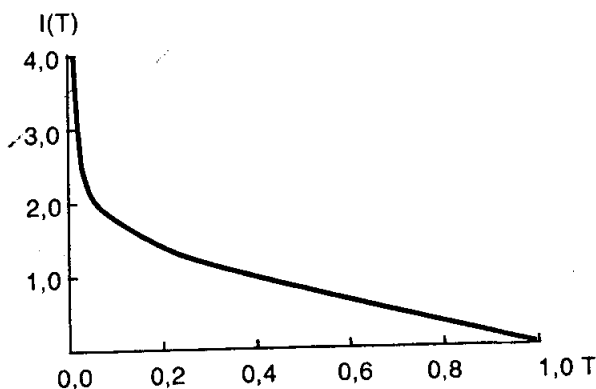
Cường độ chọn lọc cắt:

$$I(T) = \frac{1}{T} \frac{1}{\sqrt{2\pi}} e^{-(f_c^2/2)} \quad (4.10)$$

trong đó f_c xác định bởi biểu thức:

$$T = \int_{f_c}^{\infty} \frac{1}{\sqrt{2\pi}} e^{-(f^2/2)} df \quad (4.11)$$

Hình 4.7 biểu diễn quan hệ giữa cường độ chọn lọc và mức ngưỡng T , theo hình vẽ ta thấy mức ngưỡng càng bé cường độ chọn lọc càng cao.



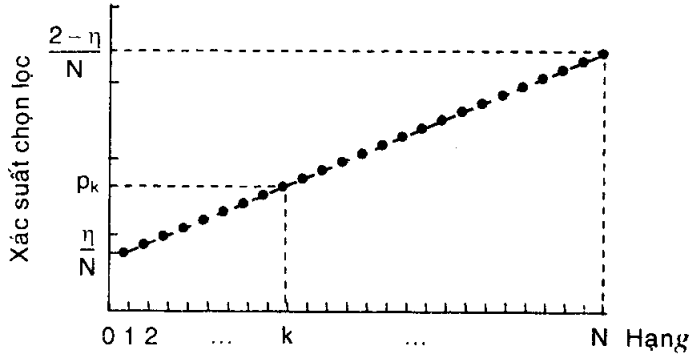
Hình 4.7 Cường độ chọn lọc cắt

4- Chọn lọc sắp hạng tuyến tính (Linear Ranking Selection)

Sắp xếp các cá thể theo thứ tự tăng dần độ thích nghi, và gán cá thể tốt nhất hạng N , cá thể xấu nhất hạng 1. Xác suất chọn lọc của mỗi cá thể tỷ lệ tuyến tính với hạng của nó:

$$p_k = \frac{1}{N} \left[\eta + 2(1 - \eta) \frac{k - 1}{N - 1} \right] \quad (4.12)$$

trong đó $0 < \eta < 1$. Xác suất chọn lọc của cá thể xấu nhất và tốt nhất lần lượt là η/N và $(2 - \eta)/N$.



Hình 4.8 Xác suất chọn lọc sắp hạng tuyến tính

Thuật toán 4.4: Thuật toán chọn lọc sắp hạng tuyến tính

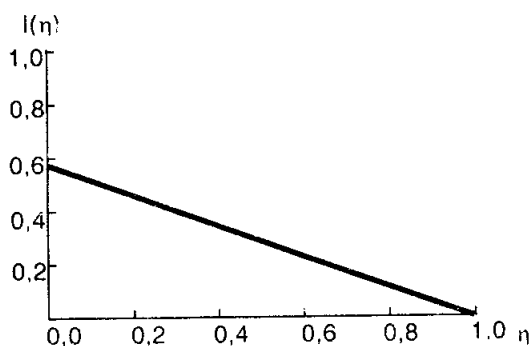
```

P' = chon_loc_sap_hang_tuyen_tinh (P, η)
{
    P̄ ← sắp xếp P theo thứ tự tăng dần độ thích nghi
    s0 ← 0;
    for k ← 1:N {
        sk ← sk-1 + pk  (pk xác định bởi công thức (4.12))
    }
    for k ← 1:N {
        r ← random[0, sN);
        I'k = Ij, với j thỏa sj-1 ≤ r < sj
    }
    P' ← (I'1, I'2, ..., I'N)
}

```

Cường độ chọn lọc sắp hạng tuyến tính giảm khi hệ số η tăng:

$$I(\eta) = \frac{(1 - \eta)}{\sqrt{\pi}} \quad (4.13)$$



Hình 4.9 Cường độ chọn lọc sắp hạng tuyến tính

5- Chọn lọc sắp hạng lũy thừa (Exponential Ranking Selection)

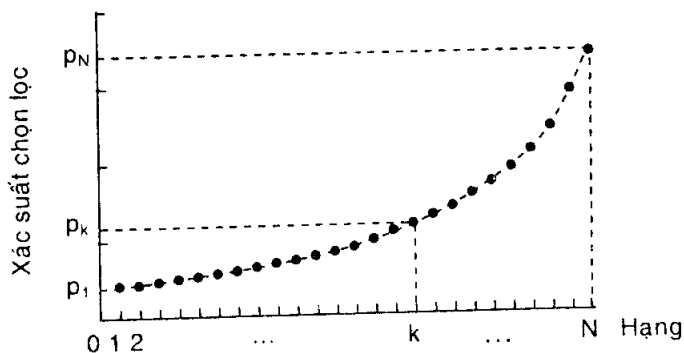
Sắp xếp các cá thể theo thứ tự tăng dần độ thích nghi, và gán cá thể tốt nhất hạng N , cá thể xấu nhất hạng 1. Xác suất chọn lọc của mỗi cá thể tỷ lệ với hạng của nó theo quy luật hàm mũ:

$$p_k = \frac{c^{N-k}}{\sum_{j=1}^N c^{N-j}}; \quad (k = 1..N) \quad (4.14)$$

hay:
$$p_k = \frac{c-1}{c^N-1} c^{N-k}; \quad (k = 1..N) \quad (4.15)$$

trong đó c là hệ số lũy thừa ($0 < c < 1$). Hệ số c càng gần bằng 1, tính chất “chọn lọc lũy thừa” càng mờ nhạt.

Thuật toán chọn lọc sắp hạng lũy thừa hoàn toàn tương tự thuật toán chọn lọc sắp hạng tuyến tính, hai thuật toán chỉ khác nhau ở công thức tính xác suất.



Hình 4.10 Xác suất chọn lọc sắp hạng lũy thừa

Thuật toán 4.5: Chọn lọc sắp hạng lũy thừa

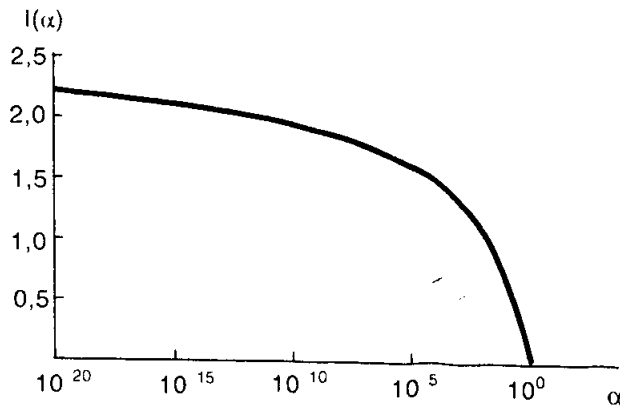
```

 $P' = \text{chon\_loc\_sap\_hang\_luy\_thua}(P, c)$ 
{
     $\bar{P} \leftarrow$  sắp xếp  $P$  theo thứ tự tăng dần độ thích nghi
     $s_0 \leftarrow 0$ ;
    for  $k \leftarrow 1:N$  {
         $s_k \leftarrow s_{k-1} + p_k$  ( $p_k$  xác định bởi công thức (4.15))
    }
    for  $k \leftarrow 1:N$  {
         $r \leftarrow \text{random}[0, s_N]$ ;
         $I'_k = I_j$ , với  $j$  thỏa  $s_{j-1} \leq r < s_j$ 
    }
     $P' \leftarrow (I'_1, I'_2, \dots, I'_N)$ 
}

```

Đặt $\alpha = c^N$, cường độ chọn lọc sắp hạng lũy thừa có thể xấp xỉ bằng công thức dưới đây với sai số nhỏ hơn 6% khi $\alpha \in [10^{-20}; 0,8]$:

$$I(\alpha) \approx 0,588 \frac{\ln \ln(\pi / \alpha)}{3,69^\alpha} \quad (4.16)$$



Hình 4.11 Cường độ chọn lọc sắp hạng lũy thừa

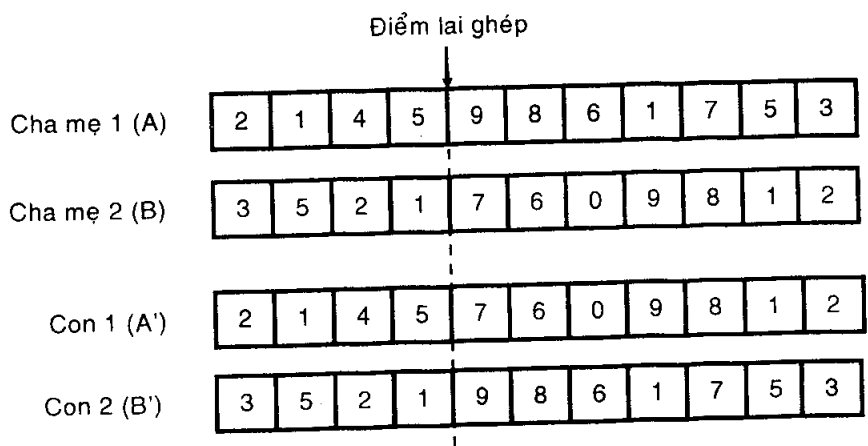
4.2.5 Lai ghép

Lai ghép là phương thức chia sẻ thông tin giữa các NST, phép toán này kết hợp đặc điểm của hai NST cha mẹ để tạo ra hai NST con với triển vọng cha mẹ tốt sẽ tạo ra con tốt hơn. Phép lai ghép

thường không tác động đến tất cả các NST mà trái lại chỉ xảy ra giữa hai NST cha mẹ được lựa chọn ngẫu nhiên với xác suất p_c (gọi là xác suất lai ghép). Lai ghép đóng vai trò trung tâm trong GA, phép toán này là một trong những yếu tố cần ghi nhớ khi muốn cải thiện chất lượng của GA. Phép lai ghép được thực hiện theo nhiều cách khác nhau, tùy thuộc theo cách mã hóa. Nguyên tắc thực hiện phép lai ghép là bắt cặp ngẫu nhiên hai NST trong quần thể sau khi đã qua bước chọn lọc để tạo ra hai NST con, mỗi NST con thừa hưởng một phần gen của cha, một phần gen của mẹ.

1- Lai ghép một điểm (One point Crossover)

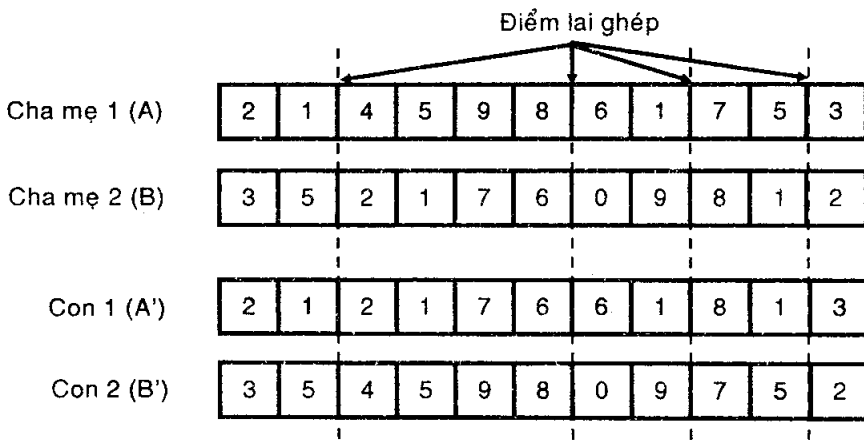
Đầu tiên ta chọn hai cá thể cha mẹ A, B. Hai cá thể này được lấy ra từ tập cá thể đã được chọn lọc bằng một trong những phương pháp đã trình bày ở mục 4.2.4. Tiếp theo, ta xác định điểm lai ghép ngẫu nhiên k trong đoạn $[2, L-1]$ với L là chiều dài của NST. Điểm k này sẽ chia gen A thành A_1, A_2 và B thành B_1, B_2 . Hai cá thể con được tạo ra từ phép lai này sẽ là A_1B_2 và B_1A_2 .



Hình 4.12 Lai ghép một điểm

2- Lai ghép nhiều điểm (Multipoint Crossover)

Trong phương pháp này, thay vì chỉ chọn một điểm lai ghép, ta chọn m điểm lai ghép $k_1, k_2, \dots, k_m$. m điểm lai ghép này sẽ chia chuỗi NST của hai cá thể cha mẹ A, B thành $m+1$ đoạn. Chuỗi NST của hai con sẽ được tạo ra bằng cách ghép các đoạn này của hai NST A, B theo quy tắc: các đoạn ở vị trí lẻ được giữ nguyên, các đoạn ở vị trí chẵn sẽ được hoán chuyển với nhau.



Hình 4.13 Lai ghép nhiều điểm

3- Lai ghép đều (Uniform Crossover)

Lai ghép đều là phương pháp lai ghép tổng quát nhất. Trong lai ghép đều ta sử dụng hai mặt nạ mA và mB để tạo ra hai chuỗi nhiễm sắc thể con A' và B' . Hai mặt nạ này là một chuỗi số nhị phân có cùng chiều dài với A, B . Mặt nạ này có thể được phát sinh một cách ngẫu nhiên khi tiến hành lai ghép hoặc kế thừa từ thế hệ trước. Giá trị bit của mặt nạ sẽ quyết định thành phần gen nào của cá thể con sẽ được trích ra từ gen của cha mẹ. Quy tắc này như sau:

- Giá trị bit thứ i trong mặt nạ mX (X bằng A hoặc B) có giá trị 1 sẽ cho biết: gen thứ i trong chuỗi nhiễm sắc thể của cá thể con X' được sao chép từ gen thứ i của cá thể cha mẹ A .

- Ngược lại, giá trị bit thứ i trong mặt nạ mX (X bằng A hoặc B) có giá trị 0 sẽ cho biết: gen thứ i trong chuỗi nhiễm sắc thể của cá thể con X' sẽ được sao chép từ gen thứ i của cá thể cha mẹ B .

Về nguyên tắc, mặt nạ mA và mB độc lập với nhau. Tuy nhiên, trên thực tế, người ta thường dùng cặp mặt nạ đảo nhau, nghĩa là $mA = \text{NOT}(mB)$.

Dễ nhận ra rằng lai ghép một điểm, lai ghép nhiều điểm là các trường hợp đặc biệt của lai ghép đều. Chẳng hạn như với lai ghép một điểm với vị trí lai ghép là k , ta chỉ việc dùng mặt nạ mA có $k-1$ bit đầu là 1 và các bit còn lại là 0 và mặt nạ $mB = \text{NOT}(mA)$.

Cha mẹ 1 (A)	2	1	4	5	9	8	6	1	7	5	3
Cha mẹ 2 (B)	3	5	2	1	7	6	0	9	8	1	2
Mắt nạ 1 (mA)	0	1	1	1	0	0	1	1	0	1	0
Mắt nạ 2 (mB)	1	0	0	1	1	0	0	1	0	1	1
Con 1 (A')	3	1	4	5	7	6	6	1	8	5	2
Con 2 (B')	2	5	2	5	9	6	0	1	8	5	3

Hình 4.14 Lai ghép đều

Nhận xét: Mục đích của phép toán lai ghép là tìm kiếm không gian lời giải đồng thời bảo toàn thông tin chứa trong các chuỗi NST cha mẹ (vì đây những các thể tốt được chọn lựa nhờ phép toán chọn lọc). Phép toán lai ghép một điểm có khả năng tìm kiếm không rộng nhưng thông tin được bảo toàn tối đa từ NST cha mẹ sang NST con. Ngược lại, phép lai ghép đều có khả năng tìm kiếm rất rộng nhưng ít bảo toàn thông tin từ NST cha mẹ sang NST con.

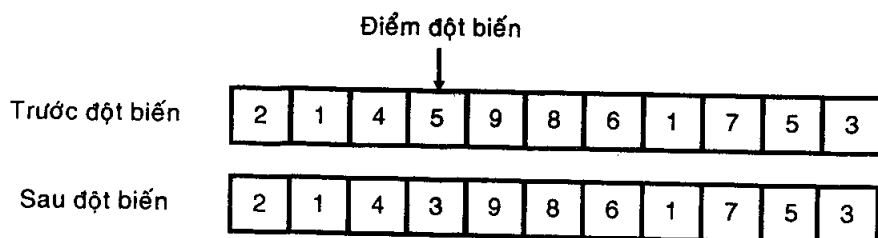
4.2.6 Đột biến

Thế hệ con được tạo ra qua quá trình chọn lọc tự nhiên và lai ghép sẽ mang những “phẩm chất tốt” của thế hệ cha mẹ. Tuy nhiên, nhiều khi do thế hệ khởi tạo ban đầu có đặc tính chưa phong phú và chưa phù hợp nên các cá thể không rải đều được hết không gian của lời giải, từ đó khó có thể tìm ra lời giải tối ưu. Phép toán đột biến thay đổi ngẫu nhiên một hoặc nhiều gen của một cá thể để làm tăng sự đa dạng về cấu trúc trong quần thể. Vai trò của đột biến là khôi phục chất liệu di truyền bị mất hoặc chưa khai phá để ngăn hiện tượng GA hội tụ sớm vào lời giải tối ưu cục bộ. Phép đột biến đảm bảo xác suất đạt tới bất cứ điểm nào trong không gian tìm kiếm cũng khác 0. Tuy nhiên, đột biến chỉ được phép xảy ra với xác suất p_M thấp, vì hiện tượng này có thể gây xáo trộn và làm mất đi những cá thể đã chọn lọc và lai ghép có tính thích nghi cao, dẫn đến GA không còn hiệu quả. Có nhiều cách thực hiện phép toán đột biến tùy thuộc

vào cách mã hóa, nguyên tắc chung để thực hiện phép đột biến là thay đổi ngẫu nhiên chuỗi NST với xác suất đột biến p_M .

1- Đột biến một điểm

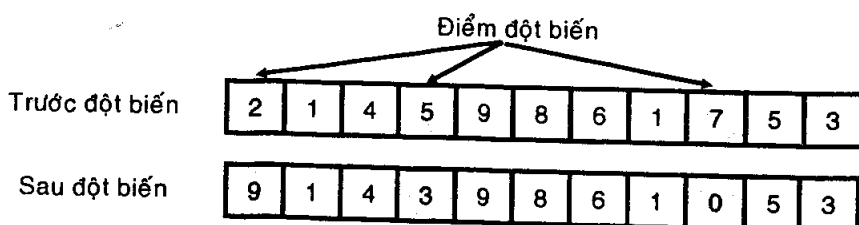
Điểm đột biến được chọn ngẫu nhiên, giá trị gen tại điểm đột biến sẽ thay đổi ngẫu nhiên bằng giá trị bất kỳ trong bộ gen (khác với giá trị gen hiện tại). Nếu chuỗi nhiễm sắc thể được mã hóa nhị phân (bộ gen chỉ có hai ký hiệu 0 và 1) thì hiện tượng đột biến làm đảo bit tại điểm đột biến.



Hình 4.15 Đột biến một điểm

2- Đột biến nhiều điểm

Tương tự như đột biến một điểm, chỉ khác là trong trường hợp này số gen bị đột biến nhiều hơn 1.



Hình 4.16 Đột biến nhiều điểm

4.2.7 Các thông số của giải thuật di truyền

1- Kích thước quần thể

Kích thước quần thể là một trong những tùy chọn quan trọng nhất và có thể đóng vai trò quyết định trong nhiều ứng dụng. Nếu kích thước quần thể quá nhỏ, GA có thể hội tụ quá nhanh; ngược lại nếu kích thước quần thể quá lớn, GA sẽ hội tụ chậm và có thể lãng phí tài nguyên máy tính. Có một số nghiên cứu đề nghị kích thước

quần thể thích hợp. Goldberg [36] đề nghị kích thước quần thể chỉ phụ thuộc vào chiều dài của chuỗi L :

$$N = 1,65 * 2^{0,21L} \quad (4.17)$$

Schaffer et.al (1989) thực hiện nhiều mô phỏng trên một số hàm kiểm tra và kết luận rằng GA hoạt động tốt với kích thước quần thể nhỏ từ 20 đến 30, xác suất lai ghép nằm trong khoảng 0,75 đến 0,95, và một xác suất đột biến từ 0,005 đến 0,01. Sau đó, Goldberg, Deb, and Clark [24] đã tính kích thước quần thể dựa vào phi tuyến của bài toán cần giải dùng GA như sau:

$$N \approx O(2^k \frac{\sigma^2}{d^2}) \quad (4.18)$$

trong đó k là bậc phi tuyến, σ^2 là phương sai của bài toán, d^2 là sai lệch giữa giá trị thích nghi của lời giải tối ưu cục bộ và toàn cục. Theo công thức (4.18), kích thước của quần thể tăng khi tính phi tuyến của bài toán cần giải tăng. Harik et.al. (1996) đề xuất công thức cải biến (4.18) tính kích thước quần thể như sau:

$$N \approx O(2^k \frac{\sigma}{d}) \quad (4.19)$$

Mô phỏng với một số bài toán kiểm tra cho thấy GA với kích thước quần thể tính theo công thức (4.19) cho kết quả tốt hơn so với công thức (4.18).

Một số công trình nghiên cứu thực nghiệm gợi ý cho ta chọn $N \leq 100$ và theo Goldberg thì hiệu quả của giải thuật di truyền tỷ lệ với bậc ba của kích thước quần thể.

Một trong những nghiên cứu thay đổi thích nghi kích thước quần thể trong quá trình tiến hóa đầu tiên là công trình của Baker [10]. Baker nhấn mạnh tầm quan trọng của việc ngăn ngừa hiện tượng GA hội tụ nhanh hơn là phục hồi sau khi GA đã hội tụ. Baker nhận thấy rằng hiện tượng hội tụ nhanh thường xuất hiện sau khi một NST hoặc một nhóm NST đóng góp một lượng lớn cá thể con qua phép chọn lọc. Do kích thước quần thể là hữu hạn nên một NST có quá nhiều con dẫn đến nhiều cá thể còn lại trong quần thể không có con, kết quả là quần thể mất tính đa dạng rất nhanh và GA bị hội tụ sớm. Để nhận ra hiện tượng hội tụ nhanh trước khi chất liệu di

truyền bị mất, Baker đưa ra số đo “*phần trăm tái sinh*”, chính là phần trăm của quần thể hiện tại có đóng góp con cho thế hệ sau. Baker đề xuất GA có kích thước động để khống chế chặn dưới của “*phần trăm tái sinh*”. Điều này được thực hiện bằng cách thêm NST vào quần thể cho đến khi đạt được giá trị “*phần trăm tái sinh*” chấp nhận được. Kích thước quần thể giảm trong những chu kỳ chậm hội tụ.

2- Xác suất lai ghép

Lai ghép tìm kiếm lời giải mới có xu hướng tốt hơn bằng cách kết hợp hai NST cha mẹ. Có nhiều phép lai ghép đã được đưa ra (xem mục 4.2.5), mỗi phép lai ghép có “*khả năng tìm kiếm*” khác nhau. “*Khả năng tìm kiếm*” của phép lai ghép là đại lượng tỷ lệ với không gian tìm kiếm có thể đạt tới bằng cách lai ghép hai cá thể cha mẹ một lần. Bảng 4.1 là “*khả năng tìm kiếm*” của ba phương pháp lai ghép phổ biến áp dụng cho chuỗi NST nhị phân có chiều dài L .

Bảng 4.1 Khả năng tìm kiếm của các phép lai ghép thường dùng

Phép lai ghép	Khả năng tìm kiếm
Một điểm	$(L - 1) / 2^{L-1}$
Hai điểm	$(L - 1) / 2^L$
Đều	1.00

Bảng 4.1 cho thấy “*khả năng tìm kiếm*” của phép lai ghép đều là cực đại. Điều này có nghĩa là phương pháp lai ghép đều phá hủy cấu trúc của các cá thể cha mẹ rất lớn và có một xác suất bảo tồn rất thấp. “*Khả năng tìm kiếm*” của phép lai ghép cũng bị ảnh hưởng xác suất lai ghép p_c . Nếu giá trị p_c gần bằng 1 thì lai ghép được thực hiện trên toàn quần thể và “*khả năng tìm kiếm*” là cực đại. Thường p_c có giá trị khá lớn, khoảng từ 0,7 đến 1,0 (Goldberg, [36]).

Trong một số trường hợp, GA với xác suất lai ghép thích nghi có nhiều lợi ích. Có thể dựa vào số đo entropy của quần thể để điều chỉnh xác suất lai ghép: tăng nhẹ p_c nếu entropy giảm và giảm p_c nếu entropy không đổi hoặc tăng. Một cách khác để điều chỉnh thích nghi xác suất lai ghép là dựa vào số đo “*phần trăm tái sinh*” (xem mục 1 của phần 4.2.7). Nếu “*phần trăm tái sinh*” giảm thì xác suất lai ghép tăng và ngược lại (Booker, [15]).

3- Xác suất đột biến

Sự đột biến duy trì sự đa dạng của quần thể và vì vậy nó được sử dụng với một xác suất nhỏ p_M . Qua nhiều nghiên cứu thực nghiệm lẫn toán học, người ta đã nhận xét được rằng, để quá trình đột biến có hiệu quả thì xác suất đột biến nên tỷ lệ nghịch với chiều dài nhiễm sắc thể. Chiều dài nhiễm sắc thể càng lớn thì xác suất diễn ra đột biến càng nhỏ. Một xác suất đột biến được dùng rộng rãi là $1/L$ với L là chiều dài của chuỗi NST. Hơn nữa, người ta cũng khẳng định rằng xác suất đột biến nên độc lập với kích thước quần thể N , nghĩa là số lượng cá thể trong quần thể tăng hay giảm không làm ảnh hưởng đến khả năng đột biến của các cá thể trong quần thể đó.

Một trong những cách tránh hiện tượng GA hội tụ sớm là tăng xác suất đột biến để khôi phục các đoạn gen tốt bị mất. Tuy nhiên, nếu xác suất đột biến quá cao, việc đưa vào quần thể quá nhiều đoạn gen ngẫu nhiên sẽ ảnh hưởng đến quá trình tìm kiếm di truyền. Để đảm bảo tính đa dạng gen, đồng thời không ảnh hưởng lớn đến sự di truyền các tính trạng tốt trong quần thể, giải pháp được đưa ra là cho xác suất đột biến thay đổi trong quá trình tiến hóa. Nguyên tắc đơn giản nhất là giảm dần xác suất đột biến qua các thế hệ. Ban đầu xác suất đột biến cao để đảm bảo GA tìm kiếm rộng trong không gian lời giải, sau đó xác suất đột biến giảm để GA hội tụ đến lời giải tối ưu. Xác suất đột biến có thể giảm tuyến tính hoặc giảm theo quy luật hàm mũ.

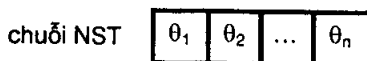
4.3 GIẢI THUẬT DI TRUYỀN MÃ SỐ THỰC

4.3.1 Mã hóa số thực

Cách mã hóa nhị phân dùng trong GA cơ bản có khuyết điểm là không phù hợp để giải các bài toán tìm kiếm lời giải tối ưu trong không gian liên tục, nhiều chiều và yêu cầu độ chính xác cao. Trong trường hợp này lời giải bài toán được mã hóa thành chuỗi NST rất dài, dẫn đến GA chạy chậm. Cách mã hóa thập phân đã khắc phục phần nào khuyết điểm của cách mã hóa nhị phân, tuy nhiên đây cũng chưa phải là cách hiệu quả nhất để mã hóa lời giải của bài toán tối ưu hóa hàm trong không gian liên tục nhiều chiều vì độ chính xác của lời giải vẫn phụ thuộc vào chiều dài của chuỗi NST.

Một cách tự nhiên hơn, có thể biểu diễn các gen trực tiếp là các số thực cho bài toán tối ưu hóa thông số với các biến nằm trong miền

liên tục, khi đó chuỗi NST là vector gồm các thành phần là các số thực (H.4.17), độ chính xác của lời giải sẽ chỉ bị giới hạn bởi khả năng của máy tính thực thi GA. Ưu điểm của cách mã hóa số thực là việc sử dụng các biến thực làm cho GA có thể tìm kiếm các miền rộng lớn trong không gian lời giải, điều này rất khó đạt được bằng cách mã hóa nhị phân khi mà việc mở rộng miền tìm kiếm dẫn đến lời giải kém chính xác với nếu chuỗi NST có chiều dài cố định. Bên cạnh đó việc sử dụng mã số thực để biểu diễn lời giải rất gần với phát biểu tự nhiên của nhiều bài toán, do đó không cần quá trình mã hóa và giải mã, vì vậy tăng tốc độ của GA. Phương pháp mã hóa số thực có khuyết điểm là không thể sử dụng các phép toán lai ghép và đột biến cơ bản đã đưa ra cách mã hóa nhị phân.



Hình 4.17 Mã hóa số thực

4.3.2 Lai ghép NST mã số thực

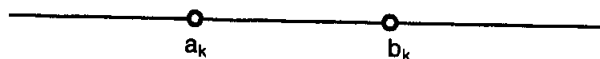
Giả sử $A = (a_1, a_2, \dots, a_n)$ và $B = (b_1, b_2, \dots, b_n)$ là hai chuỗi NST cha mẹ được chọn lọc ngẫu nhiên, A và B có thể lai ghép theo một trong các cách dưới đây để tạo ra NST con $C = (c_1, c_2, \dots, c_n)$.

Lai ghép đơn giản (Simple Crossover): một điểm lai ghép $k \in [2, n - 1]$ được chọn ngẫu nhiên và NST con được tạo ra như sau:

$$C = (a_1, \dots, a_k, b_{k+1}, \dots, b_n) \quad (4.20)$$

Lai ghép rời rạc (Discrete Crossover): gen c_k của NST con được chọn ngẫu nhiên một trong hai giá trị a_k và b_k

$$c_k = \text{random}(\{a_k, b_k\}) \quad (4.21)$$



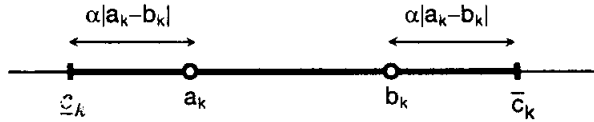
Hình 4.18 Lai ghép rời rạc (o cha mẹ, • con)

Lai ghép BLX- α (BLX- α Crossover): gen c_k của NST con được chọn ngẫu nhiên trong đoạn $[\underline{c}_k, \bar{c}_k]$:

$$c_k = \text{random}([\underline{c}_k, \bar{c}_k]) \quad (4.22)$$

trong đó:
$$\underline{c}_k = \min(a_k, b_k) - \alpha|a_k - b_k| \tag{4.23}$$

$$\bar{c}_k = \max(a_k, b_k) + \alpha|a_k - b_k| \tag{4.24}$$

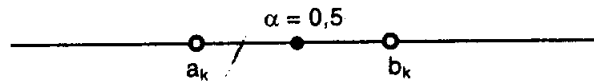


Hình 4.19 Lai ghép BLX- α (o cha mẹ, • con)

Lai ghép số học (Arithmetical Crossover): gen thứ k trên hai NST con được tạo ra như sau:

$$c_k = \alpha a_k + (1 - \alpha)b_k \tag{4.25}$$

trong đó α là hằng số (lai ghép số học đồng nhất) hoặc có thể thay đổi qua các thế hệ (lai ghép số học không đồng nhất).



Hình 4.20 Lai ghép số học (o cha mẹ, • con)

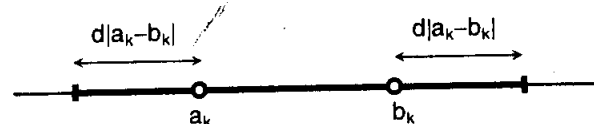
Lai ghép đường thẳng (Line Crossover): gen thứ k trên hai NST con được tạo ra như sau:

$$c_k = \min(a_k, b_k) + \alpha|a_k - b_k| \tag{4.26}$$

trong đó α được chọn ngẫu nhiên trong đoạn $[-d, 1 + d]$, $d > 0$.

Nếu mỗi gen được tạo ra tương ứng với hệ số α_k khác nhau, ta có phiên bản lai ghép đường thẳng tức thời:

$$c_k = \min(a_k, b_k) + \alpha_k|a_k - b_k| \tag{4.27}$$

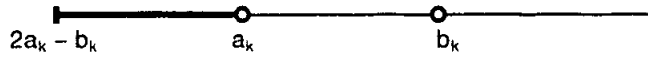


Hình 4.21 Lai ghép đường thẳng (o cha mẹ, • con)

Lai ghép trực giác (Heuristic Crossover): Không mất tính tổng quát, nếu A là NST cha mẹ có độ thích nghi tốt hơn thì gen thứ k trên NST con được tạo ra như sau:

$$c_k = a_k + \alpha(a_k - b_k) \quad (4.28)$$

trong đó α là số ngẫu nhiên thuộc đoạn $[0,1]$



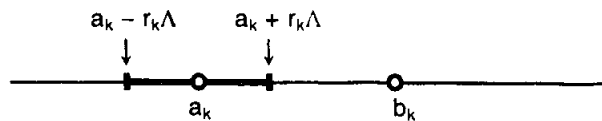
Hình 4.22 Lai ghép trực giác (o cha mẹ, • con)

Lai ghép tuyến tính BGA (Linear BGA Crossover): Với giả thiết giống như trên, NST con được gồm các gen được tạo ra như sau:

$$c_k = a_k \pm r_k \cdot \gamma \cdot \Lambda \quad (4.29)$$

trong đó $\Lambda = \frac{r_k - b_k}{\|A - B\|}$.

Dấu ‘ $\pm$ ’ trong công thức (4.29) được chọn với xác suất 0,9. Thông thường $r_k = 0,5(b_k - a_k)$ và $\gamma = \sum_{i=0}^{15} \alpha_i 2^{-i}$ với α_i được chọn ngẫu nhiên trong tập $\{0,1\}$ với xác suất $p(\alpha_i = 1) = 1/16$.



Hình 4.23 Lai ghép tuyến tính BGA (o cha mẹ, • con)

4.3.3 Đột biến mã số thực

Gọi $C = (c_1, c_2, \dots, c_n)$ là chuỗi NST có gen $c_k \in [c_{k \min}, c_{k \max}]$ bị đột biến thành c'_k . Có nhiều cách để thực hiện phép toán đột biến số thực, các phương pháp đột biến sau đây được sử dụng nhiều nhất:

Đột biến ngẫu nhiên (Random Mutation): c'_k là giá trị ngẫu nhiên nằm trong đoạn $[c_{k \min}, c_{k \max}]$.

$$c'_k = \text{random}([c_{k \min}, c_{k \max}]) \quad (4.30)$$



Hình 4.24 Đột biến ngẫu nhiên (o trước đột biến, • sau đột biến)

Đột biến không đồng nhất (Non-uniform Mutation): Nếu phép toán đột biến này được áp dụng ở thế hệ thứ t và $g_{\max}$ là số thế hệ tối đa thì:

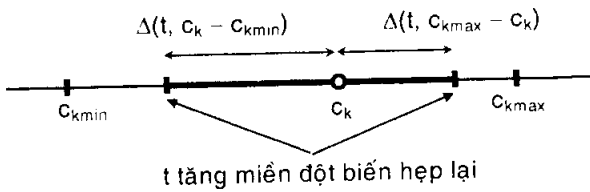
$$c'_k = \begin{cases} c_k + \Delta(t, c_{k\max} - c_k) & \text{nếu } \tau = 0 \\ c_k - \Delta(t, c_k - c_{k\min}) & \text{nếu } \tau = 1 \end{cases} \quad (4.31)$$

trong đó τ là số ngẫu nhiên có giá trị bằng 0 hoặc 1 và:

$$\Delta(t, x) = x(1 - r^{(1 - \frac{t}{g_{\max}})^\alpha}) \quad (4.32)$$

với r là giá trị ngẫu nhiên nằm trong đoạn $[0,1]$ và α là tham số chọn bất kỳ.

Để ý hàm $\Delta(t, x)$ cho giá trị ngẫu nhiên nằm trong đoạn $[0, x]$ với xác suất gần bằng 0 tăng lên khi t càng tăng, do đó GA chạy càng lâu thì đột biến càng ít ảnh hưởng.



Hình 4.25 Đột biến không đồng nhất (o trước đột biến, • sau đột biến)

Đột biến Mühlennein (Mühlennein's Mutation):

$$c'_k = c_k \pm r_k \cdot \gamma \quad (4.33)$$

trong đó dấu '-' hoặc '+' trong công thức (4.33) được chọn với xác suất 0,5 và:

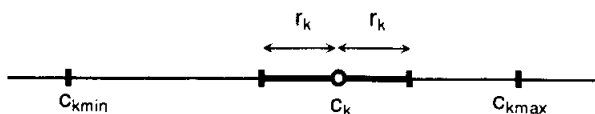
$$r_k = 0.1(c_{k\max} - c_{k\min}) \quad (4.34)$$

$$\gamma = \sum_{i=0}^{15} \alpha_i 2^{-i} \quad (4.35)$$

với α_i được chọn ngẫu nhiên trong tập $[0;1]$ với xác suất

$$p(\alpha_i = 1) = 1/16$$

r_k gọi là tầm đột biến. Phép đột biến Mühlennein tạo ra gen $c'_k \in [c_k - r_k, c_k + r_k]$ với xác suất gần bằng c_k rất cao.



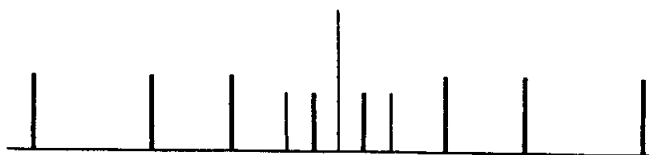
Hình 4.26 Đột biến Mühlennein (o trước đột biến, • sau đột biến)

Đột biến rời rạc (Discrete Modal Mutation): Đây là một phiên bản của phép đột biến Mühlennein, trong đó:

$$\gamma = \sum_{i=0}^{\pi} \alpha_i B_m^i \quad (4.36)$$

với:
$$\pi = \frac{\log(r_{\min})}{\log(B_m)} \quad (4.37)$$

Tham số $B_m > 1$ gọi là hệ số cơ sở của phép đột biến và $r_{\min}$ là giới hạn dưới của tầm đột biến tương đối.



Hình 4.27 Đột biến rời rạc

Đột biến liên tục (Continuous Modal Mutation): Đây cũng là một phiên bản của phép đột biến Mühlennein, trong đó:

$$\gamma = \sum_{i=0}^{\pi} \alpha_i \phi(B_m^i) \quad (4.38)$$

với π tính theo công thức (4.37), tham số $B_m > 1$ gọi là hệ số cơ sở của phép đột biến và $\phi(x_i)$ là phân bố xác suất dạng tam giác với:

$$\frac{B_m^i - B_m^{i-1}}{2} \leq x_i \leq \frac{B_m^{i+1} - B_m^i}{2} \quad (4.39)$$



Hình 4.28 Đột biến liên tục

4.3.4 Một số nhận xét

Qua các ứng dụng thực nghiệm, một số nhận xét đã được rút ra như sau:

- Đối với bài toán tối ưu hóa thông số, GA mã hóa số thực cho kết quả tốt hơn GA mã hóa nhị phân hoặc thập phân về cả hai phương diện tốc độ tính toán và độ chính xác của lời giải.

- Phép lai ghép BLX- α (đặc biệt là trường hợp $\alpha = 0,5$) và lai ghép đường thẳng là thích hợp nhất cho GA mã hóa số thực. Các phép toán này có khả năng tạo ra các cá thể con ngoài miền $[\min(a_k, b_k), \max(a_k, b_k)]$ nên không bỏ sót các lời giải trong quá trình tìm kiếm.

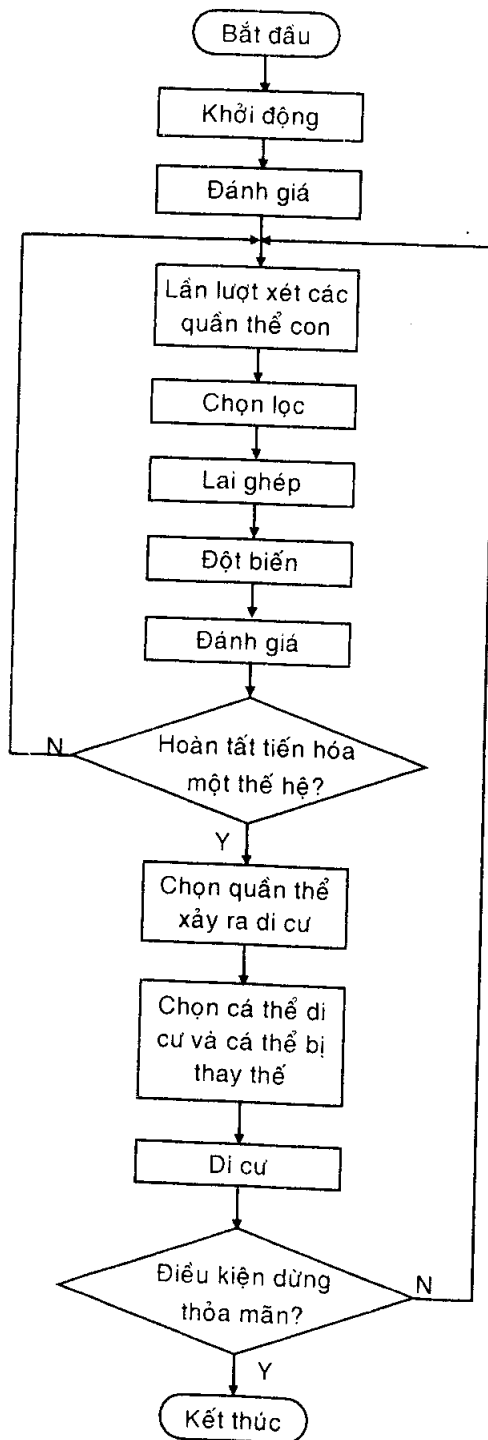
- Phép đột biến không đồng nhất rất thích hợp cho GA mã số thực vì phép toán này có đặc điểm là ảnh hưởng của hiện tượng đột biến càng giảm khi GA chạy càng lâu.

4.4 GIẢI THUẬT DI TRUYỀN SONG SONG

4.4.1 Khái niệm về GA song song

Theo di truyền học, mô hình sinh thái đảo là mô hình mà trong đó các quần thể con riêng biệt và cô lập tiến hóa một cách độc lập và song song. Người ta tin rằng nhiều quần thể con phân tán với các quy luật và sự tương tác cục bộ là mô hình có tính thực tế của các loài trong tự nhiên hơn là một quần thể đơn lớn. GA song song phỏng theo sự tiến hóa của quần thể theo mô hình sinh thái đảo.

GA song song là GA trong đó quần thể được chia ra làm nhiều quần thể con, mỗi quần thể con được khởi động ngẫu nhiên và tiến hóa độc lập và thỉnh thoảng có vài cá thể di cư giữa các quần thể con (H.4.29). Sự di cư của các cá thể giữa các quần thể con là đặc điểm quan trọng của GA song song. Phép toán di cư cho phép phân tán và chia sẻ các đoạn gen giữa các quần thể con thông qua các cá thể di cư. Điều này làm tăng áp lực chọn lọc chung vì có thêm phép chọn lọc được thực hiện để chọn các cá thể di cư. Đồng thời, việc đưa các cá thể di cư vào các quần thể con giúp duy trì tính đa dạng gen, vì các cá thể di cư đến từ một quần thể con khác được tiến hóa độc lập. Nếu chọn các thông số thích hợp, hiện tượng di cư giúp GA tránh rơi vào cực trị cục bộ, tăng xác suất tìm được lời giải tối ưu toàn cục.



Hình 4.29 Lưu đồ giải thuật di truyền song song

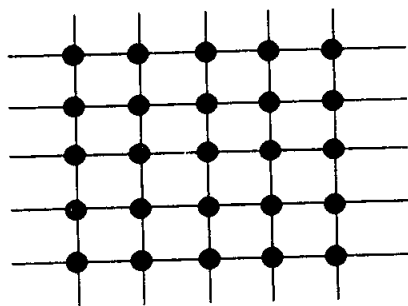
GA song song là mô hình logic, tức là không nhất thiết phải thực thi GA song song trên máy tính song song mà hoàn toàn có thể thực thi GA song song trên máy tính tuần tự bằng cách phân chia thời gian tính toán quá trình tiến hóa của các quần thể con.

4.4.2 Các tùy chọn của GA song song

Đặc điểm của GA song song được xác định bởi nhiều tùy chọn, bên cạnh tùy chọn các phép toán di truyền tác động lên từng quần thể con thì các tùy chọn sau đây là đặc trưng của GA song song:

- Cấu hình phân bố các quần thể con.
- Cách chọn lựa các quần thể con mà giữa chúng xảy ra hiện tượng di cư.
- Cách chọn lựa cá thể di cư và cá thể bị thay thế
- Số lượng cá thể di cư và tần suất di cư của chúng

Số quần thể con và cấu hình phân bố: Số lượng quần thể con càng nhiều, hiệu quả của GA song song càng cao. Cấu hình phân bố các quần thể con đơn giản nhất là phân bố theo cấu hình lưới trong không gian hai chiều. Với cấu hình này, hiện tượng di cư xảy ra ngẫu nhiên giữa hai quần thể lân cận nhau hoặc xảy ra tuần tự từ một quần thể con này lần lượt đến các quần thể con khác trong lân cận của nó.



Hình 4.30 Phân bố các quần thể con trong không gian hai chiều

Chọn cá thể di cư: Có hai lý do để cho một cá thể di cư từ quần thể con này sang quần thể con khác: thứ nhất là để làm tăng độ thích nghi trung bình của quần thể mà cá thể di cư sang, thứ hai là để giúp quần thể nhận cá thể di cư sang đảm bảo tính đa dạng. Có nhiều cách chọn tùy theo ta muốn cá thể di cư nhằm mục đích gì, sau đây là ba cách điển hình:

- Chọn cá thể có độ thích nghi cao nhất: Nếu một quần thể con thường xuyên nhận cá thể di cư có độ thích nghi cao, các cá thể này trở nên cá thể nổi trội trong quần thể, GA sẽ tập trung tìm kiếm vào các cá thể này và do đó mất tính đa dạng, GA dễ rơi vào cực trị cục bộ.

- Chọn cá thể ngẫu nhiên: nếu quần thể nhận một cá thể di cư ngẫu nhiên thì tính đa dạng được đảm bảo, tuy nhiên không chắc rằng độ thích nghi trung bình của quần thể được cải thiện.

- Chọn theo phương pháp đấu vòng ngẫu nhiên với xác suất 0,6: cách chọn này đảm bảo tính đa dạng, đồng thời cá thể di cư có độ thích nghi trên trung bình.

Chọn cá thể bị thay thế: có thể chọn bằng một trong hai cách sau đây:

- Thay thế cá thể có độ thích nghi kém nhất: cách này làm tăng áp lực chọn lọc, quần thể dễ rơi vào cực trị cục bộ.

- Thay thế cá thể “thắng cuộc” theo cách chọn đấu vòng ngẫu nhiên với xác suất 0,4: cách này không loại bỏ cá thể kém nhất mà chỉ loại bỏ có thể có độ thích nghi dưới trung bình, do đó làm giảm áp lực chọn lọc, đảm bảo tính đa dạng của quần thể.

Số lượng cá thể di cư: Thông thường số lượng cá thể di cư bằng 1, tuy nhiên cũng có thể chọn nhiều hơn 1.

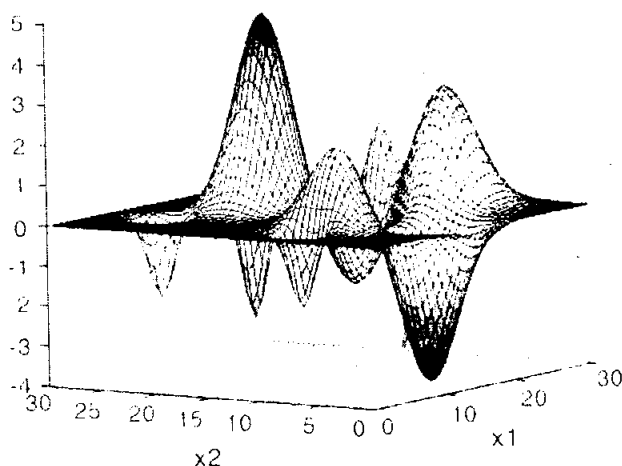
Tần suất di cư: Hiện tượng di cư không nên xảy ra quá thường xuyên.

4.5 MỘT SỐ VÍ DỤ

Để minh họa khả năng của GA, mục này trình bày một ví dụ ứng dụng GA để giải các bài toán trong nhiều lĩnh vực khác nhau như tối ưu hóa hàm, tìm đường đi ngắn nhất, và đo lường tốc độ lưu chất. Các ứng dụng cụ thể của GA trong các hệ thống điều khiển sẽ được đề cập đến trong các chương sau.

Ví dụ 4.3 Tìm cực tiểu hàm

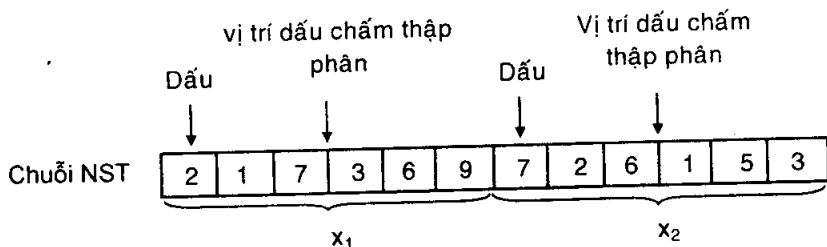
$$\begin{aligned}
 y = f(x_1, x_2) = & 5e^{-0,1[(x_1-15)^2+(x_2-20)^2]} - 2e^{-0,08[(x_1-20)^2+(x_2-15)^2]} \\
 & + 3e^{-0,08[(x_1-25)^2+(x_2-10)^2]} + 2e^{-0,1[(x_1-10)^2+(x_2-10)^2]} - 2e^{-0,5[(x_1-5)^2+(x_2-10)^2]} \\
 & - 4e^{-0,1[(x_1-15)^2+(x_2-5)^2]} - 2e^{-0,5[(x_1-8)^2+(x_2-25)^2]} \\
 & - 3e^{-0,5[(x_1-21)^2+(x_2-25)^2]} + 2e^{-0,5[(x_1-25)^2+(x_2-16)^2]}
 \end{aligned}$$



Hình 4.31 Hàm cần tìm cực tiểu

Hình 4.31 là đồ thị biểu diễn giá trị hàm $y = f(x_1, x_2)$ cần tìm cực tiểu, ta thấy hàm này là tổng của các hàm mũ nên có rất nhiều cực trị cục bộ. Đây là dạng hàm cần tìm cực trị rất điển hình trong điều khiển, chẳng hạn như trong nhận dạng hệ phi tuyến hoặc chỉnh định thông số bộ điều khiển mờ. Nếu giải bài toán tìm cực tiểu dùng thuật toán tối ưu hóa thông thường như các phiên bản của thuật toán Newton thì lời giải thường rơi vào các cực tiểu cục bộ (trừ khi giá trị khởi động ban đầu của thuật toán tối ưu hóa nằm gần điểm cực tiểu toàn cục). GA với đặc điểm là một thuật toán tìm lời giải tối ưu toàn cục có thể giải quyết rất tốt bài toán này.

Mã hóa: Sử dụng phương pháp mã hóa thập phân, mỗi lời giải của bài toán tìm cực trị là một cặp (x_1, x_2) được mã hóa thành chuỗi NST có dạng như hình 4.32. NST ở hình 4.32 tương ứng với lời giải $(x_1, x_2) = (-17.369; 26.153)$.



Hình 4.32 Mã hóa lời giải bài toán tìm cực tiểu ở ví dụ 4.2

Hàm thích nghi: Biến đổi bài toán tìm cực tiểu hàm $f(x_1, x_2)$ thành bài toán tìm cực đại của hàm thích nghi (để có thể áp dụng phương pháp chọn lọc tỷ lệ). Hàm thích nghi được chọn như sau:

$$fitness = \frac{1}{f(x_1, x_2) + C} \quad (4.40)$$

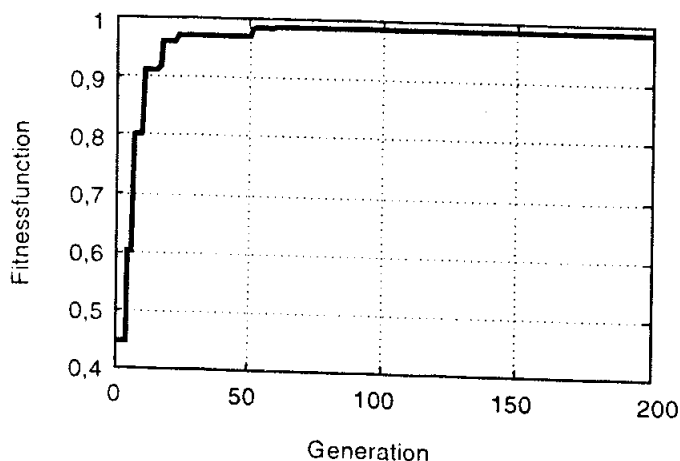
Hệ số C ở mẫu số hàm thích nghi được chọn để đảm bảo hàm thích nghi luôn dương. Trong ví dụ này có thể chọn $C = 5$.

Khởi động: Kích thước quần thể là 20, thế hệ đầu khởi động ngẫu nhiên.

Các phép toán di truyền: Chọn lọc tỷ lệ, lai ghép hai điểm với xác suất $p_C = 0,9$; đột biến đều với xác suất $p_M = 0,1$; lưu giữ NST ưu việt trong quá trình tiến hóa.

Chương trình thực hiện giải thuật di truyền trình bày ở phụ lục A.1. Kết quả chạy GA tìm được lời giải tối ưu sau khoảng 100 thế hệ. Hình 4.33 là đồ thị sự thay đổi của hàm thích nghi. Ta thấy qua các thế hệ, giá trị hàm thích nghi tăng dần tiến đến giá trị cực đại, điều này đồng nghĩa với hàm cần tìm cực trị $f(x_1, x_2)$ tiến dần đến giá trị cực tiểu.

Lời giải tối ưu tìm được dùng GA là $(x_1^*, x_2^*) = (14.998; 5.004)$, sai lệch không đáng kể so với cực tiểu đúng của hàm $f(x_1, x_2)$ tính toán dựa vào biểu thức toán học là $(x_1^*, x_2^*) = (15.000; 5.000)$.



Hình 4.33 Hàm thích nghi của NST tốt nhất trong mỗi thế hệ

Ví dụ 4.4 Tìm đường đi tối ưu cho robot di động

Xét bài toán hoạch định đường đi ngắn nhất cho robot di động giữa M điểm có tọa độ biết trước để robot thực hiện một tác vụ nào đó. Nếu M nhỏ chúng ta dễ dàng xác định đường đi ngắn nhất cho robot trong số các đường đi có thể. Ngược lại, nếu M lớn thì không gian tìm kiếm rất lớn, bài toán tìm đường đi ngắn nhất bằng cách thông thường trở nên khó khăn. GA là một trong những giải pháp thích hợp nhất để giải bài toán trên.

Mã hóa: Đặt tên cho các điểm mà robot cần phải di chuyển tới là các số tự nhiên $1, 2, \dots, M$. Một lời giải của bài toán tìm đường đi được mã hóa thành một chuỗi NST gồm có các gen là tên của các điểm, ví dụ hình 4.34 là chuỗi NST mã hóa đường đi $2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow \dots \rightarrow 10 \rightarrow 8 \rightarrow 2$. Chú ý do đường đi phải qua tất cả các điểm nên chuỗi NST hợp lệ phải có tên của tất cả các điểm và không có gen nào có giá trị giống nhau. Ta cần tìm chuỗi NST hợp lệ tương ứng với đường đi ngắn nhất.

Chuỗi NST

2	1	4	3	6	9	7	11	5	10	8
---	---	---	---	---	---	---	----	---	----	---

Hình 4.34 Mã hóa lời giải bài tìm đường đi ngắn nhất cho robot

Hàm thích nghi: Gọi $(x(i), y(i))$ là tọa độ của điểm thứ i ($i = 1..M$), D là độ dài đoạn đường mà robot phải di chuyển. Độ dài đường đi tương ứng với chuỗi NST gồm các gen $[g_1, g_2, \dots, g_M]$ được tính theo công thức:

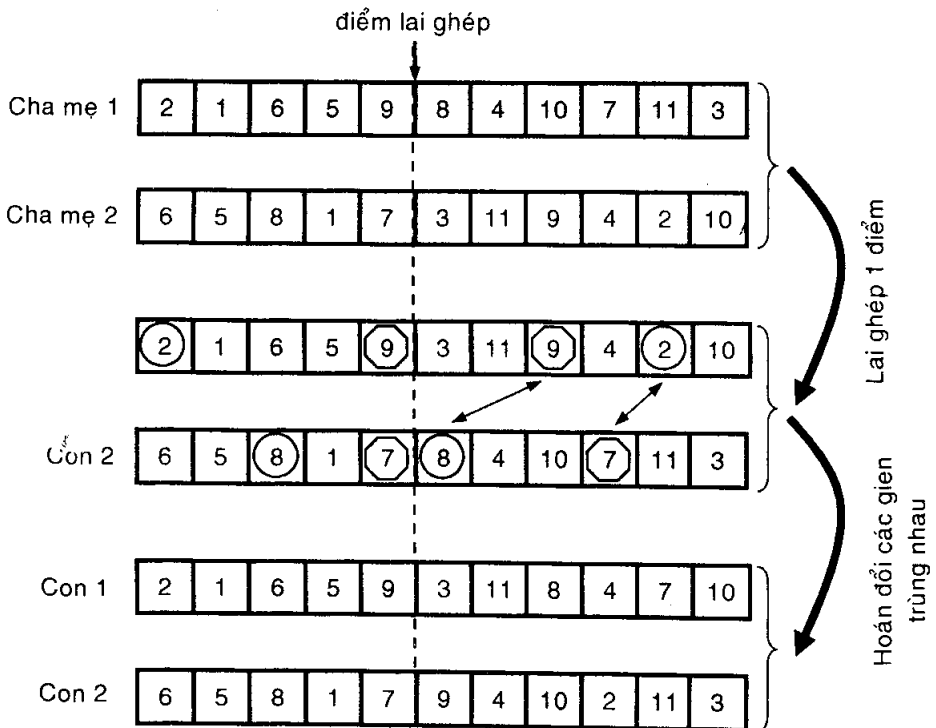
$$D = \sum_{i=2}^M \sqrt{[x(g_i) - x(g_{i-1})]^2 + [y(g_i) - y(g_{i-1})]^2} + \sqrt{[x(g_1) - x(g_M)]^2 + [y(g_1) - y(g_M)]^2} \quad (4.41)$$

Bài toán tìm cực tiểu đường đi D được chuyển thành bài toán tìm cực đại hàm thích nghi định nghĩa như sau:

$$fitness = \frac{1}{D} \quad (4.42)$$

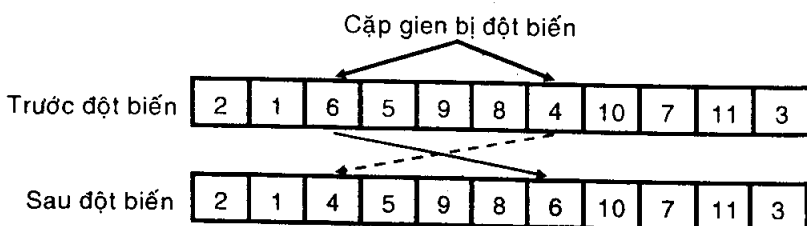
Chọn lọc: Áp dụng phương pháp chọn lọc sắp hạng tuyến tính với hệ số chọn lọc là $\eta^- = 0,5$ (chú ý là các phương pháp chọn lọc khác cũng có thể sử dụng được).

Lai ghép: Do chuỗi NST hợp lệ không được phép chứa các gen có giá trị giống nhau nên cần một cải tiến nhỏ các phép lai ghép cơ bản mô tả ở mục 0 để áp dụng vào bài toán tìm đường đi ngắn nhất cho robot. Cụ thể, sau khi áp dụng phép lai ghép thông thường, các gen trùng nhau trên hai chuỗi NST con sẽ được trao đổi cho nhau để được hai chuỗi NST hợp lệ, xem hình 4.35.



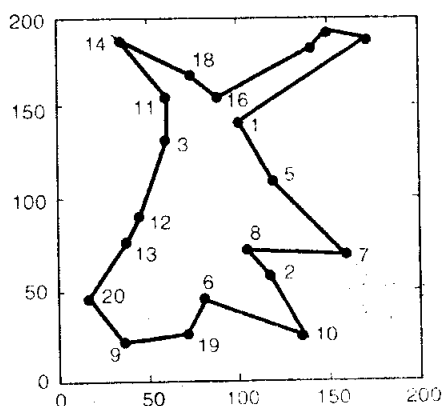
Hình 4.35 Phép lai ghép dùng trong GA hoạch định đường đi robot

Đột biến: Phép đột biến áp dụng cho bài toán hoạch định đường đi cho robot là hoán đổi ngẫu nhiên một hoặc nhiều cặp gen trên chuỗi NST cho nhau, xem hình 4.36.

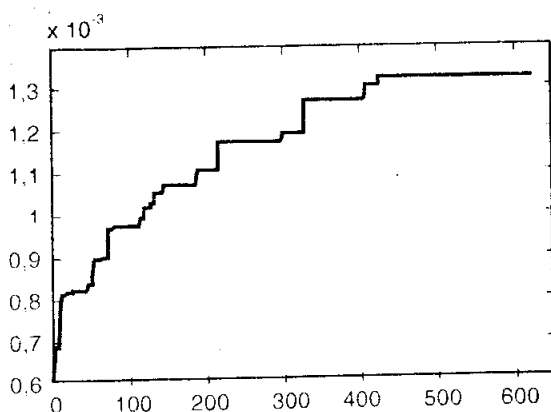


Hình 4.36 Phép đột biến dùng trong GA hoạch định đường đi robot

Kết quả: Hình 4.37 là đường đi ngắn nhất giữa $M = 20$ điểm theo kết quả chạy GA (xem chương trình ở phụ lục A.2). Qua các thế hệ giá trị hàm thích nghi tăng dần đến giá trị cực đại (H.4.38), điều đó có nghĩa là đoạn đường đi giảm dần đến giá trị cực tiểu.



Hình 4.37 Đường đi ngắn nhất giữa 20 điểm hoạch định bởi GA



Hình 4.38 Sự thay đổi hàm thích nghi của bài toán hoạch định đường đi robot

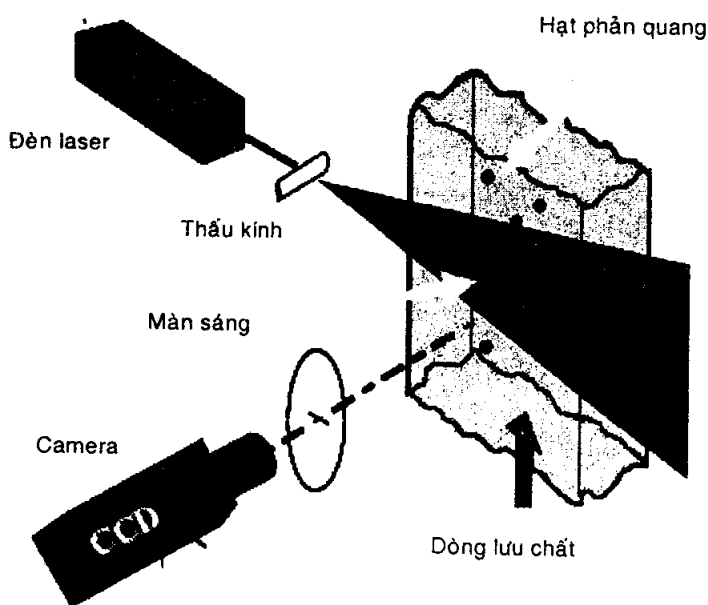
Ví dụ 4.5 Đo tốc độ lưu chất dùng phương pháp xử lý ảnh

Nguyên tắc của phương pháp đo tốc độ lưu chất dùng phương pháp xử lý ảnh (*Particle Image Velocimetry – PIV*) như sau:

- Dùng một đèn laser chiếu vào dòng lưu chất cần đo tốc độ, bên trong lưu chất có các hạt phản quang, các hạt này có kích thước đủ nhỏ và khối lượng đủ nhẹ để chuyển động cùng tốc độ với lưu chất (H.4.39).

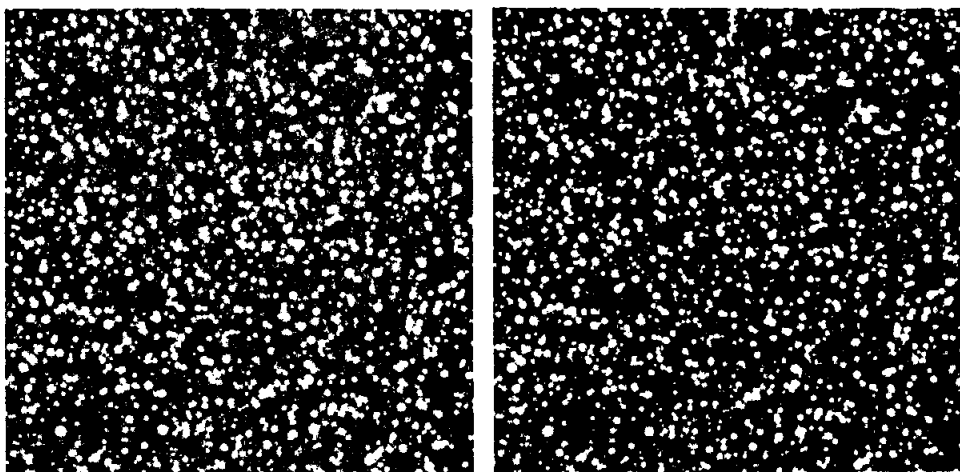
- Hai ảnh liên tiếp của các hạt phản quang bên trong lưu chất được chụp cách nhau một khoảng thời gian Δt dùng camera (H.4.40).

- Xử lý hình ảnh để xác định tốc độ lưu chất qua các bước: xác định vị trí trọng tâm của các hạt phản quang trong hai ảnh chụp (H.4.41); xác định các điểm phản quang tương ứng giữa hai ảnh, từ đó xác định được dịch chuyển của các hạt phản quang; chia dịch chuyển cho thời gian Δt sẽ được tốc độ tức thời của lưu chất tại mọi điểm bên trong vùng được chụp ảnh.

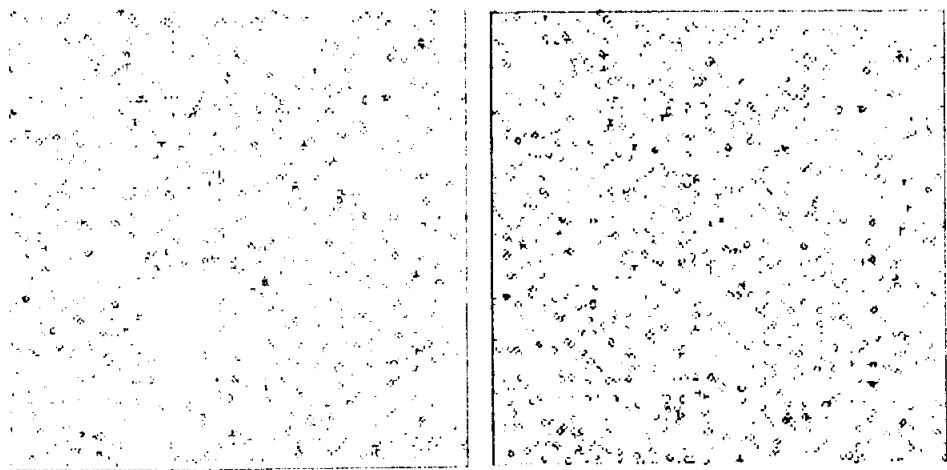


Hình 4.39 Nguyên tắc đo tốc độ lưu chất bằng phương pháp xử lý ảnh

Việc xác định tọa độ các điểm phản quang trên hai ảnh có thể giải được dễ dàng bằng cách chuyển ảnh thành ảnh nhị phân với mức ngưỡng chọn phù hợp. Vấn đề đặt ra là sau khi đã xác định được tọa độ các điểm phản quang ta phải xác định điểm tương ứng giữa hai ảnh. Có nhiều phương pháp giải bài toán tương ứng điểm giữa hai ảnh như phương pháp tương quan, phương pháp dùng mạng thần kinh tự tổ chức (SOM)... Ở đây chúng ta sử dụng giải thuật di truyền để giải bài toán này.



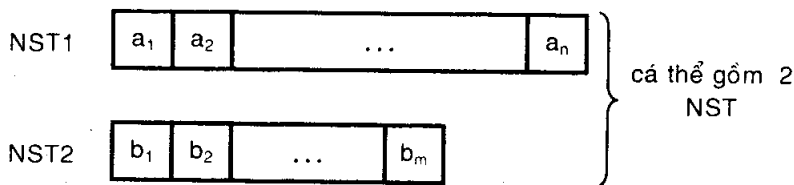
Hình 4.40 Hai ảnh chụp liên tiếp các điểm phản quang trong lưu chất



Hình 4.41 Vị trí trọng tâm của các điểm phản quang trong lưu chất

GA giải bài toán tương ứng hạt phản quang trong phương pháp đo PIV

Mã hóa: Giả sử các hạt phản quang phát hiện được trong hai ảnh lần lượt được đánh số thứ tự từ 1 đến n và 1 đến m , không mất tính tổng quát giả sử $n \geq m$. Lời giải của bài toán tương ứng hạt phản quang là các cặp (a_i, b_i) , với a_i và b_i là số nguyên hai bất kỳ thuộc đoạn $[1, n]$ và $[1, m]$, lời giải này có thể mã hóa thành một cá thể gồm hai chuỗi NST như hình 4.42.



Hình 4.42 Mã hóa lời giải bài toán PIV

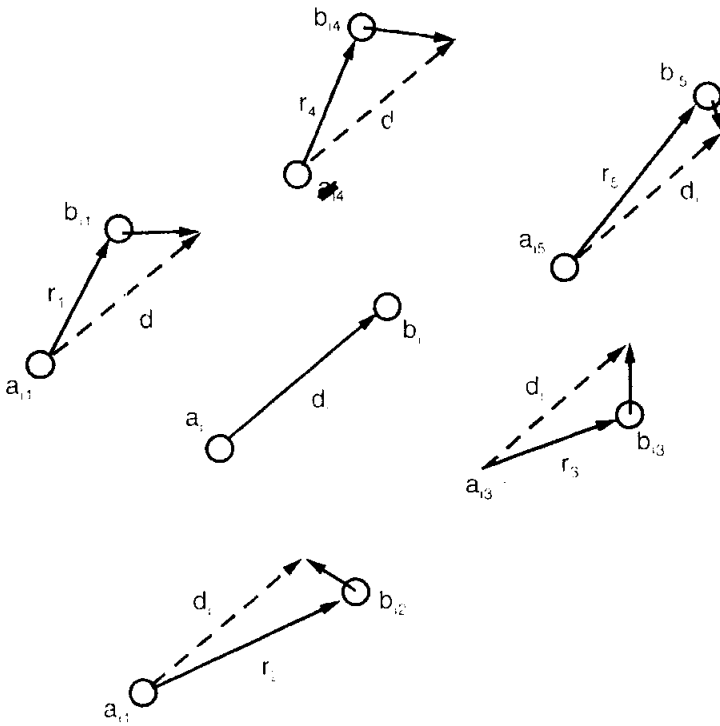
Hàm thích nghi: Hàm thích nghi đơn giản nhất là nghịch đảo tổng khoảng cách giữa các cặp tương ứng:

$$fitness = \left[\sum_{i=1}^m \sqrt{[x(a_i) - x(b_i)]^2 + [y(a_i) - y(b_i)]^2} \right]^{-1} \quad (4.43)$$

Tuy nhiên, hàm thích nghi trên không phù hợp để giải bài toán PIV trong trường hợp mật độ điểm phản quang trong ảnh cao.

Chú ý, dòng chuyển động của lưu chất là dòng chuyển động có hướng, do đó nếu hai điểm phản quang trong hai ảnh là một cặp tương ứng đúng thì trong lân cận xung quanh hai điểm này sẽ có nhiều cặp khác cũng chuyển động cùng hướng. Trên cơ sở nhận xét này, một hàm thích nghi phù hợp hơn với bài toán PIV mật độ cao có thể định nghĩa là nghịch đảo của tổng modul sai lệch giữa vector dịch chuyển của hạt phản quang từ ảnh 1 sang ảnh 2 và các vector dịch chuyển cùng hướng nhất với nó xuất phát từ các hạt phản quang trong lân cận của hạt phản quang ở ảnh 1. Hình 4.43 minh họa ý tưởng hàm thích nghi và nếu trong đó vector dịch chuyển từ hạt phản quang a_i ở ảnh 1 đến hạt phản quang b_i ở ảnh 2 là $\vec{d}_i$, và các vector dịch chuyển từ các hạt phản quang $(a_{i1}, a_{i2}, \dots, a_{ir})$ trong lân cận của hạt a_i ở ảnh 1 đến các hạt phản quang $b_{i1}, b_{i2}, \dots, b_{ir}$ trong lân cận của hạt b_i ở ảnh 2 là $\vec{r}_k$. Sử dụng các ký hiệu trên, hàm thích nghi được xác định bởi biểu thức:

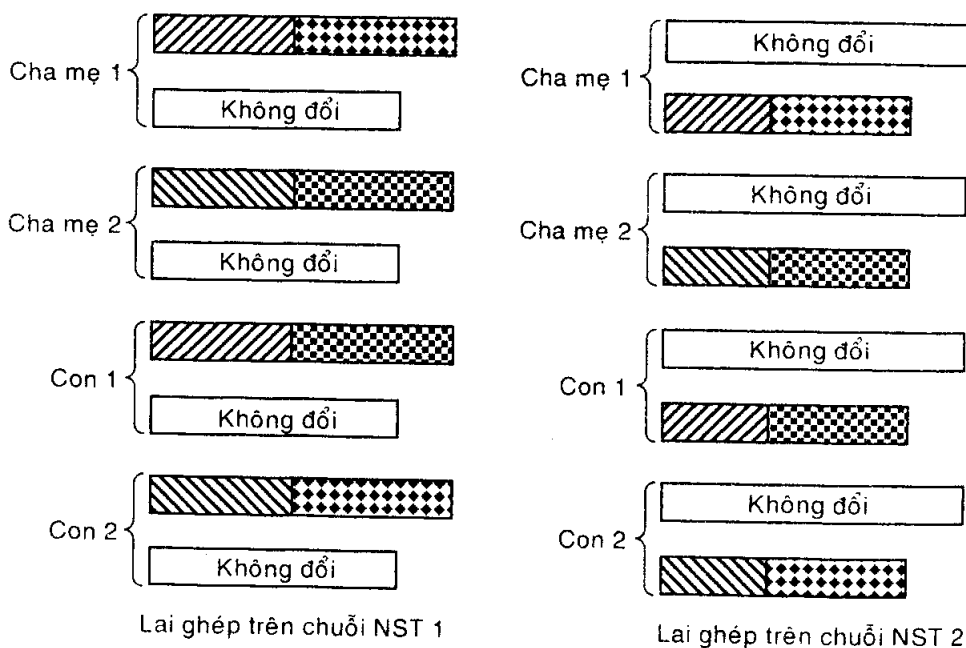
$$fitness = \left[\sum_{i=1}^m \sum_{k=1}^l \left| \vec{d}_i - \vec{r}_k \right| \right]^{-1} \quad (4.44)$$



Hình 4.43 Ý tưởng tính hàm thích nghi

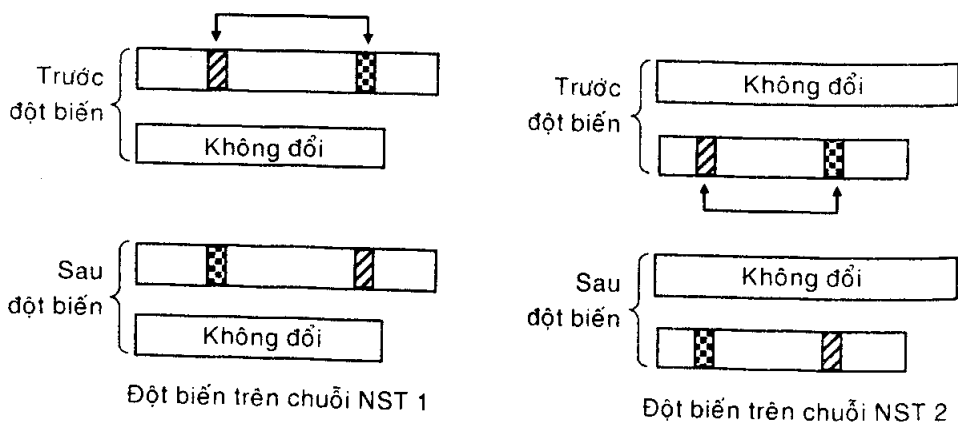
Chọn lọc tự nhiên: Áp dụng phương pháp chọn lọc sắp hạng lũy thừa, đồng thời bảo toàn phần tử ưu việt để đảm bảo thế hệ sau luôn có độ thích nghi tốt hơn thế hệ trước.

Lai ghép: Do mỗi cá thể gồm hai chuỗi NST nên ta cần một biến thể của phép lai ghép thông thường. Phép lai ghép sẽ được thực hiện theo hai hướng lần lượt trên từng chuỗi NST: ở thế hệ này thực hiện trên chuỗi NST thứ nhất, sang thế hệ kế tiếp thực hiện trên chuỗi NST thể thứ hai, và cứ thế tiếp tục. Hình 4.44 minh họa ý tưởng cải tiến phép toán lai ghép một điểm áp dụng vào bài toán PIV. Chú ý rằng sau khi thực hiện phép lai ghép có thể trên chuỗi NST có thể xuất hiện các gen trùng nhau, do đó cần trao đổi các gen trùng nhau giữa hai cá thể con để được chuỗi NST hợp lệ theo cách đã trình bày ở ví dụ 4.3.



Hình 4.44 Phép lai ghép thực hiện theo hai hướng áp dụng vào bài toán PIV

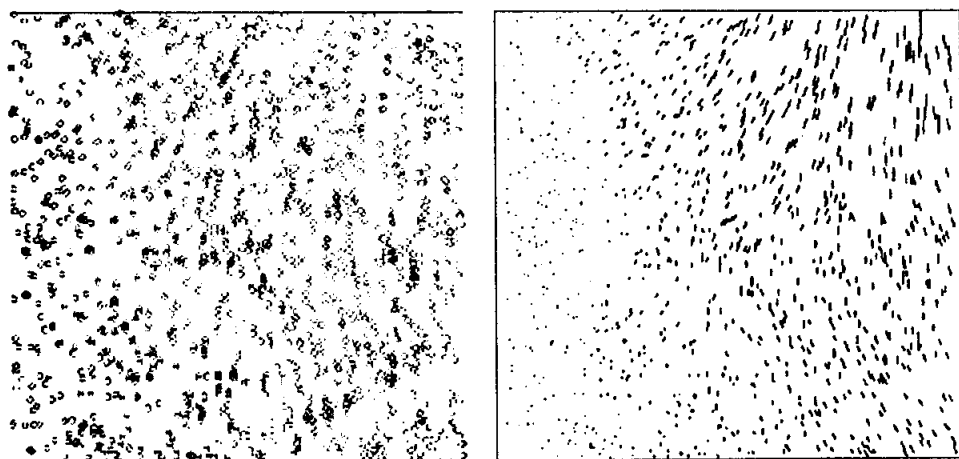
Đột biến: Phép đột biến ngẫu nhiên đổi vị trí của các gen trên chuỗi NST. Tương tự như phép lai ghép, do mỗi cá thể có hai chuỗi NST nên phép lai ghép sẽ lần lượt được thực hiện trên từng chuỗi NST.



Hình 4.45 Phép đột biến thực hiện theo hai hướng áp dụng vào bài toán PIV

Kết quả: Hình 4.46b là kết quả tương ứng hạt phân quang dùng GA. Các thông số cơ bản của GA như sau: kích thước quần thể là 10, số thế hệ tối đa là 2400, số lượng hạt trong lân cận để tính hàm thích nghi là 5, xác suất lai ghép là 0,95; xác suất đột biến là 0,5.

Bằng quan sát đánh giá trực giác, ta thấy kết quả tương ứng hạt ở hình 4.46b phù hợp với tọa độ của các hạt ở hình 4.46a (màu đỏ là các hạt ở ảnh 1, màu xanh là các hạt ở ảnh 2). Theo tính toán kết quả tương ứng đúng 99%.



Hình 4.46 Kết quả tương ứng hạt phản quang trong bài toán PIV

a) Tọa độ các hạt phản quang; b) Kết quả tương ứng hạt phản quang

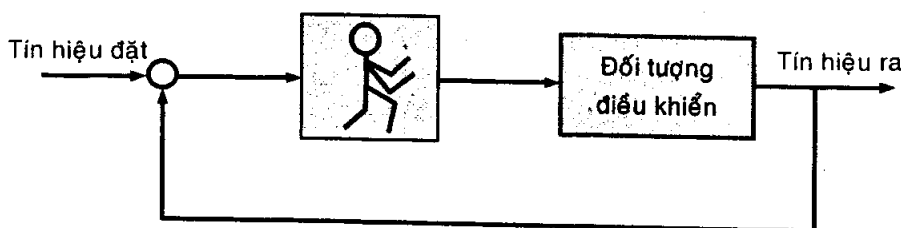
4.6 TÓM TẮT

Chương 4 đã trình bày một cách có hệ thống các vấn đề liên quan đến thuật toán di truyền. Các phương pháp chọn lọc, lai ghép, đột biến sử dụng trong các thuật toán di truyền mã số nhị phân/thập phân và mã số thực được mô tả chi tiết để có thể dựa vào đó lập trình được. Các mở rộng của giải thuật di truyền cũng được đề cập. Một số ví dụ ứng dụng minh họa khả năng của giải thuật di truyền trong việc giải các bài toán các nhau: tìm cực trị hàm, tìm đường đi ngắn nhất, đo tốc độ lưu chất. Trong điều khiển, giải thuật di truyền được sử dụng chủ yếu để chỉnh định tối ưu các bộ điều khiển hoặc có thể sử dụng để ước lượng mô hình. Chương 5 và chương 6 sẽ trình bày cụ thể hơn về các vấn đề này.

ĐIỀU KHIỂN TRỰC TIẾP

5.1 GIỚI THIỆU

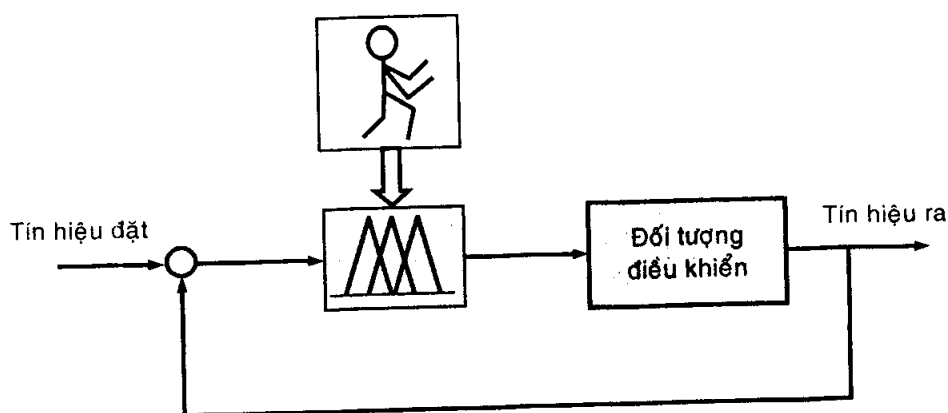
Trong công nghiệp có những đối tượng điều khiển rất phức tạp, việc thiết kế các bộ điều khiển thông thường hầu như không thể thực hiện được do không thể xác định được mô hình toán của đối tượng, hoặc mô hình toán của đối tượng quá phức tạp, có tính phi tuyến cao. Trong khi đó con người với khả năng xử lý tình xảo của bộ não sau khi trải qua quá trình đào tạo và tích lũy kinh nghiệm lại có thể điều khiển được các đối tượng phức tạp mà không cần biết gì về mô tả toán học của đối tượng (H.5.1). Thực tế trên là động lực để các nhà nghiên cứu đưa ra các sơ đồ điều khiển có khả năng phỏng theo khả năng xử lý dựa vào kinh nghiệm và/hoặc khả năng học của con người, đó là các sơ đồ điều khiển dùng logic mờ và/hoặc mạng thần kinh.



Hình 5.1 Hệ thống điều khiển bởi người vận hành

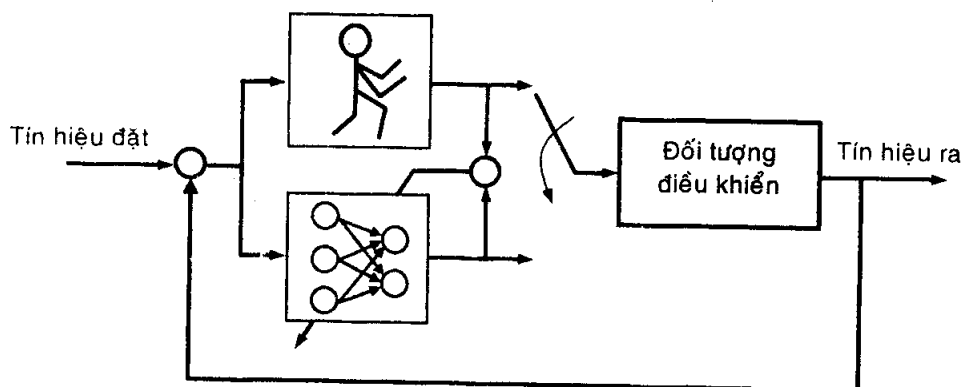
Cách thiết kế hệ thống điều khiển mờ và điều khiển dùng mạng thần kinh hoàn toàn khác nhau. Điều khiển mờ cung cấp phương pháp để biểu diễn, xử lý và thực thi tri thức trực giác của con người. Khác với mạng thần kinh, bộ điều khiển mờ không tự học được kinh nghiệm chuyên gia mà ngược lại kinh nghiệm chuyên gia phải được

tích hợp vào bộ điều khiển mờ trong quá trình thiết kế hệ thống (H.5.2). Nhờ logic mờ, các phát biểu ngôn ngữ định tính mô tả tác động điều khiển được chuyển thành mô hình toán học. Quá trình thiết kế bộ điều khiển mờ dựa vào kinh nghiệm là quá trình thử sai, và khi quá trình thiết kế thành công ta được bộ điều khiển mờ có thể điều khiển được đối tượng với chất lượng chấp nhận được. Về nguyên tắc, hệ thống điều khiển mờ cũng không có gì khác so với hệ thống điều khiển thông thường khác. Sự khác biệt ở đây là bộ điều khiển mờ làm việc có tư duy như “bộ não” dưới dạng trí tuệ nhân tạo. Nếu khẳng định với bộ điều khiển mờ có thể giải quyết mọi vấn đề từ trước đến nay chưa giải quyết được theo phương pháp kinh điển thì không hoàn toàn chính xác, vì hoạt động của bộ điều khiển phụ thuộc vào kinh nghiệm và phương pháp rút ra kết luận theo tư duy con người, sau đó được cài đặt vào máy tính dựa trên cơ sở logic mờ.



Hình 5.2 Bộ điều khiển mờ được thiết kế dựa vào kinh nghiệm chuyên gia

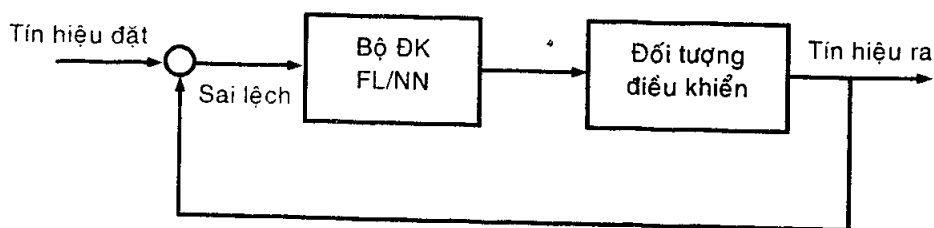
Trái với bộ điều khiển mờ, bộ điều khiển mạng thần kinh được thiết kế nhờ vào khả năng học của mạng thần kinh. Qua quá trình huấn luyện mạng dựa vào dữ liệu là sai số giữa tín hiệu đặt và tín hiệu ra và tác động điều khiển của con người, trọng số của mạng thần kinh được cập nhật và mạng học được kinh nghiệm của chuyên gia trong việc điều khiển đối tượng (H.5.3). Sau khi quá trình huấn luyện kết thúc thành công, có thể sử dụng mạng thần kinh để điều khiển đối tượng thay thế con người.



Hình 5.3 Mạng thần kinh học kinh nghiệm điều khiển của chuyên gia

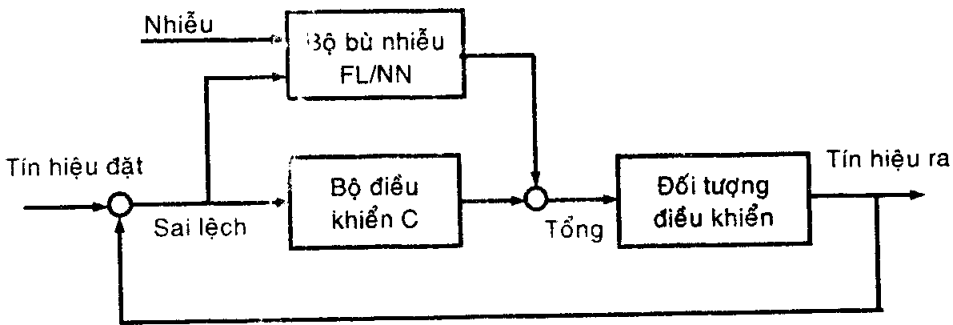
Bộ điều khiển mờ/thần kinh có thể dùng trong các sơ đồ điều khiển khác nhau. Sau đây là ba sơ đồ điều khiển thường gặp.

Điều khiển trực tiếp (Direct Control): Bộ điều khiển mờ/thần kinh được dùng trong đường thuận của hệ thống điều khiển hồi tiếp. Tín hiệu ra của đối tượng điều khiển được so sánh tín hiệu đặt, nếu có sai lệch thì bộ điều khiển mờ/thần kinh sẽ xuất tín hiệu tác động vào đối tượng nhằm mục đích làm sai lệch giảm về 0. Đây là sơ đồ điều khiển rất quen thuộc, trong sơ đồ này, bộ điều khiển mờ/thần kinh được dùng để thay thế bộ điều khiển kinh điển.



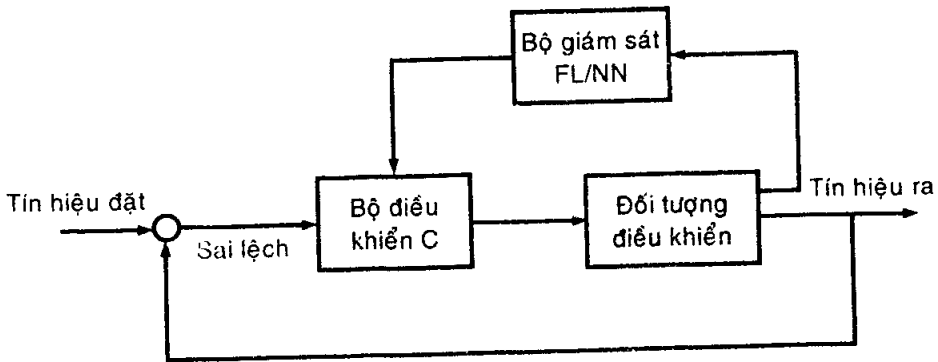
Hình 5.4 Điều khiển trực tiếp (Direct Control)

Điều khiển bù nhiễu: Sơ đồ điều khiển này nhằm mục đích bù ảnh hưởng của nhiễu đo được. Điều này cần mô hình chính xác, nhưng nếu việc xây dựng mô hình toán quá khó khăn hoặc đắt tiền thì có thể dùng mô hình mờ. Hình 5.5 trình bày hệ thống điều khiển với bộ điều khiển kinh điển và bộ bù nhiễu mờ/thần kinh. Khi bỏ qua ngõ vào nhiễu, hệ thống trên có thể xem là sự kết hợp tác động điều khiển tuyến tính và phi tuyến; bộ điều khiển C có thể là bộ điều khiển PID và bộ bù nhiễu FL/NN đóng vai trò bộ điều khiển phi tuyến phụ.



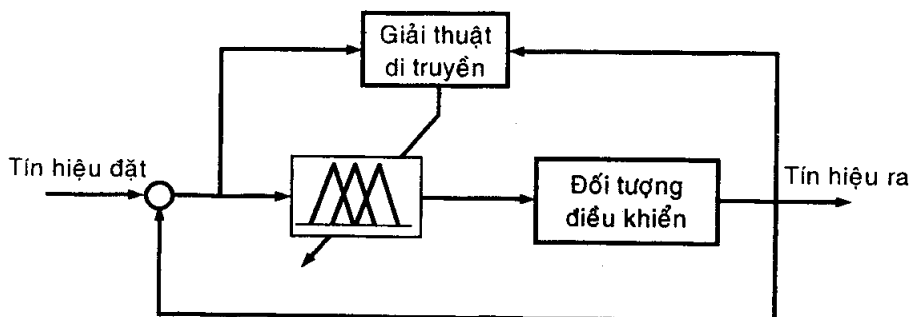
Hình 5.5 Điều khiển bù nhiễu mờ

Điều khiển thích nghi: Các quy tắc mờ cũng có thể dùng để hiệu chỉnh thông số của bộ điều khiển tuyến tính trong sơ đồ điều khiển thích nghi. Nếu một đối tượng phi tuyến thay đổi điểm làm việc, để chất lượng điều khiển tốt thì thông số của bộ điều khiển phải thay đổi theo. Hình 5.6 là sơ đồ điều khiển thích nghi với bộ giám sát mờ/thần kinh.



Hình 5.6 Điều khiển thích nghi mờ/thần kinh

Một trong những khó khăn khi thiết kế bộ điều khiển mờ là đưa ra hệ quy tắc điều khiển và chỉnh định thông số của các hàm liên thuộc mô tả các giá trị ngôn ngữ sử dụng trong các quy tắc để hệ thống điều khiển cho kết quả tốt. Để giải quyết khó khăn này có thể sử dụng giải thuật di truyền để tự chỉnh bộ điều khiển mờ sao cho tối thiểu một chỉ tiêu chất lượng nào đó, thông thường là hàm toàn phương của sai số và/hoặc năng lượng điều khiển (H.5.7). Thuật toán di truyền cũng rất hữu ích trong việc huấn luyện mạng thần kinh để tìm bộ trọng số tối ưu toàn cục.



Hình 5.7 Tự chỉnh bộ điều khiển mờ dùng giải thuật di truyền

Khuyết điểm của bộ điều khiển dùng mạng thần kinh và logic mờ vừa trình bày ở trên là rất khó đánh giá được tính ổn định của hệ thống, nguyên nhân chủ yếu là do không xác định được mô hình toán học của đối tượng. Và giả sử nếu biết được đặc tính động của đối tượng thì việc phân tích tính ổn định của hệ thống cũng gặp nhiều khó khăn do tính phi tuyến của bộ điều khiển. Thông thường việc phân tích tính ổn định chỉ có thể thực hiện được với các hệ thống đơn giản (cả đối tượng và bộ điều khiển). Các kết quả phân tích ổn định này chủ yếu có ý nghĩa về mặt lý thuyết mà không có ý nghĩa thực tế vì nếu đối tượng đơn giản thì không cần đến bộ điều khiển mờ hay mạng thần kinh.

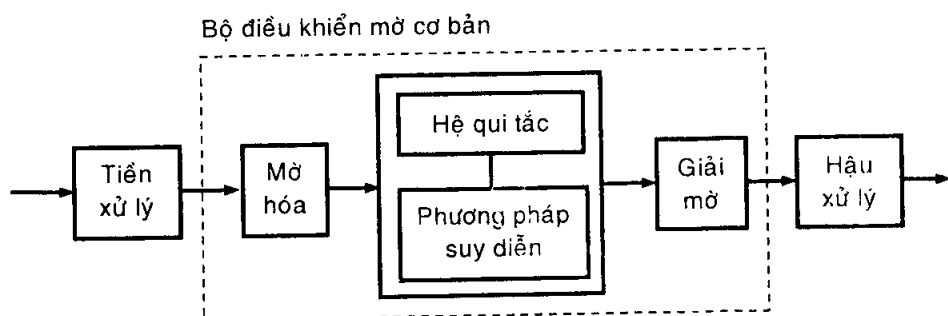
Hiện nay các bộ điều khiển trực tiếp dùng logic mờ/thần kinh được sử dụng rất rộng rãi trong công nghiệp và dân dụng vì sự đơn giản và tính hiệu quả của phương pháp điều khiển này. Từ năm 1990 đến nay, rất nhiều nghiên cứu đưa ra các sơ đồ điều khiển mờ/thần kinh thích nghi thiết kế dựa vào lý thuyết ổn định Lyapunov, các sơ đồ điều khiển này sẽ được trình bày ở chương 8. Với sự xuất hiện của các sơ đồ điều khiển thích nghi ổn định, các bộ điều khiển mờ/thần kinh mở rộng phạm vi ứng dụng cả sang lĩnh vực quân sự và chinh phục vũ trụ. Dưới đây là một số ứng dụng thành công của phương pháp điều khiển mờ/thần kinh đã được ghi nhận:

- Điều khiển quá trình: nhiệt độ, áp suất, mức chất lỏng, chuẩn đoán lỗi, điều khiển cột chưng cất...
- Hệ thống sản xuất
- Hệ thống điện: điều khiển động cơ, điều khiển/phân phối điện năng, ước lượng phụ tải

- Điều khiển máy bay, tàu vũ trụ
- Giao thông: điều khiển lái tự động, thắng
- Robot: điều khiển vị trí và hoạch định quỹ đạo
- Các thiết bị điện tử dân dụng: máy giặt, máy lạnh, máy ảnh, tivi...

5.2 ĐIỀU KHIỂN MỜ TRỰC TIẾP

5.2.1 Cấu trúc bộ điều khiển mờ trực tiếp



Hình 5.8 Sơ đồ khối bộ điều khiển mờ

Sơ đồ khối của bộ điều khiển mờ trình bày ở hình 5.8 gồm thành phần chính là bộ điều khiển mờ cơ bản với ba khối chức năng là mờ hóa, hệ quy tắc và giải mờ. Thực tế trong một số trường hợp khi ghép bộ điều khiển mờ vào hệ thống điều khiển cần thêm hai khối tiền xử lý và hậu xử lý. Chức năng của từng khối trong sơ đồ trên được mô tả sau đây:

Bộ điều khiển mờ cơ bản

Thành phần chính của bộ điều khiển mờ cơ bản là hệ quy tắc điều khiển, hệ quy tắc này có thể rút ra từ kinh nghiệm chuyên gia trong việc điều khiển đối tượng. Khâu mờ hóa chuyển giá trị rõ phản hồi từ ngõ ra của đối tượng thành giá trị mờ để hệ quy tắc có thể suy luận được. Khâu giải mờ chuyển giá trị mờ suy luận được ở ngõ ra của hệ quy tắc thành giá trị rõ để điều khiển đối tượng. Chi tiết về các loại quy tắc điều khiển, các phương pháp mờ hóa và giải mờ đã trình bày ở chương 2.

Khối tiền xử lý (Preprocessing)

Bộ điều khiển mờ cơ bản là bộ điều khiển tĩnh. Để có thể điều khiển động, cần có thêm các tín hiệu vi phân, tích phân của giá trị đo, những tín hiệu này được tạo ra bởi các mạch vi phân, tích phân trong khối tiền xử lý. Ngoài ra khối tiền xử lý còn có thể lượng tử hóa hoặc làm tròn giá trị đo, chuẩn hóa hoặc tỷ lệ giá trị đo vào tầm giá trị chuẩn, lọc nhiễu. Các tín hiệu ra của bộ tiền xử lý sẽ được đưa vào bộ điều khiển mờ cơ bản, và cần chú ý rằng các tín hiệu này vẫn là giá rõ.

Việc lượng tử hóa cần thiết để biến đổi giá trị đo liên tục thành giá trị thích hợp nhất trong tập cơ sở rời rạc. Ví dụ, giả sử biến x có giá trị là 4.6, nhưng tập cơ sở rời rạc của biến x là $\{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}$, bộ tiền xử lý sẽ là tròn x thành 5. Trong ví dụ trên, nếu không lượng tử hóa thì x sẽ có vô số giá trị trong đoạn $[-5, 5]$, còn việc lượng tử hóa làm cho x chỉ có thể nhận 11 giá trị rời rạc, và như vậy sẽ dẫn đến việc tính toán dễ dàng hơn. Tuy nhiên, nếu bước lượng tử quá lớn có thể làm hệ thống dao động thậm chí không ổn định.

Khối hậu xử lý (Postprocessing)

Trong trường hợp các giá trị mờ ở ngõ ra của các quy tắc được định nghĩa trên tập cơ sở chuẩn thì giá trị rõ sau khi giải mờ phải được nhân với một hệ số tỷ lệ để trở thành giá trị vật lý. Ví dụ, cơ sở chuẩn $[-1, 1]$ (không thứ nguyên) cần phải được tỷ lệ để trở thành giá trị vật lý $[-10, 10]$ (volt).

Khối hậu xử lý thường gồm các mạch khuếch đại (có thể chỉnh độ lợi), đôi khi khối hậu xử lý có thể có khâu tích phân.

5.2.2 Thiết kế bộ điều khiển mờ trực tiếp dựa vào kinh nghiệm chuyên gia

Khi thiết kế bộ điều khiển mờ, chúng ta chỉ mong muốn được bộ điều khiển cho kết quả **“chấp nhận được”** chứ không phải kết quả **“tốt nhất”**. Mặt khác, bài toán ổn định của hệ thống điều khiển mờ vẫn còn là bài toán mở. Vì vậy chỉ nên sử dụng bộ điều khiển mờ khi kết quả điều khiển bằng các phương pháp kinh điển không thỏa mãn yêu cầu thiết kế.

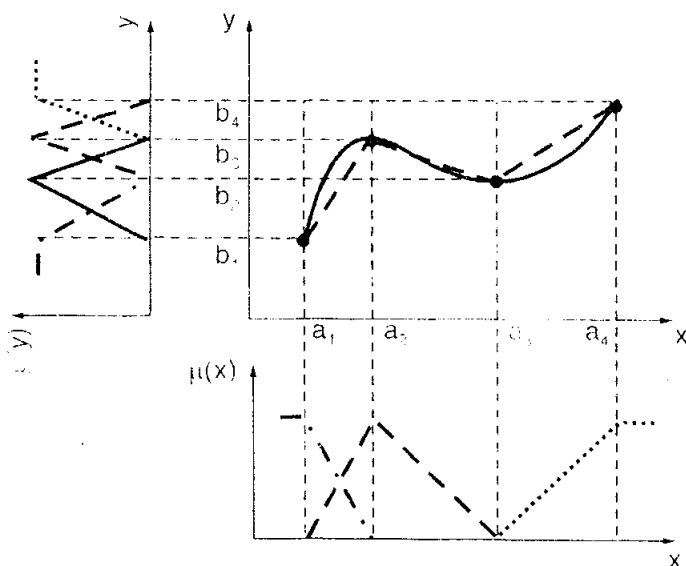
Rất khó có thể đưa ra được phương pháp thiết kế hệ thống điều khiển mờ tổng quát. Một bộ điều khiển mờ được thiết kế tốt hay không hoàn toàn phụ thuộc vào kinh nghiệm của người thiết kế. Mục này chỉ đưa ra một số đề nghị về trình tự thiết kế một bộ điều khiển mờ.

Trình tự thiết kế

- 1- Xác định biến vào, biến ra (và biến trạng thái, nếu cần) của đối tượng.
- 2- Chuẩn hóa biến vào, biến ra về miền giá trị $[0,1]$ hay $[-1,1]$ để sau này có thể lập trình dễ dàng bằng vi xử lý hoặc tích hợp vào các hệ PLC.
- 3- Định nghĩa các tập mờ trên tập cơ sở đã chuẩn hóa của các biến, và gán cho mỗi tập mờ một giá trị ngôn ngữ. Số lượng, vị trí và hình dạng của các tập mờ tùy thuộc vào từng ứng dụng cụ thể. Một đề nghị là nên bắt đầu bằng 3 tập mờ có dạng hình tam giác cho mỗi biến và các tập mờ này nên được phân hoạch mờ. Nếu không thỏa mãn yêu cầu thì có thể tăng số lượng tập mờ, thay đổi hình dạng.
- 4- Gán quan hệ giữa các tập mờ ở ngõ vào và ngõ ra, bước này xây dựng được hệ quy tắc mờ. Bước này có thể thực hiện tốt nếu người thiết kế có kinh nghiệm về các phát biểu ngôn ngữ mô tả đặc tính động của đối tượng và các hệ quy tắc mờ thông dụng, ví dụ các hệ quy tắc PD, PI hoặc PID mờ (mục 5.3.1).
- 5- Chọn phương pháp suy diễn. Trong thực tế người ta thường chọn phương pháp suy diễn cục bộ nhằm đơn giản trong việc tính toán và áp dụng công thức hợp thành MAX-MIN hay MAX-PROD.
- 6- Chọn phương pháp giải mờ. Trong điều khiển người ta thường chọn phương pháp giải mờ “thỏa hiệp” như phương pháp trọng tâm, phương pháp trung bình có trọng số...

Trong các bước thiết kế vừa nêu trên, bước 1, 3 và 4 là quan trọng nhất. Bước 2 chuẩn hóa ngõ vào, ngõ ra nên làm chứ không bắt buộc. Ta hoàn toàn có thể thiết kế bộ điều khiển mờ với các biến vào ra là các biến vật lý chứ không phải là biến chuẩn hóa và kết quả điều khiển là như nhau.

Chú ý: Nếu chúng ta biết trước quan hệ giữa ngõ vào và ngõ ra của bộ điều khiển cho kết quả chấp nhận được (ví dụ như quan hệ vào ra ở hình 5.9) thì bước 1, 3 và 4 có thể thực hiện như ví dụ sau (chú ý là bộ điều khiển trong ví dụ này có một ngõ vào một ngõ ra, tuy nhiên phương pháp này có thể mở rộng cho hệ nhiều ngõ vào, nhiều ngõ ra):



Hình 5.9 Cách chọn các tập mờ và xây dựng hệ quy tắc mờ khi biết quan hệ giữa ngõ vào và ngõ ra của bộ điều khiển

- Chọn các điểm đặc tính, trong hình 5.9 ta chọn các điểm đặc tính là (a_1, b_1) , (a_2, b_3) , (a_3, b_2) , (a_4, b_4) .
- Tại các điểm đặc tính ta có thể phát biểu:

Nếu x bằng a_i thì y bằng b_j ($i = 1..4$; $j = 1..4$)

Các phát biểu trên có thể chuyển thành các quy tắc mờ như sau:

Nếu x “gần bằng a_i ” thì y “gần bằng b_j ”

- Từ các quy tắc mờ vừa đưa ra ở trên ta thấy cần phải định nghĩa các tập mờ “gần bằng a_i ” ($i = 1..4$) và các tập mờ “gần bằng b_j ” ($j = 1..4$). Các tập mờ này có thể chọn có dạng tam giác và được phân hoạch mờ.

- Cuối cùng nếu cần thiết thì gán giá trị ngôn ngữ cho các tập mờ vừa định nghĩa và phát biểu lại hệ quy tắc mờ dùng các giá trị ngôn ngữ này.

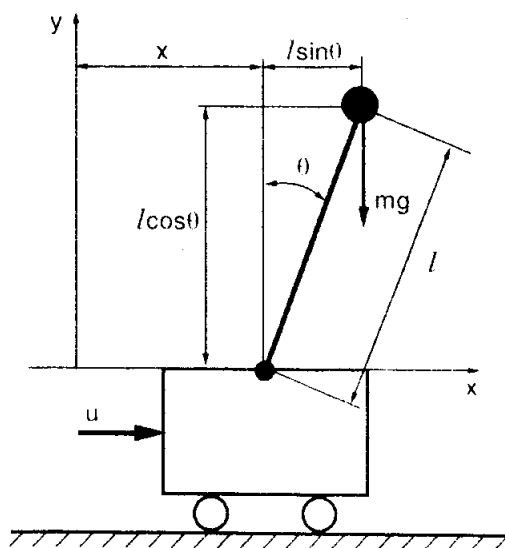
Qua phần trình bày ở trên ta thấy việc thiết kế bộ điều khiển mờ trực tiếp mang tính thử sai vì việc chọn số lượng, hình dạng, vị trí tập mờ; chọn hệ quy tắc,... đều phụ thuộc vào kinh nghiệm của người thiết kế, mà kinh nghiệm chỉ có được qua quá trình “thử và sai”. Ngay cả khi có thể chọn tập mờ và hệ quy tắc mờ từ quan hệ giữa ngõ vào và ngõ ra thì việc thiết kế cũng mang tính thử sai vì mối quan hệ giữa ngõ vào và ngõ ra cho đáp ứng tốt chấp nhận được không phải tự nhiên mà có được, quan hệ này phải được tích lũy từ kinh nghiệm của người vận hành.

Tóm lại, bộ điều khiển mờ là bộ điều khiển dựa trên cơ sở toán học là lý thuyết tập mờ và logic mờ. Bộ điều khiển mờ bắt chước sự xử lý thông tin và điều khiển của con người, vì vậy thích hợp để điều khiển những đối tượng phức tạp mà các phương pháp kinh điển không cho được kết quả mong muốn. Phương pháp thiết kế bộ điều khiển mờ mang nặng tính thử sai, vì vậy nếu người thiết kế không có kinh nghiệm thì rất khó có thể tổng hợp được bộ điều khiển mờ cho kết quả tốt.

Bộ điều khiển mờ có ưu điểm là có tính bền vững, tuy nhiên không có phương pháp nào cho phép phân tích tính ổn định của hệ thống điều khiển mờ nếu không biết mô hình toán của đối tượng. Trong trường hợp có thể xác định được mô hình toán của đối tượng thì cũng không có kết quả tổng quát trong việc phân tích và thiết kế hệ thống điều khiển mờ do bộ điều khiển mờ là bộ điều khiển có tính phi tuyến cao. Việc chứng minh tính ổn định cho các bộ điều khiển mờ trong các công trình nghiên cứu đã được công bố chỉ hạn chế trong trường hợp bộ điều khiển mờ đơn giản, như điều khiển PID mờ, và bản thân đối tượng là ổn định. Trong phần lớn các trường hợp như trên, việc chứng minh tính ổn định không có ý nghĩa đáng kể vì sự đơn giản của bộ điều khiển và đối tượng. Nếu đối tượng điều khiển không thể mô tả bằng mô hình toán học thì việc chứng minh tính ổn định không thể thực hiện được. Thật ra đối với các hệ thống điều

khuyến trong công nghiệp, người ta không xem việc chứng minh hệ thống ổn định về mặt toán học như là một điều kiện cần và đủ để chấp nhận hệ thống. Việc chứng minh tính ổn định về mặt toán học của hệ thống đơn thuần chỉ là quan điểm của các nhà nghiên cứu. Trong thực tế, cách kiểm tra tính ổn định thực nghiệm quan trọng hơn việc phân tích ổn định về mặt toán học.

Ví dụ 5.1 Thiết kế bộ điều khiển mờ giữ cân bằng hệ con lắc ngược.



M- trọng lượng xe [Kg]; m- trọng lượng con lắc [Kg]; l- chiều dài con lắc [m];
u- lực tác động vào xe [N]; g- gia tốc trọng trường [m/s^2]; x- vị trí xe [m];
 θ - góc giữa con lắc và phương thẳng đứng [rad]

Hình 5.10 Hệ con lắc ngược

Xét hệ thống con lắc ngược như hình 5.10, hệ thống gồm một con lắc có trục quay tự do được gắn vào xe kéo bởi động cơ điện. Chúng ta chỉ xét bài toán trong không gian hai chiều, nghĩa là con lắc chỉ di chuyển trong mặt phẳng. Con lắc không ổn định vì nó luôn ngã xuống trừ khi có lực tác động thích hợp. Giả sử khối lượng con lắc tập trung ở đầu thanh như hình vẽ (khối lượng thanh không đáng kể). Lực điều khiển u tác động vào xe. Yêu cầu của bài toán là điều khiển vị trí của xe và giữ cho con lắc luôn thẳng đứng. Bài toán

điều khiển hệ con lắc ngược chính là mô hình của bài toán điều khiển định hướng tàu vũ trụ khi được phóng vào không gian.

Mô hình toán học của hệ con lắc ngược cho bởi các phương trình sau:

$$(M + m)\ddot{x} - ml(\sin \theta)\dot{\theta}^2 + ml(\cos \theta)\ddot{\theta} = u \quad (5.1)$$

$$m\ddot{x} \cos \theta + ml\ddot{\theta} = mg \sin \theta \quad (5.2)$$

Từ (5.1) và (5.2), ta có thể dễ dàng tính được:

$$\ddot{x} = \frac{u + ml(\sin \theta)\dot{\theta}^2 - mg \cos \theta \sin \theta}{M + m - m(\cos \theta)^2} \quad (5.3)$$

$$\ddot{\theta} = \frac{u \cos \theta - (M + m)g(\sin \theta) + ml(\cos \theta \sin \theta)\dot{\theta}}{ml(\cos \theta)^2 - (M + m)l} \quad (5.4)$$

Hai biểu thức (5.3) và (5.4) được sử dụng để viết chương trình mô phỏng đặc tính động học hệ con lắc ngược.

Thiết kế bộ điều khiển mờ:

Áp dụng phương pháp thiết kế đã trình bày trong mục trước:

1- Chọn các biến vào/ra: Ta chọn bốn biến vào và góc lệch của con lắc, vận tốc góc của con lắc, vị trí xe và tốc độ xe, biến ra là lực tác dụng vào xe. Tập cơ sở của các biến phụ thuộc chủ yếu vào đối tượng, ta chọn như sau:

Góc lệch: $[-0,3, 0,3]$ (rad)

Vận tốc góc: $[-1, 1]$ (rad/sec)

Vị trí xe: $[-3, 3]$ (m)

Vận tốc xe: $[-3, 3]$ (m/sec)

Lực tác dụng: $[-40, 40]$ (N)

2- Chuẩn hóa tập cơ sở của các biến vào/ra về miền $[-1, 1]$, do đó các ma trận hệ số khuếch đại của các khối tiền xử lý và hậu xử lý như sau:

Các hệ số tiền xử lý: $K_1 = \frac{1}{0,3}$; $K_2 = 1$; $K_3 = \frac{1}{3}$; $K_4 = \frac{1}{3}$

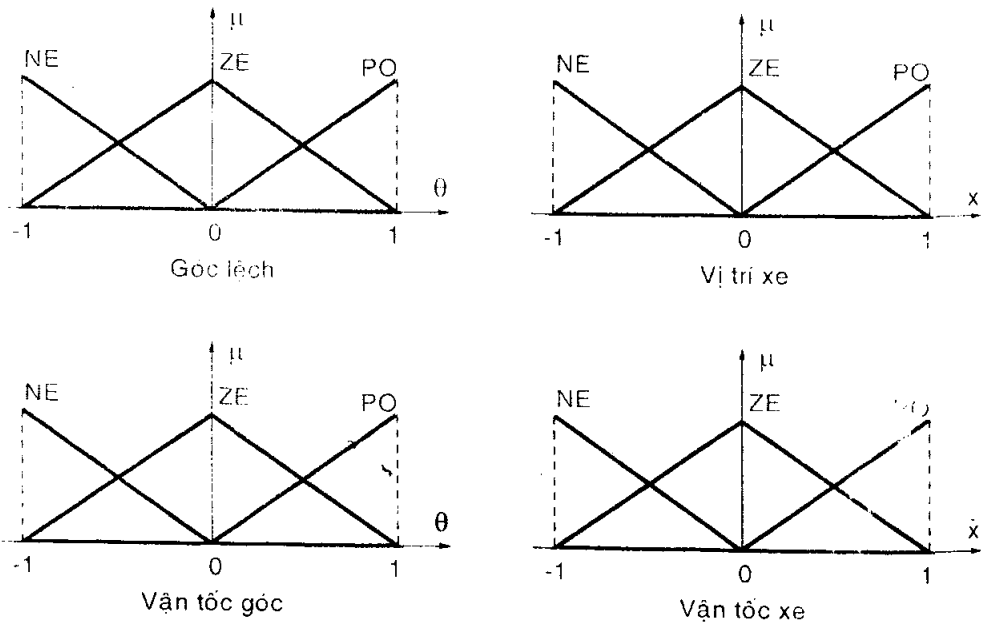
Hệ số hậu xử lý: $K_5 = 40$

3- Định nghĩa các tập mờ mô tả các giá trị ngôn ngữ của các biến vào và biến ra của bộ điều khiển mờ. Chọn 3 tập mờ (giá trị ngôn ngữ) cho mỗi biến vào, các tập mờ này được phân hoạch mờ trên tập cơ sở chuẩn hóa và hàm liên thuộc có dạng tam giác (H.5.11). Số tập mờ cho biến ở ngõ ra được chọn bằng 7, dạng hàm liên thuộc của tập mờ ở ngõ ra được chọn là dạng vạch để thuận lợi trong việc giải mờ (H.5.12).

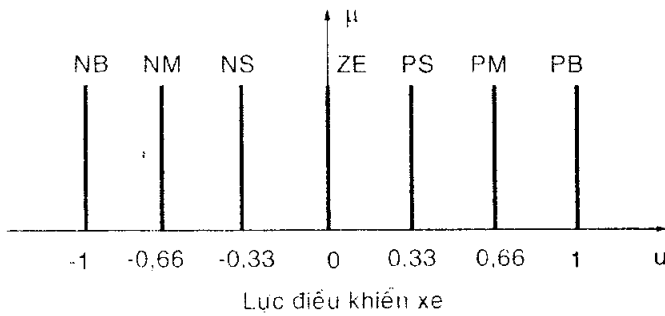
4- Đưa ra hệ quy tắc mờ bằng phương pháp thử sai, dựa vào kinh nghiệm, xem hình 5.13. Với ba tập mờ ở ngõ vào cho mỗi biến, để thỏa mãn tính hoàn chỉnh thì hệ quy tắc phải gồm 81 quy tắc (xem bảng 5.1). Chúng ta cũng không nên tăng số tập mờ ở ngõ vào vì như thế quy tắc mờ sẽ tăng lên rất lớn.

5- Chọn *phương pháp suy diễn MAX-MIN* (cũng có thể chọn phương pháp suy diễn MAX-PROD và kết quả thu được không sai khác nhau nhiều).

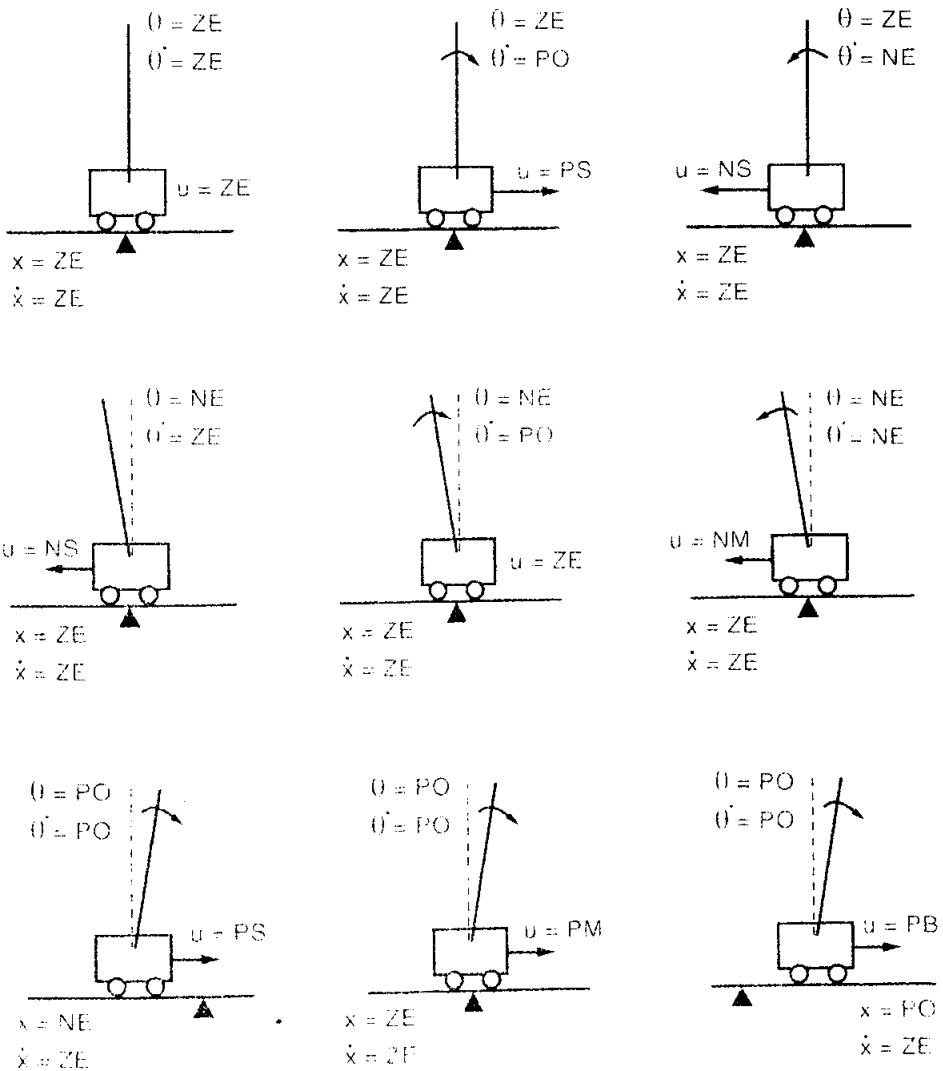
6- Vì ta ứng dụng hệ quy tắc mờ trong điều khiển và khối hậu xử lý không có khâu tích phân nên chọn *phương pháp giải mờ trọng tâm (COA)*.



Hình 5.11 Các tập mờ chuẩn hóa của các biến ở ngõ vào



Hình 5.12 Các tập mờ chuẩn hóa của biến ở ngõ ra

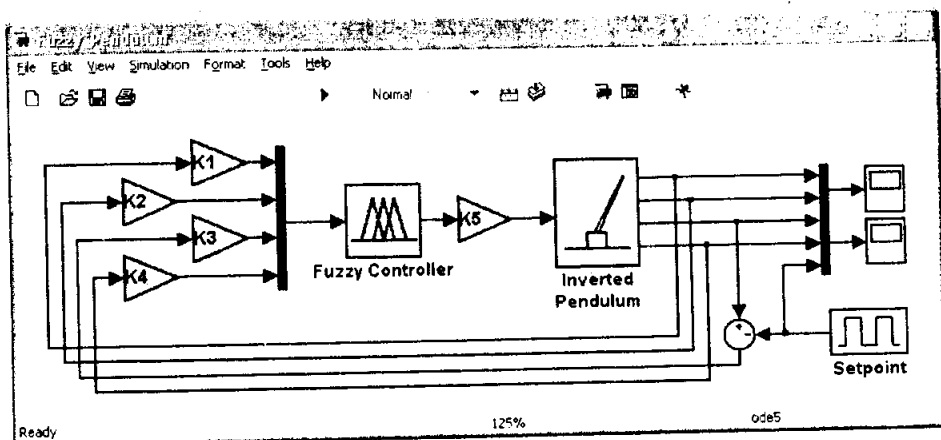


Hình 5.13 Một số quy tắc điều khiển

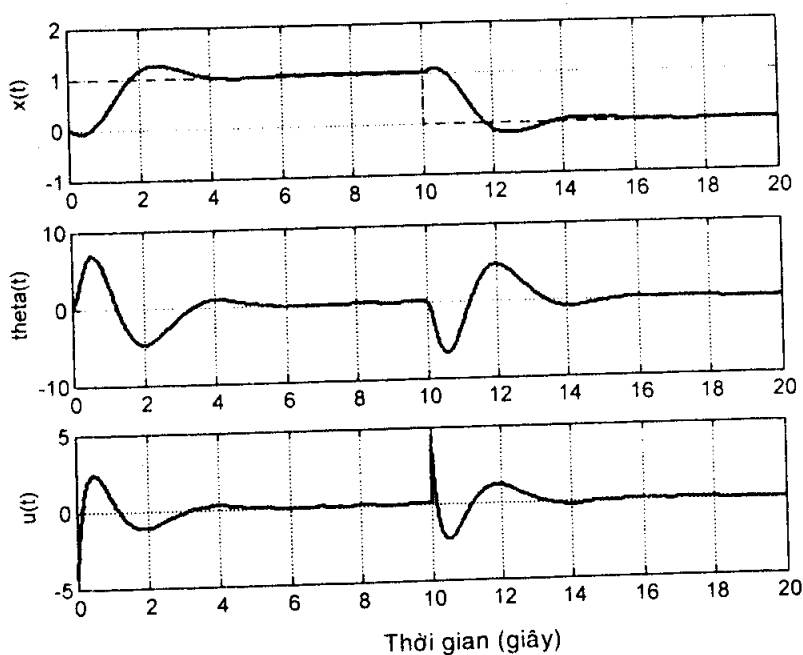
Bảng 5.1 Các quy tắc giữ cân bằng hệ con lắc ngược

STT	θ	$\Delta\theta$	x	Δx	u
1	NE	NE	NE	NE	NB
2	NE	NE	NE	ZE	NB
3	NE	NE	NE	PO	NM
4	NE	NE	ZE	NE	NB
5	NE	NE	ZE	ZE	NM
6	NE	NE	ZE	PO	NE
7	NE	NE	PO	NE	NM
8	NE	NE	PO	ZE	NE
9	NE	NE	PO	PO	ZE
10	NE	ZE	NE	NE	NB
11	NE	ZE	NE	ZE	NM
12	NE	ZE	NE	PO	NE
13	NE	ZE	ZE	NE	NM
14	NE	ZE	ZE	ZE	NE
15	NE	ZE	ZE	PO	ZE
16	NE	ZE	PO	NE	NE
17	NE	ZE	PO	ZE	ZE
18	NE	ZE	PO	PO	PO
19	NE	PO	NE	NE	NM
20	NE	PO	NE	ZE	NE
21	NE	PO	NE	PO	ZE
22	NE	PO	ZE	NE	NE
23	NE	PO	ZE	ZE	ZE
24	NE	PO	ZE	PO	PO
25	NE	PO	PO	NE	ZE
26	NE	PO	PO	ZE	PO
27	NE	PO	PO	PO	PM
28	ZE	NE	NE	NE	NB
29	ZE	NE	NE	ZE	NM
30	ZE	NE	NE	PO	NE
31	ZE	NE	ZE	NE	NM
32	ZE	NE	ZE	ZE	NE
33	ZE	NE	ZE	PO	ZE
34	ZE	NE	PO	NE	NE
35	ZE	NE	PO	ZE	ZE
36	ZE	NE	PO	PO	PO
37	ZE	ZE	NE	NE	NM
38	ZE	ZE	NE	ZE	NE
39	ZE	ZE	NE	PO	ZE
40	ZE	ZE	ZE	NE	NE
41	ZE	ZE	ZE	ZE	ZE

STT	θ	$\Delta\theta$	x	Δx
42	ZE	ZE	ZE	PO
43	ZE	ZE	PO	NE
44	ZE	ZE	PO	ZE
45	ZE	ZE	PO	PO
46	ZE	PO	NE	NE
47	ZE	PO	NE	ZE
48	ZE	PO	NE	PO
49	ZE	PO	ZE	NE
50	ZE	PO	ZE	ZE
51	ZE	PO	ZE	PO
52	ZE	PO	PO	NE
53	ZE	PO	PO	ZE
54	ZE	PO	PO	PO
55	PO	NE	NE	NE
56	PO	NE	NE	ZE
57	PO	NE	NE	PO
58	PO	NE	ZE	NE
59	PO	NE	ZE	ZE
60	PO	NE	ZE	PO
61	PO	NE	PO	NE
62	PO	NE	PO	ZE
63	PO	NE	PO	PO
64	PO	ZE	NE	NE
65	PO	ZE	NE	ZE
66	PO	ZE	NE	PO
67	PO	ZE	ZE	NE
68	PO	ZE	ZE	ZE
69	PO	ZE	ZE	PO
70	PO	ZE	PO	NE
71	PO	ZE	PO	ZE
72	PO	ZE	PO	PO
73	PO	PO	NE	NE
74	PO	PO	NE	ZE
75	PO	PO	NE	PO
76	PO	PO	ZE	NE
77	PO	PO	ZE	ZE
78	PO	PO	ZE	PO
79	PO	PO	PO	NE
80	PO	PO	PO	ZE
81	PO	PO	PO	PO



Hình 5.14 Sơ đồ SIMULINK mô phỏng hệ thống điều khiển hệ con lắc ngược dùng logic mờ



Hình 5.15 Kết quả mô phỏng hệ thống điều khiển giữ cân bằng con lắc ngược

Hình 5.14 là sơ đồ Simulink (MATLAB) mô phỏng hệ thống điều khiển mờ giữ cân bằng hệ con lắc ngược. Khối **[Inverted Pendulum]** mô phỏng đặc tính động học của hệ con lắc ngược theo các phương trình (5.3) và (5.4), bộ điều khiển mờ cơ bản (khối **[Fuzzy Controller]**)

gồm 81 quy tắc trình bày trong bảng 5.1, các ma trận tiền xử lý và hậu xử lý có giá trị như đã trình bày ở bước 2. Kết quả mô phỏng ở hình 5.15 cho thấy bộ điều khiển đã thiết kế có thể điều khiển vị trí xe đúng vị trí đặt đồng thời giữ được trạng thái con lắc cân bằng quanh vị trí thẳng đứng.

5.2.3 Thiết kế bộ điều khiển mờ trực tiếp dùng lý thuyết ổn định Lyapunov

Bên cạnh phương pháp thiết kế được sử dụng rất phổ biến là thử sai dựa vào kinh nghiệm như đã trình bày ở mục 5.2.2, bộ điều khiển mờ trực tiếp còn có thể được thiết kế dựa vào lý thuyết ổn định Lyapunov. Trái với phương pháp thử sai, phương pháp thiết kế dùng lý thuyết Lyapunov có tính hệ thống, đồng thời đảm bảo hệ thống điều khiển sau khi thiết kế ổn định về mặt toán học. Tuy nhiên, để có thể áp dụng được phương pháp thiết kế này điều kiện cần là phải biết được mô tả toán học của đối tượng điều khiển. Thực tế rất khó xác định được mô hình toán học của các đối tượng phức tạp, đây chính là rào cản lớn khiến phương pháp thiết kế Lyapunov mặc dù có nhiều ưu điểm nhưng lại không được sử dụng phổ biến bằng phương pháp thử sai.

Lý thuyết ổn định Lyapunov

Xét hệ thống mô tả bởi phương trình biến trạng thái:

$$\begin{cases} \dot{x}(t) = f(x, u) \\ y(t) = g(x, u) \end{cases} \quad (5.5)$$

Quỹ đạo trạng thái: Nghiệm của (5.5) với giá trị đầu x_0 , ký hiệu là $x(x_0, t)$ được gọi là quỹ đạo trạng thái của hệ tại thời điểm $t=t_0$ đi qua điểm x_0 .

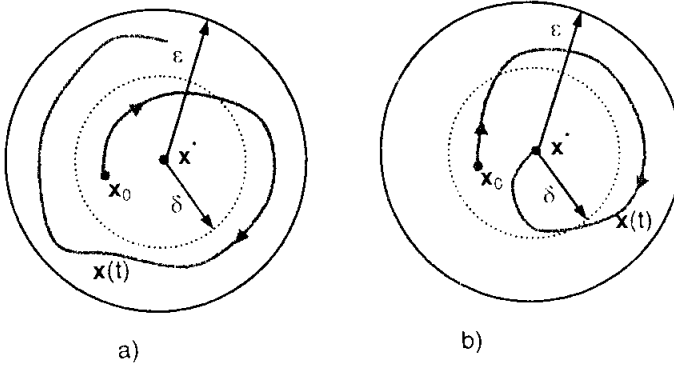
Điểm cân bằng: Trạng thái x^* được gọi là điểm cân bằng của hệ (5.5) nếu:

$$f(x^*, u) \Big|_{u=0} = 0 \quad (5.6)$$

Định nghĩa ổn định Lyapunov: Hệ thống (5.5) được gọi là ổn định (Lyapunov) tại điểm cân bằng x^* nếu và chỉ nếu cho trước $\varepsilon > 0$ bất kỳ, tồn tại $\delta > 0$ sao cho với mọi điểm đầu x_0 thỏa $\|x_0 - x^*\| < \delta$ thì:

$$\|x(x_0, t) - x^*\| < \varepsilon, \quad \forall t > 0 \quad (5.7)$$

Nếu (5.7) được thay bằng $\lim_{t \rightarrow \infty} x(x_0, t) = x^*$ thì hệ (5.5) **ổn định tiệm cận** tại x^* .



Hình 5.16 Mô tả tính ổn định (a) và ổn định tiệm cận (b) theo Lyapunov

Định lý ổn định Lyapunov: Cho hệ phi tuyến không kích thích:

$$\dot{x}(t) = f(x) \quad (5.8)$$

có điểm cân bằng $x^* = 0$. Nếu tồn tại hàm Lyapunov $V(x)$ là hàm vô hướng khả vi liên tục của biến trạng thái x sao cho:

i) $V(x) \geq 0, \forall x$ và $V(x) = 0 \Leftrightarrow x = 0$.

ii) $V(x) \rightarrow \infty$ khi $\|x\| \rightarrow \infty$.

iii) $\dot{V}(x) \leq 0, \forall x$

thì hệ thống (5.8) ổn định (theo Lyapunov) tại điểm cân bằng $x^* = 0$.

Nếu điều kiện iii) được thay bằng $\dot{V}(x) < 0, \forall x$ thì hệ thống (5.8) ổn định tiệm cận tại điểm cân bằng $x^* = 0$.

Thiết kế bộ điều khiển mờ trực tiếp dùng lý thuyết Lyapunov

- 1- Xác định phương trình trạng thái mô tả đặc tính động học của đối tượng.
- 2- Chọn hàm Lyapunov bán xác định dương, hàm Lyapunov là hàm của các biến trạng thái và có liên quan đến tín hiệu điều khiển $u(t)$.
- 3- Tìm điều kiện ràng buộc của $u(t)$ sao cho đạo hàm theo thời gian của hàm Lyapunov bán xác định âm.
- 4- Nếu không tìm được biểu thức tường minh $u(t)$ thỏa mãn điều kiện ràng buộc ở bước 3 thì ta chọn một số điểm đặc

tính và tính giá trị tín hiệu điều khiển $u(t)$ thỏa mãn điều kiện ràng buộc tại các điểm đặc tính.

- 5- Thiết kế bộ điều khiển mờ có mặt điều khiển chứa các điểm đặc tính tính toán được ở bước 4.

Ví dụ 5.2 Thiết kế bộ điều khiển mờ giữ cân bằng hệ con lắc ngược dùng lý thuyết ổn định Lyapunov, biết rằng góc lệch của con lắc quan hệ với lực điều khiển xe theo phương trình vi phân sau:

$$\ddot{\theta} = \frac{u \cos \theta - (M + m)g(\sin \theta) + ml(\cos \theta \sin \theta)\dot{\theta}}{ml(\cos \theta)^2 - (M + m)l} \quad (5.9)$$

trong đó: $M = 1 [Kg]$ - trọng lượng xe

$m = 0,1 [Kg]$ - trọng lượng con lắc

$l = 1 [m]$ - chiều dài con lắc

$g = 9,81 [m/s^2]$ - gia tốc trọng trường

u - lực tác động vào xe $[N]$

θ - góc giữa con lắc và phương thẳng đứng $[rad]$

Đặt $x_1 = \theta$ và $x_2 = \dot{\theta}$ ta được phương trình trạng thái:

$$\begin{cases} \dot{x}_1 = x_2 \\ \dot{x}_2 = \frac{u \cos x_1 - (M + m)g(\sin x_1) + ml(\cos x_1 \sin x_1)x_2}{ml(\cos x_1)^2 - (M + m)l} \end{cases} \quad (5.10)$$

Chọn hàm Lyapunov bán xác định dương:

$$V = \frac{1}{2}(x_1^2 + x_2^2) \quad (5.11)$$

Đạo hàm theo thời gian của hàm Lyapunov là:

$$\begin{aligned} \dot{V} &= x_1 \dot{x}_1 + x_2 \dot{x}_2 \\ &= x_1 x_2 + x_2 \frac{u \cos x_1 - (M + m)g(\sin x_1) + ml(\cos x_1 \sin x_1)x_2}{ml(\cos x_1)^2 - (M + m)l} \\ &= x_2 \left[\frac{x_1 ml(\cos x_1)^2 - x_1(M + m)l + u \cos x_1}{ml(\cos x_1)^2 - (M + m)l} + \right. \\ &\quad \left. + \frac{-(M + m)g(\sin x_1) + ml(\cos x_1 \sin x_1)x_2}{ml(\cos x_1)^2 - (M + m)l} \right] \end{aligned}$$

Để hệ thống ổn định ta phải chọn $u(t)$ sao cho $\dot{V} \leq 0$. Do mẫu số nhỏ hơn 0 nên để $\dot{V} \leq 0$ ta phải chọn tín hiệu điều khiển $u(t)$ sao cho tử số lớn hơn hoặc bằng 0. Để thấy rằng $u(t)$ chọn như sau sẽ thỏa mãn điều kiện ổn định:

- Nếu $x_2 > 0$:

$$u > \frac{(M+m)[x_1 l + g(\sin x_1)] - x_1 ml(\cos x_1)^2 - ml(\cos x_1 \sin x_1)x_2}{\cos x_1} \quad (5.12)$$

- Nếu $x_2 < 0$:

$$u < \frac{(M+m)[x_1 l + g(\sin x_1)] - x_1 ml(\cos x_1)^2 - ml(\cos x_1 \sin x_1)x_2}{\cos x_1} \quad (5.13)$$

- Nếu $x_2 = 0$:

$$u = \frac{(M+m)[x_1 l + g(\sin x_1)] - x_1(t)ml(\cos x_1)^2}{\cos x_1} \quad (5.14)$$

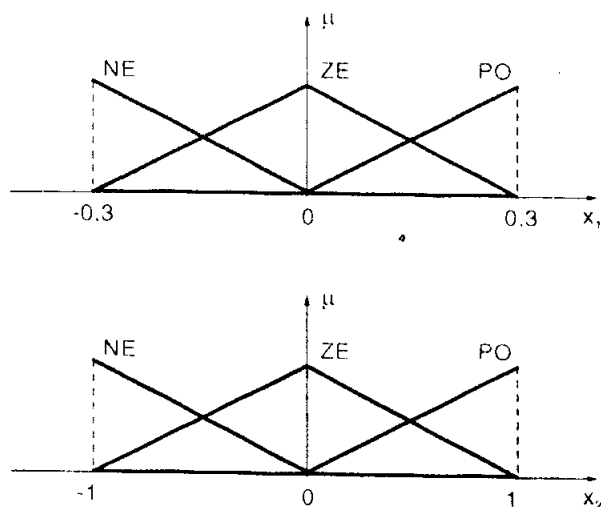
Chọn giá trị $u(t)$ thỏa mãn điều kiện ổn định (5.12), (5.13) và (5.14) tại các điểm đặc tính (x_1, x_2) như trình bày ở bảng 5.2.

Bảng 5.2 Các điểm đặc tính thỏa mãn điều kiện ổn định

Điểm đặc tính		Điều kiện ổn định	Chọn tín hiệu điều khiển u
x_1	x_2		
-0,6	-1,0	$u < -8,3021$	-12,00
-0,6	0,0	$u = -8,1327$	-8,13
-0,6	1,0	$u > -7,9633$	-6,00
0,0	-1,0	$u < 0$	-2,00
0,0	0,0	$u = 0$	0,00
0,0	1,0	$u > 0$	2,00
0,6	-1,0	$u < 8,3021$	6,00
0,6	0,0	$u = 8,1327$	8,13
0,6	1,0	$u > 7,9633$	12,00

Bước tiếp theo chúng ta cần thiết kế bộ điều khiển mờ có mặt đặc tính đi qua các điểm đã chọn trong bảng 5.2. Để dễ dàng chỉnh

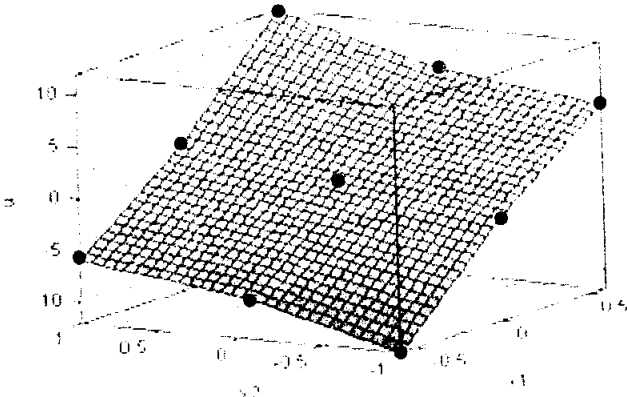
định mặt đặc tính ta sử dụng bộ điều khiển mờ Sugeno với kết luận là các hằng số. Bộ điều khiển có hai ngõ vào là $x_1(t)$ và $x_2(t)$; ngõ ra của bộ điều khiển là $u(t)$. Các tập mờ mô tả các giá trị ngôn ngữ của các biến vào được định nghĩa như hình 5.17, hệ mờ gồm 9 quy tắc liệt kê trong bảng 5.3 (để ý mối liên hệ giữa thông số của hàm liên thuộc ở mệnh đề điều kiện và giá trị hằng số ở mệnh đề kết luận của các quy tắc với giá trị x_1, x_2, u tại các điểm đặc tính). Chọn phương pháp suy diễn SUM-PROD và phương pháp giải mờ trung bình có trọng số. Hình 5.18 là mặt đặc tính của bộ điều khiển mờ vừa thiết kế, dễ thấy rằng mặt đặc tính này chứa các điểm đã tính toán ở bảng 5.2. Hình 5.19 là kết quả mô phỏng hệ thống điều khiển mờ vừa thiết kế, các đồ thị cho thấy bộ điều khiển mờ có thể giữ con lắc cân bằng xung quanh vị trí thẳng đứng với các điều kiện đầu khác nhau. Như vậy bộ điều khiển đã thiết kế thỏa mãn yêu cầu.



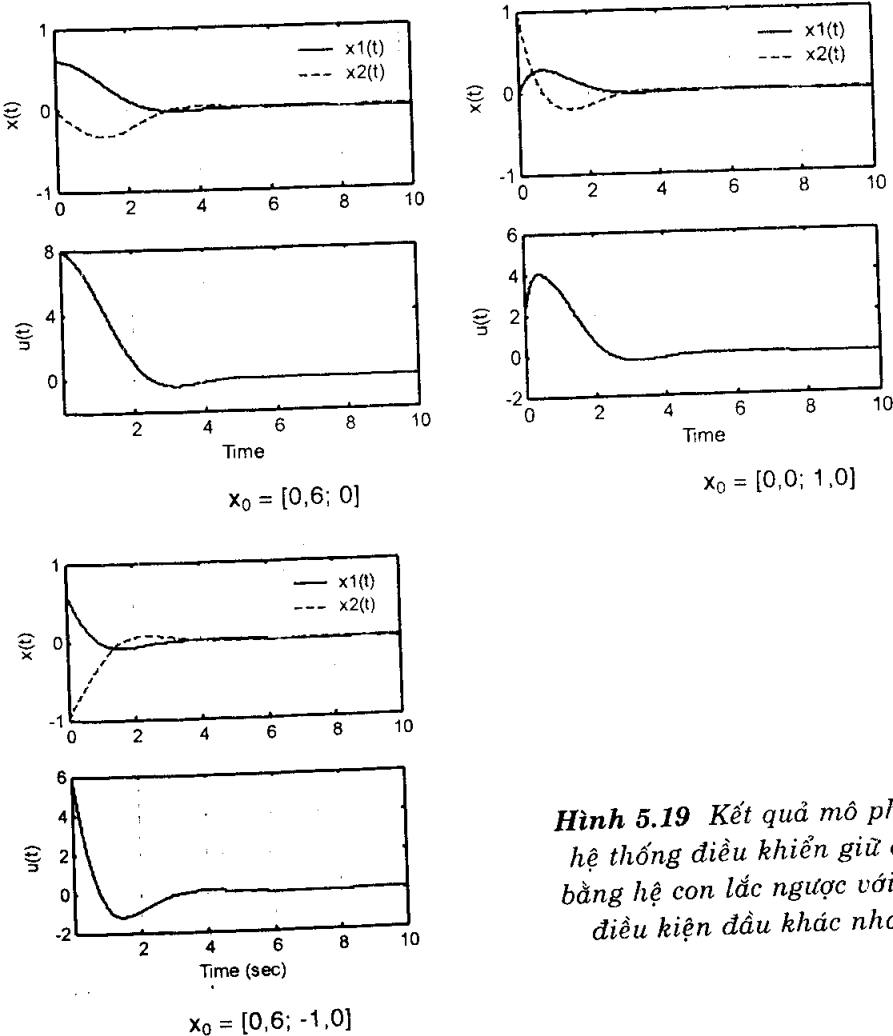
Hình 5.17 Hàm liên thuộc của các tập mờ

Bảng 5.3 Hệ quy tắc mờ

u		x ₂		
		NE	ZE	PO
x ₁	NE	-12,00	-8,13	-6,00
	ZE	-2,00	0,00	2,00
	PO	6,00	8,13	12,00



Hình 5.18 Mặt đặc tính của bộ điều khiển mờ



Hình 5.19 Kết quả mô phỏng hệ thống điều khiển giữ cân bằng hệ con lắc ngược với các điều kiện đầu khác nhau

5.3 ĐIỀU KHIỂN PID MỜ

5.3.1 Điều khiển PID mờ dùng hệ quy tắc Mamdani

Theo lý thuyết điều khiển kinh điển, bộ điều khiển PID là bộ điều khiển có tín hiệu ra tỷ lệ tuyến tính với tín hiệu vào, vi phân tín hiệu vào và tích phân tín hiệu vào theo biểu thức:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \frac{de(t)}{dt} \quad (5.15)$$

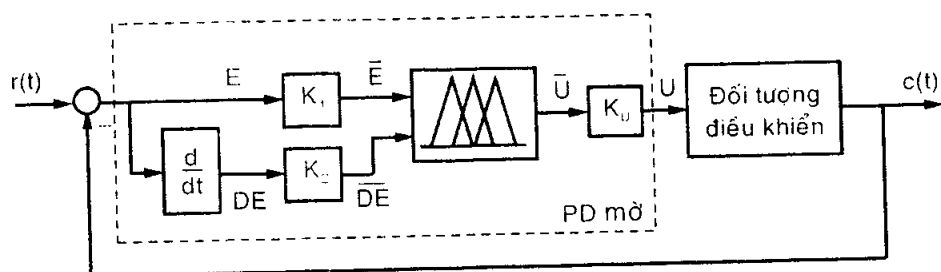
Bộ điều khiển PID nếu được thiết kế tốt có khả năng điều khiển hệ thống với chất lượng quá độ tốt (đáp ứng nhanh, độ vọt lố thấp) và triệt tiêu sai số xác lập, vì lý do đó các bộ điều khiển PID được sử dụng rất phổ biến trong các quá trình công nghiệp. Một nghiên cứu cho thấy hơn 90% các bộ điều khiển được sử dụng hiện nay là bộ điều khiển PID. Tuy nhiên bộ điều khiển PID kinh điển chỉ khống chế tín hiệu ra của đối tượng với chất lượng tốt nếu hệ thống làm việc trong miền tín hiệu nhỏ (hệ tuyến tính). Nếu đối tượng điều khiển là phi tuyến thì bộ điều khiển PID kinh điển không thể đảm bảo chất lượng điều khiển tại mọi điểm làm việc. Do vậy, để điều khiển các đối tượng phi tuyến ta cần sử dụng các bộ điều khiển PID mờ. Thuật ngữ PID mờ được hiểu theo nghĩa tín hiệu ra của bộ điều khiển tỷ lệ phi tuyến với tín hiệu vào, với tích phân tín hiệu vào và vi phân tín hiệu vào theo quy luật xác định bởi hệ quy tắc mờ.

1- Điều khiển PD mờ

Bộ điều khiển ở hình 5.20 là bộ điều khiển PD mờ vì tín hiệu ra của bộ điều khiển mờ cơ bản phụ thuộc vào tín hiệu vào và vi phân của tín hiệu vào. Bộ điều khiển PD mờ có thể điều khiển vô sai trong các trường hợp sau đây:

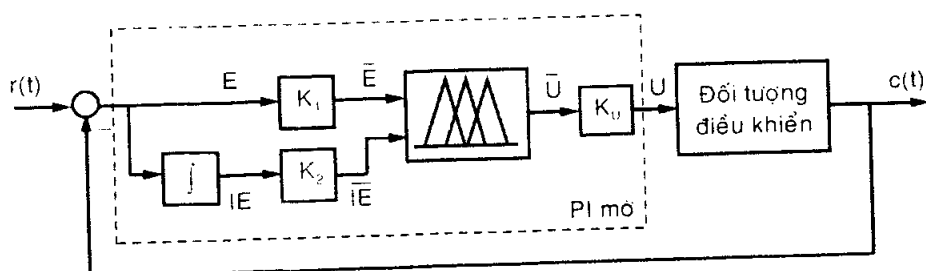
- Đối tượng có khâu tích phân lý tưởng, hoặc
- Ổn định hóa trạng thái của đối tượng xung quanh điểm cân bằng $(\bar{u}, \bar{x})$, trong đó $\bar{u} = 0$.

Nếu đối tượng không có khâu tích phân lý tưởng (như lò nhiệt, bồn chứa chất lỏng,...) thì sử dụng bộ điều khiển PD mờ không thể triệt tiêu sai số xác lập.

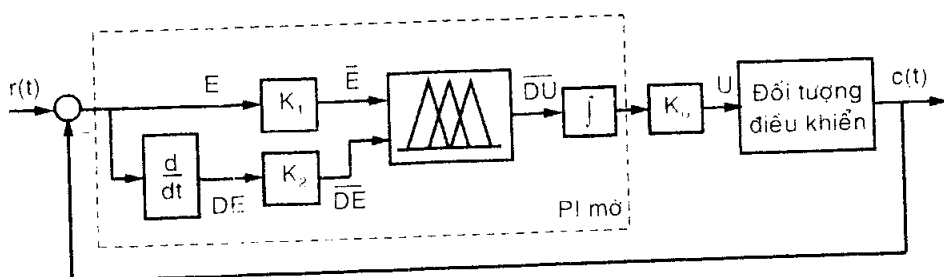


Hình 5.20 Bộ điều khiển PD mờ dùng hệ quy tắc Mamdani

2- Điều khiển PI mờ



a)



b)

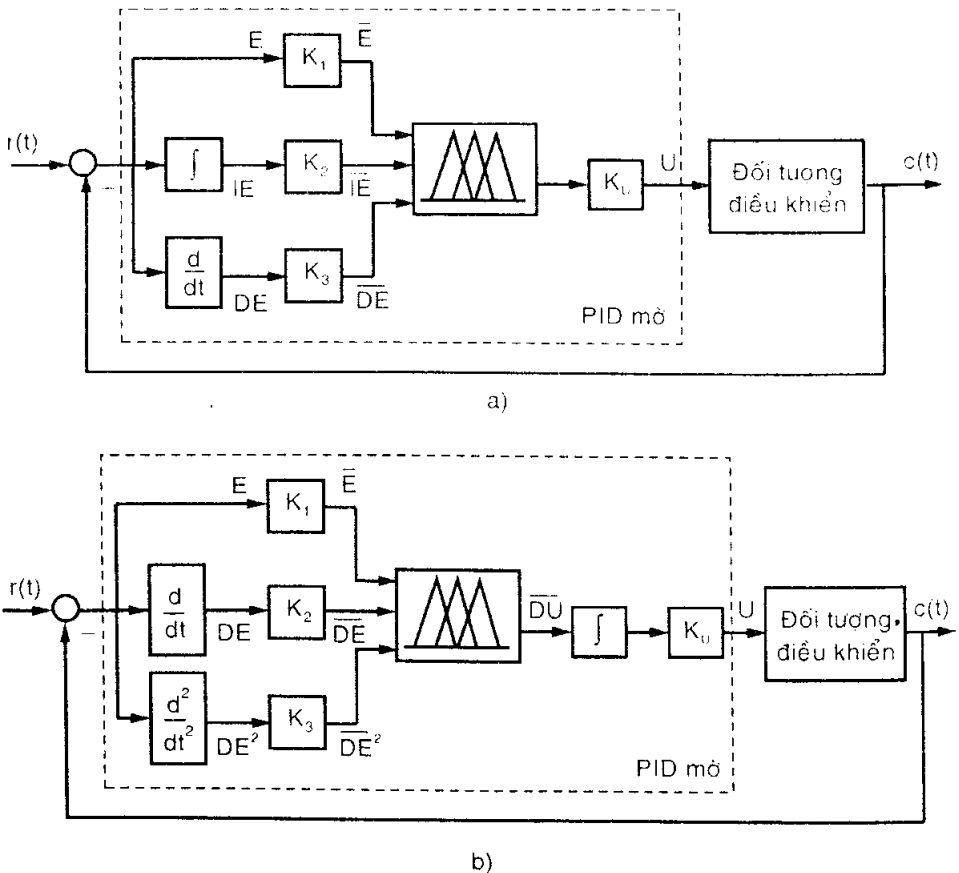
Hình 5.21 Bộ điều khiển PI mờ dùng hệ quy tắc Mamdani

Có hai cách để thực hiện bộ điều khiển PI mờ như trình bày ở hình 5.21, dễ dàng thấy rằng trong hai sơ đồ trên tín hiệu ra của bộ điều khiển có quan hệ phi tuyến với tín hiệu vào và tích phân tín hiệu vào. Tuy nhiên, sơ đồ ở hình 5.21a rất khó thực hiện vì kinh nghiệm chuyên gia hầu như không thể đưa ra các quy tắc điều khiển dựa vào tích phân của sai số. Do đó chỉ có sơ đồ điều khiển PI mờ ở hình 5.21b mới được sử dụng trong thực tế. Bộ điều khiển PI mờ nếu thiết kế tốt có thể điều khiển đối tượng trong miền làm việc rộng với sai số

xác lập bằng 0. Tuy nhiên cũng cần để ý rằng, bộ điều khiển PI làm chậm đáp ứng của hệ thống và làm cho quá trình quá độ có dao động.

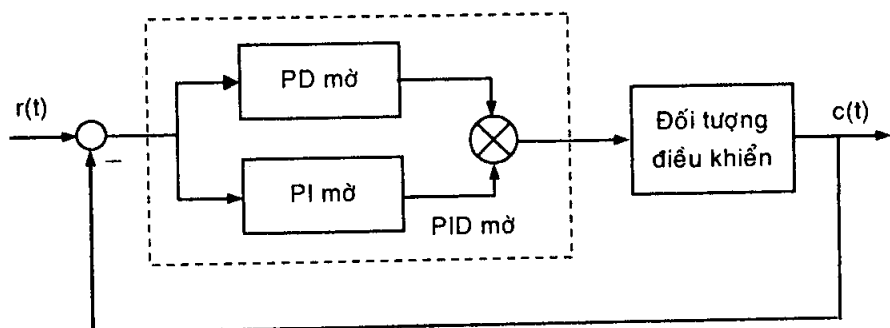
3- Điều khiển PID mờ

Bộ điều khiển PID mờ có các ưu điểm của bộ điều khiển PI mờ và PD mờ, tức là có thể điều khiển vô sai, thời gian đáp ứng nhanh, độ vọt lố thấp. Sơ đồ khối bộ điều khiển PID mờ được trình bày ở hình 5.22, trong đó sơ đồ PID ở hình 5.22b được sử dụng phổ biến hơn sơ đồ PID ở hình 5.22a. Thực tế, việc đưa ra các quy tắc Mamdani mô tả quan hệ giữa vi phân của tín hiệu ra theo tín hiệu vào, vi phân bậc 1 và vi phân bậc 2 của tín hiệu vào (H.5.22b) dựa vào kinh nghiệm chuyên gia cũng không dễ dàng. Do đó chỉ nên sử dụng bộ điều khiển PID mờ khi bộ điều khiển PI mờ hoặc PD mờ không thể điều khiển đối tượng với chất lượng như mong muốn.



Hình 5.22 Bộ điều khiển PID mờ dùng hệ quy tắc Mamdani

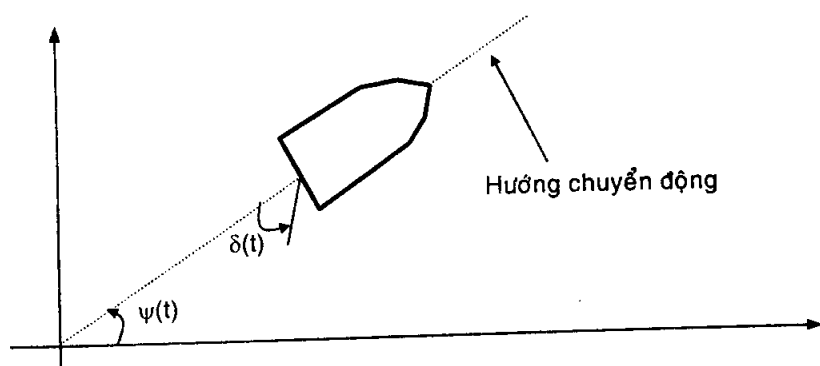
Một giải pháp khác để thực thi bộ điều khiển PID mờ là sử dụng bộ điều khiển PI mờ ghép song song với bộ điều khiển PD mờ như sơ đồ ở hình 5.23.



Hình 5.23 Bộ điều khiển PID mờ gồm bộ điều khiển PD mờ và PI mờ ghép song song

Chú ý: Nếu thực hiện bộ điều khiển PID mờ rời rạc thì phép tính vi phân được tính gần đúng bằng phép sai phân, phép tính tích phân được thay thế bằng phép tổng (tích phân gần đúng).

Ví dụ 5.3 Điều khiển lái tàu



Hình 5.24 Bài toán điều khiển lái tàu

Phương trình vi phân mô tả hệ thống lái tàu là:

$$\ddot{\psi}(t) + \left(\frac{1}{\tau_1} + \frac{1}{\tau_2} \right) \dot{\psi}(t) + \left(\frac{1}{\tau_1 \tau_2} \right) (\psi^3(t) + \psi(t)) = \left(\frac{k}{\tau_1 \tau_2} \right) (\tau_3 \delta(t) + \delta(t)) \quad (5.16)$$

trong đó:

- Tín hiệu vào $\delta(t)$ (*radian*) là góc của bánh lái.
- Tín hiệu ra $\psi(t)$ (*radian*) là góc (hướng) chuyển động của tàu.
- Các hằng số:

$$k = k_0 \frac{v}{l} \quad (k_0 = -3,86)$$

$$\tau_i = \tau_{i0} \frac{l}{v} \quad (i = 1, 2, 3) \quad (\tau_{10} = 5,66, \tau_{20} = 0,38, \tau_{30} = 0,89)$$

l - chiều dài tàu, $l = 161m$

v - vận tốc tàu, giả sử tàu chuyển động đều với vận tốc $v=5m/s$

Bài toán đặt ra là thiết kế bộ điều khiển mờ lái tàu di chuyển theo hướng mong muốn. Dễ thấy trong mô hình toán hệ thống lái tàu có khâu tích phân lý tưởng, do đó có thể có thể sử dụng bộ điều khiển PD mờ (xem sơ đồ khối ở hình 5.20).

Thiết kế bộ điều khiển PD mờ

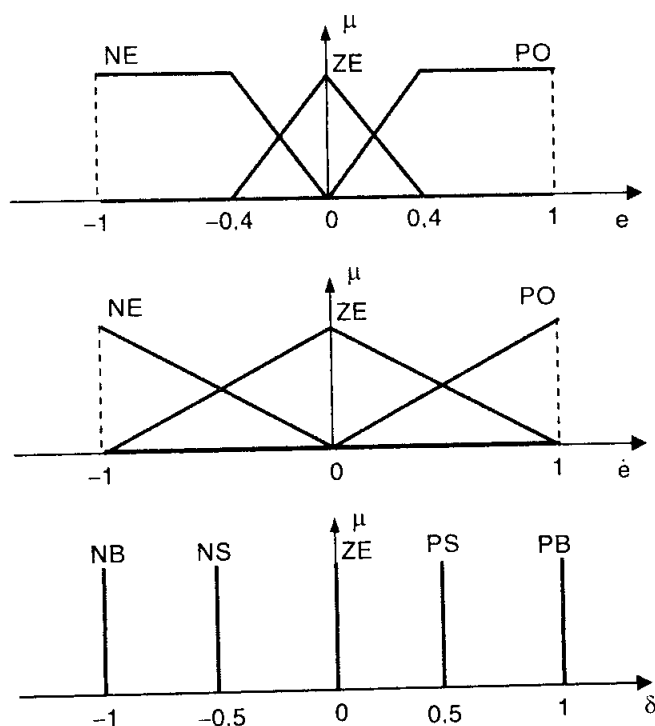
- Gọi $\psi_r(t)$ là tín hiệu đặt (hướng chuyển động mong muốn), tín hiệu vào của bộ điều khiển PD mờ là sai số $e(t) = \psi_r(t) - \psi(t)$ và vi phân sai số $\dot{e}(t)$.

- Các độ lợi tiền xử lý, hậu xử lý được chọn như sau: $K_1 = 1/\pi$ (do $-\pi \leq e(t) \leq \pi$), $K_2 = 100$ (để chuẩn hóa $\dot{e}(t)$ về tầm $[-1,1]$), $K_U = 80\pi/180$ (do góc bánh lái nằm trong giới hạn vật lý $-80^\circ \leq \delta(t) \leq 80^\circ$).

- Các tập mờ biểu diễn các giá trị ngôn ngữ của các biến vào và biến ra được chọn như hình 5.26. Thông số của các hàm liên thuộc được chỉnh định bằng phương pháp thử sai để đáp ứng của hệ thống là “tốt nhất”.

- Hệ quy tắc mờ (bảng 5.4) được rút ra từ kinh nghiệm của người lái tàu.

- Chọn phương pháp suy diễn MAX-MIN, phương pháp giải mờ trọng tâm.

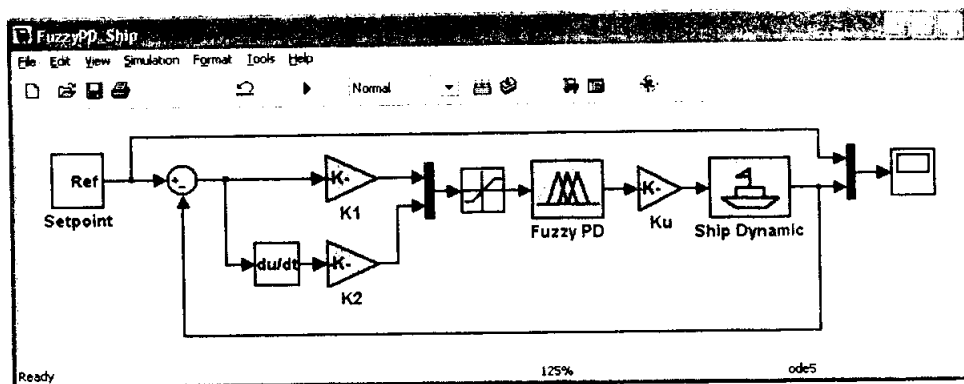


Hình 5.25 Các giá trị ngôn ngữ trong bài toán lái tàu

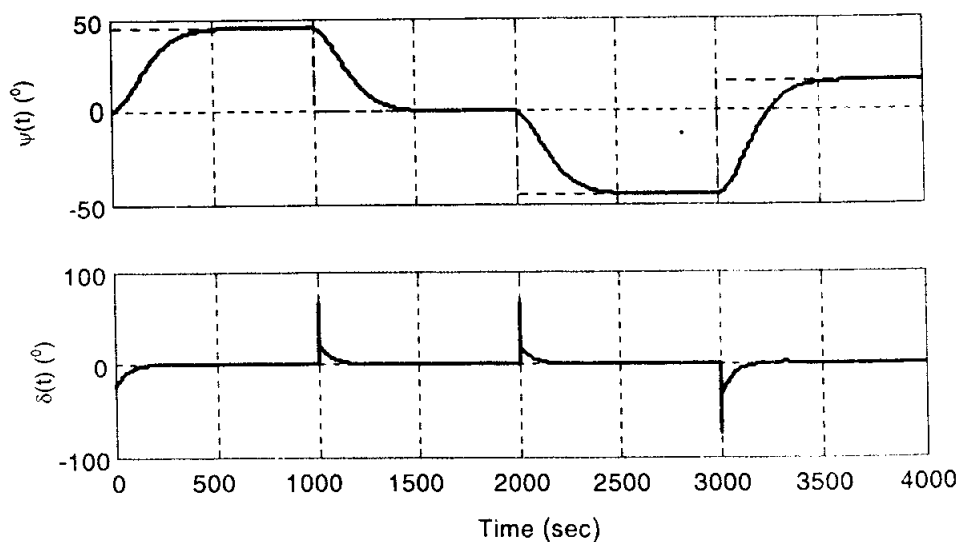
Bảng 5.4 Các quy tắc mờ của bài toán lái tàu

U		E		
		NE	ZE	PO
DE	NE	PB	PS	ZE
	ZE	PS	ZE	NS
	PO	ZE	NS	NB

Hình 5.26 là sơ đồ Simulink mô phỏng hệ thống điều khiển lái tàu vừa thiết kế, trong đó khối [Ship Dynamic] mô phỏng đặc tính động học của tàu theo phương trình (5.16), bộ điều khiển mờ với các quy tắc trình bày ở bảng 5.2. Hình 5.27 cho thấy hệ thống điều khiển có thể lái tàu theo hướng mong muốn với sai số xác lập bằng 0, không vọt lố, thời gian quá độ khoảng 8 phút (500sec). Chất lượng điều khiển được đảm bảo với giá trị đặt là các hướng di chuyển khác nhau của tàu.

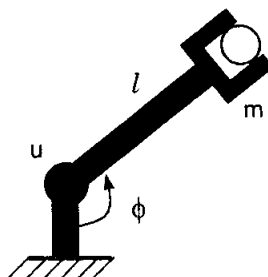


Hình 5.26 Sơ đồ Simulink điều khiển PD mờ hệ thống lái tàu



Hình 5.27 Kết quả mô phỏng lái tàu dùng bộ điều khiển PD mờ

Ví dụ 5.4 Điều khiển vị trí cánh tay máy một bậc tự do



Hình 5.28 Cánh tay máy 1 bậc tự do

Đặc tính động học của hệ tay máy một bậc tự do cho bởi phương trình:

$$(J + ml^2)\ddot{\phi}(t) + B\dot{\phi}(t) + (ml + Ml_C)g \sin \phi(t) = u(t) \quad (5.17)$$

trong đó:

$u(t)$ - moment tác động lên trục quay của cánh tay máy

$\phi(t)$ - góc quay (vị trí) của cánh tay máy, giả sử tầm làm việc của cánh tay máy là từ 0^0 (vị trí thẳng đứng phía dưới) đến 180^0 (vị trí thẳng đứng phía trên)

J - moment quán tính của cánh tay máy ($J = 0,05kg.m^2$)

M - khối lượng của cánh tay máy ($M = 1,0kg$)

m - khối lượng vật nặng ($m = 0,1kg$)

l - chiều dài cánh tay máy ($l = 0,4m$)

l_C - khoảng cách từ trọng tâm cánh tay máy đến trục quay ($l=0,15m$)

B - hệ số ma sát nhớt ($B = 0,2kg.m^2/s$)

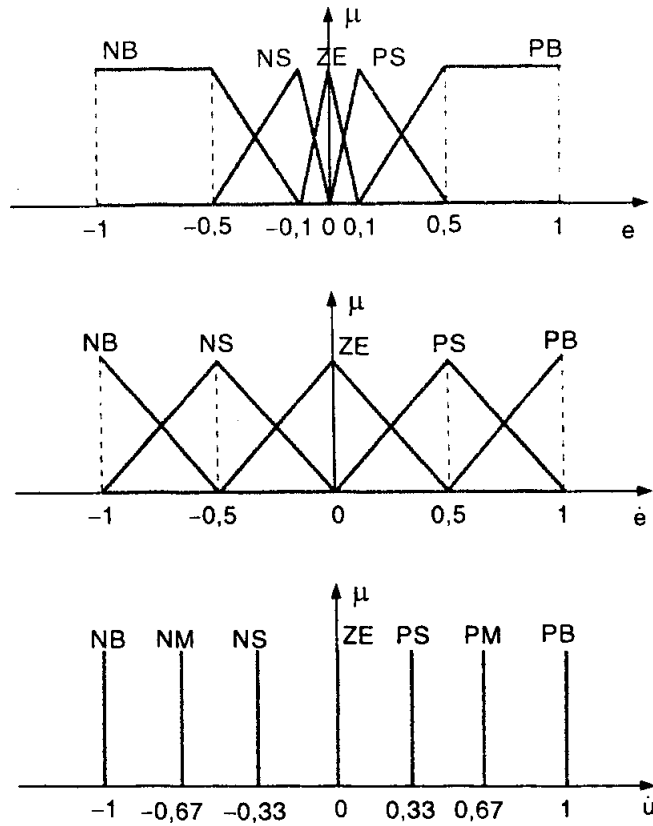
g - gia tốc trọng trường ($g = 9,81m/s^2$)

Do mô hình toán của cánh tay máy 1 bậc tự do không có khâu tích phân lý tưởng nên để điều khiển cánh tay máy di chuyển theo tín hiệu đặt với sai số xác lập bằng 0 ta phải sử dụng bộ điều khiển PI mờ có sơ đồ khối ở hình 5.21b.

Thiết kế bộ điều khiển PI mờ:

- Bộ điều khiển mờ cơ bản có tín hiệu vào sai số (E) và vi phân sai số (DE), tín hiệu ra là vi phân điện áp điều khiển (DU).
- Do cánh tay máy có thể di chuyển trong tầm $[0 \div \pi]$ nên $-\pi \leq E \leq \pi$, do đó hệ số chuẩn hóa $K_1 = 1/\pi$. Chọn hệ số chuẩn hóa $K_2 = 5/\pi$ (hệ số này phụ thuộc vào thông số của cánh tay máy). Hệ số $K_U = 8$ được chọn trong quá trình tinh chỉnh chất lượng của hệ thống điều khiển.
- Các tập mờ biểu diễn các giá trị ngôn ngữ của các biến vào và biến ra được chọn có dạng như trình bày ở hình 5.29. Do cánh tay máy có tính phi tuyến cao nên ta chọn số tập mờ cho mỗi biến vào là 5. Số tập mờ cho biến ra là 7.
- Hệ quy tắc điều khiển được suy ra từ kinh nghiệm.

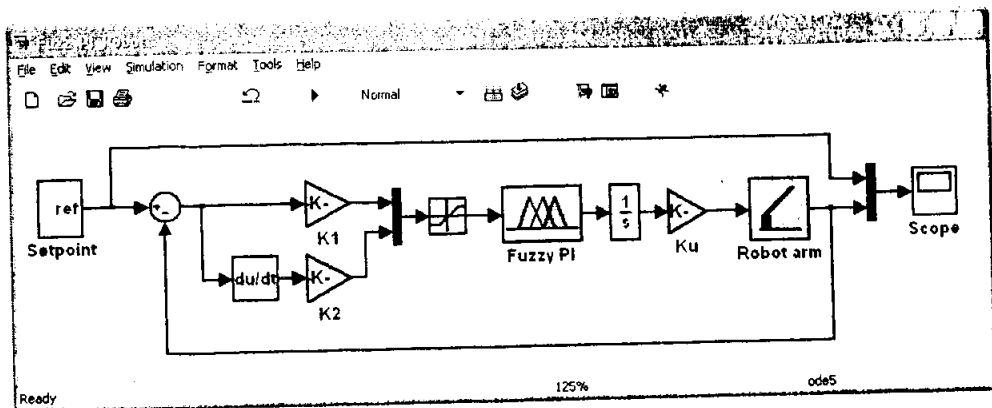
Hình 5.31 là kết quả điều cánh tay máy dùng thuật toán PI mờ vừa thiết kế. Ta thấy từ vị trí ban đầu $\phi = 0^\circ$, vị trí cánh tay bám theo giá trị đặt rất tốt, đáp ứng không vọt lố.



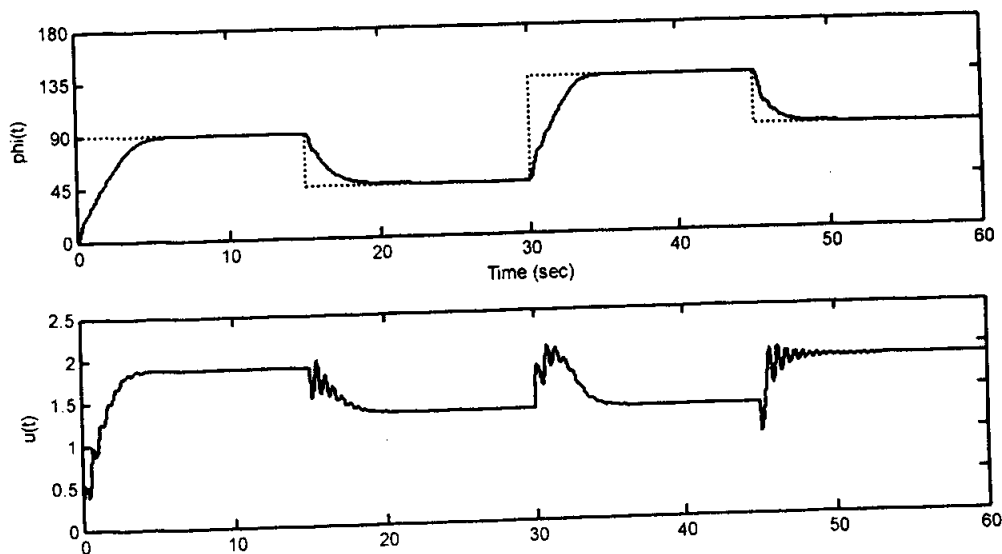
Hình 5.29 Các giá trị ngôn ngữ trong bài toán điều khiển cánh tay máy

Bảng 5.5 Các quy tắc mờ điều khiển cánh tay máy

DU		E				
		NB	NE	ZE	PO	PB
DE	NB	NB	NB	NM	NS	ZE
	NE	NB	NM	NS	ZE	PS
	ZE	NM	NS	ZE	PS	PM
	PO	NS	ZE	PS	PM	PB
	PB	ZE	PS	PM	PB	PB



Hình 5.30 Sơ đồ Simulink điều khiển PI mờ hệ tay máy



Hình 5.31 Kết quả mô phỏng điều khiển cánh tay máy

5.3.2 Điều khiển PID mờ dùng hệ quy tắc Sugeno

Bộ điều khiển PID kinh điển được sử dụng rất rộng rãi trong các hệ thống điều khiển ổn định hóa hệ tuyến tính do có khả năng làm hệ thống đáp ứng nhanh, triệt tiêu sai số xác lập. Tuy nhiên, khi áp dụng bộ điều khiển PID kinh điển để điều khiển đối tượng phi tuyến thì chất lượng của hệ thống thường không tốt tại mọi điểm làm việc. Lý do là vì mô hình tuyến tính của đối tượng phi tuyến tại mỗi điểm làm việc là khác nhau, do đó để đảm bảo chất lượng thông số

bộ điều khiển PID cần thay đổi theo điểm làm việc. Cách đơn giản nhưng rất hiệu quả để thiết kế bộ điều khiển PID có thông số thay đổi theo điểm làm việc là sử dụng bộ điều khiển mờ với các quy tắc Sugeno có mệnh đề kết luận là luật điều khiển PID ứng với các điểm làm việc. Quy tắc điều khiển thứ k có dạng:

Nếu x_1 là $\tilde{A}_{1k}$ và... x_n là $\tilde{A}_{nk}$ thì:

$$u_k(t) = K_{Pk}e(t) + K_{Ik} \int e(t)dt + K_{Dk}\dot{e}(t) \quad (5.18)$$

Trong quy tắc trên, mệnh đề điều kiện xác định điểm làm việc của bộ điều khiển PID thứ k . Gọi β_k là độ đúng của mệnh đề điều kiện của quy tắc thứ k , tín hiệu ra của bộ điều khiển PID mờ gồm R quy tắc dạng (5.18) xác định bởi công thức giải mờ trung bình có trọng số:

$$u(t) = \frac{\sum_{k=1}^R \beta_k u_k(t)}{\sum_{k=1}^R \beta_k} \quad (5.19)$$

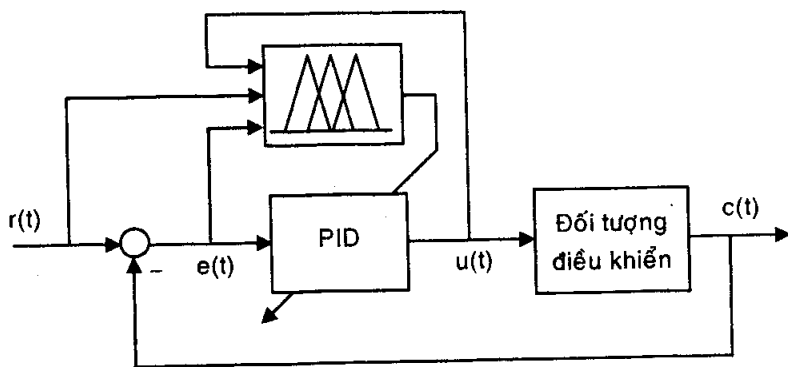
Thay $u_k(t)$ ở mệnh đề kết luận của quy tắc (5.18) vào (5.19) ta được:

$$u(t) = \tilde{K}_P e(t) + \tilde{K}_I \int e(t)dt + \tilde{K}_D \dot{e}(t) \quad (5.20)$$

trong đó:

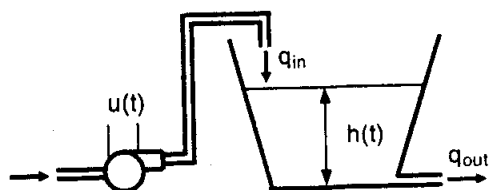
$$\tilde{K}_P = \frac{\sum_{k=1}^R \beta_k K_{Pk}}{\sum_{k=1}^R \beta_k}; \quad \tilde{K}_I = \frac{\sum_{k=1}^R \beta_k K_{Ik}}{\sum_{k=1}^R \beta_k}; \quad \tilde{K}_D = \frac{\sum_{k=1}^R \beta_k K_{Dk}}{\sum_{k=1}^R \beta_k} \quad (5.21)$$

Công thức trên chứng tỏ bộ điều khiển PID có thông số $\tilde{K}_P$, $\tilde{K}_I$, $\tilde{K}_D$ thay đổi tùy theo điều kiện làm việc và có thể xem hệ quy tắc mờ như là bộ giám sát thay đổi thông số bộ điều khiển PID như sơ đồ khối ở hình 5.32.



Hình 5.32 Bộ điều khiển PID mờ dùng hệ quy tắc Sugeno

Ví dụ 5.5 Điều khiển mực chất lỏng trong bồn chứa dùng bộ điều khiển PI mờ.



Hình 5.33 Hệ bồn chứa chất lỏng

Xét hệ bồn chứa chất lỏng có tiết diện ngang thay đổi theo độ cao như hình 5.33. Phương trình vi phân mô tả hệ thống là:

$$\dot{h}(t) = \frac{1}{A(h)} (ku(t) - C_D a \sqrt{2gh(t)}) \quad (5.22)$$

$$A(h) = \frac{A_{\max} - A_{\min}}{h_{\max}} h + A_{\min} \quad (5.23)$$

trong đó: $u(t)$ - điện áp điều khiển máy bơm ($0 \leq u(t) \leq 12V$)

$h(t)$ - độ cao mực chất lỏng trong bồn (cm)

$A(h)$ - tiết diện ngang bồn chứa (cm^2)

$h_{\max}$ - độ cao cực đại của bồn chứa

$A_{\max}, A_{\min}$ - tiết diện ngang cực đại và cực tiểu

k - hệ số tỷ lệ với công suất máy bơm

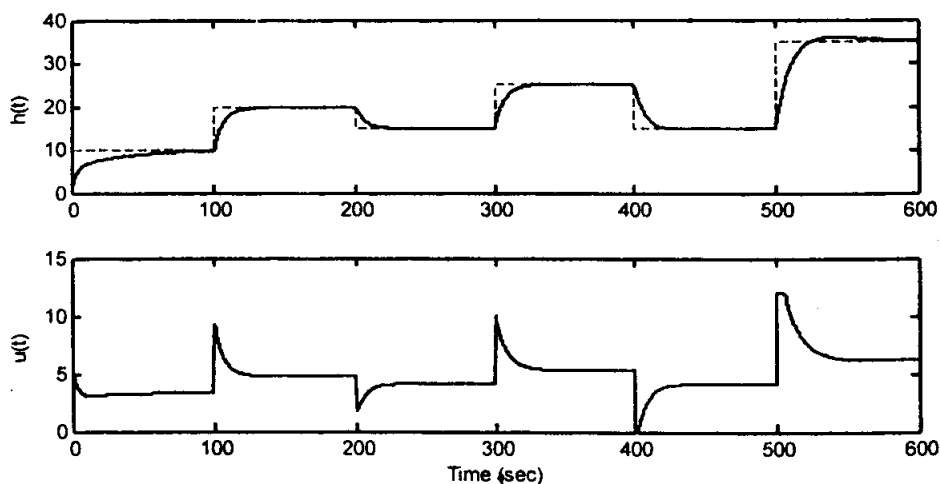
a - tiết diện van xả (cm^2)

g - gia tốc trọng trường ($981cm/sec^2$)

C_D - hệ số xả.

Thông số của hệ bồn đơn được chọn như sau $h_{max} = 50cm$; $A_{max} = 200cm^2$; $A_{min} = 100cm^2$; $a = 1cm^2$; $k = 300cm^3/sec$; $C_D = 0,6$. Bài toán đặt ra là điều khiển mực chất lỏng trong bồn bám theo tín hiệu đặt.

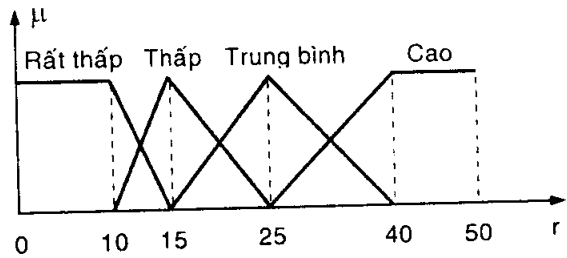
Do hệ bồn chứa là đối tượng phi tuyến nên nếu ta sử dụng bộ điều khiển PI kinh điển để điều khiển mực chất lỏng trong bồn thì chất lượng của hệ thống không thể tốt tại mọi điểm làm việc. Hình 5.34 là đáp ứng của hệ thống với thông số bộ điều khiển PI được chỉnh phù hợp với điểm làm việc $\bar{h} = 20cm$. Rõ ràng tại các điểm làm việc cách xa điểm $\bar{h} = 20cm$, chất lượng của hệ thống rất kém, đáp ứng của hệ thống hoặc rất chậm hoặc có vọt lố.



Hình 5.34 Kết quả điều khiển hệ bồn đơn dùng PI kinh điển

Thiết kế bộ điều khiển PI mờ

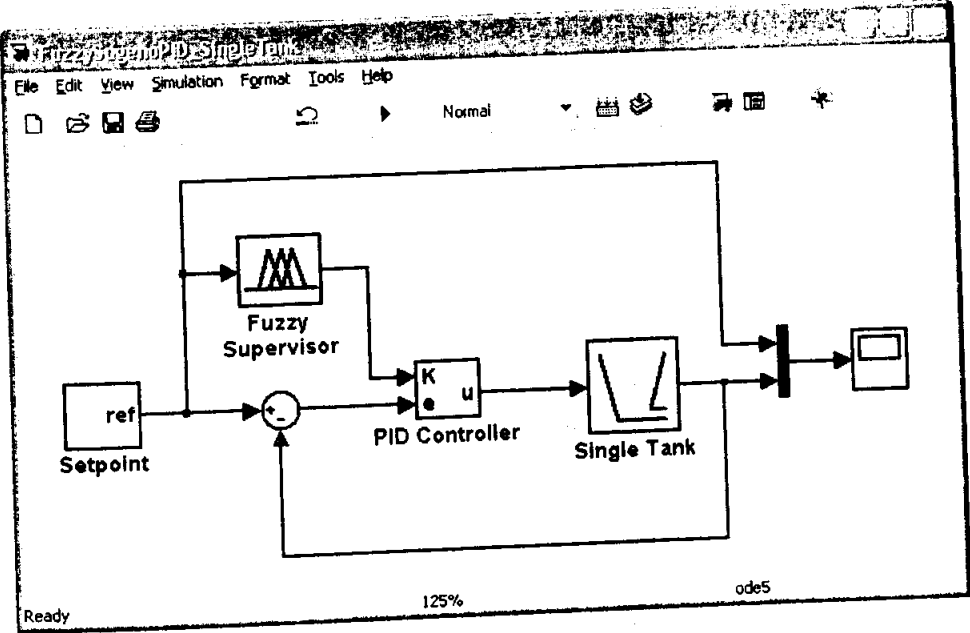
Ta thiết kế hệ thống điều khiển PI mờ theo sơ đồ khối ở hình 5.32. Ngõ vào của bộ giám sát mờ là điểm làm việc, các tập mờ mô tả các giá trị ngôn ngữ của biến vào được định nghĩa ở hình 5.35. Ngõ ra của bộ giám sát mờ là giá trị của các hệ số $\tilde{K}_P$ và $\tilde{K}_I$ thay đổi theo hệ quy tắc cho ở bảng 5.6. Hình 5.36 là sơ đồ Simulink mô phỏng hệ thống điều khiển mờ vừa thiết kế. Kết quả mô phỏng (H.5.37) cho thấy bộ điều khiển PI mờ có thể điều khiển mực chất lỏng trong bồn chứa với chất lượng đạt yêu cầu tại mọi điểm làm việc.



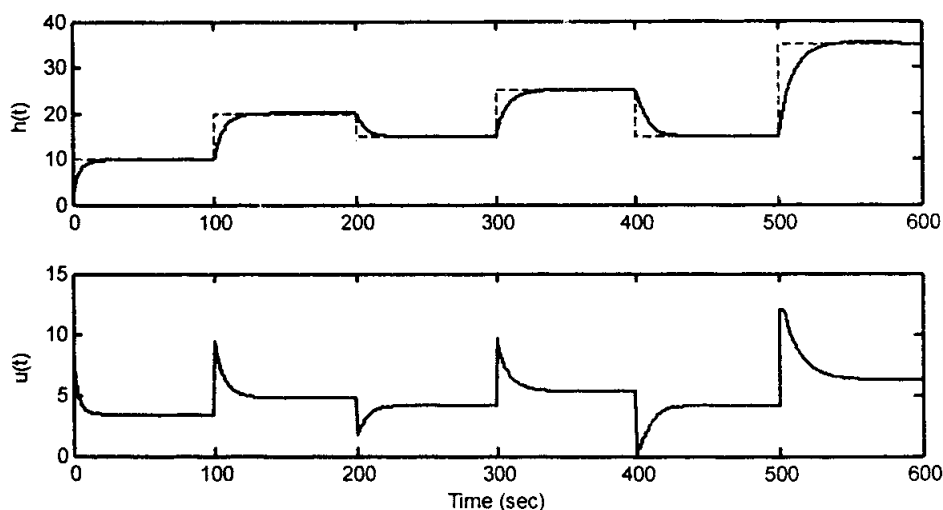
Hình 5.35 Các tập mờ mô tả giá trị ngôn ngữ của điểm làm việc

Bảng 5.6 Hệ quy tắc mờ

		Thông số bộ điều khiển PI	
		K_P	K_I
Điểm làm việc	Rất thấp	0,900	0,080
	Thấp	0,800	0,055
	Trung bình	0,700	0,035
	Cao	0,600	0,020



Hình 5.36 Sơ đồ mô phỏng hệ thống điều khiển PID mờ dùng quy tắc Sugeno

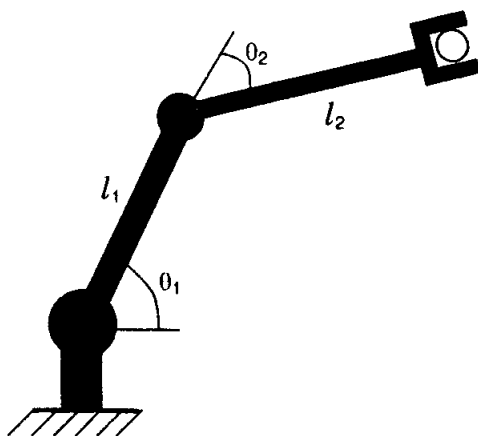


Hình 5.37 Kết quả mô phỏng hệ thống điều khiển PID mờ hệ bồn chứa

5.3.3 Điều khiển hệ MIMO

Các mục trước chỉ đề cập đến bộ điều khiển mờ trực tiếp cho hệ SISO, trong trường hợp đối tượng điều khiển là hệ MIMO (ví dụ như hệ tay máy nhiều bậc tự do, hệ bồn chứa liên kết...) vẫn có thể sử dụng các bộ điều khiển mờ SISO theo nguyên tắc tách rời. Sau đây là một ví dụ.

Ví dụ 5.6 Điều khiển vị trí cánh tay máy hai bậc tự do



Hình 5.38 Cánh tay máy hai bậc tự do

Hệ tay máy hai bậc tự do có mô hình toán học như sau:

$$\begin{bmatrix} H_{11} & H_{12} \\ H_{21} & H_{22} \end{bmatrix} \begin{bmatrix} \ddot{\theta}_1 \\ \ddot{\theta}_2 \end{bmatrix} + \begin{bmatrix} -h\dot{\theta}_2 & -h\dot{\theta}_1 - h\dot{\theta}_2 \\ h\dot{\theta}_1 & 0 \end{bmatrix} \begin{bmatrix} \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} + \begin{bmatrix} g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_2 \end{bmatrix} \quad (5.24)$$

trong đó:

$$H_{11} = J_1 + J_2 + m_1 l_{c1}^2 + m_2 [l_1^2 + l_{c2}^2 + 2l_1 l_{c2} \cos(\theta_2)] + m_3 [l_1^2 + l_2^2 + 2l_1 l_2 \cos(\theta_2)] \quad (5.25)$$

$$H_{22} = J_2 + m_2 l_{c2}^2 + m_3 l_2^2 \quad (5.26)$$

$$H_{12} = H_{21} = J_2 + m_2 [l_{c2}^2 + l_1 l_{c2} \cos(\theta_2)] + m_3 [l_2^2 + 2l_1 l_2 \cos(\theta_2)] \quad (5.27)$$

$$h = m_2 l_1 l_{c2} \sin(\theta_2) \quad (5.28)$$

$$g_1 = m_1 l_{c1} g \cos(\theta_1) + m_2 g [l_{c2} \cos(\theta_1 + \theta_2) + l_1 \cos(\theta_1)] \quad (5.29)$$

$$g_2 = m_2 l_{c2} g \cos(\theta_1 + \theta_2) \quad (5.30)$$

Thông số vật lý của cánh tay máy mô phỏng:

J_1, J_2 : moment quán tính của hai khớp

$$(J_1 = 0,05 \text{ kg.m}^2; J_2 = 0,05 \text{ kg.m}^2)$$

m_1, m_2 : khối lượng của hai khớp ($m_1 = 1,0 \text{ kg}; m_2 = 1,0 \text{ kg}$)

m_3 : khối lượng vật nặng mà tay máy gắp ($m_3 = 0,1 \text{ kg}$)

l_1, l_2 : chiều dài hai khớp ($l_1 = 1,0 \text{ m}; l_2 = 1,0 \text{ m}$)

l_{c1}, l_{c2} : khoảng cách từ trọng tâm của mỗi khớp đến trục quay

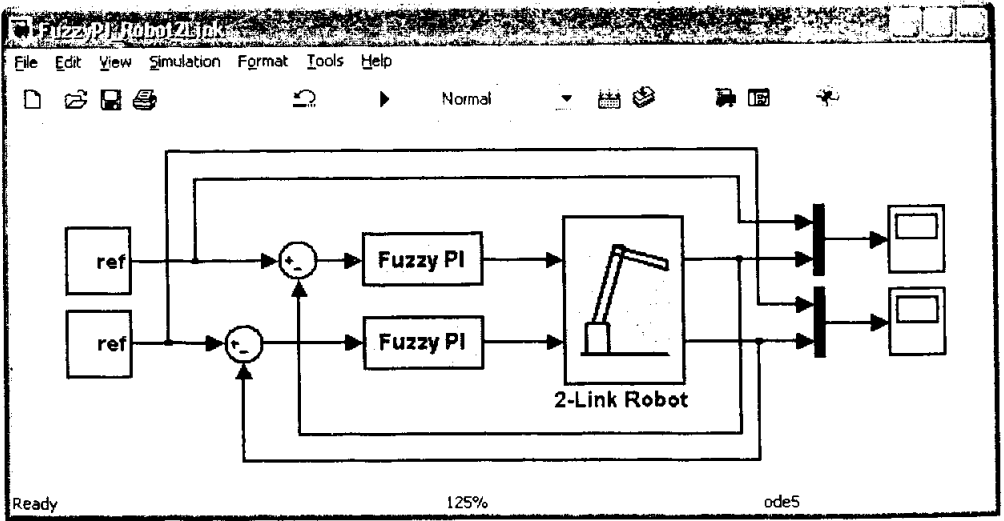
$$(l_{c1} = 0,4 \text{ m}; l_{c2} = 0,4 \text{ m})$$

g : gia tốc trọng trường ($g = 9,81 \text{ m/s}^2$).

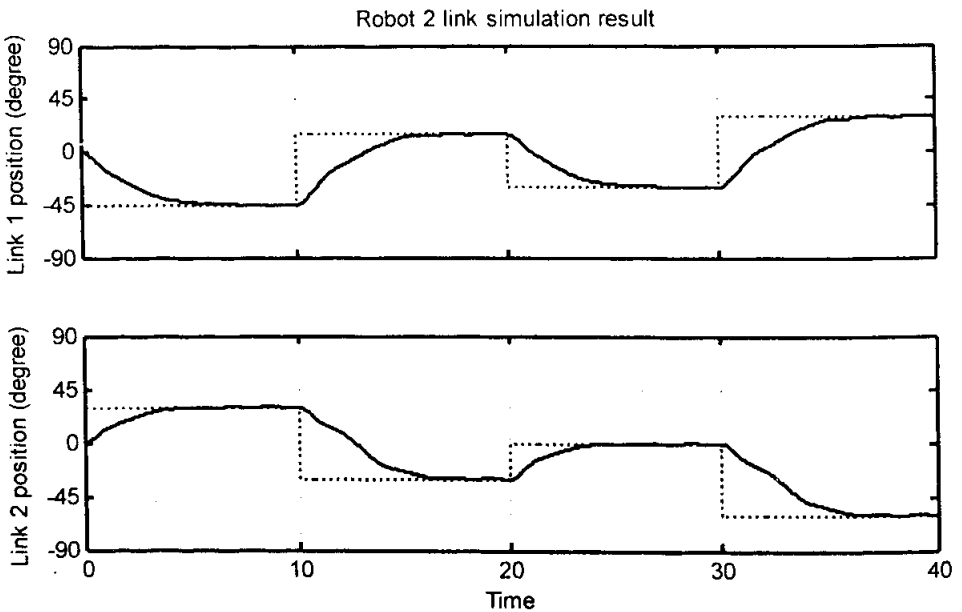
Thiết kế bộ điều khiển

Hệ tay máy xét ở ví dụ này là hệ MIMO hai vào hai ra, ta có thể sử dụng hai bộ điều khiển SISO tương tự như đã thiết kế ở ví dụ 5.4 để điều khiển cánh tay máy này theo nguyên lý tách rời, mỗi bộ điều khiển sẽ kiểm soát hoạt động một khớp liên kết của tay máy. Cụ thể sơ đồ Simulink mô phỏng hệ thống điều khiển như hình 5.39. Bằng phương pháp chỉnh định thử sai, ta có thể chọn thông số chuẩn hóa và thông số các tập mờ mô tả các giá trị ngôn ngữ tương tự như

đã trình bày ở ví dụ 5.4. Hình 5.40 cho thấy hai bộ điều khiển mờ PI có thể điều khiển vị trí cánh tay máy hai bậc tự do bám theo tín hiệu đặt cho trước.



Hình 5.39 Sơ đồ Simulink điều khiển cánh tay máy hai bậc tự do



Hình 5.40 Kết quả mô phỏng điều khiển cánh tay máy hai bậc tự do

5.4 ĐIỀU KHIỂN TRỰC TIẾP DÙNG MẠNG THẦN KINH

Mạng thần kinh có thể được sử dụng để học kinh nghiệm chuyên gia trong việc điều khiển các đối tượng phức tạp, khó xác định mô hình toán hoặc sao chép một bộ điều khiển vật lý có sẵn. Ví dụ mạng thần kinh có thể học kinh nghiệm của người lái tàu, lái xe hoặc học kinh nghiệm điều khiển máy móc thiết bị của các chuyên gia lành nghề (H.5.3). Sau khi huấn luyện, mạng thần kinh có thể điều khiển các đối tượng thay thế con người.

Để huấn luyện mạng thần kinh học một bộ điều khiển, trước tiên ta phải cho hệ thống điều khiển hoạt động và thu thập dữ liệu vào ra của bộ điều khiển đó. Sau đó sử dụng dữ liệu đã thu thập để huấn luyện mạng thần kinh sao cho sai lệch giữa ngõ ra $u_c(t)$ của bộ điều khiển và ngõ ra $u_{nn}(t)$ của mạng là tối thiểu.

$$J(w) = \sum_{t=1}^N [u_c(t) - u_{nn}(t)]^2 \rightarrow \min \quad (5.31)$$

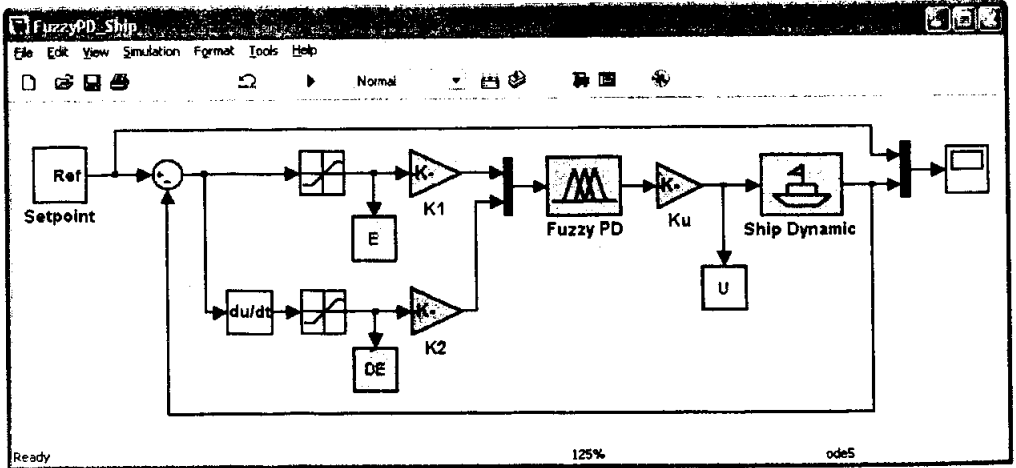
Chú ý là hầu hết các thuật toán huấn luyện mạng thần kinh đều là thuật toán tìm cực trị cục bộ nên trong một số trường hợp có thể cần phải chạy thuật toán huấn luyện mạng nhiều lần với các bộ trọng số ban đầu khác nhau mới có thể tìm được mạng thần kinh điều khiển được đối tượng đúng yêu cầu.

Để minh họa khả năng sao chép bộ điều khiển của mạng thần kinh, hai ví dụ huấn luyện mạng thần kinh học bộ điều khiển PD và PI được trình bày dưới đây.

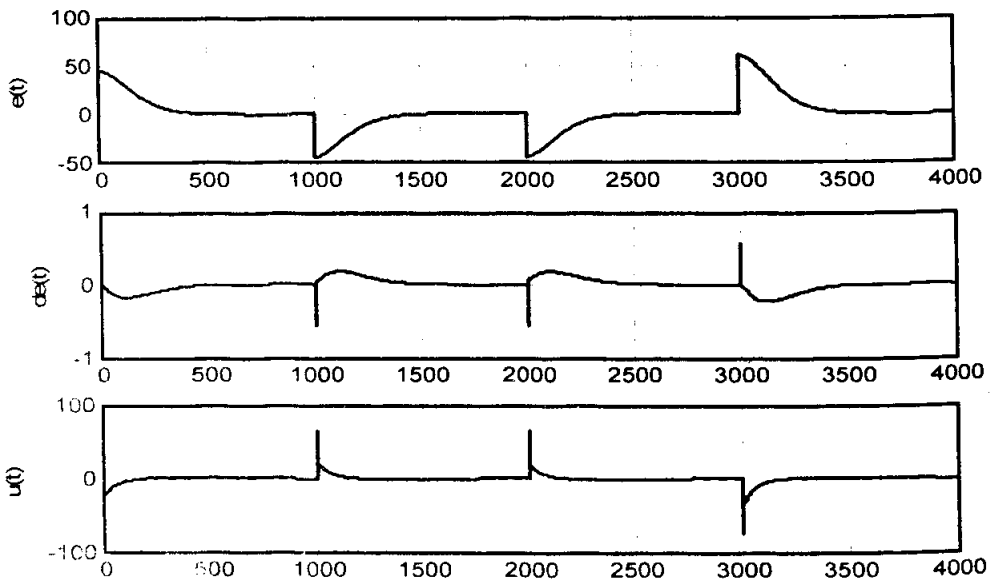
Ví dụ 5.7 Huấn luyện mạng thần kinh sao chép bộ điều khiển PD mờ điều khiển hệ thống lái tàu ở ví dụ 5.3.

- Dữ liệu vào ra của bộ điều khiển PD mờ được thu thập để huấn luyện mạng thần kinh gồm dữ liệu vào là sai số $e(t)$ và đạo hàm sai số $\dot{e}(t)$, dữ liệu ra là tín hiệu điều khiển $u(t)$ (H.5.41, H.5.42).
- Cấu trúc mạng thần kinh (H.5.43): mạng thần kinh cần huấn luyện là mạng hai lớp có hai ngõ vào là $e(t)$, $\dot{e}(t)$ và một ngõ ra là $u(t)$; lớp ẩn gồm năm tế bào thần kinh có hàm kích hoạt dạng sigmoid, lớp ra gồm một tế bào thần kinh có hàm kích hoạt tuyến tính.

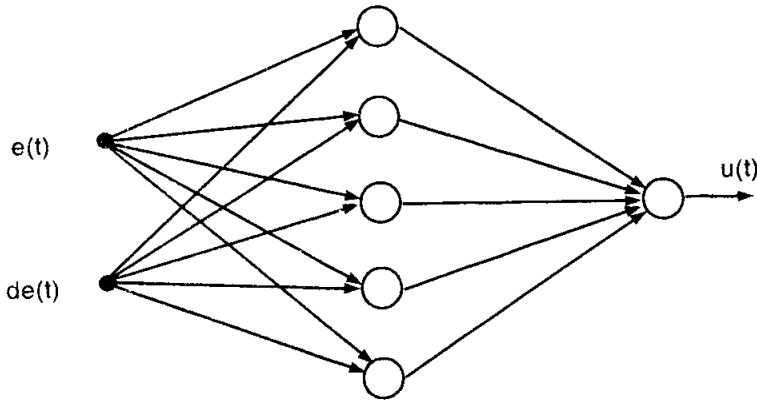
Chú ý là số tế bào thần kinh ở lớp ẩn có thể nhiều hơn, tùy theo tính phi tuyến của bộ điều khiển PD mờ mà mạng thần kinh cần học.



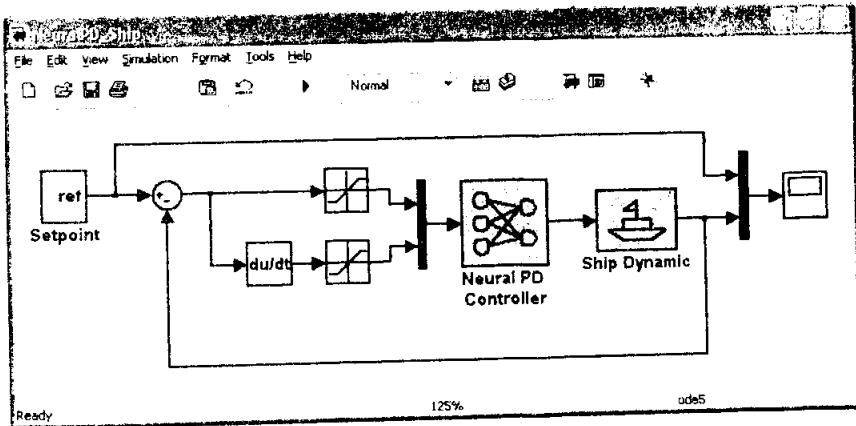
Hình 5.41 Sơ đồ thu thập dữ liệu vào-ra của bộ điều khiển mờ PD



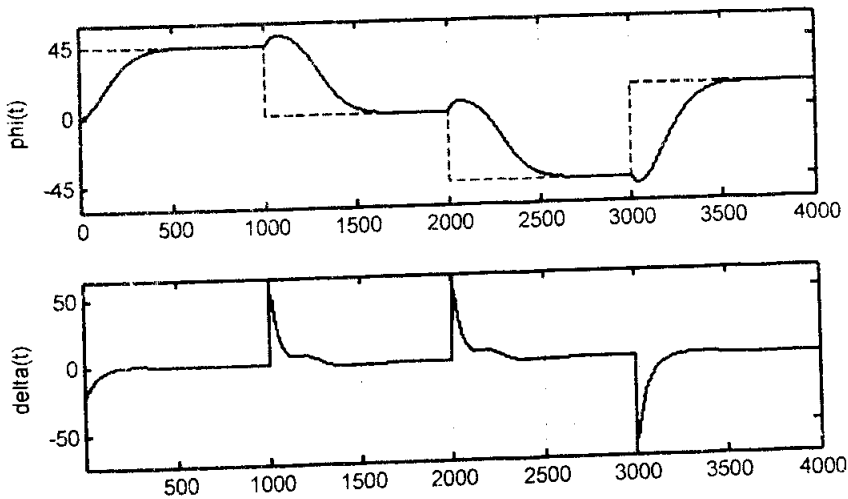
Hình 5.42 Dữ liệu vào-ra của bộ điều khiển PD mờ dùng để huấn luyện mạng thần kinh



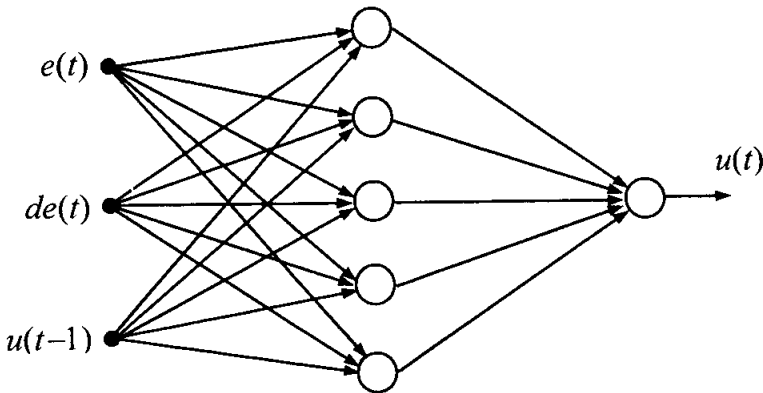
Hình 5.43 Cấu trúc mạng thần kinh sao chép bộ điều khiển PD mờ



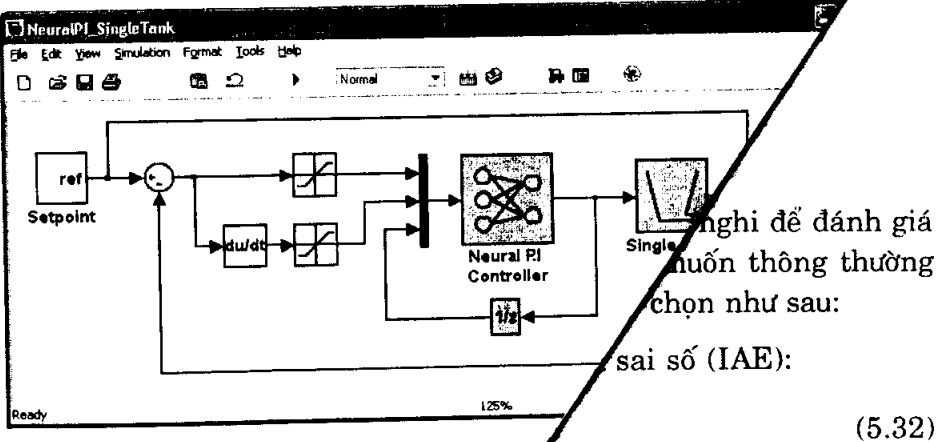
Hình 5.44 Bộ điều khiển neural PD sau khi huấn luyện



Hình 5.45 Kết quả điều khiển lái tàu dùng bộ điều khiển neural PD



Hình 5.48 Cấu trúc mạng thần kinh sao chép bộ điều khiển mờ



Hình 5.49 Bộ điều khiển neural

sai số (IAE):

$$(5.32)$$

sai số (ISE):

$$(5.33)$$

giao nhân với trị tuyệt đối của sai số

$$(5.34)$$

lượng khác cũng rất thường dùng là tối thiểu
ra và năng lượng của tín hiệu điều khiển:

$$fitness = \int_0^{\infty} e^2(t)dt + \rho \int_0^{\infty} u^2(t)dt \quad (5.35)$$

Hình 5.50 K

5.5 TỰ CHỈNH BỘ ĐIỀU KHIỂN TRỰC TIẾP DÙNG GIẢI THUẬT DI TRUYỀN

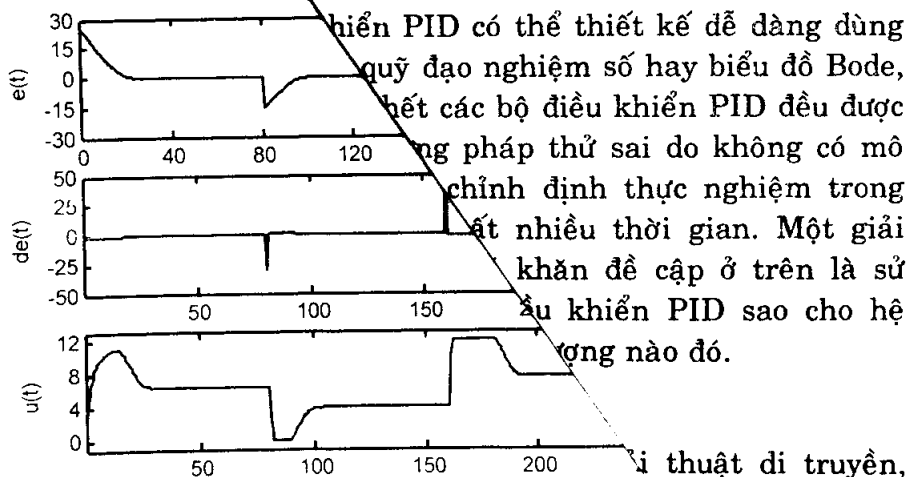
5.5.1 Giới thiệu

Thiết kế hệ thống điều khiển liên quan đến nhiều vấn đề như tính ổn định của hệ thống, chất lượng quá độ và xác lập, tính bền vững của hệ thống. Mỗi vấn đề phụ thuộc rất nhiều vào cấu trúc và thông số của bộ điều khiển. Tuy nhiên, sự phụ thuộc này thường không thể biểu diễn dưới dạng công thức toán học. Ngoài ra khi thiết kế cần dung hòa giữa các chỉ tiêu chất lượng mâu thuẫn nhau.

Rõ ràng việc thiếu các phương pháp mang tính hệ thống cũng như trực giác để chọn giá trị cho nhiều thông số điều khiển là một

lớn khi muốn được hệ thống điều khiển thỏa mãn yêu cầu. Để giải bài toán này dùng giải thuật di truyền, chúng ta mã hóa cấu trúc của bộ điều khiển thành chuỗi nhiễm sắc thể, và một hàm thích nghi là hàm của các yêu cầu chất lượng, bài toán thiết kế thành bài toán cực tiểu một hàm thông số bộ điều khiển. Vì giải thuật di truyền chỉ thực hiện trong quá trình tối ưu hóa, chúng có thể thực hiện được bài toán này. Sự kết hợp sáng tạo các phương pháp điều khiển và giải thuật di truyền có thể tạo nên một công cụ được nhiều bài toán điều khiển thực tế.

Hình 5.46 Thu thập dữ liệu PID kinh điển

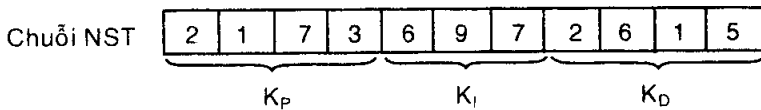


Hình 5.47 Dữ liệu vào-ra của bộ điều khiển

dùng để huấn luyện mạng thần kinh

mã hóa thông dụng là mã hóa nhị phân, mã hóa thập phân và mã hóa số thực. Nếu mã hóa thập phân, chuỗi nhiễm sắc thể biểu diễn lời giải của bài toán chỉnh định thông số bộ điều khiển PID có dạng như hình 5.51. Chú ý rằng, do ta đã biết trước các thông số của bộ điều khiển PID phải dương nên không cần mã hóa dấu của các thông số. Số gen tương ứng với mỗi thông số được chọn tùy theo tầm giá trị và độ chính xác cần đạt được của thông số đó.

Giải mã thực hiện các phép toán ngược lại so với mã hóa để tính các thông số (K_P, K_I, K_D) từ chuỗi nhiễm sắc thể.



Hình 5.51 Mã hóa thông số bộ điều khiển PID

Hàm thích nghi

Bước tiếp theo ta phải định nghĩa một hàm thích nghi để đánh giá các nhiễm sắc thể. Do chất lượng điều khiển mong muốn thông thường là tối thiểu sai số ngõ ra nên hàm thích nghi có thể chọn như sau:

Tiêu chuẩn tích phân giá trị tuyệt đối của sai số (IAE):

$$fitness = \int_0^{+\infty} |e(t)| dt \tag{5.32}$$

Tiêu chuẩn tích phân bình phương sai số (ISE):

$$fitness = \int_0^{+\infty} e^2(t) dt \tag{5.33}$$

Tiêu chuẩn tích phân thời gian nhân với trị tuyệt đối của sai số (ITAE):

$$fitness = \int_0^{+\infty} t|e(t)| dt \tag{5.34}$$

Một chỉ tiêu chất lượng khác cũng rất thường dùng là tối thiểu năng lượng sai số ngõ ra và năng lượng của tín hiệu điều khiển:

$$fitness = \int_0^{\infty} e^2(t) dt + \rho \int_0^{\infty} u^2(t) dt \tag{5.35}$$

Một cải tiến chỉ tiêu (5.35) được sử dụng rất hiệu quả trong thực tế là:

$$fitness = \rho_1 \int_0^{T_1} e^2(t)dt + \rho_2 \int_{T_1}^{T_2} e^2(t)dt + \rho_3 \int_0^{T_1} u^2(t)dt + \rho_4 \int_{T_1}^{T_2} u^2(t)dt \quad (5.36)$$

trong đó $T_1 = T_{qd}$, $T_2 = 2T_{qd}$ với T_{qd} là thời gian quá độ của hệ thống. Bằng cách thay đổi các trọng số ρ_i ta có thể thay đổi được chất lượng của bộ điều khiển PID cần chỉnh định.

Chú ý: Khi thực thi giải thuật di truyền trên máy tính, ta phải xấp xỉ phép tích phân liên tục trong hàm thích nghi bằng tích phân rời rạc.

Các phép toán di truyền

Về nguyên tắc có thể sử dụng tất cả các phép toán di truyền phù hợp với phương pháp mã hóa thập phân để tạo ra các lời giải mới tiệm cận đến lời giải tối ưu. Tuy nhiên, thực tế trong đa số các trường hợp ta có thể chọn các phép toán di truyền như sau:

- Phép chọn lọc: chọn lọc sắp hạng tuyến tính hoặc sắp hạng lũy thừa
- Phép lai ghép: lai ghép hai điểm
- Phép đột biến: đột biến đều.

Chú ý rằng, thông số của các phép toán di truyền ảnh hưởng rất lớn kết quả tìm kiếm lời giải tối ưu do đó cần chọn lựa phù hợp với mỗi bài toán cụ thể.

Khởi động thế hệ đầu tiên

Thông thường thế hệ đầu tiên được khởi động ngẫu nhiên. Tuy nhiên có thể khởi động thế hệ ban đầu dựa vào suy đoán của người thiết kế hoặc vận hành hệ thống về giá trị thông số của bộ điều khiển PID để thuật toán hội tụ nhanh hơn.

Ví dụ 5.9 Tự chỉnh bộ điều khiển PID điều khiển động cơ DC.

Cho động cơ DC có đặc tính động mô tả bởi hai phương trình vi phân:

$$\frac{di(t)}{dt} = -\frac{R}{L}i(t) - \frac{K_b}{L}\omega(t) + \frac{1}{L}u(t) \quad (5.37)$$

$$\frac{d\omega(t)}{dt} = \frac{K_m}{J} i(t) - \frac{K_f}{J} \omega(t) \quad (5.38)$$

trong đó: $u(t)$ - điện áp phần ứng (tín hiệu vào)

$\omega(t)$ - tốc độ quay của động cơ (tín hiệu ra)

$i(t)$ - dòng điện phần ứng.

Các thông số của động cơ như sau:

$R = 2 (\Omega)$ điện trở phần ứng

$L = 0,5 (H)$ điện cảm phần ứng

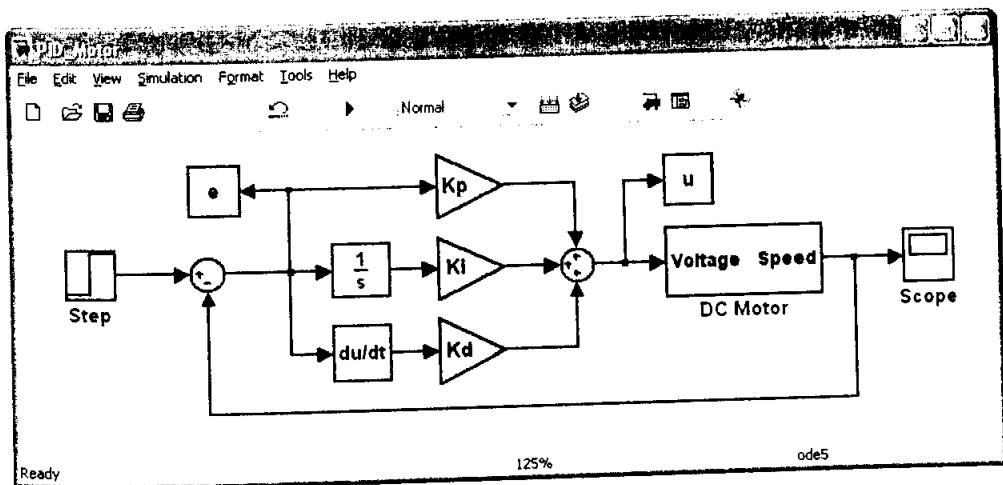
$K_m = 0,015$ hằng số moment

$K_b = 0,015$ hằng số sức điện động

$J = 0,02 (kg.m^2)$ moment quán tính của tải

$K_f = 0,2 (Nms)$ hệ số ma sát nhớt.

Yêu cầu đặt ra là thiết kế bộ điều khiển PID điều khiển tốc độ động cơ.



Hình 5.52 Sơ đồ mô phỏng điều khiển PID tốc độ động cơ

Hình 5.52 là sơ đồ mô phỏng hệ thống điều khiển PID tốc độ động cơ, thông số của bộ điều khiển PID được chỉnh định tự động dùng giải thuật di truyền với các tùy chọn như sau:

- Mã hóa thập phân: dấu chấm thập phân ở vị trí 2, lấy 5 chữ số có nghĩa.

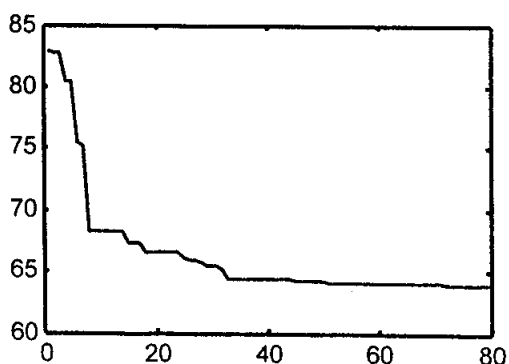
- Hàm thích nghi: (5.35) với giá trị trọng số $\rho = 0,02$. Khi thực thi giải thuật di truyền, để đánh giá độ thích nghi của nhiễm sắc thể trong mỗi thế hệ, trước tiên thông số của bộ điều khiển PID trong sơ đồ ở hình 5.52 được đặt bằng giá trị (K_P, K_I, K_D) tương ứng với nhiễm sắc thể đó; kế đến hệ thống điều khiển PID được mô phỏng, giá trị sai số $e(t)$ và tín hiệu điều khiển $u(t)$ được lưu vào bộ nhớ; cuối cùng độ thích nghi của NST được tính theo công thức (5.35).

- Kích thước quần thể: $N = 20$
- Chọn lọc xếp hạng tuyến tính: hệ số chọn lọc $\eta = 0,5$
- Lai ghép hai điểm: xác suất lai ghép $p_c = 0,9$
- Đột biến đều: xác suất đột biến $p_m = 0,1$
- Khởi động quần thể đầu tiên: ngẫu nhiên.

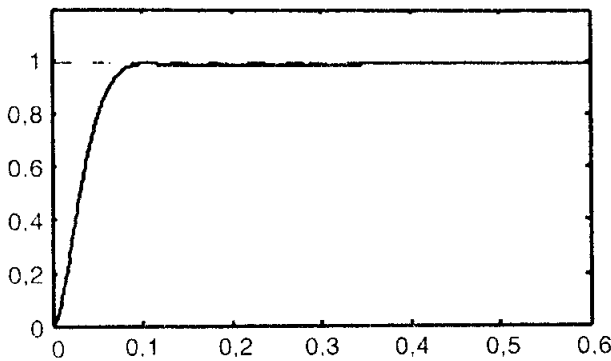
Hình 5.53 là kết quả chạy giải thuật di truyền tự chỉnh bộ điều khiển PID (xem chương trình GA ở phụ lục A.3). Ta thấy giá trị hàm thích nghi của nhiễm sắc thể tốt nhất trong quá trình tiến hóa từ thế hệ này sang thế hệ khác tiệm cận tiến đến giá trị cực tiểu. Đáp ứng thời gian của hệ thống của hệ thống sau khi tự chỉnh thông số bộ điều khiển PID rất tốt (H.5.54), thời gian quá độ của hệ thống chỉ khoảng 0,1 giây, không vọt lố, sai số xác lập bằng 0. Chú ý rằng có thể thay đổi chất lượng đáp ứng thời gian của hệ thống bằng cách thay đổi trọng số ρ của hàm thích nghi.

Hàm truyền của bộ điều khiển PID sau khi tự chỉnh dùng GA là:

$$G(s) = 7,25 + \frac{31,01}{s} + 0,12s$$



Hình 5.53 Sự thay đổi hàm thích nghi qua các thế hệ



Hình 5.54 Đáp ứng nấc của hệ thống điều khiển PID chỉnh định dùng GA

5.5.3 Chỉnh bộ điều khiển LQR

Bộ điều khiển LQR

Cho đối tượng điều khiển mô tả bởi phương trình trạng thái:

$$\begin{cases} \dot{x}(t) = Ax(t) + Bu(t) \\ y(t) = Cx(t) \end{cases} \quad (5.39)$$

Bộ điều khiển hồi tiếp trạng thái

$$u(t) = -Kx(t) \quad (5.40)$$

Trong số các phương pháp thông thường dùng để thiết kế bộ điều khiển hồi tiếp trạng thái (5.40), phương pháp LQR (*Linear Quadratic Regulator*) được sử dụng rất phổ biến. Theo phương pháp LQR, vector K được tính toán sao cho chất lượng của hệ thống điều khiển cực tiểu hóa chỉ tiêu chất lượng dạng toàn phương tuyến tính sau đây:

$$J = \int_0^{\infty} x^T Q x dt + \int_0^{\infty} u^T R u dt \quad (5.41)$$

Lý thuyết điều khiển tối ưu chứng minh được vector K cực tiểu hóa chỉ tiêu chất lượng (5.41) xác định bởi biểu thức:

$$K = R^{-1} B^T P \quad (5.42)$$

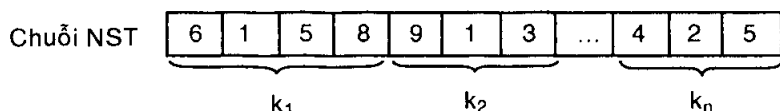
trong đó P là nghiệm của phương trình Riccati:

$$A^T P + PA - PBR^{-1}B^T P = -Q \quad (5.43)$$

Giải thuật di truyền tự chỉnh bộ điều khiển LQR

Nếu không biết mô hình toán tuyến tính của đối tượng điều khiển thì không thể tìm lời giải tối ưu cho bài toán điều khiển LQR theo các công thức giải tích (5.42) và (5.43), trong trường hợp này ta có thể tìm lời giải tối ưu cho bài toán LQR dùng giải thuật di truyền.

Mã hóa: Lời giải của bài toán LQR (vector $K = [k_1 \ k_2 \ \dots \ k_n]$) được mã hóa thành chuỗi nhiễm sắc thể mã thập phân:



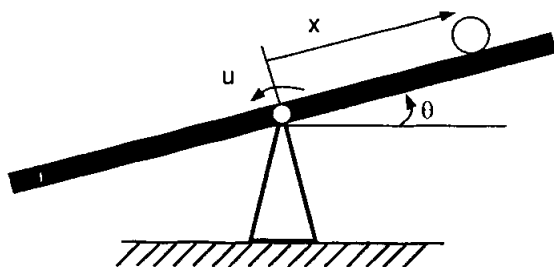
Hình 5.55 Mã hóa lời giải bài toán LQR

Hàm thích nghi: chính là tiêu chuẩn LQR

$$fitness = \int_0^{\infty} x^T Q x dt + \int_0^{\infty} u^T R u dt \quad (5.44)$$

Các phép toán di truyền: chọn tương tự như GA chỉnh bộ điều khiển PID.

Ví dụ 5.10 Tự chỉnh bộ điều khiển LQR giữ cân bằng hệ bóng và thanh.



Hình 5.56 Hệ bóng và thanh (ball and beam)

Hệ bóng và thanh gồm một thanh dầm gắn vào trục quay song song với phương nằm ngang, góc quay của thanh dầm có thể thay đổi bằng cách điều khiển động cơ DC tác động moment $u(t)$ vào trục quay, trên thanh dầm có một quả cầu đường kính R , khối lượng m có thể lăn tự do (không trượt). Tương tự hệ con lắc ngược, hệ bóng và thanh cũng là một trong những đối tượng phi tuyến không ổn định điển hình thường được sử dụng để kiểm chứng lý thuyết điều khiển tự động.

Phương trình trạng thái mô tả đặc tính động học của hệ bóng-và-thanh như sau (bỏ qua ma sát):

$$\ddot{\theta} = \frac{u - 2mx\dot{\theta} - mgx \cos \theta}{J_B + mx^2} \quad (5.45)$$

$$\ddot{x} = \frac{mx\dot{\theta}^2 - mg \sin \theta}{(J/R^2) + m} \quad (5.46)$$

trong đó: u - moment tác động vào trục quay

θ - góc tạo bởi thanh và phương nằm ngang

x - vị trí bóng trên thanh

$m = 0,1 (kg)$ - khối lượng của bóng

$J = 10^{-5} (kgm^2)$ - moment quán tính của bóng

$R = 0,015 (m)$ - đường kính bóng

$J_B = 0,05 (kgm^2)$ - moment quán tính của thanh

$g = 9,81 (m/s^2)$ - gia tốc trọng trường.

Trong ví dụ này bộ điều khiển LQR giữ cân bằng hệ bóng và thanh sẽ được chỉnh định tự động dùng GA. Chú ý rằng các phương trình vi phân mô tả đặc tính động học của hệ bóng và thanh chỉ sử dụng để mô phỏng (H.5.57), khi ứng dụng thực nghiệm ta không cần biết các phương trình này.

Các thông số của GA chỉnh bộ điều khiển LQR được chọn như sau:

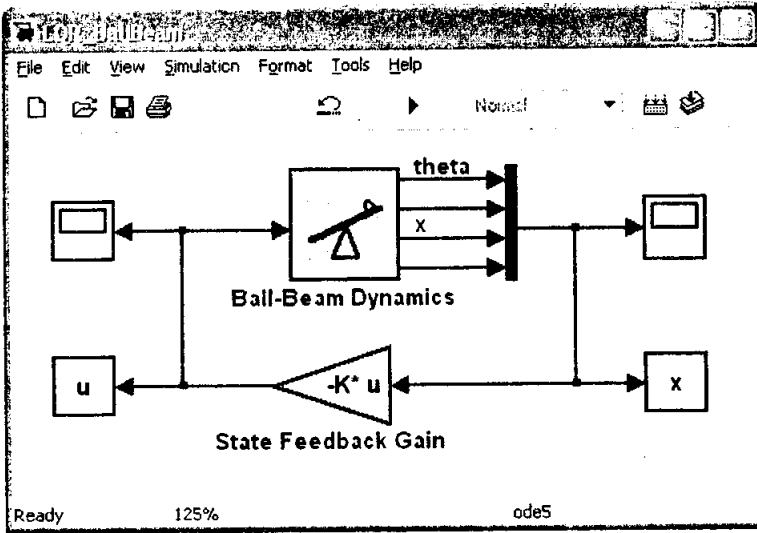
- Mã hóa thập phân (5 chữ số có nghĩa, 2 chữ số trước dấu chấm thập phân)
- Hàm thích nghi là chuẩn LQR với:

$$Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$R = 0,1$$

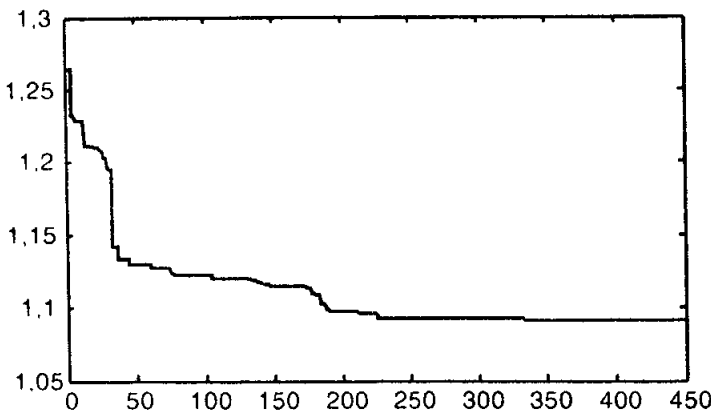
- Kích thước quần thể $N = 20$
- Chọn lọc sắp hạng tuyến tính, hệ số chọn lọc $\eta = 0,2$
- Lai ghép hai điểm, xác suất lai ghép $p_C = 0,9$
- Đột biến đều, xác suất đột biến $p_M = 0,1$.

Kết quả chạy chương trình GA hội tụ sau khoảng 350 thế hệ, vector hồi tiếp trạng thái K là: $[4.000 \quad 1.160 \quad -2.923 \quad -2.024]$

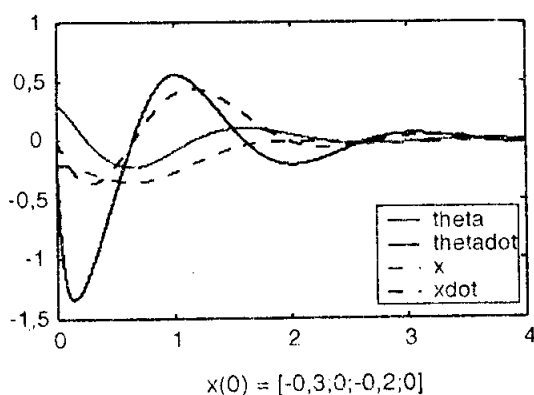
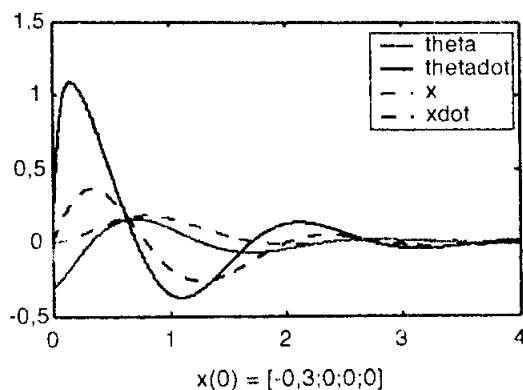


Hình 5.57 Sơ đồ mô phỏng điều khiển LQR giữ cân bằng hệ bóng và thanh

Hình 5.58 cho thấy chuẩn LQR giảm dần đến giá trị cực tiểu qua các thế hệ. Kết quả mô phỏng cho thấy bộ điều khiển LQR chỉnh bằng GA có thể điều khiển hệ bóng và thanh về trạng thái cân bằng từ các điều kiện đầu khác nhau với thời gian quá độ khoảng 3 giây.



Hình 5.58 Chuẩn LQR giảm dần qua các thế hệ



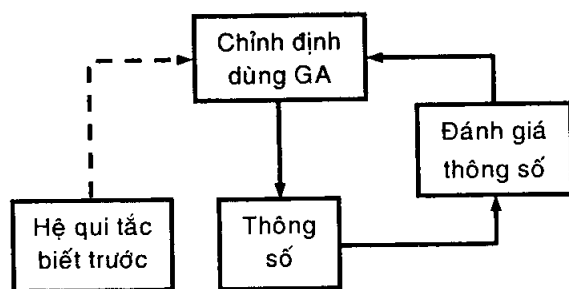
Hình 5.59 Đáp ứng của hệ thống LQR sau khi chỉnh định dùng GA

5.5.4 Tự chỉnh bộ điều khiển mờ

Như đã trình bày ở mục 5.2, điều khiển mờ là một phương pháp điều khiển thông minh phỏng theo quá trình xử lý thông tin không rõ ràng và ra quyết định điều khiển của con người. Phương pháp này rất thích hợp để điều khiển các đối tượng phức tạp, không xác định được mô hình toán. Tuy nhiên, bộ điều khiển mờ trực tiếp thường được thiết kế dựa vào kinh nghiệm chuyên gia và mang nặng tính “thử sai”, do đó khi gặp các đối tượng phức tạp, người thiết kế sẽ mất rất nhiều thời gian mà kết quả có được có thể sẽ không tối ưu. Vấn đề tự chỉnh bộ điều khiển mờ đã được quan tâm nghiên cứu rất nhiều từ khi điều khiển mờ khẳng định được là một phương pháp hiệu quả để điều khiển các đối tượng phức tạp. Trong các phương pháp tự chỉnh được sử dụng hiện nay, tự chỉnh dùng GA có thể nói là phương pháp tự nhiên nhất và cũng rất hiệu quả. GA chỉnh định bộ điều khiển mờ có thể chia làm hai nhóm:

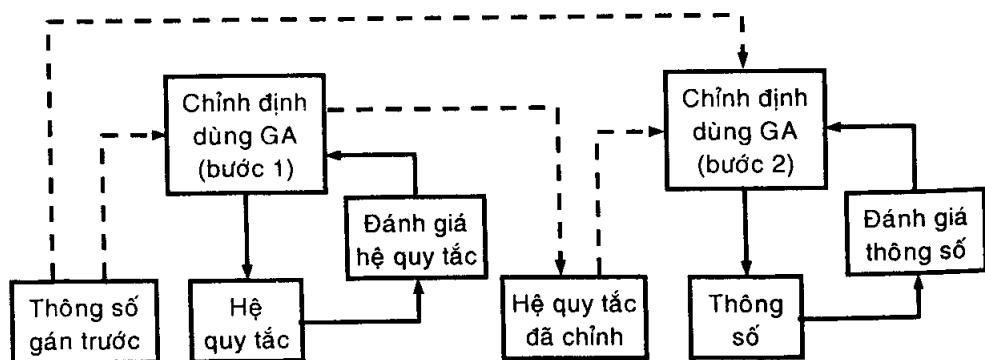
a- Chỉnh thông số

Trong một số trường hợp, người thiết kế đã đưa ra được hệ quy tắc mờ để điều khiển đối tượng dựa trên kinh nghiệm (ví dụ như điều khiển hệ con lắc ngược) hoặc hệ quy tắc mờ để giải bài toán đã quá quen thuộc (ví dụ như hệ quy tắc PI mờ, PD mờ). Vấn đề đặt ra là phải chỉnh định thông số của bộ điều khiển (bao gồm chỉnh định các hệ số chuẩn hóa và thông số của các hàm liên thuộc mô tả các giá trị ngôn ngữ) để tối ưu hóa một chỉ tiêu chất lượng nào đó. GA có thể sử dụng để giải rất hiệu quả bài toán này.



Hình 5.60 Chỉnh thông số bộ điều khiển mờ

b- Chỉnh hệ quy tắc và thông số

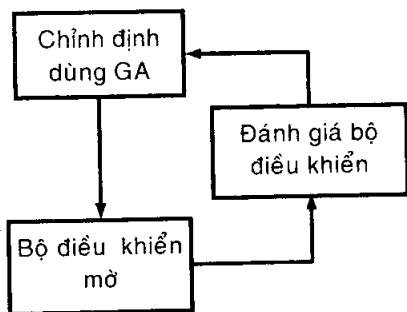


Hình 5.61 Chỉnh hệ quy tắc sau đó chỉnh thông số bộ điều khiển mờ

Trong trường hợp đối tượng quá phức tạp, người thiết kế chưa tích lũy đủ kinh nghiệm để đưa ra được các quy luật điều khiển thì có thể sử dụng GA để chỉnh định cả hệ quy tắc và thông số của bộ điều khiển mờ. Có hai cách chỉnh định:

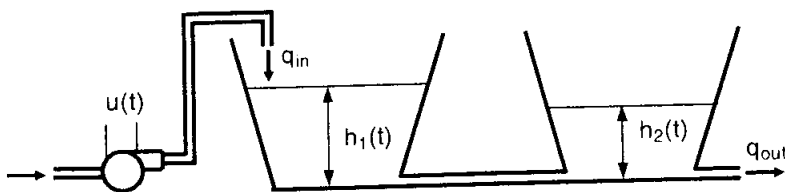
- Chỉnh định hai bước (H.5.61): bước 1 gán trước thông số hàm liên thuộc và các hệ số chuẩn hóa, sử dụng GA để chỉnh hệ quy tắc. Sau khi đã rút ra được hệ quy tắc, tiếp tục bước 2 dùng GA chỉnh thông số của bộ điều khiển để cải thiện chất lượng.

- Chỉnh định một bước (H.5.62) sử dụng GA chỉnh đồng thời cả hệ quy tắc và thông số.



Hình 5.62 Chỉnh bộ điều khiển mờ

Ví dụ 5.11 GA chỉnh thông số bộ điều khiển PI mờ hệ bồn chứa nối tiếp.



Hình 5.63 Hệ bồn chứa chất lỏng

Cho hệ bồn chứa nối tiếp có mô hình toán như sau (bỏ qua đặc tính phi tuyến của máy bơm và thời gian trễ lưu chất di chuyển trong đường ống):

$$\dot{h}_1(t) = \frac{1}{A_1(h_1)} \left(ku(t) - \text{sgn}(h_1(t) - h_2(t)) C_D a_1 \sqrt{2g | h_1(t) - h_2(t) |} \right) \quad (5.47)$$

$$\dot{h}_2(t) = \frac{1}{A_2(h_2)} \left(\text{sgn}(h_1(t) - h_2(t)) C_D a_1 \sqrt{2g | h_1(t) - h_2(t) |} - C_D a_2 \sqrt{2gh_2(t)} \right) \quad (5.48)$$

$$A_i(h_i) = \frac{A_{\max} - A_{\min}}{h_{\max}} h_i + A_{\min} \quad (i = 1, 2) \quad (5.49)$$

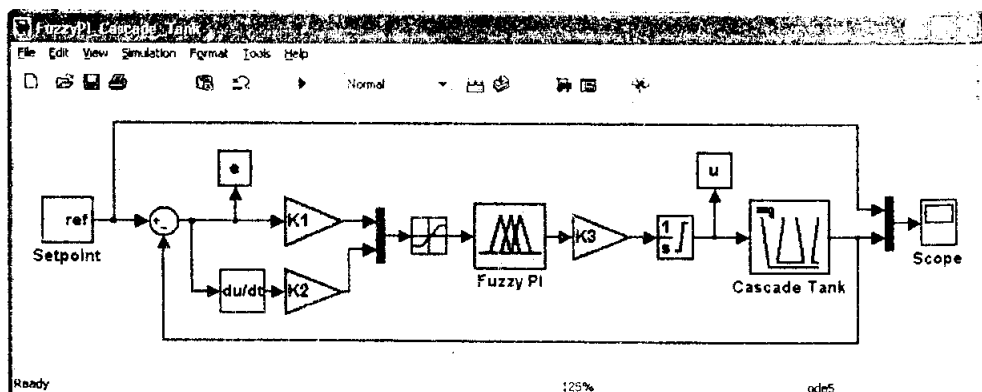
trong đó: $u(t)$ - tín hiệu điều khiển máy bơm ($0 \leq u(t) \leq 1$)
 $h_1(t)$, $h_2(t)$ - độ cao mực chất lỏng trong bồn (cm)
 $A_1(h_1)$, $A_2(h_2)$ - tiết diện ngang bồn chứa (cm^2)
 $h_{\max}$ - độ cao cực đại của bồn chứa
 $A_{\max}$, $A_{\min}$ - tiết diện ngang cực đại và cực tiểu
 k - hệ số tỷ lệ với công suất máy bơm
 a_1 , a_2 - tiết diện van xả (cm^2)
 g - gia tốc trọng trường ($981cm/sec^2$)
 C_D - hệ số xả.

Thông số vật lý của hệ thống được chọn như sau:

$$h_{\max} = 50cm; A_{\max} = 200cm^2; A_{\min} = 100cm^2; a_1 = 1cm^2$$

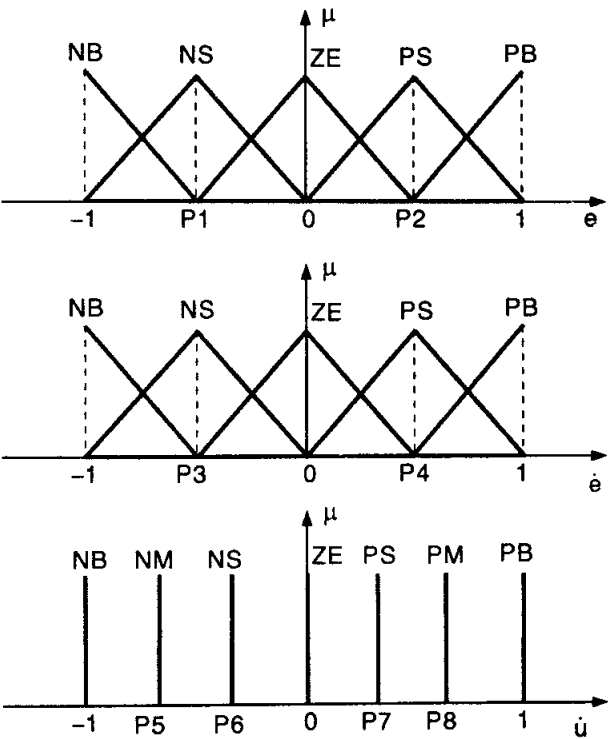
$$a_2 = 0,4cm^2; k = 300cm^3/sec; C_D = 0,6$$

Bài toán đặt ra là thiết kế bộ điều khiển mờ để điều khiển mực chất lỏng trong bồn 2 bảm theo tín hiệu đặt.



Hình 5.64 Sơ đồ mô phỏng hệ thống điều khiển PI mờ hệ bồn nối tiếp

Vì hệ bồn chứa không có khâu tích phân lý tưởng nên để hệ thống vô sai đối với tín hiệu vào dạng hàm nấc, ta phải sử dụng bộ điều khiển PI mờ. Các giá trị ngôn ngữ của các biến vào và biến ra của hệ mờ được định nghĩa ở hình 5.65, thông số của các tập mờ có thể thay đổi được. Hệ quy tắc điều khiển PI mờ được trình bày ở bảng 5.7, đây là hệ quy tắc rất quen thuộc có thể suy ra dễ dàng. Như vậy cấu trúc của bộ điều khiển PI mờ đã được xác định, vấn đề tiếp theo là chỉnh định thông số của bộ điều khiển, bao gồm các thông số của hàm liên thuộc ($P1-P8$) và các hệ số chuẩn hóa ($K1, K2, K3$).



Hình 5.65 Các giá trị ngôn ngữ trong bài toán điều khiển hệ bốn chứa nối tiếp

Bảng 5.7 Các quy tắc mờ điều khiển hệ bốn nối tiếp

DU		E				
		NB	NE	ZE	PO	PB
DE	NB	NB	NB	NM	NS	ZE
	NE	NB	NM	NS	ZE	PS
	ZE	NM	NS	ZE	PS	PM
	PO	NS	ZE	PS	PM	PB
	PB	ZE	PS	PM	PB	PB

GA chỉnh định bộ điều khiển PI mờ

Mã hóa: Mỗi thông số cần chỉnh định (các thông số tập mờ P1-P8 và các hệ số chuẩn hóa K1, K2, K3) được mã hóa thành một giên trên chuỗi NST mã số thực như hình 5.66.

P1	P2	P3	...	P8	K1	K2	K3
----	----	----	-----	----	----	----	----

Hình 5.66 Chuỗi NST mã hóa thông số bộ điều khiển PI mờ

Hàm thích nghi: chọn hàm thích nghi như sau

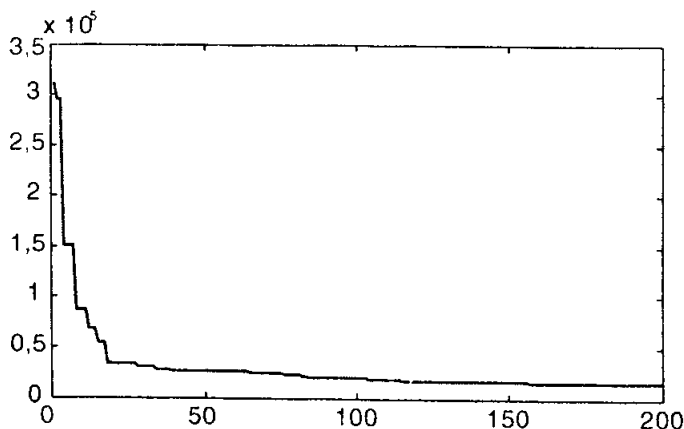
$$fitness = \int_0^{T_{qd}} e^2(t) dt + \rho \int_{T_{qd}}^{2T_{qd}} e^2(t) dt \quad (5.50)$$

trong đó $T_{qd} = 75 \text{ sec}$, $\rho = 10000$.

Lý do của việc tăng trọng số của hàm thích nghi trong khoảng thời gian từ T_{qd} đến $2T_{qd}$ là để GA tìm kiếm bộ thông số tối ưu làm cho đáp ứng của hệ thống không còn dao động từ thời điểm T_{qd} trở đi.

Các tùy chọn của GA

- Kích thước quần thể: $N = 30$, khởi động quần thể ban đầu ngẫu nhiên
- Chọn lọc sắp hạng tuyến tính: $\eta = 0,3$
- Lai ghép BLX- α ($\alpha = 0,5$); xác suất lai ghép $p_c = 0,9$
- Đột biến ngẫu nhiên, xác suất đột biến $p_m = 0,1$.



Hình 5.67 Hàm thích nghi giảm dần khi chỉnh thông số bộ điều khiển PI mờ

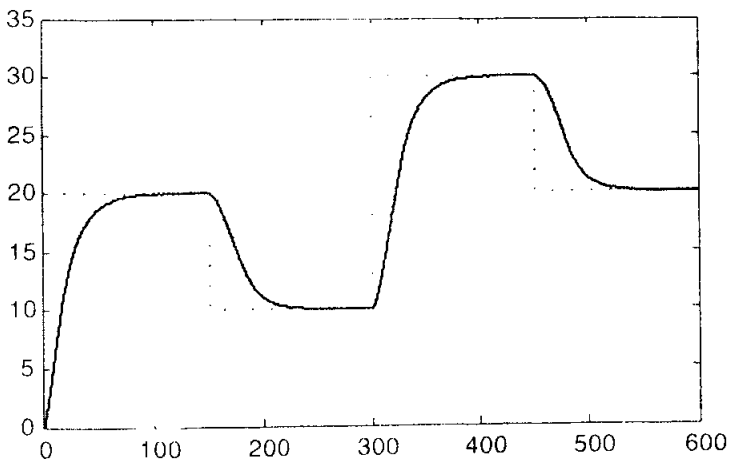
Kết quả: Chương trình GA chỉnh định bộ điều khiển PI mờ được trình bày ở phụ lục A.4. Thông số bộ điều khiển PI mờ chỉnh định dùng GA như sau:

$$P_1 = -0,2796; \quad P_2 = 0,2844; \quad P_3 = -0,6486; \quad P_4 = 0,5695$$

$$P_5 = -0,4778; \quad P_6 = -1,4999; \quad P_7 = 0,6242; \quad P_8 = 1,1791$$

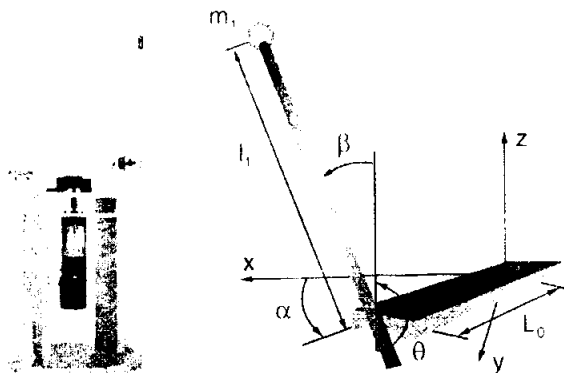
$$K_1 = 0,0879; \quad K_2 = 1,4201; \quad K_3 = 14,9504$$

Kết quả mô phỏng (H.5.68) cho thấy đáp ứng của hệ thống sau khi chỉnh định không vọt lố, thời gian quá độ khoảng 75sec, sai số xác lập bằng 0.



Hình 5.68 Đáp ứng của hệ thống sau khi chỉnh định dùng GA

Ví dụ 5.12 Tự chỉnh bộ điều khiển mờ giữ cân bằng hệ con lắc ngược quay



Hình 5.69 Hệ con lắc ngược quay

Hệ con lắc ngược quay gồm hai phần: *cánh tay* gắn vào động cơ DC quay quanh trục thẳng đứng và *con lắc* gắn vào trục quay ở cuối cánh tay quay tự do trong mặt phẳng vuông góc với cánh tay. Con lắc

không ổn định, nó luôn ngã xuống trừ khi có lực tác động thích hợp vào phần cánh tay. Bài toán đặt ra là điều khiển cánh tay để giữ cho con lắc cân bằng ở vị trí thẳng đứng. Hệ con lắc ngược quay là một phiên bản cải tiến của hệ xe-con lắc ngược trên đường ray thẳng nhằm tiết kiệm không gian và loại bỏ yếu tố bão hòa do giới hạn đường chạy của xe. Hệ con lắc ngược quay là hệ phi tuyến không ổn định rất khó điều khiển. Mô hình toán của hệ con lắc ngược quay như sau:

$$\begin{aligned} & \begin{bmatrix} J_0 + m_1 L_0^2 + m_1 l_1^2 \sin^2 \beta & -m_1 L_0 l_1 \cos \beta \\ -m_1 L_0 l_1 \cos \beta & J_1 + m_1 l_1^2 \end{bmatrix} \begin{bmatrix} \ddot{\alpha} \\ \ddot{\beta} \end{bmatrix} \\ & + \begin{bmatrix} C_0 + \frac{1}{2} m_1 l_1^2 \dot{\beta} \sin 2\beta & m_1 L_0 l_1 \dot{\beta} \sin \beta + \frac{1}{2} m_1 l_1^2 \dot{\alpha} \sin 2\beta \\ -\frac{1}{2} m_1 l_1^2 \dot{\alpha} \sin 2\beta & \end{bmatrix} \begin{bmatrix} \dot{\alpha} \\ \dot{\beta} \end{bmatrix} \\ & + \begin{bmatrix} 0 \\ -m_1 g l_1 \sin \beta \end{bmatrix} = \begin{bmatrix} \tau \\ 0 \end{bmatrix} \quad (5.51) \end{aligned}$$

trong đó: α - góc quay của cánh tay

β - góc quay của con lắc

τ - moment tác động vào trục quay của cánh tay

$m_1 = 0,0319 (kg)$ - khối lượng con lắc

$l_1 = 0,1572 (m)$ - chiều dài con lắc

$L_0 = 0,1370 (m)$ - khoảng cách từ trục quay cánh tay đến trục quay con lắc

$J_0 = 0,008591 (kg.m^2)$ - moment quán tính của cánh tay

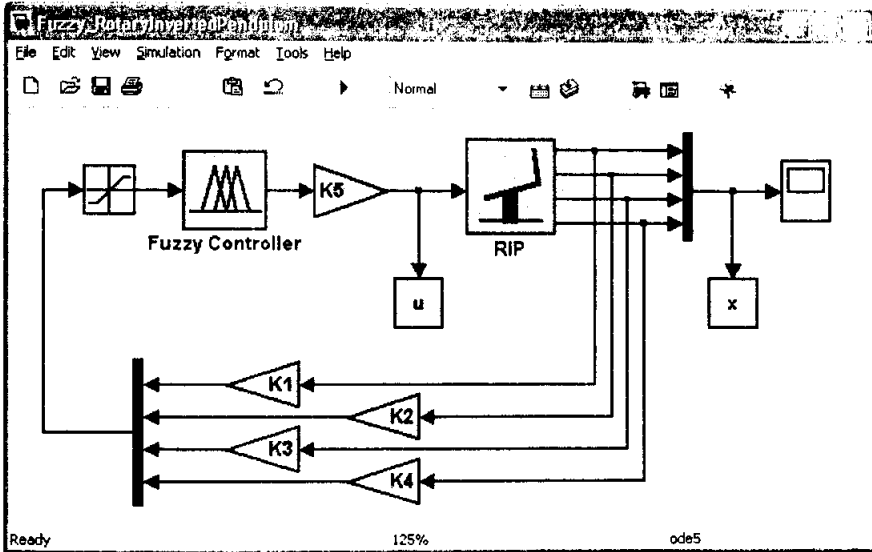
$J_1 = 0,000217 (kg.m^2)$ - moment quán tính của con lắc

$C_0 = 0,006408 (kg.m^2/s)$ - hệ số ma sát trục quay cánh tay

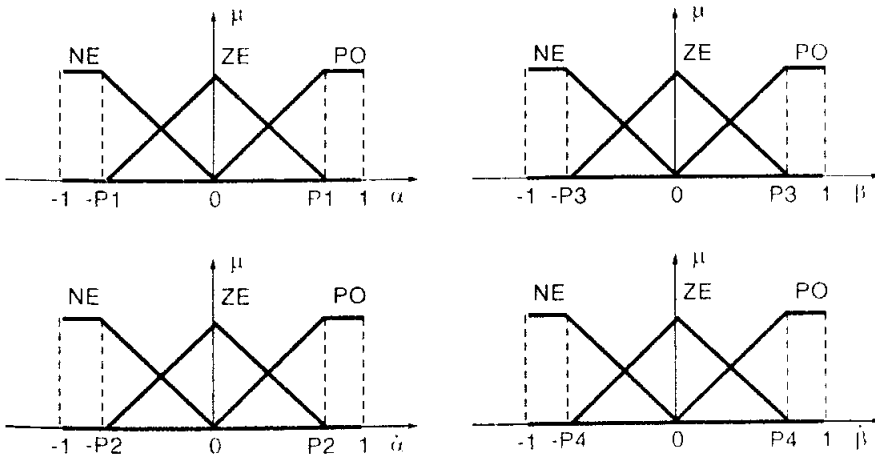
$C_1 = 0,000158 (kg.m^2/s)$ - hệ số ma sát trục quay con lắc

$g = 9,81 (m/s^2)$ - gia tốc trọng trường.

Hình 5.70 là sơ đồ mô phỏng hệ thống điều khiển mờ giữ cân bằng hệ con lắc ngược quay. Các biến vào của bộ điều khiển là α , $\dot{\alpha}$, β và $\dot{\beta}$, tương ứng với mỗi biến vào ta định nghĩa 3 giá trị ngôn ngữ là NE, ZE và PO (H.5.71). Biến ra là moment quay cánh tay τ , định nghĩa 9 giá trị ngôn ngữ cho biến τ là NE4, NE3, NE2, NE1, ZE, PO1, PO2, PO3, PO4 (H.5.72).



Hình 5.70 Sơ đồ điều khiển mờ hệ con lắc ngược quay



Hình 5.71 Các giá trị ngôn ngữ của các biến ở ngõ vào

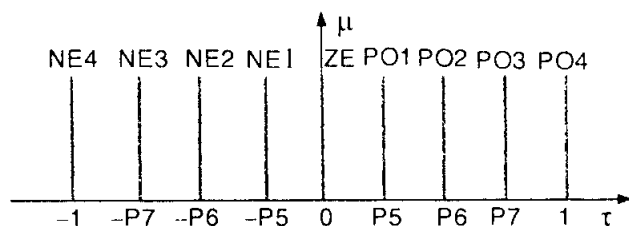
Với các biến vào ra và các giá trị ngôn ngữ định nghĩa như trên, bộ điều khiển mờ đầy đủ gồm 81 quy tắc, quy tắc thứ i có dạng:

Nếu (α là $\tilde{A}_{i1}$) và (α là $\tilde{A}_{i2}$) và (β là $\tilde{A}_{i3}$)

và (β là $\tilde{A}_{i4}$) thì (τ là $\tilde{B}_i$) (5.52)

Thông thường, hệ quy tắc (5.52) và thông số của bộ điều khiển (vị trí hàm liên thuộc, các hệ số chuẩn hóa) được rút ra qua quá trình chỉnh định thử sai. Khi chỉnh bộ điều khiển mờ bằng phương pháp thử sai đầu

tiên ta hay chọn trước các tập mờ mô tả các giá trị ngôn ngữ, các tập mờ này có dạng đơn giản như dạng tam giác và được phân chia đều trên miền xác định của biến ngôn ngữ. Sau đó bằng cách dựa vào kinh nghiệm, cố gắng đưa ra một hệ quy tắc có thể điều khiển được đối tượng. Cuối cùng tinh chỉnh thông số và hình dạng của các hàm liên thuộc (nếu cần) để được bộ điều khiển có chất lượng tốt nhất. Các nghiên cứu cho thấy đặc tính của bộ điều khiển mờ chịu ảnh hưởng quyết định bởi hệ quy tắc; hình dạng, vị trí của các tập mờ mô tả các giá trị ngôn ngữ không đóng vai trò quan trọng. Nếu hệ quy tắc không đúng, điều đó có nghĩa là kinh nghiệm chuyên gia trong việc điều khiển đối tượng là sai thì dù có chỉnh thông số và hình dạng của các hàm liên thuộc như thế nào đi nữa thì bộ điều khiển cũng sẽ không làm việc được. Trên cơ sở nhận xét trên, trong ví dụ này GA được sử dụng để chỉnh định bộ điều khiển mờ qua hai bước (H.5.61), bước 1 chỉnh hệ quy tắc với thông số hàm liên thuộc và các hệ số chuẩn hóa chọn trước, bước 2 tiếp tục chỉnh thông số sau khi đã có hệ quy tắc để được bộ điều khiển tối ưu.



Hình 5.72 Các giá trị ngôn ngữ của biến ở ngõ ra

GA chỉnh bộ điều khiển mờ

Hàm thích nghi: Để đánh giá các lời giải trong quá trình tiến hóa, sử dụng hàm thích nghi có dạng năng lượng như sau:

$$fitness = \int_0^T [\rho_1 \alpha^2(t) + \rho_2 \dot{\alpha}^2(t) + \rho_3 \beta^2(t) + \rho_4 \dot{\beta}^2(t) + \beta_5 u^2(t)] dt \quad (5.53)$$

Các trọng số tùy chọn $\rho_i > 0$ ($i = \overline{1,4}$) sẽ quyết định chất lượng quá độ của hệ thống điều khiển, do đó cần chọn phù hợp để GA chỉnh định được bộ điều khiển đúng yêu cầu.

Mã hóa hệ quy tắc: Chúng ta thiết kế một hệ quy tắc đầy đủ, nghĩa là gồm tất cả các quy tắc có thể có. Trong trường hợp này mệnh đề điều kiện của hệ quy tắc đã hoàn toàn xác định, do đó chỉ cần gán tương ứng với mỗi mệnh đề điều kiện một mệnh đề kết luận.

Ký hiệu các giá trị ngôn ngữ của mệnh đề kết luận bằng các số nguyên như bảng 5.8, khi đó hệ quy tắc mờ có thể được mã hóa thành chuỗi NST gồm 81 gene như hình 5.73a, trong hình vẽ B_i là mệnh đề kết luận của quy tắc thứ i .

Cách mã hóa hệ quy tắc như trên rất linh hoạt, có thể áp dụng để chỉnh hệ quy tắc mờ trong nhiều bài toán điều khiển khác nhau. Tuy nhiên, đối với hệ con lắc ngược quay ta thấy không cần thiết phải mã hóa đầy đủ cả hệ quy tắc như vậy. Để ý rằng bài toán giữ cân bằng hệ con lắc ngược quay có tính chất *đối xứng*, nếu sử dụng tính chất này chúng ta chỉ cần mã hóa các quy tắc từ 1 đến 40, quy tắc 42 đến 81 đối xứng với 40 quy tắc đầu, quy tắc thứ 41 chọn cố định là:

Nếu (α là ZE) và (α là ZE) và (β là ZE) và (β là ZE) thì (τ là ZE)

để con lắc có thể cân bằng. Với cách mã hóa như trên, chiều dài chuỗi NST rút ngắn còn 40 gene (H.5.73b).

Bảng 5.8 Bộ gen dùng để mã hóa hệ quy tắc

Giá trị ngôn ngữ	N4	N3	N2	N1	ZE	P1	P2	P3	P4
Giá trị của gen	1	2	3	4	5	6	7	8	9

B1	B2	B3	...	B79	B80	B81
----	----	----	-----	-----	-----	-----

a) NST 81 gene

B1	B2	B3	...	B38	B39	B40
----	----	----	-----	-----	-----	-----

b) NST 40 gene

Hình 5.73 Nhiễm sắc thể mã hóa hệ quy tắc

Mã hóa thông số: Do bài toán giữ cân bằng hệ con lắc ngược quay có tính đối xứng nên ta định nghĩa các tập mờ mô tả các giá trị ngôn ngữ của các biến vào ra có tính đối xứng như hình 5.71 và hình 5.72. Như vậy ta cần chỉnh định các thông số P1, P2,... P7 của các hàm liên thuộc và các hệ số K1, K2,... K5 của khâu tiền xử lý và hậu xử lý (H.5.70). Các thông số này được mã hóa thành chuỗi NST mã số thực, mỗi gen trên chuỗi NST tương ứng với một thông số cần chỉnh định như hình 5.74.

P1	P2	...	P7	K1	K2	...	K5
----	----	-----	----	----	----	-----	----

Hình 5.74 Nhiệm sắc thể mã hóa thông số của bộ điều khiển mờ

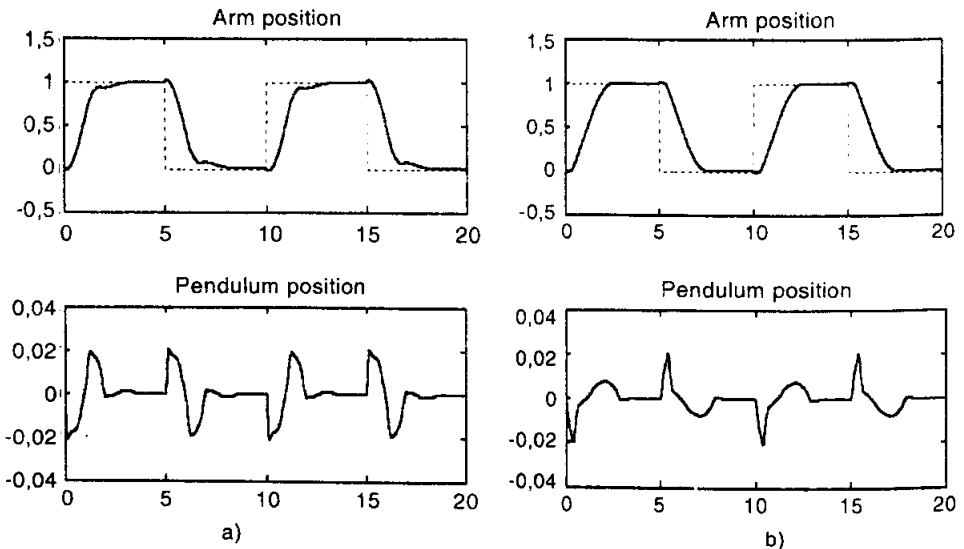
Các tùy chọn của GA:

- Kích thước quần thể: $N = 20$, khởi động quần thể ban đầu ngẫu nhiên
- Chọn lọc sắp hạng tuyến tính: $\eta = 0,3$
- Lai ghép:
 - Bước 1: Lai ghép hai điểm
 - Bước 2: Lai ghép BLX- α ($\alpha = 0,5$)
 - Xác suất lai ghép $p_c = 0,9$.
- Đột biến:
 - Bước 1: Đột biến một điểm
 - Bước 2: Đột biến ngẫu nhiên
 - Xác suất đột biến $p_m = 0,1$.
- Giá trị thông số chọn trước để chỉnh hệ quy tắc ở bước 1:

$$P_1 = P_2 = P_3 = P_4 = 0,5; \quad P_5 = 0,25; \quad P_6 = 0,5; \quad P_7 = 0,75$$

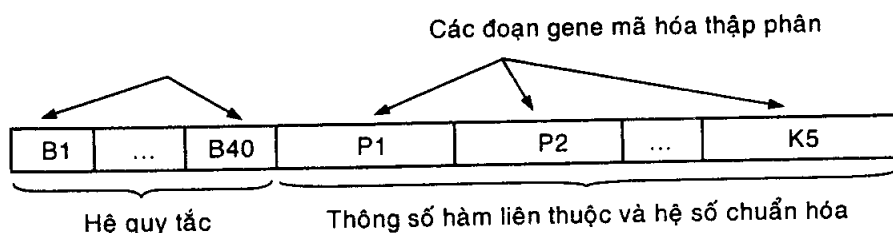
$$K_1 = 1/30; \quad K_2 = 1/2; \quad K_3 = 1/30; \quad K_4 = 1/2; \quad K_5 = 10$$

Kết quả chỉnh định dùng GA



Hình 5.75 Kết quả điều khiển giữ cân bằng hệ con lắc ngược
a) Sau khi chỉnh hệ quy tắc; b) Sau khi chỉnh thông số

Chú ý: GA chỉnh bộ điều khiển qua hai bước như vừa trình bày ở trên chỉ có thể tìm được lời giải gần tối ưu. Để tìm được bộ điều khiển tối ưu GA phải chỉnh định đồng thời hệ quy tắc và thông số tập mờ, trong trường hợp này ta mã hóa bộ điều khiển mờ thành chuỗi NST gồm hai phần, phần đầu mã hóa hệ quy tắc, phần sau mã hóa thông số dùng mã thập phân (H.5.76).



Hình 5.76 Mã hóa đồng thời hệ quy tắc và thông số của bộ điều khiển mờ

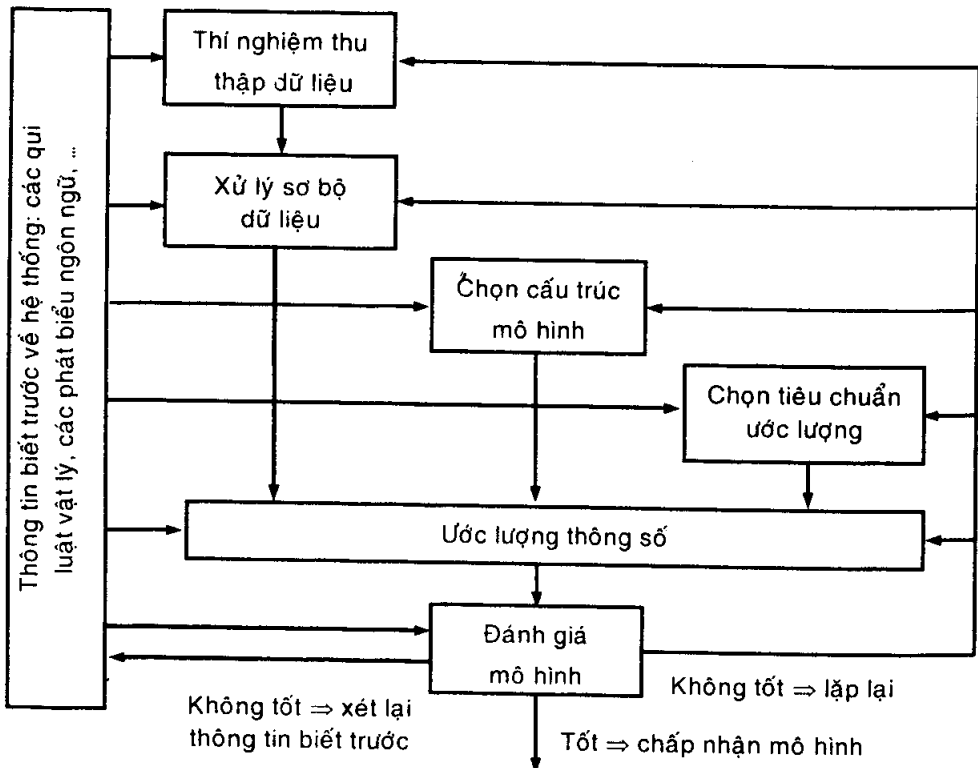
5.6 TÓM TẮT

Chương 5 trình bày phương pháp thiết kế các bộ điều khiển trực tiếp. Bộ điều khiển mờ có thể thiết kế bằng phương pháp thử sai dựa vào kinh nghiệm chuyên gia hoặc sử dụng phương pháp Lyapunov. Cách thiết kế các bộ điều khiển PID mờ dùng quy tắc Mamdani và Sugeno cũng được mô tả chi tiết. Mạng thần kinh có thể sử dụng như một bộ điều khiển trực tiếp để học kinh nghiệm chuyên gia hoặc sao chép các bộ điều khiển sẵn có. Giải thuật di truyền có thể được sử dụng để chỉnh định các bộ điều khiển khác nhau, từ các bộ điều khiển kinh điển như PID, LQR đến các bộ điều khiển mờ/mạng thần kinh. Phương pháp thiết kế các bộ điều khiển được minh họa bằng cách ví dụ cụ thể. Tuy là phương pháp điều khiển khá đơn giản nhưng điều khiển trực tiếp đã được sử dụng rất phổ biến để điều khiển nhiều lớp đối tượng khác nhau. Các phương pháp điều khiển thông minh phức tạp hơn sẽ được trình bày ở các chương sau.

NHẬN DẠNG HỆ PHI TUYẾN

6.1 GIỚI THIỆU

Nhận dạng hệ thống là xây dựng mô hình toán của hệ thống dựa trên dữ liệu quan sát được. Quá trình giải bài toán nhận dạng hệ thống là quá trình lặp theo sơ đồ trình bày ở hình 6.1. Bốn vấn đề cần giải quyết trong bài toán nhận dạng là thí nghiệm thu thập dữ liệu, chọn cấu trúc mô hình, chọn tiêu chuẩn ước lượng thông số và đánh giá chất lượng mô hình.



Hình 6.1 Vòng lặp giải bài toán nhận dạng hệ thống

Thí nghiệm thu thập dữ liệu

Để có thể nhận dạng được mô hình toán của hệ thống, trước tiên ta phải chọn tín hiệu thử có biên độ và băng thông thích hợp đặt vào các ngõ vào và quan sát đáp ứng ở các ngõ ra của hệ thống, bước này gọi là thí nghiệm thu thập dữ liệu. Tín hiệu thử ảnh hưởng rất lớn đến chất lượng của mô hình, nếu tín hiệu thử được chọn không tốt thì không thể lấy được đặc tính của hệ thống, điều này dẫn đến sai số mô hình nhận dạng ở các bước tiếp theo sẽ lớn. Dữ liệu vào ra của hệ thống được thu thập, xử lý sơ bộ (loại mức DC, lọc nhiễu...) và lưu trữ để sử dụng trong các bước ước lượng thông số và đánh giá mô hình.

Chọn cấu trúc mô hình

Vấn đề quan trọng nhất và khó nhất trong các bước giải bài toán nhận dạng hệ thống là đưa ra được cấu trúc mô hình phù hợp với hệ thống cần nhận dạng. Có thể chia bài toán chọn cấu trúc mô hình thành ba bài toán nhỏ:

- Bài toán thứ nhất là chọn loại mô hình. Dựa vào quan hệ giữa tín hiệu vào và tín hiệu ra cũng như những thông tin biết trước về hệ thống mà ta có thể chọn lựa giữa mô hình tuyến tính hay phi tuyến; mô hình hộp đen, hộp xám hay mô hình thông số vật lý...
- Bài toán thứ hai là chọn bậc của mô hình.
- Bài toán thứ ba là làm thế nào để thông số hóa mô hình để các thuật toán ước lượng cho kết quả tốt nhất.

Cấu trúc mô hình là hàm toán học được thông số hóa, hàm này có thể liên quan hoặc không liên quan gì đến các định luật vật lý. Thường người ta sử dụng thông tin biết trước về hệ thống hoặc đưa ra một số giả thiết về hệ thống cần nhận dạng để chọn cấu trúc mô hình. Khi chỉ biết trước (rất) ít thông tin người ta sử dụng mô hình hộp đen. Mô hình hộp đen là cấu trúc mô hình chuẩn linh động có thể dùng để xấp xỉ nhiều hệ thống khác nhau. Không thể xây dựng được cấu trúc mô hình có khả năng mô tả hệ thống thực tế chính xác hoàn toàn, thay vào đó người ta đưa ra một số giả thiết về hệ thống để cấu trúc mô hình gần với hệ thống thật. Giả thiết thường được đưa ra nhiều nhất là hệ thống cần nhận dạng là hệ tuyến tính. Thực tế tất cả các hệ thống đều phi tuyến, vì vậy giả thiết trên chỉ “dùng”

trong trường hợp hệ thống có thể tuyến tính hóa, hay nói các khác hệ thống hoạt động trong miền tín hiệu nhỏ xung quanh điểm tĩnh. Nếu hệ thống cần nhận dạng có độ phi tuyến cao, giả thiết hệ thống là tuyến tính không thỏa mãn, ta phải chuyển sang sử dụng cấu trúc mô hình phi tuyến và mọi việc trở nên phức tạp hơn. Dù cấu trúc mô hình được chọn là loại gì chẳng nữa thì về nguyên tắc từ cấu trúc mô hình đã chọn ta có thể xây dựng được bộ dự báo tín hiệu ra tương lai dựa vào các tín hiệu đã biết ở thời điểm hiện tại. Thông số mô hình sau đó được thay đổi bằng các thuật toán ước lượng thông số sao cho mô hình có khả năng tái tạo lại dữ liệu thực nghiệm.

Ước lượng thông số

Ở bước này chúng ta phải giải một bài toán tối ưu hóa để tìm thông số mô hình sao cho tối thiểu một hàm xác định dương đánh giá sai lệch giữa tín hiệu ra của bộ dự báo và tín hiệu ra thực nghiệm. Tùy theo cấu trúc mô hình và chỉ tiêu đánh giá mà bài toán ước lượng thông số có thể giải được dễ dàng hoặc có thể giải rất khó khăn. Nếu cấu trúc mô hình tuyến tính theo thông số và tiêu chuẩn ước lượng là chuẩn bình phương thì thông số mô hình có thể tính được dễ dàng bằng cách giải bài toán bình phương tối thiểu tuyến tính. Trong trường hợp mô hình phi tuyến theo thông số thì bài toán tối ưu hóa cần giải là bài toán quy hoạch không lồi, trong đa số trường hợp chỉ có khả năng tìm được lời giải tối ưu cục bộ bằng các thuật toán lặp.

Đánh giá mô hình

Sau khi ước lượng thông số ta có được mô hình toán của hệ thống, câu hỏi đặt ra là chất lượng của mô hình vừa nhận dạng như thế nào? Bước đánh giá mô hình trả lời câu hỏi này. Phương pháp cơ bản để đánh giá mô hình là khảo sát khả năng tái tạo đáp ứng của hệ thống với một tập dữ liệu mới không dùng ở bước ước lượng thông số bằng cách mô phỏng mô hình vừa thu được với tín hiệu vào mới và so sánh tín hiệu ra mô phỏng với tín hiệu ra thực nghiệm, phương pháp này gọi là đánh giá chéo. Một phương pháp khác cũng hay dùng để đánh giá mô hình phương pháp phân tích thặng dư, theo đó nếu mô hình nhận dạng được có chất lượng tốt thì tương quan giữa thặng dư (sai lệch giữa tín hiệu ra thực nghiệm và tín hiệu ra của mô hình) và tín hiệu vào phải xấp xỉ bằng 0.

6.2 NHẬN DẠNG HỆ PHI TUYẾN

6.2.1 Đặt bài toán

Xét hệ phi tuyến SISO ngõ vào $u(t)$, ngõ ra $y(t)$. Giả sử quan hệ vào ra của hệ mô tả bởi phương trình sai phân phi tuyến sau:

$$y(t) = f_0[u(t-1), \dots, u(t-m), y(t-1), \dots, y(t-n)] + v(t) \quad (6.1)$$

trong đó $f_0(\cdot)$ là hàm phi tuyến chưa biết và $v(t)$ là nhiễu tác động vào hệ thống. Giả sử chúng ta đã thu thập được N mẫu dữ liệu vào ra của hệ thống (6.1):

$$Z^N = \{y(1), u(1); y(2), u(2); \dots; y(N), u(N)\} \quad (6.2)$$

Bài toán đặt ra là nhận dạng hệ thống (6.1) dựa vào tập dữ liệu quan sát (6.2). Để giải bài toán trên, hệ thống (6.1) có thể được mô hình hóa bằng mô hình hộp đen phi tuyến:

$$\hat{y}(t, \theta) = f(\varphi(t), \theta) \quad (6.3)$$

trong đó:

- $\varphi(t) = \varphi(Z^{t-1}) \in \mathbb{R}^{n_\varphi}$ là vector hàm được định nghĩa dựa vào dữ liệu quan sát trong quá khứ, do đó $\varphi(t)$ được gọi là vector hồi quy và các thành phần của $\varphi(t)$ được gọi là các phần tử hồi quy. Tín hiệu ra $y(t, \theta)$ được tính dựa vào dữ liệu quan sát trong quá khứ nên được gọi là tín hiệu ra dự báo.

- $f(\cdot)$ là ánh xạ phi tuyến từ không gian hồi quy đến không gian tín hiệu ra:

$$f : \mathbb{R}^{n_\varphi} \mapsto \mathbb{R} \quad (6.4)$$

- $\theta \in D_\mathcal{M} \subset \mathbb{R}^d$ là vector thông số cần ước lượng sao cho sai lệch giữa tín hiệu ra $y(t)$ của hệ thống thực và tín hiệu ra dự báo $y(t, \theta)$ là tối thiểu, $D_\mathcal{M}$ là tập hợp tất cả các thông số chấp nhận được.

Như đã giới thiệu ở mục 6.1, bước quan trọng nhất trong quá trình giải bài toán nhận dạng hệ thống là chọn cấu trúc mô hình. Đối với hệ phi tuyến, vấn đề chọn cấu trúc mô hình được chia làm hai bài toán nhỏ là chọn vector hồi quy $\varphi(t)$ và chọn ánh xạ phi tuyến $f(\cdot)$.

6.2.2 Các phần tử hồi quy

Các phần tử hồi quy được chọn tùy thuộc vào từng đối tượng phi tuyến cần nhận dạng cụ thể. Tương tự như các phần tử hồi quy của mô hình tuyến tính, thông thường các phần tử hồi quy của mô hình phi tuyến cũng có thể được chọn từ tín hiệu vào, tín hiệu ra trong quá khứ của hệ thống và sai số dự báo trong quá khứ. Cụ thể, các phần tử hồi quy có thể chọn trong các tín hiệu sau:

- $u(t-k)$: tín hiệu vào trong quá khứ.
- $y(t-k)$: tín hiệu ra trong quá khứ.
- $\varepsilon(t-k, \theta) = y(t-k) - \hat{y}(t-k, \theta)$: sai số dự báo trong quá khứ.
- $\hat{y}(t-k, \theta)$: tín hiệu ra mô phỏng từ tín hiệu vào trong quá khứ.
- $\varepsilon_u(t-k, \theta) = y(t-k) - y_u(t-k, \theta)$: sai số mô phỏng trong quá khứ.

Tùy theo các phần tử hồi quy được chọn mà ta phân biệt các loại mô hình phi tuyến như bảng 6.1, trong đó mô hình NARX là mô hình được sử dụng phổ biến nhất trong đa số các ứng dụng thực tế.

Bảng 6.1 Các loại mô hình hộp đen phi tuyến

Mô hình	Các phần tử hồi quy
NFIR	$u(t-k)$
NAR	$y(t-k)$
NARX	$u(t-k)$ và $y(t-k)$
NARMAX	$u(t-k)$, $y(t-k)$ và $\varepsilon(t-k, \theta)$
NOE	$u(t-k)$ và $\hat{y}_u(t-k, \theta)$
NBJ	$u(t-k)$, $y(t-k, \theta)$, $\varepsilon(t-k, \theta)$ và $\varepsilon_u(t-k, \theta)$

6.2.3 Ánh xạ phi tuyến

Ánh xạ phi tuyến định nghĩa bộ dự báo (6.3) thường có dạng là tổng có trọng số của các hàm cơ sở:

$$\hat{y}(t, \theta) = f(\varphi(t), \theta) = \sum_{i=1}^{n_b} \alpha_i \xi_i(\varphi(t), \beta_i, \gamma_i) \quad (6.5)$$

Trong công thức (6.5) $\xi_i(\cdot)$ ($i = \overline{1, n_b}$) là các hàm cơ sở, mỗi hàm tương ứng với các thông số β_i xác định hướng và γ_i xác định vị trí

của hàm cơ sở trong không gian hồi quy, thông số α_i là trọng số của mỗi hàm cơ sở trong bộ dự báo; vector thông số cần ước lượng là vector cột gồm tất cả các hệ số α_i , β_i và γ_i .

$$\theta = [\alpha_1, \dots, \alpha_{n_b}, \beta_1, \dots, \beta_{n_b}, \gamma_1, \dots, \gamma_{n_b}]^T \quad (6.6)$$

Câu hỏi đặt ra là hàm cơ sở $\xi_i(.)$ được chọn như thế nào. Có rất nhiều cách chọn hàm cơ sở, tuy vậy các cách chọn hàm cơ sở có các điểm chung sau:

- Tất cả các hàm cơ sở $\xi_i(.)$ được xây dựng từ một hàm cơ sở gốc, ký hiệu hàm cơ sở gốc là $\kappa(x)$.
- Hàm cơ sở gốc $\kappa(x)$ là hàm của biến vô hướng x .
- $\xi_i(.)$ là phiên bản tỷ lệ và tịnh tiến của $\kappa(.)$. Trong trường hợp vector hồi quy chỉ có một phần tử ($n_\varphi = 1$) thì ta có thể viết:

$$\xi_i(\varphi, \beta_i, \gamma_i) = \kappa(\beta_i(\varphi - \gamma_i)) \quad (6.7)$$

Do $\xi_i(.)$ được định nghĩa ở biểu thức (6.7) nên β_i được gọi là thông số tỷ lệ và γ_i được gọi là thông số tịnh tiến của $\xi_i(.)$.

Hàm cơ sở gốc $\kappa(x)$

Hàm cơ sở gốc $\kappa(x)$ có thể chọn là hàm vô hướng bất kỳ, sau đây là một số dạng hàm cơ sở gốc thông dụng:

- Hàm tam giác:

$$\kappa(x) = \begin{cases} 1 - |x| & \text{nếu } |x| \leq 1 \\ 0 & \text{nếu } |x| > 1 \end{cases} \quad (6.8)$$

- Hàm bão hòa:

$$\kappa(x) = \begin{cases} x & \text{nếu } 0 \leq x \leq 1 \\ 0 & \text{nếu } x < 0 \\ 1 & \text{nếu } x > 1 \end{cases} \quad (6.9)$$

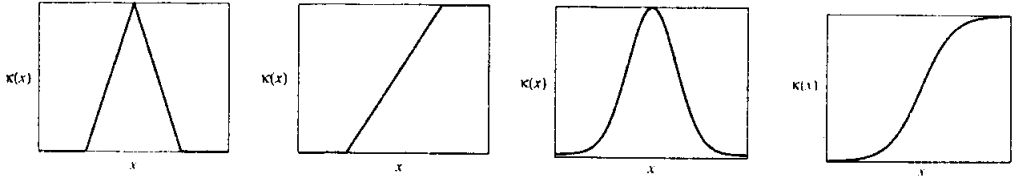
- Hàm Gauss:

$$\kappa(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \quad (6.10)$$

- Hàm Sigmoid:

$$\kappa(x) = \frac{1}{1 + e^{-x}} \quad (6.11)$$

Trong các hàm cơ sở gốc trình bày ở trên, có thể xem hàm Gauss là phiên bản trơn hóa của hàm tam giác, hàm Sigmoid là phiên bản trơn hóa của hàm bão hòa. Các hàm khác như hàm hình thang, hàm chuông, hàm pi... cũng có thể sử dụng là hàm cơ sở gốc.



Hình 6.2 Các hàm cơ sở gốc thông dụng

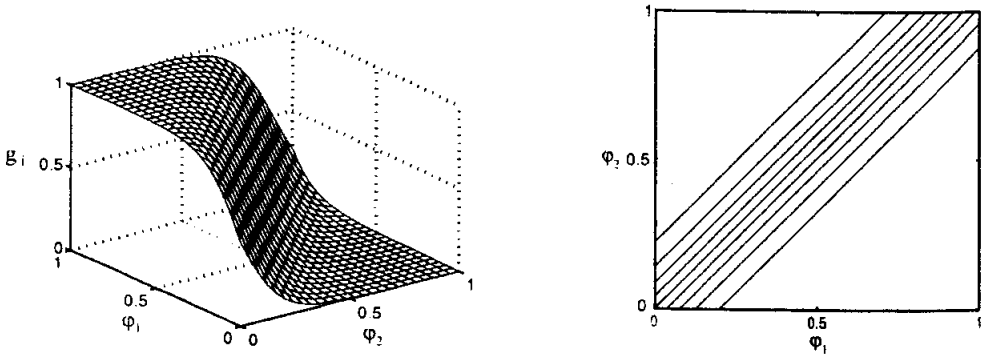
Cấu trúc hàm cơ sở nhiều biến

Trong trường hợp vector hồi quy có nhiều hơn 1 phần tử ($n_\varphi > 1$) thì $\xi_i(\cdot)$ là hàm nhiều biến. Có ba cách thường sử dụng để xây dựng hàm cơ sở $\xi_i(\cdot)$ từ hàm cơ sở gốc $\kappa(x)$:

- Cấu trúc dây:

$$\xi_i(\varphi(t), \beta_i, \gamma_i) = \kappa(\beta_i^T \varphi(t) - \gamma_i) \quad (6.12)$$

trong đó $\beta_i \in \mathcal{R}^{n_\varphi}$, $\gamma_i \in \mathcal{R}$. Hàm cơ sở cấu trúc dây có đặc điểm là giá trị của hàm bằng hằng số đối với tất cả các phần tử hồi quy nằm trên cùng một siêu phẳng $\beta_i^T \varphi(t) = \text{const}$. Số thông số tối đa cần ước lượng trong trường hợp hàm cơ sở cấu trúc dây là $d = \dim \theta = n_b(n_\varphi + 2)$.

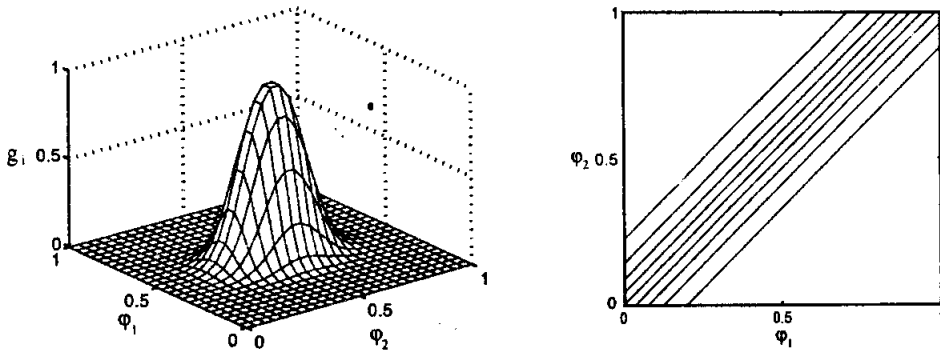


Hình 6.3 Hàm cơ sở nhiều biến cấu trúc dây

- Cấu trúc xuyên tâm:

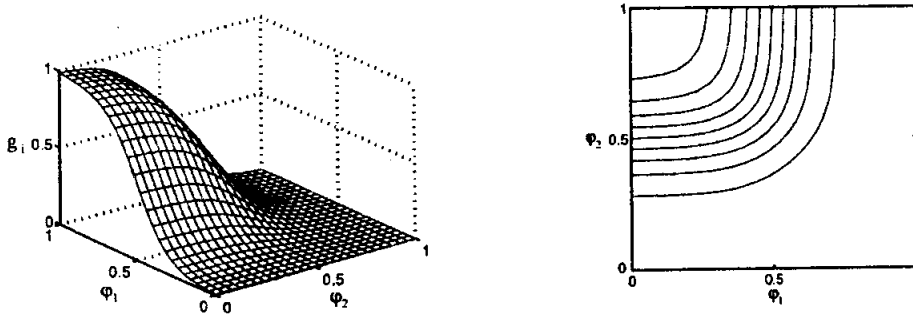
$$\xi_i(\varphi(t), \beta_i, \gamma_i) = \kappa\|\varphi(t) - \gamma_i\|_{\beta_i}^2 \quad (6.13)$$

trong đó $\|\cdot\|_{\beta_i}^2$ là ký hiệu chuẩn bình phương có trọng số ($\|x\|_{\beta_i}^2 = x^T \beta_i x$), $\beta_i \in \mathbb{R}^{n_\varphi \times n_\varphi}$ là ma trận đối xứng bán xác định dương, $\gamma_i \in \mathbb{R}^{n_\varphi}$. Hàm cơ sở cấu trúc xuyên tâm có đặc điểm là giá trị hàm bằng hằng số đối với tất cả các phần tử hồi quy nằm trên cùng một siêu cầu. Số thông số tối đa cần ước lượng trong trường hợp hàm cơ sở cấu trúc xuyên tâm là $d = \dim \theta = n_b(n_\varphi^2 + n_\varphi + 1)$.



Hình 6.4 Hàm cơ sở nhiều biến cấu trúc xuyên tâm

- Cấu trúc tích: Hàm cơ sở nhiều biến cấu trúc tích được thành lập bằng cách lấy tích của tất cả các hàm cơ sở một biến đối với từng phần tử hồi quy.



Hình 6.5 Hàm cơ sở nhiều biến cấu trúc tích

$$\xi_i(\varphi(t), \theta) = \prod_{j=1}^{n_\varphi} \xi_{ij}(\varphi_j(t), \beta_{ij}, \gamma_{ij}) \quad (6.14)$$

trong đó $\beta_{ij} \in \mathbb{R}$, $\gamma_{ij} \in \mathbb{R}$ và:

$$\xi_{ij}(\varphi_j(t), \beta_{ij}, \gamma_{ij}) = \kappa(\beta_{ij}(\varphi_j(t) - \gamma_{ij})) \quad (6.15)$$

Số thông số tối đa cần ước lượng trong trường hợp này là:

$$d = \dim \theta = n_b(2n_\varphi + 1)$$

Cách xây dựng hàm cơ sở như trình bày ở trên bao hàm hầu hết tất cả các cấu trúc mô hình hộp đen phi tuyến được sử dụng phổ biến hiện nay, chẳng hạn mô hình mạng thần kinh nhiều lớp có cấu trúc dây; mô hình mạng hàm cơ sở xuyên tâm, mô hình mạng wavelet có cấu trúc xuyên tâm; mô hình mờ có cấu trúc tích. Một câu hỏi đặt ra là mô hình hộp đen dưới dạng khai triển chuỗi hàm cơ sở có khả năng xấp xỉ quan hệ vào ra của hệ thống thật tốt như thế nào. Có nhiều tài liệu đề cập đến vấn đề này, kết luận chung là đối với hầu hết các cách chọn hàm cơ sở gốc $\kappa(x)$, mô hình khai triển chuỗi hàm cơ sở (6.5) có thể xấp xỉ hàm trơn bất kỳ với sai số nhỏ tùy ý với điều kiện số hàm cơ sở sử dụng đủ lớn.

6.2.4 Chọn cấu trúc mô hình

Một trong những nguyên tắc cơ bản của lý thuyết nhận dạng hệ thống là xây dựng mô hình toán càng đơn giản càng tốt. Khi bậc mô hình càng tăng, số thông số càng nhiều thì mô hình càng linh hoạt và sai số xấp xỉ càng giảm, tuy nhiên bậc tăng đến một mức nào đó thì dù tăng thêm sai số vẫn giảm không đáng kể, vì vậy tiêu chuẩn chọn lựa cấu trúc mô hình phải tính đến sai số mô hình và số thông số của mô hình.

Tiêu chuẩn chọn bậc mô hình tuyến tính theo thông số

Đối với các mô hình tuyến tính theo thông số, có nhiều tiêu chuẩn để chọn bậc của mô hình. Nguyên tắc chung để đưa ra các tiêu chuẩn chọn bậc mô hình là sự cân bằng giữa độ chính xác và độ phức tạp của mô hình. Tổng quát, các tiêu chuẩn này gồm hai thành phần: thành phần thứ nhất là trung bình bình phương sai số phản ánh độ chính xác của mô hình; thành phần thứ hai là hệ số phạt có đặc điểm tăng lên theo bậc hệ thống. Sau đây là một số tiêu chuẩn chọn bậc mô hình thường sử dụng:

- Tiêu chuẩn sai số dự báo cuối cùng (*Final Prediction Error-FPE*):

$$J_{FPE} = \frac{N+d}{N-d} \left[\frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t, \hat{\theta}))^2 \right] \quad (6.16)$$

trong đó N là số mẫu dữ liệu thực nghiệm, d là số thông số của mô hình và θ là giá trị thông số ước lượng được.

- Tiêu chuẩn thông tin Akaike (*Akaike Information Critetion-AIC*):

$$J_{AIC} = \log \left[\frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t, \hat{\theta}))^2 \right] + \frac{2d}{N} \quad (6.17)$$

- Tiêu chuẩn độ dài mô tả cực đại (*Maximum Description Length-MDL*):

$$J_{MDL} = \log \left[\frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t, \hat{\theta}))^2 \right] + \frac{d \log(N)}{N} \quad (6.18)$$

Tiêu chuẩn chọn bậc mô hình phi tuyến

Các tiêu chuẩn mô tả ở trên chỉ có thể áp dụng để chọn mô hình tuyến tính theo thông số. Đối với mô hình phi tuyến, vấn đề chọn bậc mô hình vẫn còn là một bài toán mở. Thông thường số thông số của mô hình phi tuyến được chọn dựa vào kinh nghiệm hoặc bằng phương pháp thử sai. Một số tác giả mở rộng tiêu chuẩn AIC cho trường hợp mô hình phi tuyến như sau:

- Tiêu chuẩn thông tin phi tuyến (*Nonlinear Information Criterion-NIC*):

$$J_{NIC} = \log \left[\frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t, \hat{\theta}))^2 \right] + \frac{2d_{eff}}{N} \quad (6.19)$$

Tiêu chuẩn NIC tương tự như tiêu chuẩn AIC cho mô hình tuyến tính theo thông số, chỉ khác là số thông số d trong tiêu chuẩn AIC được thay bằng số thông số tác động của mô hình phi tuyến được định nghĩa như sau:

$$d_{eff} = \text{tr}[(g^T g) H^{-1}] \quad (6.20)$$

trong đó:
$$g = \text{Var} \left[\frac{\partial V_N(\theta)}{\partial \theta} \right] \quad (6.21)$$

$$H = E \left[\frac{\partial^2 V_N(\theta)}{\partial \theta^2} \right] \quad (6.22)$$

$$V_N(\theta) = \frac{1}{N} \sum_{t=1}^N (y(t) - \hat{y}(t, \theta))^2 \quad (6.23)$$

Trong các công thức trên ký hiệu $\text{Var}(\cdot)$ là phương sai và $E(\cdot)$ là kỳ vọng. Nếu số thông số tác động ít hơn nhiều so với số thông số

thật thì hai tiêu chuẩn NIC và AIC sẽ chọn các mô hình khác nhau. Trong thực tế, do cần nhiều phép tính để tính số thông số tác động nên người ta thường dùng các tiêu chuẩn chọn mô hình tuyến tính (theo thông số) để chọn mô hình phi tuyến.

Các phương pháp chọn bậc mô hình trình bày ở trên có thể được sử dụng trong các phần mềm nhận dạng hệ thống để tự động chọn bậc mô hình. Việc chọn lựa tiêu chuẩn nào để sử dụng tùy thuộc vào từng ứng dụng cụ thể.

6.2.5 Ước lượng thông số

Phương pháp ước lượng thông số được sử dụng phổ biến nhất hiện nay là phương pháp sai số dự báo. Ý tưởng cơ bản của phương pháp này là ước lượng thông số sao cho cực tiểu hóa tiêu chuẩn ước lượng là hàm xác định dương của sai số dự báo (thường là hàm toàn phương), trong đó sai số dự báo là sai lệch giữa tín hiệu ra của hệ thống thật và tín hiệu ra dự báo.

Sai số dự báo:

$$\varepsilon(t, \theta) = y(t) - \hat{y}(t, \theta) \quad (6.24)$$

Tiêu chuẩn ước lượng là chuẩn bình phương tối thiểu:

$$V_N(\theta, Z^N) = \frac{1}{N} \sum_{t=1}^N \varepsilon^2(t) = \frac{1}{N} \sum_{t=1}^N \left[y(t) - \hat{y}(t, \theta) \right]^2 \quad (6.25)$$

Thông số ước lượng là:

$$\hat{\theta}_N = \arg \min_{\theta} V_N(\theta, Z^N) \quad (6.26)$$

Lời giải $\hat{\theta}_N$ của bài toán tối ưu hóa (6.26) có thể tìm được bằng thuật toán lặp Newton:

$$\hat{\theta}_N^{k+1} = \hat{\theta}_N^k - \left[V_N''(\hat{\theta}_N^k, Z^N) \right]^{-1} V_N'(\hat{\theta}_N^k, Z^N) \quad (6.27)$$

trong đó $\hat{\theta}_N^k \in \mathfrak{R}^{d \times 1}$ là thông số ước lượng ở bước lặp thứ k ; $V_N'(\cdot) \in \mathfrak{R}^{d \times 1}$ và $V_N''(\cdot) \in \mathfrak{R}^{d \times d}$ lần lượt là đạo hàm bậc 1 và bậc 2 của $V_N(\cdot)$ tại $\hat{\theta}_N^k$. Từ biểu thức (6.25) ta dễ dàng rút ra được:

- Đạo hàm bậc 1 của $V_N(\cdot)$:

$$V_N'(\hat{\theta}_N^k, Z^N) = -\frac{2}{N} \sum_{t=1}^N \psi(t, \hat{\theta}_N^k) \varepsilon(t, \hat{\theta}_N^k) \quad (6.28)$$

trong đó: $\psi(t, \hat{\theta}_N^k) = \left[\frac{\partial \hat{y}(t, \theta)}{\partial \theta} \right]_{\theta = \hat{\theta}_N^k}$ (6.29)

- Đạo hàm bậc 2 của $V_N(\cdot)$:

$$V_N''(\hat{\theta}_N^k, Z^N) = \frac{2}{N} \sum_{t=1}^N \left[\psi(t, \hat{\theta}_N^k) \psi^T(t, \hat{\theta}_N^k) - \frac{\partial \psi(t, \hat{\theta}_N^k)}{\partial \theta} \varepsilon(t, \hat{\theta}_N^k) \right] \quad (6.30)$$

Thuật toán Newton có một số dạng sau đây:

- Thuật toán suy giảm độ dốc:

$$\hat{\theta}_N^{k+1} = \hat{\theta}_N^k + \mu^k \sum_{t=1}^N \psi(t, \hat{\theta}_N^k) \varepsilon(t, \hat{\theta}_N^k) \quad (6.31)$$

- Thuật toán Gauss-Newton:

$$\hat{\theta}_N^{k+1} = \hat{\theta}_N^k + \mu^k \left[\sum_{n=1}^N \psi(t, \hat{\theta}_N^k) \psi^T(t, \hat{\theta}_N^k) \right]^{-1} \sum_{n=1}^N \psi(t, \hat{\theta}_N^k) \varepsilon(t, \hat{\theta}_N^k) \quad (6.32)$$

- Thuật toán Levenberg-Marquardt:

$$\hat{\theta}_N^{k+1} = \hat{\theta}_N^k + \left[\left(\sum_{t=1}^N \psi(t, \hat{\theta}_N^k) \psi^T(t, \hat{\theta}_N^k) \right) + \mu^k I \right]^{-1} \sum_{t=1}^N \psi(t, \hat{\theta}_N^k) \varepsilon(t, \hat{\theta}_N^k) \quad (6.33)$$

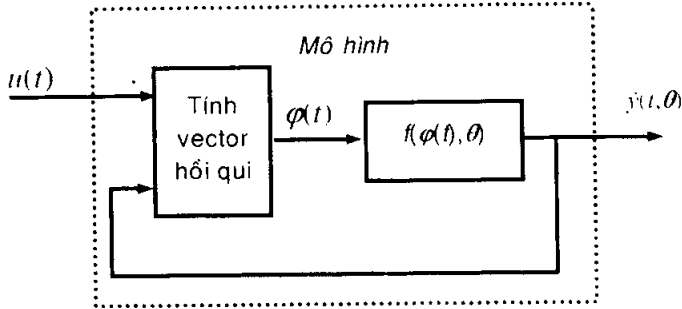
Thuật toán lặp Newton là thuật toán tìm lời giải tối ưu cục bộ, do đó thông số ước lượng được phụ thuộc vào giá trị khởi động ban đầu. Ngoài thuật toán Newton, có thể áp dụng các thuật toán khác, chẳng hạn như GA để giải bài toán tối ưu hóa (6.26).

6.2.6 Đánh giá mô hình

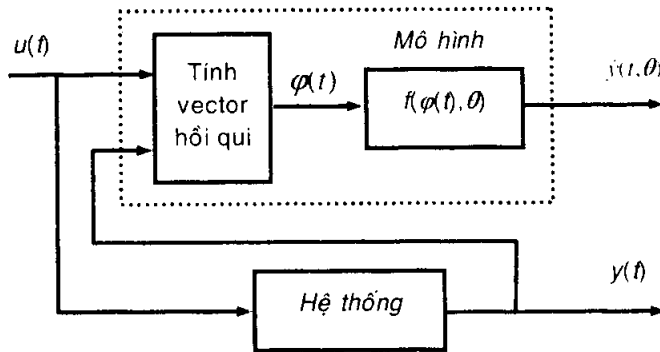
Đánh giá chéo

Một phương pháp rất phổ biến để chọn bậc mô hình là đánh giá chéo (Cross Validation). Đánh giá chéo là mô phỏng mô hình đã nhận dạng được với tập dữ liệu đầu vào không dùng ở bước ước lượng thông số. Thông thường tập dữ liệu thực nghiệm được chia làm hai phần, một phần dùng để ước lượng thông số và một phần để đánh giá chéo. Kỹ thuật đánh giá chéo có khuyết điểm là mất nhiều thời gian nhưng hiện nay vẫn là một trong những phương pháp được sử dụng phổ biến nhất để chọn bậc mô hình phi tuyến.

Cần phân biệt mô phỏng (*simulation*) và dự báo (*prediction*). Giả sử $\hat{y}(t, \theta) = f(\varphi(t), \theta)$ là mô hình của hệ thống, hình 6.6 và hình 6.7 cho thấy sự khác biệt giữa dùng mô hình để mô phỏng đáp ứng của hệ thống và dùng mô hình để dự báo đáp ứng của hệ thống:



Hình 6.6 Mô phỏng đáp ứng của hệ thống



Hình 6.7 Dự báo đáp ứng của hệ thống

Chỉ tiêu định lượng độ phù hợp của mô hình

Để đánh giá chất lượng của mô hình có thể dùng chỉ số chất lượng độ phù hợp của mô hình, chỉ tiêu này được định nghĩa như sau:

$$VAF = \left(1 - \frac{\sum_{t=1}^N [y(t) - \hat{y}(t, \hat{\theta}_N)]^2}{\sum_{t=1}^N [y(t) - \bar{y}]^2} \right) \times 100\% \quad (6.34)$$

$$\bar{y} = \frac{1}{N} \sum_{t=1}^N y(t) \quad (6.35)$$

Mô hình càng chính xác, chỉ số *VAF* (*Variance Accounted For*) càng cao.

6.3 NHẬN DẠNG HỆ PHI TUYẾN DÙNG MÔ HÌNH MỜ

6.3.1 Tại sao dùng mô hình mờ?

Một yêu cầu quan trọng khi giải bài toán nhận dạng hệ thống là sử dụng triệt để thông tin biết trước về hệ thống để đưa ra cấu trúc mô hình phù hợp. Đối với một số hệ thống kỹ thuật, có rất nhiều thông tin biết trước về đặc tính động của hệ thống được cung cấp bởi các chuyên gia là người thiết kế hay vận hành, những thông tin này không cho dưới dạng hàm toán học mà cho dưới dạng các phát biểu ngôn ngữ không chính xác. Ví dụ đối với hệ bồn chứa chất lỏng chúng ta có thể mô tả được đặc tính động của bồn chứa bằng các phát biểu ngôn ngữ có dạng như sau:

“Nếu điện áp cấp cho máy bơm tăng thì mực chất lỏng trong bồn tăng”

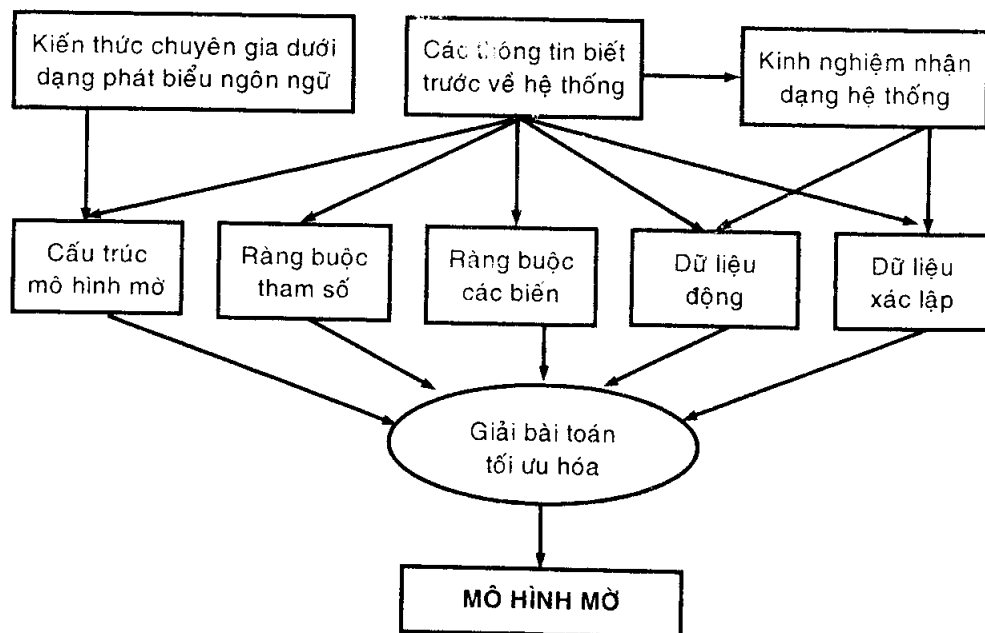
hoặc: *“Nếu điện áp cấp cho máy bơm ở thời điểm t thấp và mực chất lỏng trong bồn thời điểm $t-1$ cao thì mực chất lỏng ở thời điểm t trung bình”.*

Tập hợp những phát biểu tương tự như trên tạo thành mô hình ngôn ngữ. Đặc tính động của một số hệ thống phức tạp như hệ tay máy hay các quá trình công nghiệp có thể mô tả được bằng các phát biểu ngôn ngữ tương tự như trên từ kinh nghiệm của các chuyên gia. Vì phần lớn tri thức và kinh nghiệm của con người được biểu diễn dưới dạng các phát biểu ngôn ngữ nên dẫn đến yêu cầu tự nhiên là cần có phương pháp tích hợp thông tin dưới dạng các phát biểu ngôn ngữ vào quá trình nhận dạng. Cho đến hiện nay, phương pháp duy nhất có thể thực hiện được yêu cầu trên là dùng lý thuyết logic mờ để chuyển mô hình ngôn ngữ về dạng mô hình toán học, cụ thể là mô hình mờ Mamdani. Thông số của mô hình mờ (các hàm liên thuộc biểu diễn các giá trị ngôn ngữ) sau đó sẽ được ước lượng bằng cách sử dụng dữ liệu vào-ra đã quan sát được.

Thông tin biết trước về hệ phi tuyến còn có thể là cấu trúc mô hình tuyến tính cục bộ xung quanh các điểm làm việc tĩnh. Trong trường hợp này có thể sử dụng thông tin về các mô hình tuyến tính cục bộ để đưa ra cấu trúc mô hình mờ Takagi-Sugeno. Mô hình mờ Takagi-Sugeno còn có thể kết hợp được các thông tin biết trước về tính ổn định, độ lợi cực tiểu và cực đại, thời gian xác lập của đáp

ứng nắc để nhận dạng hệ phi tuyến. Hình 6.8 là tiến trình giải bài toán mô hình hóa mờ dựa vào thông tin biết trước về hệ thống.

Nhận dạng hệ phi tuyến dùng mô hình mờ là vấn đề nghiên cứu được rất nhiều người quan tâm trong những năm gần đây. Về mặt toán học hai loại mô hình mờ Mamdani và Takagi-Sugeno đều có khả năng xấp xỉ tổng quát nên có thể sử dụng để mô hình hóa hệ phi tuyến tron với sai số nhỏ tùy ý.



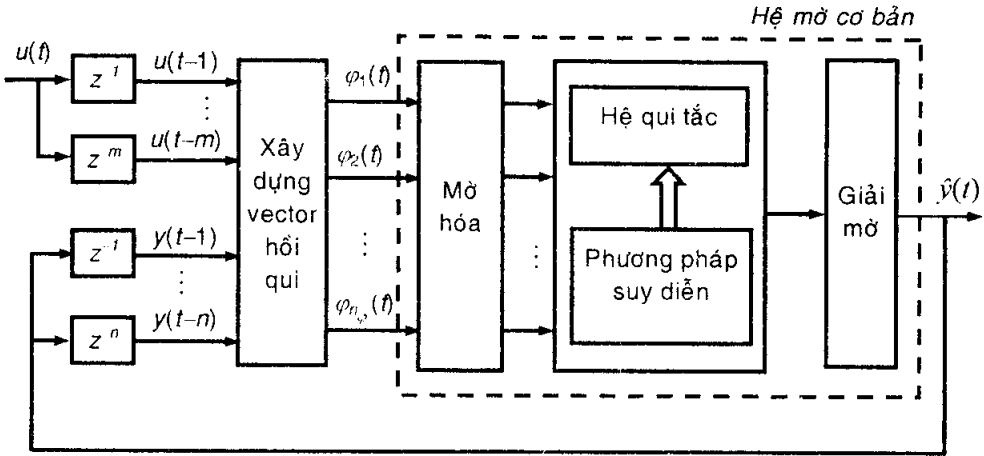
Hình 6.8 Tiến trình nhận dạng mô hình mờ dựa trên thông tin biết trước

6.3.2 Mô hình mờ Mamdani

Giả sử $\varphi(t)$ là vector hồi quy, trong đó các phần tử hồi quy $\varphi_i(t)$ được xây dựng từ tín hiệu ra và tín hiệu vào đến thời điểm $t-1$, tức là:

$$\varphi(t) = [y(t-1), y(t-2), \dots, y(t-n), u(t-1), u(t-2), \dots, u(t-m)]^T \quad (6.36)$$

Ta xây dựng bộ dự báo mờ có sơ đồ khối như trình bày ở hình 6.9. Vì bộ dự báo mờ là một hệ mờ nên cũng có ba thành phần cơ bản là khâu mờ hóa, hệ quy tắc mờ, và khâu giải mờ.



Hình 6.9 Sơ đồ khối bộ dự báo mờ

Hệ quy tắc mờ biểu diễn tri thức và kinh nghiệm của con người dưới dạng các phát biểu ngôn ngữ. Đặc tính động của hệ thống dưới dạng các phát biểu ngôn ngữ được mô tả toán học bằng n_r quy tắc mờ, quy tắc thứ i có dạng:

$$\text{Nếu } (\varphi_1(t) \text{ là } \tilde{A}_{i1}) \text{ và ... và } (\varphi_{n_{\varphi}}(t) \text{ là } \tilde{A}_{in_{\varphi}}) \text{ thì } (y(t) \text{ là } \tilde{B}_i) \quad (6.37)$$

trong đó các giá trị ngôn ngữ trong các mệnh đề điều kiện và kết luận của hệ quy tắc được mô tả bởi các tập mờ. Hàm liên thuộc của các tập mờ có thể có các dạng sau phân bố Gauss, dạng sigmoid, dạng chuông, dạng tam giác, dạng hình thang... Ký hiệu $\mu_{\tilde{A}_{ij}}(\varphi_j(t), \beta_{ij}, \gamma_{ij})$

là hàm liên thuộc của tập mờ $\tilde{A}_{ij}$, trong đó β_{ij} và γ_{ij} là các thông số xác định tỷ lệ và vị trí của hàm liên thuộc.

Nếu hệ quy tắc mờ là hệ quy tắc hoàn chỉnh, nghĩa là gồm tất cả các quy tắc có thể có; các tập mờ ở mệnh đề điều kiện của các quy tắc được phân hoạch mờ và các tập mờ ở mệnh đề kết luận của hệ quy tắc được chọn là vạch đơn tại vị trí α_i ; suy diễn dùng phương pháp SUM-PROD; giải mờ dùng phương pháp trung bình có trọng số thì áp dụng các công thức suy luận mờ đã trình bày ở chương 2 ta có thể rút ra được biểu thức quan hệ vào-ra của mô hình mờ là:

$$\hat{y}(t, \theta) = \sum_{i=1}^{n_r} \alpha_i \cdot \prod_{j=1}^{n_{\varphi}} \mu_{\tilde{A}_{ij}}(\varphi_j(t), \beta_{ij}, \gamma_{ij}) \quad (6.38)$$

$$\text{hay} \quad \hat{y}(t, \theta) = \sum_{i=1}^{n_r} \alpha_i \cdot g_i(\varphi_j(t), \beta_i, \gamma_i) \quad (6.39)$$

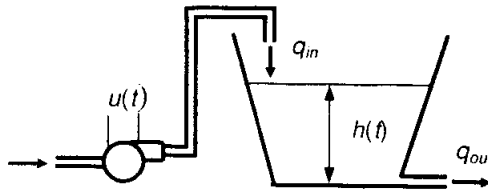
$$\text{với} \quad g_i(\varphi(t), \beta_i, \gamma_i) = \prod_{j=1}^{n_\varphi} \mu_{A_{ij}}(\varphi_j(t), \beta_{ij}, \gamma_{ij}) \quad (6.40)$$

trong các công thức trên θ là vector thông số cần ước lượng (là vector cột gồm tất cả các thông số hàm liên thuộc ở mệnh đề điều kiện và mệnh đề kết luận):

$$\theta = [\alpha_1, \dots, \alpha_{n_r}, \beta_{11}, \dots, \beta_{n_r n_\varphi}, \gamma_{11}, \dots, \gamma_{n_r n_\varphi}]^T \quad (6.41)$$

Tóm lại dùng logic mờ ta có thể chuyển các phát biểu ngôn ngữ mô tả đặc tính động của hệ thống thành bộ dự báo (6.38). Bộ dự báo này được sử dụng để ước lượng thông số.

Ví dụ 6.1 Nhận dạng hệ bồn đơn dùng mô hình mờ Mamdani.



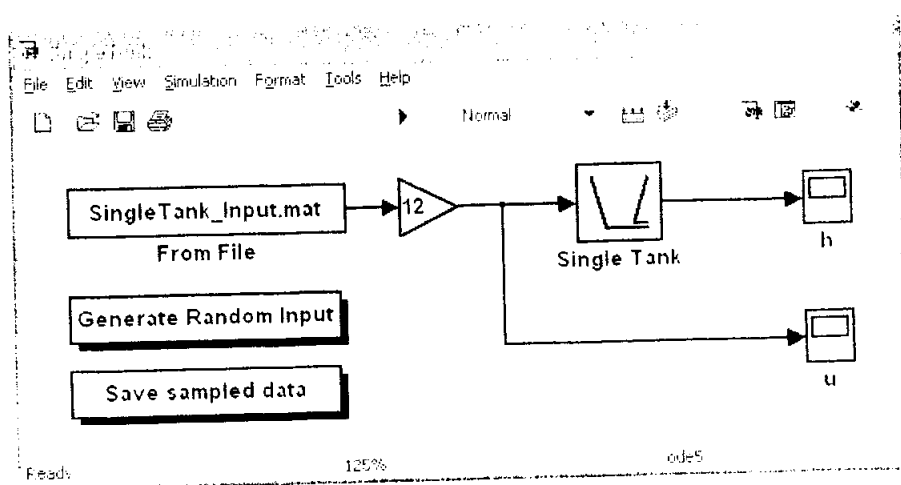
Hình 6.10 Hệ bồn đơn

Xét bài toán nhận dạng hệ bồn chứa chất lỏng có tiết diện ngang thay đổi theo độ cao (H.6.10), phương trình vi phân mô tả quan hệ vào-ra của hệ bồn chứa đã trình bày ở ví dụ 5.5.

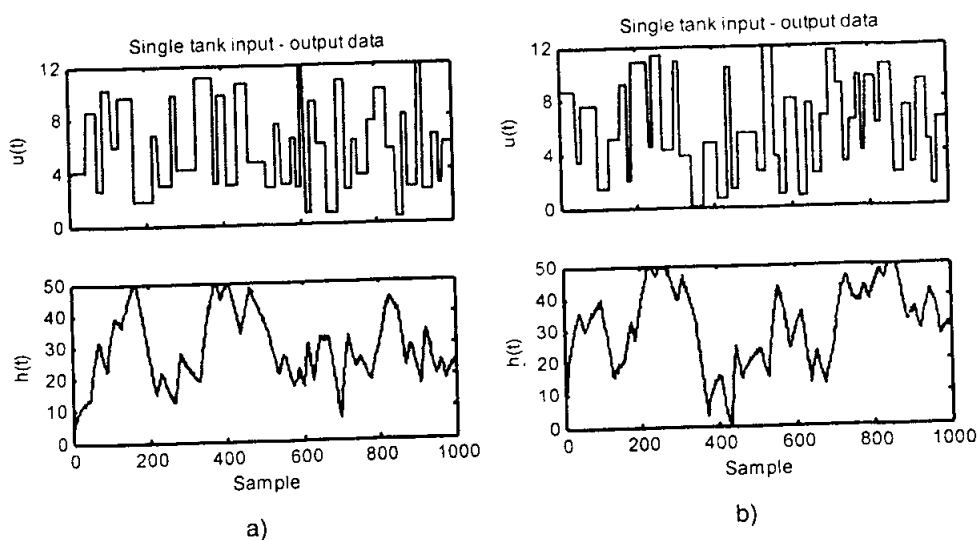
Thí nghiệm thu thập dữ liệu

Hình 6.11 là sơ đồ mô phỏng thí nghiệm thu thập dữ liệu vào ra của hệ bồn chứa, trong đó khối **[SingleTank]** có đặc tính động học mô tả bởi các phương trình (5.22), (5.23) và giả thiết tín hiệu đo độ cao mực chất lỏng trong bồn chứa bị ảnh hưởng bởi nhiễu cộng ngẫu nhiên có trung bình bằng 0 và phương sai bằng 0.5. Chú ý rằng sơ đồ ở hình 6.11 có thể dùng để thu thập dữ liệu thực nghiệm, chỉ cần lập trình lại khối **[SingleTank]** giao tiếp với các card thu thập dữ liệu (như PCL-818 hoặc PCI-1711) và sử dụng bộ công cụ xPC Target hoặc Realtime Windows Target.

Hai tập dữ liệu dùng để ước lượng thông số và đánh giá mô hình thu thập từ sơ đồ mô phỏng trình bày ở hình 6.12.



Hình 6.11 Sơ đồ thu thập dữ liệu vào ra hệ bồn đơn



Hình 6.12 Dữ liệu vào – ra của hệ bồn chứa chất lỏng
a) Dữ liệu ước lượng thông số; (b) Dữ liệu đánh giá mô hình

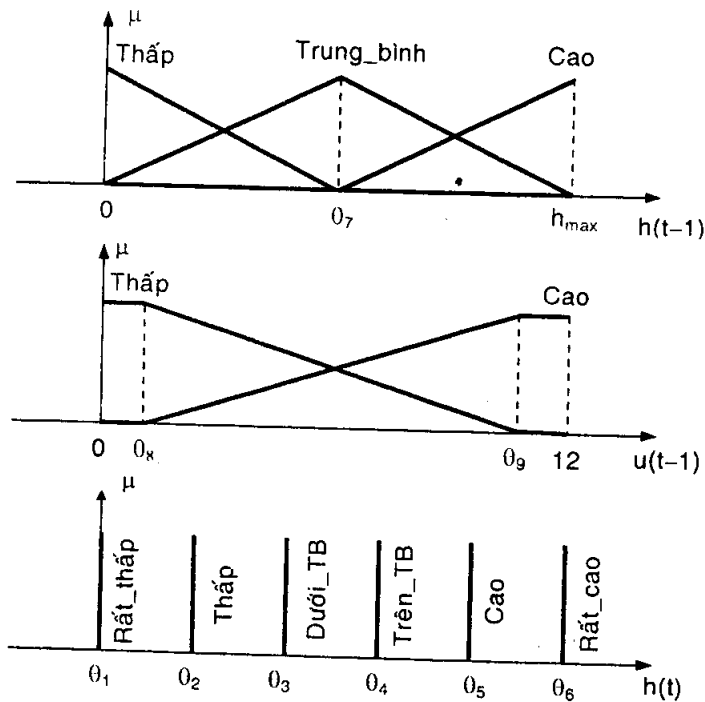
Xây dựng bộ dự báo mờ

Đặc tính động học của hệ bồn chứa có thể mô tả bởi các phát biểu định tính, Ví dụ:

“Nếu $h(t-1)$ thấp và $u(t-1)$ thấp thì $h(t)$ rất thấp” (6.42)

Sử dụng mô hình mờ ta có thể chuyển các phát biểu ngôn ngữ dạng (6.13) thành mô hình toán học để nhận dạng hệ bốn chứa. Chọn hai phần tử hồi quy là $h(t-1)$ và $u(t-1)$, tín hiệu ra của mô hình là $h(t)$. Các tập mờ mô tả các giá trị ngôn ngữ của $h(t-1)$, $u(t-1)$ và $h(t)$ định nghĩa ở hình 6.12, để ý rằng các hàm liên thuộc có thông số thay đổi. Bảng 6.2 trình bày đầy đủ tất cả các quy tắc mờ mô tả đặc tính động học của hệ bốn chứa, các quy tắc này được đưa ra dựa vào kinh nghiệm. Áp dụng công thức (6.38) ta có biểu thức bộ dự báo mờ:

$$\begin{aligned} \hat{h}(t, \theta) = & \theta_1 \cdot \mu_{Thap}(h(t-1), 0, \theta_7) \cdot \mu_{Thap}(u(t-1), \theta_8, \theta_9) + \\ & \theta_2 \cdot \mu_{Thap}(h(t-1), 0, \theta_7) \cdot \mu_{Cao}(u(t-1), \theta_8, \theta_9) + \\ & \theta_3 \cdot \mu_{TB}(h(t-1), 0, \theta_7, h_{max}) \cdot \mu_{Thap}(u(t-1), \theta_8, \theta_9) + \\ & \theta_4 \cdot \mu_{TB}(h(t-1), 0, \theta_7, h_{max}) \cdot \mu_{Cao}(u(t-1), \theta_8, \theta_9) + \\ & \theta_5 \cdot \mu_{Cao}(h(t-1), \theta_7, h_{max}) \cdot \mu_{Thap}(u(t-1), \theta_8, \theta_9) + \\ & \theta_6 \cdot \mu_{Cao}(h(t-1), \theta_7, h_{max}) \cdot \mu_{Cao}(u(t-1), \theta_8, \theta_9) \end{aligned} \quad (6.43)$$



Hình 6.13 Hàm liên thuộc của các tập mờ

Bảng 6.2 Hệ quy tắc mờ mô tả đặc tính động học hệ bốn chứa

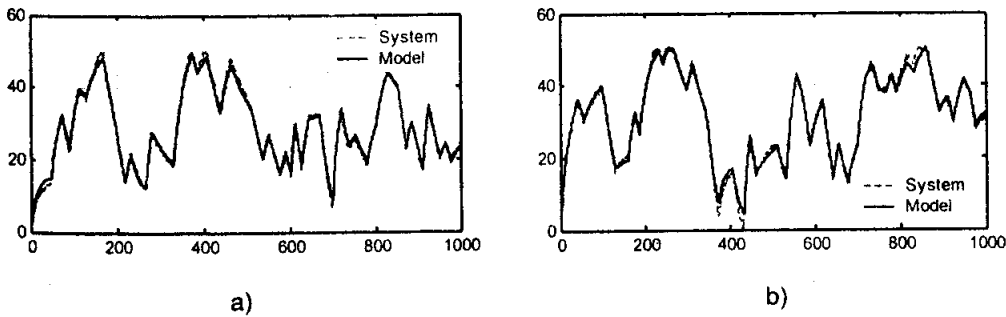
$h(t-1)$	$u(t-1)$	$h(t)$
Thấp	Thấp	Rất_thấp
Thấp	Cao	Thấp
Trung_bình	Thấp	Dưới_TB
Trung_bình	Cao	Trên_TB
Cao	Thấp	Cao
Cao	Cao	Rất_cao

Kết quả ước lượng thông số và đánh giá mô hình

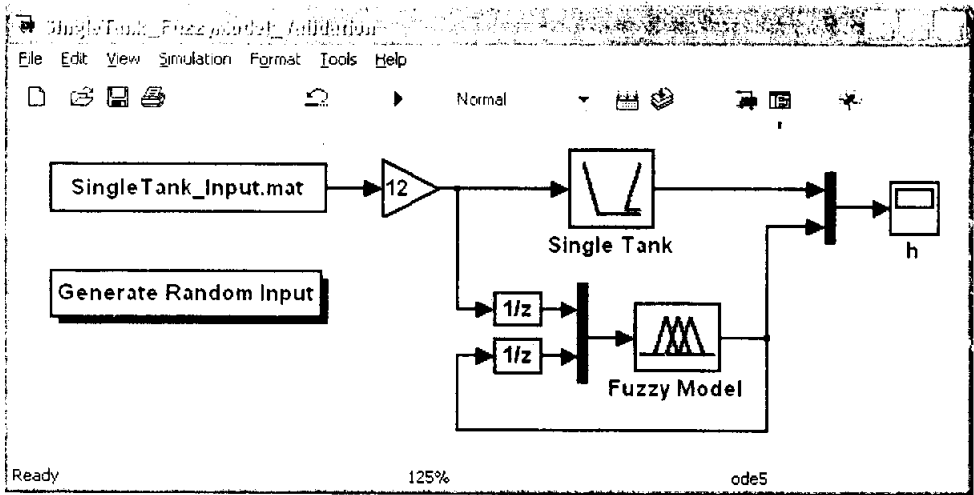
Áp dụng thuật toán Levenberg–Marquadt đã trình bày ở mục 6.2.5 để ước lượng thông số mô hình mờ (6.43) theo tiêu chuẩn bình phương tối thiểu, ta được:

$\hat{\theta} = [0 \ 3,79 \ 16,26 \ 18,96 \ 49,19 \ 50,27 \ 17,30 \ 0,38 \ 11,88]^T$

Hình 6.14 là đồ thị đánh giá chất lượng mô hình vừa nhận dạng, ta thấy sai lệch ngõ ra mô hình và hệ thống là không đáng kể. Độ phù hợp của mô hình tính theo chỉ tiêu chất lượng (6.34) là 99%. Hình 6.15 là sơ đồ mô phỏng dùng để đánh giá mô hình với tín hiệu vào ngẫu nhiên bất kỳ. Kết quả mô phỏng cho thấy ngõ ra của mô hình mờ hoàn toàn phù hợp với ngõ ra của hệ bốn chứa.



Hình 6.14 Đánh giá mô hình
a) Phân tích thành dư; b) Đánh giá chéo



Hình 6.15 Sơ đồ Simulink đánh giá mô hình mờ với tín hiệu vào ngẫu nhiên

6.3.3 Mô hình mờ Sugeno

Nếu hệ thống làm việc trong miền tín hiệu nhỏ xung quanh một điểm tĩnh nào đó thì quan hệ vào ra của hệ thống có thể mô tả bằng mô hình tuyến tính. Có nhiều cấu trúc mô hình tuyến tính như FIR, AR, ARX, ARMAX, OE, BJ, phương trình trạng thái,... Để đơn giản ở đây ta chỉ xét mô hình ARX, đây là cấu trúc mô hình tuyến tính được sử dụng phổ biến nhất trong các ứng dụng thực tế. Tuy nhiên cần nhấn mạnh rằng cơ sở lý thuyết trình bày ở mục này cũng có thể áp dụng cho các cấu trúc mô hình tuyến tính khác.

Biểu thức tổng quát mô hình ARX có dạng:

$$\hat{y}(t, \zeta) = a_1 y(t-1) + \dots + a_n y(t-n) + b_1 u(t-1) + \dots + b_m u(t-m) \quad (6.44)$$

trong đó vector thông số của mô hình tuyến tính là:

$$\zeta = [a_1 \ a_2 \ \dots \ a_n \ b_1 \ b_2 \ \dots \ b_m]^T \quad (6.45)$$

Với vector hồi quy định nghĩa ở biểu thức (6.36), mô hình ARX có thể viết gọn lại như sau:

$$\hat{y}(t, \zeta) = \varphi^T(t) \zeta \quad (6.46)$$

Nếu hệ thống làm việc trong miền tín hiệu lớn thì đặc tính phi tuyến xuất hiện, khi đó mô hình tuyến tính xung quanh điểm tĩnh không thể mô tả được quan hệ vào ra của hệ thống nữa. Vấn đề đặt

ra là làm thế nào để sử dụng thông tin biết trước về cấu trúc mô hình tuyến tính cục bộ để xây dựng mô hình phi tuyến của hệ thống. Mô hình mờ Takagi–Sugeno rất phù hợp để giải quyết bài toán nêu trên.

Gọi $z(t) = [z_1(t), z_2(t), \dots, z_{n_z}(t)]^T$ là vector xác định các điểm làm việc tĩnh của hệ phi tuyến, mô hình mờ Takagi–Sugeno mô tả đặc tĩnh động học hệ phi tuyến gồm các quy tắc có dạng:

Nếu $(z_1(t) \text{ là } \tilde{A}_{i1}) \text{ và...và } (z_{n_z}(t) \text{ là } \tilde{A}_{in_z}) \text{ thì } \hat{y}(t) = \varphi^T(t)\zeta_i$ (6.47)

Gọi $\mu_{\tilde{A}_{ij}}(z_j(t), \eta_{ij})$ là hàm liên thuộc của các tập mờ ở mệnh đề điều kiện (η_{ij} là vector cột thông số của tập mờ $\tilde{A}_{ij}$, có thể chỉnh định được). Sử dụng toán tử PROD để thực hiện phép toán logic AND, độ đúng β_i của mệnh đề điều kiện của quy tắc thứ i là:

$$\beta_i(z(t), \eta_i) = \prod_{j=1}^{n_z} \mu_{\tilde{A}_{ij}}(z_j(t), \eta_{ij}) \tag{6.48}$$

$$\eta_i = [\eta_{i1}^T \quad \eta_{i2}^T \quad \dots \quad \eta_{in_z}^T]^T \tag{6.49}$$

Áp dụng công thức giải mờ trung bình có trọng số, ta được biểu thức bộ dự báo tín hiệu ra của hệ phi tuyến mô tả bởi hệ quy tắc (6.47) là:

$$\hat{y}(t, \theta) = \frac{\sum_{i=1}^{n_r} \beta_i(z(t), \eta_i) \varphi^T(t) \zeta_i}{\sum_{i=1}^{n_r} \beta_i(z(t), \eta_i)} \tag{6.50}$$

trong đó vector thông số của mô hình gồm tất cả các thông số hàm liên thuộc ở mệnh đề điều kiện và thông số mô hình tuyến tính ARX ở mệnh đề kết luận.

$$\theta = [\eta_1^T \quad \eta_2^T \quad \dots \quad \eta_{n_r}^T \quad \zeta_1^T \quad \zeta_2^T \quad \dots \quad \zeta_{n_r}^T]^T \tag{6.51}$$

Biểu thức (6.50) chứng tỏ tín hiệu ra của mô hình mờ Takagi–Sugeno là nội suy phi tuyến (do β_i phi tuyến) giữa các mô hình tuyến tính.

Nếu các tập mờ mô tả các giá trị ngôn ngữ của các biến $z_i(t)$ được phân hoạch mờ và hệ quy tắc hoàn chỉnh thì biểu thức (6.50) có thể viết gọn lại:

$$\hat{y}(t, \theta) = \sum_{i=1}^{n_r} \beta_i(z(t), \eta_i) \varphi^T(t) \zeta_i \quad (6.52)$$

Nếu các thông số của các hàm liên thuộc ở mệnh đề điều kiện được chọn trước cố định nhờ thông tin biết trước về điểm làm việc tĩnh thì vector thông số cần ước lượng chỉ gồm các thông số tuyến tính ở mệnh đề kết luận. Trong trường hợp này có thể ước lượng thông số dễ dàng từ tập dữ liệu vào-ra bằng công thức bình phương tối thiểu tuyến tính quen thuộc:

$$\zeta = \left(\sum_{t=1}^N \Phi(t) \Phi^T(t) \right)^{-1} \left(\sum_{t=1}^N \Phi(t) y(t) \right) \quad (6.53)$$

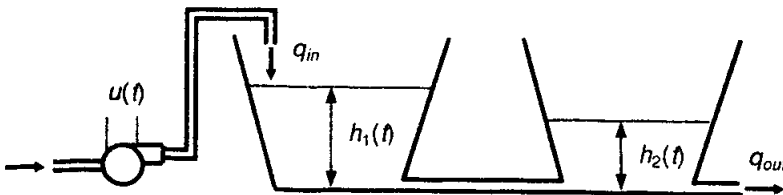
trong đó:

$$\Phi(t) = [\beta_1(z(t)) \varphi^T(t) \quad \beta_2(z(t)) \varphi^T(t) \quad \dots \quad \beta_{n_r}(z(t)) \varphi^T(t)]^T \quad (6.54)$$

$$\zeta = [\zeta_1^T \quad \zeta_2^T \quad \dots \quad \zeta_{n_r}^T]^T \quad (6.55)$$

Ngược lại, nếu các thông số của các hàm liên thuộc ở mệnh đề điều kiện chưa biết trước thì ta phải ước lượng cả các thông số ở mệnh đề điều kiện và các thông số ở mệnh đề kết luận. Trong trường hợp này vector θ chỉ có thể ước lượng bằng cách giải bài toán bình phương tối thiểu phi tuyến dùng các thuật toán lặp đã trình bày ở mục 6.2.5.

Ví dụ 6.2 Nhận dạng hệ bốn nối tiếp dùng mô hình mờ Takagi–Sugeno

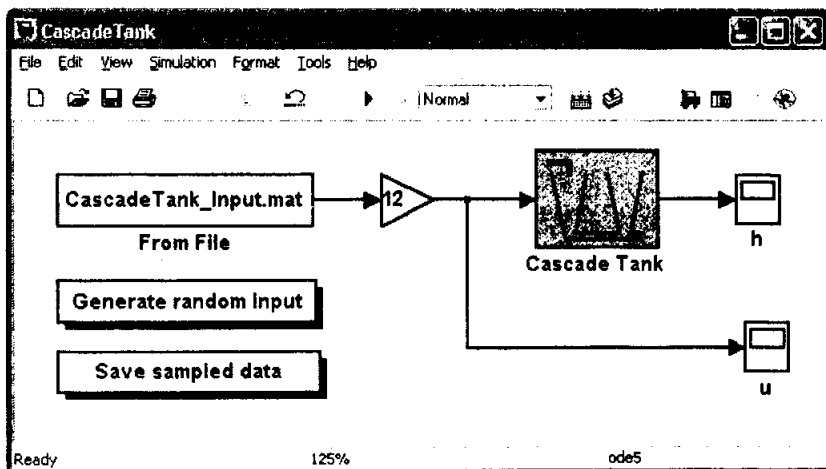


Hình 6.16 Hệ bốn nối tiếp

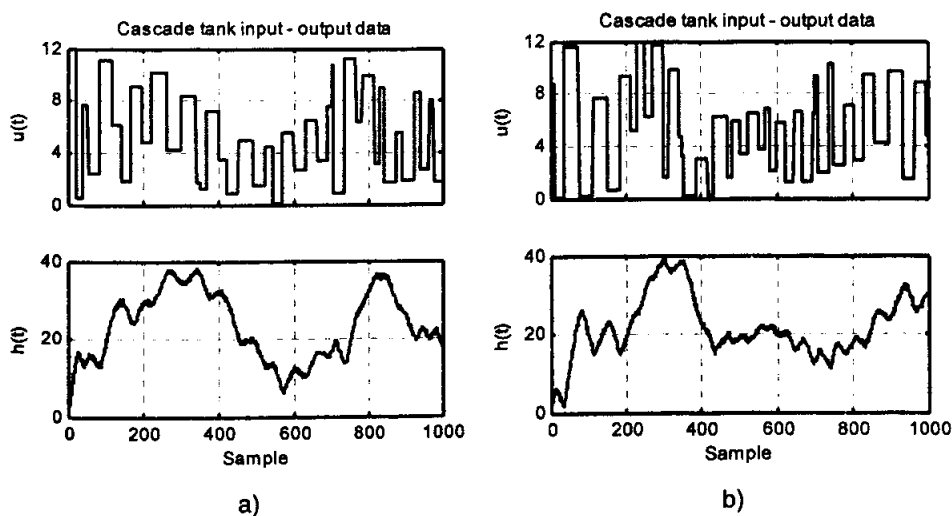
Các phương trình vi phân mô tả quan hệ vào ra của hệ bốn chứa chất lỏng nối tiếp đã trình bày ở ví dụ 5.11. Bài toán đặt ra là nhận dạng quan hệ mô hình phi tuyến giữa tín hiệu ra $y(t) = h_2(t)$ và tín hiệu vào $u(t)$.

Thí nghiệm thu thập dữ liệu

Hình 6.17 là sơ đồ mô phỏng thí nghiệm thu thập dữ liệu vào ra của hệ bồn nối tiếp, trong đó khối [Cascade Tank] mô phỏng đặc tính động học tả bởi các phương trình (5.47), (5.48) và (5.49). Giả sử tín hiệu đo mực chất lỏng ở bồn 2 bị ảnh hưởng bởi nhiễu cộng ngẫu nhiên có trung bình bằng 0 và phương sai bằng 0.1. Hai tập dữ liệu dùng để ước lượng thông số và đánh giá mô hình thu thập từ sơ đồ mô phỏng trình bày ở hình 6.18.



Hình 6.17 Sơ đồ thu thập dữ liệu vào ra hệ bồn nối tiếp



Hình 6.18 Dữ liệu vào-ra của hệ bồn chứa nối tiếp
a) Dữ liệu ước lượng thông số; b) Dữ liệu đánh giá mô hình

Xây dựng bộ dự báo mờ

Ta thấy rằng đặc tính phi tuyến của hệ bồn nối tiếp phụ thuộc vào độ cao của mực chất lỏng trong bồn, mặt khác do hệ bồn nối tiếp là hệ bậc 2 nên xung quanh mỗi điểm làm việc tính ta có thể mô hình hóa dùng mô hình ARX bậc 2. Định nghĩa các tập mờ mô tả các giá trị ngôn ngữ của biến $y(t-1)$ xác định điểm làm việc tính như hình 6.19, quan hệ vào ra của hệ bồn nối tiếp có thể mô tả bằng hệ quy tắc mờ Takagi–Sugeno như sau:

Nếu $y(t-1)$ là “Rất_thấp” **thì**

$$y(t) = \text{ARX1} = \theta_1 y(t-1) + \theta_2 y(t-2) + \theta_3 u(t-1) + \theta_4 u(t-2)$$

Nếu $y(t-1)$ là “Thấp” **thì**

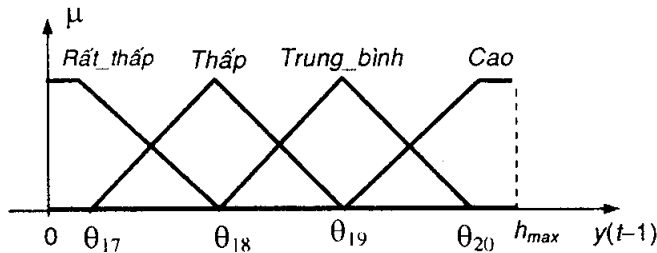
$$y(t) = \text{ARX2} = \theta_5 y(t-1) + \theta_6 y(t-2) + \theta_7 u(t-1) + \theta_8 u(t-2)$$

Nếu $y(t-1)$ là “Trung_bình” **thì**

$$y(t) = \text{ARX3} = \theta_9 y(t-1) + \theta_{10} y(t-2) + \theta_{11} u(t-1) + \theta_{12} u(t-2)$$

Nếu $y(t-1)$ là “Cao” **thì**

$$y(t) = \text{ARX4} = \theta_{13} y(t-1) + \theta_{14} y(t-2) + \theta_{15} u(t-1) + \theta_{16} u(t-2)$$



Hình 6.19 Hàm liên thuộc của các tập mờ

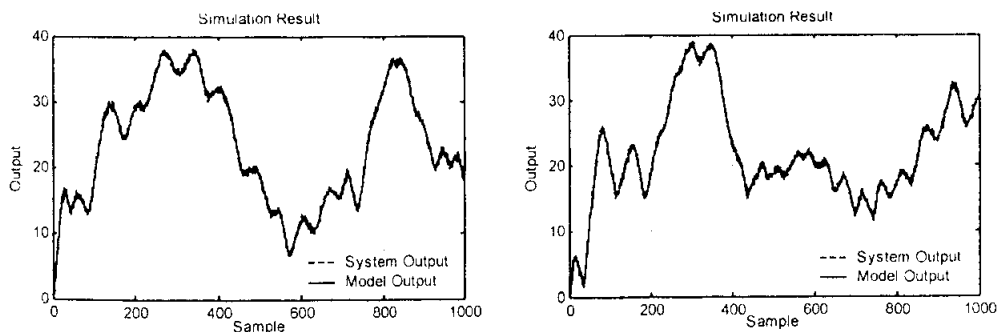
Do các tập mờ ở hình 6.19 được phân hoạch mờ và hệ quy tắc hoàn chỉnh nên áp dụng công thức (6.52) ta được biểu thức bộ dự báo như sau:

$$\begin{aligned} \hat{y}(t, \theta) = & \mu_{\text{Rất_thấp}}(y(t-1), \theta_{17}, \theta_{18}) \times \\ & [\theta_1 y(t-1) + \theta_2 y(t-2) + \theta_3 u(t-1) + \theta_4 u(t-2)] \\ & + \mu_{\text{Thấp}}(y(t-1), \theta_{17}, \theta_{18}, \theta_{19}) \times \\ & [\theta_5 y(t-1) + \theta_6 y(t-2) + \theta_7 u(t-1) + \theta_8 u(t-2)] \\ & + \mu_{\text{TB}}(y(t-1), \theta_{18}, \theta_{19}, \theta_{20}) \times \\ & [\theta_9 y(t-1) + \theta_{10} y(t-2) + \theta_{11} u(t-1) + \theta_{12} u(t-2)] \\ & + \mu_{\text{Cao}}(y(t-1), \theta_{19}, \theta_{20}) \times \\ & [\theta_{13} y(t-1) + \theta_{14} y(t-2) + \theta_{15} u(t-1) + \theta_{16} u(t-2)] \end{aligned}$$

Kết quả ước lượng thông số và đánh giá mô hình

Ta có thể chọn trước thông số các tập mờ ở mệnh đề điều kiện để xác định trước các điểm làm việc tĩnh, sau đó ước lượng thông số của các mô hình ARX tuyến tính. Như đã trình bày ở phần lý thuyết, cách này có ưu điểm là có thể ước lượng thông số mô hình tuyến tính rất dễ dàng, tuy nhiên nếu ta chọn thông số các cửa của các tập mờ ở mệnh đề điều kiện không phù hợp với đặc tính phi tuyến của đối tượng thì sai số nhận dạng sẽ lớn.

Sau đây là kết quả thu được bằng cách áp dụng thuật toán Levenberg–Marquardt để ước lượng tất cả các thông số của mô hình mờ Takagi–Sugeno. Ta thấy ngõ ra của mô hình mờ Takagi–Sugeno gần như trùng khớp với ngõ ra của hệ thống.



Hình 6.20 Đánh giá mô hình

a) Phân tích thẳng dư; b) Đánh giá chéo

6.3.4 Kỹ thuật phân nhóm mờ và ứng dụng nhận dạng hệ phi tuyến

Ở mục 6.3.3, để nhận dạng mô hình mờ Takagi–Sugeno ta chọn trước dạng tập mờ ở mệnh đề điều kiện, sau đó ước lượng thông số mệnh đề điều kiện và mệnh đề kết luận dựa vào dữ liệu vào ra quan sát được dùng thuật toán tối ưu hóa phi tuyến. Mục này trình bày một cách khác để xây dựng mô hình mờ Takagi–Sugeno, đó là dùng kỹ thuật phân nhóm mờ để xác định các tập mờ $\tilde{A}_{ij}$ ở mệnh đề điều kiện, sau đó các tham số tuyến tính ở mệnh đề kết luận được ước lượng bằng thuật toán bình phương tối thiểu. Cách xây dựng mô hình mờ này hoàn toàn dựa vào dữ liệu thực nghiệm, không cần thông tin định tính biết trước về đặc tính động học của hệ thống.

1- Định nghĩa phân nhóm rõ và phân nhóm mờ

Mục đích của phân nhóm là phân loại các đối tượng (dữ liệu) dựa vào các đặc điểm tương tự nhau và phân chúng ra thành các nhóm khác nhau. Ưu điểm của phân nhóm là phát hiện được cấu trúc bên trong của dữ liệu, không chỉ để phân loại và nhận dạng mẫu mà còn để rút gọn mô hình và tối ưu hóa.

Phân nhóm rõ

Giả sử ta có tập dữ liệu $X = \{x_k \in \mathcal{R}^p \mid k = 1, 2, \dots, N\}$. Gọi $P(X)$ là tập hợp gồm tất cả các tập con của X . Một phân nhóm rõ của X là họ các tập con $\{A_i \subset P(X), 1 \leq i \leq c\}$ sao cho:

$$\bigcup_{i=1}^c A_i = X \quad (6.56)$$

$$A_i \cap A_j = \emptyset \text{ với } 1 \leq i \neq j \leq c \quad (6.57)$$

Hai điều kiện trên có nghĩa là các tập con phải chứa tất cả dữ liệu trong X và các tập con phải độc lập nhau. Mỗi tập con A_i được xem như là một nhóm, như vậy $\{A_1, A_2, \dots, A_c\}$ phân X thành c nhóm. Phân nhóm rõ có thể định nghĩa thông qua hàm liên thuộc của các phần tử x_k trong A_i . Cụ thể, định nghĩa:

$$\mu_{ik} = \begin{cases} 1, & x_k \in A_i \\ 0, & x_k \notin A_i \end{cases} \quad (6.58)$$

trong đó $x_k \in X$, $A_i \in P(X)$, $i = 1, 2, \dots, c$, $k = 1, 2, \dots, N$. Rõ ràng, $\mu_{ik} = 1$ khi x_k thuộc phân nhóm A_i . Cho các giá trị μ_{ik} chúng ta có thể xác định duy nhất c nhóm rõ A_i và ngược lại. Các giá trị μ_{ik} phải thỏa mãn các điều kiện:

$$\mu_{ik} \in \{0, 1\}, 1 \leq i \leq c, 1 \leq k \leq N \quad (6.59)$$

$$\sum_{i=1}^c \mu_{ik} = 1, \forall k \in \{1, 2, \dots, N\} \quad (6.60)$$

$$0 < \sum_{k=1}^N \mu_{ik} < N, \forall i \in \{1, 2, \dots, c\} \quad (6.61)$$

Điều kiện (6.59) và (6.60) có nghĩa là mỗi $x_k \in X$ thuộc và chỉ thuộc về một nhóm. Điều kiện (6.61) có nghĩa là các nhóm phải khác rỗng và không được bằng X . Tập hợp các giá trị μ_{ik} với $1 \leq i \leq c, 1 \leq k \leq N$ có

thể sắp xếp thành ma trận U bậc $c \times N$, c phân nhóm rõ có thể biểu diễn dưới dạng ma trận như sau:

$$U = \begin{bmatrix} \mu_{11} & \mu_{12} & \dots & \mu_{1N} \\ \mu_{21} & \mu_{22} & \dots & \mu_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ \mu_{c1} & \mu_{c1} & \dots & \mu_{cN} \end{bmatrix} \quad (6.62)$$

Theo cách biểu diễn trên, mỗi hàng của ma trận U tương ứng với một nhóm.

Ví dụ 6.3 Giả sử X là tập hợp ba loại xe:

$$X = \{x_1 = \textit{Toyota}, x_2 = \textit{Ford}, x_3 = \textit{Honda}\}$$

Nếu phân tập X thành hai nhóm rõ ($c = 2$) thì theo các điều kiện ràng buộc (6.59) – (6.61) có ba cách phân nhóm được biểu diễn dưới dạng ma trận như sau:

$$U_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}, \quad U_2 = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad U_3 = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Các ma trận trên được hiểu như sau:

U_1 : nhóm 1 = {*Toyota*}, nhóm 2 = {*Ford*, *Honda*}

U_2 : nhóm 1 = {*Toyota*, *Ford*}, nhóm 2 = {*Honda*}

U_3 : nhóm 1 = {*Toyota*, *Honda*}, nhóm 2 = {*Ford*}

Nếu ta muốn phân ba loại xe trên thành hai nhóm xe Mỹ và xe Nhật thì cách phân nhóm thứ ba là hợp lý nhất. Tuy nhiên, sự phân biệt giữa xe Mỹ và xe Nhật không rõ ràng vì có nhiều bộ phận trong xe Mỹ được nhập khẩu từ Nhật và xe Nhật có thể sản xuất tại Mỹ. Để giải quyết vấn đề trên ta sử dụng cách phân nhóm mờ.

Phân nhóm mờ

Phân nhóm mờ có thể xem là trường hợp mở rộng của phân nhóm rõ, phân nhóm mờ cho phép μ_{ik} nhận giá trị bất kỳ trong khoảng $[0,1]$, tức là cho phép một phần tử thuộc vào một phân nhóm nào đó với độ chắc chắn nhỏ hơn 1.

Ma trận $c \times N$ $\tilde{U} = [\mu_{ik}]$ biểu diễn cho phân nhóm mờ nếu và chỉ nếu các phần tử của nó thoả mãn các điều kiện sau:

$$\mu_{ik} \in [0,1], 1 \leq i \leq c, 1 \leq k \leq N \quad (6.63)$$

$$\sum_{i=1}^c \mu_{ik} = 1, \forall k \in \{1,2,\dots,N\} \quad (6.64)$$

$$0 < \sum_{k=1}^N \mu_{ik} < N, \forall i \in \{1,2,\dots,c\} \quad (6.65)$$

Trở lại ví dụ 6.3, sử dụng phân nhóm mờ, một cách phân nhóm xe Mỹ và xe Nhật hợp lý hơn được biểu diễn dưới dạng ma trận như sau:

$$\tilde{U} = \begin{bmatrix} 0.9 & 0.2 & 0.85 \\ 0.1 & 0.8 & 0.15 \end{bmatrix}$$

2- Giải thuật phân nhóm mờ c-Means (FCM)

Giải thuật phân nhóm mờ C-means tìm ma trận liên thuộc $U = [\mu_{ik}]$ và tâm của các nhóm $V = [v_1, v_2, \dots, v_c]$ với $v_i \in \mathcal{R}^p$ sao cho tối thiểu hóa hàm mục tiêu:

$$J_m(U, V) = \sum_{i=1}^c \sum_{k=1}^N (\mu_{ik})^m D^2(x_k, v_i, A) \quad (6.66)$$

trong đó $m \in (1, \infty)$ là hệ số trọng lượng và:

$$D(x_k, v_i, A) = \sqrt{(x_k - v_i)^T A (x_k - v_i)} \quad (6.67)$$

$D(x_k, v_i, A)$ là khoảng cách theo chuẩn A từ mẫu dữ liệu thứ k (x_k) đến tâm của phân nhóm thứ i (v_i). Ma trận A là ma trận chuẩn, trường hợp đơn giản nhất A là ma trận đơn vị. Cũng có thể chọn A là ma trận chéo biểu diễn cho các trị phương sai khác nhau theo các hướng trong không gian của $\mathbf{X}$:

$$A = \begin{bmatrix} (1/\sigma_1)^2 & 0 & \dots & 0 \\ 0 & (1/\sigma_2)^2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & (1/\sigma_n)^2 \end{bmatrix} \quad (6.68)$$

Hoặc A có thể là ma trận nghịch đảo của ma trận phương sai

$$A = F^{-1} \quad (6.69)$$

$$\text{với: } F = \frac{1}{N} \sum_{k=1}^N (x_k - \bar{x})(x_k - \bar{x})^T \quad (6.70)$$

trong đó $\bar{x}$ là trị trung bình của dữ liệu:

$$\bar{x} = \frac{1}{N} \sum_{k=1}^N x_k \quad (6.71)$$

Định lý 6.1:

Cho $X = \{x_k \in \mathbb{R}^p \mid k = 1, 2, \dots, N\}$ là tập dữ liệu, $c \in \{2, 3, \dots, N-1\}$ và $m \in (1, \infty)$. Nếu $\|x_k - v_i\| \neq 0$ với mọi $1 \leq k \leq N$ và $1 \leq i \leq c$ thì $U = [\mu_{ik}]$ và $V = [v_1, v_2, \dots, v_c]$ là cực tiểu cục bộ của $J_m(U, V)$ chỉ khi:

$$u_{ik} = \frac{1}{\sum_{j=1}^c \left(\frac{D(x_k, v_i, A)}{D(x_k, v_j, A)} \right)^{\frac{2}{m-1}}}, \quad (6.72)$$

$$1 \leq i \leq c, \quad 1 \leq k \leq N$$

và

$$v_i = \frac{\sum_{k=1}^N (\mu_{ik})^m x_k}{\sum_{k=1}^N (\mu_{ik})^m}, \quad 1 \leq i \leq c \quad (6.73)$$

Chứng minh: xem [109].

Giải thuật phân nhóm mờ c-Means được đưa ra dựa vào điều kiện cần (6.72) và (6.73) như sau:

Giải thuật 6.1 Phân nhóm mờ c-Means

Cho trước tập dữ liệu $X = \{x_k \in \mathbb{R}^p \mid k = 1, 2, \dots, N\}$, chọn số nhóm $1 \leq i \leq c$, và hệ số trọng lượng $m \in (1, \infty)$, sai số kết thúc vòng lặp $\varepsilon > 0$.

Bước 1: Khởi động ngẫu nhiên $U^{(0)} = [\mu_{ik}^{(0)}]$. Gán $l = 0$.

Bước 2: Tính tâm của các nhóm:

$$v_i^{(l)} = \frac{\sum_{k=1}^N (\mu_{ik}^{(l)})^m x_k}{\sum_{k=1}^N (\mu_{ik}^{(l)})^m}, \quad 1 \leq i \leq c \quad (6.74)$$

Bước 3: Tính khoảng cách từ các mẫu dữ liệu đến tâm các nhóm:

$$D(x_k, v_i^{(l)}, A) = \sqrt{(x_k - v_i^{(l)})^T A (x_k - v_i^{(l)})} \quad (6.75)$$

$$(1 \leq i \leq c, \quad 1 \leq k \leq N)$$

Bước 4: Cập nhật ma trận phân nhóm mờ:

$$\mu_{ik}^{(l+1)} = \frac{1}{\sum_{j=1}^c \left(\frac{D(x_k, v_i^{(l)}, A)}{D(x_k, v_j^{(l)}, A)} \right)^{\frac{2}{m-1}}}, \quad (6.76)$$

$$(1 \leq i \leq c, 1 \leq k \leq N)$$

Bước 5: Nếu $\|U^{(l+1)} - U^{(l)}\| < \varepsilon$ thì dừng. Ngược lại gán $l = l + 1$ và trở lại bước 2.

3- Giải thuật phân nhóm mờ Gustafson-Kessel (GK)

Gustafson và Kessel đã mở rộng giải thuật FCM chuẩn bằng cách đưa ra một chuẩn khoảng cách thích nghi, mục đích là có thể phát hiện ra các nhóm có hình dáng khác nhau trong một tập dữ liệu. Mỗi nhóm sẽ có ma trận chuẩn A_i riêng. Các ma trận A_i cũng là các biến cần tối ưu hoá trong giải thuật GK, vì thế nó cho phép các nhóm thay đổi chuẩn khoảng cách cho phù hợp với cấu trúc và hình dáng cục bộ của dữ liệu. Đặt $A = (A_1, A_2, \dots, A_c)$, hàm mục tiêu của giải thuật GK được định nghĩa như sau:

$$J_m(U, V, A) = \sum_{i=1}^c \sum_{k=1}^N (\mu_{ik})^m D^2(x_k, v_i, A_i) \quad (6.77)$$

Ma trận A_i có thể thay đổi tương ứng với hình dạng của các nhóm trong quá trình tối ưu hóa nhưng định thức của nó phải không đổi.

$$\|A_i\| = \rho_i, \quad \rho_i > 0, \quad (6.78)$$

trong đó ρ_i là hằng số đối với mỗi nhóm. Một cách chọn A_i mở rộng công thức (6.69) và (6.70), đồng thời thỏa mãn điều kiện (6.78) như sau:

$$A_i = [\rho_i \det(F_i)]^{1/N} F_i^{-1} \quad (6.79)$$

trong đó F_i là ma trận hiệp phương sai mờ của nhóm thứ i :

$$F_i = \frac{\sum_{k=1}^N (\mu_{ik})^m (x_k - v_i)(x_k - v_i)^T}{\sum_{k=1}^N (\mu_{ik})^m} \quad (6.80)$$

Giải thuật 6.2 Phân nhóm mờ Gustafson và Kessel

Cho trước tập dữ liệu $X = \{x_k \in \mathbb{R}^p \mid k = 1, 2, \dots, N\}$, chọn số nhóm $1 \leq i \leq c$, và hệ số trọng lượng $m \in (1, \infty)$, sai số kết thúc vòng lặp $\varepsilon > 0$.

Bước 1: Khởi động ngẫu nhiên $U^{(0)} = [\mu_{ik}^{(0)}]$. Gán $l = 0$.

Bước 2: Tính tâm của các nhóm:

$$v_i^{(l)} = \frac{\sum_{k=1}^N (\mu_{ik}^{(l)})^m x_k}{\sum_{k=1}^N (\mu_{ik}^{(l)})^m}, \quad 1 \leq i \leq c \quad (6.81)$$

Bước 3: Tính các ma trận hiệp phương sai mờ:

$$F_i^{(l)} = \frac{\sum_{k=1}^N (\mu_{ik}^{(l)})^m (x_k - v_i^{(l)})(x_k - v_i^{(l)})^T}{\sum_{k=1}^N (\mu_{ik}^{(l)})^m} \quad (6.82)$$

Bước 4: Tính các ma trận chuẩn:

$$A_i^{(l)} = [\rho_i \det(F_i^{(l)})]^{1/N} [F_i^{(l)}]^{-1} \quad (6.83)$$

Bước 5: Tính khoảng cách từ các mẫu dữ liệu đến tâm các nhóm:

$$D(x_k, v_i^{(l)}, A_i^{(l)}) = \sqrt{(x_k - v_i^{(l)})^T A_i^{(l)} (x_k - v_i^{(l)})}, \quad (6.84)$$

$$(1 \leq i \leq c, 1 \leq k \leq N)$$

Bước 6: Cập nhật ma trận phân nhóm mờ:

$$\mu_{ik}^{(l+1)} = \frac{1}{\sum_{j=1}^c \left(\frac{D(x_k, v_i^{(l)}, A_i^{(l)})}{D(x_k, v_j^{(l)}, A_j^{(l)})} \right)^{\frac{2}{m-1}}}, \quad (6.85)$$

$$(1 \leq i \leq c, 1 \leq k \leq N)$$

Bước 7: Nếu $\|U^{(l+1)} - U^{(l)}\| < \varepsilon$ thì dừng. Ngược lại gán $l = l + 1$ và trở lại bước 2.

4- Nhận dạng hệ phi tuyến dùng kỹ thuật phân nhóm mờ

Các bước nhận dạng mô hình mờ Takagi–Sugeno sử dụng kỹ thuật phân nhóm mờ để phân hoạch các tập mờ ở mệnh đề điều kiện như sau:

- Thực hiện thí nghiệm thu thập dữ liệu vào ra của hệ thống cần nhận dạng. Bước này rất quan trọng và đóng vai trò quyết định chất lượng của mô hình vì phương pháp dùng kỹ thuật phân nhóm mờ là phương pháp nhận dạng dựa vào dữ liệu, không sử dụng các quy tắc phát biểu từ kinh nghiệm trực giác; nếu dữ liệu không tổng quát, không phản ánh được đặc tính động học của hệ thống thì kết quả nhận dạng sẽ không tốt.

- Chọn các phần tử hồi quy $\varphi_i(t)$ ($i = 1, 2, \dots, n_\varphi$) là các thành phần tín hiệu vào và tín hiệu ra trong quá khứ. Xây dựng tập dữ liệu dùng để phân nhóm:

$$X = \begin{bmatrix} \varphi_1(1) & \varphi_2(1) & \dots & \varphi_{n_\varphi}(1) \\ \varphi_1(2) & \varphi_2(2) & \ddots & \varphi_{n_\varphi}(2) \\ \vdots & \vdots & \dots & \vdots \\ \varphi_1(N) & \varphi_2(N) & \dots & \varphi_{n_\varphi}(N) \end{bmatrix} \quad (6.86)$$

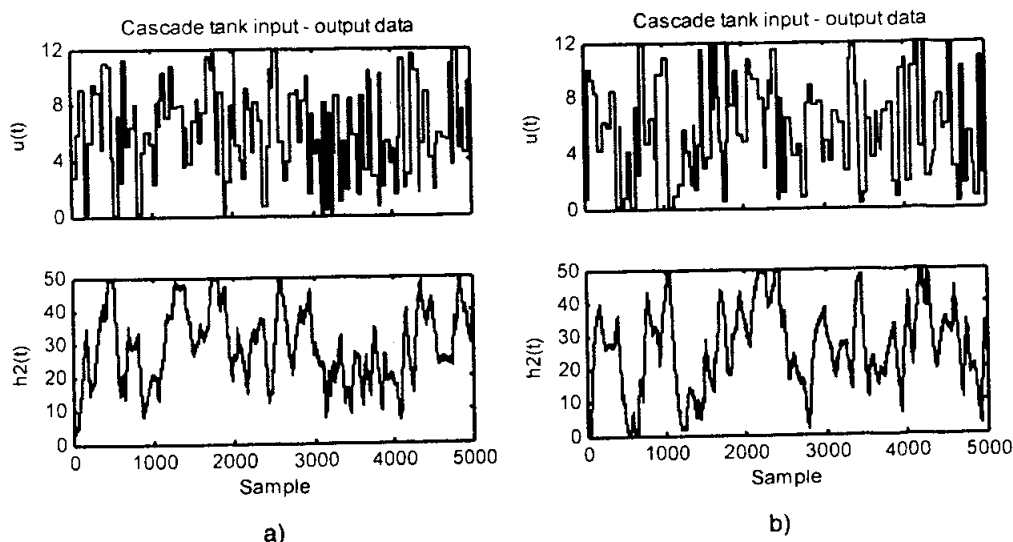
- Sử dụng giải thuật FCM hoặc GK để phân dữ liệu X ra làm c nhóm (số nhóm c chọn trước). Kết quả phân nhóm là ma trận hàm liên thuộc U gồm c hàng, mỗi hàng gồm N giá trị của hàm liên thuộc xác định một tập mờ trong không gian tích $\varphi_1 \times \varphi_2 \times \dots \times \varphi_{n_\varphi}$.

- Chiếu từng hàm liên thuộc xác định bởi các điểm trong không gian n_φ chiều xuống từng phần tử hồi quy, sử dụng bộ lọc thông thấp để trơn hóa hàm liên thuộc, sau đó xấp xỉ hàm liên thuộc xác định bởi các điểm rời rạc bằng các dạng hàm liên thuộc có thông số thường dùng (dạng chuông, dạng gauss).

- Ước lượng thông số mệnh đề kết luận của mô hình Takagi–Sugeno dùng thuật toán bình phương tối thiểu tuyến tính. Một số tác giả đề nghị ở bước này nên tinh chỉnh cả tập mờ ở mệnh đề điều kiện và ước lượng thông số của các mô hình tuyến tính ở mệnh đề kết luận dùng thuật toán ANFIS.

Ví dụ 6.4 Nhận dạng mô hình Takagi–Sugeno của hệ bốn nối tiếp dùng kỹ thuật phân nhóm mờ.

- Xét hệ bồn nối tiếp mô tả ở ví dụ 6.2. Thu thập hai tập dữ liệu vào-ra của hệ thống để phân nhóm mờ, ước lượng thông số và đánh giá mô hình, mỗi tập dữ liệu gồm 5000 mẫu (H.6.21). Chú ý rằng, vì trong ví dụ này ta đưa ra các tập mờ ở mệnh đề điều kiện dựa vào dữ liệu dùng phương pháp phân nhóm nên tập dữ liệu dùng để nhận dạng gồm nhiều mẫu hơn so với ví dụ 6.2.



Hình 6.21 Dữ liệu vào-ra của hệ bồn chứa nối tiếp

a) Dữ liệu ước lượng thông số; b) Dữ liệu đánh giá mô hình

- Chọn các phần tử hồi quy ở mệnh đề điều kiện như sau:

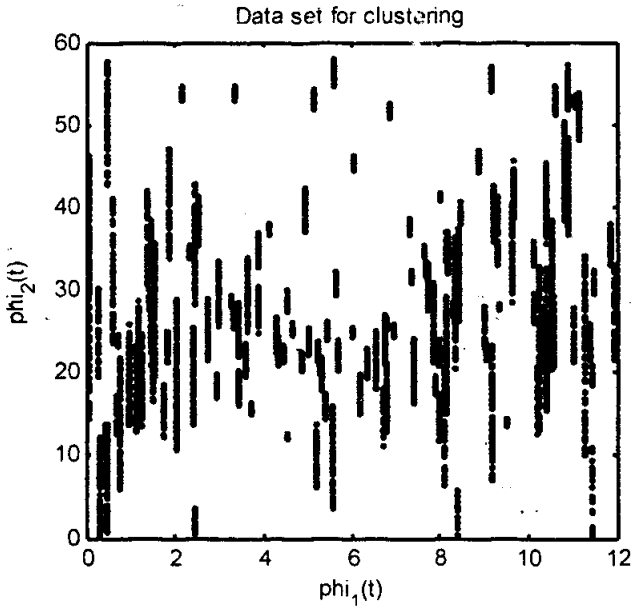
$$\begin{cases} \varphi_1(t) = u(t-1) \\ \varphi_2(t) = y(t-1) \end{cases} \quad (6.87)$$

- Tập dữ liệu X dùng để phân nhóm (biểu thức (6.86)) gồm hai cột, 4999 hàng, tập dữ liệu này được biểu diễn trong không gian hai chiều ở hình 6.22.

- Dùng lệnh **fcm** (fuzzy c-means) trong bộ công cụ Fuzzy của Matlab để phân tập dữ liệu thành ba nhóm.

$$>> [V,U] = fcm(X,3) \quad (6.88)$$

Chú ý giải thuật phân nhóm mờ c-Means là giải thuật tối ưu cục bộ nên mỗi lần thực thi lệnh (6.88) ta được kết quả khác nhau, tùy theo giá trị khởi động.



Hình 6.22 Dữ liệu phân nhóm để xác định các tập mờ ở mệnh đề điều kiện

Hình 6.23 là đồ thị biểu diễn ma trận giá trị liên thuộc sau khi phân nhóm. Chiều giá trị hàm liên thuộc xuống hai trục ta được các tập mờ ở mệnh đề điều kiện của hệ mờ như hình 6.24. Các quy tắc mờ Takagi–Sugeno mô tả đặc tính động học của hệ bốn nước nối tiếp với các thông số ở mệnh đề kết luận ước lượng dùng thuật toán bình phương tối thiểu như sau:

Nếu $y(t-1)$ là $\tilde{A}_{11}$ và $u(t-1)$ là $\tilde{A}_{21}$

$$y(t) = ARX1 = \theta_1 y(t-1) + \theta_2 y(t-2) + \theta_3 u(t-1) + \theta_4 u(t-2)$$

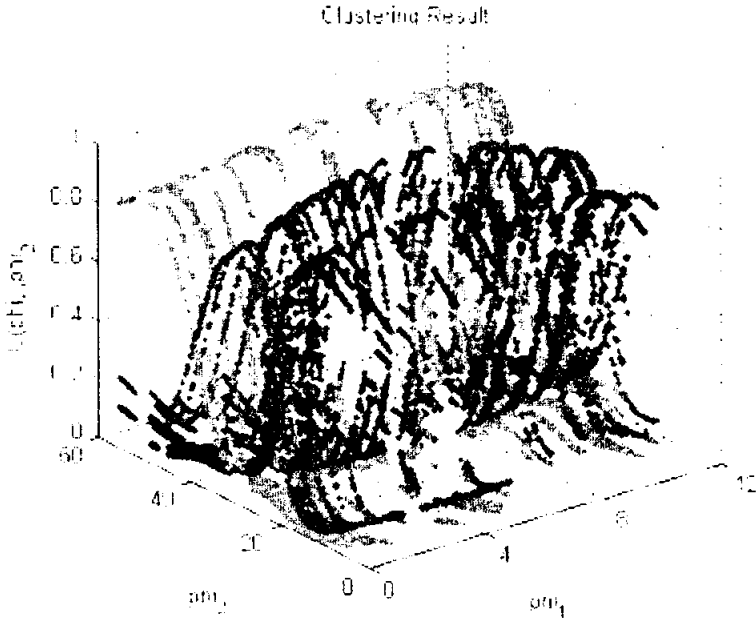
Nếu $y(t-1)$ là $\tilde{A}_{12}$ và $u(t-1)$ là $\tilde{A}_{22}$

$$y(t) = ARX2 = \theta_5 y(t-1) + \theta_6 y(t-2) + \theta_7 u(t-1) + \theta_8 u(t-2)$$

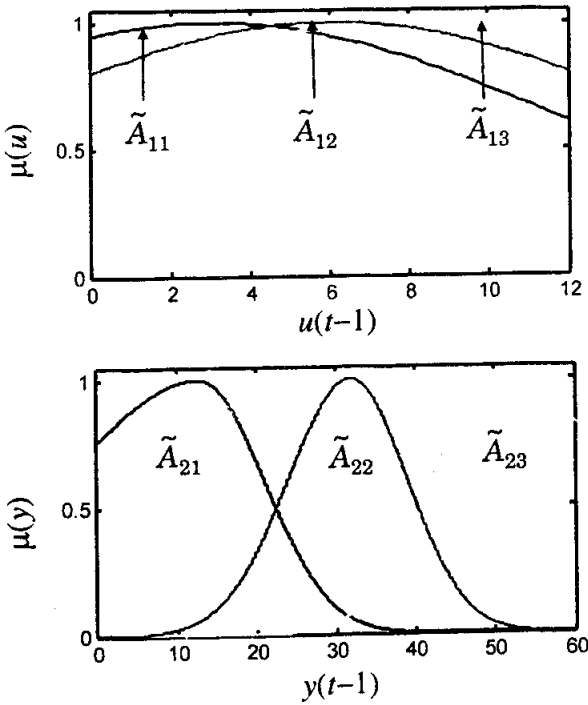
Nếu $y(t-1)$ là $\tilde{A}_{13}$ và $u(t-1)$ là $\tilde{A}_{23}$

$$y(t) = ARX3 = \theta_9 y(t-1) + \theta_{10} y(t-2) + \theta_{11} u(t-1) + \theta_{12} u(t-2)$$

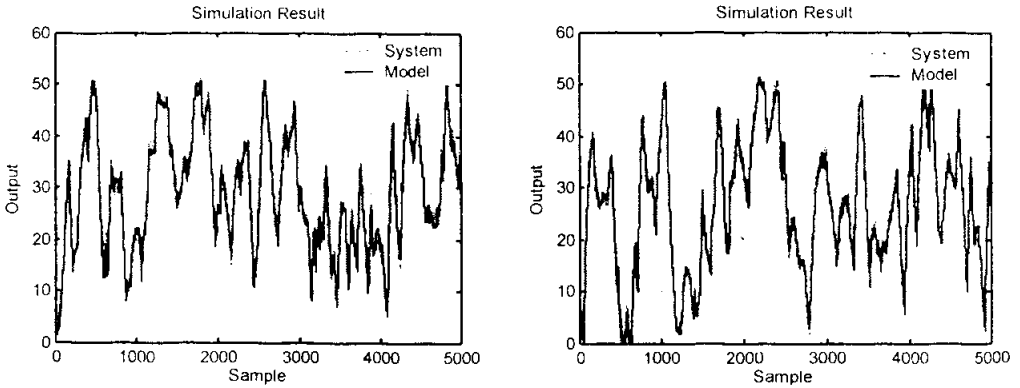
Hình 6.25 là đồ thị đánh giá mô hình sau khi ước lượng thông số của các mô hình ARX ở mệnh đề kết luận của các quy tắc. Ta thấy trong trường hợp này mặc dù mô hình mờ Takagi–Sugeno chỉ có ba quy tắc nhưng cũng mô tả được chính xác đặc tính động học của hệ bốn nối tiếp (so với ví dụ 6.2 mô hình Takagi–Sugeno) cần đến bốn quy tắc).



Hình 6.23 Tập mờ trong không gian hai chiều sau khi phân nhóm



Hình 6.24 Các tập mờ ở mệnh đề điều kiện



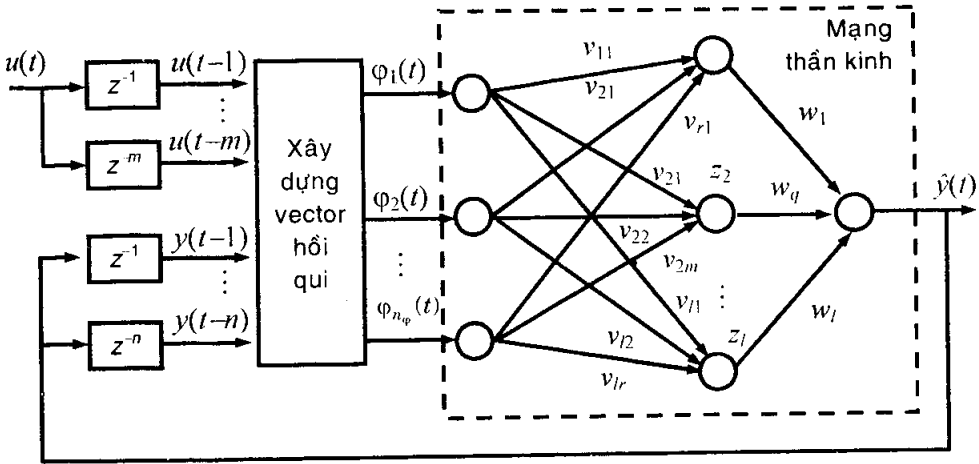
Hình 6.25 Các tập mờ ở mệnh đề điều kiện
a) Phân tích thặng dư; b) Đánh giá chéo

6.4 NHẬN DẠNG HỆ PHI TUYẾN DÙNG MÔ HÌNH MẠNG THẦN KINH

Nếu biết trước rất ít (thậm chí không biết) thông tin về đối tượng ta phải giải bài toán nhận dạng hệ thống chủ yếu dựa vào dữ liệu vào–ra thực nghiệm quan sát được. Trong trường hợp này, mạng thần kinh là cấu trúc mô hình phi tuyến được chọn lựa đầu tiên để mô hình hóa hệ thống vì mạng thần kinh có tính linh hoạt rất cao, có thể xấp xỉ hàm phi tuyến trơn với sai số nhỏ tùy ý nếu số tế bào thần kinh ở lớp ẩn đủ lớn, đồng thời trọng số của mạng có thể huấn luyện hoàn toàn dựa vào dữ liệu mà không cần thêm bất cứ thông tin gì khác về đối tượng cần nhận dạng.

6.4.1 Nhận dạng dùng mạng truyền thẳng nhiều lớp (MLP)

Tương tự như trường hợp mô hình mờ, để nhận dạng hệ thống động dùng mạng truyền thẳng nhiều lớp (*Multi Layer Perceptron-MLP*) ta phải đưa vào ngõ vào của mạng các phần tử hồi quy là tín hiệu vào và tín hiệu ra trong quá khứ. Gọi $\varphi(t) = [\varphi_1(t) \ \varphi_2(t) \ \dots \ \varphi_{n_\varphi}(t)]^T$ là vector hồi quy, trong đó các phần tử hồi quy $\varphi_i(t)$ được xây dựng từ tín hiệu vào và tín hiệu ra đến thời điểm $t-1$, bộ dự báo dùng mạng thần kinh có sơ đồ khối như trình bày ở hình 6.26. Mạng thần kinh có l tế bào ở lớp ẩn với hàm tác động là hàm Gauss hoặc Sigmoid, một tế bào ở lớp ra với hàm tác động tuyến tính.



Hình 6.26 Sơ đồ khối bộ dự báo dùng mạng thần kinh MLP

Tín hiệu ở ngõ ra của tế bào thần kinh ở lớp ẩn là:

$$z_i(t) = g_i \left(\sum_{j=1}^{n_q} v_{ij} \varphi_j(t) \right) \quad (i = 1, 2, \dots, l) \quad (6.89)$$

trong đó $g_i(.)$ là hàm tác động (hàm truyền) của tế bào thần kinh thứ i ở lớp ẩn, thông thường có thể chọn $g_i(.)$ là hàm Gauss hoặc hàm Sigmoid.

Do hàm tác động của tế bào thần kinh ở lớp ra là hàm tuyến tính nên ngõ ra của bộ dự báo dùng mạng thần kinh có thể biểu diễn bằng công thức sau:

$$y(t, \theta) = \sum_{i=1}^l w_i z_i(t) = \sum_{i=1}^l w_i g_i \left(\sum_{j=1}^{n_q} v_{ij} \varphi_j(t) \right) \quad (6.90)$$

trong đó vector thông số cần ước lượng là:

$$\theta = [v_{11}, \dots, v_{1r}, \dots, v_{l1}, \dots, v_{lr}, w_1, \dots, w_l]^T$$

Biểu thức (6.90) có dạng tổng quát (6.5) do đó có thể ước lượng thông số bằng thuật toán Newton như trình bày ở mục 6.2.5, ngoài ra cũng có thể ước lượng thông số dùng thuật toán lan truyền ngược huấn luyện mạng thần kinh đã trình bày ở chương 3.

Khi sử dụng mạng thần kinh để nhận dạng hệ thống cần phải quan tâm chọn cấu hình mạng, cụ thể:

- Số tế bào thần kinh ở lớp vào: phụ thuộc vào số phần tử hồi quy.

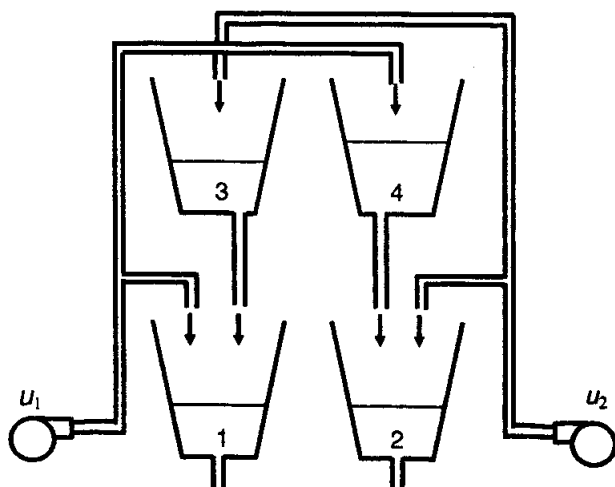
- *Số tế bào thần kinh ở lớp ẩn*: phụ thuộc vào tính phi tuyến của đối tượng cần nhận dạng, đối tượng có tính phi tuyến càng cao thì càng cần nhiều tế bào thần kinh ở lớp ẩn. Tuy nhiên không phải chọn càng nhiều tế bào thần kinh ở lớp ẩn càng tốt vì có thể dẫn đến hiện tượng quá khớp (*overfit*).

- *Hàm tác động của tế bào thần kinh ở lớp ẩn*: thực tế cho thấy hàm Gauss có thể phù hợp để giải bài toán nhận dạng rất nhiều hệ thống khác nhau. Cần nhớ hàm Gauss là chọn lựa đầu tiên nhưng không phải là chọn lựa duy nhất, tùy trường hợp cụ thể, có thể các hàm khác sẽ cho kết quả nhận dạng tốt hơn.

Khi chọn lựa cấu hình mạng cần nhớ nguyên tắc cơ bản khi giải bài toán nhận dạng hệ thống là xây dựng mô hình càng đơn giản càng tốt, điều này có nghĩa là nên chọn số phần tử hồi quy và số tế bào thần kinh ở lớp ẩn càng ít càng tốt, miễn sao sai lệch giữa ngõ ra của mạng và ngõ ra của hệ thống cần nhận dạng đủ nhỏ chấp nhận được.

Ví dụ 6.5 Nhận dạng hệ bốn bồn chứa liên kết.

Xét hệ bốn bồn chứa liên kết có sơ đồ khối ở hình 6.27, đây là hệ MIMO có hai ngõ vào là tín hiệu điều khiển hai máy bơm và hai ngõ ra là mực chất lỏng trong bồn 1 và bồn 2. Máy bơm 1 bơm chất lỏng vào bồn 1 và bồn 4, máy bơm 2 bơm chất lỏng vào bồn 2 và bồn 3.



Hình 6.27 Hệ bốn bồn chứa liên kết

Mô hình toán học của hệ bốn bồn liên kết cho bởi các phương trình sau:

$$\begin{cases} \dot{h}_1 = \frac{1}{A_1(h_1)} (\alpha_1 k_1 u_1 + C_3 a_3 \sqrt{2gh_3} - C_1 a_1 \sqrt{2gh_1}) \\ \dot{h}_2 = \frac{1}{A_2(h_2)} (\alpha_2 k_2 u_2 + C_4 a_4 \sqrt{2gh_4} - C_2 a_2 \sqrt{2gh_2}) \\ \dot{h}_3 = \frac{1}{A_3(h_3)} [(1 - \alpha_2) k_2 u_2 - C_3 a_3 \sqrt{2gh_3}] \\ \dot{h}_4 = \frac{1}{A_4(h_4)} [(1 - \alpha_1) k_1 u_1 - C_4 a_4 \sqrt{2gh_4}] \end{cases} \quad (6.91)$$

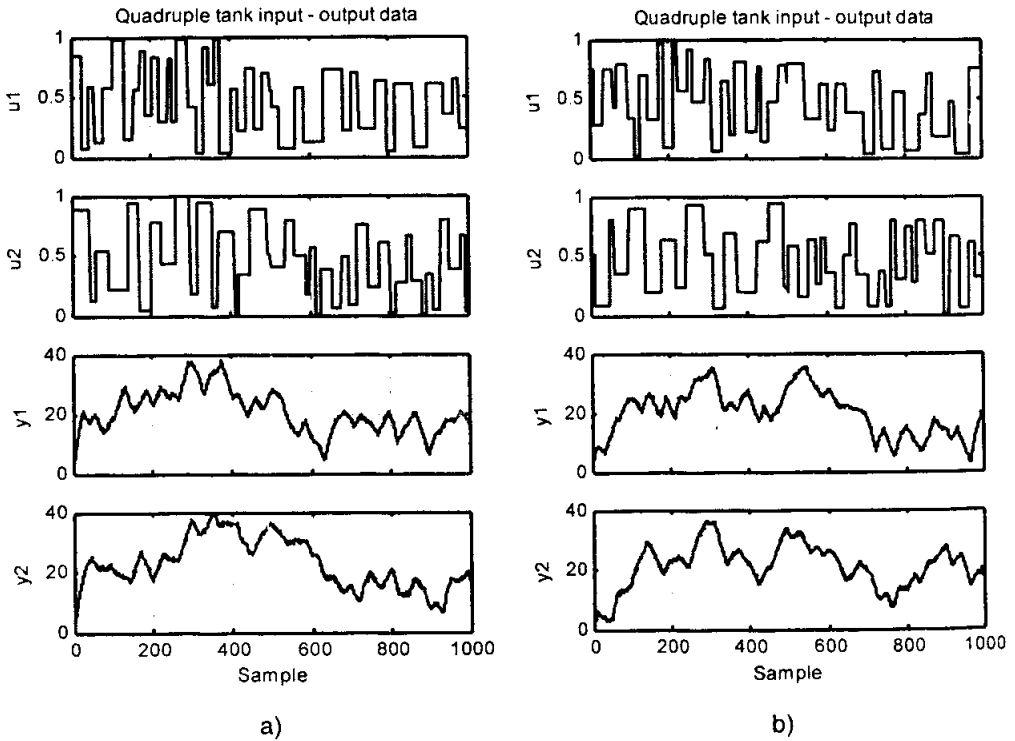
Tiết diện ngang của các bồn chứa là hình chữ nhật có chiều rộng w cố định, chiều dài thay đổi theo độ cao mực chất lỏng chứa trong bồn:

$$A_i(h_i) = w \left[\frac{(l_{\max} - l_{\min})}{h_{\max}} h_i + l_{\min} \right] \quad (i = \overline{1,4}) \quad (6.92)$$

Thông số hệ bốn bồn chứa liên kết mô phỏng cho trong bảng 6.3. Mô phỏng hệ thống dùng Matlab, dữ liệu vào ra của hệ thống dùng để nhận dạng và đánh giá mô hình được trình bày ở hình 6.28.

Bảng 6.3 Thông số vật lý hệ bốn bồn liên kết

Thông số	Giá trị	Đơn vị
Chiều rộng của tiết diện ngang (w)	9	cm
Chiều dài lớn nhất của tiết diện ngang ($l_{\max}$)	18	cm
Chiều dài nhỏ nhất của tiết diện ngang ($l_{\min}$)	9	cm
Chiều cao cực đại của bồn chứa ($h_{\max}$)	40	cm
Lưu lượng bơm cực đại	150	cm ³ /sec
Tín hiệu điều khiển máy bơm (u_1, u_2)	0 ÷ 1	—
Tiết diện van xả bồn 1 (a_1)	0,5	cm ²
Tiết diện van xả bồn 2 (a_2)	0,4	cm ²
Tiết diện van xả bồn 3 (a_3)	0,7	cm ²
Tiết diện van xả bồn 4 (a_4)	0,8	cm ²
Tỷ lệ phân lưu lượng máy bơm 1 (α_1)	0,8	—
Tỷ lệ phân lưu lượng máy bơm 2 (α_2)	0,7	—
Hệ số xả (C_1, C_2, C_3, C_4)	0,8	—



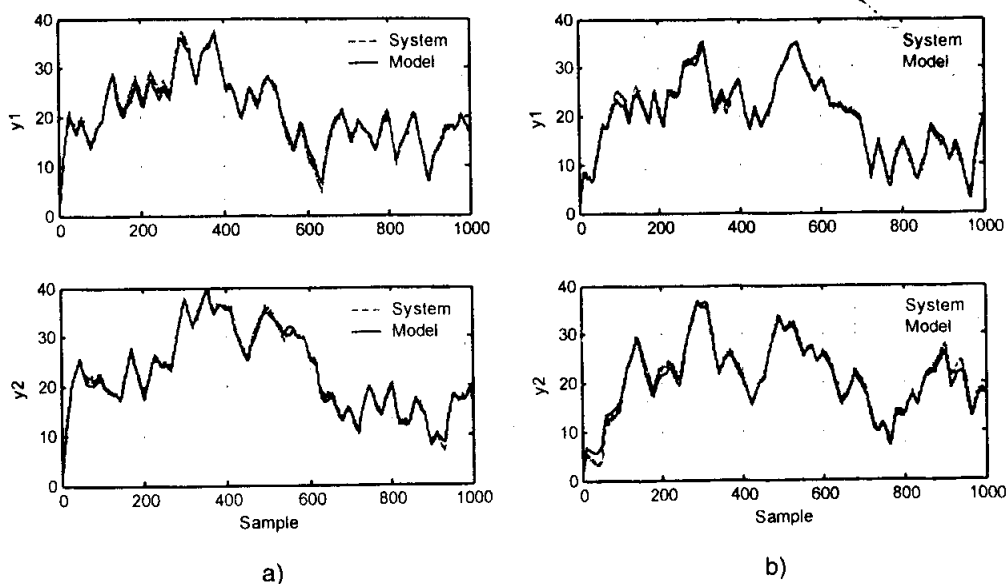
Hình 6.28 Dữ liệu vào–ra của hệ bốn bồn chứa liên kết
a) Dữ liệu nhận dạng mô hình; b) Dữ liệu đánh giá mô hình

Cấu trúc mạng thần kinh mô hình hóa hệ 4 bồn liên kết

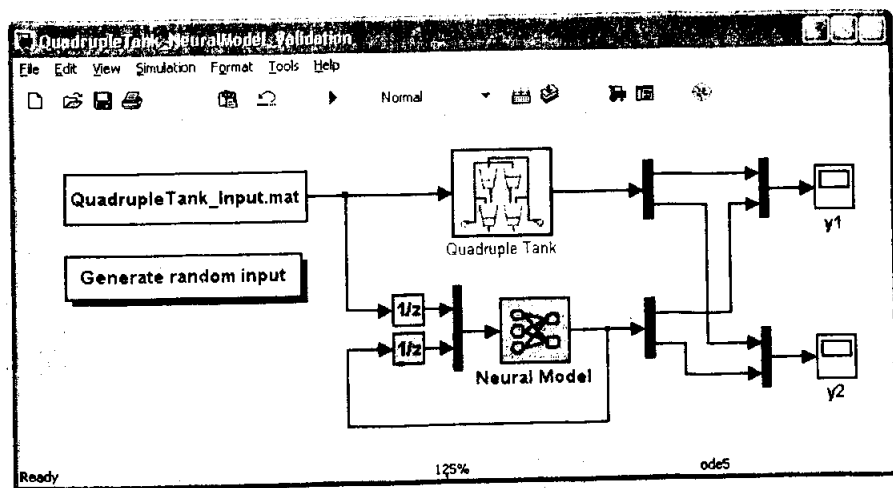
- 4 ngõ vào: $u_1(t-1)$, $u_2(t-1)$, $y_1(t-1)$, $y_2(t-1)$
- 2 ngõ vào: $y_1(t)$, $y_2(t)$
- Số tế bào thần kinh ở lớp ẩn: 6
- Hàm kích hoạt ở lớp ẩn: Sigmoid
- Hàm kích hoạt ở lớp ra: tuyến tính

Kết quả nhận dạng

Hình 6.29 là đồ thị trình bày kết quả mô phỏng ngõ ra của mạng sau khi đã huấn luyện dựa vào dữ liệu đã thu thập ở hình 6.28a. Kết quả đánh giá chéo cho thấy độ phù hợp của mô hình là 98%. Hình 6.30 là sơ đồ Simulink để đánh giá mô hình mạng thần kinh với tín hiệu vào bất kỳ. Kết quả mô phỏng cho thấy ngõ ra của mạng sai lệch so với ngõ ra của hệ bốn bồn liên kết không đáng kể.



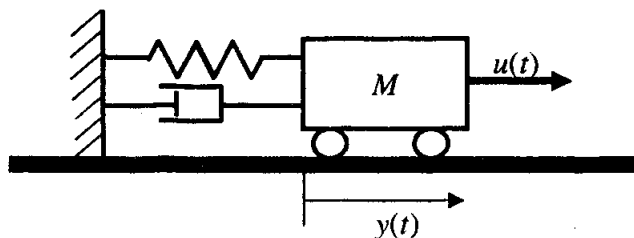
Hình 6.29 Đồ thị đánh giá mô hình
a) Phân tích thặng dư; b) Đánh giá chéo



Hình 6.30 Sơ đồ Simulink đánh giá mô hình mạng thần kinh
với tín hiệu vào ngẫu nhiên

6.4.2 Nhận dạng dùng mạng hàm cơ sở xuyên tâm (RBF)

Mạng hàm cơ sở xuyên tâm RBF là một trường hợp riêng của mạng thần kinh, do đó cơ sở lý thuyết trình bày ở mục trên cũng có thể áp dụng để giải bài toán nhận dạng hệ thống dùng mạng RBF. Sau đây là một ví dụ.

Ví dụ 6.6 Nhận dạng hệ lò xo**Hình 6.31** Hệ lò xo

Cho hệ thống ở hình 6.31. Bỏ qua khối lượng của lò xo và khâu giảm chấn, quan hệ vào ra của hệ thống có thể mô tả bởi phương trình vi phân phi tuyến sau:

$$M\ddot{y}(t) + b\dot{y}(t)|\dot{y}(t)| + k_1y(t) + k_2y^3(t) = u(t) \quad (6.93)$$

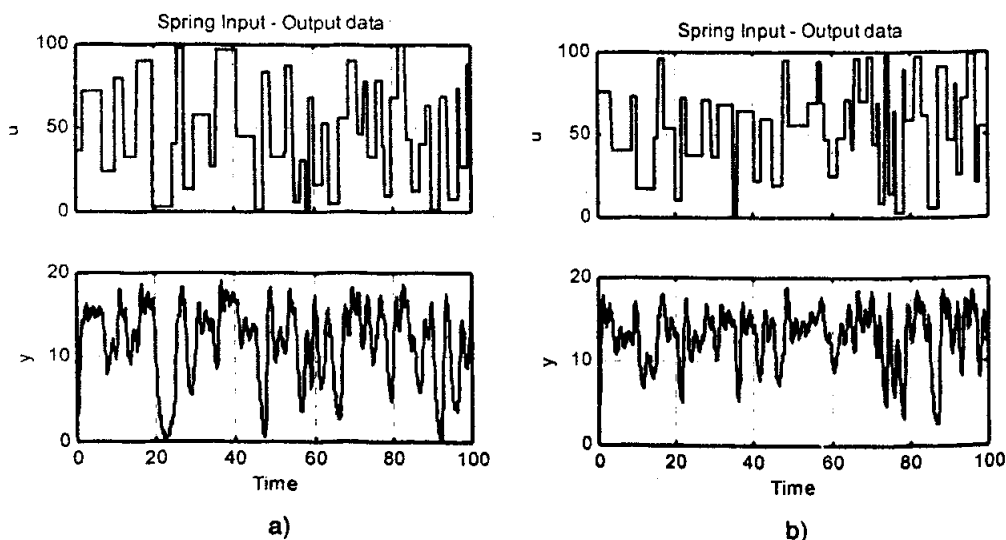
trong đó: $u(t)$ là lực kéo (tín hiệu vào)

$y(t)$ là vị trí xe (tín hiệu ra)

$M = 1kg$: khối lượng xe

$b = 0,2$: hệ số ma sát nhớt

$k_1 = 0,5$; $k_2 = 0,5$: độ cứng của lò xo

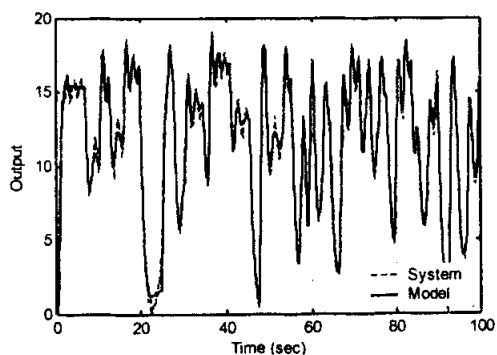
**Hình 6.32** Hai tập dữ liệu vào ra của hệ lò xo

a) Dữ liệu nhập dạng mô hình; b) Dữ liệu đánh giá mô hình

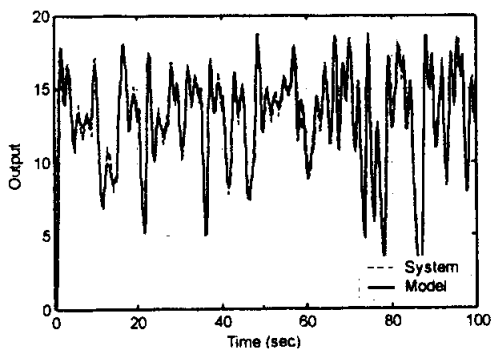
Trong ví dụ này ta nhận dạng hệ lò xo dùng mạng RBF. Tần số lấy mẫu là 0.1 sec, hai tập dữ liệu vào-ra của hệ lò xo dùng để huấn luyện mạng và đánh giá chất lượng mô hình được trình bày ở hình 6.32. Hệ lò xo mô tả bởi phương trình (6.93) là hệ phi tuyến dao động suy giảm chậm, qua quá trình thử sai tăng dần mức độ phức tạp của mạng RBF, ta chọn cấu hình mạng RBF để nhận dạng hệ lò xo như sau:

- 8 ngõ vào: $u(t-1), u(t-2), u(t-3), y(t-1), y(t-2) \dots, y(t-5)$
- 1 ngõ vào: $y(t)$
- Số hàm cơ sở xuyên tâm: 100

Hình 6.33 là kết quả đánh giá mô hình sau khi nhận dạng, độ phù hợp của mô hình khi phân tích thặng dư và đánh giá chéo tương ứng là 98,97% và 94,22%.



a)



b)

Hình 6.33 Đánh giá mô hình mạng RBF nhận dạng hệ lò xo

a) Phân tích thặng dư; b) Đánh giá chéo

6.5 ƯỚC LƯỢNG THÔNG SỐ MÔ HÌNH PHI TUYẾN DÙNG GA

GA là một công cụ tìm lời giải tối ưu đa năng, có thể áp dụng để tìm cực trị toàn cục của hàm mục tiêu có nhiều dạng khác nhau. Trong lĩnh vực nhận dạng hệ thống, GA đặc biệt hữu ích khi tiêu chuẩn ước lượng không phải dạng toàn phương. Tổng quát tiêu chuẩn ước lượng thông số có dạng:

$$V_N(\theta, Z^N) = \frac{1}{N} \sum_{t=1}^N \ell(\varepsilon(t, \theta)) \quad (6.94)$$

Trong công thức trên $l(.)$ là hàm vô hướng xác định dương. Ngoài chuẩn bình phương (công thức (6.25)), một số chuẩn khác đôi khi cũng được dùng:

- Chuẩn L_1 :

$$V_N(\theta, Z^N) = \frac{1}{N} \sum_{t=1}^N |\varepsilon(t, \theta)| \quad (6.95)$$

- Chuẩn L_∞ :

$$V_N(\theta, Z^N) = \max_{1 \leq t \leq N} |\varepsilon(t, \theta)| \quad (6.96)$$

GA ước lượng thông số mô hình phi tuyến

Để sử dụng GA giải bài toán ước lượng thông số mô hình, có hai vấn đề cần phải quan tâm là phải mã hóa lời giải bài toán như thế nào và hàm thích nghi là hàm gì.

- Mã hóa: có thể sử dụng cách mã hóa nhị phân hoặc thập phân, tuy nhiên thuận lợi nhất là cách mã hóa số thực với mỗi thông số cần ước lượng tương ứng với một gen trên chuỗi nhiễm sắc thể.

θ_1	θ_2	θ_3	...	θ_d
------------	------------	------------	-----	------------

Hình 6.34 Chuỗi NST mã số thực mã hóa lời giải bài toán ước lượng thông số

- Hàm thích nghi: chính là tiêu chuẩn ước lượng thông số. GA sẽ tìm vector thông số sao cho tối thiểu hóa hàm thích nghi.

$$fitness = V_N(\theta, Z^N) \quad (6.97)$$

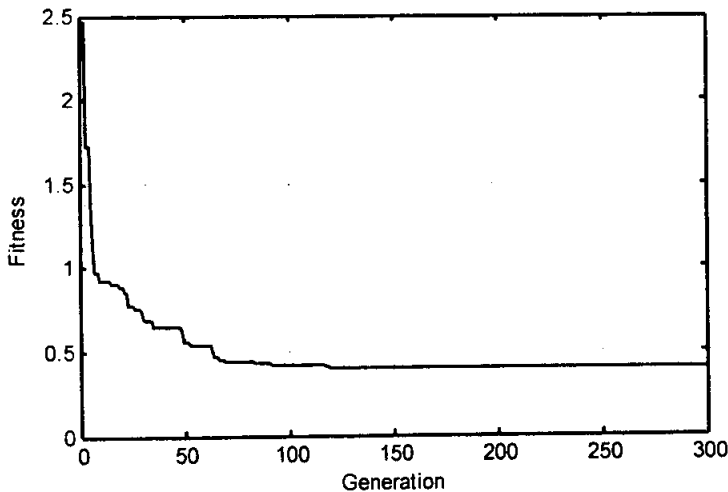
Sau khi đã mã hóa lời giải và định nghĩa được hàm thích nghi, có thể sử dụng GA với các phép toán di truyền chuẩn (tương ứng với cách mã hóa) để ước lượng thông số mô hình.

Ví dụ 6.7 Nhận dạng hệ bốn đơn dùng mô hình mờ ở ví dụ 6.1 theo tiêu chuẩn ước lượng thông số L_1 (6.95). Sử dụng GA mã số thực (xem chương 4) với các tùy chọn như sau:

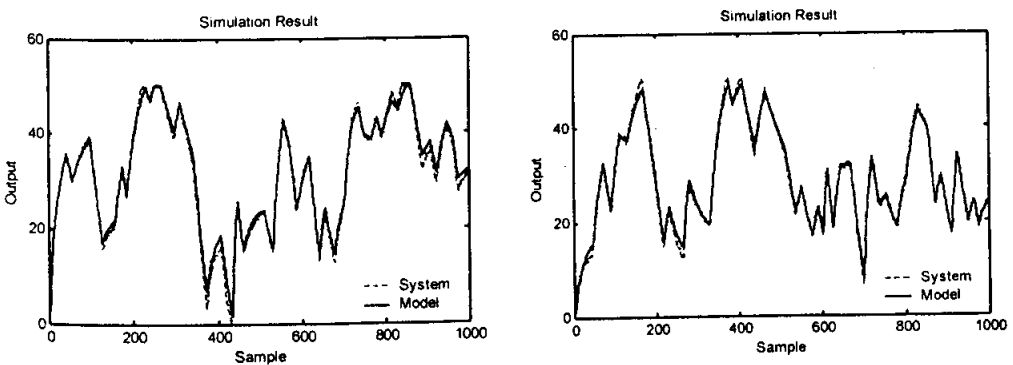
- Kích thước quần thể: $N = 20$, thế hệ đầu tiên khởi động ngẫu nhiên.

- Chọn lọc sắp hạng tuyến tính với thông số $\eta = 0,2$.
- Lai ghép BLX- α , với mức ngưỡng $\alpha = 0,5$, xác suất lai ghép $p_c = 0,9$.
- Đột biến ngẫu nhiên, xác suất đột biến $p_m = 0,1$.
- Lưu giữ cá thể tốt nhất trong mỗi thế hệ ($Elitism = 1$)

Hình 6.35 cho thấy hàm thích nghi (tức sai số ước lượng theo công thức (6.95)) giảm dần đến cực tiểu trong quá trình thực thi GA. Hình 6.36 là đồ thị đánh giá mô hình, độ phù hợp giữa mô hình và hệ thống là 99,19%.



Hình 6.35 Hàm thích nghi bài toán nhận dạng mô hình mờ theo chuẩn L_1 dùng GA



Hình 6.36 Đánh giá mô hình mờ nhận dạng dùng GA

6.6 TÓM TẮT

Sau phần giới thiệu tổng quát về vấn đề nhận dạng mô hình hộp đen phi tuyến, nội dung chính của chương 6 trình bày cụ thể phương pháp nhận dạng hệ phi tuyến dùng mô hình mờ và mạng thần kinh kèm theo các ví dụ minh họa. Mỗi phương pháp nhận dạng đã trình bày trong chương này đều có những ưu điểm riêng. Phương pháp sử dụng mô hình mờ Mamdani rất phù hợp để nhận dạng các đối tượng phi tuyến mà quan hệ vào ra của đối tượng có thể mô tả định tính bằng các phát biểu ngôn ngữ. Mô hình mờ Takagi-Sugeno phù hợp trong trường hợp đối tượng có thể mô hình hóa bằng cách nội suy phi tuyến các mô hình tuyến tính cục bộ quanh các điểm làm việc tĩnh, quy luật nội suy phi tuyến xác định bởi hệ quy tắc mờ và cách chọn hàm liên thuộc của các tập mờ ở mệnh đề điều kiện. Nếu chỉ đơn thuần nhận dạng hệ phi tuyến dựa vào dữ liệu vào-ra, và đặc biệt là các hệ MIMO trong đó ảnh hưởng qua lại giữa các ngõ vào và ngõ ra rất khó nhận xét định tính, thì giải pháp sử dụng mạng thần kinh có thể nói là dễ dàng và hiệu quả nhất. Thực nghiệm cho thấy mạng thần kinh có thể mô hình hóa rất chính xác nhiều lớp hệ phi tuyến khác nhau do mạng thần kinh có độ kinh hoạt cao hơn mô hình mờ. Cuối chương 6 trình bày ứng dụng GA ước lượng thông số mô hình phi tuyến theo các tiêu chuẩn ước lượng không toàn phương. Các mô hình đã nhận dạng được trong chương 6 có thể sử dụng trong các thuật toán điều khiển hệ phi tuyến dựa vào mô hình, vấn đề này sẽ được trình bày ở chương 7.

Chương 7

ĐIỀU KHIỂN DỰA VÀO MÔ HÌNH

7.1 GIỚI THIỆU

Điều khiển dựa vào mô hình là phương pháp điều khiển trong đó có sử dụng tường minh mô hình của đối tượng để tính toán tín hiệu điều khiển. Cần phân biệt với phương pháp điều khiển thông thường (như các bộ điều khiển sớm trễ pha hay PID kinh điển chẳng hạn) chỉ sử dụng mô hình của đối tượng khi thiết kế bộ điều khiển chứ không sử dụng mô hình khi hệ thống điều khiển đang vận hành.

Có hai loại mô hình là mô hình thuận và mô hình ngược. Hiện nay, bốn phương pháp điều khiển dựa vào mô hình được sử dụng phổ biến là:

- Điều khiển dùng mô hình ngược (*Inverse Control*)
- Điều khiển mô hình nội (*Internal Model Control – IMC*)
- Điều khiển theo mô hình chuẩn (*Model Reference Control – MRC*)
- Điều khiển dự báo dựa vào mô hình (*Model-Based Predictive Control – MPC*).

Các phương pháp điều khiển dựa vào mô hình được đưa ra từ đầu thập niên 1980 với các công trình mở đường nghiên cứu các sơ đồ điều khiển dựa vào mô hình tuyến tính. Đến thập niên 1990, khi các phương pháp nhận dạng mô hình phi tuyến (dựa vào mạng thần kinh và mô hình mờ) đã phát triển mạnh cả về lý thuyết lẫn thực nghiệm thì các phương pháp điều khiển dựa vào mô hình được mở rộng để áp dụng vào hệ phi tuyến. Đến nay, các phương pháp điều khiển dựa vào mô hình phi tuyến đã áp dụng thành công vào nhiều lĩnh vực, nhưng lĩnh vực áp dụng chủ yếu là các quá trình công nghệ hóa học (ví dụ như trong công nghiệp hóa dầu, chế biến thực phẩm, nước giải khát...).

Chương này sẽ trình bày các vấn đề cơ bản liên quan đến các sơ đồ điều khiển dựa vào mô hình phi tuyến, chủ yếu là mô tả sơ đồ, trình bày thuật toán điều khiển và ví dụ áp dụng mà không đi sâu vào vấn đề phân tích tính ổn định. Đây cũng là cách trình bày phổ biến của đa số các công trình đã công bố về điều khiển dựa vào mô hình. Lý do là vì vấn đề phân tích tính ổn định của các sơ đồ điều khiển dựa vào mô hình phi tuyến rất khó khăn và đến nay vẫn còn là bài toán mở.

7.2 ĐIỀU KHIỂN DÙNG MÔ HÌNH NGƯỢC

7.2.1 Điều khiển ngược trực tiếp (*direct inverse control*)

Phương pháp đơn giản nhất để thiết kế bộ điều khiển là dùng phương pháp điều khiển vòng hở, trong đó bộ điều khiển là mô hình ngược của đối tượng. Giả sử đối tượng phi tuyến được mô tả bởi phương trình sai phân:

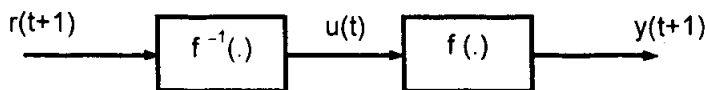
$$y(t) = f(y(t-1), \dots, y(t-n), u(t-1), \dots, u(t-m)) \quad (7.1)$$

$$\Leftrightarrow y(t+1) = f(y(t), \dots, y(t-n+1), u(t), \dots, u(t-m+1)) \quad (7.2)$$

Nếu sử dụng luật điều khiển:

$$u(t) = f^{-1}(r(t+1), r(t), \dots, r(t-n+1), u(t-1), \dots, u(t-m+1)) \quad (7.3)$$

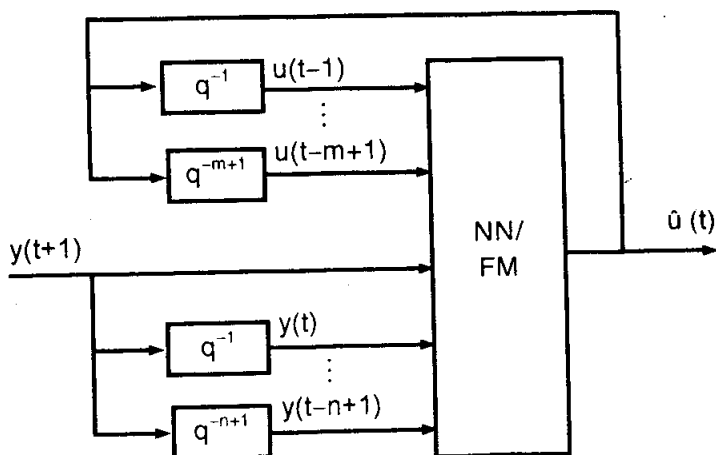
ta có sơ đồ điều khiển ngược trực tiếp (H.7.1). Dễ thấy rằng tín hiệu ra $y(t)$ của đối tượng đồng nhất bằng tín hiệu đặt $r(t)$.



Hình 7.1 Nguyên tắc điều khiển ngược trực tiếp

Mặc dù nguyên tắc điều khiển ngược trực tiếp rất đơn giản nhưng chiến lược này hầu như không thể thực hiện được trong các ứng dụng thực tế, đặc biệt là trong trường hợp đối tượng điều khiển là hệ phi tuyến, do không thể rút ra được mô hình ngược giải tích (toán học) của đối tượng. Một giải pháp được đề xuất để giải quyết vấn đề trên là sử dụng mô hình mờ (FM–*Fuzzy Model*) hoặc mạng thần kinh (NN–*Neural Network*) để nhận dạng đặc tính động học ngược của đối tượng (H.7.2):

$$\hat{u}(t, \theta) = f^{-1}(y(t+1), y(t), \dots, y(t-n+1), u(t-1), \dots, u(t-m+1)) \quad (7.4)$$



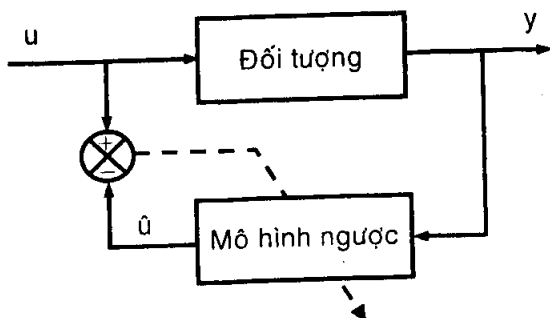
Hình 7.2 Mô hình ngược

Có hai cách ước lượng thông số của mô hình ngược:

Ước lượng off-line: thông số mô hình ngược được ước lượng dựa vào tập dữ liệu vào ra của đối tượng (đã thu thập trước) sao cho sai lệch giữa tín hiệu $u(t)$ kích thích ở đầu vào của đối tượng và tín hiệu ra $\hat{u}(t, \theta)$ của mô hình ngược là tối thiểu (H.7.3).

$$J(\theta) = \sum_{t=1}^N [u(t) - \hat{u}(t, \theta)]^2 \rightarrow \min \quad (7.5)$$

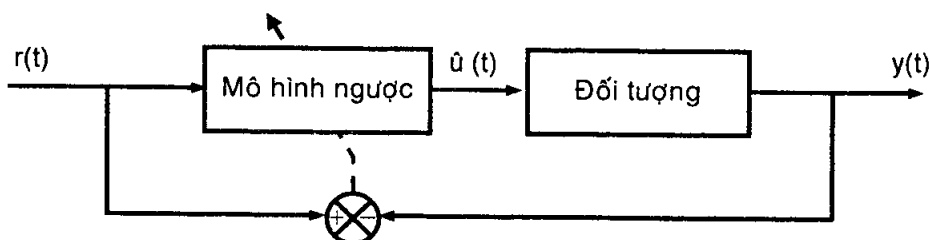
Thuật toán ước lượng thông số off-line khá đơn giản, có thể sử dụng các phiên bản của thuật toán Newton đã trình bày ở chương 6. Nếu mô hình ngược là mạng thần kinh thì có thể sử dụng các thuật toán huấn luyện mạng off-line, như thuật toán lan truyền ngược.



Hình 7.3 Nguyên tắc ước lượng thông số mô hình ngược off-line

Ước lượng on-line: thông số mô hình ngược được cập nhật trực tuyến sao cho sai lệch giữa tín hiệu ra của đối tượng và tín hiệu đặt là tối thiểu:

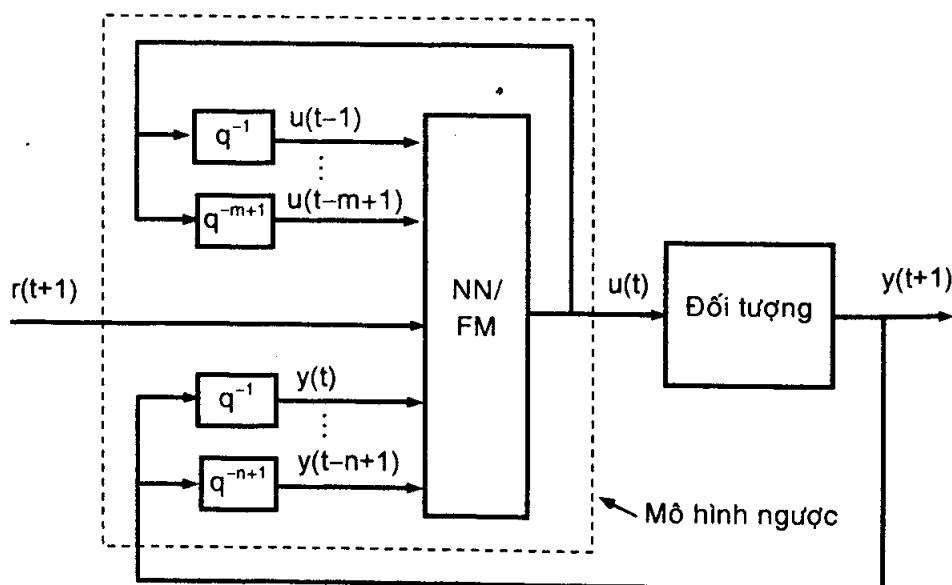
$$J(\theta) = \sum_t [r(t) - y(t)]^2 \rightarrow \min \quad (7.6)$$



Hình 7.4 Nguyên tắc ước lượng thông số mô hình ngược on-line

Nếu mô hình ngược được ước lượng on-line ta có sơ đồ điều khiển ngược thích nghi trực tiếp. Thuật toán ước lượng thông số on-line phức tạp hơn nhiều so với thuật toán ước lượng off-line.

Mô hình ngược sau khi nhận dạng sẽ được sử dụng như bộ điều khiển bằng cách thay thế tín hiệu ra $y(t+1)$ bằng tín hiệu vào chuẩn $r(t+1)$, xem hình 7.5.

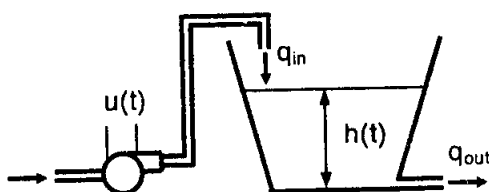


Hình 7.5 Điều khiển dùng mô hình ngược

Chú ý:

- Phương pháp điều khiển dùng mô hình ngược chỉ có thể áp dụng khi đối tượng cần điều khiển là ổn định với pha cực tiểu.
- Nếu mô hình ngược không nhận dạng đúng đặc tính động học ngược của đối tượng thì kết quả điều khiển sẽ có sai số. Giải pháp để loại trừ ảnh hưởng của sai số mô hình là dùng sơ đồ điều khiển thuận (*feed forward control*) hoặc điều khiển mô hình nội (*internal model control*).

Ví dụ 7.1 Điều khiển hệ bồn chứa dùng mô hình ngược



Hình 7.6 Hệ bồn chứa chất lỏng

Cho hệ bồn chứa có mô tả toán học trình bày ở ví dụ 5.5. Yêu cầu đặt ra là điều khiển mực chất lỏng trong bồn bám theo tín hiệu đặt. Ở ví dụ này ta sẽ điều khiển hệ thống dùng sơ đồ điều khiển ngược trực tiếp (H.7.5).

Nhận dạng đặc tính động học ngược của hệ bồn đơn

Từ các phương trình (5.22) và (5.23) ta có thể suy ra rằng:

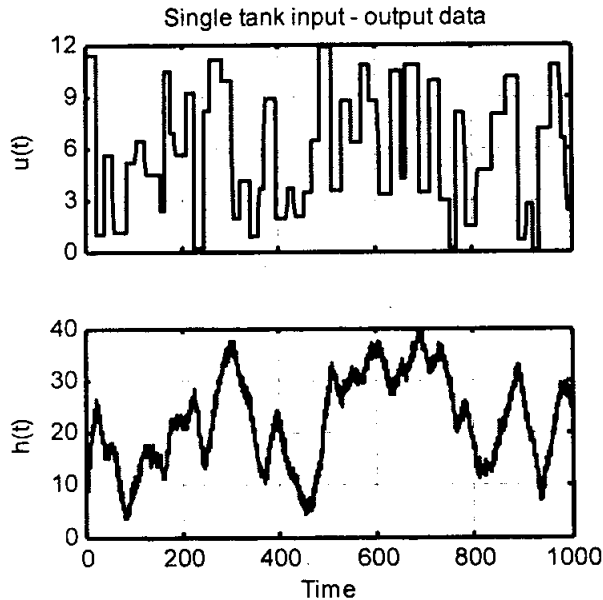
$$h(t+1) = f(h(t), u(t)) \quad (7.7)$$

Do đó dễ thấy:

$$\hat{u}(t) = \hat{f}^{-1}(h(t+1), h(t)) \quad (7.8)$$

Sử dụng mạng thần kinh để nhận dạng mô hình ngược (7.8). Dữ liệu vào ra dùng để nhận dạng mô hình ngược được trình bày ở hình 7.7. Cấu trúc mạng thần kinh như sau:

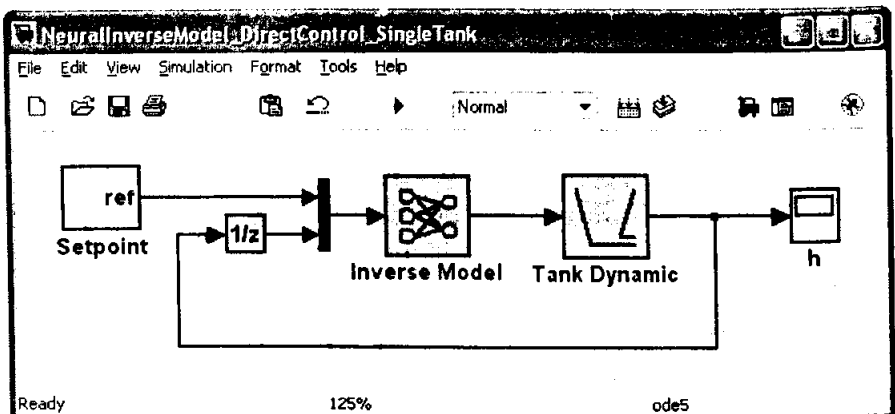
- 2 ngõ vào là $h(t+1)$, $h(t)$
- 1 ngõ ra là $\hat{u}(t)$
- Số tế bào thần kinh ở lớp ẩn là 6
- Hàm kích hoạt ở lớp ẩn là hàm sigmoid; ở lớp ra là hàm tuyến tính.



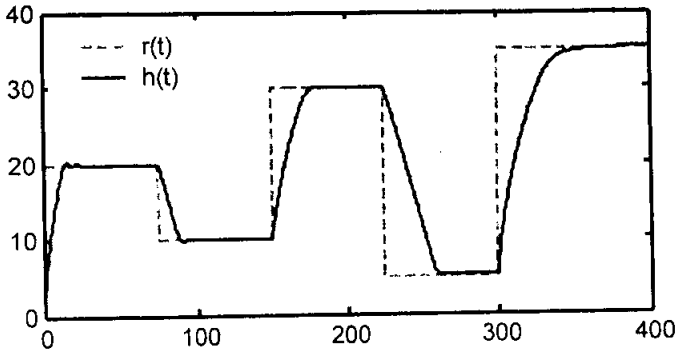
Hình 7.7 Dữ liệu vào-ra hệ bồn đơn dùng để nhận dạng mô hình ngược

Kết quả điều khiển trực tiếp dùng mô hình ngược

Mạng thần kinh sau khi huấn luyện xấp xỉ đặc tính động học ngược của bồn chứa được sử dụng để điều khiển bồn chứa theo sơ đồ ở hình 7.8. Kết quả mô phỏng ở hình 7.9 cho thấy mực chất lỏng trong bồn chứa thay đổi theo tín hiệu vào dạng hàm nấc khá tốt. Tuy nhiên đáp ứng của hệ thống có sai số xác lập tại những điểm làm việc mà mạng thần kinh chưa nhận dạng được chính xác đặc tính động học ngược của hệ bồn chứa.

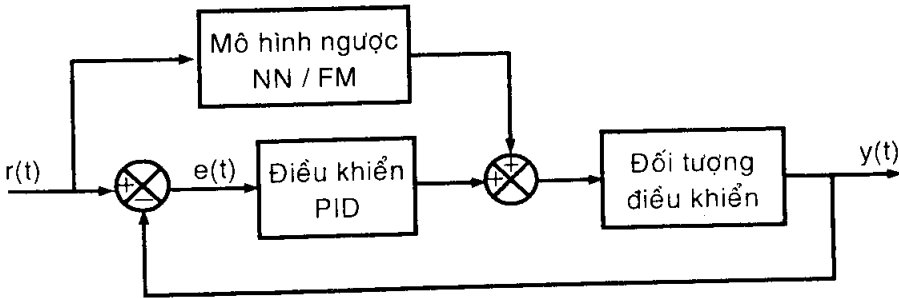


Hình 7.8 Sơ đồ điều khiển trực tiếp hệ bồn đơn dùng mô hình ngược



Hình 7.9 Kết quả điều khiển hệ bồn đơn dùng mô hình ngược trực tiếp

7.2.2 Điều khiển thuận (forward control)



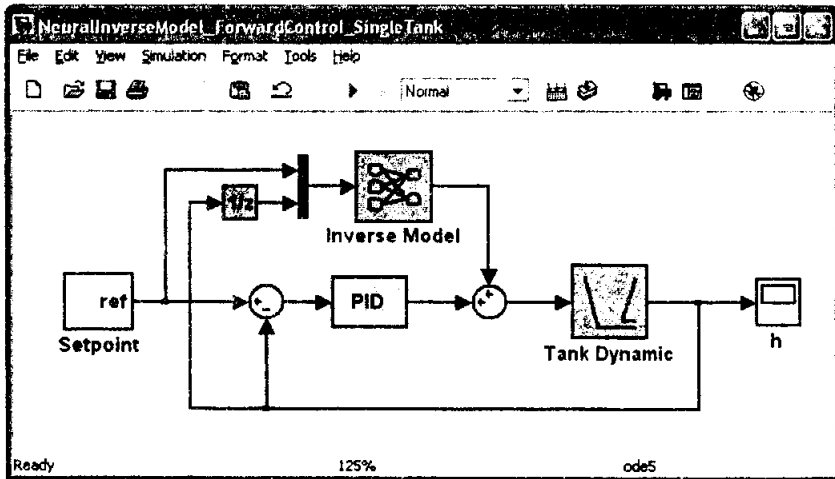
Hình 7.10 Điều khiển thuận

Trong trường hợp mô hình NN/FM không thể nhận dạng hoàn toàn chính xác đặc tính động học ngược của đối tượng, điều này thường xảy ra thực tế, có thể sử dụng thêm bộ điều khiển PID kinh điển để triệt tiêu sai số do mô hình theo sơ đồ trình bày ở hình 7.10. Sau đây là một ví dụ.

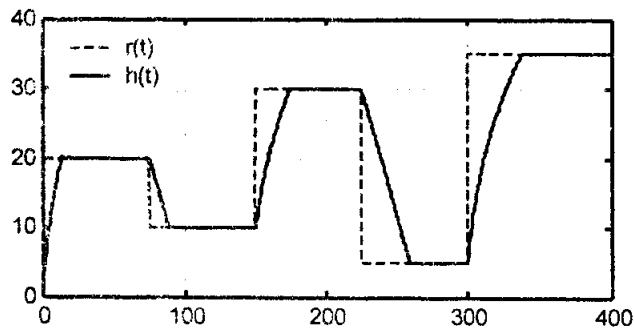
Ví dụ 7.2 Điều khiển thuận hệ bồn chứa dùng mô hình ngược

Trong ví dụ này chúng ta sử dụng mô hình ngược của hệ bồn đơn đã nhận dạng ở ví dụ 7.1 làm bộ điều khiển thuận kết hợp với bộ điều khiển hồi tiếp PID để bù sai số mô hình (xem H.7.11). Do đối tượng bồn đơn là hệ phi tuyến bậc nhất nên cụ thể trong trường hợp này chỉ cần sử dụng bộ điều khiển PI là đủ.

Thông số bộ điều khiển PI được chọn là $K_P = 10$, $K_I = 0,01$. Kết quả điều khiển ở hình 7.12 cho thấy đáp ứng của hệ thống hoàn toàn bám theo tín hiệu đặt tại mọi điểm làm việc, không còn sai số xác lập như kết quả điều khiển ở hình 7.9.



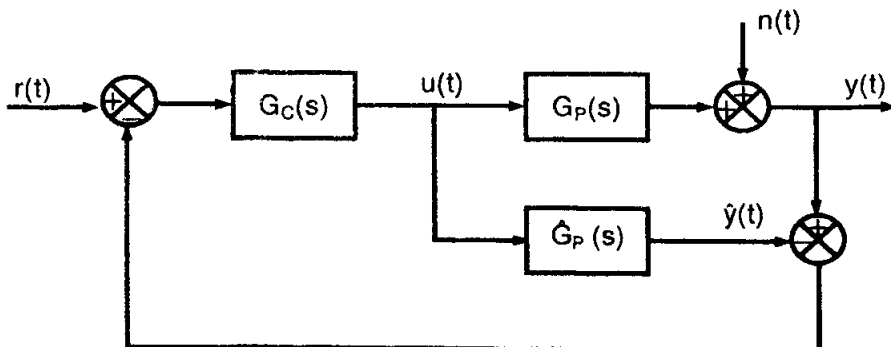
Hình 7.11 Sơ đồ điều khiển thuận hệ bồn đơn dùng mô hình ngược



Hình 7.12 Kết quả điều khiển thuận hệ bồn đơn dùng mô hình ngược

7.3 ĐIỀU KHIỂN MÔ HÌNH NỘI

7.3.1 Nguyên tắc điều khiển mô hình nội (Internal Model Control-IMC)



Hình 7.13 Sơ đồ khối điều khiển mô hình nội

Để khắc phục các khuyết điểm của phương pháp điều khiển vòng hở ở mục trên (điều khiển dùng mô hình ngược), người ta đưa ra một sơ đồ điều khiển vòng kín gọi là điều khiển mô hình nội. Để đơn giản, trước tiên xét hệ thống điều khiển mô hình nội tuyến tính có sơ đồ khối ở hình 7.13. Trong sơ đồ, $G_P(s)$ là đối tượng điều khiển, $\hat{G}_P(s)$ là mô hình của đối tượng, $G_C(s)$ là bộ điều khiển, $n(t)$ là nhiễu ảnh hưởng đến đối tượng.

Tín hiệu ra của hệ thống ở hình 7.13 là:

$$Y(s) = R(s)G_R(s) + N(s)G_N(s) \quad (7.9)$$

trong đó:

$$G_R(s) = \left. \frac{Y(s)}{R(s)} \right|_{N(s)=0} = \frac{G_C(s)G_P(s)}{1 + G_C(s)G_P(s) - G_C(s)\hat{G}_P(s)} \quad (7.10)$$

$$G_N(s) = \left. \frac{Y(s)}{N(s)} \right|_{R(s)=0} = \frac{1 - G_C(s)\hat{G}_P(s)}{1 + G_C(s)G_P(s) - G_C(s)\hat{G}_P(s)} \quad (7.11)$$

Nếu mô hình $\hat{G}_P(s)$ ổn định và không có zero nằm bên phải mặt phẳng phức, có thể chọn bộ điều khiển như sau:

$$G_C(s) = \hat{G}_P^{-1}(s) \quad (7.12)$$

Thay (7.12) vào (7.10) và (7.11) ta được:

$$G_R(s) = 1 \quad (7.13)$$

$$G_N(s) = 0 \quad (7.14)$$

Do đó suy ra:

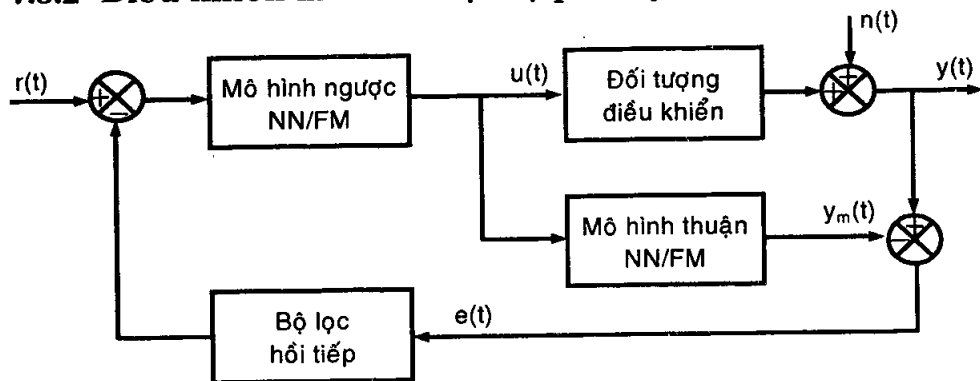
$$Y(s) = R(s) \quad (7.15)$$

Rõ ràng, chỉ cần chọn bộ điều khiển là nghịch đảo của mô hình thuận $G_C(s) = \hat{G}_P^{-1}(s)$ thì tín hiệu ra của hệ thống sẽ đồng nhất bằng tín hiệu đặt, bất chấp mô hình thuận không mô tả chính xác hoàn toàn đặc tính động của đối tượng ($G_P(s) \neq \hat{G}_P(s)$) và hệ thống có nhiễu ($n(t) \neq 0$).

Trong trường hợp bộ điều khiển $G_C(s) \neq \hat{G}_P^{-1}(s)$, thì sai số xác lập của hệ thống đối với tín hiệu vào hàm bậc sẽ bằng 0 nếu:

$$\lim_{s \rightarrow 0} G_C(s)\hat{G}_P(s) = 1 \quad (7.16)$$

7.3.2 Điều khiển mô hình nội hệ phi tuyến



Hình 7.14 Sơ đồ khối hệ thống điều khiển mô hình nội hệ phi tuyến dùng NN/FM

Sơ đồ điều khiển mô hình nội phi tuyến (H.7.14) được đưa ra dựa trên sơ đồ điều khiển mô hình nội tuyến tính. Rõ ràng khi mô hình mô tả chính xác đặc tính động của hệ thống và không có nhiễu thì tín hiệu hồi tiếp bằng 0, khi đó hệ thống trên trở thành hệ thống điều khiển vòng hở. Nếu có nhiễu $n(t)$ ảnh hưởng đến quá trình điều khiển thì tín hiệu phản hồi sẽ bằng $n(t)$ và không gây ra ảnh hưởng đến tác động điều khiển, nhiễu $n(t)$ bị trừ khỏi tín hiệu chuẩn. Do đó ảnh hưởng của nhiễu đo lường ở ngõ ra của quá trình điều khiển hoàn toàn bị triệt tiêu. Bộ lọc được sử dụng để tăng độ bền vững của hệ thống đối với sai số mô hình ở miền tần số cao và nhiễu đo lường.

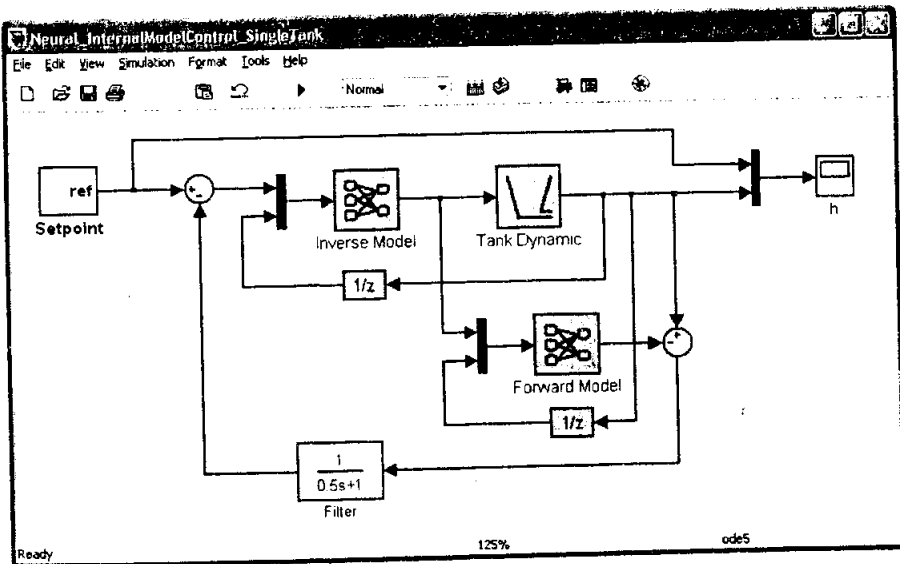
Hệ thống điều khiển mô hình nội phi tuyến có các tính chất sau:

- Nếu đối tượng điều khiển và mô hình ngược ổn định và nếu mô hình thuận mô tả chính xác đặc tính động học của đối tượng thì hệ thống điều khiển ổn định.
- Nếu đối tượng điều khiển và mô hình ngược ổn định và nếu mô hình thuận mô tả chính xác đặc tính động học của đối tượng, mô hình ngược mô tả chính xác đặc tính động học ngược của đối tượng và nếu không có nhiễu thì hệ thống tín hiệu ra của đối tượng đồng nhất bằng tín hiệu đặt.
- Nếu mô hình ngược có độ lợi tĩnh bằng nghịch đảo độ lợi tĩnh của mô hình thuận và nếu đối tượng ổn định thì hệ thống điều khiển vô sai ở trạng thái xác lập với tín hiệu vào là hàm bậc bất chấp nhiễu đo lường ở ngõ ra của đối tượng.

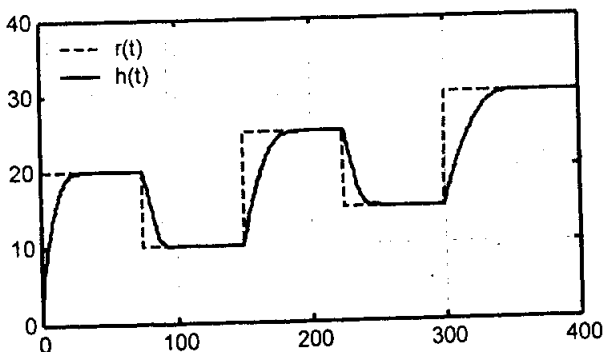
Ví dụ 7.3 Điều khiển mô hình nội hệ bồn chứa đơn

Trong ví dụ này chúng ta thiết kế hệ thống điều khiển mô hình nội bồn chứa đơn đã xét ở ví dụ 7.1. Ngoài mô hình ngược đã nhận dạng ở ví dụ 7.1, chúng ta còn nhận dạng thêm mô hình thuận từ dữ liệu thu thập được ở hình 7.7. Cấu hình mạng thần kinh nhận dạng mô hình thuận hệ bồn đơn như sau:

- Hai ngõ vào $u(t-1)$ và $h(t-1)$
- Một ngõ ra là $h(t)$
- Số tế bào thần kinh ở lớp ẩn là 6.
- Hàm kích hoạt ở lớp ẩn là hàm sigmoid; ở lớp ra là hàm tuyến tính.



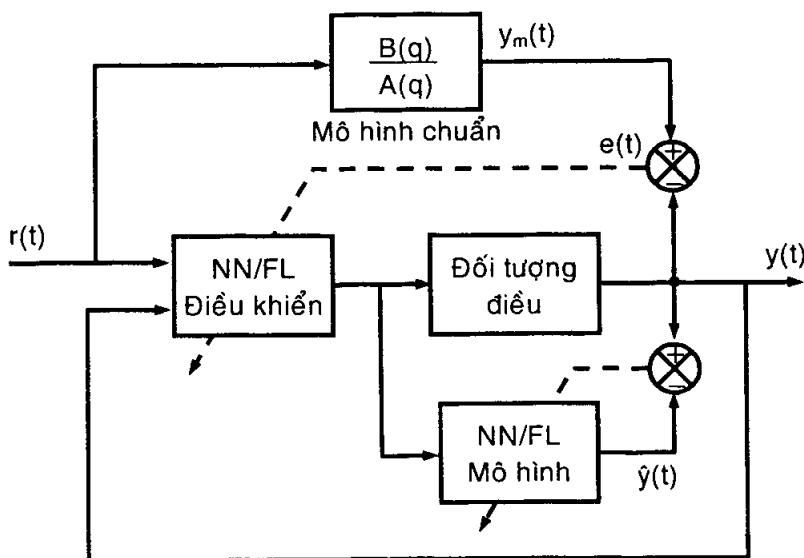
Hình 7.15 Sơ đồ mô phỏng điều khiển mô hình nội hệ bồn đơn dùng NN



Hình 7.16 Kết quả mô phỏng điều khiển mô hình nội hệ bồn đơn dùng NN

Sơ đồ mô phỏng hệ thống điều khiển mô hình nội hệ bốn đơn trình bày ở hình 7.15. Kết quả mô phỏng ở hình 7.16 cho thấy đáp ứng của hệ thống rất tốt, không có sai số xác lập.

7.4 ĐIỀU KHIỂN THEO MÔ HÌNH CHUẨN



Hình 7.17 Điều khiển theo mô hình chuẩn

Thay vì ước lượng thông số của mô hình NN/FM theo tiêu chuẩn (7.6) để được mô hình ngược của đối tượng ta có thể ước lượng thông số mô hình NN/FM sao cho sai lệch giữa tín hiệu ra của đối tượng và tín hiệu ra của mô hình chuẩn là tối thiểu (H.7.17).

$$J(\theta) = \sum_t [y_m(t) - y(t)]^2 \rightarrow \min \quad (7.17)$$

trong đó $y(t)$ là tín hiệu ra của đối tượng và $y_m(t)$ là tín hiệu ra của mô hình chuẩn:

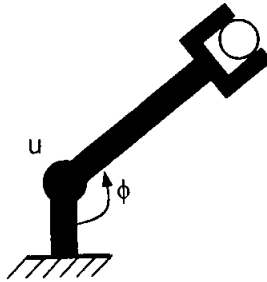
$$y_m(t) = \frac{B(q)}{A(q)} r(t) \quad (7.18)$$

Trong trường hợp mô hình chuẩn bằng 1, tức $y_m(t) = r(t)$, thì NN/FM chính là mô hình ngược của đối tượng khi lời giải bài toán tối ưu hóa (7.17) đạt cực tiểu. Thuật toán thông thường sử dụng để giải bài toán tối ưu hóa (7.17) là thuật toán suy giảm độ dốc hoặc các phiên bản của thuật toán Newton, do đó để ước lượng thông số mô

hình NN/FM theo chỉ tiêu (7.17) cần phải biết đạo hàm $\partial y(t)/\partial \theta$. Tuy nhiên do đặc tính động học của đối tượng chưa biết nên không thể tính trực tiếp $\partial y(t)/\partial \theta$ được, thay vào đó ta nhận dạng mô hình $\hat{y}(t)$ của đối tượng (dùng NN/FM) và sử dụng biểu thức gần đúng $\partial \hat{y}(t)/\partial \theta$ để ước lượng thông số.

Nếu một thuật toán ước lượng thông số trực tuyến được áp dụng để bài toán tối ưu hóa (7.17) thì ta có sơ đồ điều khiển thích nghi theo mô hình chuẩn.

Ví dụ 7.4 Điều khiển hệ tay máy một bậc tự do theo mô hình chuẩn



Hình 7.18 Hệ tay máy một bậc tự do

Cho hệ tay máy một bậc tự do có đặc tính động học mô tả bởi phương trình:

$$\ddot{\phi}(t) + 2\dot{\phi}(t) + 10 \sin \phi(t) = u(t) \quad (7.19)$$

trong đó $\phi(t)$ là vị trí của cánh tay máy và $u(t)$ là mômen do động cơ tác động lên trục quay của cánh tay.

Yêu cầu đặt ra là điều khiển cánh tay máy bám tín hiệu đặt $r(t)$ theo mô hình chuẩn:

$$G_m(s) = \frac{\Phi_m(s)}{R(s)} = \frac{9}{s^2 + 6s + 9} \quad (7.20)$$

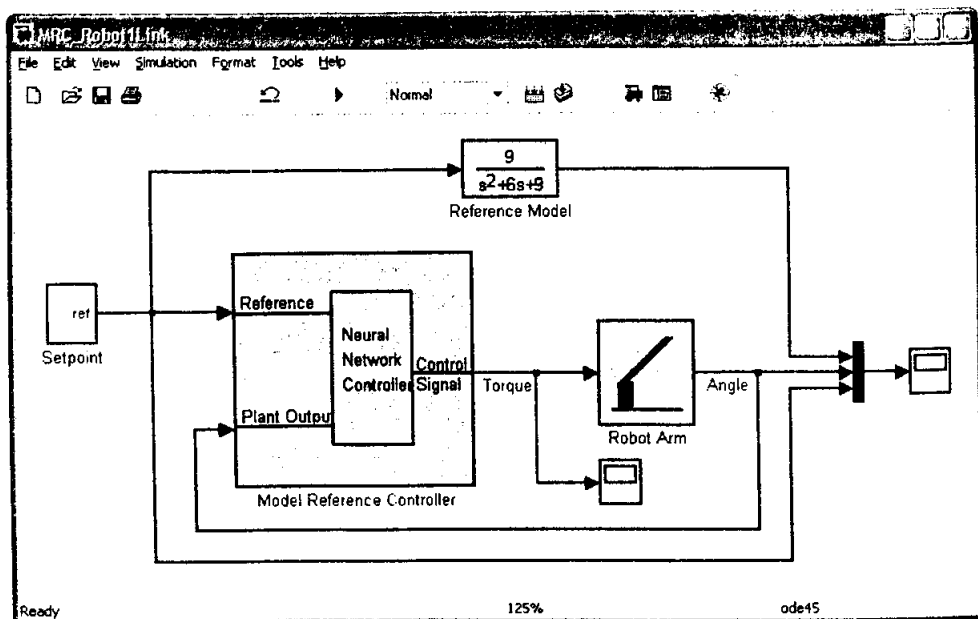
Hình 7.19 là sơ đồ Simulink mô phỏng hệ thống điều khiển cánh tay máy theo mô hình chuẩn dùng mạng thần kinh lập trình dựa theo chương trình mẫu *mrefrobotarm.mdl* của Matlab Neural Network Toolbox. Trong khối [**Model Reference Controller**] có hai mạng thần kinh:

- Một mạng thần kinh dùng để nhận dạng mô hình $\hat{\phi}(t)$ của cánh tay máy. Mạng có 4 tín hiệu vào là $\phi(t-1)$, $\phi(t-2)$, $u(t-1)$,

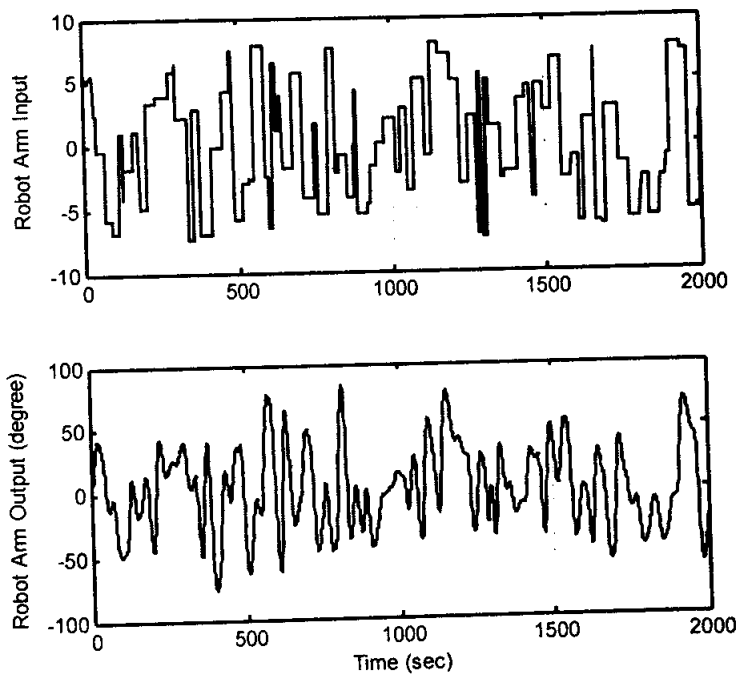
$u(t - 2)$; số tế bào thần kinh ở lớp ẩn là 10. Trong số của mạng được huấn luyện dựa vào dữ liệu vào-ra của cánh tay máy trình bày ở hình 7.20 sao cho sai lệch giữa tín hiệu ra $\phi(t)$ của cánh tay máy và tín hiệu ra $\hat{\phi}(t)$ của mạng là tối thiểu.

- Một mạng thần kinh có chức năng điều khiển. Mạng có 5 tín hiệu vào là $\phi(t - 1)$, $\phi(t - 2)$, $u(t - 1)$, $r(t - 1)$ và $r(t - 2)$. Số tế bào thần kinh ở lớp ẩn là 13. Dữ liệu huấn luyện mạng thần kinh điều khiển chính là tín hiệu vào-ra của mô hình chuẩn (H.7.21). Trọng số của mạng thần kinh điều khiển được cập nhật sao cho sai lệch giữa tín hiệu ra $\phi(t)$ của cánh tay máy và tín hiệu ra $\phi_m(t)$ của mô hình chuẩn là tối thiểu.

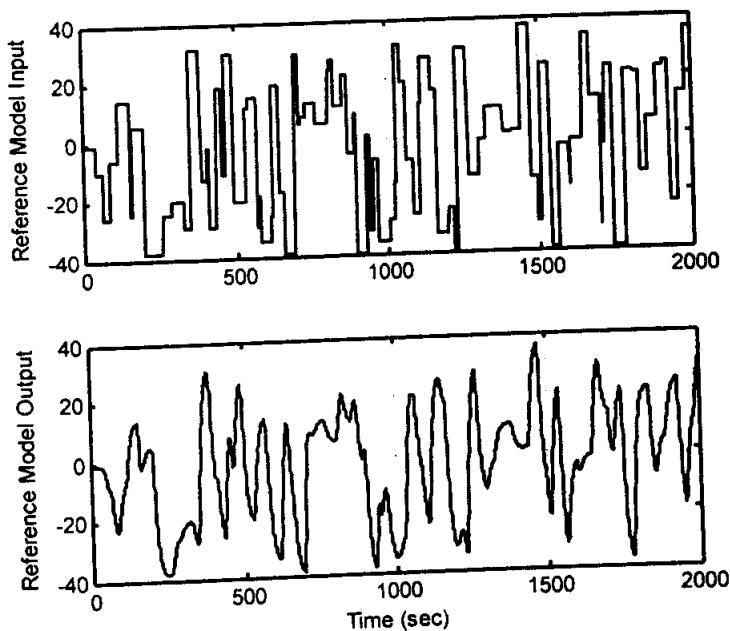
Hình 7.22 là kết quả điều khiển cánh tay máy dùng bộ điều khiển theo mô hình chuẩn dùng mạng thần kinh vừa huấn luyện. Ta thấy rằng cánh tay máy bám theo tín hiệu đặt không có vọt lố và sai số xác lập. Tuy nhiên, cũng cần chú ý mômen điều khiển tác động lên trục quay của cánh tay máy có biến động đột ngột, thực tế nên tránh tín hiệu điều khiển dạng này vì nó làm giảm tuổi thọ của các thiết bị phần cứng.



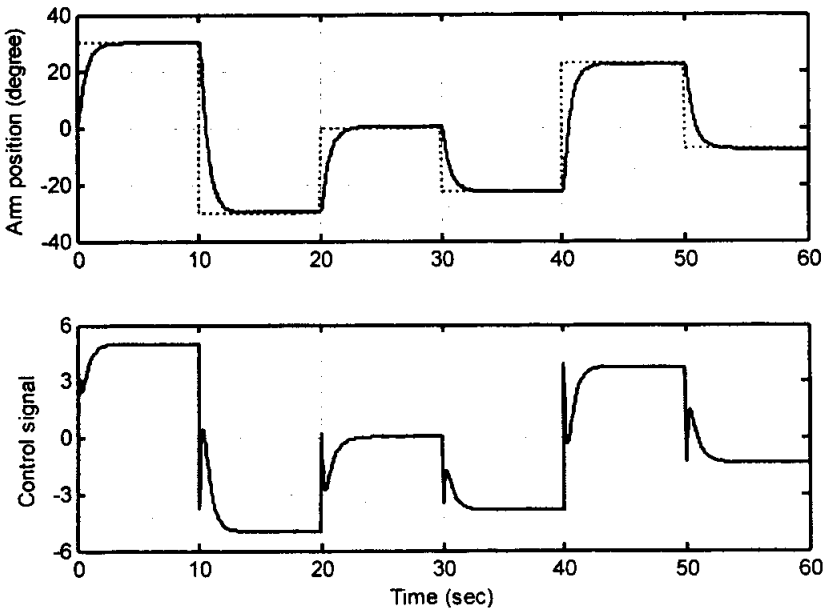
Hình 7.19 Sơ đồ mô phỏng hệ thống điều khiển cánh tay máy theo mô hình chuẩn



Hình 7.20 Dữ liệu huấn luyện mạng thần kinh nhận dạng mô hình cánh tay máy



Hình 7.21 Dữ liệu để huấn luyện mạng thần kinh điều khiển theo mô hình chuẩn



Hình 7.22 Kết quả điều khiển cánh tay máy

Chú ý: Kết quả điều khiển theo mô hình chuẩn phụ thuộc rất nhiều vào kết quả huấn luyện các mạng thần kinh. Do các thuật toán huấn luyện mạng là thuật toán tìm cực trị cục bộ nên có thể phải huấn luyện mạng nhiều lần mới được bộ điều khiển đạt yêu cầu. Ngoài ra cũng cần chú ý dữ liệu huấn luyện mạng cũng rất quan trọng, nếu dữ liệu không tổng quát thì dù có huấn luyện mạng bao nhiêu lần đi nữa thì kết quả điều khiển cũng không thể tốt được. Cuối cùng, nếu dữ liệu đã tổng quát, đã huấn luyện mạng nhiều lần mà kết quả điều khiển vẫn không tốt thì phải xem lại cấu trúc mạng: chọn lại các tín hiệu vào phù hợp và tăng số tế bào thần kinh ở lớp ẩn.

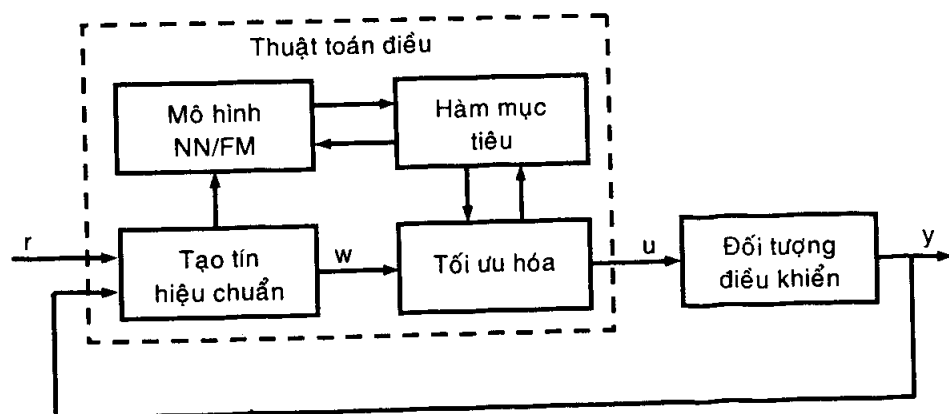
7.5 ĐIỀU KHIỂN DỰ BÁO DỰA VÀO MÔ HÌNH

7.5.1 Nguyên tắc điều khiển dự báo

Mặc dù có nhiều phương pháp điều khiển dự báo khác nhau nhưng về cơ bản ý tưởng chủ đạo của hệ thống điều khiển dự báo (H.7.23) như sau:

- Dùng một mô hình để dự báo đáp ứng của đối tượng tại các thời điểm rời rạc trong tương lai trong một phạm vi dự báo (*prediction horizon*) nhất định.

- Tính toán chuỗi tín hiệu điều khiển tương lai trong phạm vi điều khiển (*control horizon*) bằng cách tối thiểu hóa một hàm mục tiêu. Hàm mục tiêu thường dùng là yêu cầu làm cho tín hiệu dự báo đáp ứng của đối tượng phải càng gần quỹ đạo đáp ứng mong muốn càng tốt, trong điều kiện ràng buộc cho trước.
- Dời phạm vi dự báo theo thời gian, sao cho tại mỗi thời điểm lấy mẫu quá trình tối ưu hóa được lặp lại với tín hiệu đo vừa thu được, và chỉ có tác động điều khiển đầu tiên trong chuỗi tác động điều khiển đã tính toán được xuất ra để điều khiển đối tượng.

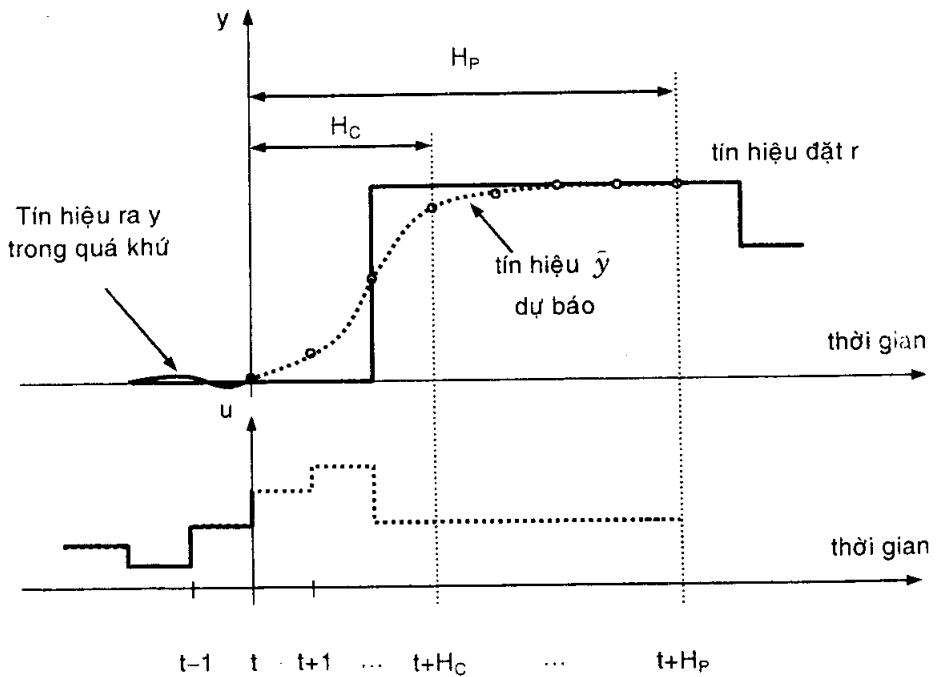


Hình 7.23 Sơ đồ khối hệ thống điều khiển dự báo

Phạm vi dự báo và phạm vi điều khiển

Bằng cách dùng một mô hình của đối tượng, có thể dự báo trước đáp ứng của hệ thống ở tương lai trong phạm vi dự báo H_P . Giá trị ngõ ra dự báo $\hat{y}(t+k)$, ($k = 1..H_P$) phụ thuộc vào đáp ứng của hệ thống trong quá khứ và tín hiệu điều khiển tương lai $u(t+k)$ trong phạm vi điều khiển H_C định trước, giả sử rằng $H_C \leq H_P$ và $u(t+k)$ giữ giá trị là hằng số trong khoảng ($k = \overline{H_C..H_P}$) (H.7.24).

- H_C thường được chọn bằng bậc của mô hình (có thể chọn nhỏ hơn nếu sử dụng hàm mục tiêu mờ). Giá trị H_C nhỏ sẽ làm giảm đáng kể số phép tính trong bước tối ưu hóa.
- H_P thường được chọn liên quan đến thời gian đáp ứng của quá trình (đáp ứng bậc thang đơn vị). Đối với hệ phi tuyến việc chọn giá trị H_P phù hợp thường mang tính thử sai do thời gian đáp ứng của hệ phi tuyến thay đổi đáng kể tùy theo điểm làm việc.



Hình 7.24 Nguyên tắc điều khiển dự báo

Hàm mục tiêu và quỹ đạo chuẩn

Chuỗi tín hiệu điều khiển trong tương lai được rút ra bằng cách tối ưu hóa một hàm mục tiêu mô tả mục đích của chiến lược điều khiển. Hàm mục tiêu thường dùng có dạng tổng bình phương của sai số dự báo và tín hiệu điều khiển:

$$J(u) = \sum_{k=1}^{H_P} \sigma_k [w(t+k) - \hat{y}(t+k)]^2 + \sum_{k=1}^{H_C} \rho_k [u(t+k-1)]^2 \quad (7.21)$$

hoặc tổng bình phương của sai số dự báo và biến thiên tín hiệu điều khiển:

$$J(u) = \sum_{k=1}^{H_P} \sigma_k [w(t+k) - \hat{y}(t+k)]^2 + \sum_{k=1}^{H_C} \rho_k [\Delta u(t+k-1)]^2 \quad (7.22)$$

trong đó các hệ số σ_k và ρ_k là trọng số của các thành phần trong hàm mục tiêu.

Vector tín hiệu điều khiển cần tìm là:

$$u = [u(t) \quad u(t+1) \quad \dots \quad u(t+H_C-1)]^T \quad (7.23)$$

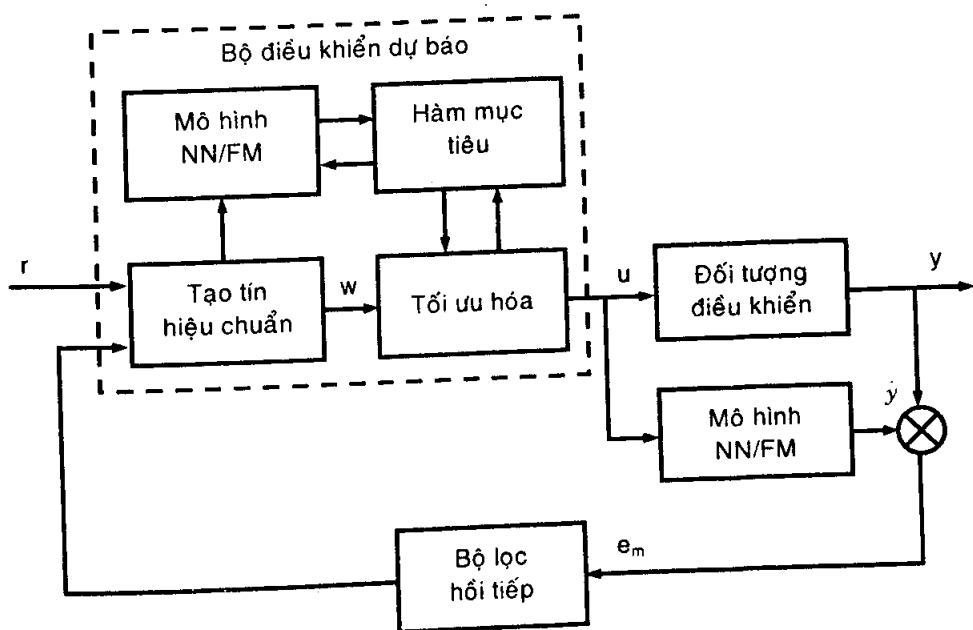
Trong đa số các trường hợp tín hiệu chuẩn mong muốn đã biết trước và hệ thống có thể đã phản ứng trước khi sự thay đổi tín hiệu chuẩn thật sự xảy ra. Một lời khuyên là nên làm trơn sự biến thiên của tín hiệu chuẩn bằng cách sử dụng bộ tạo tín hiệu chuẩn để tránh sự thay đổi đột ngột của tín hiệu điều khiển (tín hiệu chuẩn có thể tạo ra bằng cách dùng bộ lọc:

$$w(t+k) = \alpha w(t) + (1-\alpha)r(t+k), \quad (k = \overline{1, H_P}; 0 < \alpha < 1) \quad (7.24)$$

trong đó $w(t) = y(t)$, $r(t+k)$ là tín hiệu ra mong muốn của hệ thống. Bộ lọc này làm trơn tín hiệu ra mong muốn, nhờ đó làm tăng độ bền vững của hệ thống. Tuy nhiên điều này cũng dẫn đến một khuyết điểm là làm cho đáp ứng của hệ thống chậm hơn.

Vì phương pháp điều khiển dự báo sử dụng mô hình của đối tượng và phương pháp tối ưu hóa nên có thể áp dụng cho các hệ thống phức tạp như hệ đa biến, hệ không ổn định vòng hở, hệ phi tuyến hay hệ có trễ lớn. Ngoài ra phương pháp này cũng rất hiệu quả trong trường hợp phải giải các bài toán điều khiển có điều kiện ràng buộc tín hiệu vào và tín hiệu ra của đối tượng.

Có thể kết hợp chiến lược MBPC với sơ đồ điều khiển IMC để bù nhiễu và sai số mô hình (H.7.25).



Hình 7.25 Bộ điều khiển dự báo trong sơ đồ điều khiển mô hình nội

7.5.2 Giải bài toán tối ưu hóa

Mục này trình bày cách sử dụng thuật toán Levenberg–Marquardt, một phiên bản của thuật toán Newton, để tìm chuỗi tín hiệu điều khiển $\mathbf{u}$ tối thiểu hàm mục tiêu (7.21), cách tìm lời giải tối ưu hàm mục tiêu (7.22) có thể rút ra bằng cách tương tự.

Thuật toán Newton tìm vector $\hat{\mathbf{u}}$ tối thiểu hàm mục tiêu (7.21) bằng công thức lặp:

$$\hat{\mathbf{u}}^{(i+1)} = \hat{\mathbf{u}}^{(i)} - \left[\mathbf{J}''(\hat{\mathbf{u}}^{(i)}) \right]^{-1} \mathbf{J}'(\hat{\mathbf{u}}^{(i)}) \quad (7.25)$$

Trong đó:

$$\mathbf{J}'(\hat{\mathbf{u}}^{(i)}) = - \sum_{k=1}^{H_P} 2\sigma_k \psi(t+k, \hat{\mathbf{u}}^{(i)}) \varepsilon^{(i)}(t+k) + 2 \text{diag}(\rho_1, \dots, \rho_{H_C}) \hat{\mathbf{u}}^{(i)} \quad (7.26)$$

$$\varepsilon^{(i)}(t+k) = w(t+k) - \hat{y}(t+k, \hat{\mathbf{u}}^{(i)}) \quad (7.27)$$

$$\psi(t+k, \hat{\mathbf{u}}^{(i)}) = \left[\frac{\partial y(t+k)}{\partial \mathbf{u}} \right]_{\mathbf{u}=\hat{\mathbf{u}}^{(i)}} \quad (7.28)$$

$$\begin{aligned} \mathbf{J}''(\hat{\mathbf{u}}^{(i)}) = \sum_{k=1}^{H_P} 2\sigma_k \left[(t+k, \hat{\mathbf{u}}^{(i)}) \psi^T(t+k, \hat{\mathbf{u}}^{(i)}) - \right. \\ \left. - \frac{\partial \psi(t+k, \hat{\mathbf{u}}^{(i)})}{\partial \mathbf{u}} \varepsilon^{(i)}(t+k) \right] + 2 \text{diag}(\rho_1, \dots, \rho_{H_C}) \end{aligned} \quad (7.29)$$

Thuật toán Newton hội tụ rất nhanh (số bước lặp ít), tuy nhiên thời gian cần thiết để hoàn tất một bước lặp lại khá chậm do việc tính chính xác $\mathbf{J}''(\hat{\mathbf{u}}^{(i)})$ đòi hỏi rất nhiều phép tính. Thuật toán Levenberg–Marquardt có đặc tính hội tụ xấp xỉ thuật toán Newton, tuy nhiên thời gian tính mỗi bước lặp lại nhanh hơn rất nhiều do $\mathbf{J}''(\hat{\mathbf{u}}^{(i)})$ được tính gần đúng như sau:

$$\begin{aligned} \mathbf{J}''(\hat{\mathbf{u}}^{(i)}) = \sum_{k=1}^{H_P} 2\sigma_k \left[\psi(t+k, \hat{\mathbf{u}}^{(i)}) \psi^T(t+k, \hat{\mathbf{u}}^{(i)}) \right] \\ + 2 \text{diag}(\rho_0, \rho_2, \dots, \rho_{H_C}) + \mu^{(i)} \mathbf{I} \end{aligned} \quad (7.30)$$

Có nhiều chiến lược thay đổi hệ số $\mu^{(i)}$ để tăng tốc độ hội tụ. Trong thuật toán điều khiển dự báo 7.1 trình bày dưới đây $\mu^{(i)}$ thay đổi theo chiến lược đề xuất bởi Marquardt.

Thuật toán 7.1

Cho đối tượng điều khiển có mô hình $\hat{y}(t)$. Thuật toán điều khiển dự báo tối thiểu hàm mục tiêu (7.21) được tóm tắt như sau:

Bước 1: Khởi động $t = 0$.

Bước 2: Lọc tín hiệu chuẩn (7.24).

Bước 3: Gán $i = 0$, khởi động giá trị ban đầu $\mu^{(0)}$, $\hat{u}^{(0)}$.

Bước 4: Tính:

- Đáp ứng dự báo $\hat{y}(t + k, \hat{u}^{(i)})$, ($k = \overline{1, H_P}$)

- Hàm mục tiêu $J(\hat{u}^{(i)})$ (7.21)

- Sai số $\varepsilon^{(i)}(t + k)$ (7.27)

- Đạo hàm của bộ dự báo $\psi(t + k, \hat{u}^{(i)})$ (7.28); $k = \overline{1, H_P}$

- Đạo hàm bậc 1 của hàm mục tiêu $J'(\hat{u}^{(i)})$ (7.26)

Bước 5: $\mu^{(i)} = \frac{\mu^{(i)}}{\lambda}$

Bước 6: Tính:

- Đạo hàm bậc 2 của hàm mục tiêu $J''(\hat{u}^{(i)})$ (7.30)

- Tín hiệu điều khiển $\hat{u}^{(i+1)}$ (7.25)

- Đáp ứng dự báo $\hat{y}(t + k, \hat{u}^{(i+1)})$, $k = \overline{1, H_P}$

- Hàm mục tiêu $J'(\hat{u}^{(i+1)})$ (7.21)

Bước 7: Nếu $J'(\hat{u}^{(i+1)}) > J'(\hat{u}^{(i)})$ thì $\mu^{(i)} = \lambda \mu^{(i)}$ và trở lại bước 6.

Bước 8: Nếu đã hội tụ (chẳng hạn $J'(\hat{u}^{(i+1)}) < 10^{-6}$) thì sang bước 9.
Nếu chưa hội tụ thì gán $i = i + 1$; $\mu^{(i)} = \mu^{(i-1)}$ rồi trở lại bước 4.

Bước 9: Xuất tín hiệu $u(t)$ điều khiển đối tượng.

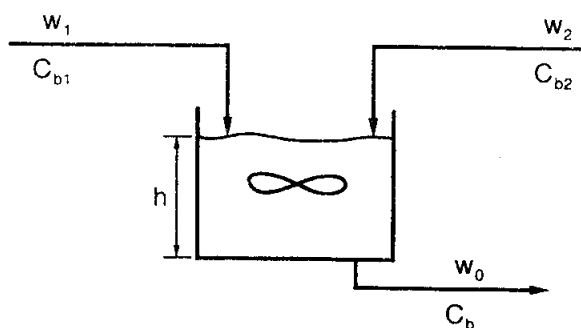
Bước 10: Nếu chưa kết thúc điều khiển thì gán $t = t + 1$ rồi trở lại bước 2.

Trong thuật toán trên, từ bước 2 đến bước 8 là vòng lặp tìm chuỗi tín hiệu điều khiển tối ưu $u(t+k)$ trong phạm vi điều khiển $k = 0, H_C - 1$, tuy nhiên chỉ có $u(t)$ được xuất ra để tác động lên đối tượng. Nếu chưa kết thúc điều khiển thì ở thời điểm lấy mẫu kế tiếp quá trình tối ưu hóa lại tiếp tục với $\hat{u}^{(0)}(t)$ chính là chuỗi tín hiệu điều khiển tối ưu ở thời điểm lấy mẫu trước đó.

Ngoài thuật toán Levenberg–Marquardt, các thuật toán khác cũng được sử dụng để giải bài toán tối ưu hóa trong sơ đồ điều khiển dự báo hệ phi tuyến như thuật toán suy giảm độ dốc (rất nhiều phiên bản khác nhau), thuật toán nhảy và chặn (*branch-and-bound*), giải thuật di truyền...

Ví dụ 7.5 Điều khiển dự báo hệ bồn phản ứng dùng mô hình mạng thần kinh.

Cho hệ bồn phản ứng có sơ đồ ở hình 7.26. Hệ thống gồm một bồn khuấy liên tục trộn dòng sản phẩm có nồng độ cao C_{b1} với dòng sản phẩm có nồng độ thấp C_{b2} để được sản phẩm có nồng độ C_b ở đầu ra. Đặc tính động học của hệ thống mô tả bởi các phương trình sau:



Hình 7.26 Bồn phản ứng

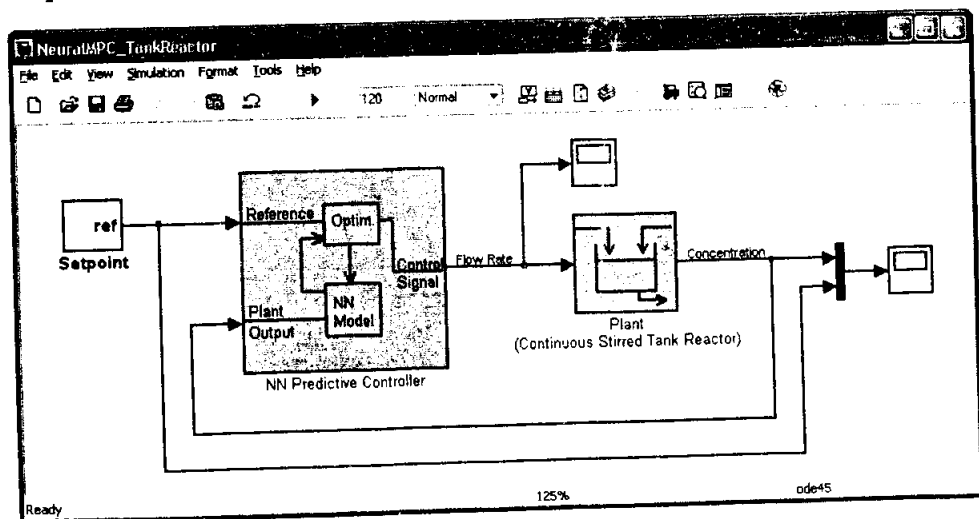
$$\frac{dh(t)}{dt} = w_1(t) + w_2(t) - 0,2\sqrt{h(t)} \quad (7.31)$$

$$\begin{aligned} \frac{dC_b(t)}{dt} = & (C_{b1} - C_b(t)) \frac{w_1(t)}{h(t)} + \\ & + (C_{b2} - C_b(t)) \frac{w_2(t)}{h(t)} - \frac{k_1 C_b(t)}{(1 + k_2 C_b(t))^2} \end{aligned} \quad (7.32)$$

trong đó $h(t)$ là độ cao mực chất lỏng trong bồn, $C_b(t)$ là nồng độ sản phẩm ở đầu ra của quá trình, $w_1(t)$ là lưu lượng vào bồn của dòng

sản phẩm có nồng độ cao C_{b1} , và $w_2(t)$ là lưu lượng vào bồn của dòng sản phẩm có nồng độ thấp C_{b2} . Giả sử nồng độ của sản phẩm vào là $C_{b1} = 24,9$ và $C_{b2} = 0,1$. Các hằng số liên quan đến mức độ tiêu hao là $k_1 = 1$ và $k_2 = 1$.

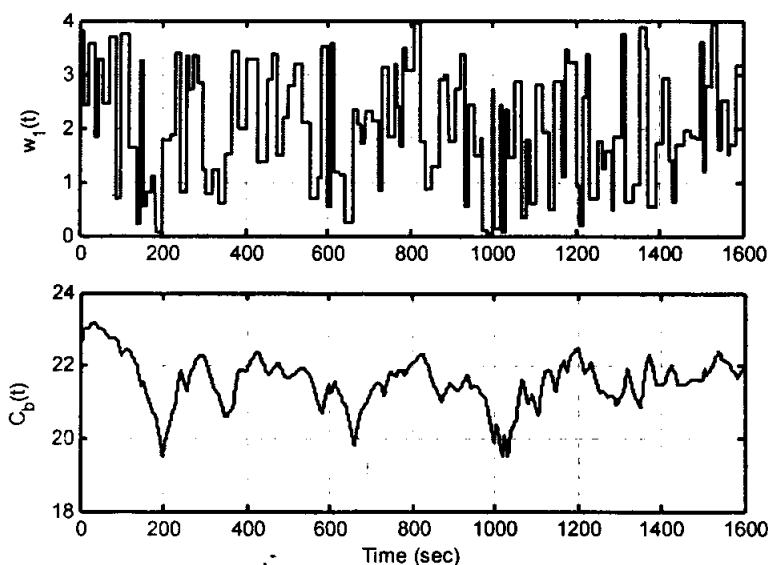
Bài toán đặt ra là điều khiển nồng độ sản phẩm ở đầu ra theo giá trị đặt bằng cách điều chỉnh lưu lượng vào $w_1(t)$. Để đơn giản, giả sử $w_2(t) = 0,1$ và không quan tâm đến việc điều khiển độ cao $h(t)$ của sản phẩm trong bồn chứa.



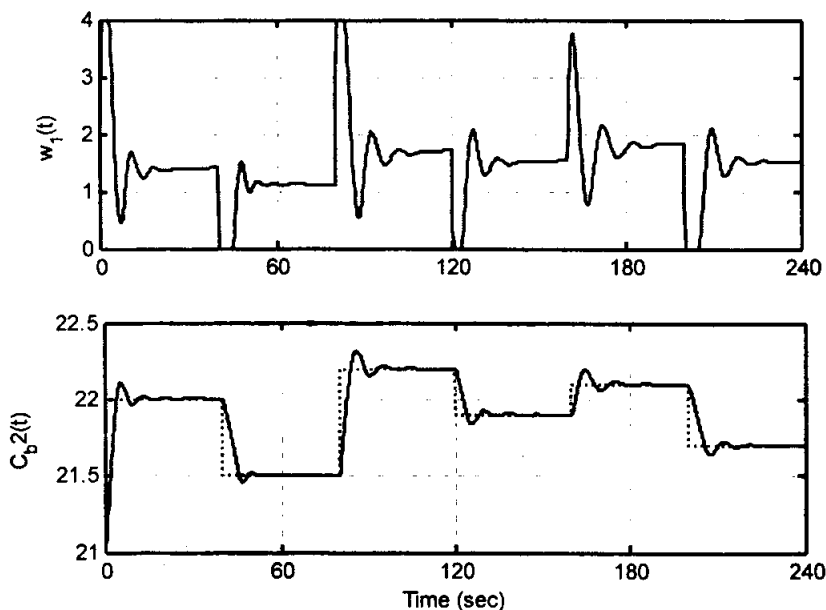
Hình 7.27 Sơ đồ mô phỏng hệ thống điều khiển dự báo bồn phản ứng dùng mô hình mạng thần kinh

Bài toán điều khiển đặt ra ở trên có thể giải bằng phương pháp điều khiển dự báo. Hình 7.27 là sơ đồ mô phỏng hệ thống điều khiển dự báo bồn phản ứng dùng mô hình mạng thần kinh (chương trình mẫu *predcstr.mdl* của Matlab Neural Network Toolbox). Trong khối **[NN Predictive Controller]** có sử dụng một mô hình mạng thần kinh nhận dạng đặc tính động học của đối tượng mô tả bởi hai phương trình (7.31) và (7.32). Mạng thần kinh có các ngõ vào là $w_1(t-1)$, $w_1(t-2)$, $C_b(t-1)$, $C_1(t-2)$ và có 7 tế bào thần kinh ở lớp ẩn. Dữ liệu vào ra của đối tượng dùng để huấn luyện mạng thần kinh trình bày ở hình 7.28. Thuật toán điều khiển dự báo tính toán tín hiệu điều khiển $w_1(t)$ sao cho tối thiểu chỉ tiêu chất lượng (7.22) với các thông số được chọn như sau: $H_C = 2$, $H_P = 7$, $\sigma = 1$, $\rho = 0,05$. Thuật toán tối ưu hóa sử dụng tìm cực tiểu chỉ tiêu (7.22) là thuật toán suy giảm độ dốc. Kết quả điều

khiến dự báo minh họa ở hình 7.29, ta thấy nồng độ của sản phẩm ra bám theo tín hiệu đặt sau thời gian quá độ khoảng 15 giây.

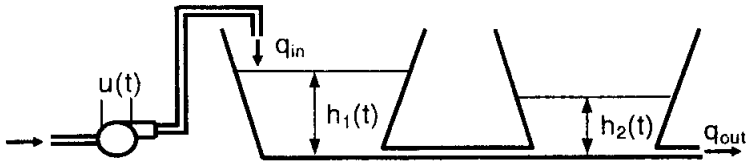


Hình 7.28 Dữ liệu vào ra của bồn phản ứng



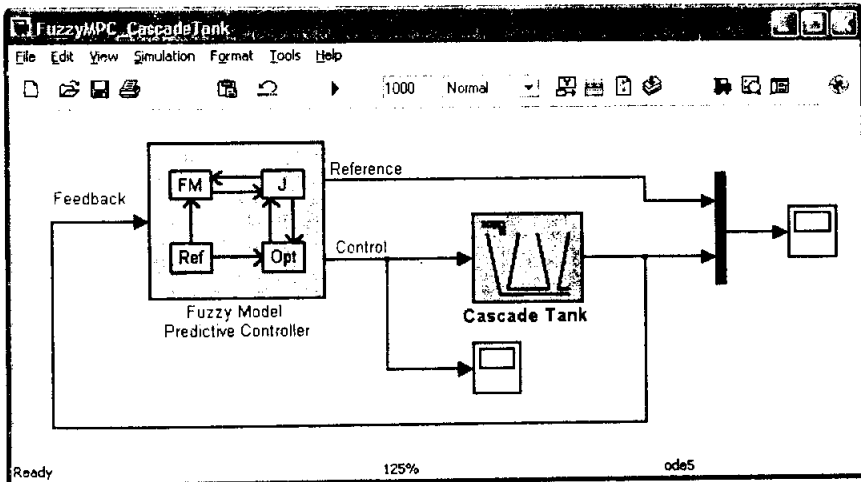
Hình 7.29 Kết quả điều khiển dự báo bồn phản ứng dùng mạng thần kinh

Ví dụ 7.6 Điều khiển dự báo hệ bồn nối tiếp dùng mô hình mờ Takagi–Sugeno



Hình 7.30 Hệ bồn nối tiếp

Ở ví dụ 6.2 chúng ta đã nhận dạng hệ bồn chứa nối tiếp dùng mô hình mờ Takagi–Sugeno, trong ví dụ này chúng ta sẽ sử dụng mô hình mờ Takagi–Sugeno đã nhận dạng để thiết kế bộ điều khiển dự báo.



Hình 7.31 Sơ đồ mô phỏng hệ thống điều khiển dự báo hệ bồn kép dùng mô hình mờ Takagi–Sugeno

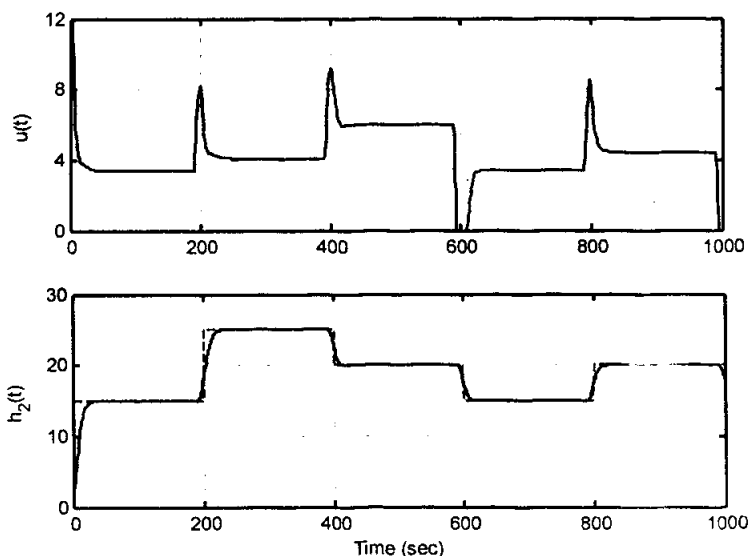
Hình 7.31 là sơ đồ mô phỏng hệ thống điều khiển dự báo bồn nối tiếp. Trong khối [Fuzzy Model Predictive Controller] có sử dụng mô hình mờ Takagi–Sugeno đã nhận dạng ở ví dụ 6.2. Khối tạo tín hiệu chuẩn lọc tín hiệu đặt có dạng xung vuông theo biểu thức:

$$w(k) = 0,7w(k - 1) + 0,3r(k) \quad (7.33)$$

Khối tối ưu hóa tính tín hiệu điều khiển tối ưu theo chỉ tiêu (7.21) với tầm dự báo $H_P = 10$, tầm điều khiển $H_C = 3$, các trọng số được chọn là $\sigma = 1$, $\rho = 400$. Bài toán tối ưu hóa được giải dùng thuật toán Levenberg–Marquardt.

Hình 7.32 là kết quả mô phỏng hệ thống điều khiển vừa thiết kế. Theo đồ thị ta thấy mực chất lỏng ở bồn 2 bắt đầu thay đổi trạng thái trước khi tín hiệu đặt thay đổi. Lý do là vì tín hiệu điều khiển

được tính toán để tối thiểu sai lệch giữa tín hiệu ra dự báo và tín hiệu đặt ở tương lai. Đây là một ưu điểm đặc trưng của phương pháp điều khiển dự báo mà các phương pháp điều khiển khác không có được. Tuy nhiên, không phải hệ thống điều khiển nào ta cũng có thể biết trước tín hiệu đặt ở tương lai, do đó phương pháp điều khiển dự báo chỉ thích hợp để điều khiển các quá trình công nghiệp, trong đó tín hiệu đặt đã biết trước do quy trình công nghệ.



Hình 7.32 Kết quả điều khiển dự báo hệ bồn nối tiếp dùng mô hình mờ TS

7.6 TÓM TẮT

Chương 7 trình bày các phương pháp điều khiển sử dụng tường minh mô hình của đối tượng để tính toán tín hiệu điều khiển. Các phương pháp điều khiển dùng mô hình ngược, điều khiển theo mô hình chuẩn, điều khiển mô hình nội và điều khiển dự báo được trình bày lý thuyết kèm theo là ví dụ minh họa cụ thể cách thiết kế. Chú ý rằng chất lượng của các phương pháp điều khiển dựa vào mô hình phụ thuộc rất nhiều vào chất lượng của mô hình được sử dụng trong hệ thống điều khiển, nếu mô hình không mô tả được đặc tính động học của đối tượng thì chất lượng điều khiển không thể nào tốt được. Các vấn đề liên quan đến nhận dạng hệ phi tuyến đã được trình bày ở chương 6. Do mạng thần kinh linh hoạt hơn mô hình mờ trong việc nhận dạng đối tượng nên thực tế có nhiều hệ thống điều khiển sử dụng mô hình mạng thần kinh hơn so với mô hình mờ.

Chương 8

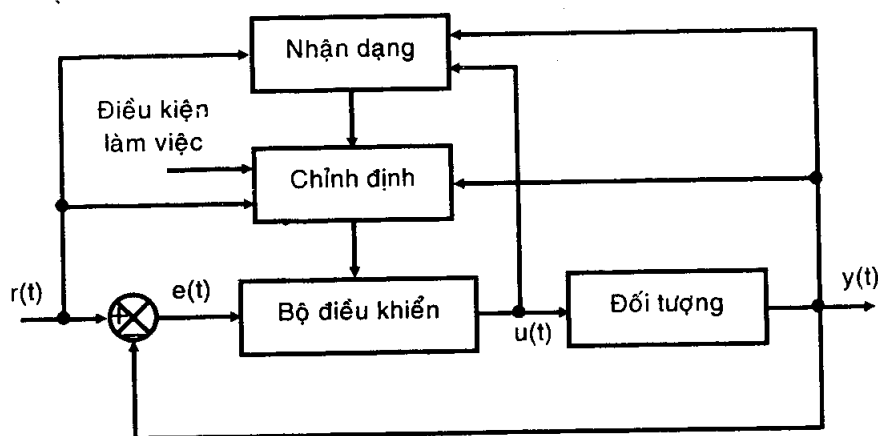
ĐIỀU KHIỂN THÍCH NGHI

8.1 GIỚI THIỆU

Bằng cách giải bài toán nhận dạng hệ thống chúng ta có thể xác định được mô hình toán của đối tượng và sử dụng thông tin này để thiết kế hệ thống điều khiển. Nếu đặc tính của đối tượng điều khiển không thay đổi trong suốt quá trình hoạt động thì hệ thống điều khiển đã thiết kế sẽ có đáp ứng thỏa mãn yêu cầu. Tuy nhiên, kinh nghiệm cho thấy trong thực tế thông số của đối tượng thường thay đổi không dự đoán trước được do ảnh hưởng của điều kiện làm việc khi hệ thống hoạt động, do đó nếu không liên tục được “thiết kế lại” thì bộ điều khiển thiết kế ban đầu có thể không thể khống chế tốt đối tượng. *Hệ thống điều khiển trong đó thông số (và cấu trúc) của bộ điều khiển thay đổi trong quá trình vận hành nhằm giữ vững chất lượng điều khiển của hệ thống có sự hiện diện của các yếu tố bất định hoặc biến đổi không biết trước trong thông số hệ thống gọi là hệ thống điều khiển thích nghi.* Vì các yếu tố bất định xuất hiện trong nhiều bài toán điều khiển thực tế, điều khiển thích nghi rất hữu ích trong nhiều lĩnh vực công nghiệp.

Hình 8.1 là sơ đồ khối tổng quát của hệ thống điều khiển thích nghi, trong sơ đồ ta thấy bộ điều khiển được chỉnh định liên tục khi hệ thống đang hoạt động. Tùy theo cách chỉnh định thông số mà ta có các phương pháp điều khiển thích nghi khác nhau. Đơn giản nhất là phương pháp hoạch định độ lợi (*gain scheduling*), theo phương pháp này thông số của bộ điều khiển thay đổi theo điều kiện làm việc, khối nhận dạng trong sơ đồ tổng quát ở hình 8.1 không cần thiết. Nếu khối nhận dạng ước lượng thông số của đối tượng và khối chỉnh định tính toán thông số của bộ điều khiển theo mô hình của đối tượng ta có sơ đồ điều khiển thích nghi gián tiếp, ngược lại nếu

khối nhận dạng ước lượng thông số của bộ điều khiển ta có sơ đồ điều khiển thích nghi trực tiếp. Nếu trong khối chỉnh định có sử dụng một mô hình chuẩn và chiến lược chỉnh định là tối thiểu sai lệch giữa đầu ra của mô hình chuẩn và đầu ra của hệ thống thật ta có sơ đồ điều khiển thích nghi (gián tiếp hay trực tiếp) theo mô hình chuẩn.



Hình 8.1 Sơ đồ khối hệ thống điều khiển thích nghi

Lịch sử phát triển lý thuyết điều khiển thích nghi

Lý thuyết điều khiển thích nghi ra đời từ những năm đầu thập niên 1950 xuất phát từ yêu cầu thiết kế hệ thống điều khiển lái máy bay tự động. Máy bay có đặc tính động học thay đổi rất mạnh khi thay đổi tốc độ và độ cao nên không thể điều khiển bằng bộ điều khiển hồi tiếp có độ lợi cố định, do đó cần bộ điều khiển phức tạp hơn có thể học và thay đổi thích ứng theo đặc tính động thay đổi của máy bay. Bộ điều khiển thích nghi theo mô hình chuẩn MRAC đã được đề xuất để giải quyết bài toán điều khiển lái tự động. Trong thập niên 1950, phương pháp độ nhảy và phương pháp độ dốc được sử dụng để thiết kế luật thích nghi cho nhiều sơ đồ điều khiển thích nghi khác nhau.

Thập niên 1960 là thời kỳ quan trọng nhất đối với sự phát triển của lý thuyết điều khiển nói chung và điều khiển thích nghi nói riêng. Kỹ thuật không gian trạng thái và lý thuyết ổn định dựa vào lý thuyết Lyapunov được đưa ra. Sự phát triển lý thuyết quy hoạch động, điều khiển ngẫu nhiên, nhận dạng hệ thống và ước lượng thông số động vai trò quan trọng trong phát triển lý thuyết điều khiển thích nghi.

Sự phát triển của lý thuyết ổn định và lý thuyết điều khiển trong thập niên năm 1960 đã hoàn thiện quan niệm về điều khiển thích nghi và góp phần tạo nên một trào lưu nghiên cứu mới trong lĩnh vực này vào những năm 1970. Nhiều sơ đồ điều khiển thích nghi ra đời là kết quả của sự kết hợp giữa các sơ đồ ước lượng thông số với các phương pháp thiết kế bộ điều khiển, đặc biệt là phương pháp thiết kế dựa vào lý thuyết Lyapunov. Mặt khác sự phát triển của kỹ thuật điện tử và máy tính trong giai đoạn này cũng cho phép thực thi các bộ điều khiển phức tạp như điều khiển thích nghi, điều này đã làm gia tăng sự quan tâm nghiên cứu về các ứng dụng của điều khiển thích nghi. Có thể nói giai đoạn 1970 là thời kỳ phát triển mạnh mẽ nhất của điều khiển thích nghi, cả về lý thuyết lẫn ứng dụng.

Tuy nhiên, sự thành công của giai đoạn 1970 nhanh chóng được nối tiếp bằng các tranh luận về tính thực tiễn của điều khiển thích nghi. Năm 1979 người ta đã chỉ ra rằng các sơ đồ điều khiển thích nghi đã đưa ra trong những năm 1970 dễ dàng trở nên mất ổn định khi có nhiễu loạn nhỏ. Tính không bền vững của điều khiển thích nghi trở nên vấn đề gây nhiều tranh luận trong những năm đầu thập niên 1980 khi ngày càng có nhiều ví dụ về tính không ổn định được công bố minh họa sự thiếu ổn định đối với sai số mô hình và nhiễu bị chặn. Điều này đã là động lực để nhiều nhà nghiên cứu tìm hiểu về cơ chế gây mất ổn định và tìm cách khắc phục. Vào những năm giữa thập niên 1980, vài sơ đồ thiết kế lại và cải tiến được đề xuất và phân tích, dẫn đến nghiên cứu về điều khiển thích nghi bền vững. Một bộ điều khiển thích nghi được gọi là bền vững nếu nó đảm bảo tất cả các tín hiệu trong hệ thống bị chặn khi có sai số mô hình và nhiễu bị chặn, đồng thời giới hạn sai số chất lượng cùng bậc với sai số mô hình hóa. Nghiên cứu về điều khiển thích nghi bền vững tiếp tục trong thập niên 1980. Lời giải của bài toán điều khiển thích nghi bền vững dẫn đến lời giải cho bài toán có từ lâu là điều khiển hệ tuyến tính có thông số bất định và biến đổi theo thời gian.

Hướng nghiên cứu lý thuyết điều khiển thích nghi từ thập niên 1990 đến nay là nghiên cứu tính chất của chất lượng của hệ điều khiển thích nghi và mở rộng các kết quả có được trong thập niên 1980 cho các lớp hệ phi tuyến có thông số không biết trước. Các cố gắng này dẫn đến hình thành lớp các sơ đồ điều khiển thích nghi

mới xuất phát từ lý thuyết điều khiển phi tuyến cũng như các sơ đồ điều khiển thích nghi cải thiện đáp ứng quá độ và xác lập.

Có rất nhiều tài liệu tham khảo về điều khiển thích nghi với các kỹ thuật thiết kế, phân tích khác nhau. Đến nay có thể nói các sơ đồ điều khiển thích nghi đối tượng tuyến tính đã được nghiên cứu khá hoàn chỉnh. Trái ngược với lý thuyết điều khiển thích nghi hệ tuyến tính, lý thuyết điều khiển thích nghi hệ phi tuyến mặc dù cũng đã được tập trung nghiên cứu rất mạnh từ đầu thập niên 1990 nhưng đến nay một số vấn đề liên quan vẫn còn là bài toán mở. Hiện nay, đa số các sơ đồ điều khiển thích nghi hệ phi tuyến đều dựa trên kỹ thuật hồi tiếp tuyến tính hóa. Theo lý thuyết điều khiển phi tuyến, nếu biết chính xác đặc tính phi tuyến của đối tượng thì có thể biến đổi quan hệ vào ra của hệ phi tuyến thành quan hệ tuyến tính nhờ vào hồi tiếp trạng thái. Trong sơ đồ điều khiển thích nghi gián tiếp, các bộ xấp xỉ tổng quát được sử dụng để nhận dạng trực tuyến mô hình toán của đối tượng và luật điều khiển hồi tiếp tuyến tính hóa được tính theo nguyên lý chắc chắn tương đương. Trái lại, trong sơ đồ điều khiển thích nghi trực tiếp, bộ xấp xỉ tổng quát được sử dụng xấp xỉ luật điều khiển hồi tiếp tuyến tính hóa lý tưởng. Nguyên tắc chung để thiết kế hệ thống điều khiển thích nghi là chọn luật cập nhật thông số sao cho hệ thống ổn định theo Lyapunov. Các bộ xấp xỉ tổng quát thường được sử dụng nhiều nhất là các mạng thần kinh và mô hình mờ. Về mặt chức năng mạng thần kinh và mô hình mờ là tương đương nhau nên đa số các trường hợp có thể sử dụng thay thế nhau trong các sơ đồ điều khiển thích nghi.

Trong các công trình đầu tiên về điều khiển thích nghi hệ phi tuyến, đối tượng nghiên cứu là hệ phi tuyến SISO liên tục có mô tả toán học có thể biến đổi về dạng chuẩn. Các bộ điều khiển thích nghi gián tiếp đảm bảo tính ổn định của hệ kín theo Lyapunov và tín hiệu ra của hệ thống tiệm cận bám theo tín hiệu chuẩn. Để hệ thống điều khiển được, điều kiện cần là độ lợi đầu vào phải khác 0, tuy nhiên trong các sơ đồ điều khiển thích nghi gián tiếp, mô hình nhận dạng độ lợi đầu vào rất dễ bị suy biến dẫn đến không thể điều khiển được thì không thể tính được tín hiệu điều khiển hồi tiếp tuyến tính hóa. Để khắc phục tình trạng này có ba hướng tiếp cận:

- Khi hệ thống bắt đầu hoạt động, mô hình xấp xỉ độ lợi đầu vào được khởi động dựa trên kết quả nhận dạng offline

- Dùng các phiên bản cải tiến luật thích nghi gián tiếp sử dụng thuật toán hình chiếu
- Dùng sơ đồ điều khiển thích nghi trực tiếp.

Các sơ đồ điều khiển thích nghi đề xuất đầu thập niên 1990 chỉ áp dụng được cho lớp rất nhỏ các hệ phi tuyến; cụ thể các sơ đồ điều khiển thích nghi gián tiếp chỉ áp dụng được cho hệ phi tuyến không có đặc tính động học không, và các sơ đồ điều khiển thích nghi trực tiếp chỉ có thể áp dụng cho lớp hệ phi tuyến có độ lợi đầu vào bằng hằng số. Trên cơ sở công trình mang tính “mở đường” vừa nêu trên, các nhà nghiên cứu đã đưa ra nhiều phiên bản cải tiến các sơ đồ điều khiển thích nghi theo hướng mở rộng lớp hệ phi tuyến có thể ứng dụng, nới lỏng các điều kiện ràng buộc và nâng cao tính bền vững của hệ thống điều khiển đối với các yếu tố bất định. Phương pháp cuốn chiếu (*back stepping*) cũng đã được áp dụng để thiết kế hệ thống điều khiển thích nghi gián tiếp và trực tiếp hệ phi tuyến SISO có mô tả toán học dưới dạng hồi tiếp chặt. Để đảm bảo tính ổn định bền vững của hệ thống điều khiển đối với sai số xấp xỉ, trong hầu hết các sơ đồ điều khiển thích nghi đã công bố người ta sử dụng thêm thành phần điều khiển chế độ trượt (hay còn gọi là thành phần điều khiển giám sát) song song với thành phần điều khiển thích nghi. Tuy nhiên khi thêm vào thành phần điều khiển chế độ trượt thì xảy ra hiện tượng “chattering” (tín hiệu điều khiển thay đổi với tần số cao). Để khắc phục hiện tượng trên có thể sử dụng phiên bản luật thích nghi cải biến vùng chết để trơn hóa tín hiệu điều khiển, tuy nhiên trong trường hợp này chỉ có thể đảm bảo sai số bám nằm trong lân cận của 0 (không thể đảm bảo vô sai về mặt lý thuyết). Đa số các sơ đồ điều khiển thích nghi đề cập đến ở trên đều giả thiết các biến trạng thái có thể đo lường được trong khi đó thực tế có rất nhiều trường hợp chỉ có thể đo được tín hiệu ra mà không thể đo được các biến trạng thái, điều này chính là động lực để các nhà nghiên cứu kết hợp các kết quả quan sát trạng thái vào các sơ đồ điều khiển thích nghi. Gần đây hơn, vấn đề điều khiển thích nghi bền vững hệ phi tuyến nhằm đảm bảo tính ổn định và chất lượng điều khiển đối với các yếu tố bất định đang được rất nhiều người nghiên cứu. Xu hướng hiện nay đang tập trung nghiên cứu các sơ đồ điều khiển thích nghi bền vững hệ MIMO; hệ rời rạc; hệ phi tuyến biến đổi theo thời gian; hệ thống lai.

8.2 HỒI TIẾP TUYẾN TÍNH HÓA HỆ PHI TUYẾN

Hồi tiếp tuyến tính hóa là phương pháp thiết kế hệ thống điều khiển áp dụng cho lớp các hệ phi tuyến pha cực tiểu có thể hồi tiếp tuyến tính hóa. trong đó tín hiệu điều khiển $u(t)$ xuất hiện tuyến tính trong phương trình trạng thái. Tổng quát, kỹ thuật hồi tiếp tuyến tính hóa nhằm biến đổi hệ phi tuyến thành hệ tuyến tính bằng cách hồi tiếp trạng thái và biến đổi hệ tọa độ. Bài toán hồi tiếp tuyến tính hóa vào ra chính xác giải được khi biết mô hình toán của đối tượng phi tuyến. Trong trường hợp không xác định được mô hình toán của hệ phi tuyến hoặc mô hình có yếu tố không chắc chắn thì áp dụng phiên bản thích nghi của cấu trúc điều khiển này. Hiện nay, đa số các công trình về điều khiển thích nghi hệ phi tuyến theo phương pháp hồi tiếp tuyến tính hóa đều giả thiết mô hình toán chưa biết của hệ phi tuyến có thể mô hình hóa tuyến tính theo thông số để đưa ra thuật toán thích nghi và chứng minh tính ổn định toàn cục. Trước khi đưa ra sơ đồ điều khiển thích nghi, phần này nhắc lại một số khái niệm cơ bản liên quan đến hệ phi tuyến.

Quy ước: Do một số công thức sẽ dẫn ra chương này nếu viết đầy đủ khá dài và gây khó khăn cho người đọc nên trong những trường hợp không thể gây nhầm lẫn đối số của các hàm sẽ được lược bớt.

Xét hệ phi tuyến SISO mô tả bởi phương trình trạng thái có dạng:

$$\begin{cases} \dot{x} = f(x) + g(x)u \\ y = h(x) \end{cases} \quad (8.1)$$

trong đó:

$x = [x_1, \dots, x_n]^T \in \mathbb{R}^n$ - vector trạng thái của hệ thống

$u \in \mathbb{R}$ - tín hiệu vào

$y \in \mathbb{R}$ - tín hiệu ra

$f(x) \in \mathbb{R}^n$, $g(x) \in \mathbb{R}^n$ - các vector hàm trơn mô tả động học của hệ thống

$h(x) \in \mathbb{R}$ - hàm trơn xác định quan hệ giữa biến trạng thái và tín hiệu ra.

Đạo hàm Lie

Định nghĩa 8.1 Đạo hàm Lie

Đạo hàm Lie của hàm $\phi(x) : \mathbb{R}^n \rightarrow \mathbb{R}$ dọc theo vector hàm $f(x) : \mathbb{R}^n \rightarrow \mathbb{R}^n$, ký hiệu $L_f \phi(x)$ được định nghĩa như sau:

$$\begin{aligned} L_f \phi(x) &= \frac{\partial \phi(x)}{\partial x} \cdot f(x) \\ &= \left[\frac{\partial \phi(x)}{\partial x_1}, \dots, \frac{\partial \phi(x)}{\partial x_n} \right] [f_1(x), \dots, f_n(x)]^T \end{aligned} \quad (8.2)$$

Đạo hàm Lie hàm $\phi(x)$ k lần dọc theo vector hàm $f(x)$, ký hiệu $L_f^k \phi(x)$, được định nghĩa như sau:

$$L_f^k \phi(x) = \frac{\partial L_f^{k-1} \phi(x)}{\partial x} \cdot f(x) \quad (8.3)$$

trong công thức đệ quy (8.3), quy ước $L_f^0 \phi(x) = \phi(x)$.

Đạo hàm Lie hàm $\phi(x)$ k lần dọc theo vector hàm $f(x)$, sau đó lấy đạo hàm Lie dọc theo vector hàm $g(x) : \mathbb{R}^n \rightarrow \mathbb{R}$, ký hiệu $L_g L_f^k \phi(x)$, được định nghĩa như sau:

$$L_g L_f^k \phi(x) = \frac{\partial L_f^k \phi(x)}{\partial x} \cdot g(x) \quad (8.4)$$

Bậc tương đối của hệ SISO

Định nghĩa 8.2 Bậc tương đối của hệ phi tuyến SISO

Hệ phi tuyến (8.1) có bậc tương đối là r tại x_0 nếu:

$$\begin{cases} L_g L_f^k h(x) = 0 & \text{khi } 0 \leq k \leq r-1 \\ L_g L_f^r h(x) \neq 0 & \text{khi } k = r \end{cases} \quad (8.5)$$

Hệ phi tuyến MIMO (8.1) có bậc tương đối bằng r trong miền $S_x \subset \mathbb{R}^n$ nếu nó có bậc tương đối bằng r với mọi $x \in S_x$.

Hệ phi tuyến (8.1) được gọi là có bậc tương đối tổng quát nếu $r < n$; được gọi là có bậc tương đối đầy nếu $r = n$.

Hồi tiếp tuyến tính hóa

Giả sử hệ phi tuyến SISO (8.1) có bậc tương đối là r tại x_0 . Lấy đạo hàm tín hiệu ra theo thời gian, ta có:

$$y = h(x)$$

$$\begin{aligned}\frac{dy}{dt} &= \frac{\partial h(x)}{\partial x} \dot{x} = \frac{\partial h(x)}{\partial x} [f(x) + g(x)u] \\ &= L_f h(x) + L_g h(x)u = L_f h(x)\end{aligned}$$

$$\begin{aligned}\frac{d^2 y}{dt^2} &= \frac{\partial L_f h(x)}{\partial x} \dot{x} = \frac{\partial L_f h(x)}{\partial x} [f(x) + g(x)u] \\ &= L_f^2 h(x) + L_g L_f h(x)u = L_f^2 h(x)\end{aligned}$$

...

$$\frac{d^{r-1} y}{dt^{r-1}} = L_f^{r-1} h(x) + L_g L_f^{r-2} h(x)u = L_f^{r-1} h(x)$$

$$\frac{d^r y}{dt^r} = L_f^r h(x) + \underbrace{L_g L_f^{r-1} h(x)u}_{\neq 0} \quad (8.6)$$

$$\text{Đặt: } a(x) = L_f^r h(x) \quad (8.7)$$

$$b(x) = L_g L_f^{r-1} h(x) \quad (8.8)$$

Biểu thức (8.6) có thể viết lại dưới dạng:

$$y^{(r)} = a(x) + b(x)u \quad (8.9)$$

Thực tế có rất nhiều đối tượng điều khiển phi tuyến mà quan hệ vào ra có thể mô tả bằng phương trình (8.9). Bài toán đặt ra là thiết kế hệ thống điều khiển sao cho tín hiệu ra $y(t)$ của đối tượng bám theo quỹ đạo chuẩn mong muốn $y_m(t) \in \mathcal{R}$. Một trong những cách đơn giản nhất để thực hiện mục tiêu điều khiển trên là dùng luật điều khiển hồi tiếp tuyến tính hóa:

$$u^*(x) = \frac{1}{b(x)} [-a(x) + v(t)] \quad (8.10)$$

trong đó $v(t) \in \mathfrak{R}$ là vector tín hiệu vào hệ thống động đã được tuyến tính hóa:

$$v(t) = y_m^{(r)} + \bar{e}_s + \eta e_s \quad (8.11)$$

với η là hệ số dương, e_s và $\bar{e}_s$ được định nghĩa như sau:

$$e_s(t) = e_o^{(r-1)} + k_1 e_o^{(r-2)} + \dots + k_{r-1} e_o \quad (8.12)$$

$$e_o(t) = y_m(t) - y(t) \quad (8.13)$$

$$\bar{e}_s(t) = \dot{e}_s - e_o^{(r)} = k_1 e_o^{(r-1)} + \dots + k_{r-1} \dot{e}_o \quad (8.14)$$

$e_o(t)$ là sai số ngõ ra và $e_s(t)$ là sai số bám. Các hệ số k_i được chọn sao cho:

$$\Delta(s) = s^{(r-1)} + k_1 s^{(r-2)} + \dots + k_{r-2} s + k_{r-1} \quad (8.15)$$

là đa thức Hurwitz, nghĩa là tất cả các nghiệm của phương trình $\Delta(s) = 0$ đều nằm bên trái mặt phẳng phức.

Để tín hiệu điều khiển hồi tiếp tuyến tính hóa lý tưởng $u^*(x)$ định nghĩa ở biểu thức (8.10) xác định, ta cần các giả thiết sau:

Giả thiết 8.1 Hàm $b(x)$ có thỏa:

$$0 < b(x) < \infty, \quad \forall x \quad (8.16)$$

Giả thiết 8.2 Quỹ đạo chuẩn mong muốn $y_m(t)$ khả vi liên tục bị chặn đến bậc r và các đạo hàm $\dot{y}_m(t), \dots, y_m^{(r)}(t)$ có thể đo được.

Thay (8.10) vào (8.9) ta được:

$$y^{(r)} = v(t) = y_m^{(r)} + \bar{e}_s + \eta e_s \quad (8.17)$$

$$\Rightarrow e_o^{(r)} + \bar{e}_s + \eta e_s = 0 \quad (8.18)$$

Từ (8.14) và (8.18) ta suy ra đặc tính động của sai số bám là:

$$\dot{e}_s + \eta e_s = 0 \quad (8.19)$$

Chọn hàm Lyapunov:

$$V = \frac{1}{2} e_s^2 \quad (8.20)$$

Đạo hàm hàm Lyapunov theo thời gian và để ý (8.19) ta có:

$$\dot{V} = e_s \dot{e}_s = -\eta e_s^2 \leq 0 \quad (8.21)$$

Do $\dot{V} \leq 0$ nên hệ thống điều khiển hồi tiếp tuyến tính hóa ổn định, do đó $e_s(t) \rightarrow 0$ khi $t \rightarrow \infty$. Do (8.12) và (8.15) nên suy ra $e_o(t) \rightarrow 0$ khi $t \rightarrow \infty$, tức là tín hiệu ra $y(t)$ tiệm cận bám theo tín hiệu chuẩn $y_m(t)$ với sai số bằng 0.

Tuy nhiên do $a(x)$ và $b(x)$ chưa biết nên bộ điều khiển hồi tiếp tuyến tính hóa lý tưởng (8.10) không thể thực hiện được. Bài toán đặt ra là nhận dạng luật điều khiển (8.10). Có hai cách để giải bài toán trên: cách thứ nhất là nhận dạng trực tuyến $a(x)$, $b(x)$ và dựa vào kết quả nhận dạng đó để tính $u^*(x)$, cách này gọi là điều khiển thích nghi gián tiếp; cách thứ hai nhận dạng trực tiếp luật điều khiển lý tưởng $u^*(x)$, cách này gọi là điều khiển thích nghi trực tiếp. Các mục tiếp theo sẽ trình bày chi tiết về hai phương pháp điều khiển thích nghi vừa nêu.

8.3 ĐIỀU KHIỂN THÍCH NGHI GIÁN TIẾP

Sơ đồ điều khiển thích nghi gián tiếp (*Indirect Adaptive Control-IAC*) nhận dạng các hàm phi tuyến $\hat{a}(x)$ và $\hat{b}(x)$, sau đó tính luật điều khiển theo nguyên lý chắc chắn tương đương:

$$u_{ce} = \hat{b}^{-1}(x)[- \hat{a}(x) + v] \quad (8.22)$$

trong công thức trên $\hat{a}(x)$ và $\hat{b}(x)$ có thể là mô hình mờ hoặc mạng thần kinh nhân tạo. Tổng quát $\hat{a}(x)$ và $\hat{b}(x)$ có thể biểu diễn dưới dạng sau:

$$\hat{a}(x) = \theta_a^T \xi_a(x) \quad (8.23)$$

$$\hat{b}(x) = \theta_b^T \xi_b(x) \quad (8.24)$$

trong đó vector thông số θ_a và θ_b là giá trị hàm liên thuộc dạng vạch ở mệnh đề kết luận của hệ mờ hoặc trọng số của các tế bào thần kinh ở lớp ra của mạng thần kinh. Vector tham số θ_a và θ_b được cập nhật trực tuyến để tiến tiệm cận đến giá trị tối ưu:

$$\theta_a^* = \arg \min_{\theta_a \in \Omega_a} \left\{ \sup_{x \in S_X} \left| \theta_a^T \xi_a(x) - a(x) \right| \right\} \quad (8.25)$$

$$\theta_b^* = \arg \min_{\theta_b \in \Omega_b} \left\{ \sup_{x \in S_x} \left| \theta_b^T \xi_b(x) - b(x) \right| \right\} \quad (8.26)$$

trong đó Ω_a, Ω_b là tập hợp thông số chấp nhận được. Trong trường hợp lý tưởng, khi $\theta_a \rightarrow \theta_a^*, \theta_b \rightarrow \theta_b^*$ thì $\hat{a}(x) \rightarrow a(x), \hat{b}(x) \rightarrow b(x)$. Tuy nhiên, do chúng ta xấp xỉ $a(x)$ và $b(x)$ bằng hệ mờ gồm một số hữu hạn các quy tắc hoặc mạng thần kinh gồm một số hữu hạn tế bào ở lớp ẩn nên sai số xấp xỉ không thể tránh khỏi. Giả sử mô hình đúng của đối tượng có thể biểu diễn như sau:

$$\hat{a}(x) = \theta_a^{*T} \xi_a(x) + \delta_a(x) \quad (8.27)$$

$$\hat{b}(x) = \theta_b^{*T} \xi_b(x) + \delta_b(x) \quad (8.28)$$

trong đó $\delta_a(x)$ và $\delta_b(x)$ là sai số cấu trúc, tức là sai số xấp xỉ do mô hình mờ chỉ gồm một số hữu hạn các quy tắc hoặc mạng thần kinh chỉ có một số hữu hạn tế bào ở lớp ẩn. Sai lệch giữa mô hình nhận dạng được và mô hình đúng của hệ thống là:

$$\hat{a}(x) - a(x) = \tilde{\theta}_a^T(t) \xi_a(x) - \delta_a(x) \quad (8.29)$$

$$\hat{b}(x) - b(x) = \tilde{\theta}_b^T(t) \xi_b(x) - \delta_b(x) \quad (8.30)$$

trong đó:

$$\tilde{\theta}_a(t) = \theta_a(t) - \theta_a^* \quad (8.31)$$

$$\tilde{\theta}_b(t) = \theta_b(t) - \theta_b^* \quad (8.32)$$

Giả thiết mô hình mờ/mạng thần kinh xấp xỉ mô hình toán của đối tượng cho bởi biểu thức (8.23) và (8.24) được chọn sao cho sai số cấu trúc bị chặn bởi các cận đã biết trước, nghĩa là:

$$|\delta_f(\underline{x})| \leq \bar{\delta}_f(\underline{x}) \in L_\infty, \quad \forall \underline{x} \in S_x \quad (8.33)$$

$$|\delta_b(\underline{x})| \leq \bar{\delta}_b(\underline{x}) \in L_\infty, \quad \forall \underline{x} \in S_x \quad (8.34)$$

Sai số xấp xỉ không thể tránh khỏi, trong nhiều trường hợp sai số xấp xỉ dù nhỏ có thể làm cho hệ kín không ổn định. Để đảm bảo tính ổn định bền vững cho hệ kín, ta sử dụng thêm thành phần điều khiển chế độ trượt để bù sai số xấp xỉ. Do đó áp dụng luật điều khiển:

$$u = u_{ce} + u_{si} \quad (8.35)$$

trong đó thành phần điều khiển thích nghi theo nguyên lý chắc chắn tương đương u_{ce} được cập nhật trực tuyến gián tiếp qua $\hat{a}(x)$ và $\hat{b}(x)$ để tín hiệu ra của đối tượng bám theo quỹ đạo chuẩn và thành phần điều khiển chế độ trượt u_{si} được chọn để đảm bảo hệ kín ổn định. Để tính toán thành phần điều khiển chế độ trượt ta cần thêm các giả thiết sau:

Giả thiết 8.3 Giả thiết ta đã biết chặn trên và chặn dưới của $b(x)$, tức là:

$$0 < \underline{b}(x) \leq b(x) \leq \bar{b}(x) < \infty \quad (8.36)$$

Thiết kế bộ điều khiển IAC

Để độc giả tiện theo dõi, các bước thiết kế bộ điều khiển thích nghi được tóm tắt như sau:

- Viết biểu thức mô tả động học sai số bám.
- Chọn hàm Lyapunov V (là hàm toàn phương theo sai số bám và sai số thông số).
- Luật thích nghi thông số được chọn sao cho sai số thông số bị triệt tiêu khỏi đạo hàm theo thời gian ($\dot{V}$) của hàm Lyapunov.
- Thành phần điều khiển chế độ trượt được chọn sao cho đảm bảo đạo hàm theo thời gian của hàm Lyapunov âm ($\dot{V} < 0$) khi sai số cấu trúc nằm trong giới hạn định trước.
- Để kết luận về chất lượng tiệm cận của hệ thống, cần sử dụng bổ đề sau:

Bổ đề Barbalat: Nếu $\lim_{t \rightarrow \infty} \left(\int_0^t f(\tau) d\tau \right)$ tồn tại và hữu hạn, và $f(t)$

là hàm liên tục đều thì $\lim_{t \rightarrow \infty} f(t) = 0$.

Sau đây là chi tiết các bước thiết kế IAC:

Thay luật điều khiển (8.35) vào (8.9) và để ý (8.22) ta được:

$$\begin{aligned} y^{(r)} &= a'(x) + b(x)[u_{ce} + u_{si}] \\ &= \hat{a}(x) + \hat{b}(x)u_{ce} - [\hat{a}(x) - a(x)] - [\hat{b}(x) - b(x)]u_{ce} + b(x)u_{si} \\ &= v - [\hat{a}(x) - a(x)] - [\hat{b}(x) - b(x)]u_{ce} + b(x)u_{si} \end{aligned} \quad (8.37)$$

Từ (8.37), suy ra sai số ngõ ra là:

$$\begin{aligned}
 e_o^{(r)} &= y_m^{(r)} - y^{(r)} \\
 &= y_m^{(r)} - v + [\hat{a}(x) - a(x)] + [\hat{b}(x) - b(x)]u_{ce} - b(x)u_{si} \\
 &= -\eta e_s - \bar{e}_s + [\hat{a}(x) - a(x)] + [\hat{b}(x) - b(x)]u_{ce} - b(x)u_{si} \quad (8.38) \\
 &\quad (\text{do 8.11})
 \end{aligned}$$

Kết hợp (8.14) và (8.38) ta rút ra đặc tính động của sai số như sau:

$$\dot{e}_s + \eta e_s = (\hat{a}(x) - a(x)) + (\hat{b}(x) - b(x))u_{ce} - b(x)u_{si} \quad (8.39)$$

Xét hàm Lyapunov:

$$V = \frac{1}{2}e_s^2 + \frac{1}{2}\tilde{\theta}_a^T Q_a \tilde{\theta}_a + \frac{1}{2}\tilde{\theta}_b^T Q_b \tilde{\theta}_b \quad (8.40)$$

trong đó Q_a , Q_b là ma trận trọng số xác định dương.

Đạo hàm hàm V theo thời gian, để ý $\dot{\tilde{\theta}}_a = \dot{\theta}_a$ và $\dot{\tilde{\theta}}_b = \dot{\theta}_b$ ta được:

$$\dot{V} = e_s \dot{e}_s + \tilde{\theta}_a^T Q_a \dot{\theta}_a + \tilde{\theta}_b^T Q_b \dot{\theta}_b \quad (8.41)$$

Do (8.39), suy ra:

$$\begin{aligned}
 \dot{V} &= -\eta e_s^2 + e_s \left[(\hat{a} - a) + (\hat{b} - b)u_{ce} - bu_{si} \right] + \\
 &\quad + \tilde{\theta}_a^T Q_a \dot{\theta}_a + \tilde{\theta}_b^T Q_b \dot{\theta}_b
 \end{aligned} \quad (8.42)$$

Thay (8.29) và (8.30) vào (8.42) ta được:

$$\begin{aligned}
 \dot{V} &= -\eta e_s^2 + e_s \left[(\tilde{\theta}_a^T \xi_a - \delta_a) + (\tilde{\theta}_b^T \xi_b - \delta_b)u_{ce} - bu_{si} \right] \\
 &\quad + \tilde{\theta}_a^T Q_a \dot{\theta}_a + \tilde{\theta}_b^T Q_b \dot{\theta}_b \\
 &= -\eta e_s^2 - bu_{si}e_s + e_s(-\delta_a - \delta_b u_{ce}) + \tilde{\theta}_a^T (Q_a \dot{\theta}_a + \xi_a e_s) \\
 &\quad + \tilde{\theta}_b^T (Q_b \dot{\theta}_b + \xi_b e_s u_{ce})
 \end{aligned} \quad (8.43)$$

Chọn luật thích nghi thông số như sau:

$$\dot{\theta}_a = -Q_a^{-1} \xi_a e_s \quad (8.44)$$

$$\dot{\theta}_b = -Q_b^{-1} \xi_b u_{ce} e_s \quad (8.45)$$

Thay (8.44) và (8.45) vào (8.43), ta được:

$$\begin{aligned}\dot{V} &= -\eta e_s^2 - bu_{si}e_s + e_s(-\delta_a - \delta_b u_{ce}) \\ &\leq -\eta e_s^2 - bu_{si}e_s + |e_s|(|\delta_a| + |\delta_b| |u_{ce}|) \\ &\leq -\eta e_s^2 - bu_{si}e_s + |e_s|(\bar{\delta}_a + \bar{\delta}_b |u_{ce}|)\end{aligned}\quad (8.46)$$

Chọn thành phần điều khiển chế độ trượt như sau:

$$u_{si} = \frac{1}{b}(\bar{\delta}_a + \bar{\delta}_b |u_{ce}|)\text{sgn}(e_s) \quad (8.47)$$

Thay (8.47) vào (8.46), để ý $e_s \text{sgn}(e_s) = |e_s|$, ta được:

$$\dot{V} \leq -\eta e_s^2 - \left(\frac{b}{b} - 1\right) |e_s| [\bar{\delta}_a + \bar{\delta}_b |u_{ce}|] \leq 0 \quad (8.48)$$

Do V là hàm toàn phương bán xác định dương và $\dot{V} \leq 0$ nên hệ thống ổn định. Từ đây ta có thể rút ra được các kết luận sau:

i) Do V bán xác định dương và đạo hàm theo thời gian bán xác định âm nên $V \in L_\infty$, do đó $e_s \in L_\infty$. Từ biểu thức định nghĩa e_s ta có $e_o^{(j)} = G_j(s)e_s$ ($j = 1, \dots, r-1$), trong đó $G_j(s) = s^j \Delta(s)$. Do $\Delta(s)$ có tất cả các cực nằm bên trái mặt phẳng phức nên $G_j(s)$ ổn định, do đó $e_o^{(j)} \in L_\infty$, $j = 1, \dots, r-1$. Do $e_o^{(j)} \in L_\infty$ ($j = 1, \dots, r-1$) và do giả thiết tín hiệu chuẩn có đạo hàm bị chặn đến bậc $r-1$ nên suy ra $y, \dots, y^{(r-1)} \in L_\infty$.

ii) Từ biểu thức (8.48) ta có:

$$\int_0^\infty \eta e_s^2 dt \leq -\int_0^\infty \dot{V} dt = V(0) - V(\infty) < \infty \quad (8.49)$$

Biểu thức trên chứng tỏ $e_s \in L_2$. Do ở trên ta đã chứng minh được:

$$e_o^{(j)} \in L_\infty, \quad j = 1, \dots, r-1$$

nên suy ra $\dot{e}_s \in L_\infty$, do đó e_s liên tục đều.

Vì $e_s \in L_2 \cap L_\infty$ và $\dot{e}_s \in L_\infty$ nên theo bổ đề Barbalat ta có:

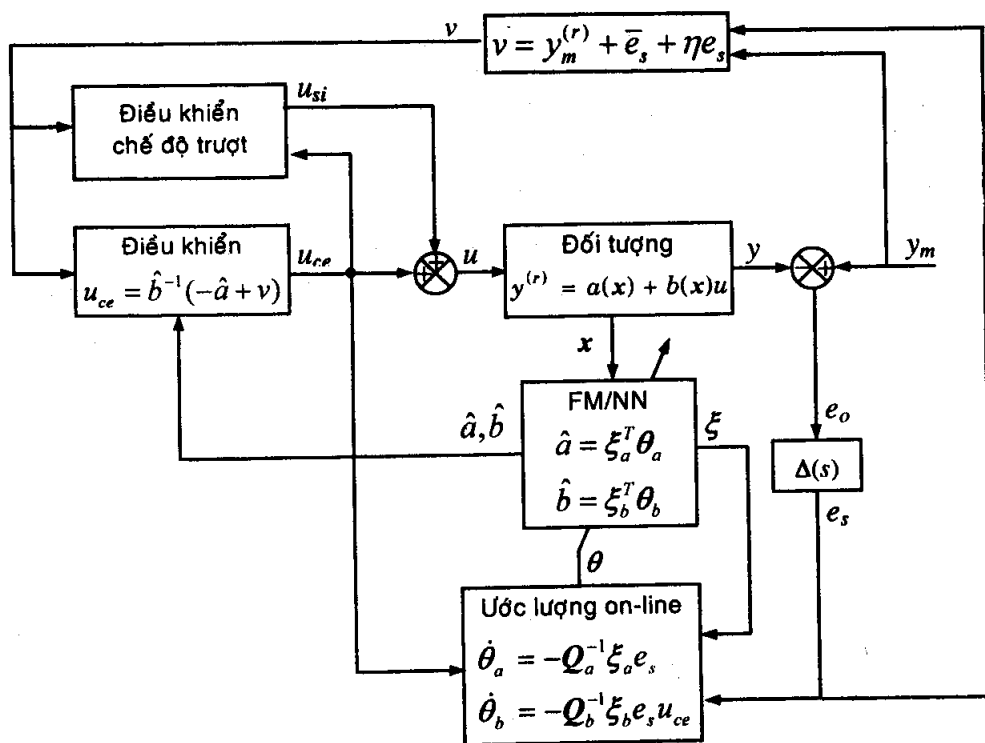
$$\lim_{t \rightarrow \infty} |e_s(t)| = 0$$

Do đó nên:

$$\lim_{t \rightarrow \infty} |e_o(t)| = 0$$

tức là sai số ngõ ra tiệm cận suy giảm về 0.

Hệ thống điều khiển thích nghi gián tiếp vừa thiết kế có sơ đồ khối trình bày ở hình 8.2.



Hình 8.2 Sơ đồ khối hệ thống điều khiển thích nghi gián tiếp

Chú ý:

- Trọng số của hàm Lyapunov có ảnh hưởng rất lớn đến chất lượng đáp ứng của hệ thống. Nếu $\|Q_a\|$, $\|Q_b\|$ lớn thì $\|Q_a^{-1}\|$, $\|Q_b^{-1}\|$ nhỏ, điều này dẫn đến hệ thống thích nghi chậm. Ngược lại nếu $\|Q_a\|$, $\|Q_b\|$ nhỏ thì $\|Q_a^{-1}\|$, $\|Q_b^{-1}\|$ lớn, kết quả là hệ thống thích nghi nhanh hơn, tuy nhiên trong trường hợp này hệ thống cũng sẽ trở nên rất nhạy với nhiễu. Vì vậy trong các ứng dụng cần chọn trọng số hàm Lyapunov thích hợp thì chất lượng của hệ thống mới thỏa mãn yêu cầu.

- Hệ số η quyết định tốc độ suy giảm về 0 của sai số bám $e_s(t)$, các hệ số k_i quyết định tốc độ hội tụ của sai số ngõ ra $e_o(t)$. Giá trị của các thông số này cũng ảnh hưởng lớn đến chất lượng đáp ứng của hệ thống.

- Thành phần điều khiển chế độ trượt đảm bảo tính ổn định của hệ thống khi có sai số cấu trúc của mô hình. Để tính toán thành phần điều khiển này phải biết cận trên, cận dưới của đặc tính động học của đối tượng. Thực tế, các cận này không phải lúc nào cũng có thể xác định được dễ dàng. Nếu các cận chọn quá nhỏ sẽ dẫn đến tín hiệu ổn định không được đảm bảo, ngược lại nếu các cận được chọn quá lớn sẽ dẫn đến hiện tượng *chattering* trong hệ thống.

Ví dụ 8.1 Điều khiển mờ thích nghi gián tiếp hệ bồn chứa nối tiếp

Xét hệ bồn chứa nối tiếp có mô hình toán trình bày ở ví dụ 5.11. Bài toán đặt ra là điều khiển mực chất lỏng trong bồn chứa bám theo tín hiệu chuẩn khi tiết diện của van xả thay đổi.

Hệ bồn nối tiếp là hệ phi tuyến SISO bậc 2. Đặt $x(t) = [h_1(t), h_2(t)]^T$ là vector trạng thái của hệ thống, từ (5.47)–(5.49) ta có thể viết lại phương trình trạng thái mô tả hệ thống dưới dạng:

$$\begin{cases} \dot{x}(t) = f(x) + g(x).u(t) \\ y(t) = h(x) \end{cases} \quad (8.50)$$

trong đó:

$$f(x) = \begin{bmatrix} \frac{(-\operatorname{sgn}(x_1(t) - x_2(t))C_D a_1 \sqrt{2g |x_1(t) - x_2(t)|})h_{\max}}{(A_{\max} - A_{\min})x_1(t) + A_{\min}h_{\max}} \\ \frac{(\operatorname{sgn}(x_1(t) - x_2(t))C_D a_1 \sqrt{2g |x_1(t) - x_2(t)|} - C_D a_2 \sqrt{2gx_2(t)})h_{\max}}{(A_{\max} - A_{\min})x_2(t) + A_{\min}h_{\max}} \end{bmatrix} \quad (8.51)$$

$$g(x) = \begin{bmatrix} \frac{kh_{\max}}{(A_{\max} - A_{\min})x_1(t) + A_{\min}h_{\max}} \\ 0 \end{bmatrix} \quad (8.52)$$

$$h(x) = x_2(t) \quad (8.53)$$

Lấy đạo hàm tín hiệu ra $y(t)$ hai lần, ta được:

$$\ddot{y} = a(x) + b(x)u \quad (8.54)$$

$$\text{trong đó: } a(x) = L_f^2 h(x) \quad (8.55)$$

$$b(x) = L_g L_f h(x) \quad (8.56)$$

Luật điều khiển hồi tiếp tuyến tính hóa là:

$$u^*(x) = b^{-1}(x)[-a(x) + v(t)] \quad (8.57)$$

$$\text{trong đó: } v(t) = \ddot{y}_m(t) + \bar{e}_s(t) + \eta e_s(t) \quad (8.58)$$

$$e_s(t) = \dot{e}_o(t) + k_1 e_o(t) \quad (8.59)$$

$$\bar{e}_s(t) = \dot{e}_s(t) - \ddot{e}_o(t) = k_1 \dot{e}_o(t) \quad (8.60)$$

$$e_o(t) = y_m(t) - y(t) \quad (8.61)$$

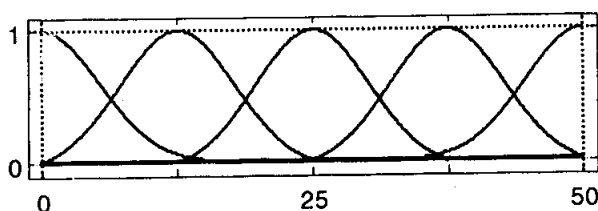
Giả sử ta không biết chính xác $f(x)$, $g(x)$ và $h(x)$ nên không thể tính được $a(x)$ và $b(x)$, do đó không thể thực thi được luật điều khiển (8.57). Trong ví dụ này ta sử dụng các mô hình mờ để nhận dạng trực tuyến $\hat{a}(x)$ và $\hat{b}(x)$, sau đó tính luật điều khiển hồi tiếp tuyến tính hóa theo nguyên lý chắc chắn tương đương.

Thiết kế bộ điều khiển mờ thích nghi gián tiếp

Mô hình mờ $\hat{a}(x)$ xấp xỉ hàm $a(x)$ được định nghĩa gồm các quy tắc có dạng tổng quát như sau:

$$\text{Nếu } (x_1 \text{ là } \tilde{A}_{1i}) \text{ và } (x_2 \text{ là } \tilde{A}_{2i}) \text{ thì } \hat{a}(x) = \theta_{ai} \quad (8.62)$$

trong đó x_1 và x_2 là các biến trạng thái của hệ bốn nối tiếp, $\tilde{A}_{1i}$ và $\tilde{A}_{2i}$ tương ứng là các tập mờ biểu diễn các giá trị ngôn ngữ của các biến trạng thái x_1 và x_2 ở quy tắc thứ i . Các tập mờ mô tả các trị ngôn ngữ của biến trạng thái x_1 và x_2 có hàm liên thuộc dạng Gauss, số tập mờ cho mỗi biến vào được chọn bằng 5, xem hình 8.3. Như vậy hệ mờ (8.62) gồm có tất cả là 25 quy tắc.



Hình 8.3 Các tập mờ dùng ở mệnh đề điều kiện của các hệ mờ

Với các quy tắc mờ và hàm liên thuộc định nghĩa như trên, sử dụng phương pháp giải mờ trung bình có trọng số, mô hình mờ $\hat{a}(x)$ xác định bởi biểu thức:

$$\hat{a}(x) = \theta_a^T \xi_a(x) \quad (8.63)$$

trong đó: $\theta_a = [\theta_{a,1} \quad \theta_{a,2} \quad \dots \quad \theta_{a,25}]^T \quad (8.64)$

$$\xi_a(x) = [\xi_{a,1}(x) \quad \xi_{a,2}(x) \quad \dots \quad \xi_{a,25}(x)]^T \quad (8.65)$$

$$\xi_{ai}(x) = \frac{\mu_{\tilde{A}_{1i}}(x_1) \cdot \mu_{\tilde{A}_{2i}}(x_2)}{\sum_{i=1}^{25} \mu_{\tilde{A}_{1i}}(x_1) \cdot \mu_{\tilde{A}_{2i}}(x_2)} \quad (8.66)$$

Mô hình mờ $\hat{b}(x)$ cũng được định nghĩa tương tự như trên. Để tăng tốc độ tính toán của thuật toán điều khiển thích nghi ta có thể chọn mệnh đề điều kiện của cả hai hệ mờ $\hat{a}(x)$ và $\hat{b}(x)$ trong sơ đồ IAFC như nhau.

Hình 8.4 là sơ đồ Simulink mô phỏng hệ thống điều khiển thích nghi gián tiếp hệ bền chứa nối tiếp. Chức năng của các khối trong sơ đồ như sau:

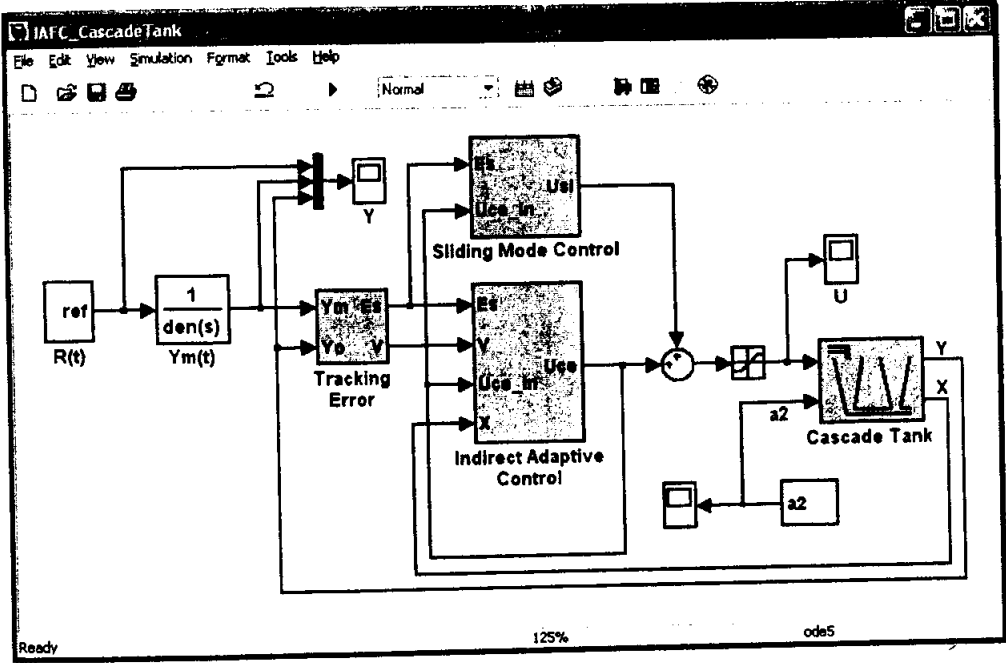
- Quỹ đạo chuẩn $y_m(t)$ được tạo ra bằng cách cho tín hiệu đặt $r(t)$ qua mô hình chuẩn tuyến tính bậc 2:

$$\frac{Y_m(s)}{R(s)} = \frac{1}{(20s + 1)^2} \quad (8.67)$$

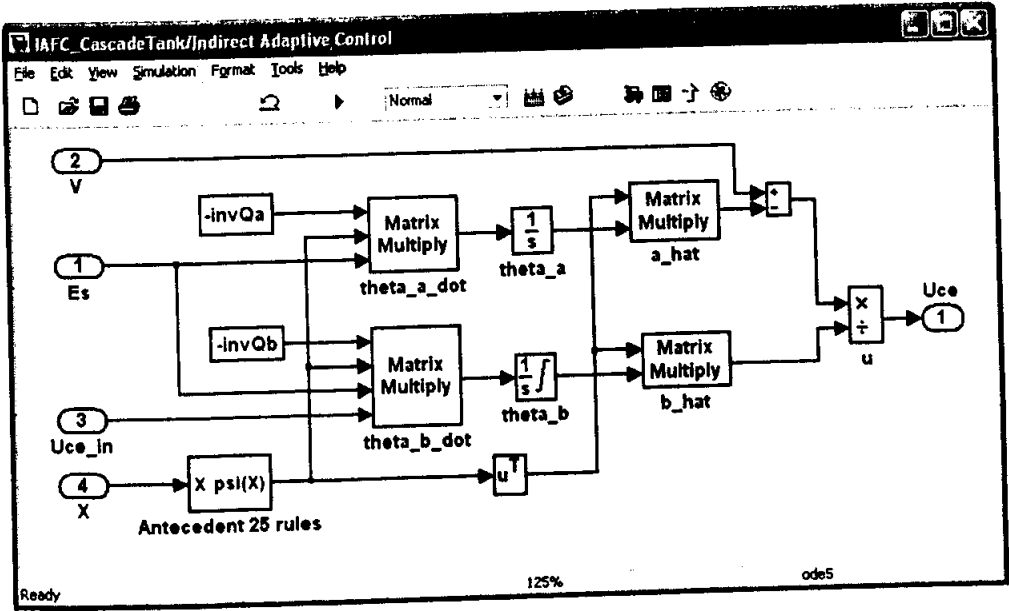
- Khối [**Tracking Error**] tính sai số bám và tín hiệu $v(t)$ theo các công thức (8.58) ÷ (8.61), trong đó $k_1 = 0,5$, $\eta = 5$.

- Khối [**Indirect Adaptive Control**] (H.8.5) tính toán thành phần điều khiển thích nghi gián tiếp (8.22), trong đó thông số của mô hình được cập nhật trực tuyến theo luật thích nghi (8.44) và (8.45) với $Q_a = 10I$, $Q_b = 20I$; khối [**Antecedent 25 rules**] tính toán độ đúng của mệnh đề điều kiện của 25 quy tắc mờ theo các công thức (8.65) và (8.66) (H.8.7), trong đó hàm liên thuộc của các tập mờ định nghĩa ở hình 8.3.

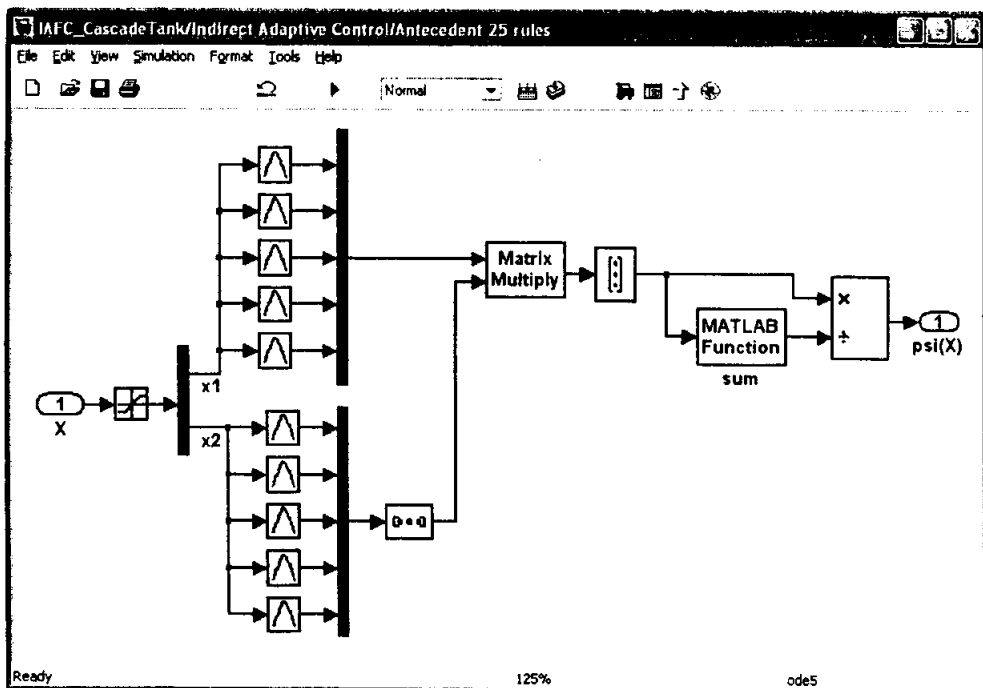
- Khối [**Sliding Mode Control**] (H.8.7) tính thành phần điều khiển chế độ trượt theo công thức (8.47). Để ý rằng thành phần điều khiển chế độ trượt chỉ có tác dụng khi [**Sliding Control Enable**] bằng 1.



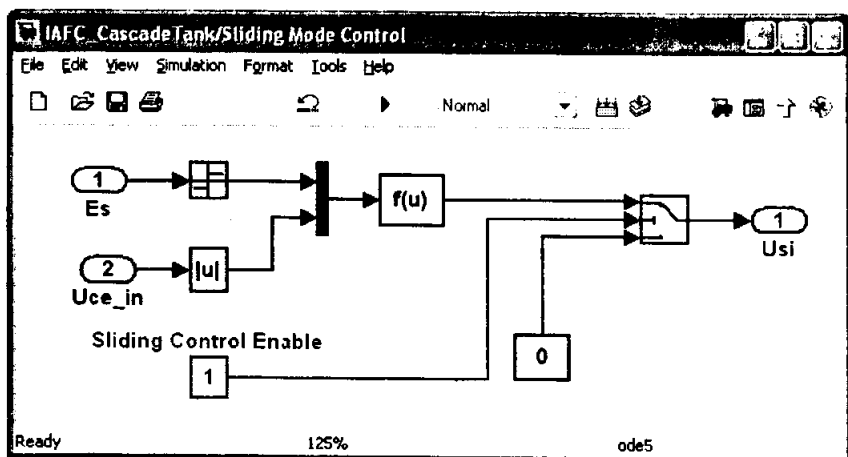
Hình 8.4 Sơ đồ mô phỏng hệ thống IAFC hệ bồn nối tiếp



Hình 8.5 Thành phần điều khiển thích nghi gián tiếp



Hình 8.6 Mệnh đề điều kiện của hệ mờ dùng trong sơ đồ IAFC

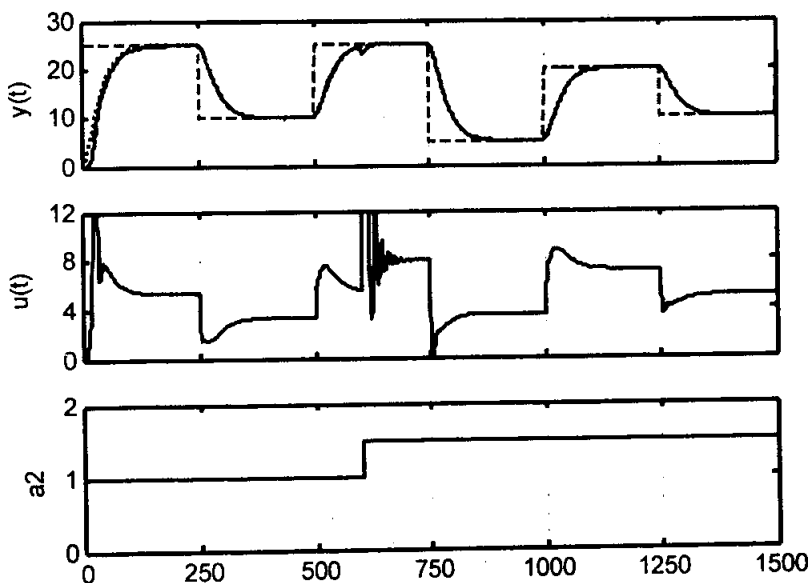


Hình 8.7 Thành phần điều khiển chế độ trượt dùng trong sơ đồ IAFC

Kết quả điều khiển

Hình 8.8 là đồ thị đáp ứng của hệ thống điều khiển mờ thích nghi bồn chứa nối tiếp vừa thiết kế. Ban đầu tín hiệu ra chưa bám tốt theo tín hiệu chuẩn, qua quá trình thông số được cập nhật trực tuyến tín hiệu ra của bồn chứa tiệm cận dần đến tín hiệu chuẩn. Tại thời

điểm tại thời điểm $t = 600\text{sec}$ tiết diện van xả bồn 2 tăng từ $1,0\text{cm}^2$ lên $1,5\text{cm}^2$, mực chất lỏng trong bồn bị tụt xuống nhưng sau đó nhanh chóng bám theo tín hiệu chuẩn nhờ thông số của mô hình được cập nhật trực tuyến.



Hình 8.8 Kết quả điều khiển mực chất lỏng hệ bồn nối tiếp dùng IAC

8.4 ĐIỀU KHIỂN THÍCH NGHI TRỰC TIẾP

Trong sơ đồ điều khiển mờ thích nghi trực tiếp (*Direct Adaptive Control-DAC*) luật điều khiển hồi tiếp tuyến tính hóa lý tưởng (8.10) được xấp xỉ bằng hệ mờ hoặc mạng thần kinh, tổng quát luật điều khiển có thể biểu diễn dưới dạng:

$$\hat{u}(x) = \theta_u^T \xi_u(x) \quad (8.68)$$

trong đó vector thông số θ_u là giá trị hàm liên thuộc dạng vạch ở mệnh đề kết luận của hệ mờ hoặc trọng số của các tế bào thần kinh ở lớp ra của mạng thần kinh, θ_u được cập nhật trực tuyến để $\hat{u}$ tiệm cận tiến đến theo u^* . Vector thông số tối ưu là:

$$\theta_u^* = \arg \min_{\theta_u \in \Theta} \left\{ \sup_{x \in S_x} |\theta_u^T \xi_u(x) - u^*| \right\} \quad (8.69)$$

Do ta xấp xỉ luật điều khiển lý tưởng u^* bằng một hệ mờ có số quy tắc hữu hạn hoặc mạng thần kinh có số tế bào ở lớp ẩn hữu hạn nên trong trường hợp tổng quát sai số cấu trúc không thể tránh khỏi, tức là khi $\theta_u \rightarrow \theta_u^*$ thì $\hat{u}$ cũng không thể đồng nhất bằng u^* . Gọi $\delta_u(x)$ là sai số cấu trúc, luật điều khiển lý tưởng có thể biểu diễn bằng tổng của thành phần có thể mô hình hóa và thành phần không thể mô hình hóa như sau:

$$u^*(x) = \theta_u^{*T} \xi_u(x) + \delta_u(x) \quad (8.70)$$

Sai lệch giữa luật điều khiển nhận dạng được và luật điều khiển lý tưởng là:

$$\tilde{u}(x) - u^*(x) = \tilde{\theta}_u^T \xi_u(x) - \delta_u(x) \quad (8.71)$$

trong đó $\tilde{\theta}_u$ là sai số thông số ước lượng:

$$\tilde{\theta}_u = \theta_u - \theta_u^* \quad (8.72)$$

Giả thiết NN/FM xấp xỉ luật điều khiển được chọn sao cho ta đã biết giới hạn của sai số cấu trúc, tức là tồn tại $\bar{\delta}_u(x)$ là hàm liên tục sao cho $|\delta_u(x)| \leq \bar{\delta}_u(x)$, với mọi x . Do có sai số cấu trúc nên để đảm bảo hệ kín ổn định ta xét luật điều khiển:

$$u = \hat{u} + u_{sd} \quad (8.73)$$

trong đó $\hat{u}$ là thành phần thích nghi trực tiếp xấp xỉ tín hiệu điều khiển lý tưởng và u_{sd} là thành phần điều khiển chế độ trượt được chọn sao cho đảm bảo hệ kín ổn định. Để tính toán thành phần điều khiển chế độ trượt, ngoài giả thiết (8.36) $b(x)$ cần thỏa mãn giả thiết sau:

Giả thiết 8.4 Giả thiết tốc độ biến thiên của $b(x)$ bị chặn, tức là tồn tại $D_b(x)$ là hàm liên tục sao cho:

$$|\dot{b}(x)| \leq D_b(x) \quad (8.74)$$

Thiết kế bộ điều khiển DAC

Trình tự thiết kế DAC tương tự như thiết kế IAC, cụ thể như sau:

Áp dụng luật điều khiển (8.73) vào hệ thống (8.9) ta được:

$$\begin{aligned} y^{(r)}(t) &= a(x) + b(x)u(t) \\ &= a(x) + b(x)u^*(t) + b(x)[u(t) - u^*(t)] \\ &= v + b(x)[u(t) - u^*(t)] \quad (\text{do 8.10}) \end{aligned} \quad (8.75)$$

Suy ra sai số ngõ ra là:

$$\begin{aligned}
 e_o^{(r)} &= y_m^{(r)} - y^{(r)} \\
 &= y_m^{(r)} - v - b(u - u^*) \\
 &= -\bar{e}_s - \eta e_s - b(\hat{u} + u_{sd} - u^*) \\
 &= -\bar{e}_s - \eta e_s - b\tilde{\theta}_u^T \xi_u + b\delta_u - bu_{sd}
 \end{aligned} \tag{8.76}$$

Kết hợp (8.14) và (8.76) ta rút ra đặc tính động của sai số như sau:

$$\dot{e}_s + \eta e_s = -b\tilde{\theta}_u^T \xi_u + b\delta_u - bu_{sd} \tag{8.77}$$

Xét hàm dạng toàn phương:

$$V = \frac{1}{2b} e_s^2 + \frac{1}{2} \tilde{\theta}_u^T Q_u \tilde{\theta}_u \tag{8.78}$$

trong đó Q_u là ma trận trọng số xác định dương.

Đạo hàm hàm V theo thời gian, để ý $\dot{\tilde{\theta}}_u = \dot{\theta}_u$ ta được:

$$\dot{V} = \frac{1}{b} e_s \dot{e}_s - \frac{\dot{b}}{2b^2} e_s^2 + \tilde{\theta}_u^T Q_u \dot{\theta}_u \tag{8.79}$$

Thay (8.77) vào (8.79) ta được:

$$\begin{aligned}
 \dot{V} &= \frac{e_s}{b} (-\eta e_s - b\tilde{\theta}_u^T \xi_u + b\delta_u - bu_{sd}) - \frac{\dot{b}}{2b^2} e_s^2 + \tilde{\theta}_u^T Q_u \dot{\theta}_u \\
 &= -\frac{\eta e_s^2}{b} - e_s u_{sd} + e_s \delta_u + \tilde{\theta}_u^T (Q_u \dot{\theta}_u - \xi_u e_s) - \frac{\dot{b}}{2b^2} e_s^2
 \end{aligned} \tag{8.80}$$

Chọn luật thích nghi thông số như sau:

$$\dot{\theta}_u = Q_u^{-1} \xi_u e_s \tag{8.81}$$

Thay luật thích nghi (8.81) vào (8.80) ta được:

$$\begin{aligned}
 \dot{V} &= -\frac{\eta e_s^2}{b} - e_s u_{sd} + e_s \delta_u - \frac{\dot{b}}{2b^2} e_s^2 \\
 &\leq -\frac{\eta e_s^2}{b} - u_{sd} e_s + |e_s| \left(|\delta_u| + \frac{|\dot{b}|}{2b^2} |e_s| \right) \\
 &\leq -\frac{\eta e_s^2}{b} - u_{sd} e_s + |e_s| \left(\bar{\delta}_u + \frac{D_b}{2b^2} |e_s| \right)
 \end{aligned} \tag{8.82}$$

Chọn thành phần điều khiển chế độ trượt như sau:

$$u_{sd} = \left(\bar{\delta}_u + \frac{D_b}{2b^2} |e_s| \right) \text{sgn}(e_s) \quad (8.83)$$

Thay (8.83) vào (8.82), để ý $e_s \text{sgn}(e_s) = |e_s|$ ta được:

$$\dot{V} \leq -\frac{\eta e_s^2}{b} \leq 0 \quad (8.84)$$

Do V là hàm toàn phương bán xác định dương và $\dot{V} \leq 0$ nên hệ thống ổn định. Từ đây ta rút ra các kết luận sau:

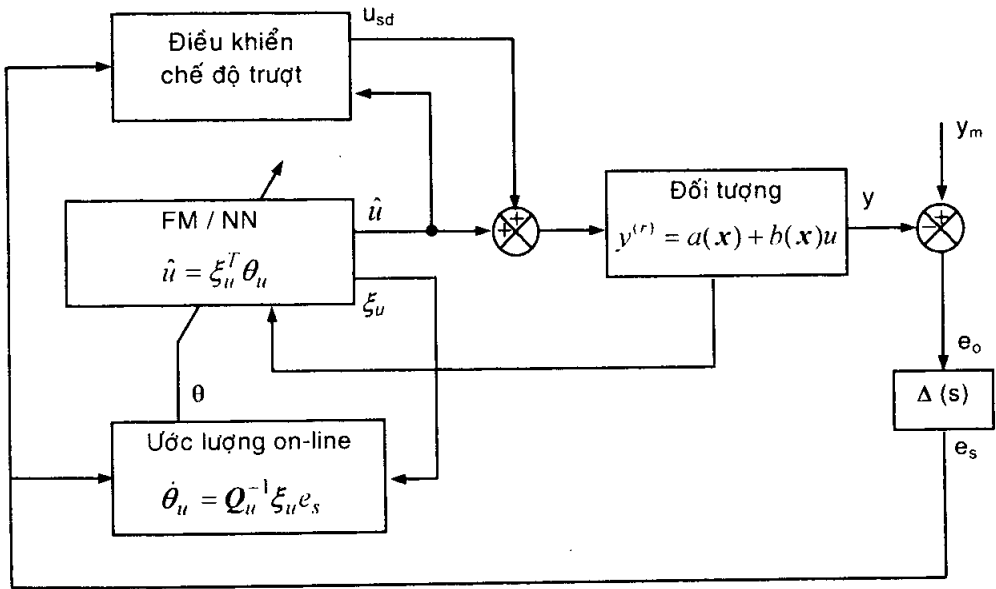
(i) Do V bán xác định dương và đạo hàm theo thời gian bán xác định âm nên điều đó có nghĩa là $V \in L_\infty$, do đó $e_s \in L_\infty$ và $\|\tilde{\theta}_u\| \in L_\infty$. Từ biểu thức định nghĩa e_s ta có $e_o^{(j)} = G_j(s)e_s$, trong đó $G_j(s) = s^j/\Delta(s)$, $j = 1, \dots, r-1$. Do $\Delta(s)$ có tất cả các cực nằm bên trái mặt phẳng phức nên $G_j(s)$ ổn định, do đó $e_o^{(j)} \in L_\infty$, $j = 1, \dots, r-1$. Do $e_o^{(j)} \in L_\infty$ ($j = 1, \dots, r-1$) và giả thiết đạo hàm của tín hiệu chuẩn bị chặn nên suy ra $y, \dot{y}, \dots, y^{(r-1)} \in L_\infty$.

(ii) Từ biểu thức (8.84) ta có:

$$\int_0^\infty \frac{\eta e_s^2}{b} dt \leq -\int_0^\infty \dot{V} dt = V(0) - V(\infty) < \infty \quad (8.85)$$

Biểu thức trên chứng tỏ $e_s \in L_2$. Do ở trên ta đã chứng minh được $e_o^{(j)} \in L_\infty$ nên suy ra $\dot{e}_s \in L_\infty$, do đó e_s liên tục đều. Vì $e_s \in L_2 \cap L_\infty$ và $\dot{e}_s \in L_\infty$ nên theo bổ đề Barbalat ta có $\lim_{t \rightarrow \infty} e_s = 0$ do đó nên $\lim_{t \rightarrow \infty} e_o(t) = 0$.

Sơ đồ khối hệ thống DAC minh họa ở hình 8.9. Ta thấy khác với trường hợp IAC, trong sơ đồ DAC thông số của bộ điều khiển mờ/thần kinh được chỉnh định trực tiếp mà không cần phải nhận dạng mô hình toán của đối tượng, chính điều này làm cho bộ điều khiển DAC dễ chỉnh định và ít nhạy với nhiễu hơn bộ điều khiển IAC.



Hình 8.9 Sơ đồ khối hệ thống điều khiển thích nghi trực tiếp

Ví dụ 8.2 Điều khiển thần kinh thích nghi trực tiếp hệ tay máy một bậc tự do.

Xét hệ tay máy một bậc tự do có mô hình toán học đã trình bày ở ví dụ 5.4. Ở ví dụ này ta sẽ thiết kế hệ thống điều khiển thích nghi trực tiếp dùng mạng thần kinh để điều khiển vị trí của cánh tay máy bám theo tín hiệu đặt cho trước, giả sử trong quá trình làm việc khối lượng của tải mà cánh tay máy gấp có khối lượng thay đổi.

Hệ tay máy một bậc tự do là hệ phi tuyến SISO bậc 2. Đặt tín hiệu ra $y(t) = \theta(t)$ và vector trạng thái là $x(t) = [x_1(t), x_2(t)]^T = [\theta(t), \dot{\theta}(t)]^T$, có thể viết lại phương trình (5.17) dưới dạng:

$$\ddot{y} = a(x) + b(x)u \quad (8.86)$$

$$\text{trong đó: } a(x) = \frac{-Bx_2(t) - (ml + Ml_C)g \cos x_1(t)}{(J + ml^2)} \quad (8.87)$$

$$b(x) = \frac{1}{(J + ml^2)}. \quad (8.88)$$

Thiết kế bộ điều khiển thích nghi trực tiếp

Sử dụng mạng thần kinh RBF với 9 hàm cơ sở ở lớp ẩn để xấp xỉ trực tiếp tín hiệu điều khiển hồi tiếp tuyến tính hóa lý tưởng (H.8.10):

$$\hat{u}(\mathbf{x}) = \theta_u^T \xi_u(\mathbf{x}) \quad (8.89)$$

$$\xi_u(\mathbf{x}) = [\xi_{u1}(\mathbf{x}) \quad \xi_{u2}(\mathbf{x}) \quad \dots \quad \xi_{u9}(\mathbf{x})]^T \quad (8.90)$$

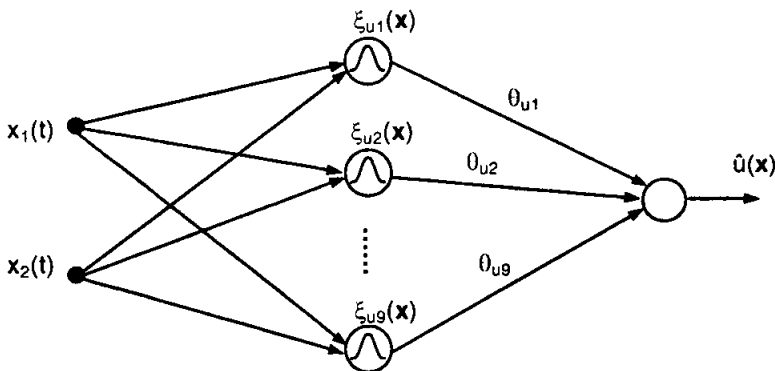
$$\theta = [\theta_{u1} \quad \theta_{u2} \quad \dots \quad \theta_{u9}]^T \quad (8.91)$$

trong đó hàm cơ sở thứ i được định nghĩa như sau:

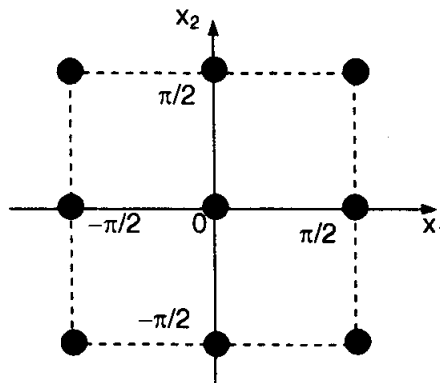
$$\xi_{ui}(\mathbf{x}) = e^{-\frac{[(x_1 - \mu_{1i})^2 + (x_2 - \mu_{2i})^2]}{\sigma_i^2}} \quad (8.92)$$

Các hàm cơ sở được chọn phân bố đều trong không gian trạng thái (H.8.11). Độ phân tán của các hàm cơ sở được chọn bằng nhau:

$$\sigma_i = 0,6 \quad (i = 1 \dots 9) \quad (8.93)$$



Hình 8.10 Mạng RBF xấp xỉ luật điều khiển hồi tiếp tuyến tính hóa



Hình 8.11 Vị trí tâm hàm cơ sở của mạng RBF

Hình 8.12 là sơ đồ Simulink mô phỏng hệ thống điều khiển thần kinh RBF thích nghi cánh tay máy một bậc tự do. Trong sơ đồ các khối con có chức năng như sau:

- Quỹ đạo chuẩn $y_m(t)$ được tạo ra bằng cách cho tín hiệu đặt $r(t)$ qua mô hình chuẩn tuyến tính bậc 2:

$$\frac{Y_m(s)}{R(s)} = \frac{1}{(0,5s + 1)^2}$$

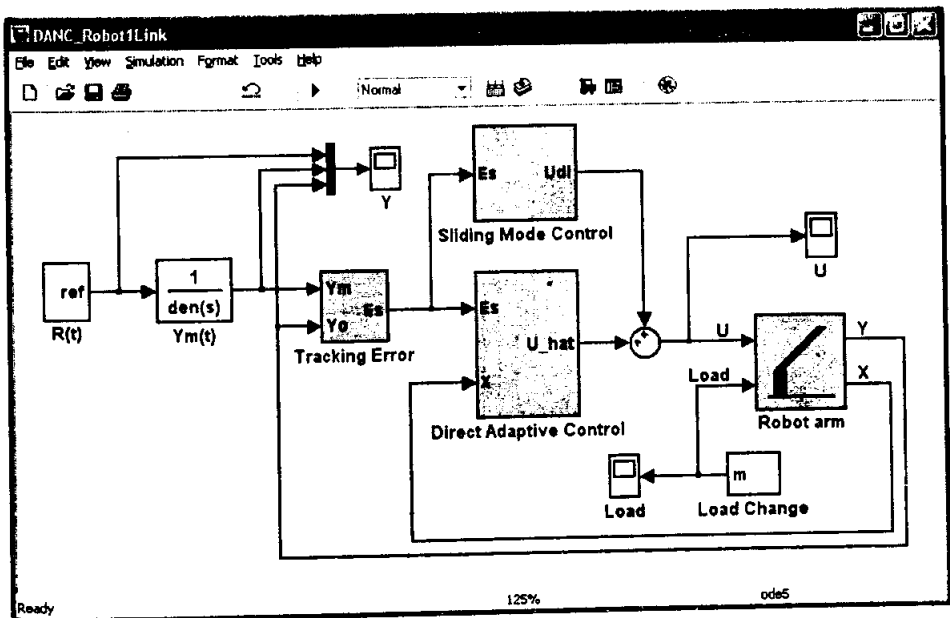
- Khối [**Tracking Error**] tính sai số bám theo các công thức sau:

$$e_s(t) = \dot{e}_o(t) + k_1 e_o(t) \quad (k_1 = 2) \quad (8.94)$$

$$e_o(t) = y_m(t) - y(t) \quad (8.95)$$

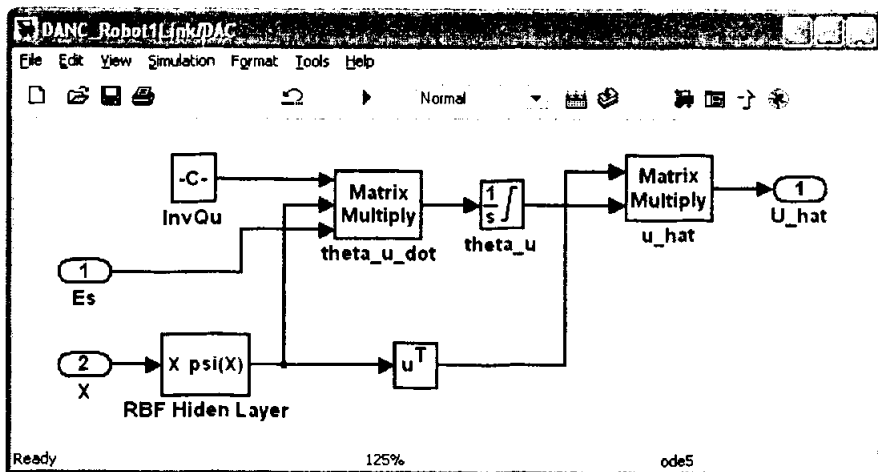
$$\bar{e}_s(t) = \dot{e}_s(t) - \ddot{e}_o(t) = k_1 \dot{e}_o(t) \quad (8.95)$$

$$e_o(t) = y_m(t) - y(t) \quad (8.96)$$

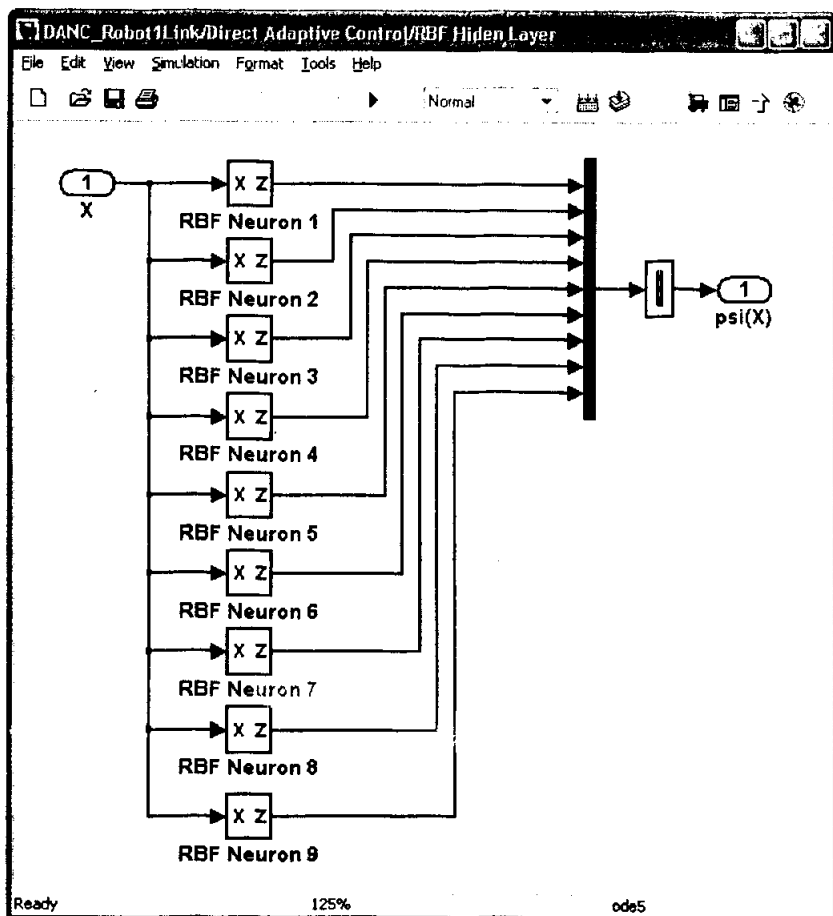


Hình 8.12 Sơ đồ mô phỏng hệ thống điều khiển thần kinh RBF thích nghi trực tiếp

- Khối [**Direct Adaptive Control**] (H.8.13) tính toán thành phần điều khiển thích nghi trực tiếp (8.89), trong đó thông số của bộ điều khiển được cập nhật trực tuyến theo luật thích nghi (8.81) với $Q_u = 0,02I$, khối [**RBF Hiden Layer**] (H.8.14) gồm 9 hàm cơ sở (8.92), mỗi hàm cơ sở được thực thi bằng sơ đồ ở hình 8.15.

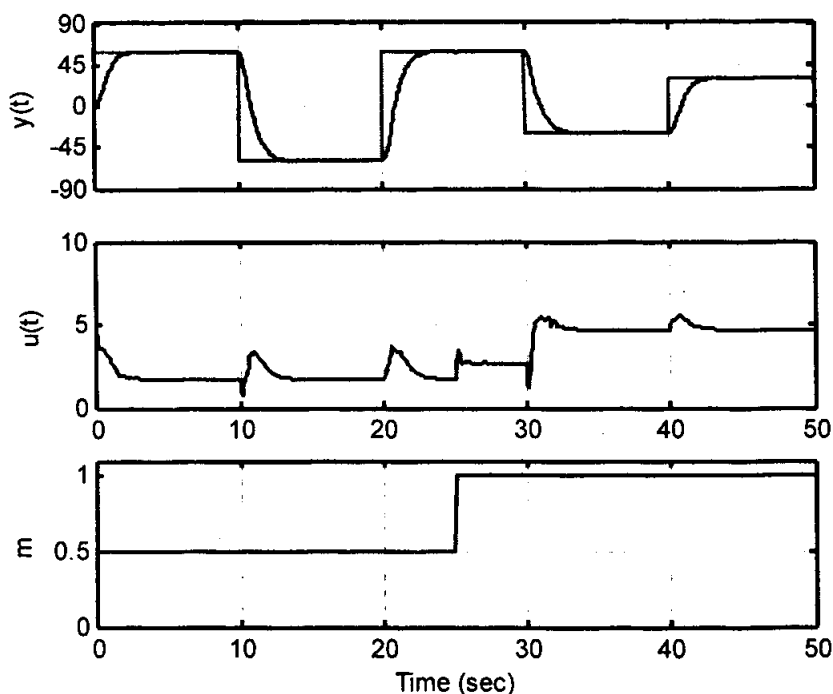


Hình 8.13 Thành phần điều khiển thích nghi trực tiếp



Hình 8.14 Các hàm cơ sở ở lớp ẩn mạng RBF

cánh tay bám rất tốt theo tín hiệu chuẩn. Tín hiệu điều khiển đã thay đổi thích ứng với sự thay đổi khối lượng của tải ở thời điểm $t = 25\text{sec}$ nhờ đó mà sự thay đổi tải hầu như không gây ảnh hưởng đến đáp ứng vị trí của cánh tay máy.



Hình 8.17 Kết quả điều khiển cánh tay máy dùng DAC

8.5 TÓM TẮT

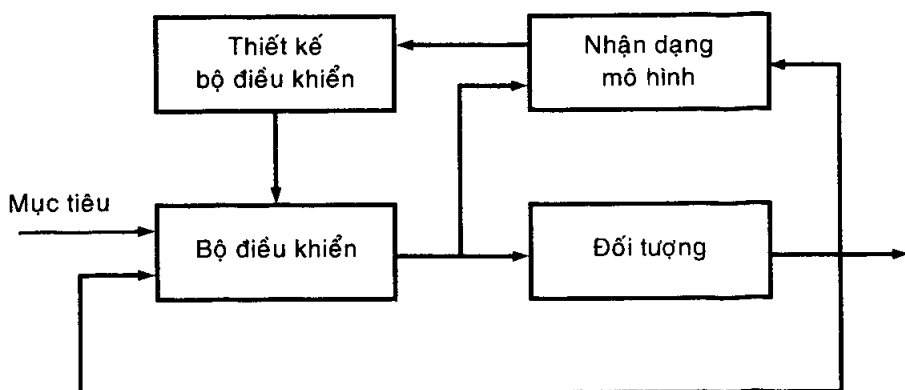
Chương 8 trình bày một số vấn đề cơ bản liên quan đến phương pháp điều khiển thích nghi gián tiếp và trực tiếp hệ phi tuyến SISO dùng mô hình mờ / mạng thần kinh. Luật thích nghi thông số và thành phần điều khiển chế độ trượt được thiết kế dựa vào lý thuyết ổn định Lyapunov. Các ví dụ minh họa được trình bày cụ thể để người đọc có thể theo đó tự lập trình để kiểm chứng lại nội dung lý thuyết. Các vấn đề nâng cao liên quan đến phương pháp điều khiển thích nghi như ước lượng trạng thái, điều khiển thích nghi có ràng buộc ngõ vào / ngõ ra, điều khiển thích nghi hệ MIMO độc giả có thể tham khảo thêm trong các tài liệu.

Chương 9

ĐIỀU KHIỂN HỌC

9.1 GIỚI THIỆU

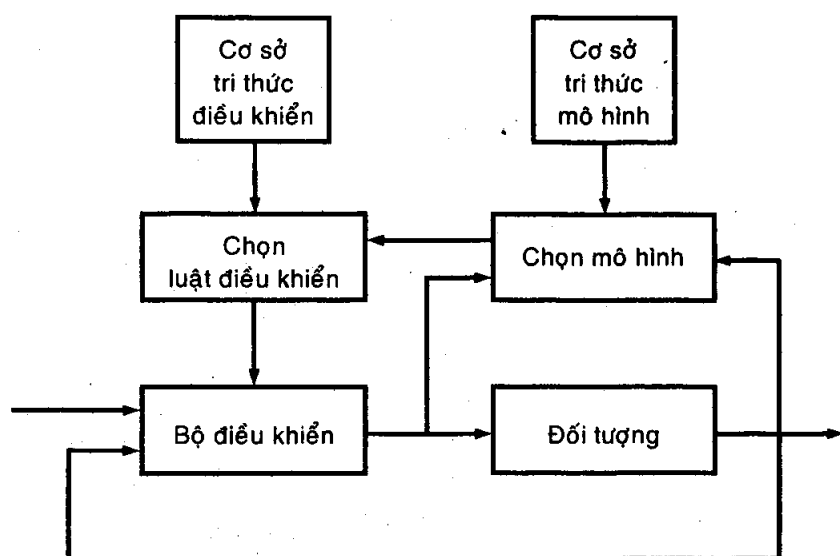
Như đã trình bày ở chương 8, điều khiển thích nghi là phương pháp điều khiển trong đó thông số bộ điều khiển được cập nhật trực tuyến thích ứng với điều kiện làm việc nhằm đảm bảo chất lượng điều khiển trong tầm làm việc rộng và các yếu tố không chắc chắn thay đổi. Điều khiển thích nghi mặc dù linh hoạt hơn nhiều so với điều khiển hồi tiếp thông thường nhưng vẫn có một số hạn chế. Hạn chế dễ thấy nhất là cơ chế nhận dạng đối tượng và tính toán luật điều khiển được lập trình khi thiết kế bộ điều khiển và không thể thay đổi trong quá trình vận hành. Do đó khả năng cập nhật luật điều khiển của bộ điều khiển thích nghi có giới hạn, nó chỉ có thể cập nhật thông số của bộ điều khiển trong một lớp mô hình định trước, lớp mô hình có thông số không chắc chắn. Điều khiển thích nghi không thể xử lý tất cả các yếu tố bất định không tham số, bao gồm động học tần số cao không mô hình hóa, động học tần số thấp không mô hình hóa, nhiễu cảm biến, và các yếu tố không chắc chắn về cấu trúc. Nếu đối tượng thực tế có cấu trúc chỉ hơi khác so với đối tượng giả thiết khi thiết kế thì kết quả điều khiển thích nghi có thể rất kém. Khi động học của đối tượng được mô hình hóa sai, ngay cả yếu tố bất định bé cũng dẫn đến trôi thông số và hệ thống không ổn định. Ngoài ra, điều khiển thích nghi cũng có một hạn chế nữa là cơ chế thích nghi lúc nào cũng làm việc ngay cả khi tình huống không chắc chắn xuất hiện lặp lại.



Hình 9.1 Sơ đồ khối hệ thống điều khiển thích nghi

Để khắc phục các khuyết điểm của phương pháp điều khiển thích nghi, phương pháp điều khiển tái cấu trúc đã được đưa ra. Động lực của việc phát triển phương pháp này là nhu cầu điều khiển các đối tượng có cấu trúc động học thay đổi. Điều này có nghĩa là tại các thời điểm khác nhau mô hình động học của đối tượng được mô tả bằng các phương trình có dạng và có các biến khác nhau. Sơ đồ khối hệ thống điều khiển tái cấu trúc trình bày ở hình 9.2. Hệ thống có bộ nhớ lưu trữ cơ sở tri thức điều khiển và cơ sở tri thức mô hình đối tượng. Ý tưởng chính là khả năng kiểm soát tình huống, nhận biết sự thay đổi cấu trúc, và sau đó thiết kế lại bộ điều khiển thời gian thực để bù các thay đổi cấu trúc.

Điều khiển tái cấu trúc có thể áp dụng trong nhiều trường hợp; trường hợp đối tượng bị hư hỏng là tình huống tiêu biểu nhất có thể áp dụng phương pháp này. Hư hỏng không nhất thiết dẫn đến làm sụp đổ hệ thống điều khiển; trong nhiều trường hợp các hư hỏng này có thể bù trừ bằng cách thay đổi đột ngột chiến lược điều khiển và điều này có thể thực hiện khi có bậc tự do thừa trong đối tượng điều khiển. Ví dụ trường hợp robot hai chân, khi một chân bị hỏng thì robot không thể di chuyển bằng cách “đi” được, nếu hệ thống điều khiển có khả năng tái cấu trúc thì có thể chuyển sang cấu trúc điều khiển mới, robot vẫn có thể di chuyển bằng cách “nhảy” một chân. Đặc tính động học của robot hoàn toàn khác nhau về cấu trúc trong hai trường hợp “đi bằng hai chân” và “nhảy bằng một chân”, do đó không thể áp dụng phương pháp điều khiển thích nghi thông thường.



Hình 9.2 Sơ đồ khối hệ thống điều khiển tái cấu trúc

Để thực thi hệ thống điều khiển tái cấu trúc, người thiết kế phải đưa ra một tập hợp các bộ điều khiển phù hợp với từng tình huống cụ thể và lưu trữ sẵn trong cơ sở tri thức. Một bộ giám sát có khả năng chọn các bộ điều khiển đã lưu trữ trong cơ sở tri thức tương ứng với tình huống cảm nhận được. Trong nhiều trường hợp rất khó thực thi phương pháp điều khiển tái cấu trúc vì khi thiết kế không thể hình dung được tất cả các tình huống có thể xảy ra khi hệ thống vận hành.

Một phương pháp điều khiển phát triển mạnh trong những năm gần đây là điều khiển học (*Learning Control*) xuất phát từ yêu cầu điều khiển hệ thống trong các tình huống thay đổi không dự tính trước. Điều khiển học không chỉ có khả năng thích nghi, nhớ các luật điều khiển mới mà còn có khả năng chọn luật điều khiển phù hợp. Các vấn đề liên quan đến điều khiển học sẽ được trình bày trong mục tiếp theo.

9.2 HỆ THỐNG ĐIỀU KHIỂN HỌC

9.2.1 Khái niệm

Rất khó đưa ra định nghĩa chính xác và thỏa mãn hoàn toàn cho thuật ngữ “*Hệ thống điều khiển học*”. Một định nghĩa phổ biến được nhiều người chấp nhận trong các tài liệu hiện nay là:

“Hệ thống điều khiển học là hệ thống điều khiển có khả năng cải thiện chất lượng điều khiển trong tương lai, dựa vào thông tin kinh nghiệm mà hệ thống thu thập được trong quá khứ, thông qua tương tác vòng kín với đối tượng và môi trường”.

Theo định nghĩa trên, hệ thống điều khiển học có khả năng tự chủ, vì nó có thể cải thiện chất lượng của chính nó. Hệ thống điều khiển học có bộ nhớ vì thông tin quá khứ là yếu tố then chốt để cải thiện chất lượng tương lai. Để cải thiện chất lượng, hệ thống điều khiển học phải nhận thông tin phản hồi chất lượng dựa trên một hàm mục tiêu mà hệ thống tìm cực trị.

Vai trò trọng tâm của học trong lĩnh vực điều khiển thông minh là cho phép thực thi các hệ thống điều khiển trong đó hệ thống hoạt động trong điều kiện không chắc chắn, và thông tin biết trước sẵn có quá ít không thể thiết kế hệ thống có những tính chất cho trước và chất lượng đủ tốt.

So sánh khả năng thích nghi và khả năng học

Cả hệ thống điều khiển thích nghi và điều khiển học đều có thể thực thi bằng cách sử dụng các thuật toán chỉnh định thông số, và cả hai phương pháp điều khiển đều sử dụng thông tin phản hồi chất lượng thu thập được qua tương tác vòng kín với đối tượng và môi trường. Khác biệt cơ bản giữa điều khiển thích nghi và điều khiển học là khả năng sử dụng thông tin kinh nghiệm trong quá khứ. Hệ thống điều khiển thích nghi xem mỗi tình huống hoạt động phân biệt đều là tình huống mới, trong khi hệ thống điều khiển học xét tương quan tình huống hoạt động hiện tại với kinh nghiệm tình huống quá khứ, và có thể nhớ lại và khai thác kinh nghiệm này. Hệ thống điều khiển thích nghi không có bộ nhớ nên phải chỉnh định lại thông số khi ngay cả khi tình huống cũ lặp lại, điều này làm giảm chất lượng của hệ thống vì đáp ứng quá độ xuất hiện trong quá trình chỉnh định thông số.

Về nguyên tắc hệ thống điều khiển thích nghi hoạt động bằng cách tối ưu hóa một tập hợp các thông số có thể chỉnh định. Để hiệu quả, bộ điều khiển thích nghi phải có đặc tính động học đủ nhanh để phản ứng kịp thời sự thay đổi đặc tính động học của đối tượng. Trong một số trường hợp thông số của đối tượng thay đổi quá nhanh (có thể do các yếu tố phi tuyến), bộ điều khiển thích nghi không thể phản

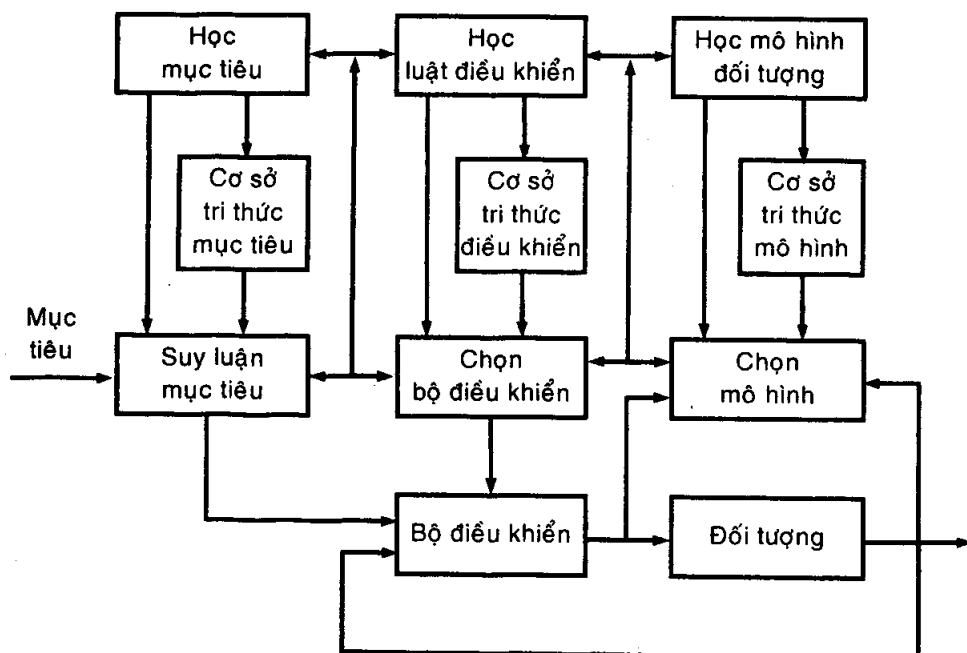
ứng kịp do đó không thể đảm bảo chất lượng mong muốn. Trong những trường hợp nêu trên sử dụng hệ thống điều khiển học sẽ phù hợp hơn. Vì hệ thống điều khiển học nhớ thông tin quá khứ nên về nguyên tắc nó phản ứng nhanh hơn trong những tình huống mà nó đã học.

Bộ điều khiển học sử dụng cơ chế tự động liên kết một tác động điều khiển thích hợp hoặc một tập hợp thông số điều khiển với tình huống hoạt động hiện tại. Khi hệ thống đã học, đáp ứng quá độ do sự thay đổi đặc tính động học của đối tượng không xuất hiện nữa, do đó hệ thống điều khiển học hiệu quả hơn và chất lượng tốt hơn so với điều khiển thích nghi.

Dưới quan điểm điều khiển thông minh quá trình thích nghi và quá trình học có những đặc điểm khác nhau và bổ sung cho nhau. Ví dụ, điều khiển thích nghi phù hợp với những trường hợp đặc tính động học của đối tượng thay đổi chậm theo thời gian và những tình huống mới (chưa gặp trước bao giờ), nhưng thường không hiệu quả trong những trường hợp đặc tính động học của đối tượng có tính phi tuyến cao. Ngược lại, điều khiển học có những đặc điểm hoàn toàn trái ngược, phương pháp này rất phù hợp với những trường hợp đối tượng có đặc tính động học phi tuyến không mô hình hóa, nhưng lại không phù hợp lắm trong những trường hợp đặc tính động học của đối tượng thay đổi theo thời gian.

9.2.2 Sơ đồ khối hệ thống điều khiển học

Sơ đồ khối hệ thống điều khiển học trình bày ở hình 9.3. Hệ thống điều khiển học tổng quát có ba khối thực hiện chức năng học: học mục tiêu, học điều khiển và học mô hình đối tượng. Tuy nhiên tùy theo ứng dụng cụ thể mà hệ thống không nhất thiết phải có đầy đủ cả ba khối học vừa nêu. Mỗi khối học có hai ngõ ra cập nhật cơ sở dữ liệu và cập nhật thủ tục chọn lựa. Ví dụ, khối học điều khiển không chỉ học luật điều khiển để cập nhật cơ sở dữ liệu mà còn học cách chọn luật điều khiển. Để thực thi hệ thống điều khiển học theo sơ đồ khối ở hình 9.3 một số vấn đề quan trọng cần giải quyết là: cách biểu diễn mô hình đối tượng và luật điều khiển không rõ cách thông số hóa, cơ chế suy diễn thỏa mãn yêu cầu áp dụng vào hệ thống thời gian thực và cơ chế học phù hợp với cấu trúc hệ thống.



Hình 9.3 Sơ đồ khối hệ thống điều khiển học

9.2.3 Thực thi hệ thống điều khiển học

Hệ thống học phải có khả năng thu thập và xử lý thông tin kinh nghiệm, lưu trữ và truy xuất lại thông tin đã xử lý, và điều chỉnh tri thức đã lưu trữ phù hợp với kinh nghiệm mới. Điểm quan trọng khi thực thi hệ thống điều khiển học là hệ thống cần lưu trữ và truy xuất thông tin một cách hiệu quả để giữ tri thức rút ra từ dữ liệu thực nghiệm. Một cách đơn giản để thực thi hệ thống học là phân chia không gian tín hiệu vào thành các miền rời nhau sao cho tín hiệu ra có thể được xác định bằng cách “tra bảng” tương ứng với miền tín hiệu vào. Rất nhiều hệ thống học trước đây đã được thực thi theo nguyên tắc này. Giả sử rằng ngõ ra của hệ thống điều khiển học được sử dụng trực tiếp cho tác động điều khiển, một luật điều khiển phi tuyến có thể phát triển bằng cách học giá trị ra xấp xỉ tương ứng với mỗi miền tín hiệu vào, kết quả là hệ thống học được xấp xỉ bậc thang luật điều khiển mong muốn. Để thấy mối quan hệ giữa học và xấp xỉ hàm trong cách thực thi hệ thống học vừa đề cập. Khuyết điểm của phương pháp này là tổ hợp các miền tín hiệu vào tăng lên khi không gian trạng thái tăng lên hoặc số miền phân chia mỗi chiều không gian trạng thái tăng lên.

Các hệ thống điều khiển học cao cấp hơn có thể phát triển dựa trên cấu trúc toán học có thể xấp xỉ một họ các hàm liên tục; cấu trúc này có thể cố định hay biến đổi và có rất nhiều thông số tự do. Cấu trúc và đặc điểm chức năng của hệ thống học một phần được xác định bởi số lượng và chất lượng của thông tin thiết kế biết trước sẵn có, bao gồm cả đặc điểm dự đoán trước của thông tin kinh nghiệm thu thập được khi hoạt động trực tuyến. Trong trường hợp này, quá trình học được thiết kế để tự động điều chỉnh thông số (hoặc cấu trúc) xấp xỉ hàm để đạt được quan hệ vào-ra mong muốn. Cách thực thi hệ thống điều khiển học như vừa mô tả có một số ưu điểm hơn so với các tra bảng đơn giản. Thứ nhất, các hàm liên tục nói chung được mô tả một cách hiệu quả hơn thông qua các thông số tự do, và do đó, cách xấp xỉ hàm cần ít bộ nhớ hơn để biểu diễn một hàm liên tục so với cách tra bảng. Thứ hai, cấu trúc xấp xỉ hàm có tính chất tổng quát hóa (tức là nội suy giữa những điểm tín hiệu vào). Do các ưu điểm nêu trên, cấu trúc xấp xỉ hàm thường được sử dụng trong các hệ thống điều khiển học. Hiện nay, các cấu trúc xấp xỉ hàm được sử dụng phổ biến nhất là mạng thần kinh (MLP, RBF) và mô hình mờ (Mamdani, Takagi-Sugeno).

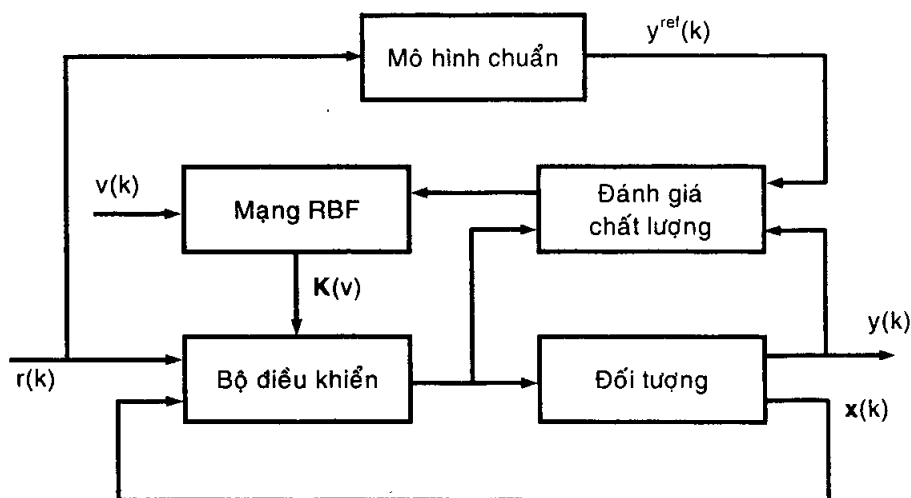
9.3 MỘT SỐ VÍ DỤ

Mục 9.2 đã trình bày sơ đồ khối và một số vấn đề liên quan đến việc thực thi hệ thống điều khiển học. Mục 9.3 sẽ trình bày một số ví dụ đơn giản để minh họa cách thiết kế và các thuật toán cần thiết để thực thi quá trình huấn luyện dùng mạng thần kinh và logic mờ. Các ví dụ được trình bày chi tiết để độc giả quan tâm có thể tự lập trình để kiểm chứng lại kết quả.

Ví dụ 9.1 Điều khiển học dùng mạng RBF

Ví dụ này trình bày kỹ thuật thiết kế hệ thống điều khiển học áp dụng cho hệ thống điều khiển máy bay hay tàu ngầm, những đối tượng mà đặc tính động học tuyến tính hóa thay đổi đáng kể theo điểm làm việc. Hoạch định độ lợi (*gain scheduling*) là phương pháp điều khiển được sử dụng hiện nay khi xác định được chính xác mô hình của đối tượng. Phương pháp điều khiển học trình bày ở ví dụ này áp dụng khi mô hình tuyến tính hóa theo điểm làm việc không thể xác định chính xác do các yếu tố không chắc chắn. Sơ đồ khối của hệ thống được trình bày ở hình 9.4. Chú ý rằng nếu không dùng khối

đánh giá chất lượng và hệ thống học được thay thế bằng một bảng hoạch định độ lợi tính thì sơ đồ điều khiển sẽ trở về phương pháp hoạch định độ lợi quen thuộc; nếu bỏ qua hệ thống học và một tập hợp các thông số của bộ điều khiển được chỉnh định độc lập với điều kiện làm việc thì sơ đồ sẽ trở thành điều khiển thích nghi trực tiếp. Việc thực thi cả hệ thống ở hình 9.4 có thể xem như là một hệ thống điều khiển thích nghi có thêm bộ nhớ hay hệ thống điều khiển hoạch định độ lợi có thêm khả năng điều chỉnh chất lượng trực tuyến.



Hình 9.4 Sơ đồ khối hệ thống điều khiển học trực tiếp

Trong ví dụ này đối tượng được mô tả bởi mô hình toán:

$$\begin{cases} \mathbf{x}(k+1) = \mathbf{A}(v)\mathbf{x}(k) + \mathbf{B}(v)u(k) \\ y(k) = \mathbf{C}\mathbf{x}(k) \end{cases} \quad (9.1)$$

$$\mathbf{A}(v) = \begin{bmatrix} 0,665v(v-2) & -0,95 \\ 1,0 & 0,0 \end{bmatrix} \quad (9.2)$$

$$\mathbf{B}(v) = \begin{bmatrix} 0,3 \left(\frac{e^{-v} - 1}{v} + 1,1 \right) \\ 0,0 \end{bmatrix} \quad (9.3)$$

$$\mathbf{C} = [1 \quad 0] \quad (9.4)$$

trong đó v là biến hoạch định điểm làm việc. Trong ví dụ này, giả thiết v thay đổi theo quy luật sau:

$$v(k+1) = 0,9v(k) + 0,1\mu(k) \quad (9.5)$$

Theo (9.5), biến hoạch định điểm làm việc v thay đổi chậm và trơn, điều này phù hợp với thực tế, chẳng hạn v biểu diễn vận tốc máy bay hoặc tàu ngầm. Biến ngẫu nhiên $\mu(k)$ được dùng trong chương trình mô phỏng để thay đổi điểm làm việc của hệ thống, giả thiết tầm giá trị của μ là $[0, 1]$. Giả thiết chúng ta chỉ biết bậc của hệ thống (bậc 2) mà không biết ảnh hưởng của điểm làm việc đến đặc tính động học của hệ thống như thế nào (tức là không biết cụ thể $A(v)$ và $B(v)$ phụ thuộc vào v như thế nào).

Mục tiêu điều khiển là bám theo tín hiệu vào với đáp ứng quá độ cho trước. Đặc tính động học mong muốn của hệ thống kín xác định bởi mô hình chuẩn trong sơ đồ khối ở hình 9.4. Bộ đánh giá chất lượng tính sai lệch giữa ngõ ra của mô hình chuẩn và ngõ ra của hệ thống. Trong ví dụ này để đơn giản chọn mô hình chuẩn có cặp cực phức $-0,4 \pm 0,05j$, tuy nhiên cần chú ý rằng trong các ứng dụng thực tế mô hình chuẩn có thể thay đổi theo điểm làm việc. Tử số của mô hình chuẩn được chọn sao cho mô hình chuẩn có độ lợi bằng 1 với một zero tại góc tọa độ.

Luật điều khiển được sử dụng là luật điều khiển hồi tiếp trạng thái:

$$u(k) = -K_1x_1(k) - K_2x_2(k) + K_3r(k) \quad (9.6)$$

trong đó: K_1, K_2 - độ lợi hồi tiếp trạng thái

K_3 - độ lợi đầu vào.

Mục đích của hệ thống điều khiển học là nhận dạng và lưu trữ vector thông số của bộ điều khiển như là hàm của biến hoạch định điểm làm việc. Hệ thống học gồm mạng RBF có một ngõ vào (v), ba ngõ ra (K_1, K_2, K_3) với 31 tế bào ở lớp ẩn phân bố đều trong đoạn $[-0,0357; 1,0357]$. Tại mỗi thời điểm lấy mẫu hệ thống học thực hiện các tác vụ sau:

- Đo lường biến hoạch định điểm làm việc $v(k)$
- Đọc độ lợi điều khiển từ ngõ ra mạng RBF:

$$K(v(k)) = [K_1(v(k)) \quad K_2(v(k)) \quad K_3(v(k))]^T \quad (9.7)$$

- Tính tín hiệu điều khiển theo công thức (9.6) và tác động vào đối tượng
- Sai lệch giữa tác động điều khiển với tín hiệu ra mong muốn của mô hình chuẩn được sử dụng để cập nhật trọng số mạng RBF.

Khối đánh giá chất lượng có nhiệm vụ biến đổi các biến trong vòng điều khiển thành tín hiệu huấn luyện hệ thống học. Điều này được thực hiện qua hai bước:

Bước 1: Giá trị hiệu chỉnh độ lợi điều khiển ở ngõ ra của mạng RBF được tính dựa vào sai số giữa tín hiệu ra của mô hình chuẩn và tín hiệu ra của hệ thống theo công thức:

$$\delta K(v(k)) = \alpha e(k) \phi(k) \quad (9.8)$$

trong đó: $e(k) = y^{ref}(k) - y(k)$

$$\phi(k) = [x_1(k) \quad x_2(k) \quad r(k)]^T$$

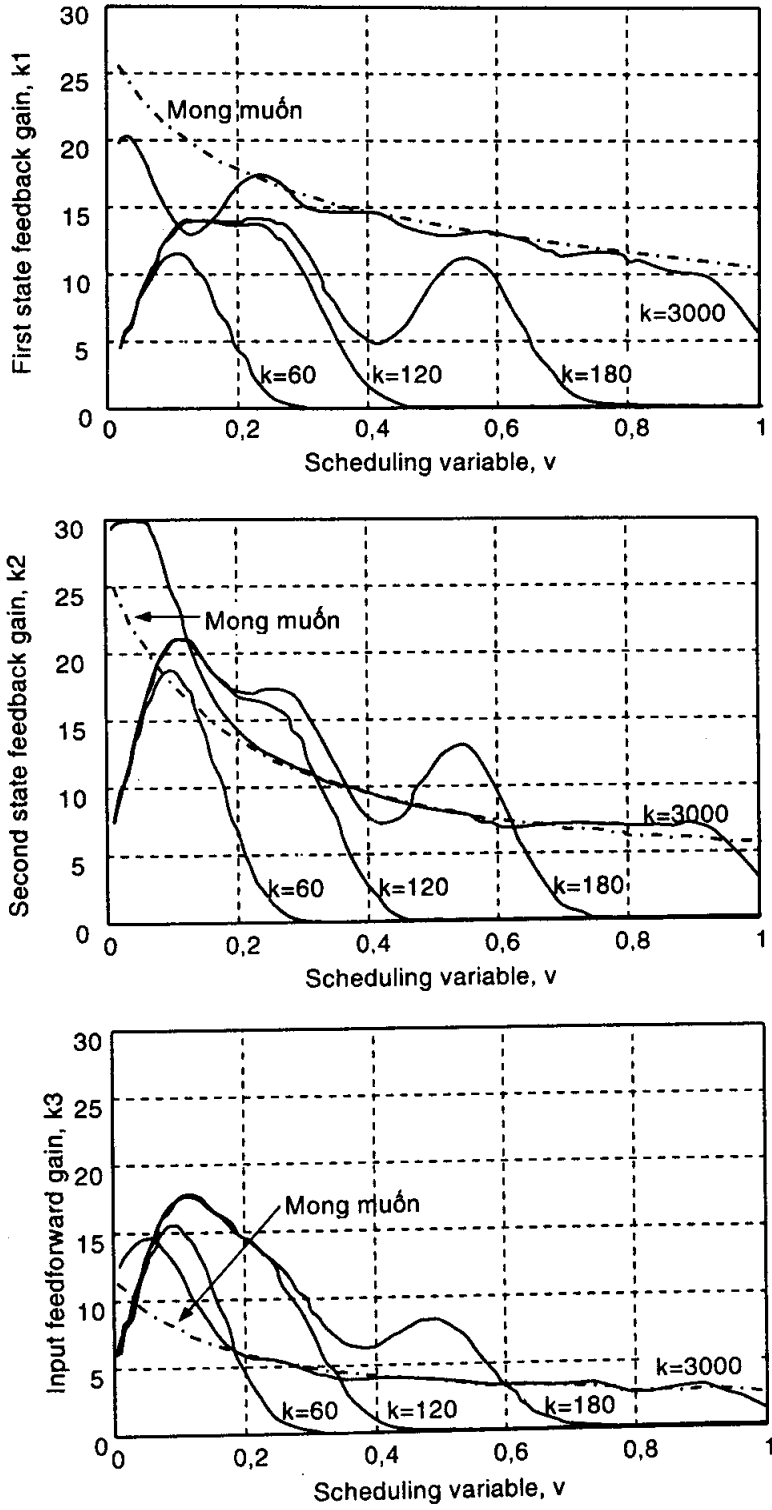
α - hệ số dương nhỏ

$K(v(k))$ - các thông số của luật điều khiển xuất ra bởi mạng RBF tương ứng với điểm làm việc $v(k)$.

Bước 2: Các trọng số lớp ra của mạng RBF được cập nhật bằng thuật toán suy giảm độ dốc để ngõ ra của mạng thay đổi một lượng là $\delta K(v(k))$.

Hình 9.5 trình bày các thông số điều khiển hoạch định theo điểm làm việc ($K_1(v)$, $K_2(v)$, $K_3(v)$) đã học theo cách đã trình bày ở trên. Trong ví dụ này, cứ 60 chu kỳ lấy mẫu giá trị $\mu(k)$ thay đổi một lần. Ba giá trị đầu tiên của $\mu(k)$ được gán bằng 0,1, 0,3 và 0,6; các giá trị còn lại được gán ngẫu nhiên (trong đoạn $[0, 1]$). Tuy nhiên do động học của biến hoạch định $v(k)$ ràng buộc $\mu(k)$ thay đổi liên tục, do đó nhiều mẫu huấn luyện tương ứng với giá trị giữa tầm của $v(k)$ được tạo ra nhiều hơn so với các giá trị biên. Trong mỗi đồ thị trình bày ngõ ra của mạng RBF sau 60, 120, 180 và 3000 bước lặp. Ta thấy sau khi học qua 3000 bước lặp, ngõ ra của mạng RBF đã có thể xấp xỉ được độ lợi điều khiển mong muốn, đặc biệt là trong miền giữa tầm của biến hoạch định v . Cụ thể trên đồ thị trong miền $0,2 \leq v \leq 0,8$ sai lệch giữa độ lợi đã học ở ngõ ra của mạng RBF và giá trị độ lợi mong muốn là không đáng kể, điều này hoàn toàn hợp lý vì các mẫu huấn luyện tương ứng với giá trị giữa tầm của biến hoạch định v được tạo ra nhiều hơn so với các giá trị biên.

Các đường cong sau 60, 120 và 180 bước lặp tương ứng xấp xỉ độ lợi mong muốn xung quanh giá trị hoạch định v bằng 0,1, 0,3 và 0,6. Chú ý rằng chỉ có các giá trị độ lợi trong miền lân cận giá trị v được điều chỉnh, điều này là do tính chất cục bộ không gian của mạng RBF.



Hình 9.5 Kết quả học thông số điều khiển ở ví dụ 9.1

Tính chất cục bộ không gian trong quá trình huấn luyện là yếu tố quan trọng trong các hệ thống học, vì chúng ta không mong muốn kết quả đã huấn luyện đã lưu trữ tại một điểm làm việc nào đó bị thay đổi khi chuyển sang huấn luyện tại một điểm làm việc khác. Ngoài ra, chú ý là kết quả huấn luyện cũng được tổng quát hóa trong miền lân cận của v .

Ví dụ 9.2 Điều khiển học mờ (*Fuzzy Learning Control*)

Ví dụ này trình bày một ứng dụng của phương pháp điều khiển học mờ áp dụng cho đối tượng phi tuyến biến đổi theo thời gian. Xét đối tượng điều khiển là tên lửa một tầng có mô hình toán học đơn giản cho bởi phương trình:

$$\frac{dv(t)}{dt} = u(t) \left(\frac{m}{M - mt} \right) - g_o \left(\frac{R}{R + y(t)} \right) - 0,5v^2(t) \left(\frac{\rho_a AC_d}{M - mt} \right) \quad (9.9)$$

trong đó:

$v(t)$: vận tốc tên lửa ở thời điểm t

$y(t)$: độ cao tên lửa ở thời điểm t

$u(t)$: vận tốc khí đốt (tạo phản lực đẩy tên lửa) ở thời điểm t

$M = 15000kg$: khối lượng ban đầu của tên lửa bao gồm của nhiên liệu

$m = 100kg/sec$: khối lượng nhiên liệu đốt trong một giây

$A = 1m^2$: tiết diện ngang xả khí đốt

$R = 6,37.10^6m$: bán kính trái đất

$\rho_a = 1,21kg/m^3$: mật độ không khí

$C_d = 0,3$: hệ số cản.

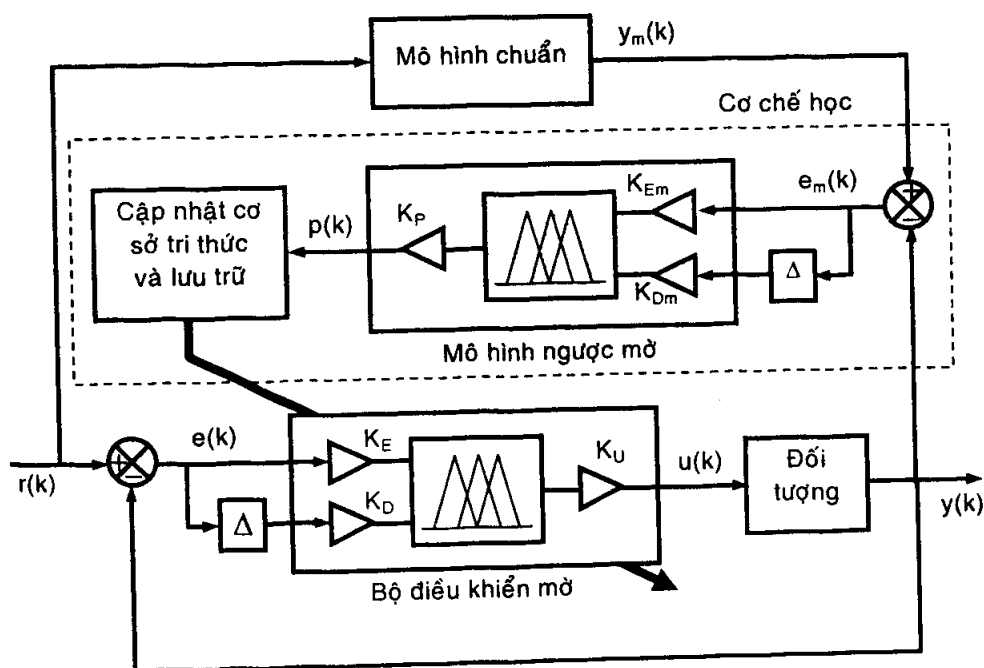
Bài toán đặt ra là điều khiển vận tốc của tên lửa (bằng cách thay đổi vận tốc khí đốt). Phương trình toán (9.9) cho thấy rằng đối tượng điều khiển có đặc tính động học phi tuyến. Mặt khác để rằng do quá trình đốt nhiên liệu khối lượng tên lửa giảm dần theo thời gian, do đó hệ thống có đặc tính động học phi tuyến biến đổi theo thời gian. Vì vậy, tên lửa là một đối tượng rất khó điều khiển.

Sơ đồ khối hệ thống điều khiển học mờ trình bày ở hình 9.6, cơ chế học của hệ thống như sau:

- Quan sát dữ liệu vào ra của hệ thống điều khiển mờ.
- Đánh giá chất lượng của hệ thống điều khiển.

- Tự chỉnh thông số bộ điều khiển mờ để đạt được chất lượng mong muốn (chất lượng mong muốn xác định bởi mô hình chuẩn trong sơ đồ ở hình 9.6).
- Lưu trữ tri thức (quy luật chỉnh định thông số) vào bộ nhớ.

Sau đây sẽ trình bày chi tiết cách thiết kế các khối chức năng trong hệ thống điều khiển học mờ. Hệ thống được lấy mẫu với chu kỳ $T = 0,1\text{sec}$.



Hình 9.6 Sơ đồ khối hệ thống điều khiển học mờ

Bộ điều khiển mờ

Bộ điều khiển được sử dụng là bộ điều khiển PD mờ gồm các quy tắc Sugeno với mệnh đề kết luận là hằng số. Như đã trình bày ở chương 5, tín hiệu vào của bộ điều khiển PD mờ là sai số và biến thiên sai số:

$$e(k) = r(k) - y(k) \quad (9.10)$$

$$\Delta e(k) = \frac{e(k) - e(k-1)}{T} \quad (9.11)$$

Tín hiệu ra của bộ điều khiển là $u(k)$. Các hệ số chuẩn hóa ở ngõ vào và ngõ ra của bộ điều khiển là K_E , K_D và K_U được chọn như sau:

$$K_E = \frac{1}{4000} \quad (9.12)$$

$$K_D = \frac{1}{2000} \quad (9.13)$$

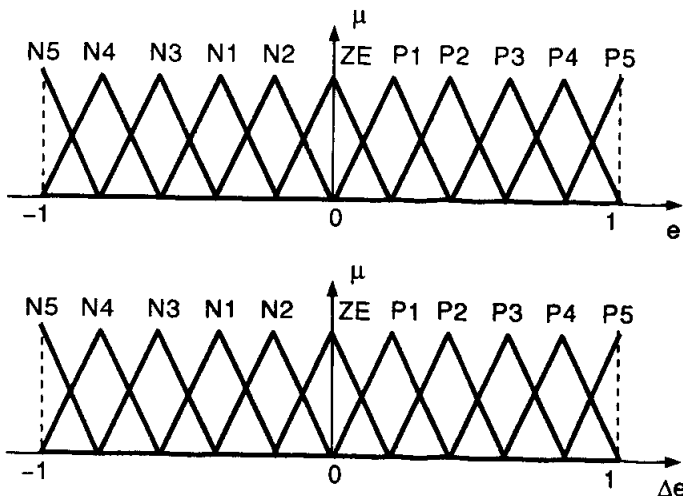
$$K_U = 10000 \quad (9.14)$$

Các giá trị ngôn ngữ của biến vào của bộ điều khiển được mô tả bởi các tập mờ định nghĩa trên miền giá trị chuẩn hóa $[-1, 1]$ như hình 9.7. Bộ điều khiển gồm các quy tắc có dạng:

Nếu sai số là $\tilde{E}_i$ **và** biến thiên sai số là $\Delta\tilde{E}_i$
thì tín hiệu điều khiển là α_i .

trong đó $\tilde{E}_i$ và $\Delta\tilde{E}_i$ là các tập mờ định nghĩa ở hình 9.7, α_i là hằng số ở mệnh đề kết luận ở quy tắc thứ i .

Ban đầu bộ điều khiển mờ với các giá trị α_i được gán dựa vào kinh nghiệm như bảng 9.1. Bộ điều khiển với các giá trị α_i ban đầu có thể không điều khiển được đối tượng với chất lượng đúng yêu cầu mong muốn. Qua quá trình học cơ sở tri thức của bộ điều khiển, cụ thể là các giá trị α_i sẽ được điều chỉnh và lưu trữ.



Hình 9.7 Các tập mờ mô tả các giá trị ngôn ngữ của các biến vào

Bảng 9.1 Cơ sở tri thức của bộ điều khiển mờ

u		Δe										
		N5	N4	N3	N2	N1	ZE	P1	P2	P3	P4	P5
e	N5	+1,0	+1,0	+1,0	+1,0	+1,0	+1,0	+0,8	+0,6	+0,4	+0,2	0,0
	N4	+1,0	+1,0	+1,0	+1,0	+1,0	+0,8	+0,6	+0,4	+0,2	0,0	-0,2
	N3	+1,0	+1,0	+1,0	+1,0	+0,8	+0,6	+0,4	+0,2	0,0	-0,2	-0,4
	N2	+1,0	+1,0	+1,0	+0,8	+0,6	+0,4	+0,2	0,0	-0,2	-0,4	-0,6
	N1	+1,0	+1,0	+0,8	+0,6	+0,4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8
	ZE	+1,0	+0,8	+0,6	+0,4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8	-1,0
	P1	+0,8	+0,6	+0,4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8	-1,0	-1,0
	P2	+0,6	+0,4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8	-1,0	-1,0	-1,0
	P3	+0,4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8	-1,0	-1,0	-1,0	-1,0
	P4	+0,2	0,0	-0,2	-0,4	-0,6	-0,8	-1,0	-1,0	-1,0	-1,0	-1,0
	P5	0,0	-0,2	-0,4	-0,6	-0,8	-1,0	-1,0	-1,0	-1,0	-1,0	-1,0

Mô hình chuẩn

Mô hình chuẩn cho phép định lượng chất lượng mong muốn của quá trình điều khiển. Tổng quát, mô hình chuẩn có thể là bất kỳ hệ thống động nào (tuyến tính hoặc phi tuyến, bất biến hoặc biến đổi theo thời gian, rời rạc hoặc liên tục...). Cụ thể, trong ví dụ điều khiển vận tốc tên lửa mô hình chuẩn được cho bởi hàm truyền:

$$\frac{Y_m(s)}{R(s)} = \frac{0,2}{s + 0,2} \quad (9.15)$$

Chất lượng của hệ thống được tính bằng sai lệch giữa tín hiệu ra của mô hình chuẩn và tín hiệu ra của đối tượng:

$$e_m(k) = y_m(k) - y(k) \quad (9.16)$$

Cho trước mô hình chuẩn phản ánh tiêu chuẩn thiết kế như độ vọt lố, thời gian lên, thời gian xác lập... và tín hiệu vào mô hình chuẩn là $r(k)$, chất lượng mong muốn của quá trình điều khiển sẽ đạt được nếu cơ chế học giữ cho $e_m(k)$ rất bé. Nếu chất lượng mong muốn được thỏa mãn ($e_m(k) \approx 0$) thì cơ chế học sẽ không điều chỉnh bộ điều khiển mờ. Ngược lại, nếu chất lượng mong muốn không thỏa mãn ($e_m(k)$ lớn) thì cơ chế học sẽ điều chỉnh bộ điều khiển mờ.

Cơ chế học

Như đã đề cập ở mục trước cơ chế học thực hiện chức năng điều chỉnh cơ sở tri thức của bộ điều khiển mờ trực tiếp để đáp ứng của hệ thống kín giống như mô hình chuẩn. Việc điều chỉnh cơ sở tri thức được thực hiện dựa trên việc quan sát dữ liệu từ quá trình điều khiển, mô hình chuẩn và bộ điều khiển mờ.

Cơ chế học gồm có hai thành phần:

- Mô hình mờ ngược
- Bộ điều chỉnh cơ sở tri thức.

Mô hình mờ ngược: mô tả định tính đặc tính động học ngược của đối tượng điều khiển. Mô hình ngược được dùng để xác định sự thay đổi ở ngõ vào của đối tượng làm cho ngõ ra của đối tượng giống như ngõ ra của mô hình chuẩn. Như trình bày ở hình 9.6, mô hình ngược ánh xạ $e_m(k)$ và $\Delta e_m(k)$ thành sự thay đổi cần thiết $p(k)$ ở ngõ vào của đối tượng điều khiển. Vì ta định nghĩa mô hình ngược trên miền giá trị chuẩn hóa nên ở ngõ vào và ngõ ra của mô hình ngược có các hệ số chuẩn hóa K_{Em} , K_{Dm} và K_P .

Bảng 9.2 Các quy tắc của mô hình mờ ngược mờ

p		Δe_m										
		N5	N4	N3	N2	N1	ZE	P1	P2	P3	P4	P5
e_m	N5	-1,0	-1,0	-1,0	-1,0	-1,0	-1,0	-0,8	-0,6	-0,4	-0,2	0,0
	N4	-1,0	-1,0	-1,0	-1,0	-1,0	-0,8	-0,6	-0,4	-0,2	0,0	+0,2
	N3	-1,0	-1,0	-1,0	-1,0	-0,8	-0,6	-0,4	-0,2	0,0	+0,2	+0,4
	N2	-1,0	-1,0	-1,0	-0,8	-0,6	-0,4	-0,2	0,0	+0,2	+0,4	+0,6
	N1	-1,0	-1,0	-0,8	-0,6	-0,4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8
	ZE	-1,0	-0,8	-0,6	-0,4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8	+1,0
	P1	-0,8	-0,6	-0,4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8	+1,0	+1,0
	P2	-0,6	-0,4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8	+1,0	+1,0	+1,0
	P3	-0,4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8	+1,0	+1,0	+1,0	+1,0
	P4	-0,2	0,0	+0,2	+0,4	+0,6	+0,8	+1,0	+1,0	+1,0	+1,0	+1,0
	P5	0,0	+0,2	+0,4	+0,6	+0,8	+1,0	+1,0	+1,0	+1,0	+1,0	+1,0

$$K_{Em} = \frac{1}{4000} \quad (9.17)$$

$$K_{Dm} = \frac{1}{2000} \quad (9.18)$$

$$K_P = 10000 \quad (9.19)$$

Các tập mờ định nghĩa cho các biến $e_m(k)$ và $\Delta e_m(k)$ trên tập cơ sở chuẩn hóa tương tự như hình 9.7. Các quy tắc mờ mô tả đặc tính động học ngược của đối tượng với mệnh đề kết luận là các hằng số được cho bởi bảng 9.2.

Khởi điều chỉnh cơ sở tri thức: có chức năng điều chỉnh bộ điều khiển mờ để hệ thống đạt được chất lượng điều khiển tốt hơn. Cho thông tin về thay đổi cần thiết ở ngõ vào của đối tượng là $p(k)$, bộ điều chỉnh cơ sở tri thức sẽ cập nhật cơ sở tri thức của bộ điều khiển mờ sao cho tác động điều khiển trước đó (tức $u(k-1)$) sẽ được thay đổi một lượng là $p(k)$.

$$u^*(k-1) = u(k-1) + p(k) \quad (9.20)$$

Muốn đạt được điều này ngõ ra của các quy tắc mờ tác động ở thời điểm lấy mẫu thứ $(k-1)$ phải được thay đổi một lượng là:

$$\alpha_i(k) = \alpha_i(k-1) + \bar{p}(k) \quad (9.21)$$

trong đó:
$$\bar{p}(k) = \frac{p(k)}{K_U} \quad (9.22)$$

Thật vậy, tín hiệu điều khiển ở thời điểm $(k-1)$ trước khi cập nhật cơ sở dữ liệu được tính theo công thức:

$$u(k-1) = K_U \frac{\sum_{i=1}^N \mu_{\tilde{E}_i}(e(k-1)) \mu_{\Delta \tilde{E}_i}(e(k-1)) \alpha(k-1)}{\sum_{i=1}^N \mu_{\tilde{E}_i}(e(k-1)) \mu_{\Delta \tilde{E}_i}(e(k-1))} \quad (9.23)$$

Tín hiệu điều khiển sau khi cập nhật cơ sở dữ liệu là:

$$\begin{aligned} u^*(k-1) &= \\ &= K_U \frac{\sum_{i=1}^N \mu_{\tilde{E}_i}(e(k-1)) \mu_{\Delta \tilde{E}_i}(\Delta(k-1)) \{\alpha(k-1) + \bar{p}(k)\}}{\sum_{i=1}^N \mu_{\tilde{E}_i}(e(k-1)) \mu_{\Delta \tilde{E}_i}(e(k-1))} \end{aligned} \quad (9.24)$$

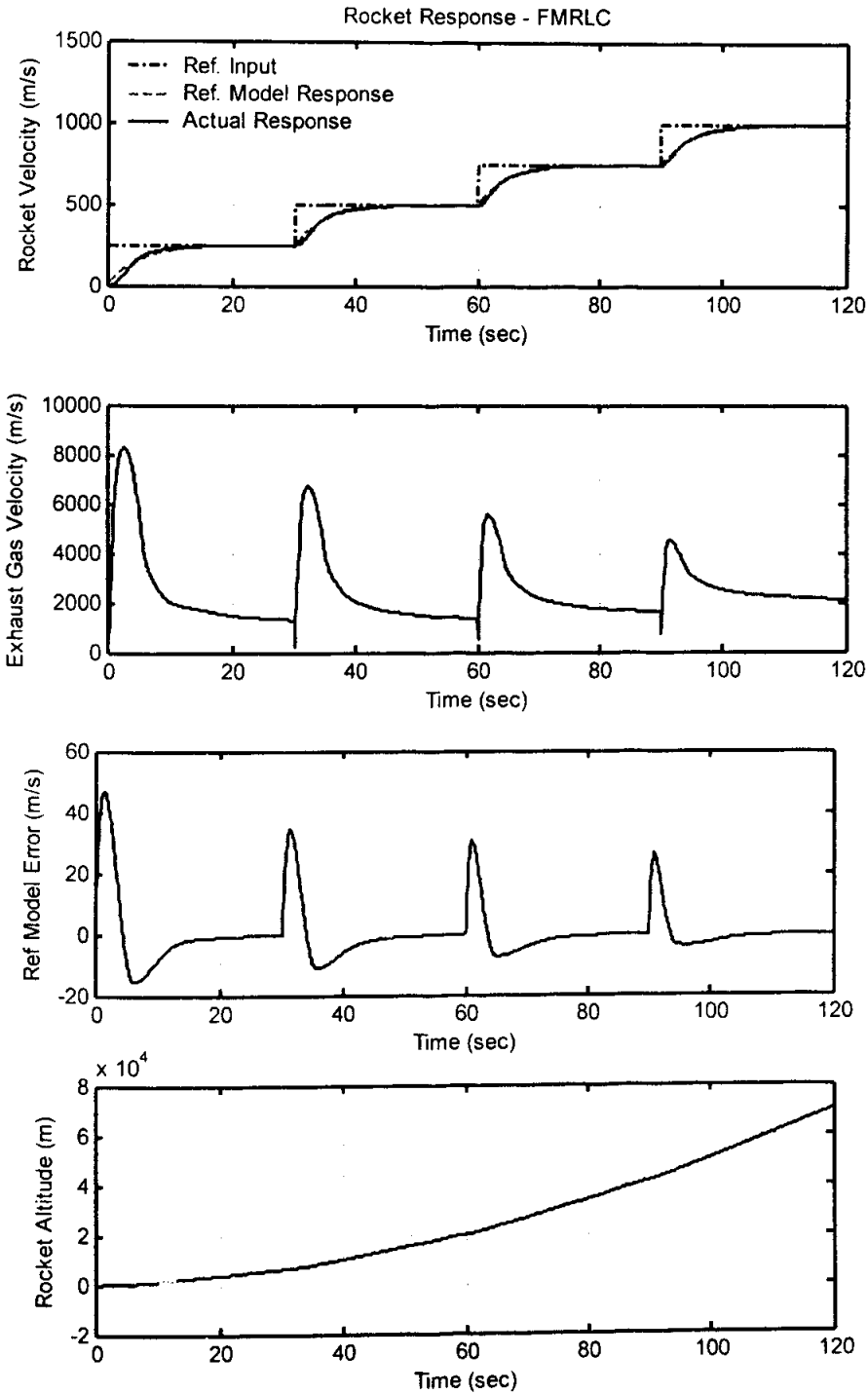
Chú ý, (9.22) và (9.23) ta dễ dàng thấy rằng (9.24) đồng nhất bằng (9.20), tức là việc cập nhật cơ sở dữ liệu ở mệnh đề kết luận của các quy tắc tác động đã thay đổi tín hiệu điều khiển đúng theo yêu cầu.

Để làm rõ hơn cơ chế cập nhật cơ sở dữ liệu ta xét ví dụ sau đây. Giả sử ở thời điểm $(k-1)$ sai số $e(k-1)$ có giá trị sao cho chỉ có các hàm liên thuộc tương ứng với các tập mờ $P3$ và $P4$ khác 0, và $\Delta e(k-1)$ có giá trị sao cho chỉ có các hàm liên thuộc tương ứng với các tập mờ $N4$ và $N5$ khác 0. Trong trường hợp này chỉ có bốn quy tắc được đánh dấu bằng các khung chữ nhật trong bảng 9.1 tác động. Giả sử $\bar{p}(k) = 0,1$, thế thì sau khi cập nhật cơ sở tri thức các giá trị +0,4, +0,2, 0,0 và +0,2 được đánh dấu bằng khung chữ nhật trong bảng 9.1 sẽ bằng +0,5, +0,3, +0,1 và +0,3 (các giá trị còn lại trong bảng 9.1 vẫn không thay đổi).

Chú ý rằng, phương pháp cập nhật cơ sở tri thức này có tính “tổng quát hóa” vì hệ thống đồng thời cũng học cách xử lý các giá trị lân cận điểm đang xét (tức là lân cận $e(k-1)$, $\Delta e(k-1)$, và $y(k-1)$). Cơ sở tri thức được điều chỉnh cục bộ, các phần khác nhau của hệ quy tắc được cập nhật dựa vào các điều kiện làm việc khác nhau của hệ thống; và khi một miền nào đó của hệ quy tắc được cập nhật các miền còn lại không bị ảnh hưởng. Hệ thống có bộ nhớ để lưu trữ cơ sở tri thức sau khi cập nhật. Bộ điều khiển được điều chỉnh thích nghi với tình huống mới và cơ sở tri thức tương ứng với những tình huống đã được điều chỉnh được lưu trữ trong bộ nhớ, do đó hệ thống điều khiển này chính là hệ thống điều khiển học.

Kết quả mô phỏng

Hình 9.8 là kết quả mô phỏng điều khiển vận tốc tên lửa dùng bộ điều khiển học mờ vừa thiết kế. Kết quả mô phỏng cho thấy vận tốc tên lửa bám theo mô hình chuẩn rất tốt mặc dù khối lượng tên lửa giảm đáng kể theo thời gian do sự tiêu hao nhiên liệu. Ví dụ này cho thấy hệ thống điều khiển học mờ có thể áp dụng để điều khiển tốt các đối tượng phi tuyến biến đổi theo thời gian.



Hình 9.8 Kết quả điều khiển tên lửa dùng bộ điều khiển học mờ

9.4 TÓM TẮT

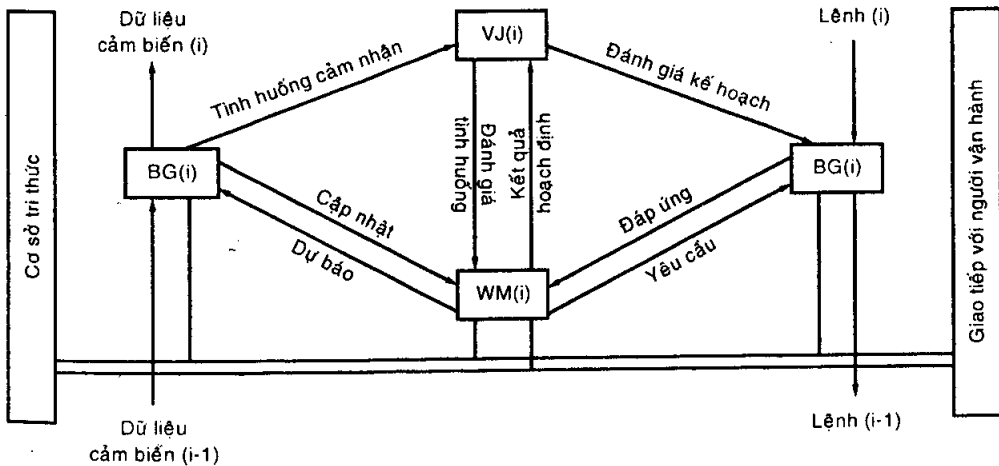
Chương 9 trình bày về hệ thống điều khiển học, một phương pháp điều khiển nâng cao dựa trên nền tảng phương pháp điều khiển thích nghi. Trong khi điều khiển thích nghi chủ yếu thay đổi thông số của bộ điều khiển để thích ứng với sự thay đổi của điều kiện làm việc thì điều khiển học có thể thay đổi và ghi nhớ cả cấu trúc và thông số của bộ điều khiển. Do hệ thống điều khiển học có bộ nhớ nên quá trình thích nghi sẽ không xảy ra khi hệ thống gặp lại tình huống cũ mà thay vào đó, cấu hình bộ điều khiển đã lưu trữ trong bộ nhớ được truy xuất để thích ứng ngay với tình huống xảy ra, nhờ đó mà hệ thống có thể đáp ứng nhanh. Điều khiển học rất phù hợp để điều khiển trong điều kiện có yếu tố bất định lớn và là phương pháp điều khiển không thể thiếu trong các hệ thống thông minh mức cao.

Chương 10

HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH

10.1 CẤU TRÚC HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH

10.1.1 Các thành phần cơ bản



Hình 10.1 Các thành phần cơ bản của hệ thống điều khiển thông minh

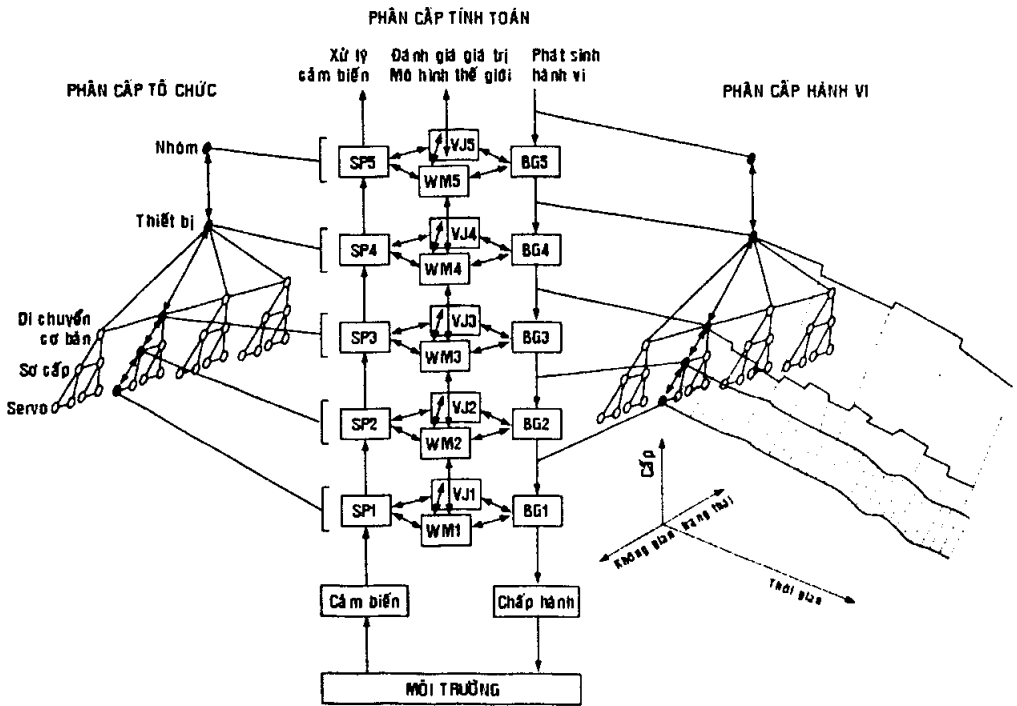
Hệ thống điều khiển thông minh được cấu thành từ bốn thành phần cơ bản là:

- Các khối xử lý cảm biến (*SP-Sensory Processing*)
- Mô hình thế giới (*WM-World Modelling*)
- Phát sinh hành vi (*BG-Behavior Generating*)
- Đánh giá giá trị (*VJ-Value Judgement*).

Hình 10.1 là sơ đồ biểu diễn mối liên hệ chức năng giữa các thành phần cơ bản trong hệ thống điều khiển thông minh và dòng tín hiệu truyền giữa chúng. Trong tất cả các hệ thống thông minh, khối xử lý cảm biến (*SP*) xử lý thông tin cảm nhận được để đưa ra và duy trì bên trong hệ thống mô hình của thế giới (*WM*) bên ngoài;

khối phát sinh hành vi (BG) điều khiển các phần tử chấp hành sao cho đạt được mục tiêu trong bối cảnh của mô hình thế giới cảm nhận được. Trong các hệ thống thông minh cao cấp hơn, khối xử lý cảm biến có thể tương tác với mô hình thế giới và khối đánh giá giá trị (VJ) để gán giá trị cho những sự kiện và tình huống cảm nhận được; khối phát sinh hành vi có thể tương tác với mô hình thế giới và khối đánh giá giá trị để suy luận về không gian và thời gian, hình học và động học, và hình thành hoặc chọn lựa kế hoạch dựa trên các chỉ tiêu như chi phí, rủi ro, tiện ích và mục đích.

10.1.2 Cấu trúc phân cấp



Hình 10.2 Cấu trúc phân cấp hệ thống điều khiển thông minh

Để điều khiển các đối tượng phức tạp hệ thống thông minh thường được tổ chức theo kiểu phân cấp, mỗi cấp gồm nhiều nút, mỗi nút gồm bốn thành phần cơ bản là:

- Các khối xử lý cảm biến (SP–Sensory Processing)
- Mô hình hóa thế giới (WM–World Modelling)
- Phát sinh hành vi (BG–Behavior Generating)
- Đánh giá giá trị (VJ–Value Judgement).

Hình 10.2 tóm tắt quan hệ giữa phân cấp tổ chức (được xác định bởi cây lệnh), phân cấp tính toán (được định nghĩa dọc theo mỗi chuỗi lệnh) và phân cấp hành vi (được tạo ra trong không gian trạng thái là hàm theo thời gian) trong hệ thống thông minh. Các mức phân cấp phụ thuộc vào độ phân giải của sự phân chia theo không gian và thời gian mục tiêu và tác vụ, cũng như sự tích hợp theo không gian và thời gian dữ liệu cảm biến. Độ phân giải không gian phụ thuộc vào mức phân nhóm các phần tử trong các hệ thống con, độ phân giải thời gian phụ thuộc vào băng thông, tần số lấy mẫu, thời đoạn chuyển trạng thái.

Ở mỗi cấp, các nút và các khối chức năng trong các nút được kết nối chặt chẽ với nhau bởi một hệ thống thông tin. Hệ thống thông tin có thể thực hiện bằng nhiều cách như bus máy tính, mạng cục bộ, bộ nhớ thông thường, hệ thống truyền thông điệp, hay các cách phối hợp. Trong mỗi nút tính toán, hệ thống thông tin cung cấp liên lạc giữa các khối chức năng. Yêu cầu và trạng thái tác vụ truyền từ khối BG đến khối WM. Thông tin phản hồi được truyền từ khối WM đến khối BG đang yêu cầu. Tín hiệu cảm biến dự báo được truyền từ khối WM đến khối SP, thông tin cập nhật mô hình thế giới được truyền từ khối SP ngược về khối WM. Các sự kiện và tình huống quan sát được truyền từ khối SP đến khối VJ. Giá trị gán cho mô hình thế giới biểu diễn bởi các sự kiện và tình huống này được truyền từ khối VJ đến khối WM. Kế hoạch giả định được truyền từ khối BG đến khối WM, kết quả được truyền ngược trở lại từ khối WM đến khối VJ. Các đánh giá sau đó được truyền từ khối VJ về khối BG đã phát sinh hành vi giả định.

Hệ thống thông tin cũng giao tiếp giữa các nút ở các cấp khác nhau. Lệnh được truyền xuống từ khối BG giám sát ở một cấp nào đó đến khối BG phụ thuộc ở cấp thấp hơn. Báo cáo trạng thái được truyền ngược lên đến WM từ các khối BG ở cấp dưới đến khối BG giám sát mà từ đó lệnh đã phát ra. Sự kiện và tình huống nhận ra bởi khối SP ở một cấp được truyền lên khối SP ở cấp cao hơn. Sự kiện và tình huống dự báo lưu trữ trong khối WM ở cấp cao hơn được truyền xuống các khối WM ở cấp thấp hơn. Tín hiệu ra từ khối BG ở cấp thấp nhất được truyền đến cơ cấu lái phần tử chấp hành. Tín hiệu vào các khối SP ở cấp thấp nhất được truyền đến từ cảm biến.

Một nguyên tắc cơ bản của hệ thống điều khiển thông minh là việc hoạch định được thực hiện đồng thời và liên tục ở mọi cấp. Tại

mỗi cấp, bộ điều khiển định kỳ phát ra kế hoạch gồm nhiều bước (thông thường là 10). Tại mỗi cấp thấp hơn kế tiếp, một hoặc hai bước đầu tiên trong kế hoạch hiện tại đã đưa ra bởi cấp trên tiếp tục được phân tích thành các kế hoạch con chi tiết hơn, trong phạm vi thời gian và không gian hẹp hơn. Kết quả là ở các cấp càng thấp, chu kỳ trung bình giữa các lệnh tác vụ ngắn.

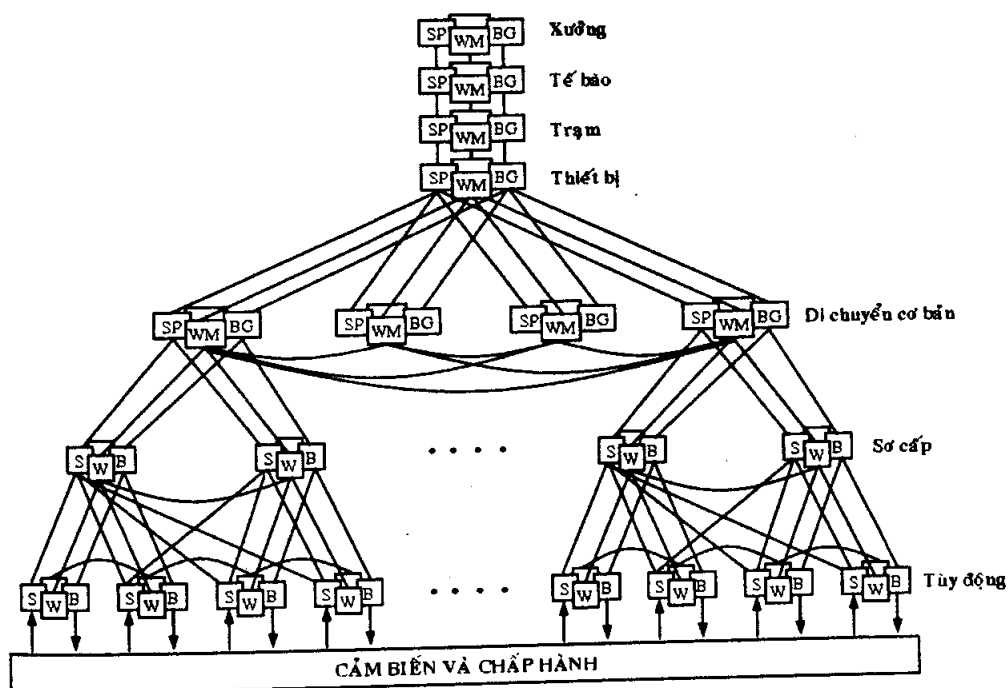
Việc tái hoạch định được thực hiện theo chu kỳ hoặc khi các tình huống khẩn cấp xuất hiện. Chu kỳ tái hoạch định có thể ngắn bằng chu kỳ cập nhật lệnh trung bình ở mỗi cấp, hoặc bằng 10% tầm hoạch định tại mỗi cấp. Điều này cho phép các bộ hoạch định thời gian thực đưa ra kế hoạch mới thường xuyên khi người vận hành đưa ra những yêu cầu mới đối với đáp ứng của hệ thống. Việc tái hoạch định càng ít thường xuyên sẽ giảm yêu cầu tốc độ tính toán đối với bộ hoạch định thời gian thực, tuy nhiên cũng có thể làm giảm hiệu quả của hệ thống điều khiển. Tái hoạch định khẩn cấp được thực hiện tức thời khi hệ thống phát hiện ra tình huống khẩn cấp.

Việc thực thi tác vụ cũng được thực hiện đồng thời và liên tục ở mọi cấp. Tại mỗi cấp, bộ điều khiển định kỳ lấy mẫu tín hiệu phản hồi, so sánh nó với tín hiệu hoạch định ở thời điểm hiện tại và ra lệnh điều khiển để hiệu chỉnh sai lệch giữa tín hiệu hoạch định và tín hiệu quan sát. Tại mỗi thời điểm lấy mẫu, tín hiệu phản hồi được kiểm tra để phát hiện các tình huống khẩn cấp, và nếu có tình huống khẩn cấp xảy ra các thủ tục phục hồi từ tình huống khẩn cấp đã định trước sẽ được gọi tức thời. Điều này cho phép hệ thống điều khiển có phản ứng xử lý tình huống khẩn cấp trong một chu kỳ điều khiển bằng các thủ tục định trước trong khi chờ các bộ hoạch định đưa ra kế hoạch mới để thực thi.

10.1.3 Phân cấp và đồng cấp

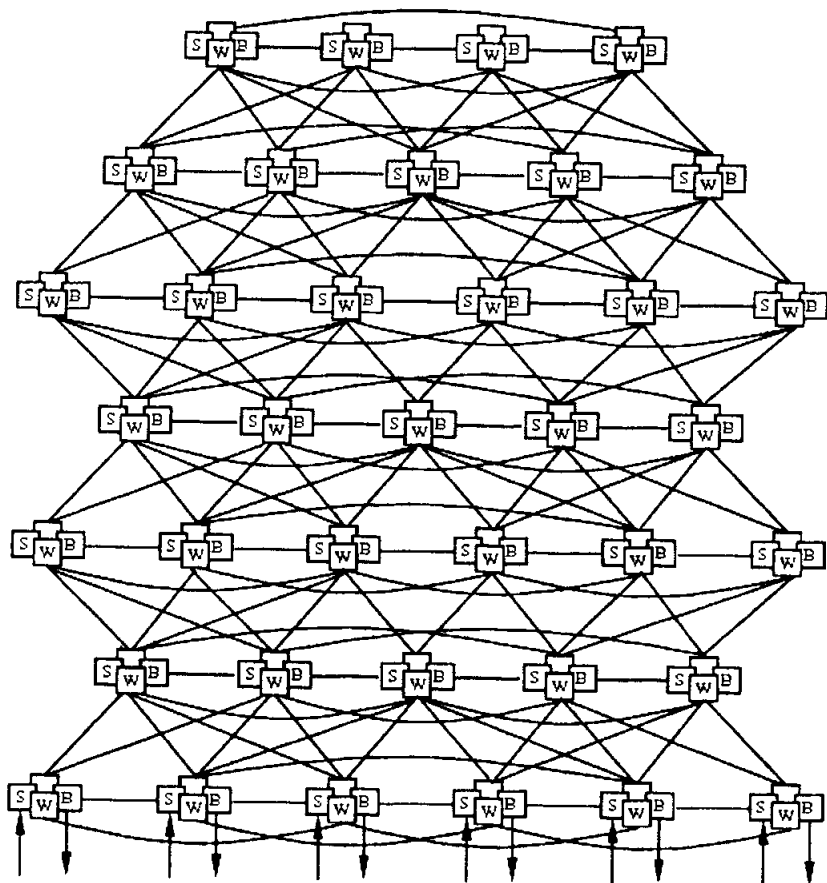
Hình 10.3 trình bày phân cấp tổ chức trong hệ thống điều khiển thông minh chi tiết hơn, bao gồm cả các quan hệ dọc và quan hệ ngang liên quan trong hệ thống. Hệ thống thông minh có tính phân cấp vì lệnh và trạng thái phản hồi truyền dọc lên xuống; chức năng xử lý cảm biến và mô hình hóa thế giới có các mức phân cấp thời gian và không gian. Hệ thống thông minh có tính đồng cấp vì dữ liệu được chia sẻ giữa các khối ở cùng mức. Ở mỗi mức phân cấp, hệ thống có những liên kết ngang bằng các đường truyền băng thông rộng giữa

các khối BG, WM, SP và VJ trong cùng một nút và giữa các nút đồng cấp. Lượng thông tin truyền ngang trong một nút rất lớn; giữa các nút trong cùng một cây lệnh ít hơn và giữa các khối tính toán trong các cây lệnh khác nhau rất ít.



Hình 10.3 Tổ chức các nút xử lý sao cho các khối BG tạo cấu trúc hình cây

Cấu hình cụ thể của cây lệnh phụ thuộc tác vụ, và do đó không nhất thiết là cấu hình tĩnh (theo thời gian). Hình 10.3 chỉ trình bày một cấu hình có thể tồn tại ở một thời điểm. Trong quá trình hoạt động, quan hệ giữa các khối trong một cấp và giữa các cấp có thể được tái cấu hình để hoàn thành các yêu cầu mục tiêu và tác vụ khác. Điều này có nghĩa là một nút tính toán bất kỳ (với các khối BG, WM, SP và VJ) có thể thuộc về một hệ thống con này tại một thời điểm và thuộc về một hệ thống con khác tại một thời điểm khác. Ví dụ các bộ phận trong cơ thể con người có thể thuộc về các hệ thống con khác nhau tùy thời điểm con người đang làm gì: miệng là một thành phần trong hệ thống vận động khi ăn và là một thành phần trong hệ thống giao tiếp khi nói; tương tự, cánh tay là một thành phần trong hệ thống vận động khi nắm và là thành phần trong hệ thống chuyển động khi bơi.

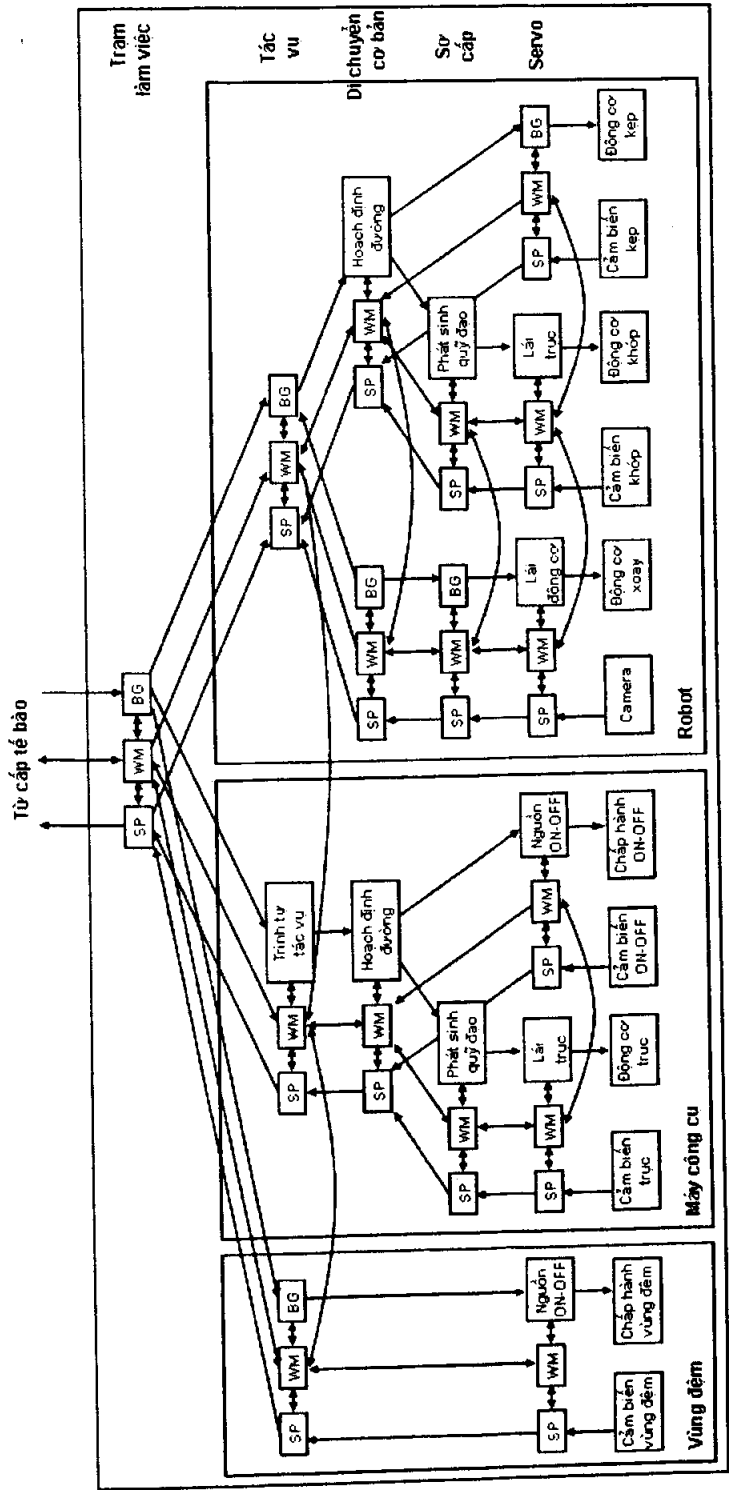


Hình 10.4 Liên kết thông tin giữa các nút trong hệ thống thông minh

Việc tái cấu hình các cây lệnh có thể thực hiện bằng cách xây dựng sẵn các kết nối giữa các khối BG ở các cấp khác nhau, nhưng các kết nối đó không phải lúc nào cũng tích cực, (H.10.4). Cấu hình này cho phép mỗi khối BG nhận thông điệp và thông số vào từ các nguồn khác nhau. Trong khi vận hành, cơ cấu chuyển mạch trong các khối BG đánh giá độ ưu tiên, phân chia tài nguyên và các hành động phối hợp để chọn tích cực các đường liên kết sẵn có. Kết quả là tại một thời điểm mỗi khối BG chỉ nhận lệnh thực hiện tác vụ từ một khối BG cấp cao hơn, và do đó các khối BG hình thành nên một cây lệnh ở mỗi thời điểm.

Các khối SP cũng được tổ chức phân cấp nhưng theo cấu trúc đồ thị phân lớp chứ không phải cấu trúc hình cây. Tại mức cao hơn, thông tin cảm biến được xử lý gia tăng mức độ trừu tượng, nhưng đường truyền thông tin cảm biến có thể phân nhánh và nhập lại bằng nhiều cách khác nhau.

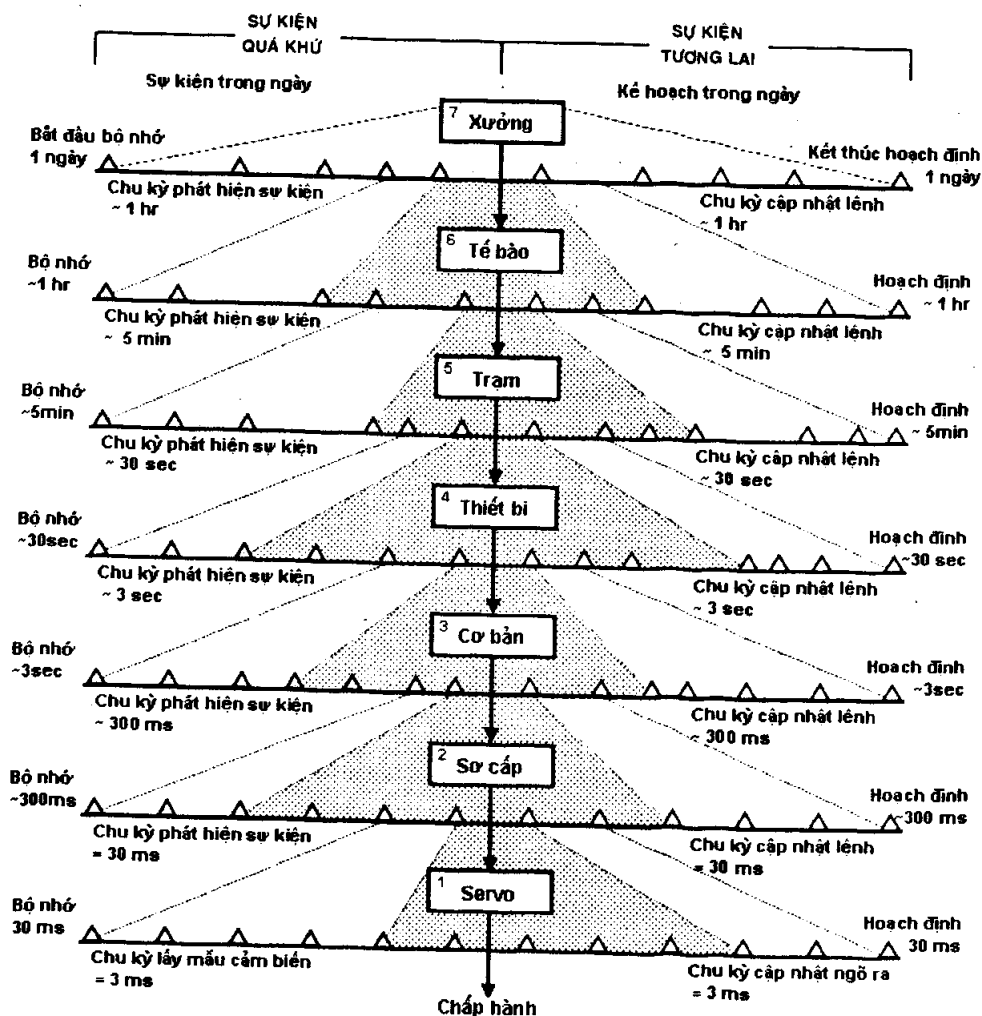
Ví dụ 10.1 Hệ thống sản xuất thông minh



Hình 10.5 Cấu trúc phân cấp một hệ thống sản xuất thông minh

Một hệ thống sản xuất thông minh thông thường có thể chia làm 7 cấp, từ cao xuống thấp bao gồm: cấp phân xưởng, cấp tế bào, cấp trạm làm việc, cấp thiết bị, cấp di chuyển cơ bản, cấp sơ cấp và cấp tùy động. Hình 10.5 minh họa năm cấp dưới cùng của hệ thống. Sau đây là sơ lược về chức năng của mỗi cấp.

Cấp 7 – Phân xưởng (Shop)



Hình 10.6 Định thời hoạt động phân chia tác vụ, xử lý cảm biến và điều khiển trong hệ thống sản xuất thông minh

Cấp phân xưởng hoạch định và điều khiển một hoặc nhiều tế bào sản xuất với chu kỳ lên đến 24 giờ (chú ý các giá trị thời gian trình bày trong ví dụ này chỉ mang tính tham khảo, thực tế chu kỳ

hoạch định và điều khiển rất khác nhau tùy ứng dụng cụ thể). Ở cấp phân xưởng, đơn đặt hàng được sắp xếp thành từng lô và lệnh được đưa ra đến cấp tế bào để triển khai kế hoạch sản xuất cho mỗi lô. Ở cấp phân xưởng, cơ sở dữ liệu mô hình thể giới gồm tên và thuộc tính của đơn đặt hàng và bảng liệt kê dụng cụ và vật liệu cần thiết để thực hiện đơn đặt hàng. Bản đồ mô tả vị trí của các tế bào sản xuất và liên kết giữa chúng.

Cấp 6 – Tế bào (Cell)

Cấp tế bào hoạch định và điều khiển hoạt động vài trạm trong khoảng thời gian 1 giờ. Các bộ phận và dụng cụ được hoạch định cho các trạm làm việc, và lệnh được đưa ra yêu cầu các trạm làm việc thực hiện gia công, kiểm tra hoặc các hoạt động vận chuyển vật liệu. Cơ sở dữ liệu mô hình thể giới gồm tên và thuộc tính của các bộ phận, dụng cụ và vật liệu cần thiết để sản xuất các bộ phận. Bản đồ mô tả vị trí của các trạm và liên kết giữa chúng. Ngõ ra của cấp tế bào là ngõ vào của cấp trạm làm việc.

Cấp 5 – Trạm làm việc (Workstation)

Trạm làm việc hoạch định tác vụ và điều khiển hoạt động trong mỗi trạm với tầm hoạch định khoảng 5 phút. Mỗi trạm làm việc có thể bao gồm một nhóm máy móc, thiết bị vận chuyển vật liệu, và các ngăn đệm chứa bộ phận và dụng cụ. Kế hoạch được triển khai và lệnh được đưa ra yêu cầu các máy hoạt động gia công vật liệu để sản xuất ra các bộ phận. Cơ sở dữ liệu mô hình thể giới gồm tên và thuộc tính của các bộ phận, dụng cụ, và các ngăn đệm tại trạm. Bản đồ mô tả vị trí của các bộ phận, dụng cụ và ngăn đệm.

Cấp 4 – Thiết bị (Equipment)

Cấp thiết bị hoạch định tác vụ và điều khiển hoạt động của mỗi máy với tầm hoạch định khoảng 30 giây. Cấp bốn phân chia mỗi tác vụ thành các di chuyển cơ bản của các bộ phận cấu thành thiết bị. Kế hoạch được triển khai thành chuỗi các di chuyển cơ bản của dụng cụ và tay gắp, và lệnh được đưa ra yêu cầu di chuyển dụng cụ và tay gắp tiếp cận, gắp, di chuyển, cắt, khoan, phay hoặc đồ các bộ phận. Cơ sở dữ liệu mô hình thể giới gồm tên và thuộc tính của các bộ phận, như kích thước, hình dạng và đặc tính vật liệu. Bản đồ gồm các bản vẽ hình dạng của các bộ phận và vị trí tương đối giữa các đặc trưng của bộ phận.

Cấp 3 – Di chuyển cơ bản (*Elementary move*)

Cấp di chuyển cơ bản hoạch định và điều khiển các chuyển động đơn giản trong thời đoạn vài giây, chẳng hạn như “di chuyển dọc theo”, “di chuyển đến điểm”, “phay bề mặt”, “khoan lỗ”, “đo bề mặt”... Kế hoạch được triển khai và lệnh được đưa ra xác định đường di chuyển an toàn cho các dụng cụ, tay gấp và các đầu dò kiểm tra để tránh tình trạng xung đột và kỳ dị, và đảm bảo chất lượng của bộ phận được gia công và an toàn quá trình sản xuất. Cơ sở dữ liệu mô hình thể giới gồm tên và thuộc tính của các đặc trưng của bộ phận như bề mặt, lỗ, hốc, rãnh, gờ, ren... Bản đồ gồm các bản vẽ hình dạng các đặc trưng và vị trí tương đối giữa biên của các đặc trưng.

Cấp 2 – Sơ cấp (*Primary*)

Cấp sơ cấp hoạch định đường di chuyển của dụng cụ, tay gấp và đầu dò kiểm tra sao cho tối thiểu thời gian và tối ưu hóa chất lượng. Cấp sơ cấp tính toán biểu đồ tăng tốc, giảm tốc của dụng cụ và tay gấp có để ý đến tương tác động học giữa khối lượng, ma sát, lực và thời gian. Tầm hoạch định ở cấp 2 khoảng 300ms. Mô hình thể giới gồm tên và thuộc tính của các đặc trưng dạng đường thẳng như các đường thẳng, các đoạn quỹ đạo và các đỉnh. Bản đồ (nếu có) gồm hình chiếu phối cảnh của các đặc trưng dạng đường thẳng như cạnh, đường thẳng, hoặc quỹ đạo của dụng cụ (hoặc bộ phận tác động đầu cuối).

Cấp 1 – Tùy động (*Servo*)

Cấp tùy động chuyển đổi các lệnh từ đường đi của dụng cụ thành tọa độ của cơ cấu chấp hành ở các liên kết. Bộ hoạch định nội suy giữa các điểm quỹ đạo sơ cấp trong 30ms. Bộ điều khiển lái từng cơ cấu chấp hành và động cơ riêng lẻ theo quỹ đạo nội suy. Điều khiển tùy động vị trí, vận tốc và lực có thể được thực hiện. Lệnh yêu cầu mô men hoặc công suất chấp hành được đưa ra với chu kỳ khoảng 3ms (hoặc tốc độ khác tùy thuộc vào đặc tính động học của máy và yêu cầu chất lượng tùy động). Cấp tùy động cũng điều khiển tín hiệu ra lái các phần tử chấp hành rời rạc như relay, van solenoid... Mô hình thể giới gồm giá trị của các biến trạng thái như vị trí liên kết, vận tốc, lực, giá trị cảm biến độ gần, vị trí các công tắc hành trình, trạng thái của các đầu dò tiếp xúc, thuộc tính hình ảnh thu thập dùng camera.

Chú ý rằng, ở cấp tùy động và cấp sơ cấp, tần xuất ra lệnh điều khiển là không đổi theo chu kỳ định trước; trong khi đó ở cấp di chuyển cơ bản hoặc cấp cao hơn tần xuất ra lệnh điều khiển có thể thay đổi vì ở các cấp này thông thường lệnh được xuất ra theo sự kiện.

10.2 PHÁT SINH HÀNH VI

Mỗi môđun phát sinh hành vi (*BG-Behavior Generation*) ở mỗi cấp gồm ba môđun con: gán việc, hoạch định và thực thi.

Môđun gán việc (JA-Job Assignment Submodule)

JA có nhiệm vụ phân chia tác vụ theo không gian. JA phân yêu cầu tác vụ ở đầu vào thành N công việc phân biệt được thực thi bởi N hệ thống con vật lý phân biệt, trong đó N là số hệ thống con được gán cho khối BG. JA cũng có nhiệm vụ gán các dụng cụ và cấp phát tài nguyên vật lý cho mỗi hệ thống con phụ thuộc vào nó để các hệ thống con sử dụng khi thực thi công việc được gán.

Môđun hoạch định (PL(j)-Planner (j) Submodule, $j = 1, 2, \dots, N$)

Đối với mỗi trong số N hệ thống con có một khối PL(j) làm nhiệm vụ phân tích công việc đã gán cho mỗi hệ thống con thành một chuỗi theo thời gian các tác vụ con. PL(j) còn có nhiệm vụ giải quyết các mâu thuẫn và các ràng buộc qua lại giữa các kế hoạch giả định của các khối con.

Môđun thực thi (EX(j)-Executor (j) Submodule, $j = 1, 2, \dots, N$)

Tương ứng với mỗi khối hoạch định PL(j) có một môđun thực thi EX(j). Môđun EX(j) có nhiệm vụ thực hiện thành công kế hoạch đưa ra bởi môđun PL(j) tương ứng. Khi môđun thực thi được mô hình thế giới thông báo một tác vụ con trong kế hoạch hiện tại đã được hoàn tất thành công, nó sẽ chuyển sang thực hiện tác vụ con kế tiếp trong kế hoạch hiện tại. Khi tất cả các tác vụ con trong kế hoạch hiện tại đã thực hiện thành công, môđun thực thi sẽ chuyển sang thực hiện tác vụ con đầu tiên trong kế hoạch tiếp theo. Nếu thông tin phản hồi cho thấy có lỗi trong một tác vụ con đã hoạch định, môđun thực thi EX(j) sẽ chuyển sang thực hiện ngay tác vụ con khẩn cấp đã hoạch định sẵn, đồng thời môđun hoạch định PL(j) tương ứng sẽ bắt đầu chọn lựa công việc hoặc tạo ra một chuỗi phục hồi sự cố có thể thay thế cho kế hoạch đã bị lỗi trước đó. Tín hiệu ra của môđun thực thi

trong khối BG ở một cấp nào đó chính là tín hiệu vào của môđun gắn việc trong khối BG ở cấp thấp kế tiếp.

Tri thức tác vụ (Task Knowledge)

Cơ sở để phát sinh hành vi là sự biểu diễn và sử dụng tri thức tác vụ. Tác vụ là một công việc phải được hoàn thành, hoặc một hành động phải được thực hiện. Đối với bất cứ khối BG nào cũng tồn tại một tập hợp các tác vụ mà nó biết phải thực hiện ra sao. Mỗi tác vụ trong tập hợp này được gán cho một tên. Bảng từ vựng tác vụ là tập hợp các tên tác vụ đã gán cho tập hợp các tác vụ mà khối BG có thể thực hiện.

Tri thức làm thế nào để thực hiện một tác vụ có thể biểu diễn dưới dạng một khung cấu trúc dữ liệu. Ví dụ một cấu trúc tác vụ được trình bày ở bảng 10.1, gồm các phần chính như sau:

- **Tên tác vụ** là một chuỗi nhận biết loại hành động được thực hiện.
- **Mục tiêu** có thể là vector xác định giá trị đặt hoặc trạng thái mong muốn có thể đạt được nhờ thực hiện tác vụ. Mục tiêu cũng có thể là bản đồ, đồ thị hoặc cấu trúc dữ liệu hình học xác định điều kiện mong muốn của một đối tượng.
- **Yêu cầu** gồm các thông tin cần thiết trong khi thực hiện tác vụ. Yêu cầu có thể gồm danh sách các biến trạng thái, bản đồ, và/hoặc cấu trúc dữ liệu hình học phản ánh tình huống thực tế đang tồn tại ở thế giới bên ngoài. Yêu cầu cũng có thể bao gồm tài nguyên, dụng cụ, vật liệu, thời gian và điều kiện cần thiết để thực hiện tác vụ.
- **Thủ tục** có thể gồm một tập hợp các kế hoạch đã tính trước hoặc các bước phân chia tác vụ, hoặc một hay nhiều thuật toán hoạch định để đưa ra kế hoạch. Ví dụ, thủ tục có thể gồm một tập hợp các luật "IF ... THEN..." để chọn một kế hoạch thích hợp với tình huống hiện tại. Thủ tục cũng có thể gồm thuật toán hoạch định để xác định sự khác biệt giữa tình huống mong muốn và tình huống hiện tại. Sự khác biệt này có thể xem như là sai số mà khối con hoạch định tác vụ cố gắng giảm thiểu. Mỗi khối con hoạch định sau đó đưa ra một chuỗi các tác vụ con được tính toán sao cho giảm thiểu sai số trong khoảng thời gian từ thời điểm hiện tại đến tầm hoạch định. Mỗi khối con thực thi sẽ hoạt động như là một bộ điều khiển hồi tiếp, cố gắng lái các hệ thống con tương ứng theo kế hoạch. Thủ tục cũng có thể gồm các thủ tục khẩn cấp sẽ được thực thi tức thời khi phát hiện ra điều kiện không cho phép.

Bảng 10.1 Một cấu trúc tác vụ tiêu biểu

TASKNAME	Tên tác vụ
Mục đích	Sự kiện hoặc điều kiện thành công kết thúc tác vụ
Đối tượng	Chỉ rõ vật bị tác động
Thông số	Độ ưu tiên Trạng thái (tích cực, chờ, không tích cực) Định thời (tốc độ, thời gian hoàn thành) Hệ tọa độ trong đó tác vụ được biểu diễn Dung sai
Tác nhân	Chỉ rõ hệ thống con sẽ thực hiện tác vụ
Yêu cầu	Thông tin phản hồi cần thiết từ mô hình thế giới khi thực hiện tác vụ Dụng cụ, thời gian, tài nguyên và vật liệu cần thiết để thực hiện tác vụ Điều kiện cho phép cần phải thỏa mãn để bắt đầu hoặc tiếp tục tác vụ Điều kiện không cho phép để ngắt hoặc hủy bỏ tác vụ
Thủ tục	Các kế hoạch tính trước hoặc các bước thực hiện tác vụ Các thuật toán hoạch định Các hàm có thể được gọi Các thủ tục khẩn cấp cho tương ứng với mỗi điều kiện không cho phép

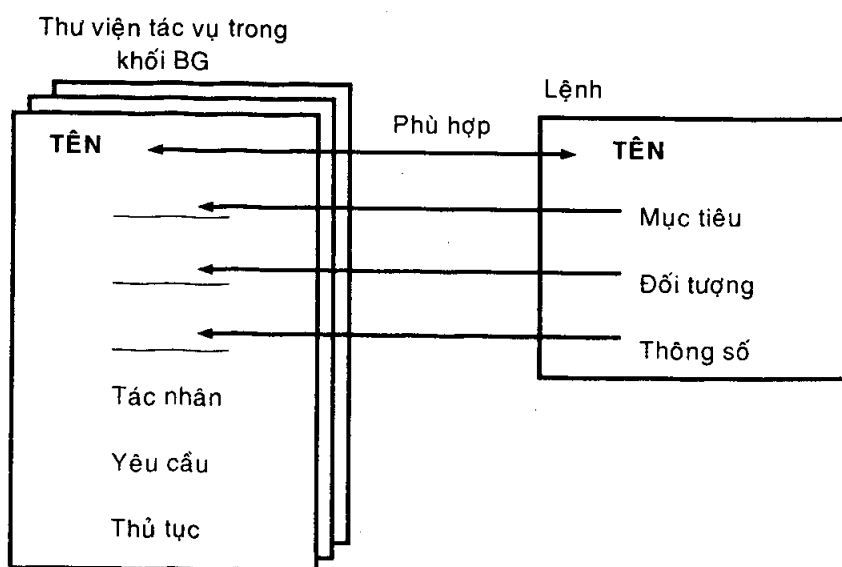
Trong các kế hoạch liên quan đến hoạt động đồng thời của các hệ thống con khác nhau, có thể có các ràng buộc qua lại. Ví dụ, sự kiện bắt đầu một tác vụ con của một hệ thống con có thể phụ thuộc và sự kiện kết thúc tác vụ con của một hệ thống con khác. Một số tác vụ có thể đòi hỏi một số hệ thống con phải hoạt động đồng thời và phối hợp nhau. Điều này yêu cầu phải có sự phối hợp trong cả việc hoạch định kế hoạch và việc thực hiện kế hoạch.

Thư viện các cấu trúc tác vụ nằm trong mỗi khối BG xác định khả năng của khối BG đó. Các tên cấu trúc tác vụ trong thư viện xác định tập hợp các lệnh tác vụ mà khối BG đó chấp nhận. Tất nhiên, có thể có một số cách hoàn tất tác vụ khác nhau. Các cách phân chia tác vụ khác nhau có thể biểu diễn bằng đồ thị AND/OR trong phần thủ tục của cấu trúc tác vụ.

Tác nhân, yêu cầu và thủ tục trong cấu trúc tác vụ chỉ rõ cho khối BG biết “làm thế nào để thực hiện” tác vụ. Mục tiêu, đối tượng và thông số chỉ rõ “phải làm gì”, “trên đối tượng nào”, “khi nào”,

“nhanh thế nào”... Hình 10.7 minh họa sự kích hoạt một cấu trúc tác vụ nằm trong thư viện tác vụ của khối BG bởi một lệnh tác vụ. Khi khối BG nhận một lệnh tác vụ, nó quét thư viện cấu trúc tác vụ để tìm một tên tác vụ phù hợp với tên lệnh. Khi tìm được tên tác vụ phù hợp, các thuộc tính mục tiêu, đối tượng và thông số từ lệnh được truyền vào cấu trúc tác vụ. Điều này kích hoạt cấu trúc tác vụ, và ngay khi các yêu cầu liệt kê trong cấu trúc tác vụ được thỏa mãn, khối BG bắt đầu thực hiện kế hoạch tác vụ.

Tri thức tác vụ để giải quyết một công việc cụ thể nào đó thông thường không dễ đưa ra. Tuy nhiên, khi tri thức tác vụ đã biết thì nó có thể được sử dụng và nhân bản.

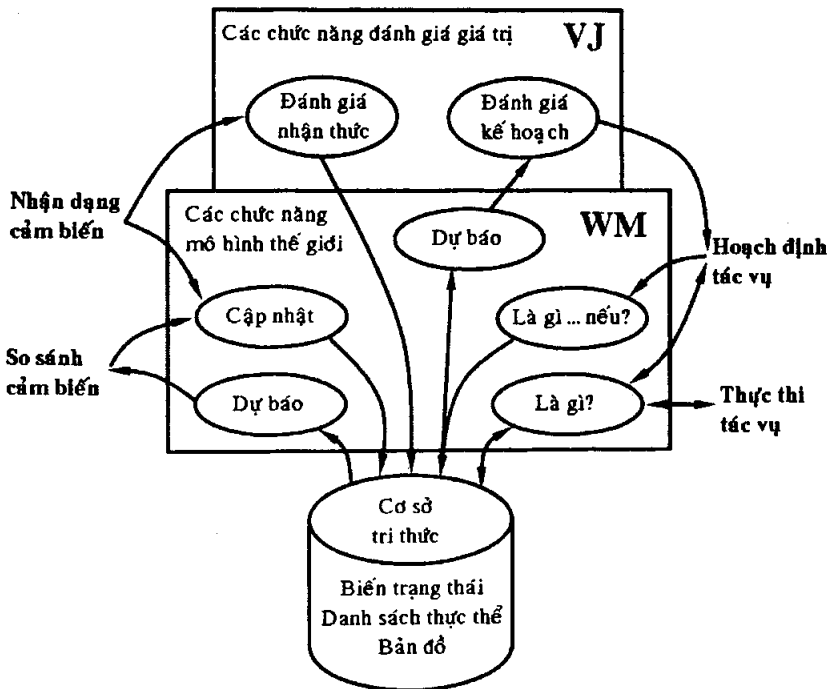


Hình 10.7 Quan hệ giữa lệnh và cấu trúc tác vụ

10.3 MÔ HÌNH THẾ GIỚI

Mô hình thế giới (WM-World Modeling) là sự mô tả thế giới bên ngoài trong hệ thống thông minh. WM hoạt động như là một bộ đệm giữa các khối xử lý cảm biến và khối phát sinh hành vi. Điều này cho phép mô đun phát sinh hành vi có thể hoạt động dựa trên thông tin có thể không quan sát được tức thời và trực tiếp từ cảm biến, và cho phép mô đun xử lý cảm biến thực hiện việc ước lượng đệ quy dựa trên các giá trị đo trong quá khứ. Mô hình thế giới có cấu trúc phân cấp và phân tán, trong mỗi nút tại mọi cấp của hệ thống điều khiển thông

minh đều có một mô đun mô hình thế giới với cơ sở tri thức (KD-*Knowledge Database*). Cơ sở tri thức hình thành nơi lưu trữ dữ liệu phân tán, phân cấp và thụ động. Mô hình thế giới quy định các hoạt động trên cơ sở tri thức. Tại mỗi cấp, mô đun mô hình thế giới thực hiện chức năng minh họa ở hình 10.8.



Hình 10.8 Các chức năng của mô hình thế giới

1- Mô đun WM duy trì cơ sở tri thức KD, giữ cho cơ sở tri thức luôn được cập nhật và phù hợp. Trong vai trò này, mô đun WM thực hiện chức năng của hệ thống quản lý cơ sở dữ liệu, nó cập nhật sự đánh giá trạng thái của cơ sở tri thức dựa trên sự tương quan và khác biệt giữa dự báo của mô hình thế giới và quan sát từ cảm biến tại mỗi mức phân cấp. Các mô đun WM cập nhật các thực thể, trạng thái, và sự kiện mới nhận biết vào cơ sở dữ liệu; và xóa khỏi cơ sở dữ liệu những thực thể và trạng thái mà mô đun xử lý cảm biến xác định rằng không còn tồn tại ở thế giới bên ngoài. Mô đun WM cũng cập nhật các ước lượng đưa ra bởi các mô đun VJ về sự tin cậy của các biến trạng thái trong cơ sở tri thức, các hệ số tin cậy khác nhau được gán cho các loại biến trạng thái khác nhau.

2- Môđun WM đưa ra dự báo về tín hiệu vào từ cảm biến để các môđun xử lý cảm biến (SP) tương ứng sử dụng. Trong vai trò này, môđun WM thực hiện chức năng của một bộ dự báo trạng thái, đưa ra dự báo cho phép môđun xử lý cảm biến thực hiện việc sửa sai và lọc dự báo. Dự báo đưa ra bởi WM dựa trên trạng thái của tác vụ và các trạng thái ước lượng về thế giới bên ngoài.

3- Môđun WM trả lời câu hỏi “Là gì?” đưa ra bởi các môđun hoạch định và môđun thực thi trong môđun BG tương ứng. Trong vai trò này, môđun WM thực hiện chức năng của một máy chủ dữ liệu (*data server*). Ước lượng của môđun WM về trạng thái thế giới hiện tại được sử dụng bởi các bộ hoạch định trong môđun BG như là điểm khởi đầu để hoạch định và được sử dụng bởi các bộ thực thi trong môđun BG để điều khiển hoặc rẽ nhánh có điều kiện.

4- Môđun trả lời câu hỏi “Là gì ... nếu” đưa ra bởi các bộ hoạch định trong môđun BG. Trong vai trò này, môđun WM thực hiện chức năng mô phỏng, đưa ra kết quả trạng thái mong đợi từ các tác động giả thiết bởi bộ hoạch định. Kết quả dự báo từ các mô phỏng của WM được gửi đến môđun đánh giá giá trị. Đối với mỗi tác động giả thiết bởi môđun BG, một dự báo của môđun WM được đưa ra, và đánh giá của môđun VJ được gửi lại cho bộ hoạch định của môđun BG. Vòng BG-WM-VJ này cho phép bộ hoạch định của môđun BG chọn được chuỗi tác động giả thiết tạo ra được đánh giá tốt nhất khi kế hoạch được thực thi.

Cơ sở tri thức (KD—Knowledge Database)

Cơ sở tri thức mô hình thế giới bao gồm cả thông tin có sẵn trong hệ thống thông minh trước khi tác động bắt đầu và tri thức có sau thu được từ việc cảm nhận môi trường khi tác động xảy ra. Cơ sở tri thức biểu diễn thông tin về không gian, thời gian, thực thể, sự kiện, trạng thái của thế giới và các quy luật tự nhiên. Tri thức về không gian biểu diễn dưới dạng bản đồ. Tri thức về thực thể, sự kiện và trạng thái được biểu diễn dưới dạng bảng liệt kê. Tri thức về các quy luật vật lý, hóa học, quang học và các quy luật logic toán học được biểu diễn dưới dạng các hàm trong WM, các hàm này dùng để mô phỏng và dự báo kết quả các tác động giả định. Quy luật tự nhiên cũng có thể biểu diễn dưới dạng các công thức, hoặc các quy tắc “nếu ... thì ...”.

Mô hình thế giới cũng bao gồm cả thông tin về chính bản thân hệ thống thông minh, như giá trị gán cho độ ưu tiên mục tiêu, giá trị

thuộc tính gán cho các đối tượng và sự kiện; thông số xác định mô hình động học và động lực học của cánh tay robot hoặc máy công cụ; các biến trạng thái mô tả áp suất bên trong, nhiệt độ, mức nhiên liệu; trạng thái của tất cả các quá trình đang thực thi trong mỗi môđun BG, SP, WM, VJ...

Tính đúng đắn và nhất quán của tri thức mô hình thế giới được kiểm tra bằng cảm biến. Cơ chế xử lý cảm biến so sánh sự khác biệt giữa dự báo của mô hình thế giới và quan sát từ cảm biến. Những khác biệt này được một thuật toán đánh giá đệ quy sử dụng để giữ các biến trạng thái của mô hình mô tả tốt nhất trạng thái của thế giới.

Thông tin trong cơ sở tri thức mô hình thế giới có thể được tổ chức thành các biến trạng thái, thông số hệ thống, cấu trúc thực thể và bản đồ.

- **Biến trạng thái:** xác định giá trị hiện tại của thực thể và thuộc tính của hệ thống. Biến trạng thái có thể cho biết trạng thái của đồng hồ; vị trí, hướng và vận tốc của tay gấp; vị trí, vận tốc và mômen của một phần tử chấp hành; hoặc trạng thái của một môđun tính toán.

- **Thông số hệ thống:** xác định đặc tính động học và động lực học của đối tượng điều khiển. Thông số hệ thống cũng xác định các biến đổi tọa độ cần thiết để chuyển lệnh và thông tin cảm biến từ một hệ tọa độ làm việc này sang hệ tọa độ làm việc khác.

- **Cấu trúc thực thể:** là một danh sách biểu tượng, trong đó tên thực thể đứng đầu danh sách và tri thức về thực thể được lưu dưới dạng cặp thuộc tính-giá trị. Mô hình thế giới gồm danh sách tất cả các thực thể mà hệ thống thông minh biết đến. Có hai loại thực thể: tổng quát và cụ thể. Ở các cấp khác nhau có các cấp thực thể khác nhau. Ví dụ ở mức 1 cấu trúc thực thể mô tả điểm, cấp 2 mô tả đường và đỉnh, cấp 3 mô tả mặt, cấp 4 mô tả đối tượng, cấp 5 mô tả nhóm đối tượng, cấp 6 và cấp cao hơn mô tả bậc cao hơn nhóm.

- **Bản đồ:** mô tả sự phân bố các thực thể trong không gian. Mỗi điểm trên bản đồ có thể có một con trỏ chỉ đến tên của thực thể tương ứng tại điểm đó. Mỗi điểm cũng có thể có một hoặc nhiều cặp thuộc tính-giá trị. Bản đồ cũng có thể biểu diễn bằng đồ thị (*graph*) chỉ rõ đường đi giữa các vị trí, ví dụ như đường đi của xe robot giữa các trạm làm việc.

10.4 XỬ LÝ CẢM BIẾN

Xử lý cảm biến là cơ chế nhận thức thiết lập và duy trì sự tương ứng giữa mô hình thế giới bên trong và thế giới thực tế bên ngoài. Chức năng của các môđun xử lý cảm biến là trích thông tin về thực thể, sự kiện, trạng thái và quan hệ ở thế giới bên ngoài, nhằm giữ cho mô hình thế giới luôn chính xác và cập nhật.

Cập nhật bản đồ

Bản đồ trong mô hình thế giới có thể được cập nhật bởi giá trị đo lường các điểm, cạnh, mặt. Các thông tin này có thể rút ra từ các cảm biến thị giác, tiếp xúc, siêu âm, radar hay laser. Phương pháp trực tiếp nhất để đo điểm, cạnh và mặt là dùng cảm biến tiếp xúc. Trong môi trường sản xuất, máy đo tọa độ sử dụng các đầu dò tiếp xúc để đo vị trí 3D của các điểm. Đầu dò tiếp xúc gắn trên máy công cụ và cảm biến xúc giác gắn trên robot cũng có thể dùng để đo vị trí. Từ các dữ liệu này, thuật toán xử lý cảm biến có thể tính được hướng và vị trí của mặt, hình dáng của các lỗ, khoảng cách giữa các mặt, kích thước của các bộ phận. Các phương pháp khác có thể đo điểm, cạnh, và mặt là thị giác lập thể, quang trắc, đo dùng laser, siêu âm...

Nhận biết và phát hiện

Nhận biết là thiết lập sự tương ứng một-một giữa một thực thể với một mô hình. Quá trình nhận thức có thể từ trên xuống, từ dưới lên hoặc kết hợp đồng thời. Quá trình nhận thức từ trên xuống bắt đầu bằng việc giả định một mô hình và so sánh thuộc tính dự báo của nó với thực thể quan sát. Một giá trị tương quan chéo giữa mô hình và thực thể được tính. Nếu giá trị tương quan vượt quá một ngưỡng chọn trước, thực thể được xem như là đã được nhận biết; ngược lại, mô hình bị loại bỏ và quá trình nhận thức được lặp lại bằng một mô hình giả định khác.

Quá trình nhận thức từ dưới lên gồm việc áp dụng các bộ lọc và mặt nạ vào dữ liệu cảm biến và tính các thuộc tính ảnh. Các thuộc tính sau đó được lưu trữ trong mô hình thế giới, hoặc so sánh với thuộc tính của các thực thể sẵn có trong mô hình thế giới.

Phát hiện sự kiện tương tự như nhận biết thực thể. Các trạng thái quan sát được của thế giới thực được so sánh với trạng thái dự báo bởi mô hình thế giới. Một hệ số tương quan giữa sự kiện quan sát thực

tế và sự kiện dự báo bởi mô hình được tính toán. Nếu hệ số tương quan vượt quá mức ngưỡng chọn trước thì sự kiện đã được phát hiện.

Môđun xử lý cảm biến

Môđun xử lý cảm biến có chức năng thu thập dữ liệu từ cảm biến; lọc và tích hợp dữ liệu, diễn giải dữ liệu. Môđun xử lý cảm biến có thể thực hiện triết tiêu nhiễu (như lọc Kalman), trích đặc trưng, nhận dạng mẫu, hay đoán hình ảnh. Ở cấp càng cao trong hệ thống phân cấp, xử lý cảm biến càng diễn dịch trừu tượng và tín hiệu vào từ nhiều cảm biến khác nhau được tích hợp theo không gian và thời gian thành mô hình về thế giới bên ngoài.

Môđun xử lý cảm biến có năm hoạt động bao gồm chuyển đổi tọa độ, so sánh, tích hợp thời gian, tích hợp không gian, nhận biết/phát hiện như trình bày ở hình 10.9.

- **Chuyển đổi tọa độ:** Trước khi một biến ước lượng bởi mô hình có thể được so sánh với một biến quan sát từ cảm biến, chúng phải được chuyển đổi về cùng hệ tọa độ.

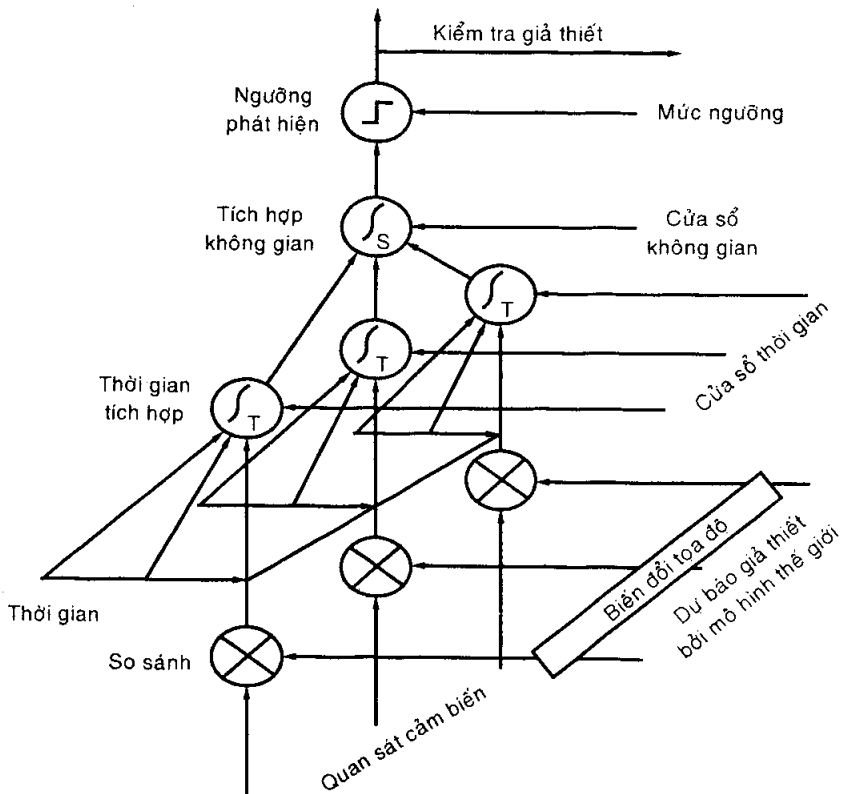
- **So sánh:** Mỗi bộ so sánh tương ứng một biến quan sát từ cảm biến với giá trị dự báo của mô hình. Biến dự báo có thể được nhân hoặc trừ với biến quan sát để tính chỉ số tương quan và chỉ số sai lệch giữa biến dự báo và biến quan sát. Chỉ số tương quan cho biết dự báo của mô hình là đúng. Chỉ số sai lệch cho biết mô hình không phù hợp với dữ liệu quan sát.

- **Tích hợp thời gian** tích lũy sự tương đồng và sự khác biệt giữa giá trị dự báo và giá trị quan sát trong một khoảng thời gian. Tích hợp thời gian tác động lên dữ liệu cảm biến có thể tính tổng hoặc giá trị trung bình của thông tin từ cảm biến trong một cửa sổ thời gian. Tích hợp thời gian tác động trên chỉ số tương quan và chỉ số sai lệch có thể tạo ra hàm tương quan chéo thời gian và hàm hiệp phương sai giữa mô hình và dữ liệu quan sát. Hàm tương quan chéo thời gian và hàm hiệp phương sai phản ánh mức độ phù hợp giữa đặc tính động của mô hình và đối tượng thực tế.

- **Tích hợp không gian** tích lũy sự tương đồng và khác biệt giữa giá trị dự báo và giá trị quan sát trong một miền không gian, lấy tổng thông tin cảm biến từ nhiều nguồn tại một thời điểm. Tích hợp không gian đưa ra hàm tương quan chéo không gian giữa mô hình và

dữ liệu quan sát, cho biết các tính chất hình học của mô hình có phù hợp với đối tượng hay không.

- **Nhận biết/phát hiện** khi các hàm tương quan thời gian–không gian vượt quá một giá trị ngưỡng chọn trước thì đối tượng được nhận biết hoặc sự kiện được phát hiện.



Hình 10.9 Các hoạt động của môđun xử lý cảm biến

10.5 ĐÁNH GIÁ GIÁ TRỊ

Đánh giá giá trị cung cấp các tiêu chuẩn để chọn lựa thông minh. Đánh giá giá trị ước lượng chi phí, rủi ro và lợi ích của kế hoạch và tác động; sự mong muốn, sự thu hút và sự không chắc chắn của đối tượng và sự kiện. Môđun đánh giá giá trị đưa ra các ước lượng có thể biểu diễn dưới dạng các biến trạng thái giá trị, các biến này có thể được gán vào bảng thuộc tính trong cấu trúc thực thể.

Thuật toán đánh giá giá trị có thể đánh giá rủi ro và tính toán mức độ không chắc chắn trong việc nhận biết thực thể và phát hiện

sự kiện. Vì vậy tri thức trong mô hình thế giới có thể được gán là tin cậy hoặc không chắc chắn.

Thuật toán đánh giá chi phí/lợi ích có thể dùng để hoạch định kế hoạch thực hiện tác vụ sao cho cực tiểu chi phí và cực đại lợi ích. Các biến trạng thái có thể được gán vào bảng thuộc tính kế hoạch và hành động trong cấu trúc tác vụ, vì vậy kế hoạch và tác vụ có thể có thuộc tính tốt hay xấu, chi phí cao hay thấp, rủi ro hay an toàn.

Đánh giá giá trị có thể đánh giá các sự kiện là quan trọng hoặc tầm thường. Việc sử dụng bộ nhớ có thể được tối ưu hóa bằng cách chỉ lưu trữ những sự kiện và tình huống được đánh giá là quan trọng. Độ ưu tiên là biến trạng thái để đánh giá tầm quan trọng. Độ ưu tiên có thể được gán cho cấu trúc tác vụ, nhờ đó mà với mỗi tác vụ bộ hoạch định và thực thi có thể quyết định phải làm gì trước, dành nỗ lực bao nhiêu, độ rủi ro phải cẩn trọng, chi phí bao nhiêu là chấp nhận được... Các biến trạng thái giá trị có thể sử dụng bởi môđun phát sinh hành vi trong việc hoạch định và thực thi. Độ ưu tiên còn có thể xác định mức độ báo động, chiến lược lựa chọn để hoạch định tác vụ trong môi trường đối nghịch.

Môđun đánh giá giá trị

Các biến trạng thái giá trị có thể được gán bởi người lập trình hoặc người vận hành; hoặc các biến này có thể được tính bởi môđun đánh giá giá trị. Tín hiệu vào môđun đánh giá giá trị mô tả thực thể, sự kiện, tình huống và trạng thái. Tín hiệu vào cũng có thể là tỷ số tín hiệu/nhiều của cảm biến, phương sai giữa quan sát của cảm biến và dự báo của mô hình, mức độ thành công trong việc hoàn thành mục tiêu, tín hiệu thưởng/phạt từ các nguồn khác nhau. Các hàm đánh giá giá trị tính chỉ số chi phí, rủi ro, lợi ích. Tín hiệu ra của môđun đánh giá giá trị là các biến trạng thái giá trị, các biến này có thể được gán cho đối tượng, sự kiện hoặc kế hoạch.

Cơ chế đánh giá giá trị thông thường có thể định nghĩa như là một hàm toán học hoặc logic dưới dạng:

$$E = V(S)$$

trong đó: E - vector trạng thái giá trị (tín hiệu ra)

S - vector trạng thái vào xác định các trạng thái trong mô hình thế giới

V - hàm đánh giá giá trị tính E khi biết S .

10.6 ỨNG DỤNG HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH

Các mục trên đã trình bày tổng quát về cấu trúc hệ thống điều khiển thông minh, tùy theo từng ứng dụng cụ thể mà hệ thống điều khiển thông minh có thể có cấu trúc đơn giản hơn cấu trúc tổng quát đã trình bày ở trên. Hiện nay lý thuyết điều khiển thông minh được ứng dụng rất rộng rãi trong các hệ thống sản xuất khác nhau. Sau đây là giới thiệu một số hệ thống điều khiển thông minh đã được thiết kế, ứng dụng thực nghiệm và công bố trên các tạp chí chuyên ngành trong vài năm gần đây. Độc giả quan tâm có thể tham khảo chi tiết các vấn đề liên quan, đặc biệt là cấu trúc phần mềm, trong các tài liệu trích dẫn.

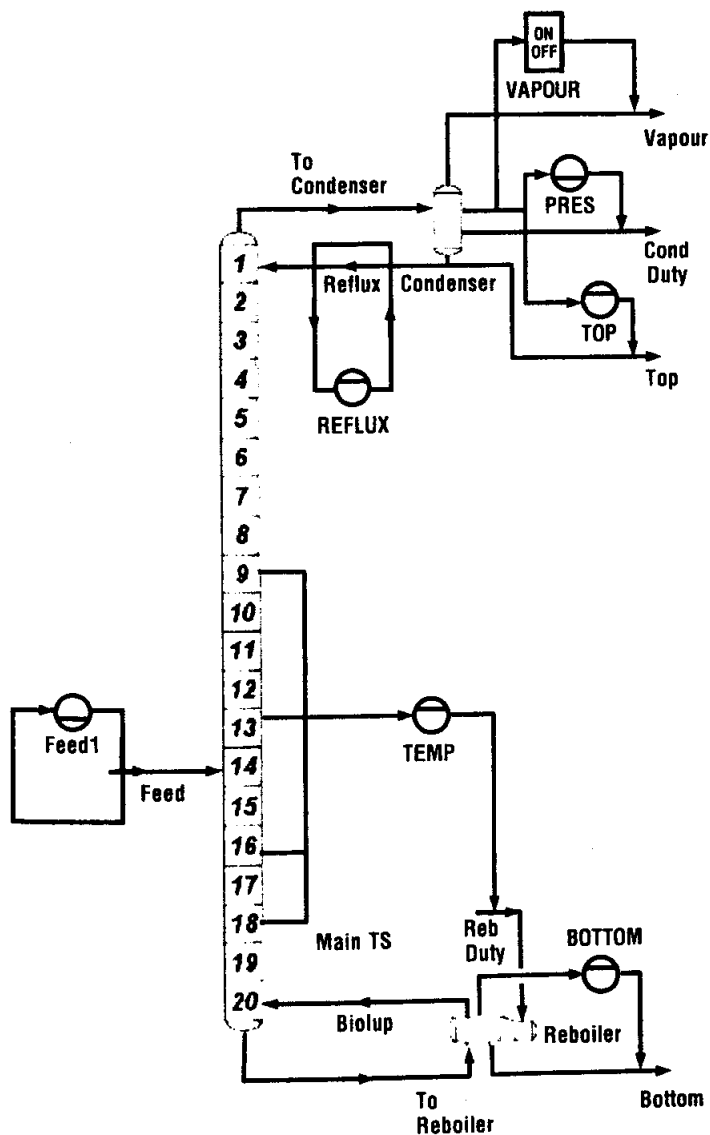
Ví dụ 10.2 Điều khiển thông minh khởi động cột chưng cất ([30])

Xét cột chưng cất gồm 20 ngăn có chức năng tách nước ra khỏi methanol (H.10.10), ở chế độ hơi lý tưởng giá trị các biến trạng thái xác lập như sau:

- Hỗn hợp chất vào: 50% nước và 50% methanol ở nhiệt độ $75,9^{\circ}\text{C}$.
- Lưu lượng vào: 236kg mol/h ($5442,9\text{kg/h}$, $6,14\text{m}^3/\text{h}$) (vào gần 14)
- Áp suất vào: 110kPa .
- Mức bình đun: 50% (thể tích chất lỏng)
- Lưu lượng và thành phần sản phẩm ở đáy cột chưng cất: 152kg mol/h (2749kg/h , $2,75\text{m}^3/\text{h}$) với hàm lượng methanol $< 0,01\%$.
- Mức bình ngưng tụ: 20% (thể tích chất lỏng)
- Lưu lượng và thành phần sản phẩm chưng cất: 85kg mol/h (2746kg/h , $3,44\text{m}^3/\text{h}$) với hàm lượng methanol $> 99,9\%$.
- Trạng thái ban đầu của bình đun, bình ngưng tụ và các ngăn là khô.
- Lưu lượng dòng ngược: 247kg mol/h (7890kg/h , $9,90\text{m}^3/\text{h}$)
- Áp suất cột chưng cất: $101,3\text{kPa}$.
- Độ giảm áp suất cột: $11,62\text{kPa}$.

Điều khiển cột chưng cất có thể chia làm hai bài toán khác nhau: điều khiển trạng thái xác lập và điều khiển động. Điều khiển trạng thái xác lập (điều khiển ổn định hóa) nhằm loại trừ ảnh hưởng của nhiễu và thường thực hiện bằng các bộ điều khiển PID. Các bộ điều khiển này được chỉnh định để hoạt động xung quanh điểm làm việc. Khi điều kiện làm việc thay đổi thì cần thiết phải điều khiển

động. Trong trường hợp này, bộ điều khiển dự báo được sử dụng để tạo ra tín hiệu đặt cho bộ điều khiển PID. Điều khiển động cũng cần thiết khi cột chưng cất ở ngoài điều kiện làm việc bình thường, chẳng hạn như khi khởi động hoặc kết thúc, trong những tình huống này hệ thống ở xa điểm làm việc bình thường và có tính phi tuyến cao.



Hình 10.10 Sơ đồ khối cột chưng cất

Ví dụ này xét bài toán khởi động cột chưng cất, đây là một trong những bài toán điều khiển phức tạp nhất trong công nghệ hóa

học, liên quan đến điều khiển nhiệt độ và khối lượng trong phạm vi làm việc rộng. Hoạt động khởi động cột chưng cất có thể chia làm ba giai đoạn:

- Giai đoạn 1 (giai đoạn không liên tục): cột chưng cất được cất nhiệt, máy bơm khởi động; các biến lưu lượng và áp suất thay đổi nhanh, nhiệt độ tăng từ từ đến giá trị xác định trước.

- Giai đoạn 2 (giai đoạn bán liên tục): các biến nhiệt động học (áp suất và nhiệt độ) có tính phi tuyến mạnh, các biến thủy học đạt giá trị xác lập, nồng độ từ từ đạt đến mức mong muốn nhưng có xu hướng rất nhạy với sự thay đổi của các biến khác.

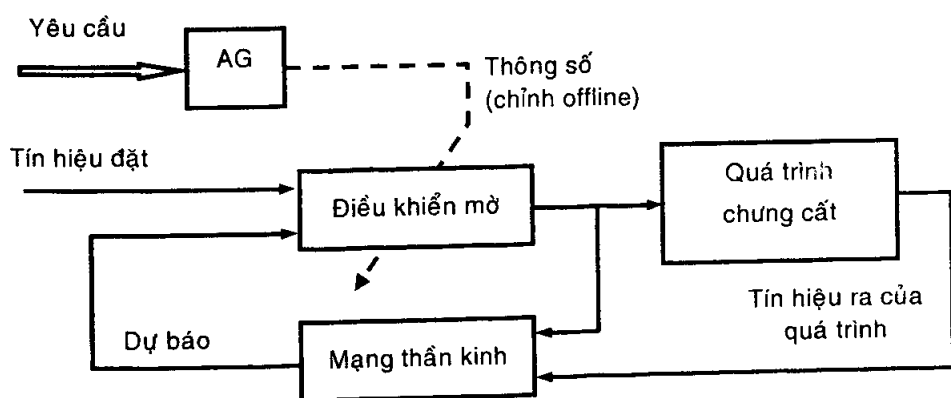
- Giai đoạn 3: kết thúc quá trình khởi động, cột chưng cất có thể xem như đã ở trạng thái xác lập với các nhiễu loạn. Vì vậy hệ thống điều khiển phải hoạt động để giữ cột chưng cất ở lân cận của trạng thái xác lập mong muốn.

Có thể điều khiển khởi động cột chưng cất bằng tay, tuy nhiên có nhiều yếu tố ảnh hưởng đến quá trình khởi động và không có chiến lược khởi động tổng quát nào để khởi động tất cả các loại cột chưng cất. Do đó để có thể khởi động được cột chưng cất bằng tay phải mất hàng giờ thực nghiệm và kết quả là tạo ra nhiều phế phẩm.

Thay vì điều khiển bằng tay, có thể thiết kế hệ thống điều khiển tự động. Các bộ điều khiển được sử dụng để kiểm soát hoạt động của cột chưng cất bao gồm (ký hiệu bởi các hình tròn ở H.10.10):

- FEED1: điều khiển bơm đưa chất lỏng vào cột chưng cất, lưu lượng tối đa 500kg mol/h .
- PRES: điều khiển áp suất trong cột bằng cách hóa lỏng hơi trong bình ngưng tụ.
- VAPOUR: điều khiển ON-OFF, dùng để đưa khí trơ ra khỏi cột chưng cất (hoạt động làm tinh khiết).
- REFLUX: điều khiển lưu lượng chất lỏng ngược từ bình ngưng tụ về cột chưng cất.
- TEMP: điều khiển nhiệt độ cột chưng cất bằng cách cấp nhiệt cho chất lỏng tại bình đun.
- BOTTOM: điều khiển van xả sản phẩm dư (nước) ở đáy cột chưng cất.
- TOP: điều khiển van đưa sản phẩm (methanol) ra khỏi cột chưng cất.

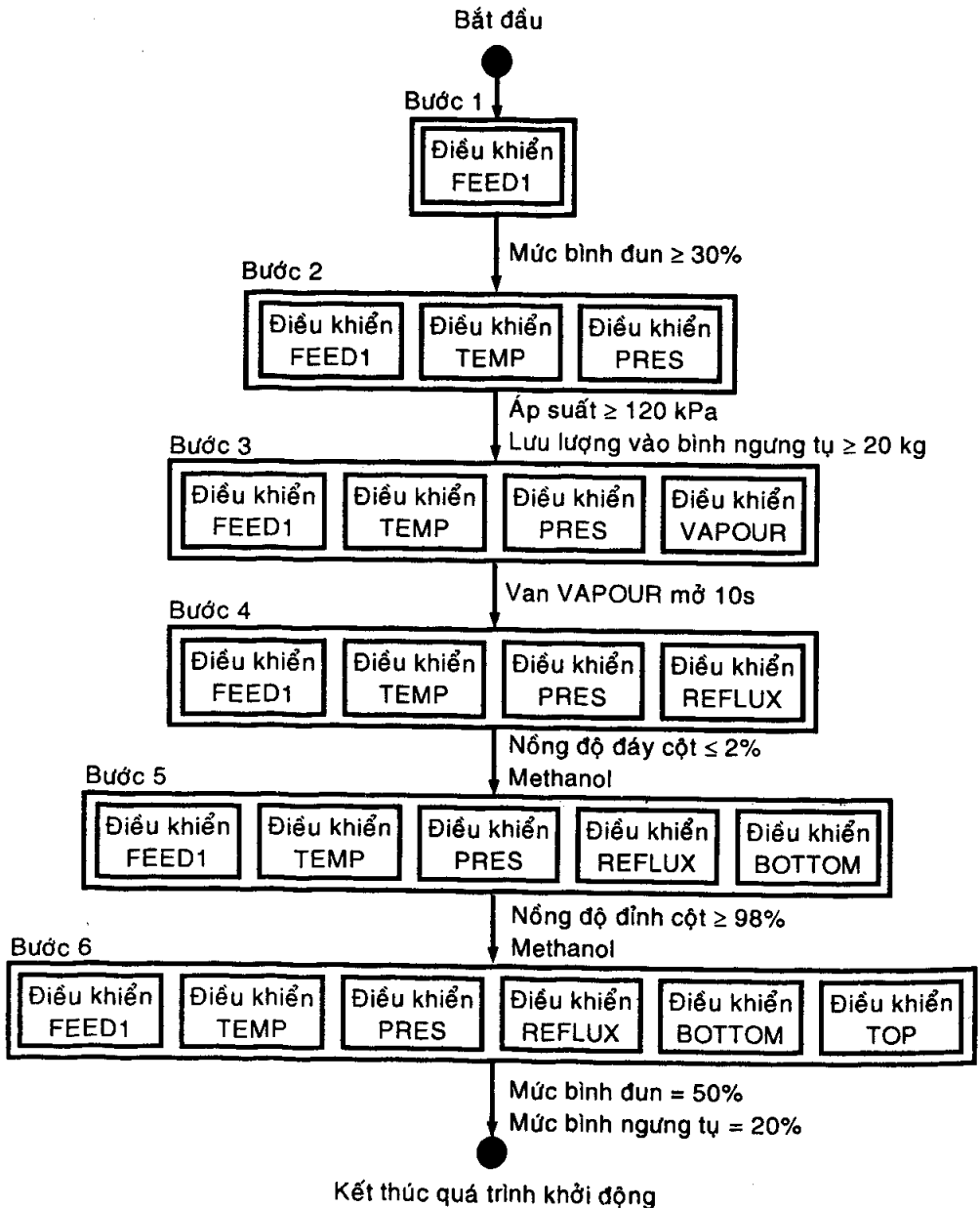
Trong ví dụ này, cột chưng cất được khởi động dùng các kỹ thuật điều khiển thông minh như mạng thần kinh, hệ mờ và giải thuật di truyền. Hệ thống sử dụng bộ điều khiển dự báo mờ để khởi động cột chưng cất. Mạng thần kinh mô hình hóa đặc tính động học của đối tượng được sử dụng để đưa các tín hiệu vào các bộ điều khiển mờ. Tương ứng với mỗi biến cần điều khiển có một mạng thần kinh và một bộ điều khiển mờ tạo thành một vòng điều khiển đơn. Chất lượng của toàn hệ thống điều khiển được tối ưu hóa dùng giải thuật di truyền bằng cách chỉnh định đồng thời tất cả các bộ điều khiển mờ. Hàm thích nghi liên quan đến mục tiêu và yêu cầu ở từng giai đoạn khởi động cột chưng cất ngay cả khi các mục tiêu này mâu thuẫn nhau. Một thủ tục thêm vào để chuyển đổi qua lại giữa bộ điều khiển mờ và mạng thần kinh. Sơ đồ khối một vòng điều khiển được minh họa ở hình 10.11 (hệ thống gồm nhiều vòng như vậy). Giải thuật di truyền được sử dụng để tối ưu hóa hệ thống off-line dựa trên các thí nghiệm mô phỏng.



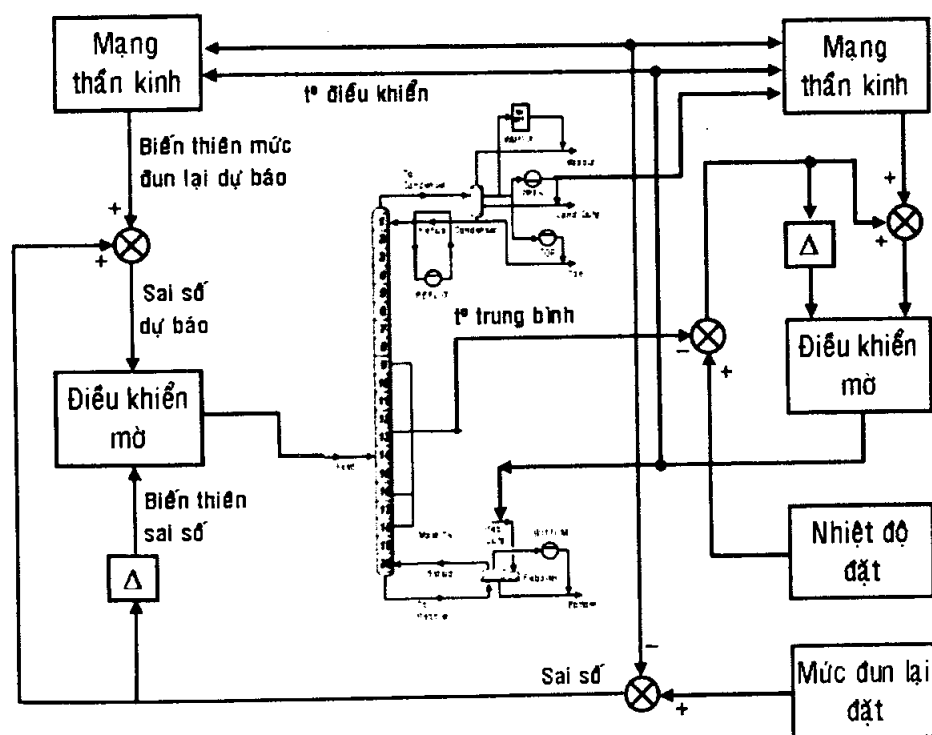
Hình 10.11 Sơ đồ khối hệ thống điều khiển thông minh cột chưng cất

Quá trình khởi động cột chưng cất có thể chia làm 6 bước. Hình 10.12 là trình tự các bước, mục tiêu cần đạt khi kết thúc mỗi bước và các bộ điều khiển liên quan. Hình 10.13 minh họa một phần hệ thống điều khiển thông minh thực hiện khởi động bước 2 và 3, trong hình 10.13 có hai vòng điều khiển đơn ở hình 10.11. Hình 10.14 là kết quả điều khiển nhiệt độ cột chưng cất trong quá trình khởi động dùng phương pháp điều khiển thông minh (hình bên trái) và điều khiển

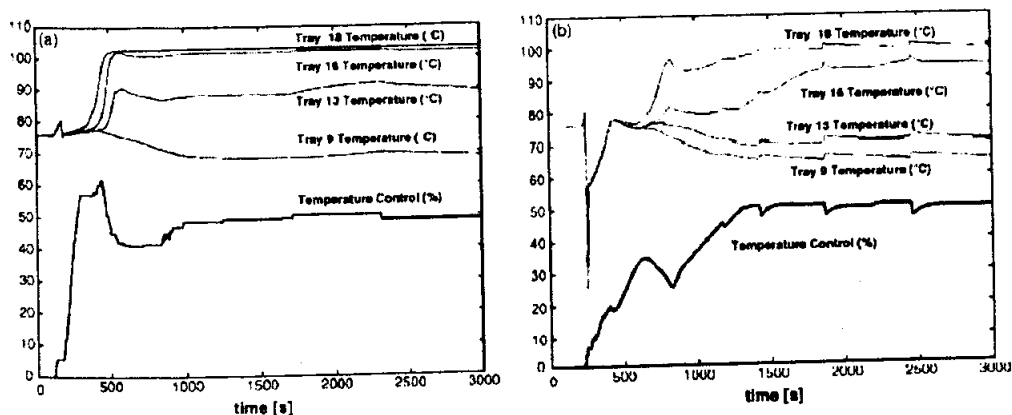
thông thường (hình bên phải). Đồ thị biến đổi nhiệt độ theo thời gian cho thấy điều khiển thông minh rút ngắn quá trình quá độ khởi động nhiệt và trạng thái nhiệt độ xác lập của hệ thống rất ổn định, không bị ảnh hưởng bởi nhiễu loạn.



Hình 10.12 Các bước khởi động cột chưng cất và các bộ điều khiển liên quan



Hình 10.13 Hệ thống điều khiển thông minh thực hiện khởi động bước 2 và 3

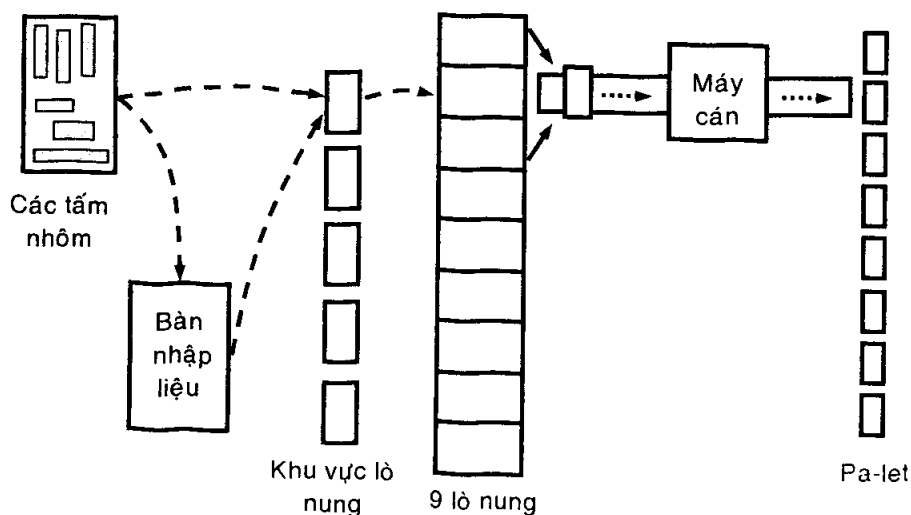


Hình 10.14 Kết quả điều khiển nhiệt độ cột chưng cất trong quá trình khởi động

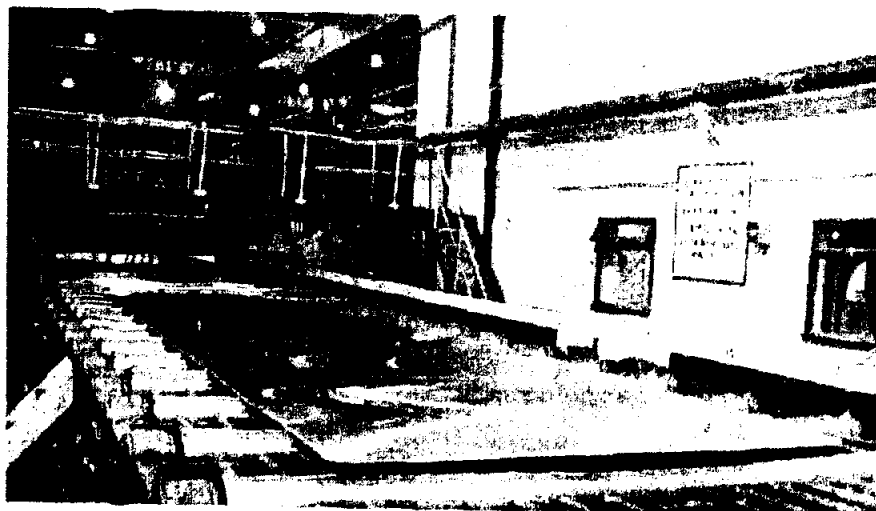
a) Điều khiển thông minh; b) Điều khiển thông thường

Ví dụ 10.3 Điều khiển giám sát thông minh hệ thống cán nhôm ([118])

Hệ thống khảo sát gồm một bàn nhập liệu, chín lò nung chảy và một máy cán nóng bố trí như hình 10.15. Cần cầu chuyển tấm nhôm từ bàn nhập liệu vào lò nung. Các tấm nhôm (kích thước tối đa rộng $1,5m$, dài $4,5m$, dày $525mm$) được nung để làm mềm trong các hầm nung và đưa lên nhiệt độ phù hợp để cán. Kích thước của tấm nhôm sau khi cán có độ dày tối thiểu là $5,7mm$, chiều rộng cực đại là $3,75m$, chiều dài cực đại là $32m$ (H.10.16).



Hình 10.15 Bố trí các lò nung và máy cán

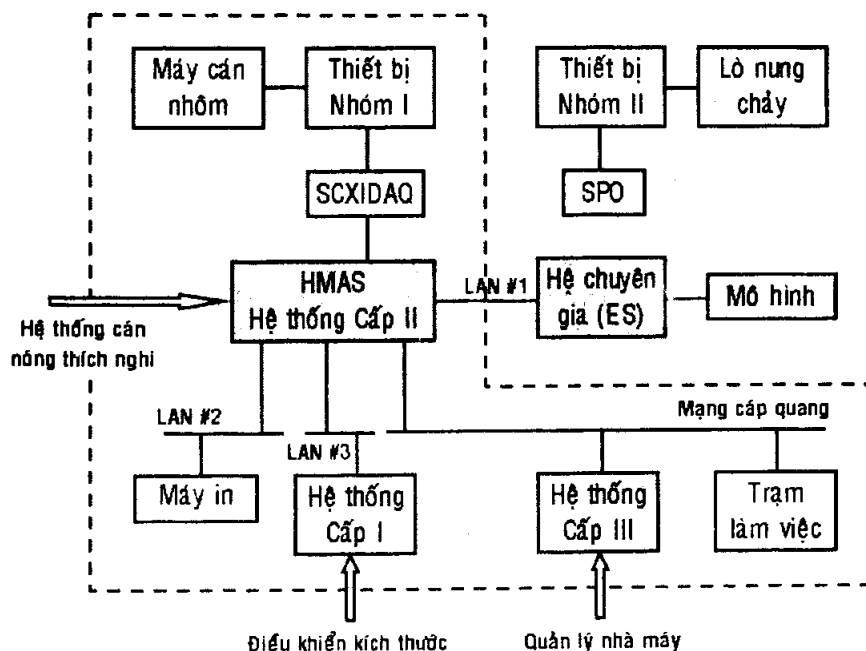


Hình 10.16 Tấm nhôm sau khi cán

Hệ thống điều khiển quá trình cán nhôm chia làm ba cấp:

- Cấp I điều khiển trực tiếp phần cứng với giá trị đặt đưa ra bởi cấp II.
- Cấp II điều khiển giám sát có nhiệm vụ đưa ra giá trị đặt cho các bộ điều khiển ở cấp I, sao cho các hệ thống con có thể hoạt động phối hợp với nhau để tạo ra những sản phẩm có chất lượng.
- Cấp III là cấp điều khiển cao nhất, có chức năng hoạch định và tối ưu hóa hoạt động của hệ thống.

Cấu trúc hệ thống điều khiển giám sát dựa vào tri thức thông minh



Hình 10.17 Hệ thống điều khiển giám sát thông minh quá trình cán nhôm

Hình 10.17 là sơ đồ khối cấu trúc tổng thể hệ thống điều khiển giám sát dựa vào tri thức thông minh (IKBSC–Intelligent Knowledge-Based Supervisory Control) quá trình nung và cán nhôm. Hệ thống gồm ba thành phần chính:

- Hệ chuyên gia (ES–Expert System) là hệ thống con của HMAS

- Hệ thống cán thích nghi (*HMAS–Hot Mill Adaptation System*): đây là hệ thống điều khiển cấp II.
- Tối ưu hóa quá trình nung chảy (*SPO–Soaking Pit Optimisation*) hoạch định hoạt động nung chảy và cán dùng các kỹ thuật thông minh.

10.7 TÓM TẮT

Chương 10 trình bày cấu trúc hệ thống điều khiển thông minh phân cấp và chức năng của các thành phần trong hệ thống. Khác với các chương trước chủ yếu đề cập đến kỹ thuật thông minh điều khiển từng đối tượng riêng lẻ, chương này quan tâm đến quá trình điều khiển hệ thống tổng thể gồm nhiều hệ thống con. Kỹ thuật điều khiển thông minh phân cấp đã áp dụng thành công để điều khiển nhiều quá trình công nghiệp, hệ thống điều khiển máy bay, tàu vũ trụ... Do các hệ thống tổng thể rất phức tạp nên trong khuôn khổ quyển sách này các ví dụ minh họa chỉ được trình bày sơ lược mang tính giới thiệu nhằm mục đích cho thấy rằng các kỹ thuật điều khiển thông minh đã trình bày ở các chương trước rất hữu ích khi thực thi hệ thống điều khiển thông minh thực tế. Chi tiết về cách thiết kế và thực thi các hệ thống điều khiển thông minh độc giả có thể tham khảo các tài liệu đã trích dẫn.

Phụ lục A: CHƯƠNG TRÌNH MATLAB

Phụ lục A gồm các chương trình thực thi một số giải thuật di truyền đã trình bày trong các ví dụ tối ưu hóa hàm (mục A.1), tìm đường đi ngắn nhất cho robot (mục A.2), chỉnh định thông số bộ điều khiển PID (mục A.3), và chỉnh định thông số bộ điều khiển PID mờ (mục A.4).

A.1 Giải thuật di truyền mã thập phân giải bài toán tối ưu hóa hàm

A.1.1 Chương trình chính

```
% GA_example.m
% Chương trình tìm cực tiểu hàm dùng GA
% Lập trình : Huỳnh Thái Hoàng, HCMUT.
% Xem ví dụ 4.3

clc;
clear all
rand('state',sum(100*clock));

% Điều kiện dừng: GA dừng khi đã chạy được 1000 thế hệ hoặc
% trong 50 thế hệ liên tiếp hàm thích nghi thay đổi không quá 0.00001
max_generation=1000;
max_stall_generation=50;
epsilon=0.00001;

N=20; % kích thước quần thể
d = 2; % số thông số của hàm cần tìm cực trị
range=[ 0 0;... % tầm giá trị tìm kiếm của các thông số
       30 30];
dec=[2 2]; % vị trí dấu chấm thập phân khi mã hóa
sig=[6 6]; % số chữ số có nghĩa khi mã hóa

Pc = 0.9; % xác suất lai ghép
Pm = 0.2; % xác suất đột biến
elitism = 1; % lưu giữ cá thể tốt nhất trong khi chạy GA

% Khởi động quần thể ban đầu, xem chương trình con ở mục A.1.2
par=lnit(N,d,range);
```

```

Terminal=0;           % cờ báo chưa kết thúc thuật toán
generation = 1;       % biến đếm số thế hệ
stall_generation=0;    % biến đếm số thế hệ hàm thích nghi không đổi

% Đánh giá độ thích nghi của quần thể ban đầu dựa vào hàm cần tìm cực tiểu
% Hàm cần tìm cực tiểu xem chương trình con ở mục A.1.8
for pop_index=1:N,
    fitness(pop_index)=1/(testfcn(par(pop_index,1),par(pop_index,2))+5);
end;
bestfit(generation),bestchrom]=max(fitness);
bestpar(generation,:)=par(bestchrom,:);

while ~Terminal,
    generation = generation+1

    % Mã hóa thập phân, xem chương trình con ở mục A.1.3
    pop=Encode_Decimal(par,sig,dec);

    % Chọn lọc tỉ lệ, xem chương trình con ở mục A.1.4
    parent=Select_Proportional(pop,fitness,elitism,bestchrom);

    % Lai ghép hai điểm, xem chương trình con ở mục A.1.5
    child=Cross_Twopoint(parent,Pc,elitism,bestchrom);

    % Đột biến đều, xem chương trình con ở mục A.1.6
    pop=Mutate_Uniform(child,Pm,elitism,bestchrom);

    % Giải mã thập phân, xem chương trình con ở mục A.1.7
    par=Decode_Decimal(pop,sig,dec);

    % Tính hàm thích nghi sau 1 thế hệ tiến hóa
    for pop_index=1:N,
        fitness(pop_index)=1/(testfcn(par(pop_index,1),par(pop_index,2))+5);
    end;
    bestfit(generation),bestchrom]=max(fitness);
    bestpar(generation,:)=par(bestchrom,:);

    % Xét điều kiện dừng
    if generation == max_generation
        Terminal = 1;
    elseif generation>1,

```

```

    if abs(bestfit(generation) - bestfit(generation-1)) < epsilon,
        stall_generation = stall_generation + 1;
        if stall_generation == max_stall_generation, Terminal = 1; end
    else
        stall_generation = 0;
    end;
end;
end;
par(bestchrom,:)      % Hiển thị điểm cực tiểu mà GA tìm được
plot(bestfit)         % Vẽ đồ thị hàm thích nghi tốt nhất
xlabel('Generation');
ylabel('Fitness function');

```

A.1.2 Chương trình con khởi động ngẫu nhiên thể hệ đầu tiên

% Init.m: chương trình con khởi động ngẫu nhiên thể hệ ban đầu
 % Lập trình: Huỳnh Thái Hoàng, HCMUT.

```

function par=Init(N,d,range)
% Biến vào:
% N      : kích thước quần thể
% d      : số thông số của hàm cần tìm cực trị
% range  : mảng hai chiều [2×d] lưu miền giá trị của các thông số
% Biến ra:
% par    : mảng hai chiều [N×d] lưu quần thể khởi tạo ngẫu nhiên
%        (chưa mã hóa)

for pop_index = 1:N
    for par_index = 1:d,
        par(pop_index,par_index)=...
            (rand-0.5)*(range(2,par_index)-range(1,par_index))+...
            0.5*(range(2,par_index)+range(1,par_index));
    end
end

```

A.1.3 Chương trình con mã hóa thập phân

% Encode_Decimal.m: chương trình con mã hóa thập phân
 % Lập trình: Huỳnh Thái Hoàng, HCMUT

```

function pop = Encode_Decimal(par,sig,dec)
% Biến vào:
% par      : mảng hai chiều [N×d] lưu các thông số cần mã hóa
%           N là kích thước quần thể, d là số thông số
% sig      : vector hàng [1×d] lưu số chữ số có nghĩa tương ứng với mỗi thông số
% dec      : vector hàng [1×d] lưu vị trí dấu chấm thập phân
% Biến ra:
% pop      : mảng hai chiều [N×L] lưu quần thể sau khi mã hóa thập phân
%           mỗi hàng của pop là một cá thể (L là chiều dài chuỗi NST)

if (nargin < 3),
    error(['Thiếu đối số. Cú pháp: pop=Encode_Decimal(par,sig,dec)']);
end;

if size(sig)~=size(dec),
    error(['Đối số sig và dec không phù hợp nhau']);
end;

[N,d]=size(par);          % xác định kích thước quần thể

for pop_index = 1:N,      % quét từ đầu đến cuối quần thể
    gene_index = 1;
    for par_index = 1:d,   % quét các thông số

        % Gene đầu tiên mã hóa dấu của thông số
        if par(pop_index,par_index) < 0,
            pop(pop_index,gene_index)=0;
        else
            pop(pop_index,gene_index)=9;
        end
        gene_index=gene_index+1;

        % Các gen tiếp theo là các chữ số có nghĩa
        temp(par_index)=abs(par(pop_index,par_index))/10^dec(par_index);
        for count = 1:sig(par_index),
            temp(par_index)=temp(par_index)*10;
            pop(pop_index,gene_index)=temp(par_index)-...
                rem(temp(par_index),1);
            temp(par_index)=temp(par_index)-...
                pop(pop_index,gene_index);
        end
    end
end

```

```

        gene_index=gene_index+1;
    end
end
end

```

A.1.4 Chương trình con chọn lọc tỉ lệ

% Select_Proportional.m: chương trình con chọn lọc tỉ lệ

% Lập trình : Huỳnh Thái Hoàng, HCMUT.

```
function parent=Select_Proportional(pop,fitness,elitism,bestchrom)
```

% Giải thuật chọn lọc tỉ lệ: xem giải thuật 4.1

% Biến vào:

% pop : mảng hai chiều [N×L], quần thể trước khi chọn lọc

% N là kích thước quần thể, L là chiều dài chuỗi NST

% fitness : vector cột [N×1], độ thích nghi của các cá thể trong quần thể

% elitism : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa

% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa

% Biến ra:

% parent : mảng hai chiều [N×L], quần thể sau khi chọn lọc

```
if (nargin < 4),
```

```
    error(['Không đủ đối số']);
```

```
end;
```

```
N=length(fitness);          % N là kích thước quần thể
```

```
s=zeros(N+1,1);
```

```
for k=1:N,
```

```
    s(k+1)=s(k)+fitness(k);
```

```
end;
```

```
for k = 1:N,
```

```
    if (elitism==1) && (k==bestchrom),
```

```
        parent(k,:)=pop(k,:);
```

```
    else
```

```
        r=rand*s(N+1);
```

```
        index=find(s<r); j=index(end);
```

```
        parent(k,:)=pop(j,:);
```

```
    end
```

```
end
```

A.1.5 Chương trình con lai ghép hai điểm

% Cross_TwoPoint.m: chương trình con lai ghép hai điểm

% Lập trình: Huỳnh Thái Hoàng, HCMUT

```
function child=Cross_TwoPoint(parent,Pc,elitism,bestchrom)
% Biến vào:
% parent      : mảng hai chiều [N×L], quần thể trước khi lai ghép
%              : N là kích thước quần thể, L là chiều dài chuỗi NST
% Pc          : xác suất lai ghép
% elitism      : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa
% bestchrom    : biến lưu cá thể tốt nhất trong quá trình tiến hóa
% Biến ra:
% child       : mảng hai chiều [N×L], quần thể sau khi lai ghép

if (nargin < 4),
    error(['Không đủ đối số']);
end;
[N,L]=size(parent);           % N: kích thước quần thể
                                % L: chiều dài chuỗi NST
for p1 = 1:N,                 % Cá thể cha mẹ 1
    % Nếu cờ elitism=1 và p1 là cá thể ưu việt
    % thì không lai ghép (để bảo toàn gen tốt nhất)
    if (elitism==1) & (p1 == bestchrom)
        child(p1,:)=parent(p1,:);
    else
        if Pc > rand           % lai ghép xảy ra với xác suất Pc
            p2=p1;             % Chọn ngẫu nhiên cá thể cha mẹ 2
            while p2 == p1,    % khác cá thể cha mẹ 1
                p2 = rand*N;
                p2 = p2 - rem(p2,1)+1;
            end

            k1 = rand*(L-1);    % chọn ngẫu nhiên điểm lai ghép 1
            k1 = k1-rem(k1,1)+1;
            k2=k1;
            while k2==k1,      % chọn ngẫu nhiên điểm lai ghép 2
                k2 = rand*(L-1); % khác điểm lai ghép 1
                k2 = k2-rem(k2,1)+1;
            end;
        end;
    end;
end;
```

```
% nếu k2<k1 thì chuyển đổi k1 và k2 để đảm bảo rằng k1<k2
if k1>k2, t=k2; k2=k1; k1=t; end;
```

```
% cá thể con thừa hưởng gen của cha mẹ
child(p1,1:k1)=parent(p1,1:k1);
child(p1,k1+1:k2)=parent(p2,k1+1:k2);
child(p1,k2+1:L)=parent(p1,k2+1:L);
```

```
else % lai ghép không xảy ra
    child(p1,:)=parent(p1,:);
```

```
end
```

```
end
```

```
end
```

A.1.6 Chương trình con đột biến đều

% Mutate_Uniform.m: chương trình con đột biến đều

% Lập trình: Huỳnh Thái Hoàng

```
function newpop=Mutate_Uniform(pop,Pm,elitism,bestchrom)
```

```
% Biến vào:
```

```
% pop : mảng hai chiều [N×L], quần thể trước khi đột biến
```

```
% N là kích thước quần thể, L là chiều dài chuỗi NST
```

```
% Pm : xác suất đột biến
```

```
% elitism : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa
```

```
% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa
```

```
% Biến ra:
```

```
% newpop : mảng hai chiều [N×L], quần thể sau khi đột biến
```

```
if (nargin < 4),
```

```
    error(['Thiếu đối số.']);
```

```
end;
```

```
[N,L]=size(pop); % N: kích thước quần thể
```

```
% L: chiều dài chuỗi NST
```

```
newpop=pop;
```

```
for pop_index= 1:N,
```

```
    if (elitism==0) || (elitism==1 && pop_index~=bestchrom),
```

```
        for gene_index = 1:L,
```

```
            if Pm > rand, % Đột biến xảy ra với xác suất Pm
```

```
                % Phát sinh 1 gen ngẫu nhiên khác gen đang xét
```

```

        rand_gene=rand*10;
        while(pop(pop_index, gene_index)==...
            rand_gene-rem(rand_gene,1) || rand_gene==10),
            rand_gene=rand*10;
        end;

        newpop(pop_index, gene_index)=...
            rand_gene-rem(rand_gene,1);
    end
end
end
end

```

A.1.7 Chương trình con giải mã thập phân

% Decode_Decimal.m: chương trình con giải mã thập phân

% Lập trình: Huỳnh Thái Hoàng

```

function par=Decode_Decimal(pop,sig,dec)
% Biến vào:
% pop      : mảng hai chiều [N×L] lưu quần thể dưới dạng các chuỗi NST
%           : mỗi hàng của pop là một cá thể (L là chiều dài chuỗi NST)
% sig       : vector hàng [1×d] lưu số chữ số có nghĩa tương ứng với mỗi thông số
% dec       : vector hàng [1×d] lưu vị trí dấu chấm thập phân
%           : d là số thông số
% Biến ra:
% par       : mảng hai chiều [N×d] lưu quần thể sau khi giải mã
%           : mỗi hàng của par gồm d thông số

if (nargin < 3),
    error(['Thiếu đối số. Cú pháp: par=Decode_Decimal(pop,sig,dec)']);
end;

if size(sig)~=size(dec),
    error(['Đối số sig và dec không phù hợp nhau']);
end;

[N,L]=size(pop);                                % N: kích thước quần thể
                                                % L: chiều dài chuỗi NST
d=length(sig);                                   % d: số thông số

```



```

if L~=sum(sig)+d,
    error('Chiều dài chuỗi NST không phù hợp với sig');
end

par=zeros(N,d); % Khởi tạo mảng chứa thông số giải mã

for pop_index = 1:N, % Quét tất cả các cá thể
    start_gene = 1;
    for par_index = 1:d, % Giải mã từng thông số
        % Mỗi thông số bằng tổng các gen nhân với trọng số tương ứng
        for count = 1:sig(par_index),
            gene_index = start_gene+count;
            weight = dec(par_index)-count; % Tính trọng số của gen
            par(pop_index,par_index) = par(pop_index,par_index)+...
                (pop(pop_index,gene_index))*10^weight;
        end
        % Xác định dấu của thông số
        if pop(pop_index,start_gene) < 5,
            par(pop_index,par_index) = -par(pop_index,par_index);
        end

        % gen đầu tiên của thông số kế tiếp
        start_gene=start_gene+1+sig(par_index);
    end
end
end

```

A.1.8 Chương trình con tính hàm cần tìm cực tiểu

```

function y = testfcn(x1,x2);
y = +5*exp(-0.1.*((x1-15).^2+(x2-20).^2))...
    -2*exp(-0.08.*((x1-20).^2+(x2-15).^2))...
    +3*exp(-0.08.*((x1-25).^2+(x2-10).^2))...
    +2*exp(-0.1.*((x1-10).^2+(x2-10).^2))...
    -2*exp(-0.5.*((x1-5).^2+(x2-10).^2))...
    -4*exp(-0.1.*((x1-15).^2+(x2-5).^2))...
    -2*exp(-0.5.*((x1-8).^2+(x2-25).^2))...
    -3*exp(-0.5.*((x1-21).^2+(x2-25).^2))...
    +2*exp(-0.5.*((x1-25).^2+(x2-16).^2));

```

A.2 Giải thuật di truyền tìm đường đi ngắn nhất

A.2.1 Chương trình chính

% ga_RobotPath.m: chương trình GA giải bài toán tìm đường đi ngắn nhất

% của robot qua tất cả các trạm làm việc và trở về điểm xuất phát

% Lập trình: Huỳnh Thái Hoàng, HCMUT

% Xem ví dụ 4.4

clc;

clear all;

rand('state',sum(100*clock));

% Điều kiện dừng: GA dừng khi đã chạy được 2000 thế hệ hoặc

% trong 500 thế hệ liên tiếp hàm thích nghi thay đổi không quá 10^{-8}

max_generation=2000;

max_stall_generation=500;

epsilon=0.00000001;

dispON=1; % hiển thị đường đi tốt nhất tìm được qua từng thế hệ

N=20; % kích thước quần thể

Pc = 0.9; % xác suất lai ghép

Pm = 0.2; % xác suất đột biến

elitism = 1; % lưu giữ cá thể tốt nhất trong quá trình tiến hóa

load Position; % đọc tọa độ các trạm làm việc đã lưu sẵn trong file

% vector cột X lưu các tọa độ theo trục X

% vector cột Y lưu các tọa độ theo trục Y

nPos = length(X); % số trạm làm việc chính bằng số hàng của X (hoặc Y)

% Khởi động ngẫu nhiên quần thể đầu tiên, xem chương trình con ở mục A.2.2

pop=Init_RobotPath(N,nPos);

Terminal=0; % cờ báo chưa kết thúc thuật toán

generation = 1; % biến đếm số thế hệ

stall_generation=0; % biến đếm số thế hệ hàm thích nghi không đổi

% Tính hàm thích nghi của quần thể ban đầu theo công thức (4.42)

% Hàm TravelDistance.m tính quãng đường di chuyển ,

% xem chương trình con ở mục A.2.3

for pop_index=1:N,

```

    fitness(pop_index)=1/TravelDistance(pop(pop_index,:),X,Y);
end;
[bestfit(generation),bestchrom]=max(fitness);

while ~Terminal,
    generation = generation+1;

    % Chọn lọc sắp hạng tuyến tính, xem chương trình con ở mục A.2.4
    parent = Select_Linear_Ranking(pop,fitness,0.5,elitism,bestchrom);

    % Lai ghép 1 điểm cải biên phù hợp với bài toán tìm đường đi,
    % xem chương trình con ở mục A.2.5
    child = Cross_Onepoint_RobotPath(parent,Pc,elitism,bestchrom);

    % Đột biến cho bài toán tìm đường đi,
    % xem chương trình con ở mục A.2.6
    pop = Mutate_RobotPath(child,Pm,elitism,bestchrom);

    % Tính độ thích nghi của các cá thể sau 1 thế hệ tiến hóa
    for pop_index=1:N,
        fitness(pop_index)=1/TravelDistance(pop(pop_index,:),X,Y);
    end;
    [bestfit(generation),bestchrom]=max(fitness);

    if (dispON==1)      % Vẽ đường đi ngắn nhất tìm được ở mỗi thế hệ
        travel_distance=1/bestfit(generation);
        figure(1);
        plot(X,Y,'ro')
        hold on;
        for k=1:nPos,
            text(X(k)+3,Y(k),num2str(k));
        end;
        path=[pop(bestchrom,:) pop(bestchrom,1)];
        plot(X(path),Y(path))
        title(['Generation: ' num2str(generation) ...
            ', Stall generation: ' num2str(stall_generation) ...
            ', Travel distance: ' num2str(travel_distance)]);
        axis equal
        hold off;
        drawnow;
    end

```

```

% Kiểm tra điều kiện dừng
if generation == max_generation
    Terminal = 1;
elseif generation > 1,
    if abs(bestfit(generation) - bestfit(generation - 1)) < epsilon,
        stall_generation = stall_generation + 1;
        if stall_generation == max_stall_generation, Terminal = 1; end
    else
        stall_generation = 0;
    end;
end;
end; %While

path=[pop(bestchrom,:) pop(bestchrom,1)];    % Đường đi ngắn nhất tìm được
travel_distance = 1/bestfit(generation);      % Quãng đường di chuyển

% Vẽ đường đi ngắn nhất
figure(1);
plot(X,Y,'ro')
hold on;
for k=1:nPos,
    text(X(k)+3,Y(k),num2str(k));
end;
plot(X(path),Y(path))
title(['Travel distance: ' num2str(travel_distance)])
axis equal
hold off;

% Vẽ đồ thị hàm thích nghi
figure(2);
plot(bestfit)

```

A.2.2 Chương trình con khởi động quần thể ban đầu cho bài toán tìm đường đi

% Init_RobotPath.m: chương trình khởi động ngẫu nhiên thể hệ đầu tiên
 % cho bài toán tìm đường đi ngắn nhất của robot
 % Lập trình: Huỳnh Thái Hoàng

```
function pop = Init_RobotPath(N,nPos)
```

```
% Biến vào:
% N      : kích thước quần thể
% nPos   : số trạm làm việc (số vị trí mà robot phải đi qua)
% Biến ra:
% pop    : mảng hai chiều [N×nPos], quần thể ngẫu nhiên
```

```
for pop_index = 1:N
    [temp,order]=sort(rand(1,nPos));
    pop(pop_index,:)=order;
end
```

A.2.3 Chương trình con tính chiều dài quãng đường đi

```
% Travel_Distance.m: tính chiều dài quãng đường đi theo công thức (4.41)
% Lập trình: Huỳnh Thái Hoàng
```

```
function d = TravelDistance(TravelRoute,X,Y)
% Biến vào:
% TravelRoute : đường đi, mảng 1 chiều lưu thứ tự các trạm làm việc
% X, Y        : tọa độ của các trạm làm việc
% Biến ra:
% d           : quãng đường, tính theo công thức (4.41)
```

```
NPos = length(TravelRoute);    % NPos: số trạm làm việc
if (NPos ~= length(X)) || (NPos ~= length(Y)),
    error('Chiều dài mảng Travel, X, Y phải bằng nhau!');
end;
```

```
d = 0;
for i = 2:NPos,
    d = d + sqrt((X(TravelRoute(i))-X(TravelRoute(i-1)))^2+...
                (Y(TravelRoute(i)) -Y(TravelRoute(i-1)))^2);
end;
```

```
d = d + sqrt((X(TravelRoute(NPos))-X(TravelRoute(1)))^2+...
                (Y(TravelRoute(NPos))-Y(TravelRoute(1)))^2);
```

A.2.4 Chương trình con chọn lọc sắp hạng tuyến tính

```
% Select_Linear_Ranking.m: chương trình con chọn lọc sắp hạng tuyến tính
% Lập trình: Huỳnh Thái Hoàng
```

```

function parent = Select_Linear_Ranking(pop,fitness,eta,elitism,bestchrom)
% Chọn lọc tuyến tính, xem giải thuật 4.4
% Biến vào:
% pop      : mảng hai chiều [N×L], quần thể trước khi chọn lọc
%          N là kích thước quần thể, L là chiều dài chuỗi NST
% fitness  : vector cột [N×1], độ thích nghi của các cá thể trong quần thể
% eta      : tham số của phép chọn lọc sắp hạng tuyến tính
% elitism   : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa
% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa
% Biến ra:
% parent    : mảng hai chiều [N×L], quần thể sau khi chọn lọc

if (nargin < 5), error('Thiếu đối số'); end;

N = length(fitness);           % kích thước quần thể
[fitness,order] = sort(fitness); % sắp xếp NST tăng dần độ thích nghi

% Tính xác suất chọn lọc của các NST sau khi sắp xếp theo công thức (4.12)
for k = 1:N,
    p(k) = (eta+(2-eta)*(k-1)/(N-1))/N;
end

s = zeros(1,N+1);
for k = 1:N,
    s(k+1) = s(k)+p(k);
end;

for k = 1:N,
    if elitism == 1 & order(k) == bestchrom,
        % Nếu elitism = 1 và NST đang xét là NST tốt nhất thì NST đó được chọn
        parent(order(k),:) = pop(order(k),:);
    else
        % Nếu elitism = 0 hoặc NST đang xét không phải là NST tốt nhất
        % thì NST đó sẽ được chọn với xác suất tỉ lệ với hạng của nó
        r = rand*s(N+1);
        index = find(s<r); j = index(end);
        parent(order(k),:) = pop(order(j),:);
    end
end
end

```

A.2.5 Chương trình con lai ghép một điểm cho bài toán tìm đường đi

% Cross_Onepoint_RobotPath.m: chương trình con lai ghép 1 điểm

% cho bài toán tìm đường đi ngắn nhất của robot

% Lập trình: Huỳnh Thái Hoàng

```
function child = Cross_Onepoint_RobotPath (parent,Pc,elitism,bestchrom)
```

% Giải thuật: xem ví dụ 4.4

% Biến vào:

% parent : mảng hai chiều [N×L], quần thể trước khi lai ghép

% N là kích thước quần thể, L là chiều dài chuỗi NST

% Pc : xác suất lai ghép

% elitism : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa

% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa

% Biến ra:

% child : mảng hai chiều [N×L], quần thể sau khi lai ghép

```
if (nargin < 4), error('Thiếu đối số'); end;
```

```
child = parent;
```

```
[N,L] = size(parent);
```

% N: kích thước quần thể

% L: chiều dài chuỗi NST

```
for p1 = 1:N, % cha mẹ 1
```

% Chỉ lai ghép khi elitism = 0 hoặc p1 không phải là NST tốt nhất

```
if (elitism == 0) || (p1 ~= bestchrom),
```

```
    p2 = p1;
```

```
    while p2 == p1,
```

```
        p2 = rand*N;
```

```
        p2 = p2 - rem(p2,1) + 1; % p2 ngẫu nhiên, khác p1
```

```
    end
```

```
    if rand < Pc
```

% Lai ghép xảy ra với xác suất Pc

```
        k = rand*(L-1);
```

% Chọn ngẫu nhiên điểm lai ghép

```
        k = k-rem(k,1) + 1;
```

% Điểm lai ghép là số nguyên

```
        child(p1, k+1:L) = parent(p2,k+1:L);
```

```
% Tìm các gen trùng nhau trên con 1
```

```
index1 = [];
```

```
for j=k+1:L,
```

```
    if sum(find(parent(p1,1:k)==parent(p2,j)))>0
```

```

        index1 = [index1 j];
    end
end;

% Tìm các gen trùng nhau trên con 2
index2 = [];
for j = k+1:L,
    if sum(find(parent(p2,1:k) == parent(p1,j))) > 0
        index2 = [index2 j];
    end
end;

% Đổi chỗ các gen trùng nhau
for j = 1:length(index1),
    child(p1,index1(j)) = parent(p1,index2(j));
end
end
end % End the "if ELITISM..." statement
end % End "for p1=..." loop

```

A.2.6 Chương trình con đột biến cho bài toán tìm đường đi

% Mutate_RobotPath.m: chương trình con đột biến cho bài toán
 % tìm đường đi ngắn nhất cho robot
 % Lập trình: Huỳnh Thái Hoàng

```

function newpop = Mutate_RobotPath(pop,Pm,elitism,bestchrom)
% Giải thuật: xem thí dụ 4.4
% Biến vào:
% pop      : mảng hai chiều [N×L], quần thể trước khi đột biến
%          : N là kích thước quần thể, L là chiều dài chuỗi NST
% Pm       : xác suất đột biến
% elitism   : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa
% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa
% Biến ra:
% newpop    : mảng hai chiều [N×L], quần thể sau khi đột biến

if (nargin < 4), error('Thiếu đối số'); end;

newpop = pop;
[N,L] = size(pop);
% N: kích thước quần thể
% L: chiều dài chuỗi NST

```



```

for pop_index = 1:N,
    % Đột biến chỉ có thể xảy ra khi elitism=0
    % hoặc NST đang xét không phải là NST tốt nhất
    if (elitism == 0) || (pop_index~=bestchrom),
        k1 = floor(1+(L-1)*rand);
        k2 = floor(1+(L-1)*rand);

        if rand < Pm,                % Xác suất đột biến là Pm
            t = newpop(pop_index,k1);
            newpop(pop_index,k1) = newpop(pop_index,k2);
            newpop(pop_index,k2) = t;
        end
    end % if (elitism ==...)
end % for pop_index ...

```

A.3 Giải thuật di truyền chỉnh định bộ điều khiển PID

Chương trình chính

```

% ga_PID_Motor: GA chỉnh định bộ điều khiển PID điều khiển động cơ.
% GA sử dụng phương pháp mã hóa thập phân
% Lập trình: Huỳnh Thái Hoàng, HCMUT
% Xem ví dụ 5.9

```

```

clc;
clear all
rand('state',sum(100*clock));

```

```

% Điều kiện dừng: GA dừng khi đã chạy được 200 thế hệ hoặc.
% trong 50 thế hệ liên tiếp hàm thích nghi thay đổi không quá  $10^{-4}$ 
max_generation = 200;
max_stall_generation = 50;
epsilon = 0.0001;

```

```

N = 20;                % Kích thước quần thể là 20
npar = 3;              % Có 3 thông số cần chỉnh định Kp, Ki, Kd
range=[ 0   0   0;...  % Tầm khởi động giá trị thông số ban đầu
       99 99 99];
dec = [2 2 2];         % Mỗi thông số được mã hóa thành đoạn gien
sig = [5 5 5];         % có 5 chữ số có nghĩa, vị trí dấu chấm thập phân là 2

```

```

Pc = 0.9; % Xác suất lai ghép là 0.9
Pm = 0.1; % Xác suất đột biến là 0.1
elitism = 1; % Lưu giữ cá thể tốt nhất trong quá trình tiến hóa

rho = 0.02; % Trọng số của hàm thích nghi

% Khởi động ngẫu nhiên thế hệ đầu tiên, xem chương trình con ở mục A.1.2
par = Init(N,npar,range);

Terminal = 0;
generation = 0;
stall_generation = 0;

% Tính độ thích nghi của quần thể ban đầu
for pop_index = 1:N,
    Kp = par(pop_index,1);
    Ki = par(pop_index,2);
    Kd = par(pop_index,3);
    sim('PID_Motor'); % Mô phỏng hệ thống ở hình 5.52
    J = (e*e) + rho*(u*u);
    fitness(pop_index) = 1/(J+eps); % GA tìm cực đại hàm thích nghi
end;
[bestfit0,bestchrom] = max(fitness);

while ~Terminal, % Chạy đến khi điều kiện dừng thỏa mãn
    generation = generation+1;

    % Mã hóa thập phân, xem chương trình con ở mục A.1.3
    pop = Encode_Decimal (par,sig,dec);

    % Chọn lọc sắp hạng tuyến tính, xem chương trình con ở mục A.2.4
    parent = Select_Linear_Ranking(pop,fitness,0.5,elitism,bestchrom);

    % Lai ghép hai điểm, xem chương trình con ở mục A.1.5
    child = Cross_Twopoint(parent,Pc,elitism,bestchrom);

    % Đột biến đều, xem chương trình con ở mục A.1.6
    pop=Mutate_Uniform(child,Pm,elitism,bestchrom);

    % Giải mã thập phân, xem chương trình con ở mục A.1.7
    par = Decode_Decimal(pop,sig,dec);

```

% Đánh giá lại độ thích nghi của quần thể sau mỗi thế hệ tiến hóa

```
for pop_index = 1:N,
    Kp = par(pop_index,1);
    Ki = par(pop_index,2);
    Kd = par(pop_index,3);
    sim('PID_Motor');
    J = (e'*e) + rho*(u'*u);
    fitness(pop_index) = 1/(J+eps);
end;
[bestfit(generation),bestchrom] = max(fitness);
```

% Kiểm tra điều kiện dừng

```
if generation == max_generation
    Terminal = 1;
elseif generation > 1,
    if abs(bestfit(generation) - bestfit(generation-1)) < epsilon,
        stall_generation = stall_generation+1;
        if stall_generation == max_stall_generation, Terminal = 1;end
    else
        stall_generation = 0;
    end;
end;
```

end; %While

```
plot(1./bestfit) % Vẽ đồ thị hàm thích nghi
```

```
Kp = par(bestchrom,1) % Hiển thị thông số sau khi chỉnh định
```

```
Ki = par(bestchrom,2)
```

```
Kd = par(bestchrom,3)
```

```
sim('PID_Motor'); % Mô phỏng kết quả điều khiển sau khi chỉnh định
```

A.4 Giải thuật di truyền chỉnh định bộ điều khiển PI mờ

A.4.1 Chương trình chính

% ga_FuzzyPI_CascadeTank: GA chỉnh định bộ điều khiển PI mờ

% điều khiển mực chất lỏng trong bốn chứa nối tiếp

% GA sử dụng phương pháp mã hóa số thực

% Lập trình: Huỳnh Thái Hoàng, HCMUT

% Xem ví dụ 5.11

```
clc;
clear all
rand('state',sum(100*clock));
Ts = 0.5;
```

```
% Điều kiện dừng: GA dừng khi đã chạy được 1000 thế hệ hoặc
% trong 100 thế hệ liên tiếp hàm thích nghi thay đổi không quá  $10^{-4}$ 
max_generation = 1000;
max_stall_generation = 100;
epsilon = 0.0001;
```

```
N = 30;
npar = 11;
range=[ 0 0 0 0 0 0 0 0 0 0 0;...
        -1 1 -1 1 -1 -1 1 1 0.05 1 1];
```

```
Pc = 0.9;      % Xác suất lai ghép 0.9
Pm = 0.1;      % Xác suất đột biến 0.1
elitism = 1;    % Lưu giữ phần tử tối ưu
```

```
rho = 100000;  % trọng số hàm thích nghi (5.50)
disp('GA is running...');
```

```
Terminal = 0;
generation = 0;
stall_generation = 0;
```

```
% Đọc cấu trúc hệ mờ cần chỉnh định
FuzzyPI = readfis('FuzzyPI_CascadeTank.fis');
```

```
% Khởi động ngẫu nhiên thế hệ đầu tiên, xem chương trình con ở mục A.1.2
pop = Init(N,npar,range);
```

```
for pop_index = 1:N,
    % Giải mã các NST thành thông số bộ điều khiển PI mờ
    % Xem chương trình con ở mục A.4.2
    Decode_FuzzyPI;
```

```
% Mô phỏng bộ điều khiển PI mờ vừa giải mã
% Sơ đồ Simulink ở hình 5.64
```

```

sim('FuzzyPI_CascadeTank');

% Tính giá trị hàm thích nghi của mỗi NST ban đầu
Index1=[1:150 301:450 601:750 910:1050];
Index2=[151:300 451:600 751:900 1051:1200];
J = Ts*(e(Index1)*e(Index1) + rho*e(Index2)*e(Index2);
fitness(pop_index) = 1/(J+eps);
end;
[bestfit0,bestchrom] = max(fitness);

while ~Terminal,
    generation = generation+1;
    disp(['generation #' num2str(generation) ' ...
        of maximum ' num2str(max_generation)]);

    % Chọn lọc sắp hạng tuyến tính, xem chương trình con ở mục A.2.4
    parent = Select_Linear_Ranking(pop,fitness,0.4,elitism,bestchrom);

    % Lai ghép BLX, xem chương trình con ở mục A.4.3
    child = Cross_BLX(parent,Pc,0.5,elitism,bestchrom);

    % Đột biến ngẫu nhiên, xem chương trình con ở mục A.4.4
    pop = Mutate_Random(child,Pm,range,elitism,bestchrom);

    % Tính lại giá trị hàm thích nghi của quần thể sau khi tiến hóa 1 thế hệ
    for pop_index = 1:pop_size,
        Decode_FuzzyPI;
        sim('FuzzyPI_CascadeTank');
        Index1 = [1:150 301:450 601:750 910:1050];
        Index2 = [151:300 451:600 751:900 1051:1200];
        J = Ts*(e(Index1)*e(Index1) + rho*e(Index2)*e(Index2)+...
            u(Index2)*u(Index2));
        fitness(pop_index) = 1/(J+eps);
    end;
    [bestfit(generation),bestchrom] = max(fitness);

    % Kiểm tra điều kiện dừng
    if generation == max_generation
        Terminal = 1;
    elseif generation>1,

```

```

    if abs(bestfit(generation)-bestfit(generation-1))<epsilon,
        stall_generation=stall_generation+1;
        if stall_generation == max_stall_generation, Terminal = 1;end
    else
        stall_generation=0;
    end;
end;
end; %While

% Vẽ đồ thị hàm thích nghi
plot(1./bestfit)
pop_index=bestchrom;

% Hiển thị thông số của các tập mờ và các hệ số chuẩn hóa sau khi chỉnh định
Decode_FuzzyPI;
P = [P1 P2 P3 P4 P5 P6 P7 P8]
K = [K1 K2 K3]

J = 1/bestfit(end)
% Mô phỏng kết quả điều khiển sau khi chỉnh định
sim('FuzzyPI_CascadeTank');

```

A.4.2 Chương trình con giải mã NST thành thông số bộ điều khiển PI mờ

```

% Decode_FuzzyPI: Chương trình con giải mã NST mã số thực thành
% thông số hàm liên thuộc và gán vào bộ điều khiển PI mờ
% Lập trình: Huỳnh Thái Hoàng, HCMUT
% Xem ví dụ 5.11

```

```

% Lấy giá trị của các thông số
P1=pop(pop_index,1);
P2=pop(pop_index,2);
P3=pop(pop_index,3);
P4=pop(pop_index,4);
P5=pop(pop_index,5);
P6=pop(pop_index,6);
P7=pop(pop_index,7);
P8=pop(pop_index,8);
K1=pop(pop_index,9);
K2=pop(pop_index,10);
K3=pop(pop_index,11);

```

```
% Gán vào hệ mờ
FuzzyPI= setfis(FuzzyPI,'input',1,'mf',1,'Params',[-2 -1.5 -1 P1]);
FuzzyPI= setfis(FuzzyPI,'input',1,'mf',2,'Params',[-1 P1 0]);
FuzzyPI= setfis(FuzzyPI,'input',1,'mf',3,'Params',[P1 0 P2]);
FuzzyPI= setfis(FuzzyPI,'input',1,'mf',4,'Params',[0 P2 1]);
FuzzyPI= setfis(FuzzyPI,'input',1,'mf',5,'Params',[P2 1 1.5 2]);

FuzzyPI= setfis(FuzzyPI,'input',2,'mf',1,'Params',[-2 -1.5 -1 P3]);
FuzzyPI= setfis(FuzzyPI,'input',2,'mf',2,'Params',[-1 P3 0]);
FuzzyPI= setfis(FuzzyPI,'input',2,'mf',3,'Params',[P3 0 P4]);
FuzzyPI= setfis(FuzzyPI,'input',2,'mf',4,'Params',[0 P4 1]);
FuzzyPI= setfis(FuzzyPI,'input',2,'mf',5,'Params',[P4 1 1.5 2]);

FuzzyPI= setfis(FuzzyPI,'output',1,'mf',1,'Params',[-1]);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',2,'Params',[P5]);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',3,'Params',[P6]);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',4,'Params',0);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',5,'Params',[P7]);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',6,'Params',[P8]);
FuzzyPI= setfis(FuzzyPI,'output',1,'mf',7,'Params',[1]);
```

A.4.3 Chương trình con lai ghép BLX

% Cross_BLX.m: chương trình con lai ghép BLX, nhiệm sắc thể mã số thực

% Giải thuật xem mục 4.3.2

% Lập trình: Huỳnh Thái Hoàng, HCMUT

```
function child=Cross_BLX(parent,Pc,alpha,elitism,bestchrom)
% Biến vào:
% parent      : mảng hai chiều [N×L], quần thể trước khi lai ghép
%              N là kích thước quần thể, L là chiều dài chuỗi NST
% Pc          : xác suất lai ghép
% alpha       : hệ số  $\alpha$  của phép lai ghép BLX
% elitism      : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa
% bestchrom    : biến lưu cá thể tốt nhất trong quá trình tiến hóa
% Biến ra:
% child       : mảng hai chiều [N×L], quần thể sau khi lai ghép

if (nargin < 5),
    error(['Không đủ đối số để thực thi chương trình con']);
end;
```

```

[N,L] = size(parent);
for p1 = 1:N,
    % Cá thể cha mẹ 1
    % Nếu cờ elitism=1 và p1 là cá thể ưu việt
    % thì không lai ghép (để bảo toàn gen tốt nhất)
    if (elitism == 1) & (p1 == bestchrom)
        child(p1,:) = parent(p1,:);
    else
        p2 = p1;
        % Chọn ngẫu nhiên cá thể cha mẹ 2
        while p2 == p1,
            % khác cá thể cha mẹ 1
            p2 = rand*N;
            p2 = p2 - rem(p2,1)+1;
        end

        if Pc > rand
            % Lai ghép xảy ra
            for gene = 1:L,
                pMin = min(parent(p1,gene),parent(p2,gene)) - ...
                    alpha*abs(parent(p1,gene)-parent(p2,gene));
                pMax = max(parent(p1,gene),parent(p2,gene)) + ...
                    alpha*abs(parent(p1,gene)-parent(p2,gene));
                child(p1,gene) = pMin + rand*(pMax - pMin);
            end
        else
            % Lai ghép không xảy ra
            child(p1,:)=parent(p1,:);
        end
    end
end % End the "if (elitism..." statement
end % End "for p1=..." loop

```

A.4.4 Chương trình con đột biến ngẫu nhiên mã số thực

% Mutate_Random.m: chương trình con đột biến ngẫu nhiên, NST mã số thực

% Giải thuật xem mục 4.3.3

% Lập trình: Huỳnh Thái Hoàng, HCMUT

```
function mchrom = Mutate_Random(chrom,Pm,range,elitism,bestchrom)
```

% Biến vào:

% chrom : mảng hai chiều [N×L], quần thể trước khi đột biến

% N là kích thước quần thể, L là chiều dài chuỗi NST

% Pm : xác suất đột biến

% range : tầm giá trị của các thông số

% elitism : cờ lưu giữ cá thể ưu việt trong quá trình tiến hóa

% bestchrom : biến lưu cá thể tốt nhất trong quá trình tiến hóa


```

% Biến ra:
% mchrom : mảng hai chiều [N×L], quần thể sau khi đột biến

if (nargin < 5),
    error(['Không đủ đối số để thực thi chương trình con.']);
end;

[N,L] = size(chrom);
mchrom = chrom;
for p = 1:N,
    % Đột biến chỉ có thể xảy ra khi không lưu giữ cá thể tốt nhất
    % hoặc cá thể đang xét không phải là cá thể tốt nhất
    if elitism == 0 | p ~= bestchrom,
        if Pm > rand % Xác suất xảy ra đột biến
            for gene = 1:L,
                mchrom(p,gene) = ...
                    (rand-0.5)*(range(2,gene) - range(1,gene))+...
                    0.5*(range(2,gene) + range(1,gene));
            end
        end
    end
end
end
end

```

TÀI LIỆU THAM KHẢO

1. J. Abonyi, H. Andersen, L. Navy, F. Szeifert (1999), "Inverse Fuzzy-Process-Model Based Direct Adaptive Control", *Mathematics and Computers in Simulation*, 51(1-2), pp. 119-132.
2. J. Abonyi, R. Babuska, H. Verbruggen, F. Szeifert (2000), "Incorporating Prior Knowledge in Fuzzy Model Identification", *Int. J. of Systems Science*, 31(5), pp. 657-667.
3. J. S. Albus (1991), "Outline for a theory of intelligence", *IEEE Trans. Sys. Man. Cyber.*, 21(3).
4. K.J. Astrom and B. Wittenmark (1994), *Adaptive Control*, Addison-Wesley.
5. P. J. Antsaklis (1993), "Defining Intelligent Control", Report of the Task Force on Intelligent Control, IEEE Control Systems Society.
6. P.J. Antsaklis and K.M. Passino (Eds) (1993), "An Introduction to Intelligent and Autonomuos Control", Kluwer Academic Publishers.
7. P.J. Antsaklis (1997), "Intelligent Control", *Encyclopedia of Electrical and Electronics Engineering*, John Wiley @ Sons, Inc.
8. R. Babuska (2002), "Neuro-fuzzy methods for modeling and identification", In A. Abraham, L.C. Jain, and J. Kacprzyk, Editors, *Recent Advances in Intelligent Paradigms and Applications*, Springer-Verlag, pp 161-186.
9. T. Back (1993), "Optimal Mutation Rates in Genetic Search", *Technical Report*, University of Dortmund, Gernamy.
10. J.E. Baker (1985), "Adaptive Selection methods for genetic algorithm", In *Proc. First Int. Conf. On Genetic Algorithms*, pp. 101-111.
11. D. Beasley, R. Bull, R.R. Martin (1993), "An Overview of Genetic Algorithms: Part 1, Fundamentals", *University Computing*, 15(2), pp. 58-69.

12. D. Beasley, R. Bull, R.R. Martin (1993), "An Overview of Genetic Algorithms: Part 2, Research Topics", *University Computing*, 15(4), pp. 170-181.
13. T. Blickle and L. Thiele (1995), "A Comparison of Selection Schemes used in Genetic Algorithms", *Technical Report*, Swiss Federal Institute of Technology.
14. U. Bodenhofer (2003), "Genetic Algorithms: Theory and Application", *Lecture Notes*, Johannes Kepler University, Austria.
15. L. Booker (1987), "Improving search in genetic algorithms", in L. Davis Ed. *Genetic Algorithms and Simulated Annealing* (Morgan Kaufmann Publishers), pp.61-73.
16. K.M. Bossley (1997), *Neurofuzzy Modelling Approaches in System Identification*, PhD Thesis, University of Southampton
17. S. Boyd and L. Vandenberghe (2003), *Convex Optimization*, Cambridge University Press.
18. P.J.C Branco and J.A. Dente (1998), "The application of fuzzy logic in automatic modeling electromechanical systems", *Fuzzy Sets and Systems*, 95(3), pp. 273-293.
19. H. Chekireb, M. Tadjine and D. Bouchaffra (2003), "Direct adaptive fuzzy control of nonlinear system class with applications", *Control and Intelligent Systems*, 31(2).
20. Y. Chen (2001), *Hybrid Soft Computing Approach to Identification and Control of Nonlinear Systems*, Ph.D. Thesis, Kumamoto University, Japan.
21. M. G. Cooper and J.J. Vidal (1993), "Genetic Design of Fuzzy Controllers", 2nd Int. Conf. Fuzzy Theory and Technology.
22. Cordon, F. Gomide, F. Herrera, F. Hoffmann, L. Magdalena (2004), "Ten years of genetic fuzzy systems: current framework and new trends", *Fuzzy Sets and Systems*, 141, pp. 5-31
23. F. Cus, M. Milfelner, J. Balic (2005), "An intelligent system for monitoring and optimization of ball-end milling process", *Journal of Materials Processing Technology*.
24. K. Deb, "Introduction to Genetic Algorithms", Technical Report, Indian Institute of Technology Kanpur.

25. K. Deb, "Genetic Algorithm in Search and Optimization: The Technique and Applications", Technical Report, Indian Institute of Technology Kanpur.
26. M. A. Denai and S. A. Attia (2002), "Fuzzy And Neural Control Of An Induction Motor", *Int. J. Appl. Math. Comput. Sci.*, 12(2), pp. 221-233
27. Y.-S. Ding, H. Ying and S.-H. Shao (2000), "Necessary Conditions on Minimal System Configuration for General MISO Fuzzy Systems as Universal Approximators", *IEEE Trans. Syst., Man, Cybern.*, 30, pp. 857-864.
28. D. Dubois and H. Prade (1980), *Fuzzy Sets and Systems: Theory and Applications*, Academic Press.
29. J.M. Evans, E.R. Messina, J. S. Albus (2002), "Knowledge Engineering for Real Time Intelligent Control", *Proceedings of the International Symposium on Intelligent Control*, Vancouver, Canada, October 27 - 30.
30. J.A. Fabro, L.V.R. Arruda, F. Neves Jr. (2005), "Startup of a distillation column using intelligent control techniques", *Computers and Chemical Engineering*, 30, pp. 309-320.
31. J. Farrell and W. Baker (1993), "Learning Control Systems", In P.J. Antsaklis and K.M. Passino (Eds), *An Introduction to Intelligent and Autonomuos Control*, Kluwer Academic Publishers.
32. Fletcher (1987), *Practical Methods of Optimization*, John Wiley & Sons.
33. S.S. Ge and C. Wang (2002), "Direct Adaptive NN Control of a Class of Nonlinear Systems", *IEEE Trans. Neural Networks*, 13(1), pp. 214-221.
34. S. Ghosh and T. Lee (2000), *Intelligent Transportation Systems: New Principles and Architectures*, CRC Press.
35. N. Goléa, A. Goléa and K. Benmahammed (2002), "Fuzzy Model Reference Adaptive Control", *IEEE Trans. Fuz. Syst.*, 10(4), pp. 436-444.
36. D.E. Goldberg (1989), *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.

37. E. Grant (1993), "Learning in Control", In P.J. Antsaklis and K.M. Passino (Eds), *An Introduction to Intelligent and Autonomuos Control*, Kluwer Academic Publishers.
38. M. T. Hagan, H. E. Demuth and O. D. Jesus, "An introduction to the use of neural networks in control systems", *Technical Report*, Colorado State University.
39. S. Haykin (1994), *"Neural networks: A comprehensive Foundation"*, Prentice-Hall.
40. F. Herrera, M. Lozano and J.L. Verdegay (1995), "Tuning Fuzzy Logic Controllers by Genetic Algorithms", *Int. J. Approximate Reasoning*.
41. F. Herrera, M. Lozano and J.L. Verdegay (1996), "Tackling Real-Coded Genetic Algorithms: Operators and Tools for Behavioural Analysis", *Technical Report*, University of Granada, Spain.
42. F. Herrera and M. Lozano, "Adaptation of Genetic Algorithm Parameters Based on Fuzzy Logic Controllers", *Technical Report*, University of Granada, Spain.
43. A. Hopgood (2001), *"Intelligent Systems for Engineers and Scientists"*, CRC Press.
44. Y.-C. Hsu, G. Chen, S. Tong and H.-X. Li (2003), "Integrated fuzzy modeling and adaptive control for nonlinear systems", *Information Sciences*, 153, pp. 217-236.
45. H.-M Huang (1996), "An Architecture And A Methodology For Intelligent Control", *Issue of IEEE Expert*.
46. Y. L. Huang, H. H. Lou, J. P. Gong, and T. F. Edgar (2000), "Fuzzy Model Predictive Control", *IEEE Trans. Fuzzy System*, 8(6), pp. 665-678.
47. M. A. Hussain, P. Kittisupakorn and W. Daosud (2001), "Implementation of Neural-Network-Based Inverse-Model Control Strategies on an Exothermic Reactor", *ScienceAsia*, 27, pp. 41-50
48. P.A. Ioannou and J. Sun (1996), *Robust Adaptive Control*, Prentice Hall.
49. Isidori (2001), *"Nonlinear Control Systems"*, 3rd Edition, Springer.

50. R. Jager (1995), *"Fuzzy Logic in Control"*, PhD Thesis, Technische Universiteit Delft, The Netherlands.
51. J.-S.R. Jang (1992), *Neuro-Fuzzy Modeling: Architectures, Analyses, and Applications*, Ph.D. Thesis, EECS Dept., Univ. of California at Berkeley.
52. J.-S. R. Jang (1993), "ANFIS: Adaptive-Network-Based Fuzzy Inference System", *IEEE Trans. on Systems, Man, and Cybernetics*, vol. 23, pp. 665-685.
53. J.-S. R. Jang and C.-T. Sun (1993), "Functional Equivalence Between Radial Basis Function Networks and Fuzzy Inference Systems", *IEEE Trans. Neural Networks*, vol. 4, pp. 156-159.
54. J.-S. R. Jang and C.-T. Sun (1995), "Neuro-Fuzzy Modeling and Control", *The Proceedings of the IEEE*, 83, pp. 378-406.
55. T.A Johansen (1996), "Identification of Non-linear Systems using Empirical Data and Prior Knowledge - An Optimization Approach", *Automatica*, 32, pp. 337-356.
56. T.A. Johansen and M. M. Polycarpou (1996), "Stable Adaptive Control of a General Class of Non-linear Systems", In R. W. Zbikowski and K. J. Hunt (Eds.), *Neural Adaptive Control Technology I*, World Scientific.
57. T.A. Johansen, R. Shorten and R. Murray-Smith (2000), "On the Interpretation and Identification of Takagi-Sugeno Fuzzy Models", *IEEE Trans. Fuzzy Systems*, 8, pp. 297-313.
58. K. H. Johansson (2000), "The quadruple-tank process: A multivariable laboratory process with an adjustable zero", *IEEE Trans. Control Systems Technology*, 8(3), pp. 456-465.
59. R. Johansson (1993), *System Modeling and Identification*, Prentice-Hall.
60. Juditsky, H. Hjalmarsson, A. Benveniste, B. Delyon, L. Ljung, J. Sjöberg, and Q. Zhang (1995), "Nonlinear Black-box Models in System Identification: Mathematical Foundations", *Automatica*, 31(12), pp. 1725-1750.
61. H. K. Khalil (2002), *"Nonlinear Control"*, 3rd Edition, Prentice-Hall.

62. H. Kiểm và L. H. Thái (2000), *Thuật giải di truyền – Cách giải tự nhiên các bài toán trên máy tính*, NXB Giáo dục.
63. C.C Ko, Chen B. M., Chen J., Zhuang Y. and Tan K. C. (2001), "Development of a web-based laboratory for control experiments on a coupled tank apparatus", *IEEE Trans. Education*, 44(1), pp. 76-86.
64. H. Koivisto (1995), "A Practical Approach to Model Based Neural Network Control", *PhD Thesis*, Tampere Univ. of Technology, Finland.
65. M. M. Kokar (1993), "Learning Control: Methods, Needs and Architectures", In P.J. Antsaklis and K.M. Passino (Eds), *An Introduction to Intelligent and Autonomuos Control*, Kluwer Academic Publishers.
66. B. Kosko (1994), "Fuzzy Systems as Universal Approximators", *IEEE Transactions on Computers*, 43(11), pp. 1329-1333.
67. R. Krishnapuram and J. Kim (1999), "A Note on the Gustafson-Kessel and Adaptive Fuzzy Clustering Algorithms", *IEEE Trans. Fuzzy Systems*, 7(4), pp. 453-461.
68. K. Krishnakumar (1997), "Levels of Intelligent Control: A Tutorial", New Orleans, LA.
69. J.R. Layne, K.M. Passino (1996), "Fuzzy Model Reference Learning Control," *Journal of Intelligent andF uzzy Systems*, 4(1), pp. 33-47.
70. E.G. Laukonen, K.M. Passino (1995), "Training Fuzzy Systems to Perform Estimation and Identification", *Engineering Applications of Artificial Intelligence*, 8(5), pp. 499-514.
71. W.K. Lennon and K.M. Passino (1999), "Genetic Adaptive Identification and Control", *Engineering Applications of AI*, 12(2), pp. 185-200.
72. D. Levine (1994), "A parallel Genetic Algorithm for the Set Partitioning Problem", *PhD Thesis*, Illinois Institute of Technology.

73. H.-X. Li and C. L. P. Chen (2000), "The equivalence between fuzzy logic systems and feedforward neural networks", *IEEE Trans. Neural Networks*, 11(2), pp. 356-365.
74. H. Li, C.L.P. Chen, H.-P. Huang (2001), *Fuzzy Neural Intelligent Systems: Mathematical Foundation and the Application in Engineering*, CRC Press.
75. C.J. Lin (2004), "A GA-based neural fuzzy system for temperature control", *Fuzzy Sets and Systems*, 143, pp. 311-333.
76. C.-T. Lin and C.S. George-Lee (1996), *Neural Fuzzy Systems: A Neuro-Fuzzy Synergism to Intelligent Systems*, Prentice-Hall.
77. P. Lindskog (1996), *Methods, Algorithms and Tools for System Identification Based on Prior Knowledge*, PhD. Thesis, Linköping University, Sweden.
78. L. Ljung (1996), "Development of system identification", *Technical Report*, Dept of EE. Linköping University.
79. L. Ljung (1999), *System Identification – Theory for the users*, 2nd Edition, Prentice-Hall, Englewood Cliffs, New Jersey.
80. L. Ljung (2002), "Prediction Error Methods", *Circuits, Systems and Signal Processing*, 21, pp. 11-21.
81. L. Ljung (2002), "Recursive Identification Algorithms", *Circuits, Systems and Signal Processing*, 21, pp 57-68.
82. P. X. Minh và N. D. Phước (2003), *Lý thuyết điều khiển mờ*, NXB Khoa học và Kỹ thuật.
83. C. G. Moore (1992), *Indirect Adaptive Fuzzy Controller*, PhD Thesis, University of Southampton.
84. M. Morari and J.H. Lee (1999), "Model predictive control: Past, Present and Future", *Computer and Chemical Engineering*, 23, pp. 667-682.
85. E. Mosca (1995), *Optimal, Predictive, and Adaptive Control*, Prentice-Hall.
86. M. M. Mwembeshi, C. A. Kent, S. Salhi (2004), "A genetic algorithm based approach to intelligent modelling and control of pH in reactors", *Computers and Chemical Engineering*, 28, pp. 1743-1757.

87. F. Nardi (2000), *Neural Network Based Adaptive Algorithms for Nonlinear Control*, Ph.D Thesis, Georgia Institute of Technology, USA.
88. R. Ordídez (1999), *On-Line Approximation Based Control of Nonlinear Systems*, PhD Thesis, The Ohio State University, USA.
89. K.M. Passino (2001) "Intelligent Control: An Overview of Techniques", Chapter in *Perspectives in Control: New Concepts and Applications*, IEEE Press, NJ.
90. K.M. Passino and S. Yurkovich (1998), *Fuzzy Control*, Addison-Wesley.
91. K.M. Passino (1993), "Bridging the gap between conventional and intelligent control", *IEEE Trans. Control Systems*.
92. S.J. Qin and T.A. Badgwell (2000), "An overview nonlinear model predictive control applications". In F. Allgower and A. Zheng, editors, *Nonlinear Predictive Control*, pp. 369-393, Birkhauser.
93. Rivals and L. Personnaz (2000), "Nonlinear Internal Model Control Using Neural Networks: Application To Processes With Delay And Design Issues", *IEEE Transactions on Neural Networks*, 11(1), pp. 80-90.
94. J.A. Roubos, S. Molloy, R. Babuska and H. B. Verbruggen (1999), "Fuzzy Model Based Predictive Control using Takagi-Sugeno Model", *Int. J. Approximate Reasoning*, 22, pp. 3-30.
95. S. Sastry and M. Bodson (1989), *Adaptive Control: Stability, Convergence, and Robustness*, Prentice-Hall, Englewood Cliffs, NJ.
96. Sala, T. M. Guerra, R. Babuka (2005), "Perspectives of fuzzy systems and control", *Fuzzy Sets and Systems*, 156, pp. 432-444.
97. G.N. Saridis and K.P. Valavanis (1998), "Analytical Design of Intelligent Machines", *Automatica*, 24(2), 123-133.
98. H. Sarimveis and G. Bafas (2003), "Fuzzy model predictive control of non-linear processes using genetic algorithms", *Fuzzy Sets and Systems*, 139, pp. 59-80.

99. S. Setnes and H. Roubos (2000), "GA-Fuzzy Modeling and Classification: Complexity and Performance", *IEEE Trans. Fuz. Syst.*, 8(5), pp. 509-522.
100. R. Sindelar and R. Babuska (2004), Input Selection for Nonlinear Regression Models, *IEEE Trans. Fuz. Syst.*, 12(5), pp. 688-696.
101. J. Sjöberg (1995), *Non-linear System Identification with Neural Networks*, Ph.D Thesis, Linköping University, Sweden.
102. J. Sjöberg, Q. Zhang, L. Ljung, A. Benveniste, B. Deylon, P.Y. Glorennec, H. Hjalmarsson, A. Juditsky (1995), "Nonlinear Black-box Modeling in System Identification: a Unified Overview", *Automatica*, 31(12), pp. 1691-1724.
103. J.T. Spooner and K.M. Passino (1996), "Stable adaptive control using fuzzy systems and neural networks", *IEEE Trans. Fuz. Syst.*, 4(3), pp. 339-359.
104. C-Y. Su and Y. Stepanenko (1994), "Adaptive control of a class of nonlinear systems with fuzzy logic", *IEEE Trans. Fuz. Syst.*, 2(4), pp. 285-294.
105. H. Surmann and M. Maniadakis (2001), "Learning feed-forward and recurrent fuzzy systems: a genetic approach", *Journal of Systems Architecture*, 47(7), pp. 649-662.
106. S. Tan, J. Suykens, Y. Yu and J. Vandewalle (1998), "Neural Network Techniques for Nonlinear System Modeling", *Control and Dynamic Systems*, (Ed. C. T. Leondes), pp. 383-433, Academic Press.
107. S. Tsutsui and D. E. Goldberg, "Search Space Boundary Extension Method in Real-Coded Genetic Algorithms" *Technical Report*, University of Illinois at Urbana-Champaign.
108. J. Van Gorp (2000), *Nonlinear Identification with Neural Networks and Fuzzy Logic*, PhD Thesis, Vrije Universiteit Brussel.
109. L.-X. Wang (1997), *A course in fuzzy systems and control*, Prentice-Hall.

110. Q. Wang, P. Spronck, R. Tracht (2003), "An overview of genetic algorithms applied to control engineering problems", *Int. Conf. Machine Learning and Cybernetics*, pp. 1651-1656.
111. K. Warwick, Y.-H. Kang, R.J. Mitchell (1999), "Genetic least squares for system identification", *Soft Computing*, 3(4), pp. 200-205.
112. D. Whitley, "A Genetic Algorithm Tutorial", Technical Report, Colorado State University.
113. W.F. Xie and A.B. Rad (2000), "Fuzzy Adaptive Internal Model Control", *IEEE Trans. Industrial Electronics*, 47(1), pp.192-202.
114. L. Yao and W.A. Sethares (1994), "Nonlinear parameter estimation via the genetic algorithm", *IEEE Trans. Signal Processing*, 42(4), pp. 927-935.
115. Y. Yi (1998), *On fuzzy modeling of nonlinear dynamical systems*, PhD Thesis, National University of Singapore.
116. H. Ying (1998), "Sufficient Conditions on Uniform Approximation of Multivariate Functions by General Takagi-Sugeno Fuzzy Systems with Linear Rule Consequent", *IEEE Trans. Syst., Man, Cybern.*, 28, pp. 515-520.
117. H. Ying, Y.-S. Ding, S.-K. Li and S.-H. Shao (1999), "Comparison of Necessary Conditions for Typical Takagi-Sugeno and Mamdani Fuzzy Systems as Universal Approximators", *IEEE Trans. Syst., Man., Cybern.*, 29, pp. 508-514.
118. L. I. Yao, Postlethwaite, W. Browne, D. Gu, M. Mar, S. Lowes (2005), "Design, implementation and testing of an intelligent knowledge-based system for the supervisory control of a hot rolling mill", *Journal of Process Control*, 15, pp. 615-628.
119. L.A. Zadeh (1965), "Fuzzy Sets", *Information and Control*, 8, pp. 338-353.
120. Zilouchian and M. Jamshidi (Edited) (2001), *Intelligent Control Systems Using Soft Computing Methodologies*, CRC Press.

HỆ THỐNG ĐIỀU KHIỂN THÔNG MINH

Huỳnh Thái Hoàng

NHÀ XUẤT BẢN ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH

Khu phố 6, phường Linh Trung, quận Thủ Đức, TP HCM

ĐT: 7 242 181, 7 242 160 + (1421, 1422, 1423, 1425, 1426)

Fax: 7 242 194 - Email: vnuhp@vnuhcm.edu.vn

Chịu trách nhiệm xuất bản

PGS TS NGUYỄN QUANG ĐIỂN

Biên tập

TRẦN VĂN THẮNG

Sửa bản in

THÙY DƯƠNG

Trình bày bìa

TRƯƠNG NGỌC TUẤN

Đơn vị/Người liên kết

Trường DH Bách khoa – ĐH Quốc gia TP Hồ Chí Minh

In 200 cuốn, khổ 16 x 24 cm

Số đăng ký KHXB số: 732-2006/CXB/109-72/ĐHQG TPHCM

Quyết định xuất bản số: 633/QĐ/ĐHQGTPHCM

ngày 24/11/2006 của Giám đốc NXB ĐHQG TP. Hồ Chí Minh

In tại Xưởng in Đại học Bách khoa - ĐHQG TP. HCM

Nộp lưu chiếu tháng 12 năm 2006.